CMF 3D-QSAR

The continuous molecular fields approach, as a program

Desktop application, library and command-line tool
for the Continuous Molecular Fields approach to 3D-QSAR
User manual

`github.com/igbaskin/cmf-qsar`

Contents

1 What this program is **7**

1.1 The idea in one page 7
1.2 What you can do with it 7
1.3 How this manual is arranged 8

2 How a prediction is made **9**

2.1 The fields 9
2.2 The overlap integral 10
2.3 Normalisation and scaling 10
2.4 Mixing the fields 11
2.5 Fitting: what is learned from the training set 11
2.6 Predicting a new molecule 11
2.7 The same prediction, as a field 12
2.8 A worked example 12
2.9 What changes for the other tasks 13
2.10 What has to match between fitting and predicting 14

3 Installation and first run **16**

3.1 Where the program lives 16
3.2 Requirements 16
3.3 Starting the application 16
3.4 A first model in five minutes 17

4 The application window **18**

4.1 The Data set panel 18
4.2 The Model panel 19
4.3 The 3D view 20
4.4 The Results panel 21
4.5 The menus 22
4.6 Crystals and other periodic structures 23
4.7 A metal surface behind the molecules 28
4.8 The vector co-field: a direction instead of a value 29
4.9 The co-pattern as a chart 30
4.10 Projects: a session that can be reopened, a step that can be undone 31

5 Preparing the data **33**

5.1 What the program reads 33
5.2 Activities 33
5.3 Alignment 33
5.4 Judging an alignment, and aligning flexibly 34
5.5 Charges and per-atom properties 39
5.6 Quantum-chemical properties with ORCA 41
5.7 A simulation as a conformational ensemble 43

5.8 The water as a field 44
5.9 Does a trajectory beat the crystal pose? 49
5.10 A cheap quantum stage: GFN2-xTB 53
5.11 Can a Fukui function be a field? 55
5.12 Where the added electron went: the spin field, and the series that does not need it 56
5.13 A metal in the middle: coordination complexes 59
5.14 The field a reaction centre feels: a vector kernel, and why it changes almost nothing 61
5.15 Reading a few places instead of all of them 63
5.16 A metal surface as the receptor, and a score that needs the answer to compute 65
5.17 A kinetic selectivity as the activity: a catalyst series in the frame of its reaction centre 69
5.18 Three tests to run before you build a new field or a new score 70

6 Before you model: `cmf diagnose` **72**

6.1 Two things the command protects you from 72
6.2 Reading the answer 73
6.3 Does the advice work? 73

7 Building a model 74

7.1 Choosing the fields 74
7.2 GRID-like probe fields 75
7.3 Weighting the fields by solvent exposure 78
7.4 Descriptors that are not fields 78
7.5 Attenuation, regularisation and the grids 80
7.6 How long the kernel takes, and the three settings that change it 81
7.7 The learners 83
7.8 Kernel partial least squares: CoMFA's learner on the continuous field 84
7.9 A classifier with a co-field: kernel Fisher discriminant analysis 85
7.10 Kernel logistic regression: the Gaussian process with its ridge chosen on the folds 89
7.11 Molecules without an activity: the Laplacian-regularised ridge 91
7.12 Fitting the order instead of the values: the ranking SVM 93
7.13 Nonlinear kernel layers and multiple kernel learning 95
7.14 Which fields does this series need? 95
7.15 Choosing α and λ without holding anything out 97
7.16 Validation 99
7.17 Is it chance? y-randomisation 100
7.18 Where does the model apply? 100
7.19 Is the whole library inside? The two-sample test 101
7.20 How certain is that number? 102
7.21 How certain is *one* prediction? Conformal intervals 102
7.22 Three estimates of the error, compared 105
7.23 The model as the fast level of a hierarchy: a standard deviation in the units of the activity 107

7.24 What the model thinks the groups are: clustering the kernel 111
7.25 Where the validation appears 111
7.26 Statistics reported 112
7.27 Saving and reusing 113
7.28 A report to hand to a colleague 113

8 Reading the model 115

8.1 The three decomposition rules 115
8.2 Co-fields 118
8.3 Shapley values 125
8.4 A pharmacophore out of the co-field 126
8.5 What a candidate is missing 127
8.6 Several endpoints at once: the selectivity co-field 127
8.7 Comparing two models by their co-fields 133
8.8 What the field sees that another view does not 134
8.9 Chemical space maps 137
8.10 The pre-image kernel PCA lacks: a decoder of pseudo-atoms 138

9 Classification and virtual screening 142

9.1 Setting up a classification task 142
9.2 One-class models: screening with actives alone 142
9.3 Decoys and the figures that matter 143
9.4 Measured inactives, and learning from positives alone 144
9.5 Fitting in the space of the landmarks 146
9.6 Screening a library that does not fit in memory 148
9.7 Which molecule to make next 149
9.8 Designing the next compound 151
9.9 Replacing the scaffold: scoring only where the co-field has weight 153
9.10 A design loop with more than one objective 155
9.11 An anti-target beside the potency 156
9.12 Choosing ten compounds at once 157
9.13 Growing a bound fragment into the gap 158
9.14 A matched pair as a field placed in space 160
9.15 Which groups are interchangeable: a bioisostere table 161
9.16 The gradient of a prediction 162

10 Conformations: the fourth dimension 167

10.1 The variance of an ensemble: a flexibility map 168
10.2 Scoring a conformational restriction 169
10.3 Protonation and tautomers: ensembles of different molecules 170

11 Working without an alignment 173

11.1 Why the frame is the problem 173
11.2 The pose-free count kernel 173
11.3 The rotation-invariant autocorrelation kernel 173
11.4 Molecules on a surface: the in-plane invariant kernel 181
11.5 Posed by rule: the surface frame without the slab 183

11.6 A frame by rule: the gap of polybenzenoid hydrocarbons 184
11.7 Crystals: the periodic autocorrelation kernel 190
11.8 A series on one lattice: the periodic overlap kernel 195
11.9 A site of the cell as the sample: the split of the autocorrelation 201
11.10 Order the cutoff cannot reach: the reciprocal-space channels 205
11.11 A measured curve as the sample: the untyped descriptor 208
11.12 A microstructure as the sample: the field on a voxel grid 210
11.13 Three atoms at a time: what a histogram of distances cannot see 212
11.14 Designing a crystal: a prototype, a budget and a loop 216
11.15 An atom instead of a molecule: the local invariant kernel 218
11.16 A co-field around one atom: the local alignment 228
11.17 A migration path as the sample: the barrier of a Li hop 233
11.18 Reading the model from another program 247
11.19 The voltage of an insertion electrode: the cell against its composition 250
11.20 An ion as the probe: where a cation sits, and what it gains 253
11.21 The ion with its shell as the molecule: a solvation complex 255
11.22 A distribution of shells instead of one: sampling a bulk electrolyte 257
11.23 Which atom is attacked: a site of metabolism 264
11.24 Topomer fields 266
11.25 What the three routes are worth, measured 267
11.26 Heterogeneous series, step by step 268

12 Protein pockets 269

12.1 Getting a pocket on screen 269
12.2 Complementarity as numbers 269
12.3 A macrocyclic host as a pocket 271
12.4 Docking 272

13 Three series, worked through 273

13.1 The case that does not survive its own check: `ptp1b` 273
13.2 Shape and lipophilicity in a groove: `mcl1` 274
13.3 An acceptor-driven kinase: `cdk8` 274
13.4 What the three have in common 275
13.5 One command for the alignment: `cmf align` 275
13.6 Co-fields without the application: `cmf cofield` 276
13.7 The benchmarks from the tool: `cmf benchmark` 277

14 Command-line reference 280

14.1 The options of `build`, by group 281
14.2 Worked examples 281

15 Using it as a library 283

15.1 The stable surface 284

16 The example data sets 288

17 When something goes wrong 289

Appendix A — Field parameterisation **290**

A.1 Increments fitted to the data 290

Appendix B — Files and tags the program understands **292**

Appendix C — Glossary **293**

Appendix D — Bibliography **297**

D.1 The method 297

D.2 One-class classification and virtual screening 297

D.3 Methods and data taken from other work 297

1 What this program is

CMF 3D-QSAR is a desktop application, a Python library and a command-line tool implementing the **Continuous Molecular Fields** approach to three-dimensional structure–activity modelling. The method was introduced by Zhokhova, Baskin and co-workers [1, 2] and extended with continuous indicator fields [4] and co-fields [6], and the program follows those papers closely enough that their published statistics can be reproduced from the same data — `docs/benchmark.md` records that comparison set by set.

1.1 The idea in one page

A molecule is represented not by a table of descriptors and not by values on a lattice, but by **continuous functions of space**. Each field is a sum of Gaussians centred on the atoms,

$$X_f(\mathbf{r}) = \sum_l w_{lf} \exp\left(-\frac{\alpha_f}{2} |\mathbf{r} - \mathbf{r}_l|^2\right) \tag{1.1}$$

where w_{lf} is the weight of atom l in field f — its partial charge for the electrostatic field, its Crippen logP contribution for the hydrophobic field, 1 for an indicator field of a given atom type, or any number the file carries per atom — and α_f is the attenuation factor that sets how far the field reaches.

Two molecules are compared by the **overlap integral** of their fields, which for Gaussians has a closed form:

$$K_f(A,B) = \int X_{fA}(\mathbf{r})\, X_{fB}(\mathbf{r})\, \mathrm{d}\mathbf{r} = \left(\frac{\pi}{\alpha}\right)^{3/2} \sum_l \sum_m w_{lf} w_{mf} \exp\left(-\frac{\alpha}{4} |\mathbf{r}_l - \mathbf{r}_m|^2\right) \tag{1.2}$$

No grid is ever evaluated for this: the kernel is a sum over atom pairs, computed exactly. The kernels of the individual fields are mixed with coefficients h_f into one kernel, and that kernel goes into a kernel method — ridge regression, a support vector machine, a Gaussian process. Three consequences follow, and they are what distinguishes the approach from CoMFA:

- **Nothing is discretised.** There is no lattice spacing to choose, no alignment of grids, no variable selection among tens of thousands of columns, and no truncation at a cut-off distance.
- **The model is a function of space again.** A kernel ridge model predicts $y(X) = \Sigma_j\, a_j\, K(X, X_j) + b$, and because the kernel is an integral of a product, the whole sum can be rewritten as the overlap of the molecule's field with one function, the **co-field** $C_f(\mathbf{r}) = \Sigma_j\, a_j\, X_{fj}(\mathbf{r})$. The co-field is the program's map: it plays the role of CoMFA's coefficient contour, but it is the model rather than a projection of it.
- **The decomposition is exact.** Because the prediction is bilinear in the fields, it can be split without approximation into contributions of training molecules, of field types, of regions of space, or of individual atoms.

1.2 What you can do with it

Task	How
Build a 3D-QSAR regression model on an aligned congeneric series	Model panel, or `cmf_cli.py build`
Work without an alignment	topomer fields, the rotation-invariant autocorrelation kernel, or the pose-free count kernel
Classify actives against inactives, or screen with actives alone	binary (SVC, ν-SVC, Laplace GP) and one-class (1-SVM, centroid, k-NN, GP, kernel PCA) tasks
See what the model says about space	co-fields as isosurfaces, molecular fields, overlap maps, Gaussian-cube export
See what the model says about a molecule	training-molecule contributions, atom contributions, Shapley values and interaction indices
Turn a model into design instructions	pharmacophore extraction from co-field lobes, field-difference maps against a reference ligand
Use the protein	fetch or load a pocket, draw it behind the fields, score field complementarity, dock
Average over conformations	conformational ensembles (4D) and multi-instance learning
Map a data set	kernel GTM chemical-space maps with activity, class and density landscapes
Model crystals and layers	the periodic-structures window: the autocorrelation kernel for any cells, the overlap kernel with its co-field inside the cell for a series on one lattice, the sites coloured by their contributions; `cmf build-crystals` and `cmf predict`

1.3 How this manual is arranged

How a prediction is made follows one prediction from the atoms to the number, which is the method in full. *Installation and first run* and *The application window* get you running and name every part of the window. *Preparing the data*, *Before you model*, *Building a model* and *Reading the model* are the working cycle: prepare the data, check what it can support, fit a model, read it. *Classification and virtual screening*, *Conformations*, *Working without an alignment* and *Protein pockets* are the specialised routes, and *Three series, worked through* applies the tools to three series from start to finish. Then come the complete *Command-line reference*, the library (*Using it as a library*), *The example data sets*, and *When something goes wrong*, which collects the things that go wrong. The appendices hold the field parameterisation, the file conventions, a glossary of the terms this manual uses in a particular sense, and the bibliography. In the typeset manual every chapter and section carries a number; the numbers are assigned when the PDF is built, so this text refers to chapters by their titles.

Conventions: *Menu → Item* names a menu command, `monospace` names a file, an option or a piece of code. Equations are numbered by chapter when the PDF is built, (2.3) being the third equation of chapter 2, and the text refers to them by that number. References are cited by their number in the bibliography of Appendix D.

2 How a prediction is made

This chapter follows one number — the activity predicted for one molecule — from the atoms it starts in to the value that appears in the table. Nothing here is a sketch: every formula is the one the program evaluates, and the worked example at the end was computed by running it.

The chain has six links:

1. each molecule becomes a set of **fields**, one per field type, each a sum of Gaussians on its atoms;
2. every pair of molecules gets a **field kernel** per field type, the overlap integral of those two fields, computed in closed form;
3. each field kernel is **normalised** and **scaled**;
4. the field kernels are **mixed** into one kernel matrix;
5. the learner turns that matrix and the activities into **dual coefficients**;
6. a new molecule is pushed through steps 1 to 4 against the training set, and its prediction is the **weighted sum of its kernel values** with those coefficients.

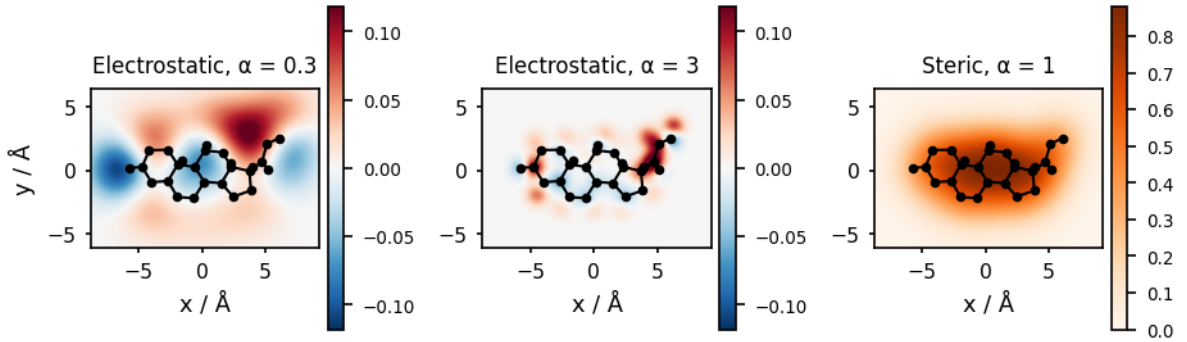
2.1 The fields

Atom l of a molecule carries a weight w_{lf} in field f — its partial charge in the electrostatic field, its Crippen logP contribution in the hydrophobic field, an Abraham-type increment in the hydrogen-bond fields, the Tripos van der Waals well depth in the steric field, 1 in an indicator field for atoms of that type and 0 elsewhere, or a number read from the file in a `prop:` field. Appendix A lists them.

The field itself is the sum of Gaussians those weights define,

$$X_f(\mathbf{r}) = \sum_l w_{lf} \exp\left(-\frac{b_{lf}}{2}|\mathbf{r}-\mathbf{r}_l|^2\right) \tag{2.1}$$

with width $b_{lf} = \alpha_f$ for every field except the steric one, where it is scaled by the atom's van der Waals radius, $b_{lf} = \alpha_f / R_l^2$, so that a large atom spreads its bulk over a correspondingly larger region.



The electrostatic field of cortisol in the mean plane of the steroid series at α = 0.3 and at α = 3, and its steric field at α = 1; red is positive, blue negative, and the skeleton of the molecule is drawn over the map. A small attenuation spreads the Gaussian of every atom over its neighbours, a large one keeps each atom's contribution to itself. Drawn by `docs/make_figures.py`*, as are the other computed figures of this manual.*

There is no grid. The field is never evaluated anywhere in order to make a prediction; it is evaluated only when a map has to be *drawn*.

2.2 The overlap integral

Two Gaussians have an overlap integral in closed form, so the inner product of two whole fields is a double sum over atom pairs:

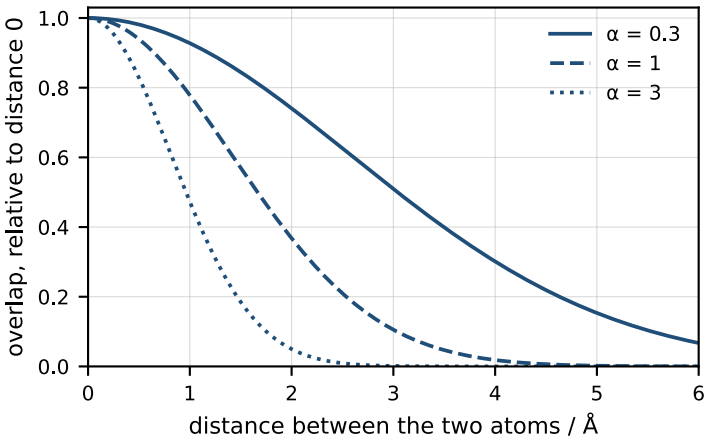
$$K_f(A,B) = \int X_{fA}(\mathbf{r}) X_{fB}(\mathbf{r})\,\mathrm{d}\mathbf{r} = \sum_{l\in A}\sum_{m\in B} w_{lf} w_{mf} \left(\frac{2\pi}{b_{lf}+b_{mf}}\right)^{3/2} \exp\left(-\frac{b_{lf} b_{mf}}{2(b_{lf}+b_{mf})} |\mathbf{r}_l - \mathbf{r}_m|^2\right) \tag{2.2}$$

When every atom shares one width, $b_{lf} = b_{mf} = \alpha$, this collapses to the form the papers quote,

$$K_f(A,B) = \left(\frac{\pi}{\alpha}\right)^{3/2} \sum_l \sum_m w_{lf} w_{mf} \exp\left(-\frac{\alpha}{4} |\mathbf{r}_l - \mathbf{r}_m|^2\right) \tag{2.3}$$

and the general form is what the steric field needs, since its widths differ atom by atom. A unit test checks both against numerical quadrature of the fields on a fine grid.

Two things follow immediately. The cost of a kernel is the number of atom pairs, not the number of grid points, which is why a 300-molecule set takes seconds. And the attenuation α is the only thing that decides how far apart two atoms may be and still contribute: large α demands close matching, small α lets distant atoms see each other, and in the limit α → 0 the geometry drops out altogether and the kernel becomes a product of atom counts — which is the alignment-free baseline of *Working without an alignment*.



The overlap of the Gaussians of two atoms of the same width against the distance between them, relative to its value at distance zero, exp(−αd²/4), for three attenuations. At α = 3 two atoms 2 Å apart no longer see each other; at α = 0.3 their overlap is still 74 % of the maximum.

2.3 Normalisation and scaling

Two corrections stand between the raw integrals and a usable kernel, and they do different jobs.

Cosine normalisation (*Normalise kernels*) divides each field kernel by the norms of the two fields,

$$\hat{K}_f(A,B) = \frac{K_f(A,B)}{\sqrt{K_f(A,A)\,K_f(B,B)}} \tag{2.4}$$

so that every molecule has unit self-similarity in every field -- the normalised kernel of a molecule with itself is 1 by construction -- and a large molecule cannot look important merely for being large.

Field scaling (*Scale fields*) then multiplies each field kernel by a constant c_f chosen so that its mean diagonal over the training set is 1. It equalises the fields' contributions before they are mixed, which matters when one field's raw magnitudes dwarf another's — an electrostatic kernel built from partial charges and a steric kernel built from well depths are not naturally on the same scale. With cosine normalisation on, every diagonal is already 1 and c_f = 1; the two options are therefore alternatives in practice, and unticking both gives the raw-kernel convention of the papers.

The scaling constants are computed **on the training set and then stored**: a prediction must not rescale anything according to the molecules being predicted, or two test molecules would be scored on different scales.

2.4 Mixing the fields

The joint kernel is a weighted sum over field types,

$$K(A,B) = \sum_f h_f c_f \hat{K}_f(A,B) \tag{2.5}$$

with the mixing coefficients h_f left at 1 by default, edited by hand in the field table, chosen by coordinate descent on the cross-validated error, or set by centred kernel alignment. Since each normalised field kernel has unit diagonal, the joint self-kernel of any molecule is $\Sigma_f h_f c_f$ — five, for the five default fields with unit coefficients, which is the 5 on the diagonal of the matrix in the worked example below.

2.5 Fitting: what is learned from the training set

Kernel ridge regression with a bias term solves one linear system:

$$(\mathbf{K} + \lambda \mathbf{I})\,\mathbf{a} + b\,\mathbf{1} = \mathbf{y}, \qquad \mathbf{1}^{\mathrm{T}}\mathbf{a} = 0 \tag{2.6}$$

The outcome is one **dual coefficient** a_j per training molecule and one bias b. The last row of the system is the constraint $\Sigma_j a_j = 0$, which is what makes the bias meaningful: the coefficients describe only *departures* from a constant level, and b carries the level itself. The ridge parameter λ adds to the diagonal, which keeps the system solvable when molecules are nearly identical — as they are in a congeneric series, where the off-diagonal kernel values run above 0.97 — and controls how closely the model follows the training activities.

The other learners differ only in how **a** and b are obtained, never in what they are: ε-SVR and ν-SVR get them from a quadratic program in which most a_j come out exactly zero, C-SVC and ν-SVC the same for class labels, and the Laplace Gaussian-process classifier from the posterior mode of a latent function. In every case the fitted model is the pair (**a**, b) over the training molecules.

2.6 Predicting a new molecule

For a molecule the model has never seen, the program:

1. builds its fields **with the same field set** — the same α, the same charge model, the same parameterisation, the same atom typing;

2 computes the per-field overlap integrals between its atoms and the atoms of every training molecule, giving one row $K_f(x, j)$ per field;

3 normalises that row with the molecule's own self-kernels and the **stored** training self-kernels, and multiplies by the **stored** scales c_f;

4 mixes those normalised, scaled fields into one row $k_j(x)$ with the mixing coefficients h_f, which is the joint kernel $K(x, j)$ of *Mixing the fields* above;

5 returns

$$\hat{y}(x) = \sum_j a_j\, k_j(x) + b = \mathbf{k}(x)^{\mathrm{T}}\mathbf{a} + b \tag{2.7}$$

That is the whole prediction: a similarity-weighted combination of the training activities, where "similarity" is the overlap of molecular fields and the weights were fixed when the model was fitted. Molecules with $a_j = 0$ — most of them, after an SVM fit — contribute nothing, which is why the *Contributions* tab can list the handful that actually make a given prediction.

2.7 The same prediction, as a field

Because the kernel is an integral of a product and the prediction is linear in it, the sum over training molecules can be moved inside the integral. Writing $s_j = 1/\sqrt{(K_f(j,j))}$ for the normalisation of training molecule j, define the **co-field**

$$C_f(\mathbf{r}) = \sum_j a_j\, s_j X_{fj}(\mathbf{r}) \tag{2.8}$$

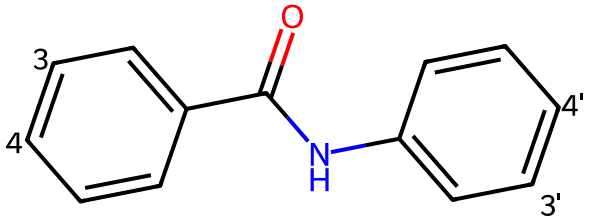
and the prediction becomes

$$\hat{y}(x) = \sum_f \frac{h_f c_f}{\sqrt{K_f(x,x)}} \int X_{fx}(\mathbf{r})\, C_f(\mathbf{r})\, \mathrm{d}\mathbf{r} + b \tag{2.9}$$

The model is therefore a *function of space*, one per field type, and a molecule is scored by how much of its own field lies in the positive regions of that function and how much in the negative ones. This is the identity that makes the pictures of *Reading the model* the model rather than an illustration of it, and the division by the molecule's own field norm is the only thing the normalisation adds to the picture. It also explains what co-fields cannot survive: a nonlinear kernel layer, which is not an integral of a product, and the autocorrelation kernel source, which has no frame to draw in.

2.8 A worked example

Four molecules from `examples/benzamides_synthetic.sdf` as the training set, the five default fields, α = 0.5, cosine normalisation, λ = 0.1, and the fifth molecule as the one to predict. The numbers below are the program's own, printed from the objects it fits.



The parent of the worked example, N-phenylbenzamide, with the positions the series substitutes marked as the molecule names of the file count them: 3 and 4 on the ring of the acid, 3' and 4' on the ring of the amine. The names in the tables below are the substituents at those positions, so 4'Cl is a chlorine para to the nitrogen and 4F a fluorine para to the carbonyl group.

The joint kernel of the training set, the sum of the five normalised field kernels, so the diagonal is 5:

```
                parent      4F      4'F      4Cl          y
parent          5.0000  2.8362  3.2195  2.5368       5.585
4F              2.8362  5.0000  2.3310  3.9647       5.230
4'F             3.2195  2.3310  5.0000  2.8882       6.471
4Cl             2.5368  3.9647  2.8882  5.0000       5.355
```

Solving the bordered system gives

```
a = [-0.1358, -0.0581, +0.3487, -0.1549]      sum(a) = 0      b = 5.7124
```

The test molecule 4'Cl is then compared with each training molecule, field by field:

```
                        parent      4F      4'F     4Cl
electrostatic           0.0878  0.6028  0.1009  0.3915
steric                  0.9793  0.9824  0.9863  0.9579
hydrophobic             0.8936  0.9412  0.9443  0.8437
H-bond acceptor         0.2600  0.9895  0.1927  0.6539
H-bond donor            0.5815  1.0000  0.4424  0.7379
                        ------  ------  ------  ------
k(x)  (their sum)       2.8022  4.5159  2.6667  3.5849
```

and the prediction is that row times the coefficients, plus the bias:

```
y(x) = 2.8022(-0.1358) + 4.5159(-0.0581) + 2.6667(0.3487) + 3.5849(-0.1549) + 5.7124
     = 5.4444
```

which is exactly what *Pred (test)* shows for that molecule. Two things can be checked directly in this example rather than taken on trust. The steric and hydrophobic rows are nearly constant across the training set — these molecules differ by one halogen — so almost all the discrimination comes from the electrostatic and hydrogen-bond fields, which is visible in the spread of their rows. And the prediction, 5.44 against an observed 7.21, is poor, because four training molecules and no hyper-parameter search is not a model: the arithmetic is the lesson here, not the accuracy.

2.9 What changes for the other tasks

Model	What the prediction is
Kernel ridge	$\hat{y} = \mathbf{k}^{\mathrm{T}}\mathbf{a} + b$, as above
Gaussian process	the same mean, plus the variance $\sigma^2(x) = K(x,x) - \mathbf{k}^{\mathrm{T}}(\mathbf{K}+\lambda\mathbf{I})^{-1}\mathbf{k}$, which the *Align by prediction confidence* tool minimises
ε-SVR, ν-SVR	the same formula; most a_j are zero
C-SVC, ν-SVC	the same formula, read as a decision value: its sign is the class, its magnitude the margin
Laplace GP classifier	the same formula gives the latent mean, which is squashed to a probability and averaged over the latent variance
Kernel Fisher discriminant	the same formula gives the position along the discriminant direction, in units of the within-class spread; the bias is the threshold chosen out of fold, and a logistic map turns the score into a probability
Kernel logistic regression	the same formula gives the latent function, and the probability is the likelihood at it, $1/(1+e^{-f})$, with no variance averaged in; the boundary is $f = 0$, which is $p = 0.5$
One-class methods	a score built from the same kernel row, but also from the molecule's own self-kernel: distance to the centroid, to the k-th nearest active, to the principal subspace, or the 1-SVM decision value
Nonlinear kernel layer	the joint kernel is passed through $\exp(-\gamma\, d^2)$ or an arc-cosine function first; the prediction formula is unchanged, the co-field identity is lost; the derivative of the prediction at one molecule is still a field, the local co-field
Autocorrelation source	the kernel row comes from the rotation-invariant distance profiles instead of the overlap integrals; everything downstream is identical
Surface source	for molecules over a surface at $z = 0$: the in-plane distance profiles resolved by the heights of both atoms, invariant in the plane and to rotation about the normal only; heights and tilts are kept, no in-plane alignment is needed (command line: `--kernel surface --surface-z ZMIN ZMAX ZSTEP`)
Periodic source	for crystals: the distance profiles summed over the neighbour list of the cell with its images, per atom, background removed, cutoff tapered; invariant to the choice of cell, origin and orientation (command line: `build-crystals`). For layers and chains the same, with the background per unit area or repeat length and no pair across the vacuum (`--pbc ab`); a layer can be resolved by the heights of both atoms above its mean plane (`--heights`)
Two endpoints	the same formula once per endpoint, with coefficients tied through a task correlation; the difference of the two coefficient vectors gives the selectivity co-field

2.10 What has to match between fitting and predicting

Because the kernel row is computed against the stored training atoms with the stored constants, a prediction is only meaningful if the new molecule was prepared the way the training molecules were. The saved `.cmf` model therefore carries the field set, the α values, the charge model, the parameterisation, the normalisation and scaling flags, the mixing coefficients, the training atoms and the coefficients — everything in steps 1 to 5 — and reuses them. It also carries a record of how the training molecules were prepared: the version of the program that wrote it, the date, how many molecules, where their partial charges came from (the file, or computed), whether they carried

explicit hydrogens, and the box their atoms occupy. Before any prediction the program compares the new molecules with that record and reports what does not match:

- **The chemistry behind the weights.** Charges read from a file for the training set and Gasteiger charges for the new molecules are two different electrostatic fields under one name. A model fitted on file charges therefore **refuses** a molecule that carries none, rather than falling back to Gasteiger charges silently; `cmf_cli.py predict --allow-mismatch` overrides the refusal, and the application asks. Supply the charges (a MOL2 charge column or a `partial_charge` atom property) or build the model with computed charges.
- **Explicit hydrogens.** The donor field sits on the hydrogens and the steric field counts them, so a set with hydrogens and a set without are two different sets of fields. A mismatch is a **warning**.
- **The frame.** The overlap integral is not invariant to rotation or translation, so a molecule predicted in a different frame from the training set scores low on every field and is predicted towards the mean. A molecule whose centroid lies more than 4 Å outside the box of the training atoms is reported as a **warning**.
- **The pose inside the frame.** A molecule can be in the right place and still be turned the wrong way, and a rotation about its own centre moves no centroid, so the box test above sees nothing. The check that does see it compares the molecule with the *shape* the training set occupies rather than with its box: the share of its atoms that come within 1.5 Å of some training atom, against the share this training set asks of its own molecules — the fifth percentile of the same quantity measured on each training molecule against the others, less a small slack, so that a series of close analogues gets a strict rule, exactly as the applicability domain's threshold does. Measured on the eight Sutherland sets [17], turning every held-out molecule 90° about its own centre is caught **95 % of the time, for a false-positive rate of 2.6 %** on the molecules left as they were. If the new molecules are not in the frame, align them (*Preparing the data*) or use a frame-free kernel source (*Working without an alignment*).
- **Per-atom property fields** that are zero on a molecule, because it carries no value of that property, are a warning; a file written by another version of the program, or by a version that recorded no provenance, is a note.

The findings appear in the log of the application and on the console of the command-line tool, each marked `[refuse]`, `[warn]` or `[note]`. A model file written by a newer version of the program than the one reading it is not opened.

3 Installation and first run

3.1 Where the program lives

The repository is github.com/igbaskin/cmf-qsar — private, so the link asks for a GitHub account with access:

```
git clone https://github.com/igbaskin/cmf-qsar.git
cd cmf-qsar
```

Everything this manual describes is in that checkout: the `cmfqsar` package with the command-line tool inside it (`cmfqsar/cli.py`), the two launchers (`cmf_app.py`, `cmf_cli.py`), the example data under `examples/`, the benchmark scripts under `benchmarks/` with their recorded results, the tests, and this manual with the script that typesets it.

3.2 Requirements

Python 3.10 or newer. The package is declared in `pyproject.toml`, so from the checkout

```
pip install -e .[gui]
```

installs the library with RDKit (chemistry), scikit-image (marching cubes for the isosurfaces), scikit-learn (the SVM solvers), NumPy, SciPy and joblib, and with the `gui` extra PySide6 (the interface) and pyqtgraph with PyOpenGL (the 3D view); it also puts the commands `cmf-app` and `cmf` on the path. The library alone, `pip install -e .`, needs no Qt. The program is pure Python and needs no compilation step; it has been used on Windows, Linux and macOS.

Further extras, only needed for particular work:

- `docs` — reportlab and matplotlib (which supplies the DejaVu fonts) to typeset this manual;
- `dev` — pytest, for the test suite;
- `examples` — `dimorphite_dl`, protonation states when importing SMILES for a given pH;
- a local ORCA or VASP installation if you intend to compute per-atom quantum properties to use as fields (see *Preparing the data* and appendix A); these are programs, not packages.
- a local xtb installation to build and optimise solvation complexes (`cmfqsar/solvation.py`, *The ion with its shell as the molecule*). `cmfqsar/xtb.py` finds it by the `XTB_EXE` environment variable, on the path, or at its built-in default; it is also a program and not a package.

`pip install -r requirements.txt` installs the working checkout, `.[gui,docs,dev]`. `requirements-lock.txt` pins the exact versions the benchmark tables were produced with: `pip install -r requirements-lock.txt` and then `pip install --no-deps -e .` reproduces that environment. `CITATION.cff` says how to cite the software and the papers.

3.3 Starting the application

```
cmf-app
```

or `python cmf_app.py` from the checkout, which takes the same arguments; or directly with a data set, which skips two dialogs:

```
cmf-app examples/benzamides_synthetic.sdf --activity pActivity --test-fraction 0.25
```

The command-line tool is `cmf` once installed and `python cmf_cli.py` from the checkout; the rest of this manual writes the latter, and the two are the same program.

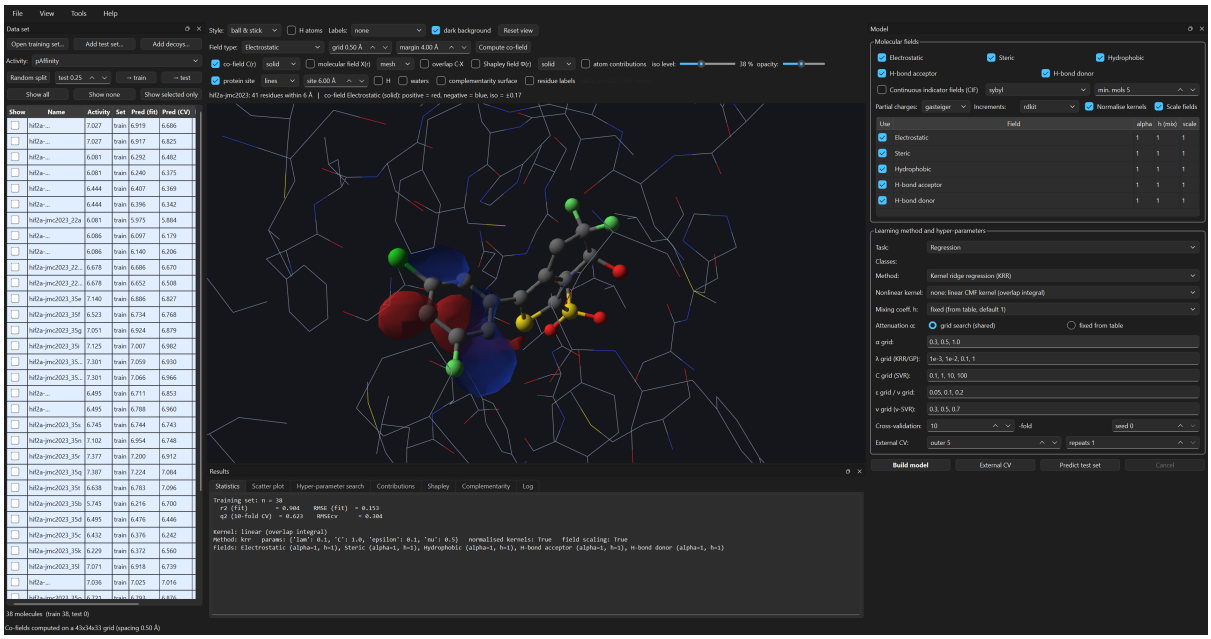
If the 3D view stays black, the OpenGL driver is the usual cause; see *When something goes wrong*.

3.4 A first model in five minutes

1. *File → Open training set…* and choose `examples/benzamides_synthetic.sdf`. It holds 109 aligned N-phenylbenzamides with a synthetic activity, so you know the answer the model should find. Pick `pActivity` as the activity property.
2. In the data-set table, press *Random split* and set a quarter of the molecules aside as a test set.
3. In the Model panel leave the five physico-chemical fields ticked, leave the method at kernel ridge regression, and press **Build model**.
4. Read the *Statistics* tab: q^2 should be about 0.87 and R^2_{pred} about 0.84 — the noise ceiling of that set is 0.94, so the model has recovered nearly all of the signal that was put into it.
5. Press **Compute co-field**, choose *Electrostatic*, and move the *iso level* slider. The red and blue lobes that appear are the pseudo-receptor zones the activity was generated from.

Nothing in that sequence is specific to the example: it is the working cycle of the program.

4 The application window



The HIF-2α series of the Merck free-energy benchmark, 38 ligands: a kernel ridge model with $q^2 = 0.62$, its electrostatic co-field drawn as ± isosurfaces, and the binding site of the reference crystal structure, which the data set brings with it and which needs no superposition.

The window is a 3D view with three dockable panels around it. The **3D view** and its control strip occupy the centre and are always there; the **Data set** panel docks on the left, the **Model** panel on the right inside a scroll area, and the **Results** panel across the bottom under the view. All three are ordinary dock widgets: drag one by its title bar to another edge, drop it on another to stack them as tabs, or pull it off entirely to float it on a second monitor. The layout is remembered between sessions, and *View → Restore default layout* puts it back when an experiment goes wrong.

At the bottom, the **status bar** shows what the program is doing, with a progress bar on the right during a long task. Every long task — a grid search, a co-field, an alignment, a conformer generation — runs in a worker thread, so the window stays responsive and the *Cancel* button in the model panel stops the work rather than the application.

4.1 The Data set panel

Three buttons at the top load structures: **Open training set…**, **Add test set…** and **Add decoys…**, the same actions as the first three entries of the *File* menu. Beside them the *Activity* selector names the SD property the activities are read from; changing it re-reads the column without reloading the file.

The table has one row per molecule and seven columns:

Column	Contents
Show	the tick that decides whether this molecule is drawn in the 3D view
Name	the molecule's title from the file
Activity	the experimental value, from the chosen property

Column	Contents
Set	train, test or decoy
Pred (fit)	the model's prediction for the molecule it was fitted on
Pred (CV)	its prediction when it was in the held-out fold
Pred (test)	its prediction as a test molecule
AD	its similarity to the training set, when the applicability domain was computed

An eighth column, **Chemotype**, appears once the chemotypes have been found. Rows are selected singly or in blocks, and the selection drives everything else: the co-field overlap, the atom contributions, the Shapley decomposition and the complementarity numbers all refer to the molecule currently selected.

Sorting. Clicking a column header sorts by it, a second click reverses the order, and a third restores the order of the file. Molecules with no value in that column are always placed at the bottom, whichever direction is in force, so an unpredicted molecule never hides among the predictions. The *Show* ticks, the selection and the 3D view follow the molecule and not the row, so sorting never disturbs what is on screen.

Splitting. *Random split* assigns a fraction to the test set; the → *train* and → *test* buttons move the selected rows by hand. A `subset`, `set` or `split` tag in the file is honoured on loading, which is how the published splits of the example sets arrive without any clicking.

Showing. *Show all*, *Show none* and *Show selected only* control the view in bulk. With one molecule shown, the maps in the viewer refer to it; with all of them shown, the co-field sits in the middle of the series as it should.

A line under the table reports the state of the data set — how many molecules, how many in each subset, which activity property — and a note in brackets records any transformation that has been applied, for instance *[topomer substituents of …]*, so that a changed data set never passes for the original.

4.2 The Model panel

Everything that defines a model, in the order one normally works down it. The panel scrolls; on a small screen nothing is hidden, it is merely below.

Molecular fields. Five tick boxes for the physico-chemical fields — Electrostatic, Steric, Hydrophobic, H-bond acceptor, H-bond donor — then *Continuous indicator fields (CIF)* with its typing scheme (`sybyl`, `element`, `element+aromatic`) and the minimum number of training molecules an atom type must occur in to be given a field. Below them, *Per-atom property fields* lists the per-atom properties the loaded file carries, each of which can be turned into a field of its own.

Partial charges chooses between Gasteiger, MMFF94 and the charges stored in the file; *Increments* chooses the parameterisation (`rdkit`, `tripos`, `tripos-legacy`). Two tick boxes set the kernel conventions: *Normalise kernels* for cosine-normalised field kernels, *Scale fields* to rescale every field kernel to unit mean diagonal before mixing — untick it for the raw-kernel convention of the papers.

Conformations switches between one conformation per molecule and an ensemble, with the weighting (uniform or Boltzmann), kT, a cap on the number of conformations, and the multi-instance settings (the softness β, and *one conformation* for the hard selection). A label beside

them reports how many conformations the data set currently holds.

A table at the foot of the group lists the fields that will actually be built, with the columns **Use**, **Field**, **alpha**, **h (mix)** and **scale**: per-field attenuation and mixing coefficients are editable there, which is how a field is given more or less weight than the others without leaving the panel.

Learning method and hyper-parameters. A form, row by row:

Row	What it sets
Task	regression, binary classification, or one-class (virtual screening)
Classes	for classification: a threshold on the activity, with *active below* for data where small is potent, or a class property from the file
Method	the learner; the list changes with the task — KRR, GP, ε-SVR, ν-SVR for regression; C-SVC, ν-SVC, Laplace GP for classification; 1-SVM, centroid, k-NN, one-class GP and kernel PCA for one-class work
Nonlinear kernel	none (the linear CMF kernel), a Gaussian layer, or one or two arc-cosine layers
Mixing coeff. h	fixed from the table, optimised by coordinate descent, or set by centred kernel alignment
Attenuation α	a single value, or the grid below
Kernel from	overlap integrals of the fields, which need one frame; the rotation-invariant autocorrelation, which does not; or the surface autocorrelation for poses over a surface at $z = 0$, which needs only the normal (command line)
α grid, λ grid, C grid, ε/ν grids	the search ranges, as comma-separated lists
Cross-validation	the number of folds and the shuffling seed
External CV	the outer folds and the number of repeats for the nested procedure
Probe fields	GRID-like fields for the water, methyl, carbonyl, amine and cation probes, and the number of Gaussians per atom
Computed descriptors	molecular weight, clogP, TPSA and rotatable bonds, computed by RDKit; all off by default
Descriptors (per molecule)	which numeric molecule properties enter as non-field descriptors, and the block's mixing coefficient
Validation	*shuffles* for y-randomisation, *bootstrap* for confidence intervals, and the *applicability domain* tick — all three off by default, all three run as part of **Build model** rather than from a button of their own

Four buttons close the panel: **Build model** runs the grid search, the final fit, the internal cross-validation and the test prediction; **External CV** runs the nested procedure and reports q^2_{ex}; **Predict test set** applies the current model to the test molecules; **Cancel** stops whatever is running.

4.3 The 3D view

The view itself is an OpenGL scene: drag to orbit, the wheel to zoom, middle-drag (or shift-drag) to pan. *Reset view* reframes the whole scene, *Tools → Center view on current molecule* reframes the selected one, and *File → Save 3D view as image…* writes what you see to PNG.

The control strip above the view is five rows, and everything on it redraws immediately:

- **Molecules.** Drawing style (ball & stick, sticks, lines, space fill), *H atoms*, *Labels* (none, element + number, atom type, partial charge), *dark background*, *Reset view*.
- **Maps.** *Field type* chooses which field the maps refer to; **Compute co-field** evaluates the co-fields on a grid around the training set; then the maps themselves — *co-field C(r)*, *molecular field X(r)*, *overlap C·X*, *Shapley field Φ(r)*, *atom contributions*, *difference map*, *pharmacophore*, *vector co-field (arrows)* — each with its own solid-or-mesh style where that applies. After a two-endpoint fit an *Endpoint* box joins the row and chooses whose co-field the maps show: either endpoint, or their difference, the selectivity co-field.
- **Levels.** *iso level*, the isosurface threshold as a percentage of the largest absolute value rather than an absolute number, so the surfaces stay comparable when the field changes; and *opacity*.
- **Protein site.** *protein site* draws the residues within the chosen distance of the molecules on screen, as lines or sticks, with *H*, *waters* and residue labels optional, and *complementarity surface* paints the molecule's surface by how well it complements the pocket.
- **Surface slab.** *surface slab* draws a patch of a loaded or built metal surface behind the molecules, as balls or dots, with the *patch* radius, the *frame axes* of the surface frame and the *surface sites* of the top layer. The section *A metal surface behind the molecules* is about this row.

Co-field colours are fixed, so that maps made in different sessions can be compared at a glance:

Field	Positive lobe	Negative lobe
Electrostatic	red	blue
Steric	green	yellow
Hydrophobic	orange	light blue
H-bond acceptor	magenta	dark red
H-bond donor	cyan	purple
Indicator field	red: an atom of that type is favoured here	blue: it is disfavoured here

4.4 The Results panel

Fourteen tabs, each filled by the work that produces it:

Tab	What it holds
Statistics	every figure of merit, for the fit, the cross-validation, the test set and the external CV; the y-randomisation, bootstrap and applicability-domain results follow at the bottom, under a *Validation* heading, when they were asked for
Scatter plot	observed against predicted, by subset, with the molecule under the cursor named
ROC	the receiver-operating curves of a classification model, per subset
Scores	the ranked decision values or calibrated probabilities: rank, molecule, score, set, class
Enrichment	the enrichment curve against a decoy set, with EF at 0.5, 1, 5 and 10 %, BEDROC and RIE

Tab	What it holds
Map	the kernel-GTM chemical-space map, with the activity, class and density landscapes, and a control for how the molecules themselves are drawn over it
Hyper-parameter search	the whole grid — α, the other parameters, RMSEcv and q^2 — so that an optimum at the edge of the grid is visible rather than hidden
Contributions	two tables for the selected molecule: the training molecules that make its prediction, with `a_j` and `a_j K(X, X_j)`, and its own atoms with their contributions
Co-pattern	the reading of a kernel that has no co-field: the channel weight against distance, for one pair of field types at a time. See *The co-pattern as a chart*
Surface poses	the height and the tilt of every molecule in the surface frame, filled by *Tools → Pose the molecules in the surface frame*
Shapley	the Shapley decomposition, over atoms, substituents of the common scaffold or field types, with second-order interactions
Complementarity	the field complementarity of the molecules and of the co-field with the pocket, by sign agreement, cosine or correlation
Chemotypes	the Murcko scaffolds found in the set, their sizes, and the leave-one-chemotype-out result beside the alignment-free baseline
Log	everything the program did, with timings and warnings — including the names of the molecules that fell outside the applicability domain — the first place to look when a result surprises you

4.5 The menus

File — *Open training set…* (Ctrl+O), *Add test set…*, *Add decoys (background set)…*, *Import activities from CSV…*; *Crystals and other periodic structures…*, which opens the window of the section below; *Load protein structure (PDB/mmCIF)…*, *Fetch protein from the PDB…*, *Forget protein structure*; *Load a surface slab (POSCAR/CIF/XYZ)…*, *Build a bcc(110) surface slab…*, *Forget the surface slab*; *Open project…* (Ctrl+Shift+O), *Save project* (Ctrl+Shift+S), *Save project as…*; *Load model…* (Ctrl+L), *Save model…* (Ctrl+S); *Export predictions (CSV)…*, *Export aligned data set (SDF)…*, *Export current co-field (Gaussian cube)…*, *Export current Shapley field (Gaussian cube)…*, *Save 3D view as image…*; *Quit* (Ctrl+Q).

View — show or hide each panel, *Results panel spans full width* for when the tables and plots matter more than the view, and *Restore default layout*.

Tools — *Generate conformations (4D)…*; *Dock into the pocket…*; *Align data set to reference…*; *Pose the molecules in the surface frame…*; *Align by prediction confidence (GP variance)…* and its cross-validated form for the training set; *Replace molecules by their topomer substituents…*; *Field difference from a reference molecule…*; *Extract pharmacophore from the co-field…*; *Model several endpoints (selectivity co-field)…*; *Chemical space map…*; *Align decoys to the training set…*; *Unalign (randomise poses)…*; *Undo last alignment step* (Ctrl+Z); *Compare two models on these molecules…*; *Center view on current molecule*; *Quantum-chemical properties with ORCA…* and *Save the data set with its properties (SDF)…*; and the submenu *Heterogeneous sets (several chemotypes)* with its four numbered steps.

Help — *About CMF 3D-QSAR / method summary*, which names the equations of the papers that each part of the program implements.

Every *Tools* entry opens a dialog that states what it is about to do and on which molecules, and each one runs in the background with the progress bar and the *Cancel* button live.

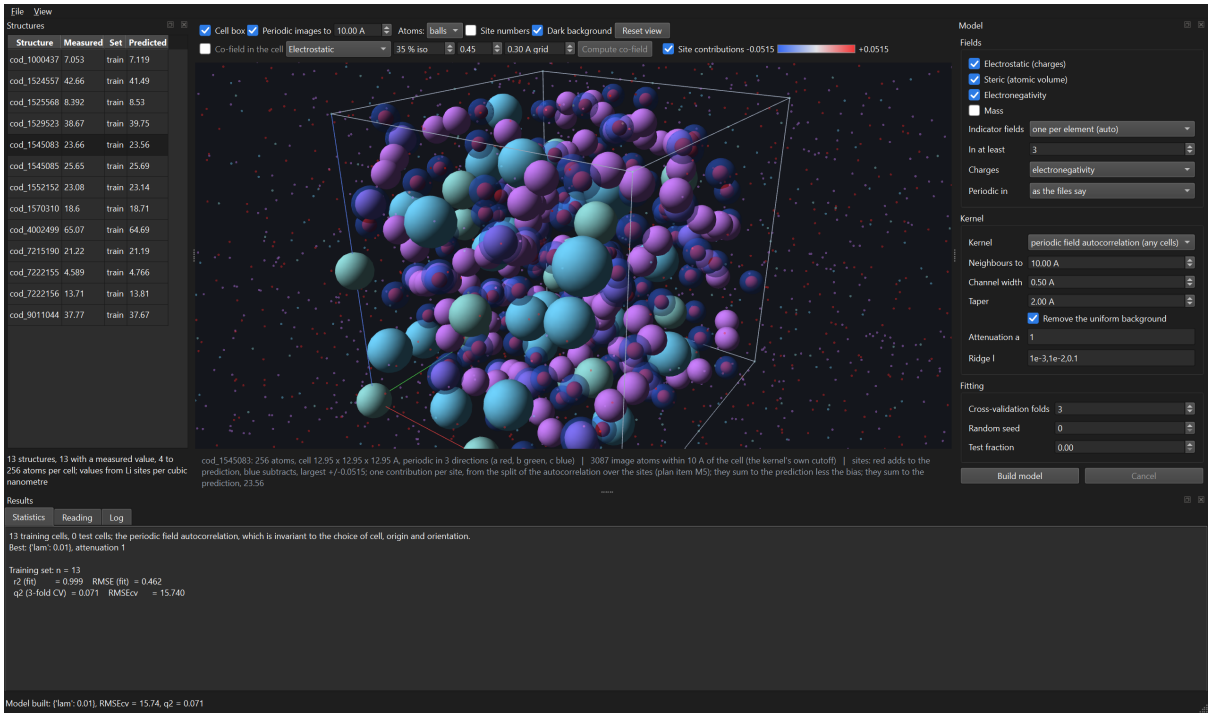
4.6 Crystals and other periodic structures

Everything the program does with crystals — the periodic autocorrelation kernel, the periodic overlap kernel with its co-field inside the cell, the split of a prediction over the sites — lived on the command line and in the library until version 1.0. *File → Crystals and other periodic structures...* opens a window of its own for that side of the program. It is a window and not a mode of the main one, because a crystal is not a molecule with a box around it: it has a lattice instead of a pose, no bond table, and half of what its kernel sees lies outside the cell, in the periodic images. The two windows do not share a data set.

Opening a set. *File → Open a folder of structures...* reads every structure file in a folder (CIF, POSCAR or CONTCAR, extended XYZ, ASE JSON, and the other formats ASE reads), one structure per file, named after its file; this is how a set downloaded from a database arrives. *Open one structure file...* reads a file that holds one structure or many, as `examples/sac/` and `examples/hops/` do. Both routes read through ASE, so any format ASE reads is accepted. A folder is searched for the file endings `.cif`, `.vasp`, `.poscar`, `.contcar`, `.extxyz`, `.xyz`, `.json` (an ASE database), `.traj` (an ASE trajectory), `.cell` (CASTEP), `.res` (SHELX), `.xsf` (XCrySDen) and `.pwo` (Quantum ESPRESSO output), and for files named `POSCAR` or `CONTCAR`, which VASP writes without an ending; a file with any other ending is left alone, so notes and tables may sit beside the structures. The file dialog lists the common endings; a file of another ASE format is opened through *All files*. A CIF with partly occupied sites is read with its occupancies, as the section on partial occupancy describes. The *Structures* panel lists what came back with the measured value, the subset and the prediction of each cell, and the line under it says how many atoms the cells have.

The measured values. *File → Read the measured values from a CSV...* takes a table whose first column holds the structure names as the panel shows them. A plain `name,value` file needs no further choice; a prepared table with one column per property asks which column is the property. A per-structure property stored in the file itself (the `info` key of an extended-XYZ file) is read with `--target KEY` on the command line and is not offered in the window.

What the view draws. The cell is drawn as the twelve edges of its box, with the first, second and third lattice vectors in red, green and blue. Its atoms are spheres coloured by element, with the colours the molecular view uses for the organic elements and the standard Jmol colours for the rest. Outside the box, *Periodic images* draws every copy of the cell's atoms within the radius beside it, as dots: these are the atoms the periodic kernel sums over, and seeing them is how one checks that the cutoff reaches as far as it should. Once a model is fitted the radius is set to the kernel's own cutoff — the one of the neighbour list for the autocorrelation, the one the overlap kernel's Gaussians reach to for the overlap — and the line under the view says so. The legend also names the cell: its atoms, its lattice lengths, and in how many directions it is periodic. *Site numbers* writes the index and element of every site beside it, which is how a site in the Reading table is found in the picture.



The window for crystals with the thirteen solid electrolytes of `examples/hops/disordered` *and a periodic-autocorrelation model fitted to the number of lithium sites per cubic nanometre, the target* `benchmarks/run_cell_view.py` *uses. On screen is the largest cell, the cubic garnet Li6.5La3Zr1.5Ta0.5O12 with 256 atoms: the cell box with its lattice vectors, the atoms as spheres coloured by element, the periodic images out to the kernel's cutoff as dots, and over every site the sphere of its contribution to the prediction, red where the site adds to it and blue where it subtracts. The line under the view names the cell and says that the contributions sum to the prediction. Taken by* `docs/make_screenshot.py --what crystal`.

The images are cheap. `benchmarks/results/cell_view/table.md` measures the window on the three example sets: a 256-atom cell of the garnet $Li_{6.5}La_3Zr_{1.5}Ta_{0.5}O_{12}$ has 6,581 image atoms within 14.9 Å, expanded in 6 ms and drawn, with the cell and its atoms, in 11 ms; the 112- to 128-atom supercells of the three solid electrolytes of the hop section take 10 to 13 ms; a 127-atom sheet of the single-atom catalysts, periodic in two directions, has 836 images and draws in 3 ms. The thirteen CIF files of `examples/hops/disordered/` are read in half a second.

How the atoms are drawn, and the bonds. The *Atoms* box offers four representations. *balls* is one sphere per atom, coloured by element and sized by its covalent radius, and *dots* is one point per atom, which stays fast for a cell of thousands. Those two show the atoms alone, which is the honest picture of a file that states where the atoms are and nothing about which of them are bonded. *ball and stick* and *sticks* add the bonds.

The bonds are perceived from the distances, because no CIF or extended-XYZ file carries a bond table: a pair is bonded when its distance is below the sum of the two covalent radii plus 0.45 Å, which is the rule the protein reader already applies, with no stick drawn longer than 2.4 Å. Two things make it work for a periodic structure. The pairs are taken over the cell **and its periodic images**, so a ring that the file leaves split across the cell boundary is drawn as a ring, with the sticks leaving the box and the image atoms they end on drawn beside them; a bond that crosses the boundary is drawn from both of its ends, because both halves of it are inside the box, and the legend states the number of bonds and the number of sticks apart. And **a metal atom gets no bonds**: a distance rule gives every atom of a close-packed metal about twelve neighbours, and twelve sticks per atom hide the structure they are meant to show. So a slab stays spheres and the organic molecule adsorbed on it gets its bonds, which is the one picture an adsorbate study cannot

do without. A cell whose atoms get no bonds at all falls back to *balls* and the legend says why rather than showing an empty box.

This perception is **for drawing only**. No bond order is assigned, no molecule is sanitised and no charge is perceived from it; a stick on screen never reaches a field, a kernel or a prediction. Turning a periodic structure into a molecule is a different job (plan item M8) and this is not it.

What a metal gets instead: coordination polyhedra. *Coordination polyhedra* draws the first coordination shell of every metal atom as a translucent polyhedron. The shell is found without any chemistry: the nearest neighbour of the atom sets a distance, and every neighbour within 1.3 times that distance is a vertex, with the non-metal neighbours preferred where the atom has any, so the vertices of an ionic structure are its anions. Neighbours are taken over the periodic images, so a polyhedron at the edge of the cell is whole. A flat shell -- square-planar, trigonal-planar -- has no volume, so it is drawn as a polygon and the legend says how many were. A shell of fewer than three neighbours is not a shape and gets nothing, and the legend counts those atoms too: on a disordered conductor whose lithium sites are partly occupied, the nearest oxygen of a site can be more than 1.3 times closer than the next one, and then the rule leaves a shell of one atom. Widening the ratio would draw those and inflate every well-behaved shell as well, so the convention stays and the number is stated instead. This is how an inorganic structure is read: a chain of corner-sharing octahedra says more than any number of sticks would, and it is the reading the bonds deliberately do not offer for a metal.

What the polyhedra are *not* for is a metal slab. Every atom of a close-packed metal has twelve or fourteen neighbours, so its polyhedron says only that the metal is close-packed, which the spheres already showed; and on a slab with an adsorbate the few iron atoms under the molecule take its carbon and nitrogen atoms as their vertices, because non-metal neighbours are preferred, which is the rule working as written and not a reading worth having. The option is for an ionic or framework structure -- an oxide, a garnet, a zeolite -- and it is off until it is asked for.

What the drawing is worth, measured. `benchmarks/results/surface_view/table.md` checks the perception against a bond table that was not perceived. The molecules of the two Fe(110) sets came from SDF files, which do carry one, so there is a ground truth on 138 real molecules with their hydrogens: in the open the rule returns the molecule's own bonds for 72 of 73 and 63 of 65 of them, the misses being long bonds in a relaxed pose that the radii plus 0.45 Å do not reach across. Put each molecule on the Fe(110) slab in its 16.68 x 11.79 Å cell and wrap it so that the box cuts it in two, and the perception over the images returns the molecule's own bonds for **65 of the 65** and **57 of the 58** molecules that fit inside that cell. The ones it gets wrong are almost exactly the molecules longer than a cell edge: those touch their own periodic image, so two atoms far apart in the molecule become neighbours across the boundary and the rule joins them. That is a property of the cell and not of the rule, and it is the one caution to carry into a picture: a long inhibitor in a small surface cell needs a wider cell, or sticks read with care. No bond ever reaches the 144-atom slab, which the benchmark asserts rather than reports. Perceiving the bonds of one molecule costs 0.1 ms in the open and about 1 ms in a cell; on the largest cells of `examples/hops/disordered/`, 192 and 256 atoms, the bonds take 2 ms and the polyhedra 29 to 59 ms, so neither option makes the window wait. Those garnets are also the case the two options were built as a pair for: only their oxygen is a bonding element and no two oxygen atoms are within 2.4 A, so the distance rule gives them no bonds at all and the polyhedra are the whole of their reading.

An organic molecule on a surface, in the window. This is the case the representations exist for, and the repository ships the file to open:

```
cmf-app                # File -> Crystals and other periodic structures -> Open one structure file
#   examples/corrosion_fe110/benzimidazole_on_fe110.extxyz
#   Periodic in: ab: a layer      Atoms: ball and stick
```

It holds eight of the benzimidazoles of the Fe(110) study on the slab they were relaxed on: the 6 x 6 x 4 Fe(110) slab of 144 atoms with one molecule at its relaxed pose, periodic in the two surface directions and open along the normal, with the adsorption energy of each in an info field, so *Read the measured values* is not even needed -- `--target E_ads_kJmol` on the command line reads it. With *ball and stick* the molecule comes out with its rings while the 144 iron atoms stay spheres, because a metal atom gets no perceived bonds. That picture is the whole point of the two representations.

`examples/corrosion_fe110/make_adsorbate_cell.py` writes that file from the two halves the repository already had -- the relaxed poses in `157_benzimidazole_uma.sdf`, which hold the molecules alone, and `cmfqsar.slab.bcc110_slab` -- because a data set of adsorbates keeps the molecules alone and leaves out the slab every one of them shares. Two of its options matter here: `--count` writes more molecules, and `--place corner` keeps each pose where the SDF has it, which is about x = y = 0 and therefore at the corner of the slab's cell, so the molecule straddles the boundary. That is the file to open to see a bond drawn across the cell boundary with the image atoms it ends on beside it; the shipped file uses `--place centre`, a translation by half a cell that changes no distance and leaves the molecule in the middle of the picture.

Your own relaxed state needs no script: one file holding a slab and an adsorbate is what a relaxation writes, so open its `CONTCAR`, CIF or extended-XYZ file directly.

Fitting a model. The *Model* panel is the crystal form of the main window's: the fields (electrostatic, steric, electronegativity, mass, and one indicator field per element that occurs in at least the stated number of cells, plus one for the rest), where the electrostatic weights come from (the oxidation-state guess, the electronegativity proxy, or the charges a calculation wrote into the file; the guess is exact for a small ionic cell and slow for a large cell of many elements), in how many directions the cells are periodic (a layer is summed only in its own plane, and the uniform background is taken per unit area), and the kernel:

- **periodic field autocorrelation (any cells)** — the distance descriptor of the section *Crystals: the periodic autocorrelation kernel*, with its cutoff, channel width, taper and background switch. It needs no common frame, so it is the kernel for a set of different structures, such as the thirteen conductors of the example folder.
- **periodic overlap (cells on one lattice)** — the overlap integral over one cell with the images summed in, from the section *A series on one lattice*. It compares two cells point by point, so it is for a doped, substituted or defective series on one parent lattice, such as the single-atom sheets of `examples/sac/`, and it is the kernel that has a co-field.

The attenuation and the ridge are grids, as in the main window; *Build model* runs the search and the final fit in the background with the progress bar live, and the *Statistics* tab shows the summary. The fitted model is saved with *File → Save model...* and is the same file `build-crystals` writes, so `cmf predict` applies it to new cells.

Reading the model. Which reading applies depends on the kernel, and the window offers the one that exists rather than both:

- The **co-field inside the cell** exists for the overlap kernel. *Compute co-field* evaluates $C_f(\mathbf{r})$ of the chosen field on a grid over the box of the selected cell (the grid is cut to the atoms along a direction with vacuum, so a layer does not spend its points on empty space), and the two

isosurfaces are drawn in the fixed colours of the field at the chosen percentage of the largest value, as the main window draws them: a positive lobe is a place where more of that field raises the property, a negative lobe a place where it lowers it. The co-field of a periodic model is a sum over the images of the training cells out to the kernel's cutoff; the benchmark doubles that cutoff and finds that the images beyond it change the co-field inside the cell by 8×10^{-15} of its peak, so the sum is complete. On twenty sheets of 127 atoms the steric co-field on 30,744 points at 0.5 Å takes 1.4 s. The autocorrelation has no co-field, because a distribution of distances cannot say *where*; the box is disabled for it, and clicking the button says why.

- The **site contributions** exist for both kernels and are the same decomposition seen from two sides. *Site contributions* draws one translucent sphere per site, red where the site adds to the prediction and blue where it subtracts, its radius growing with the square root of the magnitude so that the area on screen follows the value, and the colour scale on the control strip states the value the largest sphere stands for, in the units of the property. For the overlap kernel the numbers are the atom contributions of decomposition rule 3; for the autocorrelation they are the split of the descriptor over the sites of plan item M5. In both cases they sum to the prediction less the bias, and the legend says whether they do: for the autocorrelation the residual is rounding (6×10^{-14} on the thirteen conductors); for the overlap kernel the integral runs over the cell of one structure and the images of the other, so the two orders agree only on exactly one lattice, and on the relaxed sheets, whose lattice vectors differ by 0.4 %, the sum misses the prediction by 0.007 on a property whose spread is 0.34. The legend reports that difference whenever it exceeds a tenth of a per cent of the prediction, with the reason.
- The **Reading** tab holds the same numbers as a table: for the autocorrelation the co-pattern of the model — the channels (a pair of fields at a distance) that move the prediction most, with the weight, its spread over the training set and their product, which is what `build-crystals` prints — and for either kernel the sites of the selected cell, largest magnitude first, with their positions. Beside that table the same co-pattern is drawn as a curve of weight against distance, one pair of field types at a time, so that the table says which channel and the chart says what the profile over distance is; the section *The co-pattern as a chart* is about it.

A check the reading passes on its own: the property of the single-atom sheets used by the benchmark is the height of the metal above the sheet, and the largest per-site contribution of the first cell is the metal itself, at −0.39 Å against a largest sphere of ±0.39. The colouring points at the atom the property belongs to.

A worked example. Open `examples/hops/disordered/` with *Open a folder of structures…*, leave the periodic directions *as the files say*, choose the autocorrelation kernel, and build: the thirteen conductors have no common lattice and a measured property of your own goes in from a CSV. Then open `examples/sac/substrates.extxyz` with *Open one structure file…*, set the periodic directions to *ab: a layer*, read `examples/sac/sac_hoer.csv` and pick the column `height`, choose the overlap kernel, set the charges to *electronegativity* (a sheet of carbon with one metal has no oxidation states worth guessing), and build; then *Compute co-field* on any sheet and tick *Site contributions*. The benchmark's twenty sheets fit in 2 s with $q^2 = 0.99$ on three folds, which says only that the height of a metal above a sheet is an easy property for a field that sees the metal; the kernel costs grow with the square of the number of cells.

`cmf predict` **on a crystal.** A model fitted with `build-crystals`, or saved from this window, is applied from the command line like any other:

```
python cmf_cli.py predict cells.cmf new_cells/ --activity-file values.csv --activity-column dG
python cmf_cli.py predict sheets.cmf more_sheets.extxyz --pbc ab --out predictions.csv
```

The structures are read the way `build-crystals` reads them — one file or a folder of files, `--pbc` for the periodic directions — and the model decides the path: a periodic model takes crystals, a molecular model takes molecules, and the options that only mean something for a molecule (`--align-variance` and the rest) are refused for a crystal rather than ignored. The contract check of the section *What has to match between fitting and predicting* asks the questions a crystal raises instead of the ones a molecule raises: a structure without a lattice is refused; an element the field set has no indicator field for is reported, as a warning when it falls into the `cif:other` field and as a refusal when no field collects it; an element the training cells never contained is named as an extrapolation; electrostatic weights from another source than the training cells' are a warning; a cell whose lattice vectors differ from the training lattice by more than the training set's own spread, or 10 %, is a warning for the overlap kernel, which compares cells point by point; and partly occupied sites in a set fitted on ordered cells, or the reverse, are reported. A model fitted on sites of a cell (plan item M5) is not a crystal model, and `predict` says so instead of guessing.

What the window does not do. It fits the two whole-cell kernels and nothing else: no site-resolved model, no difference fields of a defect series, no three-body or reciprocal-space channels, which stay in the library and the benchmarks. It draws one cell at a time. The images are dots and not spheres, because a cell with its images is thousands of atoms, and only the image atoms a bond ends on are drawn as atoms. The bonds it draws are perceived for the picture and are not a bond table the program will use for anything else. And it keeps no project file: the model is saved, the structures are the files they came from.

4.7 A metal surface behind the molecules

The 3D view draws a protein pocket behind a ligand. An electrode is the same problem with a different receptor: a slab, periodic in the two directions of the surface, with no residues and no edge. The section *A metal surface as the receptor* is about what such a receptor is worth; this one is about seeing it.

Loading one. *File → Load a surface slab (POSCAR/CIF/XYZ)…* reads a slab from any structure file ASE reads and shifts it along z so that its highest atom sits at $z = 0$. *Build a bcc(110) surface slab…* builds one instead, and its defaults are the slab the Fe(110) adsorption energies of `examples/corrosion_fe110` were relaxed on: iron, a 6 × 6 orthogonal surface cell, four layers, a = 2.78 Å, which is the minimum of the UMA potential for body-centred cubic iron rather than the experimental 2.87. *Forget the surface slab* takes it away. The slab is not part of a project file: it is the file it came from, or four numbers in a dialog.

The frame. The surface frame of this program is the normal along z with the top atomic layer at $z = 0$, which is the frame `examples/uma_fe110_adsorption.py` writes its poses in and the frame `cmfqsar/planealign.py` poses a molecule in. A frame that is not drawn is a frame the reader has to take on trust, so *frame axes* draws the three axes at the origin, red for x and green for y in the surface and blue for z along the normal, the same colours the lattice vectors of a cell carry in the crystal window. If the molecules on screen reach below the top layer they are not in this frame, and the log says so when the slab is installed.

The patch. A surface has no edge, so there is no cell to draw. What is drawn is the top two layers repeated over the surface lattice and then cut to the atoms whose distance to a molecule is at most the *patch* radius — measured **in the plane**, not in space, so the surface stays a surface under a molecule that sits high above it instead of thinning out to nothing. A small radius shows the patch under the molecule and a large one shows that the surface continues. `benchmarks/results/surface_view/table.md` measures the cost on both Fe(110) sets: on the benzimidazole set a 6 Å patch is 93 surface atoms and a 24 Å patch 845, chosen in 1 ms either way,

and the whole view — the patch, the molecule, the frame — is drawn in 3 ms and 11 ms. Every drawn atom lies at or below $z = 0$, and all of them are atoms of the complementarity receptor `cmfqsar.slab.slab_receptor` builds, so the picture shows what a score against the surface is computed from and not something that merely looks like it.

The sites. *surface sites* marks the top, bridge and hollow sites of the top layer as small spheres, yellow, cyan and magenta. They come from a Delaunay triangulation of the top layer, so they are found for any surface and not only for the lattice the program can build, and what a molecule sits over is the surface's answer to a residue profile.

The pose, in two numbers. A molecule in the surface frame has four free motions and the surface kernel sees two of them: how high it sits and how far it is tilted. Where it sits in the plane and how it is turned about the normal are what that kernel is invariant to. So the line under the view states the two the kernel sees, for the selected molecule: the height of its lowest heavy atom above the surface, the height of its heavy-atom centroid, and the tilt — the angle between the molecule's own plane, spanned by its two longest principal axes, and the surface, so 0° is a molecule lying flat and 90° one standing on edge. A molecule with no plane to speak of is told apart rather than given a meaningless angle: the line then says how thick it is, and the *Surface poses* tab has a column for it.

Tools → Pose the molecules in the surface frame… puts every molecule on $z = 0$ by rule — the rules of the section *Posed by rule: the surface frame without the slab* — and fills that tab with one row per molecule. The step moves the molecules rigidly, so it joins the pose history and *Undo last alignment step* takes it back.

The two numbers are checked against something the window never reads. The relaxation that produced the Fe(110) poses wrote the height of each one into the data set as a column of its own; the window computes it from the coordinates, and on all 138 molecules of the two sets the two agree to 0.005 Å. The same measurement says what the poses are: the relaxed heights run from 1.23 to 4.83 Å, the median tilt is 18° on the benzimidazole set and 27° on the quinoline set, and only 29 % and 14 % of the molecules have a flat plane at all. A molecule relaxed on iron is therefore not lying flat, and the flat rule pose differs from the relaxed tilt by that same median — which is the honest form of what the section on the rule poses says about them.

What this does not do. It does not dock: the pose search constrained to a surface is `cmfqsar.slab.dock_on_surface` and is measured in the slab benchmark, not offered in a dialog. It draws one slab at a time. And the drawing says nothing about the fields: on a one-element metal four of the five physicochemical fields are identically zero, so a complementarity score against a clean metal is a shape score, and the section *What a clean metal surface offers* states why.

4.8 The vector co-field: a direction instead of a value

Every field of this program is a scalar except one. The electric-field kind of the section *The field a reaction centre feels* carries the same charges as the electrostatic field and pairs two molecules by the overlap of the **gradients** of their fields rather than of the fields themselves. It is offered in the Model panel as *Electric field (vector)*, and it starts unticked, because whether it is worth adding is a question per series that `benchmarks/results/efield/table.md` answers.

Its co-field cannot be an isosurface. The co-field of a scalar field is a value at a place and a surface at one value is the natural drawing of it; the co-field of this kind is a **vector** at a place, and what it says there is a direction: which way the molecule's own electric field should point to raise the predicted property. So it is drawn as arrows. Tick *vector co-field (arrows)* in the maps row, with *Compute co-field* already done and the *Field type* box on the electric-field kind, and the number

beside the box says how many arrows are drawn: the longest ones, kept at least 1.5 Å apart so that they spread over the molecule instead of crowding one peak.

The arrows are scaled so that the longest one is 2.5 Å on screen, and the legend states what that length stands for, in activity units per unit of the field per angstrom. A length scale on screen means nothing without that number, and a vector field drawn without its unit is a picture rather than a reading.

`benchmarks/results/surface_view/table.md` measures the drawing on 95 Hammett benzenes with the electric-field kind as the only field and σ_m as the property. Sixteen arrows on a 0.5 Å grid of 21,600 points take 0.7 s. The longest arrow is the largest gradient the grid holds, at every spacing tried, so the arrows do stand on the strongest places; the weakest arrow drawn is 40 to 50 % of the longest, and the gradient is at least that strong in 1.5 to 2.2 % of the volume, so the drawing is the strong part of the reading and not a sample of all of it. One thing the measurement found is worth knowing before an angle is read off the screen: refining the grid from 0.7 Å to 0.35 Å turns the arrows by about 16° on average, because an arrow is taken at the grid point nearest a peak and a finer grid has a nearer one. An arrow states a direction to roughly fifteen degrees and not better. That is enough for the question it answers and not enough to measure an angle from the picture.

Ticking the box while the *Field type* box is on a scalar field draws nothing and the legend says why: that field's co-field is a value at a place, and the isosurface is its drawing.

4.9 The co-pattern as a chart

The invariant kernels have no co-field — the autocorrelation of a molecule, the surface autocorrelation of an adsorbate, the periodic autocorrelation of a cell, the reciprocal-space channels. They compare two structures by a distribution of distances, and a distribution of distances cannot say *where* in a structure anything is. What they have instead is the co-pattern: one weight per channel, a channel being one pair of field types at one distance. `periodic.top_channels` ranks those weights and the tables print them.

Twenty ranked channels out of several thousand are a table, not a shape. The same numbers for one pair of field types, drawn against distance, are the shape: a positive part at 3 Å and a negative part at 6 Å says that the property wants those two fields close together and not far apart. The *Co-pattern* tab of the Results panel draws that curve in the main window, and in the crystal window the chart sits in the *Reading* tab beside the ranked table, so the table says which channel and the chart says what the profile over distance is.

Two boxes above the chart: which **pair of field types** the curve is for, and whether the *y* axis is the **weight** — the model's own coefficient of the channel — or the **weight times spread**, the coefficient multiplied by how much that channel varies over the training structures, which is how much it actually moves a prediction and is the ranking the table uses. A weight at a distance no structure varies over means nothing, so the second is usually the one to read and the first is the one to quote.

The *x* axis is whatever the kernel's channels are indexed by, and the label says which: the distance between two atoms for the molecular and the periodic autocorrelation, the in-plane distance for the surface kernel, the reciprocal length *q* for the reciprocal-space channels. Two kernels have no such axis and the chart says so instead of drawing something: the three-body channels of a cell index a channel by two radii and a separation, and the signed chirality channels by a radial shell. For the kernels that resolve a distance **further** — the surface kernel resolves it by the heights of both atoms — a point of the curve is the sum of the weights of the channels at that distance, and the note under the chart says how many were summed. The sum is the right summary because a

prediction is a sum over channels, and the table beside the chart still names the individual height channels.

What makes this a reading and not a picture is that the weights reproduce the prediction: the bias plus the sum over every channel of weight times channel is the prediction itself. `benchmarks/results/surface_view/table.md` measures the residual of that identity on two kernels that have no co-field — 8×10^{-14} on the periodic autocorrelation of the thirteen COD electrolytes, whose property has a spread of 16.3, and 2×10^{-12} on the surface autocorrelation of the 73 Fe(110) adsorbates, whose property has a spread of 63.8 kJ/mol. Both are rounding. The same table says what the chart opens on: on the thirteen conductors the leading pair is the electrostatic field with itself, 26 % of the whole reading, peaking at 8.0 Å; on the Fe(110) adsorbates it is the electrostatic field with the hydrophobic one, 28 %, peaking at 2.25 Å in the plane. One curve costs between 0.1 and 1.1 ms to build, so the chart follows the box without a wait.

The same numbers are available without the window. `cmfqsar.copattern.copattern` returns one curve and `copattern_rows` returns it as a table, with the distance, the weight, the spread and their product per row — which is what goes into a paper.

4.10 Projects: a session that can be reopened, a step that can be undone

A session is more than a model. It is the molecules with the poses they have now, the poses they had before each step that moved them, the settings of every panel, the model with the molecules it was fitted and tested on, the search table, the validation report and the log. *File → Save project* writes all of it to one file (`.cmfproj`), and *Open project* brings the session back as it was: the panels show the settings, the Results panel shows the model's statistics and plots, the log of the saved session is appended to the new one, and the protein structure comes back with the molecules. A project file given on the command line is opened the same way:

```
python cmf_app.py series.cmfproj
```

The model inside a project is the same object a model file holds, so *Save model* still exists for the case where only the model is wanted, for `cmf predict`, `cmf screen` or a colleague.

Undo. Every step that moves molecules — *Align data set to reference*, the two confidence-based alignments, *Unalign*, *Dock into the pocket*, *Generate conformations*, *Align decoys* and the two chemotype steps that move anything — first keeps a snapshot of every conformer of every molecule. *Tools → Undo last alignment step* (Ctrl+Z) puts the molecules back to the latest snapshot and names the step it undid; a second Ctrl+Z undoes the step before, and so on. The molecules move, so the model fitted on the previous poses is discarded, as it is after any alignment; the fields are rebuilt and the view follows. The snapshots travel with the project, so a step can be undone after the session has been closed and reopened. The last fifty steps are kept; a snapshot taken before conformations were generated removes them again when it is restored.

Two models on the same molecules. *Tools → Compare two models on these molecules...* takes model A and model B — the model of this session, a model file or a project file for either — and predicts every loaded molecule with both. The window that opens lists, per molecule, the experimental value, the two predictions and their difference, sorted by the difference so the molecules the two models disagree about come first; above the table are the RMSE and r^2 of each model on the training, test and whole set, the correlation of the two prediction vectors, the mean and largest difference, and on how many molecules B is nearer the measurement than A. The same lines go to the log, and *Save as CSV...* keeps the table.

Both models see the molecules as they are posed now. That is what makes the comparison useful after a realignment — build a model, realign, build again, compare the two on the new poses — and it is also the caveat: a model fitted in another frame is being asked about poses it never saw, and will look wrong because the frame moved, not because the model is worse. Compare models that share a frame, or compare on the frame of the model being judged.

5 Preparing the data

5.1 What the program reads

Format	Notes
`.sdf`, `.sd`, `.mol`	3D coordinates used as they are; hydrogens are added with coordinates if missing
`.mol2`	Sybyl atom types and partial charges are read and can be used directly; a tolerant fallback reader handles blocks RDKit refuses
`.smi`, `.csv`, `.tsv`, `.txt`	2D input: each molecule is embedded with ETKDG and minimised with MMFF94, which means **the poses are arbitrary** until you align them
`.pdb`, `.mmcif`	protein structures only, read by a deliberately permissive reader

A CSV needs a column of SMILES; a column called `name` or the first non-SMILES column names the molecules, and every other column is kept as a molecule property, so activities and metadata travel with the file.

5.2 Activities

Three ways, in order of convenience:

1. **An SD property.** The loader guesses the activity property and the dialog lets you change it. Anything numeric will do: `pIC50`, `pKi`, `pActivity`, `CBG_pK`.
2. **A separate table** (*File → Import activities from CSV…*, `--activity-file`): molecule name, activity, and optionally a third column with the split (1 = train, 2 = test, or the words).
3. **A class property** for classification tasks, or a threshold on the activity.

A per-molecule tag named `subset`, `set` or `split` is honoured automatically, which is how the published splits of the example sets arrive.

5.3 Alignment

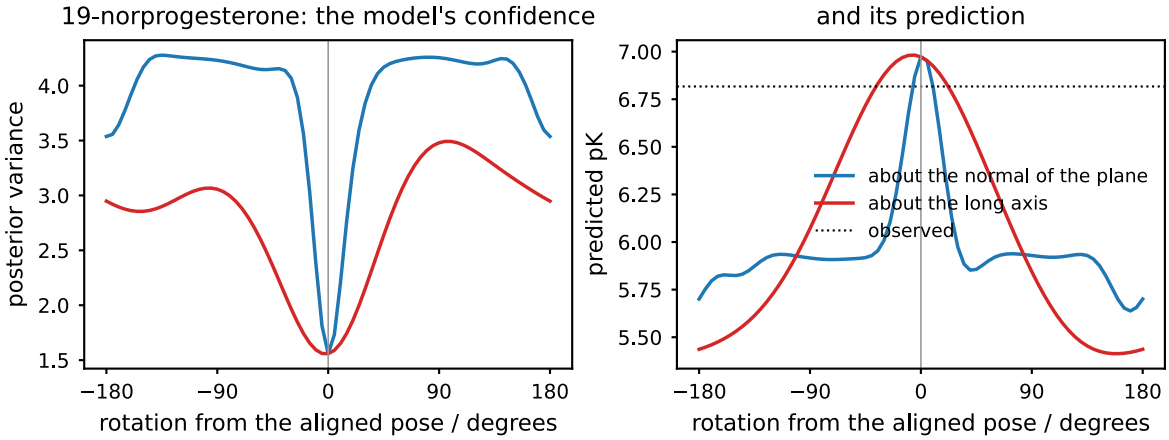
CMF compares fields in one frame of reference. If your molecules came from a superposition, from a crystallographic series or from a docking run, that frame already exists and you should leave it alone. If they came from SMILES, they have no frame at all and you must make one — or use one of the frame-free routes of *Working without an alignment*, which is often the better answer.

The tools, in increasing order of sophistication:

- *Tools → Align data set to reference…* — rigid superposition on the maximum common substructure with a reference molecule, optionally refined by maximising the CMF kernel similarity, which is the alignment criterion proposed in [2].
- *Tools → Align by prediction confidence…* — minimises the Gaussian-process posterior variance $\sigma^2(x) = K(x,x) - \mathbf{k}(x)^{\mathrm{T}}(\mathbf{K} + \lambda\mathbf{I})^{-1}\mathbf{k}(x)$ of a molecule under an existing model, which moves it into the model's applicability domain. It uses no activity value, so it is valid for prospective prediction. The program refuses to move training molecules, because the optimum there is degenerate: superimposing everything maximises mutual similarity and destroys the signal. The

search tries a few orientations near the pose it is given, which is what a molecule already placed in the site needs; for a molecule whose orientation is unknown, see *What a bounded restart cannot do*.

- *Tools → Align decoys to the training set…* — the same idea for a background set, so that no decoy is rejected merely for having been posed badly.
- *Tools → Unalign (randomise poses)…* — the control experiment. It gives every molecule a random rigid pose, which leaves internal geometry and self-kernels untouched and scrambles only the mutual arrangement. Randomise, realign, and see how much comes back; the program remembers the poses from before and reports the recovery.



The confidence criterion on one test molecule of the steroid set under the model of chapter 8: the posterior variance of its prediction as the molecule is turned away from its aligned pose about the normal of the mean plane and about the long axis of the series, and the prediction itself. The variance is lowest at the aligned pose and rises within a few tens of degrees, so the aligned pose can be found from the variance without using the activity, and the prediction is closest to the observed value there; a half turn about the long axis costs less than a quarter turn in the plane, because the skeleton is nearly flat.

5.4 Judging an alignment, and aligning flexibly

Everything in this manual says the alignment decides more than the kernel, the learner or the fields. What it could not say until recently is whether one alignment method is better than another, because the obvious measure cannot tell you: realigning changes the geometry q^2 is computed on, so a method is rewarded for making the data look like the model, and q^2 and the external R^2pred have been seen to move in opposite directions under exactly that operation.

The criterion has to come from outside the model, and the FEP benchmark sets supply one: every ligand of `examples/openff` is co-crystallised in the same frame, so the pose a method *should* have found is known. `benchmarks/run_poses.py` fits a model on training molecules in their crystal poses, destroys the poses of the held-out ones, and asks each method to recover them without ever seeing a held-out activity. Two numbers come back for every method — how near the crystal pose it got, measured over heavy atoms, symmetry-aware and **without superposition**, and what the model then predicts — and both are reported between two fixed references: the crystal poses as the ceiling and the scrambled poses as the floor.

That last detail is what makes the protocol work. Superposing before measuring would report 0.00 Å for a molecule placed in the wrong site: on a ligand displaced by 1 Å, `CalcRMS` returns 1.0 and `GetBestRMS` returns 0.0.

What the methods are worth

Measured this way over sixteen FEP sets, with the held-out **conformations regenerated** rather than read from the crystal — which is the case a new molecule actually presents:

method	median RMSD	within 2 Å	R^2pred	conformer limit
crystal poses (the ceiling)	0.00 Å	100 %	+0.334	0.00 Å
scrambled (the floor)	6.12 Å	1 %	−0.696	1.73 Å
substructure (MCS)	1.94 Å	54 %	−0.391	1.73 Å
MCS + rigid confidence	2.27 Å	45 %	−0.396	1.73 Å
MCS + flexible confidence	1.41 Å	74 %	+0.065	1.20 Å
MCS + flexible, 5 conformer starts	**0.86 Å**	**88 %**	**+0.264**	**0.71 Å**

Three things follow. **Freeing the torsions makes the confidence criterion work**: with a conformation to fix, the rigid search of [6] is worse than not refining at all, which is the failure the protocol was built to expose; the same objective with the rotatable bonds free is the best method available. The **conformer generator is what makes it work** — five generated starting conformations improve the median pose by 0.55 Å, better on 15 of 16 sets, and the combination beats plain substructure alignment on 16 of 16. And the **displacement guard has little left to prevent**: with these starts no set is worse than 1.52 Å, a 1 Å guard costs 0.6 Å, and 2 Å is indistinguishable from none.

What none of it does is replace knowing the conformation: +0.264 against the crystal poses' +0.334. The conformational residue is 0.71 Å, and that is where the remaining error comes from.

What a bounded restart cannot do

The protocol destroys a pose by placing the molecule at random within 2 Å of the site **and giving it a uniformly random orientation**. Until 30 October 2026 three of the searches it judges restarted by turning the pose they had arrived at by at most about 63°, and restarted twice. A bounded turn cannot undo an orientation that was drawn uniformly, however many restarts are spent on it, so the measurement was about the starting points of those searches and not about the criteria they optimise. The searches now accept an unbounded restart — `--any-orientation` on `cmf align`, `--align-any-orientation` on `cmf predict`, and **every restart from a completely different orientation** in *Tools → Align by prediction confidence…*, which also removes the displacement limit and raises the restarts to twenty, because those are what the measurement needed. The protocol itself uses twenty restarts from any orientation.

Both halves are needed, and the control says why. The number of restarts and the turn they are allowed were varied separately, on two sets, with the confidence criterion alone and the same held-out molecules:

set	restarts, and the turn each may make	median RMSD	within 1 Å	R^2pred	the worst molecule
thrombin	2, at most 63°	5.00 Å	43 %	−2.804	7.62 Å
thrombin	2, any orientation	0.11 Å	71 %	−0.014	7.62 Å

set	restarts, and the turn each may make	median RMSD	within 1 Å	R²pred	the worst molecule
thrombin	20, at most 63°	0.10 Å	86 %	−0.011	7.62 Å
thrombin	20, any orientation	**0.10 Å**	**100 %**	**+0.440**	**0.33 Å**
mcl1	2, at most 63°	0.29 Å	62 %	+0.207	7.65 Å
mcl1	2, any orientation	0.22 Å	88 %	+0.249	7.63 Å
mcl1	20, at most 63°	0.17 Å	100 %	+0.387	0.34 Å
mcl1	20, any orientation	0.17 Å	100 %	+0.387	0.34 Å

Either change on its own brings the median down. On thrombin only the two together remove the last misplaced molecule, and that molecule is what the prediction depends on: with one ligand of seven left 7.6 Å away, R²pred stays at about zero whichever half was changed, and when it comes back the same model predicts at +0.440. On mcl1, which never failed outright, twenty restarts are enough whichever turn they may make. This is also why the protocol reports the error of the prediction beside the RMSD: a median of 0.10 Å can hide a molecule in the wrong site.

```
# the control above: the two settings varied separately, two sets each
python benchmarks/run_poses.py --sets mcl1 thrombin --restarts 2 --max-rotation-deg 180
python benchmarks/run_poses.py --sets mcl1 thrombin --restarts 20 --max-rotation-deg 30
```

What it changes, over the same sixteen FEP sets but with the held-out **conformations taken from the crystal**, so that the pose is the only thing left to find:

method	median RMSD	within 1 Å	recovery	R²pred
crystal poses (the ceiling)	0.00 Å	100 %	—	+0.334
scrambled (the floor)	5.29 Å	0 %	0 %	−0.683
substructure (MCS)	0.25 Å	94 %	95 %	+0.329
confidence, restarts turned by at most 63°	1.35 Å	64 %	74 %	−0.219
confidence, restarts from any orientation	**0.11 Å**	**100 %**	**98 %**	**+0.312**

The confidence criterion reaches the crystal-pose ceiling with no substructure to align on and no receptor. No set is worse than it was, and the four sets where it had failed outright are the four that gain most: syk 6.26 → 0.13 Å, thrombin 5.00 → 0.10 Å, jnk1 3.49 → 0.10 Å, pde2 3.14 → 0.12 Å.

Two cautions. This table is about the pose and not the conformation, because it reads the held-out conformation from the crystal; the table before it, which regenerates the conformation, is the harder and more realistic case, and there the wider search improves the flexible route from 2.47 Å to 1.51 Å and leaves the substructure-started rows where they were. And an unbounded restart is for a molecule whose orientation is genuinely unknown: it needs the displacement guard switched

off, since a reorientation is a large displacement, and it needs about ten times the restarts.

Inside the pocket

Both criteria above know only half of the problem. The confidence criterion knows the series and not the receptor, so nothing stops it putting a molecule where no molecule could go; the docking score of `cmfqsar/docking.py` knows the receptor and not the series, so nothing makes the pose it finds comparable with the poses the model was fitted on. The item that joins them reads the complementarity as a **prior** over poses and the confidence as the **likelihood**, and takes the pose that maximises the product:

$$\min_{p} \left[\sigma^2(p) - w\,(\text{complementarity}(p) - \text{clash}(p))\right] \tag{5.1}$$

over the six rigid-body parameters p, with the same displacement guard. At $w = 0$ this is the confidence alignment exactly; as w grows the receptor takes over.

```
cmf predict model.cmf new.sdf --align-variance --align-pocket complex.pdb --pocket-weight 1
```

Judged by the protocol above on 4 FEP sets with a receptor structure (mcl1, p38, thrombin, tnks2), the held-out poses scrambled and recovered, with the pose search restarting twenty times from any orientation. Every number is a mean over the four sets:

method	median RMSD	within 2 Å	R²pred
crystal poses (the ceiling)	0.00 Å	100%	+0.335
scrambled (the floor)	5.00 Å	0%	-0.965
substructure (MCS)	0.24 Å	100%	+0.229
confidence alone	0.11 Å	100%	+0.286
MCS + confidence	0.11 Å	100%	+0.286
pocket alone	1.12 Å	74%	-0.241
pocket + confidence	0.15 Å	90%	-0.110
MCS + pocket + confidence	0.15 Å	100%	+0.265

Per set, the comparison that carries the answer — the receptor added to the confidence criterion, with and without a substructure frame to start from:

set	confidence alone	pocket + confidence	MCS + confidence	MCS + pocket + confidence
mcl1	0.17 Å	0.22 Å	0.17 Å	0.22 Å
p38	0.09 Å	0.12 Å	0.09 Å	0.12 Å
thrombin	0.10 Å	0.15 Å	0.10 Å	0.14 Å
tnks2	0.07 Å	0.11 Å	0.07 Å	0.09 Å

The two left columns repeat the two right ones, and that is the first thing to read here: with a search that can find the pose, the criterion no longer depends on the frame it starts from, so the substructure start adds nothing to it.

The receptor term is not worth its cost. It is behind the confidence criterion on all four sets, by 0.03 to 0.05 Å, its predictions are worse (-0.110 against +0.286), and it costs about ten times as much, because the pocket term has no analytic gradient and the search falls back on Powell. What it no longer does is ruin the pose.

Before 30 October 2026 these differences were twenty times larger, and the reason was the search and not the term. With two restarts of a bounded turn (*What a bounded restart cannot do*) the receptor alone sat at 4.22 Å and the two together at 3.22 Å, against 0.11 Å for the confidence criterion: the term costs 0.04 Å and that search cost 3.07 Å. The earlier edition of this manual gave 3.93 Å and 2.50 Å for those two rows, which are the medians over the four sets of the same measurement; every number in the table above is a mean, as `benchmarks/results/poses/table.md` prints them.

The receptor criterion needs the free orientation more than the confidence criterion does. On mcl1 the number of restarts and the turn they may make were varied separately, with the receptor in the objective, on the same 8 held-out ligands. Every number is a distance from the crystal pose:

restarts, and the turn each may make	receptor alone, median	receptor alone, worst molecule	receptor and confidence, median	receptor and confidence, worst molecule
2, at most 30°	3.00 Å	7.97 Å	0.31 Å	7.89 Å
20, at most 30°	1.00 Å	8.02 Å	0.31 Å	7.89 Å
20, any orientation	**0.93 Å**	7.82 Å	**0.22 Å**	**0.37 Å**

The receptor alone gains most from the number of restarts, and one ligand stays about 8 Å from its crystal pose whatever the search does: that placement is where the receptor term's own optimum is, and none of the three searches moved it. The two criteria together answer the other way round. Twenty bounded restarts change nothing, and the one restart setting that recovers the last ligand is the one that draws a completely different orientation. The confidence criterion alone on this same set is recovered by twenty bounded restarts (the control in *What a bounded restart cannot do*), so the receptor term is what holds the pose in an orientation a bounded turn cannot leave. This is a second reason to leave `--any-orientation` on wherever `--align-pocket` is used.

How much of the receptor the pose can stand. The weight of the receptor term was varied on two sets, first at the narrow search of two bounded restarts and then, on 30 October 2026, at the corrected search of twenty restarts from any orientation. Weight zero is the control: the joint aligner is then the confidence objective optimised by a different method, and at the corrected search it gives 0.09 Å on p38 and 0.07 Å on tnks2, which are the two numbers the confidence criterion alone gives in the table at the top of this section. Median distance from the crystal pose, the receptor and the confidence together:

weight of the receptor term	p38, narrow search	p38, corrected search	tnks2, narrow search	tnks2, corrected search
0 (the confidence criterion alone)	0.29 Å	0.09 Å	0.07 Å	0.07 Å
0.25	3.52 Å	0.10 Å	0.11 Å	0.09 Å
1	3.73 Å	0.12 Å	1.26 Å	0.11 Å
4	3.63 Å	0.24 Å	1.32 Å	0.27 Å

The weight-1 row of the two corrected columns is the *pocket + confidence* row of the table at the top of this section, measured in the same run as the rest of that table.

At the narrow search any amount of the receptor destroyed the recovery on p38 and degraded it on tnks2. At the corrected search the largest weight costs 0.2 Å of the median, so most of what the first sweep measured was the search and not the weight. What the weight still costs is one molecule. On tnks2 the eight distances end at 0.33 Å at weight zero; at weight 0.25 one ligand sits at 2.59 Å and at weight 4 the same ligand at 2.56 Å, and that one molecule takes R^2pred from +0.212 to −0.422 and −0.517, against +0.291 for the crystal poses. On p38 no weight loses a molecule (every distance stays under 0.65 Å) and R^2pred stays at the crystal value (+0.105, +0.109 and +0.088 against +0.105). So the receptor term no longer spoils the median once the search can find the pose, and it can still place one ligand of eight where the protein allows it and the crystal does not.

The reason the narrow search failed so widely is visible in what complementarity is: a weak prior over six dimensions. A ligand that fits a site fits it in many placements, so the term adds a broad, multi-modal surface to an objective that was already multi-modal, and its optimum is not where the crystal pose is — not with a scoring function that has no desolvation, no entropy and no torsional search. A search that starts from the wrong orientation follows that surface to the wrong placement; a search with twenty free restarts does not have to. What the first sweep measured is therefore how far the receptor term can lead a search that starts from the wrong orientation, and the cost of the term itself, measured where the search can find the pose, is the 0.04 Å above.

The proper use of what is implemented is therefore narrow. `--align-pocket` is for a pose that must be defensible **to the receptor** as well as to the model: it will not improve on the confidence criterion, and it will tell you when the confident pose is one the protein would not allow.

Nothing here is a docking program: there is no desolvation, no entropy and no torsional search, and the internal geometry is never touched. What the receptor adds is a term that knows where the protein atoms are, and the measurement says where that helps and where it does not.

Using it

In the application, *Tools → Align by prediction confidence…* gained a **turn the rotatable bonds as well** tick box, with the number of starting conformations and the strain allowance beside it. From the command line:

```
cmf predict model.cmf new.sdf --align-variance --align-flexible --align-conformers 5
```

The force field acts as a prior on the conformation — it may rise ten kcal/mol above where it started before being penalised, one-sided so that the aligner does not quietly become a conformer optimiser — and the displacement limit now applies to the **rigid motion only**, since a turning bond is not a translation and counting it as one would suppress exactly what this option adds.

5.5 Charges and per-atom properties

The electrostatic field needs charges: Gasteiger (the default), MMFF94, or **read from the file**, which is what you want with the Sutherland sets and their MNDO ESP-fit charges, or with anything you computed yourself.

What the Gasteiger scheme cannot do, and what the program does about it

The scheme covers **fourteen elements** — H, B, C, N, O, F, Mg, Al, Si, P, S, Cl, Br and I. That list was measured rather than read from a paper: a two-atom molecule C–X was built for every element up to radon and the charges asked for. Magnesium and aluminium are in and lithium and sodium are out, which is not a pattern anyone would guess.

Outside that list the scheme does not refuse, it answers "not a number", and it does so in two different ways:

- an **element** it has no parameters for voids *every atom of the molecule*, so the molecule has no electrostatic field at all;
- a **valence state** it has no parameters for voids *that atom alone*. The phosphorus of hexafluorophosphate is the case that matters, because phosphorus is in the covered list.

Both used to become a charge of zero, which is not an error message and not a charge. A coordination complex therefore had no electrostatic field (item EC4 found that), a selenoether in the corrosion series had none (item EC3 found the same fault outside the d block), and — worst, because it is silent rather than empty — a neutral $LiPF_6$ solvation shell presented a **net charge of +1** to the field, because the anion's phosphorus had been zeroed while the lithium kept its +1.

What the program does now (item D11): it asks the plain scheme first, so every molecule the scheme can do is answered exactly as before. Only a molecule that comes back with a "not a number" is repaired, by computing the charges with the uncovered atoms removed and giving each of those atoms its formal charge, and the source is then reported as `gasteiger+formal` rather than `gasteiger`, so a model fitted on one cannot be confused with a model fitted on the other.

The repair has one approximation worth knowing. Removing a *bonded* atom leaves its neighbours an open valence, so the remaining fragment is charged as a radical and the total comes out near the molecule's formal charge rather than exactly on it — within 0.25 of an electron on the cases measured. Where that matters, compute the charges properly and read them from the file.

Beyond that, any per-atom number can become a field of its own. `prop:<name>` reads the atomic weights from a property of the molecule — a quantum charge, a Fukui index, a local softness, an atomic polarisability, even a per-atom embedding — and that field then behaves exactly like a built-in one, with its own attenuation, its own mixing coefficient, its own co-field, atom contributions and Shapley values.

Two conventions are read, in this order:

1. a property set on **every atom**, which RDKit writes to SDF as `atom.dprop.<name>` and reads back automatically;
2. a **molecule property holding one number per atom**, separated by spaces, commas or newlines, which is how most quantum-chemistry packages dump such a table.

The hydrogens are the usual source of error. A file without 3D coordinates is embedded on loading and hydrogens are added, so a list written for the heavy atoms alone no longer matches the atom count. The program says so rather than silently mis-assigning: `--prop-missing error` (the default) stops, `--prop-missing zero` gives the unlisted atoms zero weight. The next section shows the whole route from an ORCA run to fields and descriptors, from inside the program or from the command line.

A number that belongs to an atom, not to the molecule

A field is built from a property every atom has. A **target** of that shape is a different object, and the program now carries one (plan item LP1): a carbon shift, an acidity, a local reactivity or a site's

energy is a number per atom, and a table of them differs from a per-atom field in the way that decides everything downstream -- **most atoms have no value at all**, and the atoms that do are not independent samples of each other, because they share a molecule.

`cmfqsar/atomtargets.py` holds that data model, and nothing more: there is no model over atoms yet, and the item that adds one needs this first.

- **Reading keeps the blanks.** `read_atom_target` gives one number per atom and *not a number* where the atom has none. The two conventions of the previous section are read, with a value on the atoms themselves taking precedence, and in a molecule property a blank field between two commas is an atom that was not measured -- `1.2, , 3.4` is three atoms of which the middle one has nothing. `n/a`, `-`, `none` and an empty field all mean the same. A list as long as the heavy atoms is spread over them, leaving the hydrogens unmeasured; any other length is an error naming the molecule and both counts.
- **The index remembers the molecule.** `AtomTargets` is the list of measured (molecule, atom) pairs with their values; a fit iterates over it, and the molecule each sample came from is kept beside it because every honest split needs it.
- **The folds hold out whole molecules.** `AtomTargets.folds` splits the molecules, never the samples. Two shifts of one molecule, one in the training set and one in the test set, would let a model see the molecule it is being tested on, and a per-atom q^2 obtained that way would not mean anything. This is the grouped-fold rule of the chemotype splits, with the molecule as the group.
- **A prediction goes back on its atom.** `write_atom_predictions` writes an SDF whose atoms carry the measured value and the predicted one, in RDKit's own `atom.dprop.<name>` spelling, so a predicted shift sits on the carbon it belongs to and every program that reads an SDF can see it.

```
cmf atoms shifts.sdf --list                       # what here could be a target
cmf atoms shifts.sdf --target shift --out with_atoms.sdf
```

`cmf atoms` is the inspection the data model deserves before a model exists: it says how many atoms were measured in how many molecules, which molecules carry the most, and what the folds would look like, and it writes the file back in the canonical form. Listing is a looser rule than the one for fields on purpose -- a property with gaps cannot be a field but is exactly what a target looks like.

5.6 Quantum-chemical properties with ORCA

Tools → Quantum-chemical properties with ORCA... runs ORCA over the loaded molecules and brings two kinds of number back, to the two places the model keeps them.

Per atom, and so available as `prop:` fields: Mulliken, Löwdin, Hirshfeld and CHELPG charges, the HOMO and LUMO population of every atom, and — when the N+1 and N−1 states are computed as well — the condensed Fukui indices f^+, f^-, f^0, the dual descriptor $f^+ - f^-$, the local softnesses and the local electrophilicity. **Per molecule**, and so available as descriptors: the total energy, the HOMO and LUMO energies and their gap, the dipole moment, and from the ionised states the ionisation potential, the electron affinity, the chemical hardness, the softness, the chemical potential and the electrophilicity index.

That split is the purpose of the feature. A Fukui index is a statement about one atom, so it belongs in a field, where a co-field can then say *where* a nucleophilic centre helps; the gap is a statement about the molecule, so it belongs in the descriptor block, where it cannot pretend to have a position.

The dialog asks for

- **the ORCA program** — found by itself at the standard location, in `$ORCA_EXE` or on the PATH, and otherwise given here;
- **the level of theory** — the ORCA keyword line, which may be edited: `r2SCAN-3c` (fast, with its own basis) is the one to start with, B3LYP-D3/def2-SVP the usual hybrid, and anything else ORCA accepts as a single point will do;
- **which molecules** — all of them; only those that do not carry the properties yet, which is the default, so a set extended with new compounds costs only the new ones; the selected ones; or the training set;
- **the Fukui indices and the global reactivity numbers**, which need the N+1 and N−1 states and so cost three calculations per molecule instead of one;
- **optimising the geometry first**, off by default — see the warning below;
- **how many molecules at a time**, and, where the ORCA build has MPI, how many cores per job. The Windows build is the serial one: it has no MPI helpers, `%pal` fails outright, and the way to use a many-core machine is to run several molecules at once, which is what the first box is for. The dialog disables *cores per job* when it sees a serial build;
- **memory per core**, a **time limit** per job, after which that molecule is given up on rather than holding up the set, and the **working directory**.

Before you press *Run* the dialog says what the run will cost: the number of molecules times the number of ORCA jobs each, and the wall time that comes to at the chosen parallelism. After the first run that estimate is no longer a guess — the time every job took is recorded in the working directory, and the dialog quotes the measured mean. For scale: a molecule the size of methanol at `r2SCAN-3c` takes a few seconds per job, a drug-sized one at B3LYP-D3/def2-SVP a couple of minutes, so a hundred of the latter with their Fukui indices is a few hours with four running at a time.

The dialog also states the charge and multiplicity it read off the structures — the formal charge of the molecule, and the lowest spin state its electron count allows — so an unintended anion or an odd electron is visible before the hours are spent, not afterwards in a column of nonsense.

Three things protect a long run. Every job is **cached** under a key over the molecule, its charge, the level of theory and the geometry, so *Cancel* — which kills the ORCA processes themselves, not merely the bookkeeping — loses only the jobs that were running, and starting again continues from where it stopped. A molecule that **fails** is named in the log and skipped, never fatal to the set. And the working directory keeps every input and output, so any number can be traced back to the calculation that produced it.

When the run finishes, the new quantities appear as tick boxes — the per-atom ones in *Per-atom property fields*, the per-molecule ones in *Descriptors (per molecule)* — and the program offers to write the annotated set as an SDF. Write it: the file carries the properties with it, as atom property lists and molecule properties, so opening it later brings them all back without running ORCA again. *Tools → Save the data set with its properties (SDF)…* does the same at any time.

> **About the geometry.** *Optimise the geometry first is off by default and should normally stay off. ORCA optimises each molecule on its own, which moves every atom: a series that was superimposed is not superimposed afterwards, and a model trained on the old poses no longer refers to the molecules in memory. With the box off, the properties are computed exactly where the molecules stand, which is what a field model needs. If you do optimise, align the set again before building anything.*

The same computation runs from the command line as `examples/orca_atom_properties.py` (`--help` lists its options), which reads an SDF and writes the annotated one. The script and the dialog share the module `cmfqsar/orca.py`, so the numbers are the same either way.

What the quantum stage gained, measured. On the four corrosion series with finished ORCA runs (`benchmarks/results/corrosion_qm/table.md`) it gained nothing. Mean q^2 over the four series: the five ordinary fields alone **+0.382**; the ORCA charges in place of the Gasteiger ones +0.381; the Fukui fields added +0.359; the six per-molecule descriptors added +0.371; both added +0.348. The descriptors on their own reach +0.218, so they carry rather more than half of what the fields carry and add nothing to them. On the 46 Am/Eu extractants (`benchmarks/results/am_eu/table.md`) the answer is the same: the classic fields give q^2 +0.803 and the ORCA charges +0.803, the Fukui fields +0.788, the descriptors +0.786, and everything together +0.762.

Where the stage does pay is a property that *is* electronic. On the 95 substituents of the Hammett series (`benchmarks/results/hammett/table.md`) Hirshfeld charges in place of the Gasteiger ones take q^2 for sigma-meta from 0.70 to 0.76, and to 0.79 once the charges are symmetrised over the equivalent positions. A substituent constant is a statement about resonance, and resonance is what a Gasteiger charge does not know. Spend the hours where the activity is electronic; on an adsorption potency the five ordinary fields already carry what there is.

5.7 A simulation as a conformational ensemble

The ensemble mode takes a molecule with several conformations and a weight for each, and builds the mean field over them. Where those conformations come from is of no concern to the kernel. So far they have come from the conformer generator, which samples a molecule on its own, in no surroundings. A molecular-dynamics trajectory of a ligand inside its pocket samples it where it actually sits. `cmfqsar/trajectory.py` (plan item D7) turns such a trajectory into exactly the object the ensemble mode already takes.

```
python examples/trajectory_ensemble.py sys.pdb sys.xtc --ligand LIG -o ligand.sdf
python examples/trajectory_ensemble.py --directory runs/ --template series.sdf -o series.sdf
cmf build series.sdf --activity pKi --ensemble-by name --conformations ensemble \
    --conformer-weighting given
```

The chain is five steps, and each is a decision rather than a detail.

1. **Read.** Through `mdtraj` or `MDAnalysis`, whichever is installed; XTC, DCD, NetCDF, TRR and a multi-model PDB all work, because the readers handle them. Either reader gives the same geometry — on 36 trajectories read by both, the largest difference in any coordinate is 4e-7 Å — so which one is installed does not change a result. MDAnalysis also carries the force-field charges of a topology that has them (PSF, PRMTOP, a TPR), and those are written as the per-atom property `q_mm`, so `prop:q_mm` is a field built from the charges the simulation itself was run with.
2. **Strip.** Water, the common ion residues and anything else named in `--extra-solvent`.
3. **Superimpose.** When the topology holds a receptor, every frame is superimposed on the **backbone of the binding site** — the residues near the ligand in the first frame — so what is left in the ligand's coordinates is its motion *relative to the pocket*. That is the quantity a field model of a series needs, because it is what differs between two ligands in the same site. Fitting on the ligand's own atoms instead would remove exactly that; it is what happens when there is no receptor, and the result says which was used.

4 **Subsample.** The kernel of an ensemble of k conformations is a double sum over its members and so costs k^2 atom-pair integrals. A hundred thousand frames is therefore not an option, and the thinning is part of the reader rather than left to the user. Frames are clustered by heavy-atom distance with a leader algorithm: the first frame starts a cluster, every later frame joins the first cluster whose representative is within the radius, and starts a new one otherwise. If that leaves more clusters than `--max-frames`, the radius is raised and the pass repeated, so the number asked for is always met.

5 **Weight.** The weight of a conformation is its **cluster population** divided by the number of frames, and nothing else. It is never a Boltzmann factor of the frame energy. An equilibrated trajectory has already sampled the Boltzmann distribution — that is what running it was for — so weighting the frames by $e^{-E/kT}$ a second time would square the distribution and collapse the ensemble onto its deepest basin.

What the thinning costs (`benchmarks/run_trajectory.py`, 36 molecules of 60 frames each). The error is of the joint kernel, which is what a model is fitted to, against the kernel built from every frame:

clustering radius	conformations kept	error of the kernel	kernel time
0.25 Å	34.5	1.0e-02	87 s
0.50 Å	31.2	1.7e-02	78 s
1.00 Å	21.1	4.7e-02	44 s
2.00 Å	6.4	9.7e-02	4 s
3.00 Å	1.9	1.2e-01	0.2 s

All sixty frames take 176 s. The curve does not flatten: keeping more frames keeps improving the kernel, so the number to keep is a decision about cost and not a point where the curve stops moving. At the default radius of 1 Å the kernel is within 5 % of the one from every frame for a quarter of the time. The cost follows the square of the number kept, as the algebra says: going from 1 to 27 conformations is 27 times as many and 658 times the kernel time, against 734 for an exact square.

> **What this does not yet say.** *None of this compares a trajectory with a crystal pose. The frames above are conformations of real ligands written to a real trajectory file, which exercises the reader but is not a simulation. Whether a trajectory mean field models a series better than its single crystal pose is plan item* **D8***, and it needs short simulations on a set whose ligands have crystal poses. The reader exists so that measurement can be made.*

`mdtraj` is the `md` extra; MDAnalysis does as well. Neither is required for anything else.

5.8 The water as a field

Every field described so far sits on the atoms of the molecule. The 2015 indicator-field formalism does not require that -- it accepts any continuous density -- and a water occupancy map from a simulation is one. `cmfqsar/solvent.py` (plan item D9) turns the places a ligand's water occupies into a field of the ordinary kind, so that a sixth kernel block can carry it and a co-field over it can say where displacing a water helps.

```
        python examples/make_md_trajectories.py --set mcl1        # writes mcl1_water.json too
        python benchmarks/run_solvent.py
```

What a hydration site is here. For every point of a grid around the ligand, the share of frames in which some water oxygen lies within 1 Å of it. That is an occupancy in [0, 1] with a plain reading — "a water sits here nine tenths of the time" — and bulk water scores about 14 % of it, which is why a site has to clear 50 %. The peaks are taken greedily, strongest first, with no two sites closer than 2.4 Å, so a site is a place a water occupies rather than a cluster of molecules. The coefficients are then fitted to the whole map by a non-negative least squares rather than read off the peak heights, so the field reproduces the map; non-negative because an occupancy cannot be negative and a negative coefficient would read as a place water avoids, which the map does not say.

On the mcl1 set that gives **7.8 sites per ligand** (from 4 to 12), the strongest averaging 73 % occupancy.

The water field is a block of its own, not a sixth column of the molecule's table: a table holds one set of atoms and a hydration site is not an atom of the molecule. So the kernel is mixed, $K(h) = (1-h)\ K_{\text{fields}} + h\ K_{\text{water}}$, with h searched on the same folds as everything else — the same arrangement the variance field of plan item D10 uses.

Measured (`benchmarks/run_solvent.py`), every model at its own best attenuation so the reference is not held back:

model	q²	change against the five fields
the five fields alone	**+0.575**	the reference
the five fields and the water	+0.560	−0.015 [−0.178, +0.186]
the water alone	+0.263	–
random sites alone (control)	−0.028	–
one shared set of sites alone (control)	−0.138	–

The water field does not improve the model, and the interval is wide enough that a small gain could not have been seen on 25 ligands. But the two controls make the rest of the table worth reading. The water field **on its own** reaches +0.263, while the same number of sites moved to random places in the same box reaches −0.028 and one ligand's sites given to every ligand −0.138. Both controls are below zero, as a block that cannot tell the ligands apart must be. So at this definition of a site, *where the water sits* does carry information about activity — it is simply information the ligand's own fields already carry, since adding it gains nothing.

How much of that depends on the definition of a site. Two numbers decide what a hydration site is and neither follows from anything: the share of frames a point has to be occupied, and how close two sites may be. `--sweep` runs the whole comparison again at every pair of them, and the answer is different for the two conclusions.

The first conclusion is firm. Of the sixteen definitions that leave every ligand a site, **none** beats the five fields beyond the interval and none falls behind; the change runs from −0.130 to +0.087 and every interval contains zero.

The second is not. The water field on its own runs from **−0.124 to +0.607** over those sixteen, and it is not positive at three of them. Averaged over the spacings it goes 30 % +0.080, 40 % +0.052, 50 % +0.197, 60 % +0.568, so the sensitivity has a clear direction: **the signal is in the few places a water really sits**, and adding weakly occupied points dilutes it. At the strictest threshold that still leaves every ligand a site — 60 %, about 3.6 sites a ligand — the water alone reaches +0.554 to +0.607 against a control of −0.096 to −0.055, and it does so at all four spacings rather

than at one.

A second set, with the definition fixed in advance. The paragraphs above are all from one set, and the strongest number in them was the largest of sixteen tried. So the strict-threshold finding was put to a held-out test: the definition was fixed at **60 % occupancy and 2.4 Å** before anything was run, the protocol was left exactly as it was, and the whole chain was run on **p38** — 29 ligands, receptor 3FLY, a single chain of 3,113 atoms, 67,423 atoms solvated, 13 minutes a ligand.

It failed. On p38 at that definition the water alone reaches **−0.043**, against a random-site control of +0.002: nothing. The +0.554 that mcl1 gave at the same definition does not transfer, and anyone quoting it as a property of water fields would have been wrong.

The sweep on p38 shows why, and finds something mcl1 never did. At 60 % occupancy p38 keeps **11.4 sites a ligand** where mcl1 kept 3.6 — p38's site holds far more ordered water, so the same share of frames is a far looser filter there. Going to **70 %**, which leaves 6.3 to 6.7 sites, the water alone reaches **+0.312 to +0.341** against controls near −0.02, and the mixture reaches +0.536 to +0.560, **+0.276 [+0.037, +0.560] to +0.299 [+0.063, +0.593]** against the five fields. Those four definitions are the only place in this item where a water block has cleared the interval at all.

So the threshold is a **proxy for how few sites survive, and the proxy is target-specific**. Put in terms of the count rather than the threshold, both sets say the same thing: the water field carries something when the definition leaves roughly four to seven strongly occupied sites a ligand (mcl1 +0.554 at 3.6 sites, p38 +0.312 to +0.341 at 6.3 to 6.7), and nothing when it leaves ten or more.

> **That statement was a hypothesis, and it was put to a test.** *It was formulated after seeing both sets, so it carried two observations and no estimate. A count is testable in a way a threshold is not, because a count means the same on two targets. The test is the last part of this section, and it failed.*
>
> **Two cautions, both from the sweep itself.** *At the loosest definition tried (30 % occupancy, 3.6 Å spacing, about 25 sites a ligand) the* **random-site control reaches +0.438 against −0.022 for the measured sites***. A block of two dozen plausible numbers carries what a block of two dozen random numbers carries, so a claim about where the water sits is only worth making at a definition where the control is beaten. And the +0.607 above is the largest of sixteen values tried on 25 molecules after seeing them; it should not be quoted as this item's result. The defaults of the module, 50 % and 2.4 Å, were fixed before any of this was measured and are left as they are for that reason.*

A third set, with the definition fixed as a count, and the hypothesis failed. The definition was written down before anything was run: the **five most occupied sites a ligand**, an occupancy floor of 30 % so that the count and not the floor decides, and the module's own spacing of 2.4 Å. The set was **hif2a**, chosen as the largest one available with the smallest receptor and not for anything it had shown: 37 ligands, chain A of entry 5TBM, 1,749 atoms after preparation, 22,717 atoms solvated, 5.3 minutes a ligand, about four hours in all. One ligand was lost, because `antechamber` could not charge lig_43, so 36 remain. The two earlier sets were then measured again at the same count, from the trajectories they already had.

set	ligands	sites a ligand	the five fields	with the water	change against the five fields	the water alone	random sites alone
hif2a (held out)	36	5.0	+0.342	+0.262	**−0.079 [−0.287, −0.007]**	+0.058	+0.177
mcl1	25	5.0	+0.575	+0.455	−0.120 [−0.316, +0.074]	+0.172	−0.134
p38	29	5.0	+0.261	+0.267	+0.006 [−0.100, +0.133]	−0.043	+0.089

The test failed on the held-out set. On hif2a the water alone reaches +0.058, while the same five sites moved to random places in the same box reach +0.177: the measured positions do worse than random ones. The mixture is also the first case in this item where adding the water makes a model **worse beyond the interval**, −0.079 [−0.287, −0.007].

It failed on the two sets the hypothesis was read from as well, which is the clearer result. A count of five gives mcl1 +0.172 where its own 60 % floor gave +0.607 with 3.6 sites, and p38 −0.043 where its own 70 % floor gave +0.341 with 6.3 sites. A reformulation that loses the result on both of the sets it was formed on is a mistake in reading the sweep, and not a property that failed to carry over to a third target.

Why the count was the wrong variable. The random-site control keeps each ligand's number of sites and moves them, so the count was already held fixed in every comparison in this item. At mcl1's 60 % floor the measured sites reach +0.554 and that control −0.096, which says the difference lies in the positions of strongly occupied sites and not in how many of them there are. What the two working definitions have in common is the floor itself: **every site kept is occupied in most frames**. How many survive follows from the floor and differs between targets, 3.6 on one and 6.3 on the other. Fixing the count at five does the opposite of what a floor does. A ligand with one strongly held water is made to take four weak places as well, so a count behaves like a loose threshold, which is the end of the sweep where the field carries least.

> **What is left of this item.** *A water field does not improve a model at any definition that can be fixed in advance, and at one such definition on one set it makes the model measurably worse. A high occupancy floor is the only setting in which the field carries anything; the floor that works differs between targets; and nothing here explains why. Across the two sweeps 36 definitions were tried and one cleared the interval, which is also what trying 36 definitions produces on its own. The field and its co-field stay in the program as a display tool under rule 5.*

The rule behind those definitions, applied as a rule, and it failed too. One thing survived the count: both definitions where the water alone carried anything looked like the **strictest occupancy floor that still leaves every ligand at least one site**, 60 % on mcl1 and 70 % on p38. That is a rule and not a number, so it can be carried to a new set. It also reads nothing it should not: the floor is chosen from the occupancy maps alone, with no activity and no q^2 in sight, and the five models are then measured once at the chosen floor.

set	ligands	floor the rule picks	sites a ligand	the five fields	with the water	change against the five fields	the water alone	random sites alone
hif2a (held out)	36	60 %	4.6	+0.342	+0.396	+0.054 [−0.216, +0.330]	**−0.074**	**+0.277**
mcl1	25	60 %	3.6	+0.575	+0.617	+0.042 [−0.190, +0.299]	+0.554	−0.096
p38	29	80 %	3.0	+0.261	+0.301	+0.040 [−0.176, +0.220]	+0.169	+0.100

The rule was refuted by its own evidence, before the held-out set was consulted. On p38 it picks a floor of **80 %**, not the 70 % the sweep had found, and at 80 % the water alone reaches +0.169 against a random-site control of +0.100 — nothing. The reason is a mistake of mine in reading the sweep: its grid stopped at 70 %, so a floor that was the strictest **tried** was taken for the strictest **possible**. Carried out as stated, the rule keeps raising the floor while every ligand still has a site, and on p38 it walks straight past the definition the sweep had found. The rule only reproduces its evidence on mcl1, where 70 % does leave seven ligands with no site and 60 % really is the strictest floor available.

It failed on the held-out set as well. On hif2a the rule picks 60 %, leaving 4.6 sites a ligand, and there the water alone reaches **−0.074 while the same sites moved at random reach +0.277**. The control beats the measured positions by a wide margin, which is the same inversion the loosest definitions of the sweep produced. The mixture is +0.054 [−0.216, +0.330]: positive, and the interval contains zero, as it does on all three sets.

> **Where that leaves this item.** *Two readings of the sweep were written down and tested on a set they were not read from, and both failed. The water field does not improve a model at any definition that can be fixed in advance. The one place in the whole item where a water block cleared the interval — p38 at a 70 % floor, +0.276 [+0.037, +0.560] — cannot be reached by any rule stated beforehand, and it is one result from 36 definitions tried on two sets, which is about what 36 attempts produce on their own. The field and its co-field remain in the program as a display tool under rule 5, which is what rule 5 is for.*

```
python benchmarks/run_solvent.py --sets mcl1 p38 hif2a --count
python benchmarks/run_solvent.py --sets mcl1 p38 hif2a --rule
```

Where the model asks for water. The sites of all the ligands are in one frame, so they gather into places, and the weight the water block puts on a place is what it asks of the water there: positive for more water, negative for less, which is where displacing one is worth trying. The table in `benchmarks/results/solvent/table.md` lists the places shared by at least three ligands. Under rule 5 this is a **display tool**: the block does not improve the model, so a place it names is something to look at and not a prediction.

> **One reading to avoid.** *The mixed control in which every ligand is given the same sites lands exactly on the reference, and that is arithmetic rather than a result: identical sites make every kernel entry the same, and a constant block added to a kernel is absorbed by the fit's own bias. It is a check that the mixing does nothing of itself.*

5.9 Does a trajectory beat the crystal pose?

The section above reads a simulation as an ensemble. Whether that ensemble is worth having is a separate question, and plan item D8 is the measurement that answers it. The fourth-dimension study of this program built its ensembles by generating conformations of a molecule on its own and found they did not help; what it never had was a pose known to be right. A free-energy set gives one, because every ligand is co-crystallised.

`cmfqsar/md.py` runs the simulations and `examples/make_md_trajectories.py` is the whole recipe:

```
python examples/make_md_trajectories.py --set mcl1 --ns 2
python benchmarks/run_md.py
```

What was run. The 25 ligands of `examples/openff/mcl1.sdf`, each in the site of PDB entry 4HW3, for 2 ns at 300 K and 1 bar after 200 ps of equilibration, with GAFF-2.11 and AM1-BCC charges for the ligand, Amber ff14SB for the protein and TIP3P water with 0.15 M of ions. The system is 20,284 atoms and ran at 658 ns/day on one consumer GPU, which is 351 seconds per ligand including the charges and the equilibration. `examples/md/README.md` holds the full statement.

Two choices in that are worth naming. The **whole protein** is simulated, not the trimmed pocket of `examples/pockets/`, because every cut in a trimmed site leaves a broken peptide bond that preparation turns into a charged terminus. And **only the chain that lines the site** is kept: 4HW3 holds twelve copies of Mcl-1, and simulating all of them would cost ten times what the question needs.

The ligands stayed where they started, 0.93 Å from their crystal pose in the median and 1.39 Å at worst, so every one of the 25 is kept. That matters for reading the result: this measures what an ensemble around a correct pose is worth, not whether a simulation can find a pose.

The answer is no. Six models on the same molecules, the same folds and the same grids, each difference against the crystal pose with the bootstrap interval of rule 4:

model	q^2	change against the crystal pose
the crystal pose	**+0.575**	the reference
generated conformations	+0.556	−0.019 [−0.271, +0.279]
trajectory, clusters equal	+0.391	−0.184 [−0.526, +0.136]
trajectory, by population	+0.180	−0.395 [−0.830, −0.053]
trajectory, multi-instance	+0.463	−0.112 [−0.384, +0.169]
trajectory, each in its own frame	+0.277	–

No reading of the trajectory is ahead of the crystal pose, the best is indistinguishable from it, and the population weighting is behind it by more than the interval. The control says this is not about molecular dynamics in particular: generated conformations change nothing either, which reproduces the fourth-dimension result on a set where the pose is known. Around a pose that is already right, averaging over the ligand's motion does not add information, and weighting that average badly subtracts some.

Three readings of the same frames, and the differences between them say more than the headline. **Letting the model choose the frames gained nothing**: the multi-instance search chose $\beta = 0$, which is the plain ensemble, so there was no frame worth concentrating the weight on. **The**

weighting mattered more than the simulation did: equal weights give +0.391 and weights by population +0.180, a gap of 0.211, larger than the 0.184 between the better of them and the crystal pose. That contradicts the argument in `cmfqsar/trajectory.py` for weighting by population — that an equilibrated trajectory has already sampled the distribution — and the contradiction is reported rather than resolved, because the argument is about the physics and the measurement is about the model.

And the last row is a measurement about alignment, not about simulation. Reading the same frames with each trajectory superimposed on its own first frame, rather than on the one structure every run was built from, costs 0.114 of q^2 (+0.391 to +0.277). Two runs equilibrate to protein conformations a few tenths of an ångström apart, so fitting each ligand to its own leaves the series in as many frames as it has molecules. A field kernel notices a misalignment far smaller than the ones part C of the plan is about. The subsection *What the common frame is worth*, below, measures the same control on the two further sets and gives it an interval of its own; it does not support this reading as a statement about the method.

Two more sets, and the answer is not the same on every set

The section above was one set. The trajectories built for the water field (plan item D9) cover two further free-energy sets, so the same five models were measured on them with no new simulation: **p38**, 29 ligands in the site of PDB entry 3FLY, and **hif2a**, 36 ligands in 5TBM. One hif2a ligand has no trajectory, because its charges could not be computed, so 36 of its 37 ligands are compared.

set	ligands	the crystal pose	best reading of the trajectory	change against the crystal pose
mcl1	25	**+0.575**	+0.463, multi-instance	−0.112 [−0.384, +0.169]
p38	29	**+0.261**	+0.455, multi-instance	+0.195 [−0.133, +0.522]
hif2a	36	**+0.342**	+0.534, by population	+0.192 [−0.068, +0.561]

On mcl1 no reading of the trajectory was ahead of the crystal pose. On these two the best reading is about +0.19 ahead, and on hif2a one reading clears its interval: the multi-instance fit, +0.178 [+0.003, +0.465]. **So "the answer is no" was a statement about the mcl1 set rather than about the method.** It is also the first time the multi-instance search chose a β above zero — on p38 it chose β = 1, so the model did concentrate the weight on some frames instead of averaging them.

What the three sets support together is weak, and two things say so. First the count: nine readings of a trajectory were compared with their crystal pose over the three sets, one of them clears its interval, and the near edge of that interval is +0.003 from zero. One result at the edge out of nine comparisons is about what chance produces. Second the reading itself: at β = 0 the multi-instance fit is the uniform ensemble, every conformation weighing the same, and "clusters equal" on the same set reaches +0.129 [−0.076, +0.365] and does not clear the interval. The two are the same model.

Where they differ is the normalisation of the kernel, which `--normalisation` measures at one attenuation (0.3) and one lambda (0.1) on all three sets, with no grid and nothing chosen. These numbers are therefore not comparable with the tables above, where every model is taken at its own best attenuation.

set	ligands	ensemble	multi-instance, β = 0	ensemble, no normalisation	multi-instance, no normalisation
hif2a	36	+0.471	+0.520	+0.550	+0.552
mcl1	25	−0.068	−0.049	+0.168	+0.183
p38	29	+0.198	+0.223	+0.255	+0.264

The gap between two paths through the same model is 0.019 to 0.049 of q^2 and falls to 0.002 to 0.015 when the normalisation is switched off, on all three sets. So most of it is the normalisation: the ensemble path normalises the kernel of the whole ensemble, while the multi-instance path normalises every conformation and then averages. Up to 0.015 is left over, so the normalisation is not all of it. Both places are defensible, and that is the point for reading the table: a gain of the size measured here can come from the choice alone.

```
python benchmarks/run_md.py --sets mcl1 p38 hif2a
python benchmarks/run_md.py --sets mcl1 p38 hif2a --normalisation
```

> **What this does not settle.** *Three sets now, but one force field and one length on all of them. A few nanoseconds samples the motion around a pose and not a change of binding mode. Two further questions are answered in the subsections below: the gain reported here is the largest of three readings rather than one fixed in advance, and every number rests on a crystal pose, while a set whose poses are not known is the case where a trajectory should help most.*

One reading fixed in advance

Every gain in the tables above is the largest of three readings of the same frames, and that is what makes a single interval at the edge of zero weak evidence. So the three sets are read once more with **one** reading of the trajectory, the plainest one: every conformation weighing the same, the row *trajectory, clusters equal* of the tables above. It contains no choice — there is no weighting to pick and no β to search — and it is not the largest gain on any of the three sets, so the rule does not name a winner.

set	ligands	the crystal pose	trajectory, clusters equal	change against the crystal pose
hif2a	36	+0.342	+0.471	+0.129 [−0.076, +0.365]
mcl1	25	**+0.575**	+0.391	−0.184 [−0.526, +0.136]
p38	29	+0.261	+0.239	−0.022 [−0.310, +0.299]

None of the three clears its interval. **Combined over the three sets the change is −0.006 [−0.157, +0.163]**, over 90 molecules, with each set weighted by the molecules it contributes and the molecules resampled inside their own set — three targets do not share an activity scale, so they are never mixed in a resample.

Fixed in advance, a trajectory ensemble around a crystal pose is worth nothing. The interval is now narrow enough to say that rather than to leave it open: it is 0.32 of q^2 wide over 90 molecules, against 0.66 on the 25 molecules of mcl1 alone. The +0.003 edge of the previous

subsection is what taking the best of three readings produces.

This is not a pre-registration. The sets had been measured before the rule was written down, and no rule written afterwards undoes that. What it avoids is the other fault: deciding which of several readings to believe after seeing which one won. The attenuation and lambda are still chosen by cross-validation inside every model, which the crystal pose gets in the same measure, so that choice is not what separates the two columns.

What the common frame is worth

The last row of the mcl1 table is a control and not a model: the same frames, read with each trajectory superimposed on its own first frame instead of on the one structure all the runs of its set were built from, which leaves every ligand in a frame of its own. It is now measured on all three sets. The change it causes has an interval of its own, taken against the common-frame ensemble on the same molecules and with the same weighting (clusters equal).

set	ligands	one common frame	each in its own frame	change against the common frame
hif2a	36	+0.471	+0.297	−0.175 [−0.391, −0.020]
mcl1	25	+0.391	+0.277	−0.114 [−0.386, +0.081]
p38	29	+0.239	+0.368	+0.129 [−0.207, +0.369]

The change is not in one direction. Only hif2a loses beyond the interval. On mcl1 the drop of 0.114 reported above does not clear the interval, now that the control has one; on p38 the own frame reads 0.129 *higher* than the common one. So the sentence above — that the common frame is what makes any of this measurable — was a reading of one set, and these three sets do not support it as a statement about the method. What an alignment of a few tenths of an ångström is worth is decided set by set here.

And on p38 this control is itself above the crystal pose, +0.107 [−0.351, +0.498]. That is one more reason to read the gains of that set with care: there a reading nobody would defend does no worse than the pose it was built to lose to.

The control is kept out of the counting. It is not one of the nine readings compared with their crystal pose above, and a set does not count as ahead because the control clears an interval. A model built to be wrong must make the evidence look neither stronger nor weaker than it is.

```
python examples/make_md_trajectories.py --set p38 --set hif2a --read-only --no-common-reference
python benchmarks/run_md.py --sets mcl1 p38 hif2a --force
```

Where the pose is not known

Every number above starts from a crystal pose, and an ensemble should help most where the pose is *not* known: there it covers uncertainty in the placement and not only motion around a placement that is already right. **mcl1** is therefore read that way as well, without a new set. Every ligand was posed while it was held out: its pose was destroyed — a random rigid placement within 2 Å of the site and a uniformly random orientation — and the pocket docking of part C put it back where the receptor's own fields prefer it, with no model, no activity of its own and no crystal frame of any kind.

Each ligand was then simulated from the pose that was found, for the same 2 ns and on the same receptor frame as above.

The 25 poses sit a median 1.12 Å from the crystal poses, and that median hides the shape of the test: 14 of them are within 1.3 Å, while on the other 11 the docking missed the site, 4.0 to 8.3 Å away. The table below is as much a reading of that docking as of the ensemble.

That docking has since been corrected (*What a bounded restart cannot do*). Missing the site on 11 of 25 was the bounded restart and not the scoring function: with twenty restarts from any orientation all 25 ligands come back within 2 Å of their crystal poses. The poses read below are the old ones, so the numbers are about them; re-posing the set and simulating again from the better poses costs 3.4 hours of one consumer GPU and has not been done.

model	q^2	change against the pose that was found	change against the crystal pose
the crystal pose	**+0.575**	+0.173 [−0.144, +0.567]	the ceiling
the pose that was found	+0.402	the reference	−0.173 [−0.567, +0.144]
trajectory from it, clusters equal	−0.040	−0.442 [−1.461, +0.272]	−0.615 [−1.470, −0.107]
trajectory from it, by population	+0.024	−0.378 [−0.833, +0.185]	−0.551 [−0.957, −0.197]
trajectory from it, multi-instance	−0.012	−0.413 [−1.134, +0.198]	−0.587 [−1.140, −0.215]

Where the pose is not known the ensemble does not cover for it; it makes the model worse. It is 0.44 of q^2 below the single pose it started from, and it is below the crystal pose beyond the interval, which the found pose itself is not. A pose error of a median 1.12 Å costs 0.173 of q^2; two nanoseconds of motion around those poses cost 0.442 more. The multi-instance fit chose $\beta = 1$ here, so even putting the weight on some frames rather than all does not recover the loss.

Read strictly, the comparison against the found pose does not clear its own interval: 25 molecules and an ensemble this scattered make a wide one. The statement that does clear its interval is the one against the crystal pose. The direction, however, is the opposite of the one expected on both readings, so this is an answer and not an open question.

Did the simulation repair the pose? Partly, and not in a way the model could use. The nearest kept conformation of a ligand is a median 0.79 Å from its crystal pose, against 1.12 Å for the pose the run started from — but the nearest of 11.4 conformations is a smallest of several and not a repair, and the middle conformation sits at 1.51 Å, farther out than the start. The 11 ligands the docking placed wrongly did move in, from a median 5.17 Å to 3.70 Å, and none of them came back under 2 Å in 2 ns; the 14 that started near drifted out a little, from 0.95 Å to 1.21 Å.

```
python examples/make_docked_poses.py --set mcl1            # examples/docked/mcl1_pocket.sdf
python examples/make_md_trajectories.py --set mcl1 --ligands examples/docked/mcl1_pocket.sdf \
    --tag docked --max-drift 0 --no-water
python benchmarks/run_md.py --sets mcl1 --docked
```

5.10 A cheap quantum stage: GFN2-xTB

The section above costs two to three minutes of ORCA per molecule. That is affordable for one series and not for a collection, and it is why the quantum fields were measured on four corrosion series and nowhere else. `cmfqsar/xtb.py` and the script `examples/xtb_atom_properties.py` run GFN2-xTB instead. GFN2-xTB is a semi-empirical tight-binding method: it solves a parameterised one-electron problem rather than the Kohn-Sham equations, and a molecule of forty atoms takes **0.2 seconds** including the two ionic states a Fukui index is built from.

```
python examples/xtb_atom_properties.py series.sdf -o series_xtb.sdf
cmf build series_xtb.sdf --activity pKi --fields electrostatic,steric,hydrophobic,hba,hbd,prop:
fukui_minus,prop:dual
```

What it writes. Per atom, and so available as `prop:` fields: the GFN2 partial charge `q_xtb`, the atomic polarisability `alpha_xtb`, and the condensed Fukui indices `fukui_plus`, `fukui_minus`, `fukui_zero` and `dual`. Per molecule, and so available as descriptors: the total energy, the HOMO and LUMO energies, their gap and the dipole.

The Fukui names mean the same as ORCA's. Both define $f^{+} = q(N) - q(N{+}1)$ and $f^{-} = q(N{-}1) - q(N)$, so one `prop:fukui_minus` field can be filled by either program; a test runs the three charge states separately and checks that xtb's printed value is that difference of its own charges. The charge is a different matter: `q_xtb` is a Mulliken-like tight-binding population and `q_hirshfeld` is a Hirshfeld charge, so they carry different names and a model fitted on one must be predicted with the same one.

What the two programs agree on. Measured over every atom of 101 corrosion molecules that have both (`benchmarks/run_xtb.py`): the charges correlate at **+0.95**, the dual descriptor at **+0.78**, f^{+} at **+0.78** and f^{-} at **+0.71**. The cheap method therefore orders the atoms of a molecule much as the expensive one does for charge, and roughly for reactivity.

What it is for, and what was found. A stage that costs 0.25 s instead of 360 s per molecule can be run over every series in the repository, which is what plan item D3 asked for. Over 52 series of 3511 molecules -- the eight Sutherland sets, the Hammett sets and the FEP collections -- adding the three reactivity fields to the five physico-chemical ones raises q^2 beyond the bootstrap interval on **7 series** and lowers it on 1, where chance alone would give about 1.3. The median change is +0.014, so most series gain nothing: a quantum field is not free, and on a series it does not suit it dilutes the fields that work. The series it does help are named in `benchmarks/results/xtb/table.md`; the largest gains are tnks2 (+0.181), thrombin with the nozob ligands (+0.129), mcl1 (+0.092), ACE (+0.067) and DHFR (+0.042, on 361 molecules).

> **About the geometry.** *As with ORCA, the geometry is not relaxed unless `--opt` is given, and for the same reason: xtb would relax each molecule on its own and a superimposed series would no longer be superimposed.*

What it does not compute. The ionisation potential and the electron affinity, and so not the hardness, the softness or the electrophilicity index that the ORCA stage derives from them. The energies xtb prints for the two ionic states of a Fukui run are electronic energies of the self-consistent charge step, not total energies, so a difference against the neutral's total energy would not be an ionisation potential. xtb computes those properly with `--vipea`, which needs a different Hamiltonian (GFN1 with the ipea parameters), and describing one molecule with two methods is worse than leaving the descriptor out.

xtb is not part of this repository. Its Windows and Linux builds are released at <https://github.com/grimme-lab/xtb/releases> under LGPL-3.0; give the path with `--xtb`, or set `XTB_EXE`. A path you give must exist. If it names neither a file nor a program on the path, you get an

error that quotes the path; it is never replaced by another xtb, because numbers from a program you did not ask for would not say where they came from. To search the path, leave `--xtb` out and leave `XTB_EXE` unset.

5.11 Can a Fukui function be a field?

A `prop:` field puts one isotropic Gaussian on each atom, weighted by a number: a charge, a condensed Fukui index. That is a statement about *how much* and says nothing about *where*. The quantity a condensed Fukui index is condensed from is a function of position, $f(\mathbf{r}) = \rho_{N+1}(\mathbf{r}) - \rho_N(\mathbf{r})$, ORCA will write it on a grid, and `cmfqsar/cdft.py` asks whether that function can be brought into the program's algebra the way the probe fields were — by fitting several Gaussians per atom, which keeps the overlap of two fields a closed-form sum over atom pairs and so keeps the co-field and the atom contributions.

It cannot, and the reason is a node. Fitted against the valence part of the map on the four corrosion series with finished quantum runs (`benchmarks/results/cdft_kernels/table.md`), the relative error of an isotropic fit is 84%. A Fukui function is essentially a frontier-orbital density: on acrolein it is π^*, three quarters of its squared magnitude lies more than 0.5 Å off the molecular plane, and there is a node *in* the plane. A function that is positive everywhere and centred on a nucleus cannot make a node.

Giving every atom a direction

The smallest extension that can make one is to multiply the Gaussian by a linear factor. With the width convention this program uses, atom l then carries

$$s_l(\mathbf{r}) = \exp\left(-\frac{b}{2}\,|\mathbf{r}-\mathbf{R}_l|^2\right), \qquad p_l^u(\mathbf{r}) = (r_u - R_{l,u})\exp\left(-\frac{b}{2}\,|\mathbf{r}-\mathbf{R}_l|^2\right) \tag{5.2}$$

for $u = x, y, z$: one s and three p functions per atom per width. A p function is odd about its own centre, so it has a node through the nucleus and a lobe on either side, and three Cartesian components span every direction — a lobe along a bond, along a lone pair or perpendicular to a ring plane are the same three numbers in different proportions, and nothing has to be oriented by hand.

The overlaps stay closed-form, which is the only reason this is the extension worth trying. With $p = a + b$, $\mathbf{d} = \mathbf{B} - \mathbf{A}$ and $S = (2\pi/p)^{3/2}\, e^{-ab|\mathbf{d}|^2 / 2p}$ the four blocks are

$$\langle s_A|s_B\rangle = S, \qquad \langle p_A^u|s_B\rangle = \frac{b}{p}d_u S, \qquad \langle s_A|p_B^v\rangle = -\frac{a}{p}d_v S, \qquad \langle p_A^u|p_B^v\rangle = \left[\frac{\delta_{uv}}{p} - \frac{ab}{p^2}d_u d_v\right]S \tag{5.3}$$

and `cmfqsar/anisotropic.py` evaluates them. A kernel between two such fields is still a double sum over pairs of atoms, with sixteen terms per pair of widths instead of one.

What it is worth, measured

The plan sets two conditions for keeping the extension: the fit error has to drop, and the model has to notice. Measured on 4 series, 101 molecules in all (`benchmarks/run_anisotropic.py`):

series	n	field	error, s only	error, s and p	coefficients per atom
12_c1ccoc1	27	fukui_plus	78%	**61%**	4 → 16
12_c1ccoc1	27	dual	87%	**66%**	4 → 16

series	n	field	error, s only	error, s and p	coefficients per atom
13_c1c_nH_nn1	25	fukui_plus	79%	**61%**	4 → 16
13_c1c_nH_nn1	25	dual	87%	**67%**	4 → 16
161_c1ccc2scnc2c1	20	fukui_plus	81%	**61%**	4 → 16
161_c1ccc2scnc2c1	20	dual	91%	**68%**	4 → 16
7_c1nncs1	29	fukui_plus	80%	**63%**	4 → 16
7_c1nncs1	29	dual	91%	**69%**	4 → 16

The fit improves, and by a lot: 84% to 64% on average, the best case reaching 61% (fukui_plus on `13_c1c_nH_nn1`). The kernel the fit produces also agrees better with the one the exact field produces, +0.920 to +0.977 in correlation. The diagnosis was right: what the isotropic basis was missing is a direction.

series	field	five fields	exact grid	s alone	s and p alone	five + s	five + s and p
12_c1ccoc1	fukui_plus	+0.220	+0.052	+0.103	+0.072	+0.250	+0.235
12_c1ccoc1	dual	+0.220	+0.028	+0.057	+0.012	+0.272	+0.241
13_c1c_nH_nn1	fukui_plus	+0.654	+0.636	+0.685	+0.684	+0.672	+0.681
13_c1c_nH_nn1	dual	+0.654	+0.575	+0.505	+0.567	+0.649	+0.664
161_c1ccc2scnc2c1	fukui_plus	+0.689	+0.638	+0.636	+0.627	+0.658	+0.655
161_c1ccc2scnc2c1	dual	+0.689	+0.522	+0.556	+0.573	+0.596	+0.624
7_c1nncs1	fukui_plus	-0.037	+0.121	+0.090	+0.027	-0.012	-0.024
7_c1nncs1	dual	-0.037	-0.031	+0.045	-0.021	-0.001	-0.022

And the model does not notice. Averaged over the 8 series-field pairs the p functions change q^2 of the field alone by **-0.017** (2 of 8 better), and of the field added to the five ordinary fields by **-0.004**.

The two measurements disagree for the reason the isotropic fit disagreed with itself: a kernel is an inner product between molecules, and an error that falls the same way for every molecule cancels in it. The p functions recover the π lobes above and below a ring — and every molecule of a congeneric series has a ring in the same place, so every field improves by about the same amount and the distances between the molecules stay where they were. What a model needs is not a better field but a field that differs between molecules in a way activity follows.

So the extension is implemented, tested against quadrature, and **not wired into the ordinary field machinery**, because by the plan's own test it has not earned its four-fold cost. It is there for the case that would earn it: a series whose members differ in the *shape* of a frontier orbital rather than in where their atoms are.

5.12 Where the added electron went: the spin field, and the series that does not need it

The previous section asked whether a Fukui function can be a field and answered no, for a reason about shape. This section asks a narrower question with a clearer answer. A reduction potential is the energy of adding one electron to a molecule. The **spin population** of an atom — the difference between the number of electrons of each spin direction on that atom — says how much of an unpaired electron sits there. For the radical anion of a closed-shell molecule, those populations say *where the added electron went*. That is one number per atom, so it is a `prop:` field, and it is the most direct description of a reduction a field can be.

What the quantum stage had to learn first. Three things were missing (plan item EC1).

- **The charge and the spin state come from the record.** `cmf` used to take the charge from the connection table and the lowest spin state that fits the electron count. Nothing in a connection table says that an iron(II) complex is a quintet, and a radical written as a neutral molecule with an unpaired electron would have been computed as a closed shell, which is a different substance. `charge` and `multiplicity` are now read from the SD properties when they are there. A multiplicity of the wrong parity for the electron count is refused rather than adjusted, because the likely cause is a mistake in the file.
- **A continuum solvent.** A reduction potential measured in dimethoxyethane is not the gas-phase quantity. `--solvent` names a solvent ORCA knows, which becomes a `CPCM(name)` keyword, or one of a small table of solvents it does not know, which becomes a block with that solvent's dielectric constant and refractive index. An (ε, n) pair may also be given directly.
- **Heavier elements.** A def2 basis uses an effective core potential — a function standing in for the inner electrons — from rubidium upward. The heavy level of theory names it explicitly, so that a calculation on a ruthenium or iridium complex does not silently fail.

The spin populations of both ionic states then arrive as the fields `prop:spin_anion` and `prop:spin_cation`, and `prop:spin` for a molecule that is open-shell to begin with. A closed-shell molecule has no spin density, so nothing is written for it, which is correct and not a gap.

The series. 69 substituted phenazines with a computed one-electron reduction potential (`examples/phenazine/`, built by `examples/make_phenazine_examples.py`). A phenazine is a flat three-ring system with two nitrogens in the middle ring; it is the core of several organic electrode materials. The potentials run from −2.092 to −0.160 V, and the mean of the training set alone makes an error of 0.285 V, which is the number every model has to beat. All 69 are superimposed on the phenazine core, which they sit on to within 0.08 ± 0.07 Å, so the series has a frame and the overlap kernel applies.

The spin populations are right. They sum to 1.0000 ± 0.0000 over every molecule, which is what a doublet must give and is therefore a check on the reading rather than a result. 34.5 ± 5.9 % of the unpaired electron sits on the two core nitrogens, and on average only 4 % of it leaves the core at all. The added electron goes where a chemist would put it.

What is circular here, said plainly. The target is computed, not measured, and it comes from the same kind of calculation as the spin field. A good score for the spin field would therefore not be evidence that it predicts an experiment. It would be evidence that the field carries what an anion calculation knows. The control that separates the two readings is the **electron affinity** on its own: one number per molecule, no geometry, no field, correlated +0.77 with the target. A field that does not beat that number has not earned the ten hours of DFT that produced it.

The measurement (`benchmarks/run_phenazine.py`, `benchmarks/results/phenazine/table.md`): the five ordinary fields, then the same with the Hirshfeld charge field in place of the Gasteiger one, then with the Fukui fields added, then with the spin field added, then with all of them, against the electron affinity alone and the nine global quantum numbers alone. Five random folds and the three

source tables held out one at a time. Every model chooses its ridge and its field width inside each fold.

model	random folds	one source table held out
the electron affinity alone	0.194 ± 0.023	0.238 ± 0.080
nine global quantum numbers	0.181 ± 0.030	0.231 ± 0.096
the five ordinary fields	0.158 ± 0.019	0.233 ± 0.102
with the Hirshfeld charge field	0.153 ± 0.017	0.217 ± 0.107
with the Fukui fields	0.151 ± 0.021	0.241 ± 0.104
with the spin field	0.161 ± 0.021	0.230 ± 0.106
with all of the quantum fields	0.151 ± 0.017	0.230 ± 0.120
the spin field alone	0.290 ± 0.039	0.270 ± 0.100

Mean absolute error in volts. **No quantum field improves on the five ordinary fields.** Every paired interval over the per-molecule errors holds zero: the Hirshfeld charge field gains 0.006 V [−0.016, +0.028] on random folds, the Fukui fields 0.007 V [−0.012, +0.029], all of them together 0.007 V [−0.015, +0.031]. The spin field is the worst of the three, at −0.003 V [−0.008, +0.002]. **And the spin field alone predicts nothing**: 0.290 V against the 0.285 V of the mean, an r^2 of −0.006.

Why, measured rather than argued. The kernel compares two molecules by the overlap of their fields, normalised, so a pair of molecules has a similarity between −1 and 1. The question is how much that similarity varies across the series when one field is used by itself:

field used alone	how alike two molecules are	smallest pair
`fukui_plus`	0.953 ± 0.032	+0.771
`spin_anion`	0.910 ± 0.127	−0.082
`steric`	0.902 ± 0.057	+0.702
`fukui_minus`	0.701 ± 0.201	+0.096
`q_hirshfeld`	0.344 ± 0.182	−0.323
`electrostatic`	0.315 ± 0.168	−0.105
`dual`	0.308 ± 0.307	−0.895

Through the spin field, any two molecules of this series are 91 % alike; through the f^+ field, 95 % alike. **The added electron always goes to the same place**, so the field that says where it went is nearly a constant, and a model cannot tell two molecules apart by something they share. What *does* differ between these molecules is which substituent sits at which ring position, and that is what the ordinary steric and electrostatic fields already describe. The electrostatic and Hirshfeld charge fields vary three times as much, which is why they are the fields that carry the model.

This is not a fault in the spin field and not an argument against it. It is a statement about this series, and it says exactly where the field *would* be worth its cost: a series in which reduction happens at **different places** in different members — two or more competing redox centres, a family in which the electron moves from the ring to a substituent — rather than one fixed core decorated differently. The phenazines are the opposite of that case. Running the measurement on them was still the right thing to do, because the field was cheap once the quantum stage could produce it,

and a negative result with a measured explanation is usable: it tells the next person which series not to try.

The co-field. Fitted on all 69 with every quantum field, the co-field shows the same thing from the other side. The Hirshfeld charge field carries the largest weight by a factor of two (+0.201 against +0.122 for the next), and its strongest point sits 2.87 Å from the nearest core nitrogen — out where the substituents are, not on the core. The spin field's strongest point is at the same place but a fifth of the size. The model is reading what the substituents do, which is what the ordinary fields were already telling it.

5.13 A metal in the middle: coordination complexes

Everything up to this point assumed a molecule that RDKit can read. A coordination complex is not one, and three separate things break on it. Each of them breaks quietly, which is why each is stated here with the number that measures it.

The bonds. A metal–ligand bond is not a shared electron pair. Written as an ordinary single bond it makes the donor atom one valence too high, and RDKit then refuses the molecule. The repair is a *dative* bond, which points from the donor to the metal and leaves the donor the valence it has in the free ligand. `cmfqsar/complexes.py` builds a complex from coordinates in four steps: find the bonds by distance; take the metal out, which leaves the ligands as separate fragments; work out each fragment's bond orders on its own; and put the metal back with one dative bond per donor. A fragment needs its charge before its bond orders can be found — a cyclopentadienyl ring is aromatic at −1 and a radical at 0 — so the charges are taken from the data set where it states them, and tried in turn where it does not. The charge left over goes on the metal, which for a data set that states its ligand charges is the oxidation state.

One trap in that step cost a quarter of the iron series before it was found. A chloride, bromide or iodide ligand is a single atom with no bond to determine, and bond perception leaves it neutral whatever charge it is asked for. The charge that belonged to the halide was then given to the metal: the complex had the right total charge and the wrong charge inside it. The reader now writes the stated charge onto a one-atom ligand itself, and it refuses any complex whose formal charges do not add up to the stated total.

The charge. The Gasteiger scheme has no parameters for a d-block metal, and one metal atom makes RDKit return "not a number" for *every* atom of the molecule. The electrostatic field of a complex was therefore identically zero: not an error, and not a field. Of 200 iron complexes checked, 197 lose their electrostatic field this way without the repair. The repair computes the charges with the metal atoms removed, which leaves each ligand with the charges of the free ligand, and gives each metal its formal charge; the charge source is then reported as `gasteiger+formal` rather than `gasteiger`, so a model fitted on one cannot be confused with a model fitted on the other. What this does **not** describe is the donation from the ligand to the metal, so the donor atoms are less polarised than they really are. A calculated charge, written into the per-atom property `partial_charge` and read with `charge_model="file"`, is the better description wherever one is available.

The size of the metal. The Tripos table stops at the main group, so every metal used to get the default radius. The parameters now come from the universal force field, which covers the whole periodic table, with its van der Waals distance halved to a one-atom radius and put on the scale of the Tripos radii by the ratio of the two carbon values. Iron comes out at 1.285 Å against carbon's 1.70 Å, which is what a buried metal centre should be, and close to the 1.20 Å the Tripos table already uses for the alkali metals. The well depths are taken unchanged, because the two force

fields agree on carbon. A metal carries no hydrophobic and no hydrogen-bond increment, since the published increments were fitted to organic molecules and have nothing to say about it.

The frame a metal gives, and what it is worth

A complex has one atom that every member of a series shares: the metal. Two frames are built on it. `align_on_first_sphere` puts the metal at the origin and turns each complex so that its donor atoms sit closest to a reference complex's donor atoms. Which donor answers to which reference donor is not known, and a coordination sphere is symmetric, so every assignment that pairs donors of the same element is tried and the best is kept — the same trap the phenazine core showed, where one arbitrary matching reports several ångström for two molecules that lie exactly on top of each other. `principal_metal_frame` is the rule for a series whose members share no coordination sphere: the metal at the origin and the principal axes of the whole complex, with the signs fixed by the third moment, and it reports how close the two smallest moments are, because a small gap means the frame is arbitrary in that plane.

The two are not interchangeable, and the nickel series below says so. Matching the first coordination spheres puts the donors 0.03 ± 0.04 Å apart (worst 0.23 Å), a real shared frame. The rule frame puts the same donors **1.82 Å** from where the matched frame puts them (worst 3.45 Å). A rule that uses the whole complex is pulled around by the substituents; matching the atoms that every member shares is not. On the iron series, where no match exists across the whole set, the rule frame is nearly degenerate for 11 % of the complexes.

Two series, and what they measure

series	what it is	why
`examples/complexes_fe/`	2216 iron(II) complexes from the structural database with a computed Fe(III)/Fe(II) reduction potential	they share the metal and nothing else: 377 different first coordination spheres
`examples/complexes_ni/`	25 substituted triaqua nickel–bis(pyrazolyl)pyridine complexes with the computed potentials of two couples	one ligand skeleton, one N3O3 coordination sphere, and the population analysis of the same calculation as its own control

Both targets are computed, not measured, so both say how well a representation orders complexes that one calculation produced. `benchmarks/run_metal.py` measures them with five outer folds and ten inner folds. The iron series is measured twice: with five random folds, and with five folds built from the first coordination spheres, so that no sphere in a test fold was seen in training.

The iron series: the field beats the composition, and the frame decides everything

Mean absolute error in volts, all 2216 complexes, against a training-mean error of 0.523 V: element counts and the metal site (`composition`) 0.313, the metal site alone (`site`) 0.359, and the frameless field autocorrelation (`invariant`) **0.253**. The paired difference is +0.060 V [+0.049, +0.071], so the fields beat the composition and the interval excludes zero. With the coordination spheres held out the ranking is the same: 0.336, 0.398 and 0.284, difference +0.055 V [+0.044, +0.068].

This is worth stating plainly, because it is the first inorganic target in this manual where the field wins. Items BT1 to BT3 found the opposite on a battery voltage, where the composition beat every field. The difference is what the target is: an average voltage is set by what the cell is made of, while a reduction potential of one complex is set by what sits around its metal and how.

The frame decides whether the overlap kernel works at all. In the frame each file happens to carry, the overlap kernel scores 0.490 V against a training-mean error of 0.523 V, R^2 = +0.083: a model that has learned almost nothing. The same kernel after each complex is put in its own metal-and-principal-axes frame scores 0.302 V, R^2 = +0.588, so the frame is worth +0.188 V [+0.173, +0.202]. That lifts the overlap kernel just past the composition (+0.012 V [+0.001, +0.022], an interval that excludes zero by little) and leaves it behind the frameless kernel (+0.049 V [+0.039, +0.058]). Holding the coordination spheres out of training says the same, and a little more strongly: the frame is worth +0.182 V [+0.167, +0.197] and the margin over the composition grows to +0.022 V [+0.009, +0.033]. On a series whose members share nothing but the metal, a rule frame is far better than no frame, slightly better than no geometry at all, and worse than not needing a frame.

The nickel series: the gate, and what the co-field reads

Here the series is congeneric and the frame is real. For the Ni(III)/Ni(II) couple, against a training-mean error of 0.488 V: the population analysis of the same calculation the target comes from (`quantum`) 0.345 V, the coordination geometry alone (`site`) 0.370 V, and the five ordinary fields in the shared frame (`physchem`) **0.150 V**, R^2 = +0.820. The fields beat the quantum descriptors by +0.195 V [+0.068, +0.321] and the site by +0.220 V [+0.085, +0.355], both intervals excluding zero, and leave-one-out says the same. This is the gate of the item, and it is passed: a five-field model of a metal complex beats the global quantum numbers of the calculation that produced its target.

Two readings qualify it. The frameless kernel ties the aligned one (+0.004 V [−0.076, +0.068]), so the shared frame is not what the model needed, exactly as the cyclodextrin hosts found. And the other couple is much harder: the Ni(II)/Ni(I) potentials have a standard deviation of 0.06 V, and their training-mean error is 0.050 V, so there is little to predict. The fields reach 0.029 V against the quantum descriptors' 0.043 V, which the random folds call a difference (+0.013 V [+0.006, +0.022]) and leave-one-out does not (+0.008 V [−0.000, +0.017]).

The co-field says what the model is reading, and it is the second coordination sphere. For the oxidation potential the largest weights sit 4.5 to 7.3 Å from the nickel atom — the acceptor field −0.349 at 5.60 Å, the electrostatic field −0.201 at 6.70 Å, the hydrophobic field +0.179 at 4.52 Å — which is out among the substituents, not at the metal and not on its donors. That is the right answer for this series: every member has the same metal and the same six donors, so nothing at the metal site can separate two of them, and what differs is what the substituents do. For the reduction potential the weights are five to ten times smaller and sit closer in, 2.6 to 4.7 Å, which fits a target that barely varies.

What this does not cover

The metal identity is a descriptor the reader provides and neither series exercises it: one series is all iron and the other all nickel. The gate's own data sets — iron porphyrins with the turnover frequency and overpotential of catalytic Tafel-plot benchmarking, nickel diphosphines with their hydricities — were not used, because no machine-readable table of either was found; the published turnover frequencies come from different papers under different conditions and methods, which the sources themselves warn against pooling. Both targets here are computed rather than measured. And the electrostatic field of a complex still uses the charges of the free ligands, so the first step for anyone extending this is a calculated charge per atom.

5.14 The field a reaction centre feels: a vector kernel, and why it changes almost nothing

Every field in this program is a scalar, and the kernel of two molecules is the overlap of two scalars. The quantity a reaction centre actually feels is not the potential but its gradient, the electric field. The two are not the same description: a molecule can have a large potential and no field at a point, and the oriented-electric-field literature on catalysis is about the vector.

A field type `efield` supplies the vector kernel. It carries the same charges as the electrostatic field, and what differs is the integral:

K(A, B) = integral of grad phi_A(r) . grad phi_B(r) over all space

For atom-centred Gaussians this is in closed form. The pair integral of two Gaussians of the shared width α is the ordinary overlap multiplied by $3\alpha/2 - \alpha^2 d^2/4$, which is the negative Laplacian of the overlap with respect to the separation of the two centres. So the new kernel needs no quadrature, rule 3 of the plan holds, and the co-field and the Shapley decomposition are unchanged. The closed form was checked against a grid integral of 2.7 million points on two real molecules and agreed to 0.000 %.

One property is unlike every other pair integral here: **it changes sign**. It is positive for two close atoms, zero at d = sqrt(6/α) — 2.45 Å at α = 1 — and negative beyond, because two charges far apart have gradients that point against each other. On the Hammett series 72 % of the atom pairs lie beyond that distance, so most pairs contribute a negative term. The kernel matrix is still positive semidefinite, because it is an inner product of two vector fields; a single pair may lower the kernel, and the diagonal cannot be negative.

The reading that predicts the result

Before any model is fitted, one number says what to expect. The cosine-normalised vector kernel and the cosine-normalised scalar kernel **correlate +0.980** over the 4371 pairs of the Hammett series. Two molecules are 0.312 ± 0.314 alike through the scalar kernel and 0.302 ± 0.312 through the vector one. The vector kernel orders the pairs of this series almost exactly as the scalar one does, and a model cannot do much with a description that is a near-copy of one it already has. This is the same reading that explained the spin field of item EC1, used in advance this time rather than afterwards.

What was measured

Two series where an overlap kernel is the right tool, which means a series with a shared frame: the 95 substituted benzenes of the Hammett sets, in one frame by rule, and the 25 nickel complexes of item EC4, which share one ligand skeleton and one N3O3 coordination sphere. The iron series of item EC4 is left out, because that item measured an overlap kernel of any kind to be behind the kernel that needs no frame there, and because the compiled kernel loop does not implement this field kind.

target	five fields	five + efield	efield alone	electrostatic alone
σ_m	0.0972	0.0967	0.0963	0.0977
σ_p	0.1541	0.1473	0.1624	0.1582
π	0.3129	0.3187	0.7762	0.8373
nickel E_ox (V)	0.1501	0.1638	0.2176	0.2234
nickel E_red (V)	0.0293	0.0282	0.0312	0.0294

The gate is not passed. Added to the five fields, the vector kernel changes nothing on σ_m (+0.0005 [−0.0037, +0.0051]) and on σ_p (+0.0068 [−0.0045, +0.0190]), and on the nickel oxidation potential it makes the model *worse* by −0.0138 V [−0.0342, −0.0005], an interval that excludes zero in both fold designs. That last result is the clearest statement in the table: a sixth field that is a near-copy of the first costs accuracy rather than adding any, which is what a kernel with two nearly collinear blocks should do.

Two places where it does something are worth recording. Used *in place of* the electrostatic field rather than beside it, the vector kernel ties the five-field model everywhere (nickel E_ox +0.0019 V [−0.0031, +0.0077]), so it is a legitimate substitute at the same price. And on the nickel oxidation potential, the vector kernel alone beats the scalar field alone by +0.0299 V [+0.0144, +0.0459] under leave-one-out, though not under five random folds (+0.0057 V [−0.0072, +0.0194]). That is the one measurement consistent with the through-space catalysis argument, on the one cationic series available, and it is too small and too design-dependent to carry the item.

The apparent win on π — the vector kernel alone beats the scalar field alone by +0.0620 [+0.0238, +0.1084] — is a difference between two bad models. Hydrophobicity is not an electrostatic quantity, both models score R^2 below 0.2 where the five fields reach 0.86, and nothing should be concluded from which of two failures fails less.

What the arrows say

The co-field of a scalar field is a value at a place. The co-field of this kind is an arrow field: at each point it says which way the model wants the field to change, and `cofield_arrows` returns the strongest arrows with a minimum separation so that they spread over the molecule instead of crowding one peak. On the nickel oxidation potential the strongest arrows sit 5.6 to 6.6 Å from the metal — out among the substituents, where item EC4's scalar co-field also put its weight — and they point *inward*, towards the metal. The model wants the field to rise towards the metal from the substituent positions, which is the second-sphere statement in the form the vector kernel is able to make.

Kept and labelled under rule 5. The field type stays: it is exact, it costs no quadrature, it is a working substitute for the electrostatic field, and it gives the arrow reading that no scalar field can. It is not one of the five default fields and has to be asked for by name.

One fault this measurement uncovered and did not fix

One of the 95 Hammett molecules, the selenomethyl substituent, has an electrostatic field that is identically zero. The cause is the one item EC4 found for metals: the Gasteiger scheme has no parameters for selenium, and one such atom makes it return "not a number" for every atom of the molecule. EC4's repair covers the d-block metals only, so selenium and the other elements the scheme does not know are still affected. This is left as it is rather than repaired here, because a general repair would change the published Hammett tables, and such a change belongs to its own item with its own measurement.

5.15 Reading a few places instead of all of them

A kernel compares two fields over all space. Some questions are about one place. A Hammett constant is defined by what a reacting group feels where it sits; a molecular electrocatalyst is discussed in terms of what the metal centre feels from the second coordination sphere. A series in a shared frame has such places, and the field and its gradient there are ordinary numbers, so they can go into the model as descriptors beside the fields.

`cmfqsar/descriptors.py` reads them. `field_at_points` gives the value of any field of the field set at any list of points, exactly — every atom contributes, unlike the routine that draws a field on a grid, which drops Gaussians below a tolerance. `field_gradient_at_points` gives the gradient, and for the electrostatic field that gradient is the oriented electric field the electrostatic-catalysis literature is about. `anchored_matrix` builds the block and `anchor_names` names its columns by place, field and reading: `at:para_site:electrostatic:gx`. The gradient was checked against a finite difference of the value at every anchored point of every field on real molecules, and agrees to 1e-11.

Which field, though. The Gaussian field of a CMF field set is not a Coulomb potential: its atom-centred Gaussians fall off far faster than 1/r, which is what makes the kernel a closed form and what this manual's own measurement says represents long-range tails badly. So the gradient of the Gaussian electrostatic field is proportional to an electric field near the atoms and is not one in volts per ångström. `source="coulomb"` therefore reads the true point-charge potential and its gradient instead, in e/Å and e/Å², from the same charges. Both are offered because they are different descriptions, and the measurement below compares them.

The places

For the Hammett series the frame is a rule, so the places are exact rather than read off a molecule: the ring is a regular hexagon of radius 1.395 Å in the z = 0 plane with the ipso carbon on +x. The points are the ipso carbon, the ring centroid, a point 1.7 Å above the ring face, and — 1.5 Å beyond the meta and para carbons along the same radius — the two places where the reacting group of the σ definition sits. For the nickel complexes of item EC4, which are aligned on the metal and its first coordination sphere, the points are the metal itself and the two positions where item EC5's vector co-field put its strongest arrows, which are the two substituent positions of the ligand. So the places are either given by the rule that made the frame or by an earlier measurement, and not chosen to suit the answer.

The result: reading everywhere wins, everywhere

`benchmarks/run_anchors.py` measures the anchored block against the field model with the usual protocol — five outer folds, ten inner folds for the ridge and the field width, and a paired bootstrap interval on the per-sample absolute errors.

target	five fields	anchored, all points	anchored, reaction centre	true Coulomb
σ_m	**0.0972**	0.1322	0.2328	0.1087
σ_p	**0.1541**	0.2138	0.3588	0.1801
π	**0.3129**	0.7841	0.9958	0.8985
nickel E_ox (V)	**0.1501**	0.2801	0.4718	0.4375
nickel E_red (V)	0.0293	0.0440	0.0362	0.0487

The gate is not passed, and it is not close. Against the field model the anchored block loses by 0.035 units on σ_m [−0.066, −0.008], 0.060 on σ_p [−0.119, −0.015], 0.473 on π [−0.637, −0.327] and 0.015 V on the nickel reduction potential [−0.023, −0.006], every interval excluding zero. Added to the five fields it does not help either: a tie on σ_m (−0.0029 [−0.0190, +0.0127]) and a loss everywhere else. The reaction-centre points alone are the worst model in the table, with a negative R² on four targets of five — at 2.9 Å from the ring centre the Gaussian field is small, and eight columns of small numbers with a ridge chosen inside the fold overfit.

Two things in the table are worth keeping. The **true Coulomb potential is the better anchored descriptor**, by +0.0235 units on σ_m [+0.0031, +0.0478], which is the physical argument of the item confirmed in the one place it could show: if you are going to read a potential at a point, read the real one and not a sum of Gaussians. And the field model's advantage is not that it has more numbers — the anchored block has 100 columns on 95 substituents and still loses.

The lesson about diagnostics, which is the most useful thing here

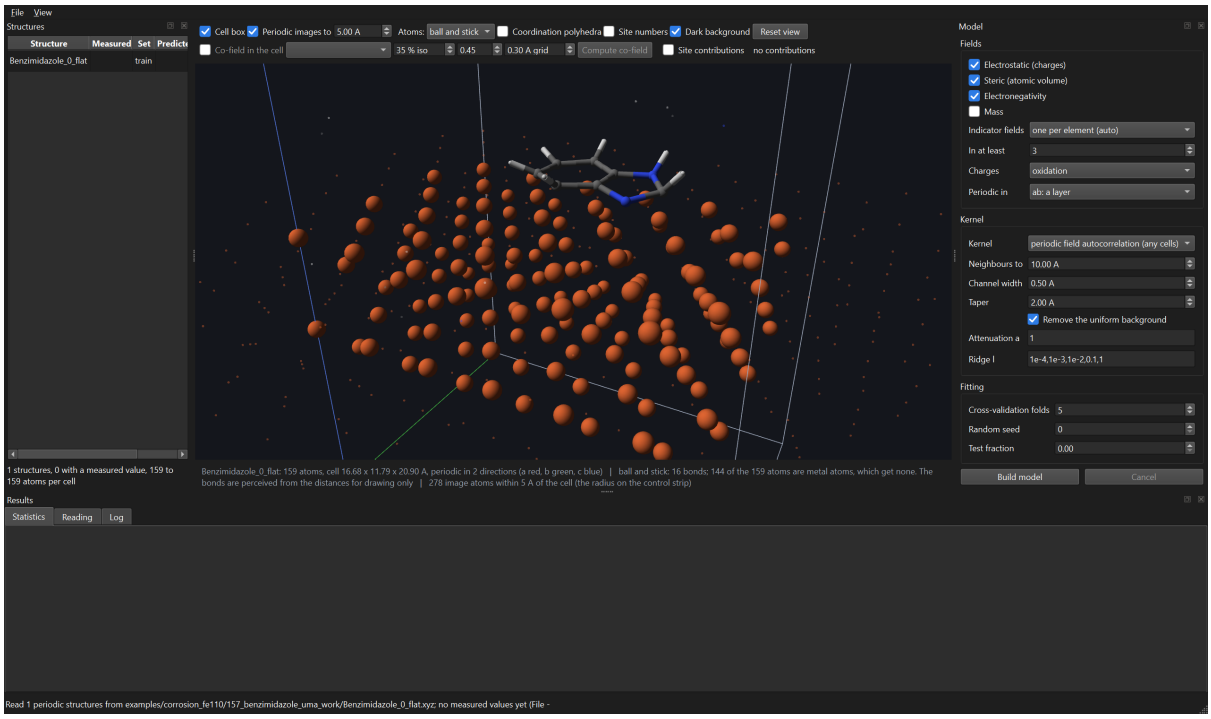
Item EC5 learned to ask, before fitting anything, whether a new description can differ from the one the model has. There it was one number — the correlation between two whole kernels — and it predicted the result exactly. The same question was asked here and the answer **misled**. The best single anchored column reaches |r| = 0.85 with the nickel oxidation potential and |r| = 0.83 with σ_p, which looks like a model waiting to be fitted; under nested cross-validation the same block gives $R^2 = -0.88$ and 0.12. The difference is selection: a correlation between two kernels is one number computed once, while the best of 60 columns on 25 molecules is a maximum over 60 tries and is inflated by exactly that. A whole-kernel correlation is a fair diagnostic; a best-of-N column correlation is not, and this measurement is what says so.

What this does not cover

The item's catalyst use is still waiting on data, not on code: the anchored reading of a metal site is exactly what a molecular-electrocatalyst series would be modelled with, and no series with a measured catalytic property was found in machine-readable form (item EC4 records the search). The application has no way to place points in a frame, so the block is available to a script and not to the window. And the block cannot be asked for by a `calc:` name like the other computed descriptors, because it needs the field set and the points as well as the molecule.

5.16 A metal surface as the receptor, and a score that needs the answer to compute

A protein pocket and an electrode are the same problem with different receptors. `cmfqsar/slab.py` turns the top layers of a slab, with their periodic images out to a cutoff, into the same `MoleculeFields` a pocket becomes, so everything downstream — the complementarity scores, the docking, the site profiles — applies to a surface without changes. A slab is read from a POSCAR or CIF, or built: `bcc110_slab()` gives the 6 × 6 × 4 Fe(110) slab at a = 2.78 Å that the reference poses of `examples/corrosion_fe110` were computed on, with its top layer at z = 0.



The Fe(110) slab of `examples/corrosion_fe110` *with a benzimidazole molecule adsorbed flat on it, the pose relaxed with the UMA potential, in the window for crystals, read as a layer periodic in two directions and drawn in ball and stick: the four layers of the slab as spheres, the molecule on the top layer with its bonds, which the window perceives from the distances and gives to the organic atoms only, the cell with its vacuum above the slab, and the periodic images of the cell within 5 Å as dots. This is the structure the slab receptor is cut from, seen whole. Taken by* `docs/make_screenshot.py --what slab`.

The molecule in that picture is drawn as spheres, which is what the *Atoms* box offers by default. *ball and stick* in the same window gives it its bonds while the slab stays spheres, because a metal atom gets none; the section *How the atoms are drawn, and the bonds* says why, and what the rule is worth on these very molecules.

What a clean metal surface offers, which is less than a protein does

Four of the five physicochemical fields are **identically zero** on a one-element metal slab, and not for want of parameters. A one-element cell has no electronegativity differences, so every charge model in the program gives it zero charges; the hydrophobic and hydrogen-bond increments are fitted to organic fragments and a metal carries none, which item EC4 made explicit rather than leaving a default in place. Only the steric field survives, from the van der Waals parameters EC4 added. All four complementary pairs of `cmfqsar/complementarity.py` therefore vanish against a clean metal, and a complementarity score of a molecule on a metal surface **is a shape score**. The benchmark prints that list rather than leaving a reader to work it out.

What a metal has instead is its image charge, and `image_charge_energy` computes it in closed form: the interaction of the molecule's charges with their mirror through the surface plane, $-\tfrac{1}{2}\,\Sigma_{ij}\, q_i q_j / |r_i - r_j^*|$, which for one charge at height h is the textbook $-q^2/4h$. Two measured facts about it decide how it can be used.

- It is **invariant to translation in the plane and to rotation about the normal** — 4×10^{-17} eV and 1×10^{-17} eV on a real molecule. A conductor is homogeneous, so the image term can set a height and a tilt and can never say where on the lattice a molecule sits. Only shape can.

- It is **small**: −0.033 eV, about −3 kJ/mol, for benzimidazole at its relaxed pose, against adsorption energies spanning ±200 kJ/mol. It is the right physics for the physisorption of a polar molecule and two orders of magnitude too small to describe chemisorption on iron.

So `dock_on_surface` optimises the four freedoms a surface leaves — height, tilt, rotation about the normal, translation in the plane — with the tilt bounded, and it uses shape alone by default, because a term that cannot place a molecule laterally should not be mixed into the search that does.

The pose: docking does not place these molecules either

Median in-plane error against the relaxed pose, over the two Fe(110) nodes (`benchmarks/run_slab.py`):

start	benzimidazole	quinoline
EC2's flat rule pose	2.46 Å	3.45 Å
that pose docked against the slab	2.50 Å	3.66 Å
docked, both faces tried	**2.37 Å**	**3.23 Å**
docked with the image term on	2.45 Å	3.78 Å
from a conformer of the SMILES, docked	2.74 Å	3.73 Å

Refining the rule pose against the slab does not improve it: on its own it is slightly worse, and trying both faces gains 0.09 Å and 0.22 Å, which takes the share within 1 Å from 0 % to 7 % and from 3 % to 5 %. The image term changes nothing, as its invariance says it must not. **Shape complementarity with the slab does not decide the face or the in-plane placement either**, which is item EC2's finding reached from the other side: EC2 found no rule and no reference decides the face, and this says complementarity does not decide it.

The energy: the score beats EC2's control, and only where the answer already is

Leave-one-out q^2 of a straight line on each term against the adsorption energy. The control is item EC2's: a line on the lowest heavy-atom height of the relaxed pose.

term	benzimidazole	quinoline
burial at the relaxed pose	**+0.638**	**+0.624**
burial and the image term	+0.661	+0.647
control: the relaxed pose's height	+0.472	+0.423
control: the chemisorbed flag	+0.345	+0.328
burial at the **docked** pose	+0.115	+0.049
burial and the image term, docked	+0.108	+0.010

Read the two halves of that table together, because separately each is misleading. Evaluated on the relaxed pose the burial score is a better readout of the relaxation than the height is — q^2 0.64 against 0.47, and 0.62 against 0.42 — and it has the right sign ($r = -0.81$: more burial, stronger

adsorption). Evaluated on a pose the program produces itself, the same score is worth +0.115 and +0.049, which is nothing. The score does not predict the adsorption energy; it reads off how close and how flat the relaxation left the molecule, and it reads that off better than the height does.

The gate is not passed, and it fails on both halves for the same reason. The pose half fails outright. The energy half passes only when it is given the pose it was supposed to find, which is not a prediction. Item EC2 warned that the relaxed frame carries the outcome of the relaxation; this item measures that the burial in that frame carries *more* of the outcome than the height does, which makes the trap worse rather than better: a score of 0.64 looks like a model and is a measurement of the input.

The measured sets: the same terms where nothing is circular

The Fe(110) target came from the same relaxation as the pose, so the first measurement could not separate a score from a readout of its input. The corrosion sets have no such circularity. They hold 23 congeneric series of iron corrosion inhibitors, 997 molecules, with a **measured** potency in acid and, for 830 of them, a **measured** Langmuir adsorption free energy. The slab terms are read at a pose this program produces itself — embedded from SMILES, laid flat by the rule of item EC2, then refined against the slab — and there is no relaxed pose to fall back on.

Six terms go in as a descriptor block: burial, the image energy, the height, and how much of the molecule sits over the top, bridge and hollow sites. The controls are the ones this repository published on these very sets, with the same grids, the same ten folds and the same selection convention, which is worth stating because the agreement is exact: atom-type counts come out at **+0.189**, the value already in `benchmarks/results/corrosion/table.md`.

model	measured potency (23 series)	measured ΔG_ads (18 series)
atom-type counts, no geometry	+0.189	+0.126
the five fields in the rule frame	**+0.279**	**+0.201**
the six slab terms	+0.032	−0.057
the five fields and the slab terms	+0.190	+0.080

The slab terms carry almost nothing on their own and **make the field model worse** when added to it: +0.279 down to +0.190, and +0.201 down to +0.080. They beat the no-geometry control in 7 of the 23 series on the potency and 3 of 18 on ΔG_ads, and they add to the five fields in 5 and 1. So on a measured target, at a pose the program can produce, the slab receptor's terms do not pay. That closes the question the Fe(110) half had to leave open.

One series looks like an exception and should be read with item EC3's lesson in hand: on a 16-molecule series the slab terms alone reach +0.544 and the combination +0.594, against counts at −0.082, with `over_top` correlating |r| 0.74 with the potency. With 23 series tried, one strong hit is what selection produces, and nothing here distinguishes it from chance.

Two smaller findings came out of this stage. The measured ΔG_ads is **harder** than the measured potency for every model, counts included (+0.126 against +0.189), although it is nominally the adsorption quantity and the potency is a rate: whatever is reproducible in these series is not mainly the adsorption free energy. And the site profile, which this item computed and had not used, is the term that most often carries what little signal there is — `over_top` and `over_bridge` appear as the best single term in several series. That is the only encouragement in the table and it is weak.

What was built, and what the slab receptor is good for

The machinery is correct and reusable: the receptor with its images, the image-charge term in closed form, the constrained pose search, and the surface-site profile — top, bridge and hollow sites from a Delaunay triangulation of the top layer, so the three kinds are found for any surface and not only the one lattice the module builds. On iron it says what the surface cannot do, on both a computed and a measured target.

Where it should earn its place is a surface that offers more than shape: an oxide or a passivated metal, where the electrostatic and hydrogen-bond fields are not zero, and physisorption rather than chemisorption, where the image term is the right size rather than a hundred times too small. Neither case has a measured series here, and after this item that is the only open question left in it — the machinery has been measured on everything the repository holds.

5.17 A kinetic selectivity as the activity: a catalyst series in the frame of its reaction centre

A rate constant across a congeneric series is a free-energy difference, as a binding constant is, so the method applies to kinetics under its usual condition: a shared frame, and one mechanism across the series. The program's first measured catalytic series is the screen of Zahrt, Henle, Rose, Wang, Darrow and Denmark (Science 363 (2019) eaau5631): 43 BINOL-derived chiral phosphoric acids, each run on the 25 combinations of 5 N-acyl imines and 5 thiols, 1,075 N,S-acetal formations with their enantiomeric excess. The target is $\Delta\Delta G‡ = RT \ln((1 + ee)/(1 - ee))$ at 298.15 K, the free-energy difference between the two diastereomeric transition states, which is the difference of two rate constants.

The frame is the reaction centre. Every catalyst is the same phosphoric acid on one of two binaphthyl scaffolds with one substituent at both 3 and 3' positions. The builder (`examples/make_denmark_examples.py --fetch`) assembles each catalyst from the published fragments, keeps the lowest of 30 MMFF conformers, gives all 43 the same axial configuration (a 2D file does not carry it, and the conformer generator assigns it at random), and puts the phosphorus at the origin with the acid oxygens on +z, so that the substituents flank the pocket in which the substrates meet. The catalyst has a C2 axis along z; both images are written as the conformers of an ensemble, which gives the field the symmetry of the molecule.

The model is the multi-endpoint model of the selectivity co-field with 25 endpoints: one kernel over the catalysts from the five physico-chemical fields, one covariance between the substrate combinations (a shared correlation of 0.95, chosen by the evidence: the endpoints are mostly the catalyst's own selectivity), and the pair kernel. The benchmark (`benchmarks/run_denmark.py`) holds out each catalyst in turn, each reaction, each substrate combination, each imine and each thiol, and scores every split against the baseline that predicts the same thing without a field, in kcal/mol:

Held out	Field model, MAD (q^2)	Baseline, MAD (q^2)	Published
each catalyst in turn	0.224 (0.82)	endpoint mean, 0.458 (0.17)	0.211 on 19 catalysts held out
reactions, 10-fold	0.148 to 0.164 (0.89 to 0.91)	additive effects, 0.183 (0.87)	0.161 on new substrate combinations
each substrate combination in turn	0.158 (0.90)	additive effects, 0.186 (0.87)	
each thiol in turn	0.406 (0.53)	additive effects, 0.399 (0.54)	

The field model is at the published accuracy on both kinds of split the paper reports. What the paper could not measure, and this program can, is what the frame is worth: on the catalyst holdout the steric field alone gives q^2 0.76, the hydrophobic 0.63, the electrostatic 0.41, and the same five fields compared without a frame, through the rotation-invariant kernel, 0.26. The selectivity is steric, and the frame is most of the result.

Reading the co-field. Averaged over the 25 endpoints and drawn in the acid's frame (`benchmarks/results/denmark/cofields.png`), the steric co-field is positive at the 3,3' ipso carbons on either side of the acid and negative 3 to 5 Å above the phosphorus, over the pocket: bulk that flanks the acid raises the selectivity, bulk that reaches over the pocket lowers it. That is a map a chemist can design against, and it is what the published grid descriptor, a steric occupancy without a learned sign, does not give.

What it does not do. A thiol no catalyst was run on is not predicted, because the substrate side of the model is a one-hot descriptor that knows nothing about an unseen thiol, and the aliphatic and aromatic thiols differ by 0.6 kcal/mol in their means; a field of the substrate aligned on its reacting group is the form that could carry it. The two worst catalysts have a methylene or a silicon between the scaffold and the aryl group, so the group's position is a conformer choice the lowest MMFF structure makes once. And the condition of one mechanism holds here because the screen was designed for it; on a series whose rate-determining step changes, the co-field of the mixture means nothing, and a curved Hammett plot is the warning sign.

5.18 Three tests to run before you build a new field or a new score

The chapter that follows diagnoses a *data set*. This short section diagnoses a *proposal*: a new field, a new kernel, a new score you are about to spend a week on. Three items of this manual each paid for an answer the hard way, and each left a test that costs minutes. They are gathered here because they are the most transferable thing in the book, and because each of them was learned after the expensive version had already been built.

1. Correlate your new kernel with the one the model already has. If the two order the pairs of your series the same way, nothing you do with the new one can change a prediction. The electric-field kernel of the section *The field a reaction centre feels* is exact, physically better motivated than the scalar field it replaces, and correlates **+0.980** with it over the 4371 pairs of the Hammett series. Every model result followed from that number: nothing gained where it was added, accuracy lost where it displaced nothing. One cosine kernel, one correlation, five minutes.

2. Do not read the same test off the best of many columns. A descriptor block is not a kernel, and the maximum correlation over its columns is not a prediction. In *Reading a few places instead of all of them* the best anchored column reaches **$|r| = 0.85$** with a nickel redox potential; the block then gives **$R^2 = -0.88$** under nested cross-validation. The difference is selection: a kernel correlation is one number computed once, the best of 60 columns on 25 molecules is a maximum over 60 tries. Report a best-of-N correlation as a ceiling on what the columns could carry, never as a forecast of what they will.

3. Report a pose-dependent score at a pose your method can produce. If the geometry you score came from the calculation that produced your target, the score can read the answer off its input. In *A metal surface as the receptor* the burial score predicts an adsorption energy with a leave-one-out q^2 of **+0.638** at the relaxed pose — beating the obvious control — and **+0.115** at a pose the program produces itself. The gap is the size of the leak, and the better the score looks at the given pose, the more of the input it is carrying. The same applies to a score used as its own

docking objective: optimising a quantity makes it nearly constant across a series, and a quantity that no longer varies cannot discriminate.

All three are instances of one habit. Before measuring whether a new description predicts, measure whether it *can*: whether it differs from what you have, whether its apparent signal survives the search that found it, and whether its inputs already contain the answer.

6 Before you model: `cmf diagnose`

Every expensive mistake recorded in this manual was visible before the model was built, in a number that takes seconds to compute. The corrosion benzenes were modelled with five fields and an automatic frame, and atom-type counts — no geometry at all — beat them. The ACE set is predicted as well by a handful of two-dimensional descriptors as by the whole 3D model. A thrombin model in the exposure study scored +0.059 with its attenuation pinned to the top of the grid, which is what a mis-specified model looks like from outside. None of these is subtle; they are simply never asked, because asking means running four baselines by hand.

```
cmf diagnose data.sdf --charges file
```

The command answers five questions and prints the numbers behind each.

Is there anything to model? The size of the set and the spread of the activity. Twenty molecules across half a log unit cannot support a conclusion, whatever q^2 comes out, and the command says so and stops.

Can a frame exist at all? The number of chemotypes, the size of the common Bemis–Murcko framework [11], and whether the molecules are actually superimposed. That last test is asked of the shared framework atoms rather than of the centroids, because `EmbedMolecule` leaves every molecule centred on the origin: a set of independently generated conformations has a centroid spread of 0.02 Å and looks perfectly aligned while every molecule points a different way. The test is also allowed to give no answer. A framework that is a small symmetric ring matches a dozen ways and two rings near the origin may coincide by luck, while a framework that is a small part of the molecule may match a *different* copy in each one — measured that way an author-aligned Sutherland set read as 9 Å apart. So a coincidence within half an ångström is a verdict on its own, a large deviation is a verdict only when the framework is a fair share of the molecule, and otherwise the answer is "this cannot tell — look at it".

Does the frame carry signal? The aligned model against the two routes that need no frame: the atom-type count kernel, which has thrown the geometry away entirely, and the rotation-invariant field autocorrelation, which keeps it without a pose. If neither is beaten, the alignment adds nothing and the cheaper route is also the more defensible one.

Does the geometry contribute anything? The same comparison against descriptors alone — what the file carries plus the handful the program computes.

Was the model specified sensibly? Whether the chosen attenuation sits on the edge of its grid, which means the search wanted a value it was not offered.

A fifth route, the topomer rule, is scored where the molecules can be cut at their framework. It covers only those molecules, so the aligned model is refitted on the same ones for the comparison: a subset against the whole set is not a comparison, and it favours whichever route happens to drop the hard compounds.

6.1 Two things the command protects you from

The activity as a descriptor. Most files store the activity as a molecule property, so the descriptors available usually include the quantity being predicted. Given it, the descriptor baseline returns $q^2 = 1.000$ and the advice becomes "use descriptors" for every set in the world. Columns

named as the activity are dropped, and so is anything correlating with it above 0.95 — but the leaked information is rarely a plain copy. The corrosion files carry the potency as `activity`, again as `mwila_pooled_raw`, and again as two laboratory-adjusted variants; excluding the first two still left a descriptor baseline at $q^2 = 0.99$, because the remainder reconstruct it between them. No per-column test can see that, so the outcome is judged as well: a descriptor baseline reaching $q^2 \geq 0.95$ on a set of this size is not chemistry but the measurement itself, and the command says so, names the nearest columns with their correlations, and refuses to recommend descriptors.

Searching harder and calling it a better model. The five routes do not carry the same number of hyper-parameters — the field routes search six attenuations and five ridges, the others search ridges only — so scoring each at the best point of its own grid biases the comparison toward whichever route searched the most. Every route therefore chooses its hyper-parameters inside each training fold and is scored outside it. On ACE this costs the aligned model 0.05 and the topomer route 0.07 while leaving counts, descriptors and the invariant kernel where they were, which is the size and the shape of the bias it removes.

6.2 Reading the answer

Each comparison is reported with a paired bootstrap interval over the molecules, because on twenty to thirty molecules a difference below about 0.1 in q^2 is not a difference. The last line recommends a route, and it should be read as a prompt rather than a verdict: it says which route to take and which question the set has not answered, never that a model is good.

One wording deserves care. When the command recommends `invariant` it often means *the frame is not worth the trouble* rather than *the invariant kernel predicts better* — on DHFR the aligned model is nominally ahead, +0.659 against +0.641, and the recommendation follows the interval rather than the point estimate. And `aligned, with the frame checked` means the frame carries signal and the molecules really are superimposed, so a frame exists whoever made it; several Murcko scaffolds only mean the program could not have built it, not that it is wrong.

6.3 Does the advice work?

A tool that reports five baselines is useful whatever its last line says, but advice that has never been followed is only a rule its author liked. `benchmarks/run_diagnose.py --validate` measures it the only way that matters: hold out 30 % of a set, run the diagnosis on the remaining 70 % alone, build the model it recommends, and score that model on the held-out molecules against every other route — thirteen sets, three splits each.

The advice named the best of the five routes on **21 of 39 splits**, and averaged over all of them the route it chose fell **0.096** in q^2 short of the best available. Always taking the aligned model — the default this command exists to question — names the best route on 12 splits and costs 0.201. Aggregated per set, which is the correct unit because three splits of one set are not three measurements, following the advice costs **0.105 less in q^2 than always aligning** (95 % bootstrap interval −0.259 to +0.019), better on six sets, worse on four, indistinguishable on three.

The shape of that distribution matters more than its mean. The advice is **insurance, not an advantage**: where it loses it loses a little — 0.02 to 0.17 on AChE, BZR, THR and the thermolysin set, all of them well-aligned series where the aligned model was always going to be right — and where it wins it wins a lot, 0.44 to 0.70 on the three corrosion sets, where the automatic frame is meaningless and the aligned model collapses. The interval crosses zero, so on thirteen sets this is not a proven improvement in the average case. What it is, is insurance against the failure described at the start of this chapter, at a price of a few hundredths of q^2.

7 Building a model

7.1 Choosing the fields

The five physico-chemical fields — electrostatic, steric, hydrophobic, H-bond acceptor, H-bond donor — are the default and reproduce the published CMF models. The steric field is special: its width is scaled by the van der Waals radius of each atom, $b_{lf} = \alpha_f / R_l^2$ in Eq. (2.1), rather than being shared.

Continuous indicator fields (CIF) [4] add one field per atom type, with weight 1 on atoms of that type and 0 elsewhere. Tick *Continuous indicator fields* and set the minimum number of training molecules an atom type must appear in (3 by default) to be given a field of its own. Three typing schemes are available: `sybyl` (the types of the original scripts, read from MOL2 or assigned), `element`, and `element+aromatic`. CIFs are the most descriptive field set the program has, and on several of the benchmark sets they match or beat the physico-chemical fields on their own.

Six pharmacophore features instead of twenty atom types (plan item D2). A Sybyl vocabulary describes what an atom *is*; a pharmacophore describes what it is *for*. `--pharm` adds one indicator field per feature — `donor`, `acceptor`, `aromatic`, `hydrophobe`, `cation`, `anion` — taken from RDKit's own feature definitions rather than from SMARTS written here, with weight one on every atom of a matching group: six atoms for a benzene ring, three for a carboxylate, because the character of a carboxylate is not on one of its atoms.

Two things follow. The fields are **dense** — a pharmacophore feature is carried by 70% of the molecules of a set on average, so each has something to say about almost every molecule, where a Sybyl field keyed to `N.am` is silent on whatever lacks an amide. And they **read as a pharmacophore with no translation step**: the co-field of `pharm:donor` is already a map of where a donor is wanted, so *A pharmacophore read out of the co-field* types its features as donors and acceptors rather than as field names.

`--pharm-min N` decides which features are used — one carried by fewer than N training molecules is left out, the rule `--cif-min` applies to atom types — and `--pharm-features donor,anion` names them explicitly. `--fields pharm:donor,steric` works too, for a field set assembled by hand.

What they are worth, and where

Measured on two collections (`benchmarks/run_pharmfields.py`, with and without `--mittal`), each set on its own alignment with the same attenuation and ridge grids and the same ten-fold cross-validation as *Does an alignment help?*. The Sutherland sets cannot answer the question, because the smallest of the eight has 66 molecules and the claim is about series below forty; the 38 Mittal sets [18] can, 15 of them being smaller than that.

sets	how many	median n	five	five + pharm	pharm	five + cif	cif
Mittal, below 40 molecules	15	29	+0.342	**+0.368**	+0.358	+0.344	+0.335
Mittal, 40 and above	23	92	+0.299	+0.305	+0.228	**+0.345**	+0.321
Sutherland (none below 66)	8	112	+0.574	+0.586	+0.538	**+0.597**	+0.580

Below forty molecules the smaller vocabulary wins. The six features add +0.027 to the five physico-chemical fields where the Sybyl indicators add +0.002, and on their own they reach +0.358 — better than the five fields themselves (+0.342) and than the Sybyl indicators (+0.335). **Above forty the order reverses**: the Sybyl indicators add +0.047 against +0.006, and the six features alone fall to +0.228. The Sutherland sets, all of them large, agree with the second half: +0.023 for the Sybyl indicators against +0.012 for the features.

Why, which is not the usual explanation

The explanation usually given is the rarity of individual Sybyl types, and that is not what is happening, because the rare types were already being dropped: a pharmacophore feature is carried by 70% of the molecules and the median Sybyl type by 47%, but `--cif-min` keeps only the types in five molecules or more — 11 of 14 on these sets — and the median coverage of what is kept is 88%.

What does change with set size is how many molecules there are per field to estimate it from:

sets	molecules per Sybyl field	molecules per pharmacophore field
Mittal, below 40	3.4	6.4
Mittal, 40 and above	8.9	20.9
Sutherland	11.8	28.5

At three or four molecules per field a vocabulary cannot be estimated, whatever each of its fields covers; at nine or twenty it can, and then the more descriptive vocabulary wins because it is more descriptive. So the rule to take away is about size, not about chemistry: `--pharm` **below about forty molecules,** `--cif` **above it**, and the six features whenever the model is going to be read as a pharmacophore, where they cost 0.040 q^2 on a large set and save the translation.

`prop:<name>` fields are added by naming them in the per-atom property row of the model panel, or in `--fields`.

7.2 GRID-like probe fields

Goodford's GRID describes a molecule by the energy a small probe — a water, a methyl, a carbonyl oxygen — would feel at every point around it. That is a field in exactly this program's sense, and it says something the built-in fields do not: not "there is hydrophobic character here" but "a probe placed here would be bound". *Probe fields* in the model panel (`--probes water,carbonyl`) adds one such field per probe. Nine are defined:

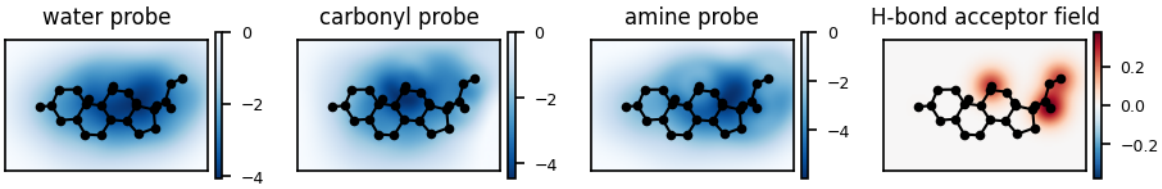
name	what it stands for	R (Å)	well (kcal/mol)	charge	hydrogen bonding
`water`	water, OH_2	1.70	0.15	0	both, weakly
`methyl`	a methyl group	2.00	0.10	0	neither
`carbonyl`	a carbonyl oxygen	1.60	0.20	−0.40	accepts
`amine`	an amine nitrogen	1.75	0.16	+0.40	donates
`cation`	a generic +1 cation	1.50	0.10	+1.00	neither

name	what it stands for	R (Å)	well (kcal/mol)	charge	hydrogen bonding
`lithium`	a lithium ion, Li^+	0.49	0.10	+1.00	neither
`sodium`	a sodium ion, Na^+	0.75	0.10	+1.00	neither
`potassium`	a potassium ion, K^+	1.11	0.10	+1.00	neither
`magnesium`	a magnesium ion, Mg^{2+}	0.45	0.10	+2.00	neither

Those four numbers are the whole definition of a probe: the radius and well depth enter the dispersion term through the usual combining rules, the charge the electrostatic term, and the hydrogen-bond character decides which atoms it is complementary to — a donor probe feels acceptor atoms and not other donors. The parameters sit in the usual GRID and Tripos ranges rather than being taken from a published GRID table, which matters if you ever want to compare numbers with GRID itself rather than shapes. Adding a probe is one entry in `PROBES` in probes.py; the fitting, the kernel, the drawing and both interfaces read that dictionary, so a new one appears everywhere at once.

The four ion probes, and how little separates them. An ion carries no hydrogen bond, and its dispersion well is small beside its charge: at the contact distance of a lithium ion and an ether oxygen the electrostatic term is about −7 kcal/mol and the dispersion term −0.14. So an ion probe field is the molecule's electrostatic potential, sampled at one distance, and the three singly charged probes differ in that distance alone. The distance is not a guess: the lithium radius of 0.49 Å is the one that puts the probe's contact with an oxygen at 2.01 Å, which is where the oxygens sit in the 1,050 relaxed Li^+–ether complexes of `examples/ether/`. The sodium and potassium radii add Shannon's six-coordinate differences, 0.26 Å and 0.62 Å, because no relaxed complexes of those ions are available to measure. The magnesium probe differs by its charge of +2 as well, which multiplies the field by two; every kernel here is normalised by the self-overlap, so a constant factor is invisible to it, and the magnesium probe is in practice the lithium probe at a slightly shorter distance. A probe cannot represent polarisation, and the difference between a hard and a soft cation of the same size and charge is polarisation.

A probe field and the alignment-free kernel. A probe field puts a short sum of Gaussians of different widths on every atom instead of one number, so its atom weight column is empty. The rotation-invariant kernel of `cmfqsar/invariant.py` therefore expands each term of that sum into a channel of its own, which is exact, because the field is the sum of the terms. Before 4 November 2026 it did not: a probe field passed to that kernel gave a matrix of zeros, which says that every molecule is alike, and nothing refused it. `feature_matrix` now needs the field set whenever a probe field is present and refuses without it.



Three probe fields of cortisol read 2 Å above the mean plane of the steroid series, where a probe can sit, beside the built-in hydrogen-bond acceptor field read at the same height. A probe field is what the probe would gain at each point and is never positive, so its scale runs from the deepest value to zero; the water and the amine probe gain most over the hydroxyl and carbonyl groups, the carbonyl probe where there is a donor to meet, and all three gain something over the whole skeleton from dispersion, which the built-in field, a sum of atom-centred Gaussians, does not carry.

The obstacle is arithmetic, not conceptual: a Lennard-Jones term is not a Gaussian, has no closed-form overlap, and diverges at the atom, so the integral of its square does not exist. The solution is to fit each atom's contribution by a few Gaussians, which keeps every kernel in the program a closed form over atom pairs. Three things had to be measured before that worked, and each one shaped what is implemented:

- **The repulsive wall cannot be represented.** Fitting a curve that is +30 kcal/mol at 2.5 Å and −0.13 at 3.5 Å with a handful of Gaussians produces coefficients of $\pm 10^{7}$ that cancel, and a worst-case error larger than the well it is meant to represent. The wall is a truncation artefact in every grid implementation anyway, and this program already includes it — the steric field *is* a Gaussian van der Waals envelope — so a probe field carries the **attraction** only.
- **A well at a radius is a shell**, and a shell is not a sum of Gaussians centred on the atom. Each term is therefore capped at its own optimum, which makes it monotone in r, and those a sum of Gaussians represents naturally.
- **The r^{-6} tail is not Gaussian either**, and is negligible beyond 6 Å, so the energy is tapered to zero between 5 and 7 Å exactly as GRID's own cut-off does.

What comes out reproduces the attraction to about 15 % in the volume-weighted norm the kernel sees (23 % on a realistic worst case), and the kernel it implies agrees with numerical integration of the fields themselves to $4 \cdot 10^{-7}$ — a test asserts both. Each atom carries K Gaussians (four by default), so a probe field costs K^2 passes over the distance matrix where an ordinary field costs one; three is the fast setting and five the accurate one.

Measured against the fields it would join, same molecules, same α and λ grids, 10-fold CV:

set	n	the five fields	+ water probe	probes only
GPB	66	+0.539	+0.539	+0.406
THER	76	+0.538	+0.543	+0.428
THR	88	+0.672	**+0.699**	+0.665
ACE	114	+0.647	+0.648	+0.607

So: adding a water probe to the five fields adds between nothing and 0.027 to q^2, and probe fields on their own are worse than the fields they would replace. That is the measured result and it was the expected one — CoMSIA replaced GRID-like energies with Gaussian similarity indices for much

the same reason, and this program's fields are closer to CoMSIA than to GRID by design. What a probe field is actually *for* is interpretation: its co-field is drawn, exported and decomposed like any other, and it says where a water or a carbonyl oxygen would be bound, which is a statement that is easier to act on than "more acceptor character here".

Two limitations are worth stating plainly. The directional part of GRID's hydrogen-bond term — the angles at the donor and at the acceptor's lone pairs — is **not** represented; the radial part is, broadened to 0.6 Å because a sharper shoulder is not representable by the shared basis. And alignment by field similarity refuses to run on a probe field rather than silently ignoring it, because the gradient of the expansion is not implemented.

7.3 Weighting the fields by solvent exposure

A kernel that integrates over all space weights an atom buried inside a molecule as heavily as one on the outside, although a binding partner can only touch the second. That is the claim the surface methods are built on, and `cmfqsar/exposure.py` tests it in the simplest way: every atomic weight multiplied by its atom's relative solvent accessibility, which keeps the kernel in closed form and keeps the co-field.

It does not help, and on the larger sets it slightly harms the model. Across sixteen FEP sets the median paired gain is −0.002 in q^2 and the weighting wins on eight of sixteen — no better than chance — while the one large win is a model whose attenuation was fixed at the edge of the grid. On the eight Sutherland sets every variant is worse, and the softened one that looked best on the FEP sets has an interval entirely below zero. The effect also shrinks as the sets grow: on DHFR (361) and COX2 (282) the change is −0.004 and +0.003, and the movers are the two smallest sets. An effect that vanishes with more data is noise.

The reason is geometric and settles the general question. Measured by group — a heavy atom and its hydrogens counted as one, because atom by atom a methyl carbon is 93 % occluded by its own hydrogens and ethanol reads as two-thirds buried — the median group exposure is 0.37 to 0.42 and **0 to 3 % of groups are buried** in either family. A ligand of thirty heavy atoms has no interior. Surface-based representations are solving a problem that molecules of this size do not have.

7.4 Descriptors that are not fields

Everything above describes a molecule as a function of space. Some things are not functions of space and never will be: a measured logP, a molecular weight, the medium or temperature a series was recorded in, a count of rotatable bonds, an energy from a quantum calculation. A model that cannot see them has to pretend the activity depends on geometry alone, which for an adsorption potency or a permeability is simply false.

The *Descriptors (per molecule)* row of the model panel (`--descriptors A,B`) offers every numeric molecule property the file carries — and hides itself when the file carries none besides the activity, which is why it is invisible on most of the example sets; the computed descriptors of the next section are always there. Ticking some adds one more block to the kernel:

$$K(A,B) = \sum_f h_f c_f \hat{K}_f(A,B) + h_d c_d \, \mathbf{z}(A) \cdot \mathbf{z}(B) \tag{7.1}$$

where **z** holds the descriptors of a molecule, standardised with the **training set's** mean and standard deviation so that a quantity in kilojoules cannot outweigh one in log units, and h_d is a mixing coefficient like a field's (the *h* box beside the row, `--descriptor-h`; zero switches the block

off without removing it).

Three properties of the block are deliberate:

- **It is scaled, not cosine-normalised.** Dividing by the two norms, as the fields are divided, would send a molecule sitting at the training mean in every descriptor — an unremarkable molecule — to a zero row and a zero diagonal, out of the model altogether. Instead the whole block is multiplied by a constant that puts its mean diagonal at 1, so ten descriptors do not outweigh the geometry merely by being ten.
- **The statistics belong to the training set** and are stored in the saved model. Standardising a test molecule by its own set would score two test sets on two different scales, which is the mistake the field scaling avoids as well.
- **The model stays linear in its inputs**, so the decomposition rules survive. A prediction splits into a field part and a descriptor part, and the descriptor part splits further into one number per descriptor, which is what the program reports.

A molecule missing a descriptor is refused by default (`prop_missing`), because a model that silently substitutes the mean is telling you nothing about that molecule while looking as confident as ever; with the `mean` policy it is substituted and the molecule simply contributes nothing through that descriptor. A descriptor that is constant over the training set is reported as inert rather than dividing by a spread of zero.

The obvious danger, stated plainly. A descriptor derived from the activity — the activity itself, a transform of it, a value fitted to it — will dominate the model and make the geometry look worthless, while q^2 goes up. The program cannot detect this, so it reports the mean absolute contribution of every descriptor beside the model statistics:

```
Non-field descriptors (mean |contribution| per training molecule):
  clogp                        0.0779
  mw                           0.0176
  tpsa                         0.0123
```

If one descriptor carries most of the prediction, the model is really a descriptor model with a 3D appearance, and it should be reported as one.

```
python cmf_cli.py build series.sdf --activity pIC50 --descriptors clogp,mw,tpsa
```

Descriptors the program computes itself

A file that carries no numeric tags has nothing to offer the row above, so four descriptors are always available, computed by RDKit from the structure: `calc:mw` (molecular weight), `calc:clogp` (Wildman–Crippen logP), `calc:tpsa` (topological polar surface area) and `calc:rotatable` (rotatable bonds). They have a row of their own in the model panel, *Computed descriptors*, and **all four are off by default**.

Those four are shortcuts, not the whole list. **Any** of RDKit's descriptors — some two hundred of them — can be asked for by its own RDKit name, typed into the box at the end of that row (with completion) or passed on the command line: `calc:NumHAcceptors`, `calc:FractionCSP3`, `calc:BalabanJ`, `calc:LabuteASA`, `calc:qed`. Names are matched case-insensitively, and a name that does not exist is refused before the fit with the nearest ones that do — `calc:NumHDonor` answers *did you mean NumHDonors?*.

The default is deliberate, and so is the warning that belongs with it. These are two-dimensional quantities: they can improve a model that has a size or lipophilicity trend in it, and they can also carry a model that then gains nothing at all from its geometry. Measured on four Sutherland sets, same α and λ grids, 10-fold CV:

set	n	five fields	+ the four computed	the four alone
THR	88	**+0.671**	+0.634	+0.620
GPB	66	**+0.534**	+0.525	+0.494
ACE	114	+0.657	+0.707	**+0.712**
THER	76	**+0.509**	+0.496	+0.480

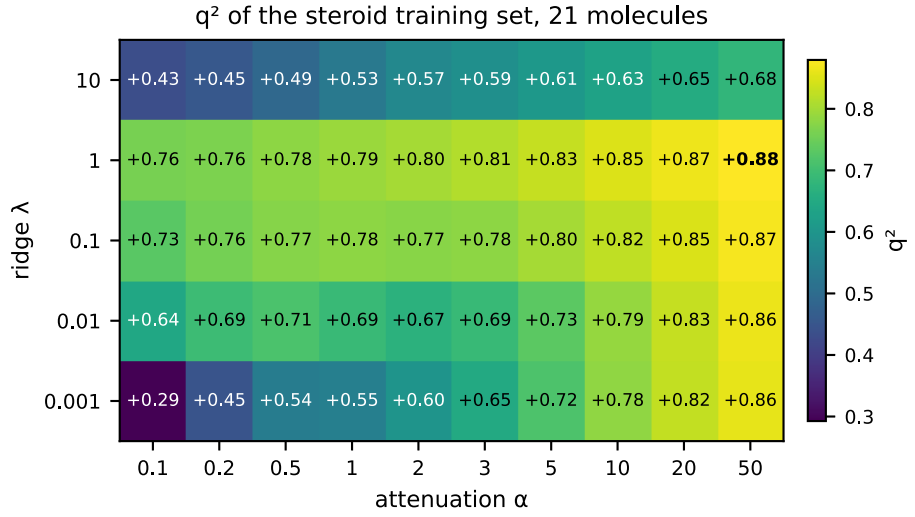
Read ACE carefully: four numbers that know nothing of the geometry predict better than the whole 3D model. That is not an argument for using them — it is a warning about that data set, and it agrees with what the alignment-free baseline already said about ACE elsewhere in this manual. On the other three sets they lower q² slightly.

So use them to *test* a model rather than to improve it: if adding them changes nothing, the geometry is carrying the model; if they take over, the 3D part was contributing nothing. The contribution table printed beside the statistics is what tells you which happened.

7.5 Attenuation, regularisation and the grids

Two hyper-parameters matter. α sets how far each field reaches: large α makes the fields sharp, so molecules must match closely, while small α makes them diffuse. λ (or *C*, ε, ν for the SVM learners) sets the regularisation. Both are chosen by cross-validated grid search, and the whole grid is shown in the *Hyper-parameter search* tab — look at it, because an optimum at the edge of the grid means the grid was too small.

```
python cmf_cli.py build data.sdf --alphas 0.1,0.3,0.5,1,2,5 --lams 1e-3,1e-2,0.1,1,10
```



The cross-validated q² of the steroid training set, 21 molecules, over a grid of attenuation and ridge; the best point is in bold. The surface is flat along α: from α = 5 on, every doubling of α adds one or two hundredths, so the best point sits on the edge of the grid and a wider grid would move it further without changing the model much. The ridge decides more: the lowest row is what too little regularisation costs.

Refinements, all optional: continuous Brent minimisation over λ as in the original R scripts (`--lam-search brent`), a Nelder–Mead search over α (`--alpha-search nelder-mead`), coordinate descent or centred kernel alignment for the mixing coefficients `h_f` (`--optimize-h`, `--mkl`), and a grid over the bandwidth of a nonlinear kernel layer (`--gammas`).

Two conventions decide how the field kernels are combined, and they change the numbers: *Normalise kernels* uses cosine-normalised field kernels, and *Scale fields* rescales each field kernel to unit mean diagonal before mixing. Untick the second to get the raw-kernel convention of the papers (`--raw-scale`).

7.6 How long the kernel takes, and the three settings that change it

A field kernel is a sum over atom pairs, so its cost is the number of atom pairs: the square of the set size times the square of the molecule size. A 200-molecule congeneric series costs under a second per attenuation and nothing has to be done about it. A library, a set of long molecules, or a grid search over six attenuations is a different matter, and three things were measured and changed (`benchmarks/run_speed.py` records all of them).

The training kernel is computed once rather than twice. $K_f(i,j) = K_f(j,i)$, so for a set against itself only the pairs with $j \geq i$ are evaluated and then mirrored. This is the largest of the three, because it removes half of the work rather than part of it, and it is exact; it also makes the matrix exactly symmetric, which it was not before, the two triangles having differed by round-off from summing the atom pairs in two different orders.

The radius-scaled prefactor is a square root rather than a power. The steric field gives every atom its own width, so $(2\pi/(b_l+b_m))^{3/2}$ differs for every atom pair, and a general power is a library call per element. Writing it as $z\sqrt{z}$ is the same number to fourteen digits and costs a third as much. Before this change that one expression was half the time of the whole kernel.

The distances are shared across the attenuation grid. The attenuation enters only through the exponential, so a grid search can compute each molecule's squared distances once and take six exponentials from them. This is the smallest of the three: the distance matrix is 3 % of a block.

Measured on 200 molecules of a congeneric series and 60 decapeptides, all six attenuations of the usual grid (0.1, 0.3, 0.5, 1, 2, 5):

shortcut	COX2 (40.8 atoms each)	AMPH1 (168.2 atoms each)	still agrees to
as before	14.3 s (1.0x)	42.2 s (1.0x)	--
+ square root	10.5 s (1.4x)	38.7 s (1.1x)	3e-16
+ symmetry	4.6 s (3.1x)	16.0 s (2.6x)	1e-15
+ shared distances	3.8 s (3.7x)	15.3 s (2.8x)	1e-15

The last column is the point: every shortcut is exact, and what is left is round-off from summing the atom pairs in a different order. End to end, a `grid_search` over the same six attenuations with five-fold cross-validation:

shortcut	seconds	speed-up	alpha chosen	q²cv
as before	13.8	1.00x	0.1	+0.504175
+ square root	9.9	1.40x	0.1	+0.504175

shortcut	seconds	speed-up	alpha chosen	q²cv
+ symmetry	5.0	2.76x	0.1	+0.504175
+ shared distances	4.1	3.40x	0.1	+0.504175

3.4 times faster, and the attenuation chosen and the cross-validated q² are identical to six decimal places.

Nothing here changes a number the program reports; the benchmark prints the chosen α and the cross-validated q² beside every timing for exactly that reason.

The cutoff, which is off unless you ask

A Gaussian overlap falls below a fraction τ of its peak beyond a cutoff radius of $\sqrt{(-4 \ln \tau / b)}$, which at $\tau = 10^{-6}$ is 10.5 Å for a shared-width field at α = 0.5 and 19 Å for the radius-scaled steric field. Atom pairs further apart than that can be dropped, and `--cutoff auto` does so, deciding per field by sampling the real distances and building a neighbour list only when most pairs would go.

Whether this helps is a property of the molecules, not of the program:

set	atoms per molecule	α	pairs kept, shared width	pairs kept, steric	exact sum	`--cutoff auto`	what auto did
COX2	40.8	0.5	98%	100%	0.95 s	0.85 s (1.12x)	kept the exact sum
COX2	40.8	2	56%	96%	0.90 s	0.91 s (0.99x)	kept the exact sum
COX2	40.8	5	24%	67%	0.84 s	0.82 s (1.02x)	kept the exact sum
AMPH1	168.2	0.5	62%	99%	2.61 s	2.64 s (0.99x)	kept the exact sum
AMPH1	168.2	2	14%	52%	2.75 s	2.15 s (1.28x)	used the neighbour list
AMPH1	168.2	5	4%	19%	2.64 s	1.21 s (2.18x)	used the neighbour list

The two columns of surviving pairs explain the rest. The sampling that decides costs about a hundredth of a block, so `auto` is free when it declines, and on these sets it declines wherever the dense sum is faster. Forcing the list on with `--cutoff 10` or `--cutoff auto` where it does not pay can cost a factor of five, which is why it is not the default.

On a congeneric series superimposed in one frame the cutoff keeps almost every pair and `auto` refuses to build a list, so the sum stays exact. On molecules five times longer it is worth a factor at a strong attenuation. A cutoff makes the kernel an approximation — each dropped pair contributed less than a millionth of a peak overlap, and the measured error is about 10^{-7} relative — so it is off by default and every table in this manual was produced without it.

The compiled kernel, if Numba is installed

The kernel above is NumPy, and NumPy is single-threaded. The same arithmetic written as one fused loop over the atom pairs and compiled with Numba can use the other cores: `cmfqsar/fastkernel.py` does that, threaded over the molecules, which is safe because every molecule writes its own row of the kernel and nothing is shared.

Two things are worth knowing before reaching for it.

Per core it is slower, not faster. `np.exp` on an array is vectorised; the same calls written one at a time in a compiled loop are not, unless Intel's SVML is installed. Measured serially on 200 molecules, the NumPy path takes 0.67 s and the compiled loop 0.90 s. All of the advantage below is the threads, and on a machine with two cores there will be almost none.

It is not the default. Install it with `pip install numba` or `pip install -e .[fast]`, and ask for it with `--engine numba` (or `--engine auto`, which uses it where it is installed and falls back silently where it is not). The default everywhere except `screen` is `numpy`, because the two paths differ by about 2e-14 relative — the compiled loop is built with `fastmath` and may add the pairs in another order — and every table in this manual was produced with NumPy. That difference is the same order as the round-off between two orderings of the NumPy sum itself, but a default that depends on whether an optional package is installed is not something a table should rest on.

What it is worth, on 32 cores:

set	atoms per molecule	NumPy	compiled	speed-up	differs by
COX2	40.8	0.50 s	0.09 s	5.6x	4e-15
AMPH1	168.2	1.99 s	0.12 s	16.6x	2e-14

The long molecules gain more because each thread then has more work to amortise the cost of being started. End to end it is less, because the kernel stops being all of the time: a whole grid search over six attenuations falls from 2.9 s to 0.7 s (4.0 times), choosing the same attenuation and the same cross-validated q^2 to six decimals, and a screening run over 4166 molecules from 46 s to 12 s (3.8 times, 91 to 347 molecules a second) — where reading the file and building the fields are now the larger half of the work.

The compiled loop also takes a cutoff without a neighbour list, since inside the loop a cutoff is one comparison per pair; `screen --cutoff R --engine numba` is therefore worth trying on long molecules, where the NumPy neighbour list of the previous section is also worth building.

7.7 The learners

Task	Methods	Key
Regression	kernel ridge regression with bias; Gaussian process (the same fit plus predictive variances); kernel partial least squares (CoMFA's learner, regularised by its number of components); ε-SVR; ν-SVR; the ranking SVM, which fits the order and not the values; the Laplacian-regularised ridge, which adds molecules that have no measured activity to the fit	`krr`, `gp`, `kpls`, `svr`, `nusvr`, `ranksvm`, `lapkrr`

Task	Methods	Key
Binary classification	C-SVC; ν-SVC; Gaussian-process classifier by the Laplace approximation; kernel Fisher discriminant analysis, the only classifier here whose score is linear in the field and which therefore has a co-field; kernel logistic regression, the same Laplace mode with its ridge chosen on the folds and no posterior variance	`svc`, `nusvc`, `gpc`, `kfda`, `klr`
One-class / screening	1-SVM; distance to the centre of the actives; distance to the k-th nearest active; one-class Gaussian process; distance from the principal subspace (kernel PCA)	`ocsvm`, `ocmean`, `ocknn`, `ocgp`, `ockpca`

Kernel ridge regression is the method of the papers and the default. The Gaussian-process view adds a posterior variance per prediction at no extra cost, which is what the confidence-based alignment of *Preparing the data* minimises. The Laplace GP classifier adds calibrated probabilities, a posterior variance and model selection by marginal likelihood — it scores no better than C-SVC on the benchmark sets (mean difference 0.001 in Matthews correlation), so it is not the default, but its probabilities are calibrated and its evidence picks hyper-parameters without holding anything out.

7.8 Kernel partial least squares: CoMFA's learner on the continuous field

CoMFA fits its grid values by partial least squares (PLS). Kernel PLS is the same method written in terms of the kernel (Rosipal and Trejo 2001). With the linear CMF kernel it is PLS in the space of the fields: every component is a direction in that space, and the regression coefficients are a field. So `--method kpls` is CoMFA's learner with the grid replaced by the continuous field, and it is the most direct comparison between CoMFA and CMF on one alignment, because the two then differ in the representation alone.

How it works. The kernel is centred, which subtracts the mean field of the training set, and the activity is centred. Each component is the direction of field space that covaries most with the current activity residual: its score vector is $t = K_c\, u$, normalised, where u is that residual. The kernel and the residual are then deflated by the component, and the next one is taken from what remains. After m components the dual coefficients are

$$\mathbf{a} = \mathbf{U}\,(\mathbf{T}^{\mathrm{T}}\mathbf{K}_c\,\mathbf{U})^{-1}\,\mathbf{T}^{\mathrm{T}}\mathbf{y} \tag{7.2}$$

with T and U the matrices of score and residual vectors. The centring is folded into **a** and a bias b, so the model predicts $K(x, \text{train})\ \mathbf{a} + b$ like every other learner here, and **a** sums to zero as the coefficients of kernel ridge regression with a bias term do. Everything that reads a kernel expansion keeps working: the co-fields, the atom and molecule contributions, the Shapley values, the pharmacophore and the applicability domain. Only the posterior variance of the Gaussian-process view belongs to ridge regression and is not available.

The one setting. Kernel PLS has no λ. Its regularisation is the number of components: a few components keep a few strong directions of the field space and drop the rest; all of them (one less

than the number of training molecules) reproduce the training activities exactly, as ridge regression does when λ goes to zero. `--pls-components` lists the counts tried (the default is 1 to 12) and the count with the lowest RMSEcv is kept, inside every outer fold of the external cross-validation, exactly as λ is for kernel ridge regression. The curve of RMSEcv against the count is often flat over several values; the search keeps the minimum, and the search table of the JSON output shows how flat it was. `--no-bias` has no effect on this learner: the centring is the bias.

```
python cmf_cli.py build data.sdf --charges file --method kpls --pls-components 1,2,3,4,5,6,8,10
```

Measured against kernel ridge regression

`benchmarks/run_kpls.py` runs both learners on the eight Sutherland sets under one protocol: the same fields, alignment, attenuation grid and folds, each learner choosing its own setting on the same 10-fold RMSEcv, with that choice repeated inside every outer fold of the external cross-validation. The attenuation α is chosen on the training set, so q^2 and R^2pred carry that choice too; q^2ex is the fair comparison.

Mean over the eight sets	kernel ridge regression	kernel PLS	sets where kernel PLS is ahead
q^2 (10-fold, training set)	0.625	0.619	4 of 8
R^2pred (test set)	0.443	0.357	0 of 8
q^2ex (external 5-fold, whole set)	0.530	0.502	1 of 8

The two tie on what was used to choose, and ridge wins on what was not. On the cross-validated q^2 of the training set kernel PLS is ahead on four sets and behind on four. On the external cross-validation it is behind by 0.03 on average, on seven of the eight sets, and on the test sets by 0.09, on every set. The gap is not in what the learner can fit but in how it regularises. The ridge shrinks every direction of the field space by a smooth amount, and a λ chosen on one subset carries over to another. PLS keeps a few directions whole and drops the rest; the count it chose here is large (4 to 11 components, where CoMFA on a grid usually takes 2 to 6), each count is a step rather than a dial, and the count chosen together with the attenuation carries over less well. The thrombin set shows the failure: kernel PLS chose a sharp attenuation (α = 0.05, 7 components) that gave the best q^2 of any model on that set and an R^2pred of 0.07, against 0.42 for ridge. Kernel PLS also costs about one and a half times the time of ridge, because every count is a separate fit.

So the measurement supports the choice of [2]: kernel ridge regression stays the default. `--method kpls` is there for what it is. It is CoMFA's learner on the continuous field, and so the right comparator when the question is what the continuous representation gains over the grid, with the learner held fixed; and it is a second opinion whose components, like CoMFA's, can be plotted and counted. The full table, per set and with the settings chosen, is `benchmarks/results/kpls/table.md`.

7.9 A classifier with a co-field: kernel Fisher discriminant analysis

A co-field exists only where the model is linear in the field. That rule leaves the program's classifiers in an awkward position. The Laplace Gaussian-process classifier passes it, because its latent function is a kernel expansion. C-SVC does not, in the sense that matters: its decision function is a kernel expansion too, but the coefficients come out of a quadratic program on the hinge loss, so they are not a linear map of the class labels and the uncertainty of the co-field cannot be written

down. Kernel Fisher discriminant analysis (`--method kfda`) is the second classifier that passes the rule, and it passes it in closed form.

What it finds. The direction in field space along which the two classes are furthest apart relative to their own spread — Fisher's discriminant, written on the kernel (Mika et al. 1999). In the dual it is one linear solve,

$$\mathbf{a} = (\mathbf{N} + \lambda t \mathbf{I})^{-1}(\mathbf{m}_+ - \mathbf{m}_-), \tag{7.3}$$

where $\mathbf{m}_c$ is the mean kernel row of class c, $\mathbf{N} = \mathbf{KWK}$ is the within-class scatter written on the kernel, $\mathbf{W}$ is the projector that centres each class on its own mean, and $t = \mathrm{tr}(\mathbf{N})/n$. The prediction is $K(x,\mathrm{train})\mathbf{a} + b$, the same expression as every other learner here, so the co-field, the atom and molecule contributions, the Shapley values and the applicability domain read a fitted classifier with no special case, and `cmf cofield` draws its map from a saved model file.

What λ is for. $\mathbf{N}$ has rank at most $n - 2$, so without the ridge the solve has no unique answer at all: the regularisation is what makes the direction defined. It also bounds the length of the discriminant, which is the usual control of overfitting, since the quotient being maximised can otherwise be driven up along directions that the training molecules happen to span and new molecules do not. λ is **relative**: the ridge added is λ times the mean diagonal of $\mathbf{N}$, so the same λ grid means the same amount of regularisation whatever the size of the kernel, exactly as the λ of kernel ridge regression does on a kernel scaled to unit mean diagonal. `--lams` supplies the grid and the value with the best cross-validated AUC is kept.

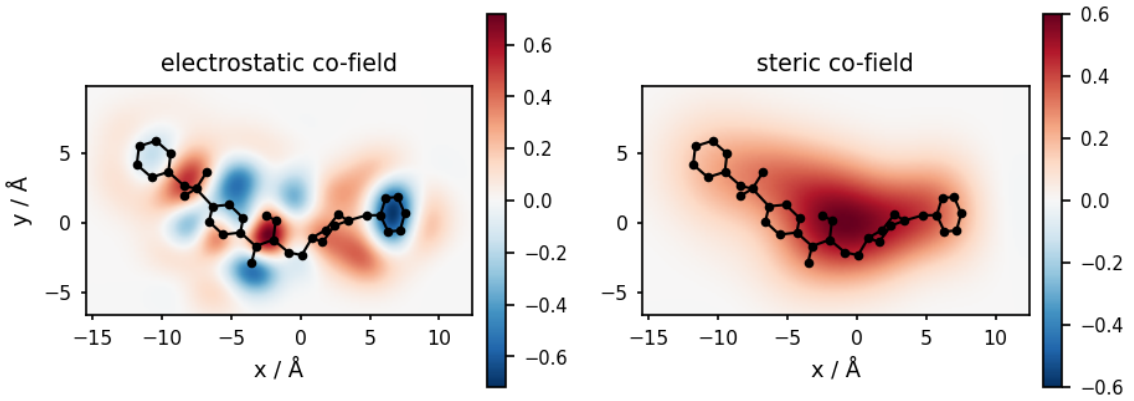
The length and the threshold. A discriminant direction is defined only up to its length, and Fisher's quotient says nothing about where to cut it. The program fixes both. The coefficients are scaled so that the within-class standard deviation of the training scores is 1, which makes the score readable and, more importantly, makes two fits on different numbers of molecules comparable. The cut is then chosen on out-of-fold scores: the training molecules are split into five folds, each fold scored by a discriminant fitted on the other four, every fold's scores expressed in that fold's own class-mean units, and the cut with the best balanced accuracy on the pooled scores is kept as the bias. Balanced accuracy, not accuracy, because a screening set with one active in twenty makes "inactive" the right answer for accuracy whatever the model says.

Probabilities. The same out-of-fold scores give a one-dimensional logistic map from score to probability — Platt's recipe [22], with his smoothed targets so that a set the score separates completely gives a finite slope. `predict_proba` and the *Scores* tab report it. Two cautions. The probability is a frequency read off cross-validation, with no term for how far a molecule lies from the training set, so unlike the Gaussian-process classifier it does not return towards one half for a molecule the model has never seen anything like. And `p ≥ 0.5` is **not** the same cut as the class the model reports: p = 0.5 is the score at which actives and inactives were equally frequent, while the class comes from the balanced-accuracy threshold, which weighs the two classes equally and on an imbalanced set sits lower. Read the probability as a frequency and the class as a decision.

Its relation to kernel ridge regression. As λ goes to zero the discriminant score becomes exactly the kernel ridge regression fitted to the labels n/n_+ and $-n/n_-$, up to a positive factor and a constant. That is a textbook identity, and it is worth knowing what it implies: the discriminant is not a different kind of model from a ridge on class labels, it is the same model with the labels weighted by class size and the ridge chosen for separation instead of for squared error. The gain it can offer over a plain ridge is therefore bounded, and the measurement below shows what it actually is.

```
python cmf_cli.py build examples/sutherland/AChE.sdf --charges file --task classification \
       --threshold 6.84 --method kfda --lams 0.01,0.03,0.1,0.3,1,3 --normalize
```

kernel Fisher discriminant of AChE: 56 actives against 55 inactives, λ = 0.01, cross-validated AUC 0.79



The co-field of the kernel Fisher discriminant on the AChE set of the command above, 56 actives above pIC50 6.84 against 55 inactives, with the charges of the file, α = 0.5 and λ = 0.01, with the most active ligand drawn over it. Red is field that moves a molecule towards the actives, blue towards the inactives: the electrostatic map alternates in sign along the chain between the two aromatic ends, and the steric map asks for bulk around its middle. Because the discriminant score is linear in the field, the map is read exactly as a regression co-field.

Measured against the other classifiers

`benchmarks/run_kfda.py` puts four learners on one kernel and one set of folds, on the two LIT-PCBA classification sets — measured actives against measured inactives, 1587 and 1564 molecules with 88 and 64 actives. The four are the discriminant, C-SVC, the Gaussian-process classifier and kernel ridge regression on the labels +1 and −1, which is the simplest learner that can be put to this task and which the discriminant is nearly a special case of. Each chooses its own setting inside every fold by cross-validated AUC.

Balanced accuracy is given twice, and the difference between the two columns is the whole result. *Own boundary* leaves every learner's decision boundary where the learner puts it. *Common rule* gives all four the same boundary, chosen out of fold by balanced accuracy.

Mean of the two sets	discriminant	C-SVC	GP classifier	ridge on ±1 labels
balanced accuracy, own boundary	0.582	0.515	0.500	0.495
balanced accuracy, common rule	0.529	0.517	0.525	0.529
AUC	0.605	0.607	0.567	0.600
Brier score	0.0457	—	0.0707	—
Brier score of the share of actives alone	0.0458			

Differences against the discriminant, with 95 % paired bootstrap intervals over the molecules, averaged over the two sets:

Difference, discriminant minus	balanced accuracy, own boundary	balanced accuracy, common rule	AUC
C-SVC	+0.067 [+0.005, +0.128]	+0.012 [−0.047, +0.070]	−0.003 [−0.071, +0.065]
GP classifier	+0.082 [+0.023, +0.140]	+0.003 [−0.037, +0.045]	+0.038 [−0.032, +0.108]
ridge on ±1 labels	+0.087 [+0.029, +0.143]	−0.001 [−0.066, +0.067]	+0.005 [−0.051, +0.063]

The discriminant does not order molecules better. Every AUC difference contains zero, against all three references, on both sets. The ordering is the kernel's, and the kernel is the same for all four.

What it does do is place its threshold. With each learner's own boundary the discriminant is ahead by 0.067 to 0.087 balanced accuracy and those intervals exclude zero — but that is not a better model. It is that the discriminant is the only one of the four that chooses a boundary at all: the others leave theirs where their fit put it, which on a set with one active in twenty calls nearly every molecule inactive and scores 0.50 whatever the ordering behind it. Give the same out-of-fold rule to all four and the advantage disappears: +0.012, +0.003 and −0.001, every interval containing zero. If you use C-SVC or the Gaussian-process classifier on an imbalanced set, move the threshold yourself; the discriminant is simply the method that does it for you.

Calibration. The discriminant's Brier score is better than the Gaussian-process classifier's on both sets, by 0.025 on average with an interval of [−0.029, −0.021] that excludes zero. Read that against the right reference. Predicting the share of actives for every molecule and saying nothing else scores 0.0524 on ESR1_ant and 0.0392 on TP53; the discriminant scores 0.0520 and 0.0394, which is that reference to three decimal places, and the Gaussian-process classifier scores 0.0630 and 0.0784, which is worse than it. So the discriminant is better calibrated only in the sense of being less overconfident. Neither classifier's probabilities carry usable information on these sets, and the comparison is not a reason to prefer either.

The co-field. Both classifiers fitted on all the molecules of a set, their co-fields compared by the cosine of *Comparing two models by their co-fields*: 0.681 on ESR1_ant and 0.233 on TP53, positive in every field on both sets. The two classifiers ask the chemistry for the same thing, clearly on one set and weakly on the other. Only the discriminant's map, however, has the standing of a co-field in the sense this manual uses the word.

Where to cut the score, measured. The threshold rule above is one of two. The other is the plain midpoint of the two projected class means, the Fisher threshold for equal class priors, which fits nothing. On eight balanced two-class problems made from the Sutherland sets by cutting each set at its median activity, the two rules give mean balanced accuracies of 0.770 and 0.773, a difference of −0.003 whose interval contains zero on seven of the eight sets. Searching for the cut gains nothing measurable over the midpoint and loses nothing. The search is the default because it needs no assumption about the class priors; `--method kfda` with the parameter `kfda_threshold` set to `midpoint` selects the other.

So what is it for. Not accuracy: on these sets the four learners are one classifier as far as the measurement can tell. It is for the picture. The discriminant is the classification model this program can draw, because its score is one linear functional of the field and is found in closed form. One limit belongs here. The error band of *How much of it is noise* propagates noise in the **activities**, and a class label carries none, so the band is not available for this classifier or for any other; `cmf cofield` says so and draws the map unmasked. The full table, per set and with the settings chosen,

is `benchmarks/results/kfda/table.md`.

7.10 Kernel logistic regression: the Gaussian process with its ridge chosen on the folds

The Gaussian-process classifier above does two things at once that can be separated. It finds the mode of the latent function under a Bernoulli likelihood, and it then averages its probability over a posterior around that mode; and it chooses its one hyper-parameter, the kernel amplitude, from the marginal likelihood rather than from held-out molecules. `--method klr` is the same mode with neither of those: the ridge comes from the λ grid on the folds, and the probability is the likelihood at the mode with nothing averaged over it.

What it minimises. The logistic loss of the latent function plus a ridge on its norm in the space the kernel defines,

$$\sum_i \log\left(1 + e^{-y_i f(x_i)}\right) + \frac{\lambda}{2}\,\|f\|^2, \qquad f(x) = \sum_j a_j K(x, x_j), \tag{7.4}$$

a smooth convex problem solved by Newton's method. The stationarity condition is worth knowing because it says what the coefficients *are*: $\lambda\, a_i = t_i - \pi_i$, the difference between the class label and the fitted probability. A molecule the model already gets right contributes almost nothing to the model.

It is the Laplace step, not a second implementation of it. The mode above is exactly the mode `cmfqsar/gpc.py` finds with the logistic likelihood and amplitude 1/λ, so `klr_fit` runs that iteration rather than repeating it, and `tests/test_klr.py` checks the two agree and that both agree with linear logistic regression on explicit features when the kernel is linear. What is skipped is the factorisation afterwards, which is what the posterior variance and the marginal likelihood are read from — so `klr` has neither, and asks you to fit `gpc` if you want them.

The boundary and the probability are one statement. $f = 0$ is $p = 0.5$, so the class this learner reports and the probability it reports never disagree, which is the one wart of the kernel Fisher discriminant above. The price is that $p = 0.5$ follows the larger class: on a set with one active in twenty, calling everything inactive is right nineteen times in twenty, and the balanced accuracy of that is 0.5. The parameter `klr_threshold` set to `cv` moves the boundary to the best out-of-fold balanced accuracy instead, at the cost of the class and the probability disagreeing again. It is off by default.

```
        python cmf_cli.py build examples/sutherland/AChE.sdf --charges file --task classification \
               --threshold 6.84 --method klr --lams 0.01,0.03,0.1,0.3,1,3 --normalize
```

Measured against the Gaussian process it comes from

`benchmarks/run_klr.py` runs the protocol of *A classifier with a co-field* on the same two LIT-PCBA sets, and holds the likelihood fixed: the Gaussian-process classifier is run with the **logistic** likelihood here, not with its own probit default, because otherwise the two would differ in three things at once and nothing could be attributed to any of them. It appears twice, once choosing its amplitude on the folds and once from the marginal likelihood, which separates the two differences.

Mean of the two sets	AUC	Brier score	balanced accuracy (common rule)
kernel logistic regression, ridge by CV	0.596	0.0519	0.511
GP classifier, amplitude by CV	0.587	0.0559	0.508
GP classifier, amplitude by the marginal likelihood	0.598	0.0472	0.514
C-SVC	0.607	—	0.517
kernel Fisher discriminant	0.605	0.0457	0.529
the share of actives, predicted for every molecule	0.500	0.0458	0.500

The expected tie on accuracy holds. Against the Gaussian process with the same likelihood and its amplitude chosen the same way, the AUC difference is −0.001 [−0.012, +0.010] on one set and +0.019 [−0.059, +0.085] on the other: the two are one classifier, which is what they should be, since they are the same mode. Against C-SVC the mean difference is −0.011 [−0.061, +0.041] and against the discriminant −0.008 [−0.065, +0.048]. No accuracy gain anywhere.

On calibration the entry's expectation fails. E9 was written expecting this learner to become the reference the Gaussian process's probabilities are judged against. It is not. Dropping the variance averaging does gain a little — the Brier score is lower than the cross-validated Gaussian process's, by differences of −0.0034 [−0.0056, −0.0010] and −0.0046 [−0.0075, −0.0015] that exclude zero on both sets — but choosing the amplitude by the marginal likelihood gains more in the other direction: the evidence-selected Gaussian process beats this learner by 0.0041 [+0.0020, +0.0064] and 0.0053 [+0.0025, +0.0083]. **The evidence is the better selector for calibration on these sets, and that is a point for the Gaussian process, not against it.**

And none of them beats the base rate. Predicting the share of actives for every molecule and saying nothing else scores 0.0458 over the two sets. Kernel logistic regression scores 0.0519, the cross-validated Gaussian process 0.0560; only the evidence-selected Gaussian process (0.0472) and the discriminant (0.0457) come back to it, and neither beats it. The reliability table in `benchmarks/results/klr/table.md` shows why: every learner puts almost every molecule in the lowest probability bin, where the observed share of actives is about 0.05, and the few molecules it places higher turn out active about one time in ten rather than the one time in two or three it predicted. These probabilities are over-confident where they are not trivial. Read them as a ranking, not as a frequency.

What "cheaper" is worth. The plan entry claims this learner is cheaper by the variance it does not compute. Measured on these kernels, with about 1570 molecules: one fit takes 490 ms against 567 ms and 499 ms against 531 ms, a gain of 6 to 16 %, because Newton's method takes the same number of steps either way and only the final factorisation is saved. Scoring every molecule takes 0.6 ms against 37.9 ms and 0.3 ms against 30.2 ms — 61 and 112 times less — because the variance costs a triangular solve for each molecule scored. **The saving is at prediction time, which is where a screen spends its time, and not at fit time.**

So what it is for. Not accuracy, and not calibration. It is the plain object: the regularised logistic regression in the field space, with one hyper-parameter chosen the way every other learner here chooses its own, no prior to argue about and no approximation in the probability beyond the mode itself. It is also the cheap way to score a large library with a probability. If you want a probability

that admits ignorance for a molecule unlike the training set, `gpc` is still the method, and its evidence is a better way of choosing its amplitude than the folds are.

7.11 Molecules without an activity: the Laplacian-regularised ridge

Every learner above fits the molecules whose activity is known and ignores the rest. A screening library, a set of decoys, a batch synthesised but not yet measured: these molecules have fields, and there are usually far more of them than there are measurements. `--method lapkrr` is the standard way of putting them to use, **manifold regularisation** (Belkin, Niyogi and Sindhwani 2006): kernel ridge regression with a second penalty that asks the prediction to change slowly between molecules that are close in field space, whether or not they carry an activity.

What it minimises. With J the diagonal indicator of the molecules that have an activity and L the Laplacian of the graph of field similarity,

$$\sum_{i \in \mathcal{L}} (y_i - f(x_i))^2 + \lambda \, \|f\|^2 + \gamma \, \mathbf{g}^{\mathrm{T}} \mathbf{L} \, \mathbf{g}, \qquad f(x) = \sum_j a_j K(x, x_j) + b, \qquad \mathbf{g} = \mathbf{K}\mathbf{a}, \tag{7.5}$$

where the sum over i runs over the labelled molecules only and the sum over j runs over **every** molecule, labelled or not. The third term is small when the prediction takes similar values at molecules the kernel calls similar. Setting the derivatives to zero gives one linear solve over all n molecules,

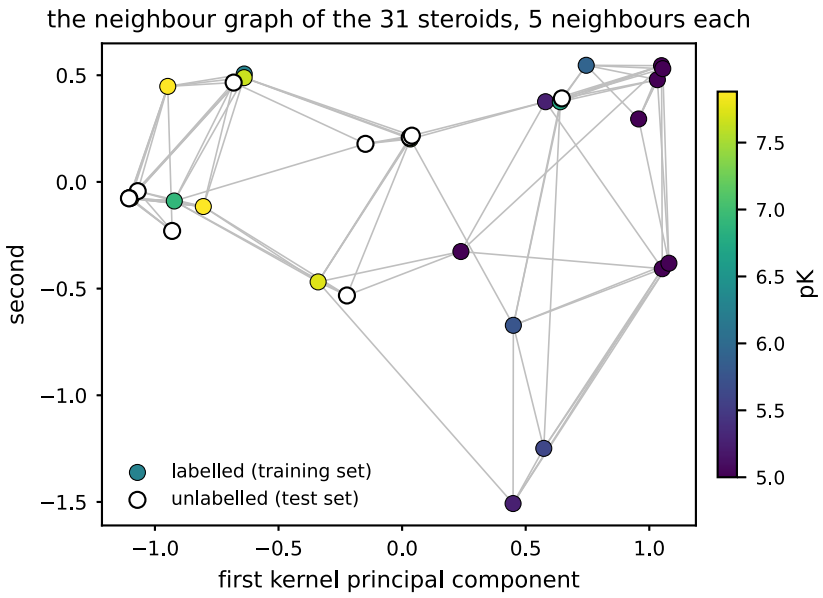
$$(\mathbf{J}\mathbf{K} + \lambda \mathbf{I} + \gamma \mathbf{L}\mathbf{K}) \, \mathbf{a} = \mathbf{J}\mathbf{y}, \tag{7.6}$$

with one extra row and column for the bias, exactly as for kernel ridge regression. So the fitted model is still one kernel expansion, and **the co-field, the molecule contributions and the atom contributions all read it with no special case** — with the unlabelled molecules now among the terms, which is what lets a co-field picture see the library.

The bias stays outside the penalty. That is what **g** = **Ka** says above: **g** holds the predicted values without the bias term. Adding the same constant to every activity does not make a model vary faster, so a bias should not be charged for. The distinction matters here, because L **1** is not zero for a normalised Laplacian, while it is zero for the unnormalised one. A bias inside the penalty would therefore be charged for being a constant, and the solve above would carry a further γ L **1** b term in its extra column.

How the activities are passed. The activity of an unlabelled molecule is `nan`. A learner that needs more than the kernel and the activities is new in this program, and the alternative — a second block of molecules handed to the learner on the side — would have put the unlabelled molecules outside the training table, where nothing that reads a model could see them. Marking the missing values in the activity vector keeps one training table and one expansion over it. Everything downstream follows: the cross-validation folds hold out only molecules that have an activity and keep the unlabelled ones in the training part of every fold, and every statistic is computed on the labelled molecules alone.

The graph. L is the normalised Laplacian of `cmfqsar/spectral.py`, built from the **cosine** of the kernel so that "close" means similar and not merely large, with every molecule keeping only its `--neighbours` closest neighbours (10 by default). Without that truncation every molecule is a neighbour of every other and the penalty asks the prediction to be flat over the whole set instead of smooth along it.



The neighbour graph the Laplacian is built from, for the 31 steroids with five neighbours each, drawn in the first two kernel principal components: an edge joins two molecules whose fields are among each other's closest, the training molecules are coloured by their affinity and the test molecules, which carry no activity, are the unlabelled ones. The penalty asks the prediction to change little along an edge, so the unlabelled molecules pass the information of their labelled neighbours on, and a molecule with no edge to a labelled one is left to the kernel alone.

The assumption, which is the whole of it. $\gamma\, \mathbf{g}^{\mathsf{T}}\mathbf{L}\,\mathbf{g}$ says the activity varies smoothly along the data. An **activity cliff** — two molecules whose fields are nearly the same and whose activities are not — says it does not, and where there is one the penalty pulls both predictions towards their average.

```
python cmf_cli.py build examples/sutherland/GPB.sdf --charges file --method lapkrr \
       --unlabelled library.sdf --lams 0.01,0.1,1 --manifold-weights 0,0.1,1 --normalize
```

The weight zero is in the grid on purpose: it is plain kernel ridge regression on the labelled molecules, so the cross-validation can decide that the unlabelled molecules do not help.

Measured, and it does not help

`benchmarks/run_lapkrr.py` asks the two questions of the plan entry. First, on the eight Sutherland sets, half of the published training activities are hidden and three models are scored on the published test set, which none of them saw: ridge on the visible half, this learner with the hidden half unlabelled, and ridge on every activity, which is the ceiling. Each chooses its own settings by cross-validation on the molecules it may see; the figures are means over five halvings and the intervals are paired bootstraps over the test molecules.

Mean of the eight sets	R2pred
ridge, the visible half only	0.258
lapkrr, the hidden half unlabelled	0.235
ridge, every activity measured (the ceiling)	0.376

The penalty recovers none of the gap. The hidden activities are worth +0.118 of R2pred. The penalty gains −0.023 on average, is ahead on two of the eight sets, and on no set does its interval exclude zero above; on two sets (BZR −0.032 [−0.062, −0.007] and GPB −0.051 [−0.177, −0.004]) the interval excludes zero **below**, so there it measurably loses.

It is the method and not the choice of its weight. Choosing both settings by looking at the test activities — an oracle that cannot be used — gains +0.009 of R2pred on average over ridge chosen the same way, and the interval excludes zero on none of the eight sets. There is no weight at which this penalty is worth anything on these series.

And the cliffs do not explain it. The plan entry expected a loss to be read against the activity cliffs, so they are counted: a pair of labelled molecules counts as similar when the cosine of their fields is 0.9 or more, and as a cliff when it is similar and the activities also differ by at least one log unit. Between 22 % (BZR) and 63 % (THER) of the similar pairs are cliffs. The rank correlation between that share and what the penalty gains is −0.33, and between the mean activity difference of a similar pair and the gain −0.38: too weak to predict anything. What does line up is the **size of the set**: the rank correlation between the number of labelled molecules and the gain is +0.74. The penalty loses most on the smallest sets and nothing at all on the largest.

One place it gives a little. Read on the hidden molecules themselves rather than on the test set — which is the transductive setting manifold regularisation is usually stated in, since those molecules are in the expansion — it is ahead on six of the eight sets, +0.383 against +0.374 on average. That is a gain of 0.009 and well inside the noise.

Second, the screening sets: the two LIT-PCBA targets of *A classifier with a co-field*, with the held-out fold of each split given to the learner as an unlabelled library. **No enrichment is lost**: no difference in AUC, in the enrichment factor at the top 1 % or 5 %, or in BEDROC against either kernel ridge regression on ±1 labels or C-SVC excludes zero below it. That part of the gate passes, but it passes quietly: on ESR1_ant the folds chose a penalty weight of zero in all five folds, so the model fitted there *is* kernel ridge regression and every difference is exactly zero.

So what it is for. Not accuracy. The learner is kept, labelled as what the measurement makes it: the standard way of using unlabelled molecules, implemented correctly — `tests/test_lapkrr.py` shows it turning an R2 of +0.02 into +0.91 on data built to satisfy its assumption — and gaining nothing on an aligned congeneric series of a few dozen molecules, which is what the data sets in this repository are. Two reasons to have it anyway. It is the only learner here whose co-field is built from molecules that were never measured, so a picture can be drawn over a library rather than over a training set. And the weight zero is inside its grid, so a user who tries it on a larger or less congeneric collection risks nothing: the cross-validation can switch the penalty off, and on half of these fits it did. Kernel ridge regression stays the default.

The full table, per set, with the oracle rows, the cliff counts and the neighbourhood sizes, is `benchmarks/results/lapkrr/table.md`.

Not in the graphical interface. The fit panel has nowhere to name a second set of molecules, so this learner is on the command line and in the library only.

7.12 Fitting the order instead of the values: the ranking SVM

A regression model is asked for the activity and judged on the squared error, which assumes the numbers are exact. Often they are not. Potencies pooled from several papers carry an offset per source that no descriptor explains, a series measured in two assays carries two scales, and an IC50 read off the end of a dose–response curve carries a bound rather than a value. In all three the *order*

survives what the values do not, and a model fitted to the values spends its capacity on the part that is noise.

`--method ranksvm` fits the order. For every pair of molecules whose activities differ it asks that the more active one score higher by a margin, and is penalised for every pair where that fails:

$$\min_{w,\,\xi} \frac{1}{2}\|w\|^2 + C\sum_p \xi_p \qquad \text{subject to} \quad w\cdot(X_i - X_j) \geq 1 - \xi_p \tag{7.7}$$

over the pairs $p = (i, j)$ with $y_i > y_j$ — an ordinary support-vector classification on the *difference* of two molecules' fields. The kernel of two such differences is a sum of four ordinary kernel entries,

$$K_{\text{pair}}((i,j),(k,l)) = K(i,k) - K(i,l) - K(j,k) + K(j,l) \tag{7.8}$$

so the whole thing is built from the kernel matrix the program already has, with no new fields and no new parameterisation.

The fitted model is still a dual expansion over molecules. The weight vector is $w = \Sigma_p\, a_p\, (X_i - X_j)$, so collecting the terms gives one coefficient per *molecule* and the score of a new one is $K(x, \text{train})\,\mathbf{a}$, exactly as for every other learner here, with no bias. Everything that reads a linear CMF model — co-fields, atom contributions, Shapley values, the pharmacophore — keeps working. What changes is what they mean: the co-field of a ranking model says where more field *promotes* a molecule, not what it adds to the activity.

What is given up is the scale. A ranking score has no units and cannot be compared with an activity; only its order means anything. The program therefore reports Spearman's ρ, Kendall's τ and the share of pairs ordered correctly, and reports no r^2 or RMSE, which would be arithmetic on a quantity that does not exist. The hyper-parameter search selects on ρ for the same reason.

```
python cmf_cli.py build data.sdf --charges file --method ranksvm --Cs 0.1,1,10 \
    --rank-gap 0.5 --rank-groups paper
```

Three options shape the pairs. `--rank-gap` drops the pairs whose activities differ by less than it: a pair separated by less than the experimental error is a matter of chance, and asking a model to call it is how a ranking model learns noise. Half a log unit suits pooled literature potencies and zero suits one careful assay. `--rank-groups PROP` forms pairs only inside a group — the same paper, assay or series — which is what makes the method useful on pooled data, because an offset between two sources then cancels instead of being compared across. `--rank-pairs` caps the number of pairs: the solver works on the kernel *between pairs*, so the cost is quadratic in that number, and the default of 4,000 already asks for 128 MB. The order of a set is heavily over-determined and a few thousand comparisons fix it, so the cap is a bound on cost rather than a loss of information.

When it is worth it, measured

`benchmarks/run_ranking.py` asks the only question that matters — does fitting the order beat fitting the values — on the eight Sutherland sets with five random 70/30 splits each, every model choosing its own hyper-parameters on its own criterion inside the training part.

On clean data it does not. Where the activities are one assay's numbers and mean what they say, Spearman's ρ on held-out molecules is **+0.713 for kernel ridge and +0.689 for the ranking SVM**, ridge ahead on five of the eight sets. That is the expected answer and worth stating plainly: a regression model is already a ranking model, since the order of its predictions is all a

ranking model ever produces, and it had the magnitudes to learn from as well. Fitting the order discards how far apart two molecules are, which on reliable data is information.

On pooled data it does, but only with the groups. The second experiment gives the same molecules to three synthetic sources — assigned *by scaffold*, because an offset that travels with the chemistry is what a model will learn as if it were an effect of the chemistry, while an offset assigned at random is simply averaged away — shifts each source by 1.5 standard deviations, and asks for the order of the true activity, which no model ever saw:

	KRR	ranking	ranking by source
same-source pairs ordered correctly	72.5 %	71.6 %	**74.3 %**
Spearman ρ over all held-out pairs	+0.574	+0.565	**+0.663**

A ranking model that is not told about the sources does no better than ridge: it is handed the same corrupted comparisons and learns them. Told which molecules share a source, it beats ridge by **0.089 in ρ**, on six of the eight sets, because the cross-source pairs — the only ones the offsets corrupt — are never formed. Note that it wins on the *global* order too, although it was never shown a cross-source comparison: it does not need one, since it fits a single scoring function over the fields and all that grouping did was stop it fitting an offset.

So the summary is narrow and useful: **the ranking SVM is for data whose numbers are not comparable across their sources, and its value is** `--rank-groups`**.** On a single clean assay, use kernel ridge.

7.13 Nonlinear kernel layers and multiple kernel learning

A layer on top of the CMF kernel makes the model more expressive than an overlap integral: a Gaussian layer, $\exp(-\gamma\, d^2)$ with $d^2 = K(i,i) + K(j,j) - 2K(i,j)$, or one or two arc-cosine layers, the kernel of an infinitely wide neural network [14]. The cost is interpretability: the model is then no longer linear in the field, so the co-field and the atom contributions of the linear model are unavailable. What takes the co-field's place is a co-field per molecule, the derivative of the prediction with respect to the field at that molecule, with the spread of those over the training set as a map of where the model is not additive and a non-linearity index per field (*The co-field of a nonlinear model: one per molecule*). The similarity decomposition stays valid, and the **Shapley tab** decomposes the prediction instead, which is why it exists.

Multiple kernel learning chooses the mixing coefficients h_f rather than leaving them at 1, either by coordinate descent on the cross-validated error, by centred kernel alignment (`--mkl align|alignf`), which is a closed-form, training-only criterion, or by the marginal likelihood of the mixture (`--mkl evidence`), which is the next section.

7.14 Which fields does this series need?

A sum of kernels is the kernel of a sum of independent processes,

$$f(x) = \sum_f f_f(x), \qquad f_f \sim \mathcal{GP}(0,\ h_f c_f K_f), \tag{7.9}$$

so h_f is not a tuning parameter but the **prior variance of the part of the model that field f carries** — how much of the activity that field is allowed to explain. Put a prior on it and integrate,

and the answer to *does this series care about the steric field at all* comes with an interval instead of a number:

```
python cmf_cli.py mixing model.cmf
```

What is sampled is each field's **share** of the kernel, written $h = F\,\mathrm{softmax}(u)$ so that the shares sum to one and can be compared with $1/F$, the share a field would have if all fields mattered equally. The prior is $u_f \sim \mathrm{N}(0, 1.5^2)$, independent and the same for every field, so it prefers no field over any other and only limits how far apart their shares can be drawn before the data are taken into account; the regularisation is sampled with them. The sampler is a slice sampler, which has no step size to tune.

Three readings come out, in increasing order of sharpness. The **shares with their intervals** say how much of the kernel each field carries. The list of fields **carrying less than an equal share** names those whose whole interval lies below $1/F$. And the **Bayes factor** against *this field does nothing* — the log evidence lost when the field is removed outright and the others are re-weighted — is the sharpest and the cheapest form of the question.

What it is good for, measured

Eight Sutherland sets, five 70/30 splits each. First, whether weighing the fields predicts better at all — the weights set four ways on the training part, each scored on the molecules held out:

weights	R^2pred
equal (h = 1)	+0.481
centred kernel alignment, per field	+0.487
centred kernel alignment, jointly	+0.485
the evidence optimum	+0.485

The evidence optimum is +0.003 in R^2pred against equal weights, 95 % interval [-0.026, +0.035], which contains zero — the same lesson as the per-field attenuation: **the evidence takes whatever freedom it is given over the shape of the kernel, improves its own number, and leaves the prediction unchanged.** Choosing the weights is not how a CMF model is made better.

What the posterior is *for* is the other question, and that one can be checked. Each field's Bayes factor against *this field does nothing*, beside what actually happens to the held-out molecules when that field is removed outright:

field	mean share	90 % interval	Bayes factor	ΔR^2pred without it
Electrostatic	0.179	[0.024, 0.432]	+1.4	-0.032
Steric	0.337	[0.091, 0.663]	+2.2	-0.012
Hydrophobic	0.184	[0.021, 0.460]	+0.7	-0.006
H-bond acceptor	0.159	[0.032, 0.385]	+1.6	-0.018
H-bond donor	0.142	[0.025, 0.349]	+1.5	-0.003

Over all 200 (set, split, field) cases the Bayes factor and the cost of removing the field correlate at Spearman ρ **+0.121** (positive means the evidence was right). The two orderings barely agree: the Bayes factor says what the training activities can see, which is not the same as what the held-out molecules need, and it should be read as the first and not the second.

The intervals are wide, and that is the answer rather than a failure of the sampler. On a congeneric series the field kernels are strongly correlated: several quite different mixtures explain the activities about equally well, and a point estimate — the alignment's, the evidence optimum, or any other — is claiming a precision the data do not support. That is the argument for integrating rather than optimising here, and it is the reason `--mkl evidence` exists as an option rather than as a default.

A prior pooled across series, and what it repairs. One series cannot say much about five weights; a collection of series can. `benchmarks/run_pooled_mixing.py` learns a two-level prior over the shares from 38 Mittal sets and applies it to a thirty-ninth, never using that set itself (`cmfqsar.mixing.pool_priors`, 300 posterior draws per set; the table is `results/pooled_mixing/table.md`). What the collection says is that the fields are not equal: the typical shares are steric 0.488, hydrophobic 0.246, electrostatic 0.109, H-bond acceptor 0.100 and H-bond donor 0.056, where equal shares would be 0.200. The spread of the log-shares between series is $\tau = 0.76$, against the 1.5 of the fixed prior above, so the series differ from one another less than that prior allows; an estimate that makes no Gaussian assumption gives $\tau = 0.86$.

The measurement says the same as the rest of this section. Against equal weights the pooled weights are **-0.003** in R^2pred, 95 % interval [-0.027, +0.017], which contains zero. Against the weights a series fits for itself by its own evidence they gain **+0.022** [+0.002, +0.047]. The prior from the other series therefore repairs the damage that fitting five weights on one series does, and does not improve on leaving the weights equal. On the 15 sets below 40 molecules, where a prior should matter most, pooled against equal is +0.009 [-0.022, +0.042]. Equal weights stay the default, and the pooled prior is the answer to *what do these series have in common*, not a way to predict better.

7.15 Choosing α and λ without holding anything out

Everything above chooses hyper-parameters by cross-validation: ten fits and ten predictions per candidate, and the best held-out score wins. A kernel ridge model is the posterior mean of a Gaussian process, and a Gaussian process scores its own hyper-parameters without holding anything out. The marginal likelihood — the probability of the training activities with the weights integrated away — is one determinant and one quadratic form on the factorisation the fit needs anyway, so a candidate costs **one factorisation instead of ten**:

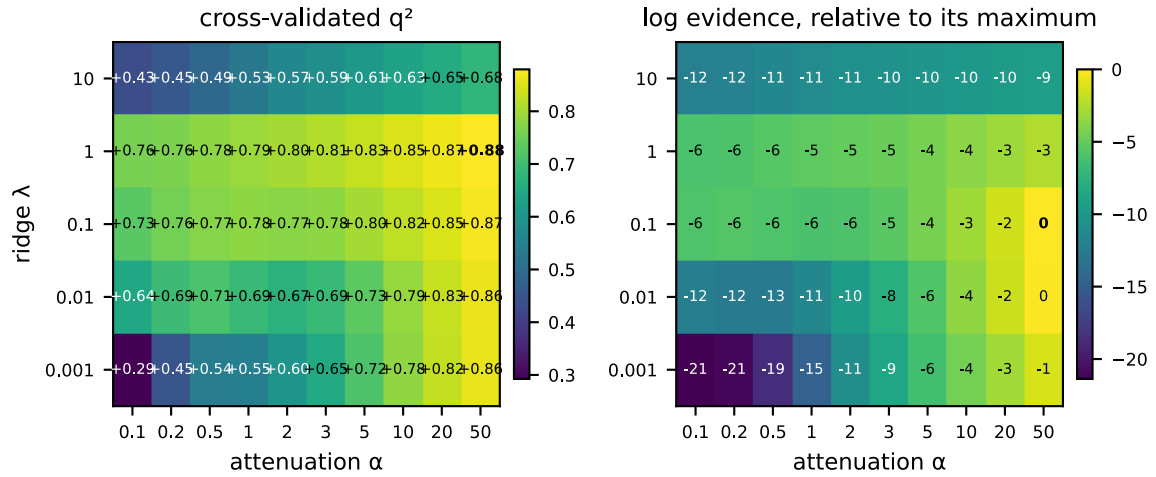
```
python cmf_cli.py build data.sdf --charges file --select evidence
```

Two adjustments are made to it, and both matter. The bias of the program's KRR is a constant mean in the Gaussian process, so it is **marginalised under a flat prior** rather than fitted and then scored, which would count it twice. And the amplitude of the kernel — unknowable, since the kernels are cosine-normalised and the activities arrive in whatever units the assay used — is **profiled out** at its closed-form optimum, so the criterion depends on α and λ alone and its ranking does not move when the activity is rescaled.

Measured, it chooses as well and costs a quarter as much. Over the eight Sutherland sets, five random 70/30 splits each, with both criteria searching the same grid on the training part and both choices scored on the held-out molecules:

	R²pred
chosen by cross-validation	+0.471
chosen by the evidence	+0.479
paired difference	**+0.007**, 95 % interval [−0.013, +0.028], better on 6 of 8 sets

The two are indistinguishable in quality and the evidence is four times faster at selecting. The detail worth more than either number is that **they agree on the point of the grid in 0 to 60 % of splits** and reach the same predictive quality anyway: the surface is flat near its optimum, so what a search costs matters more than which point it selects. That is also what makes a finer grid — a separate α per field, say — affordable for the first time, and the next section uses the saved time for exactly that and reports what it gained, which was nothing.



The two criteria over the same grid of the steroid training set: the cross-validated q² of the search figure, and the log marginal likelihood with the bias marginalised and the amplitude profiled out, written relative to its maximum, the best point of each in bold. Both surfaces are flat along α and steep along λ, and the two best points differ by one step of λ at the same α, where the q² values differ in the second decimal; a candidate on the right costs one factorisation, on the left ten.

The evidence is not a cross-validated score and does not pretend to be. It answers *how probable are these activities under this model*, not *how well would this model predict a molecule it has not seen*, and the two can disagree when the kernel is misspecified, which for molecular fields is the normal case. On these sets they do not. It is available for regression with `krr` or `gp`; the support-vector methods and the classifiers have no such quantity and are refused rather than silently cross-validated.

One width per field

One α for five fields is a convention, not a result. Electrostatics is long-ranged and a hydrogen-bond donor is nearly a point; there is no reason the same attenuation suits both. The protocol of the papers shared it because the computational cost required it — a grid over one width is five kernels, a grid over one width per field is 5^5 of them — and the evidence removes that objection twice over. The per-field kernels are independent, so every combination is assembled from the matrices the shared grid already built, and each candidate costs one factorisation. A coordinate descent over the widths, starting from the best shared one, then costs a few hundred:

```
python cmf_cli.py build data.sdf --select evidence --alpha-search per-field
```

It reports the width of each field and what the extra freedom gained, in the units it was selected in. Fields whose kernel does not depend on α at all — the probe fields, whose widths come from their own expansion — are left alone rather than searched in vain.

Measured, the extra freedom is real and gains nothing. Eight Sutherland sets, five 70/30 splits each, the shared width chosen by the evidence on the training part and the descent started from it:

log evidence gained by the per-field widths	**+2.6**, on every one of the 40 splits
widths moved away from the shared one	3.5 of 5 on average, and at least one on every split
R^2pred against the shared width	**−0.001**, 95 % interval [−0.020, +0.018], better on 4 of 8 sets
the same count per split	ahead on **20 of 40**
correlation of the evidence gained with the R^2pred gained	**−0.07**

The descent does what it was asked: it moves most of the widths, improves its criterion every time, and settles on widths a chemist would find reasonable — the steric field sharpest (mean α 2.33), the donor field widest (1.16). The molecules held out cannot tell the difference, and the gain in evidence carries no information about whether a split improved.

This is the same flat surface the previous section found in a different way. There, two different criteria picked different shared widths and predicted equally well; here, one criterion is given five widths instead of one, moves them, and predicts equally well. Around its optimum the attenuation does not decide much, and five of them decide no more than one — which is worth knowing, because it says the search is not what decides the accuracy of a CMF model. The alignment is (see *Alignment*, and the third part of `docs/benchmark.md`). Use `--alpha-search per-field` to ask the question of a new series, not as a default.

7.16 Validation

- **k-fold cross-validation** (10 by default) gives q^2 and RMSEcv. Folds are shuffled unless you ask for the interleaved folds of the original scripts (`--cv-no-shuffle`).
- **A test set** gives R^2pred and RMSEpred. Both variance conventions are reported, because they differ for sets whose test activities span a narrow range — the steroid benchmark is the classic example, where the ten test affinities cover 1.9 log units and R^2pred computed against the test variance is negative while the predictions are good.
- **External (nested) cross-validation** (`--nested`, *External CV*) wraps the whole model selection in an outer loop and reports q^2ex and RMSEcvex. This is the number to quote: the ordinary q^2 has seen the hyper-parameter choice.
- **y-randomisation**, a **bootstrap** interval and the **applicability domain** are the three sections below.
- **Leave-one-chemotype-out** folds, for a set of several scaffolds, never leave a molecule's own scaffold-mates in the training part. Random folds overstate the quality of such sets badly — by 0.11 to 0.47 q^2 on the five chemotype benchmarks here — and the program reports both, together with an alignment-free fragment baseline that the aligned model has to beat.

7.17 Is it chance? y-randomisation

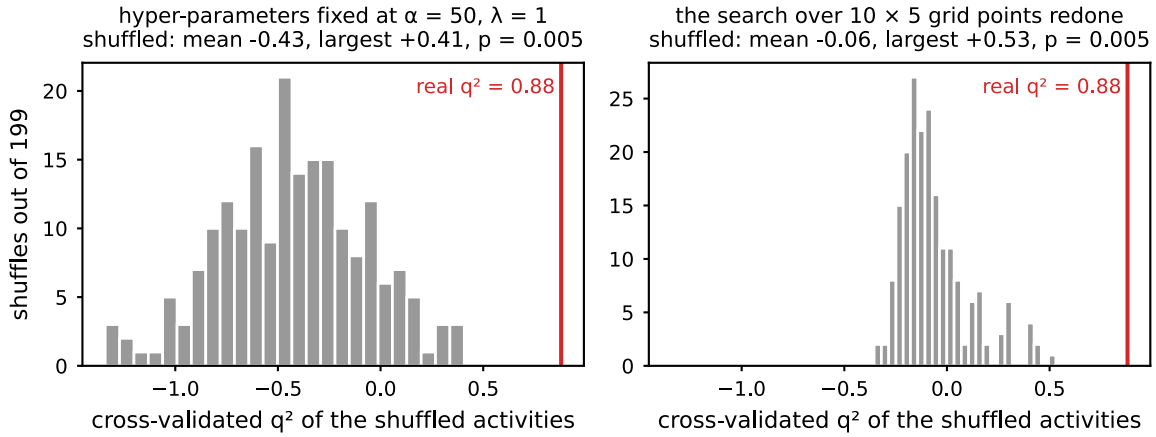
A cross-validated q² says how well the model predicts; it does not say whether a model of the same shape could have reached that number from activities that mean nothing. In a congeneric series, where kernel values above 0.97 are normal, that is a real possibility, and it is the first thing a referee asks about a 3D-QSAR model.

The **shuffles** box of the model panel's *Validation* row (`--y-scramble N`) shuffles the activities N times, refits and cross-validates each time, and reports where the real q² falls among the scrambled ones — the mean and spread of the shuffled values, the largest of them, how many standard deviations above them the real one lies, and the permutation p-value.

```
python cmf_cli.py build data.sdf --alphas 0.5 --lams 0.1 --y-scramble 100
```

Two properties of the test are worth knowing, and the program states both rather than letting the number mislead:

- **A permutation test cannot report a p below 1/(N+1).** Ten shuffles cannot go below p = 0.091, which is above every conventional threshold *however good the model is*. The program says so — "no shuffle came close, but 10 of them cannot support a p below 0.091" — instead of reporting a chance-correlation warning that the data do not support. Use 19 shuffles for p < 0.05 and 99 for p < 0.01.
- **The hyper-parameters are held fixed** by default, so the test asks whether the *fit* can be reached by chance. Tick **redo the search** beside the shuffles box (`--y-scramble-select`) for the stricter question — whether the whole procedure, model selection included, can be. It matters: on the steroid set with a 3 x 3 grid, the best scrambled q² rises from −0.08 to **+0.32** once the search is allowed to pick the grid point that best fits the noise, while the real value stays at +0.78. The kernels do not depend on the activities, so they are computed once and the extra cost is one cross-validation per grid point per shuffle.



y-randomisation of the steroid model of the search figure of Attenuation, regularisation and the grids: the cross-validated q² of 199 shuffles of the activities, with the real value as the red line. Left, the hyper-parameters held at the point the search chose; right, the whole search over the grid repeated inside every shuffle. The search finds the grid point that fits each shuffle best, so the shuffled values move up and spread less, and the largest of them rises; the real value stays where it is, and the distance between the two is the test. The mean, the largest shuffled value and the permutation p-value are in the titles.

7.18 Where does the model apply?

A kernel method always returns a number. For a molecule unlike anything in the training set, that number is the training mean presented as a prediction, and nothing in q² or R²pred reveals it.

The **applicability domain** tick of that row (`--domain`) measures how close each molecule outside the training set is to it, **in the kernel the model itself uses**: the cosine similarity of their fields, averaged over the *k* nearest training molecules (3 by default). The threshold is not an arbitrary number — it is taken from the training set's own spread, the 5th percentile of the same quantity computed among the training molecules, so a series of close analogues gets a tight domain and a diverse set a loose one. Molecules below it are listed by name in the log, their similarity appears in the data-set table's **AD** column — which sorts like any other, so the extrapolations can be brought to the top — and the exported predictions carry both the value and a yes/no verdict, so that an extrapolation stays marked after it leaves the program.

On the steroid benchmark, nine of the ten test compounds fall inside and one — 4-pregnene-3,11,20- trione — falls outside. On BZR, training on the largest chemotype and scoring the others, every molecule of the training chemotype is inside and 29 of 40 molecules from other chemotypes are outside, which is the behaviour one wants from a domain estimate.

The field-space distance is reported alongside the similarity, because the two can disagree: a molecule that is a scaled-up version of a training molecule points the same way (high cosine) but sits far away (large distance).

7.19 Is the whole library inside? The two-sample test

The applicability domain answers the question one molecule at a time. A screening run asks a different one — is this library drawn from the same population as the training set, or is the whole exercise an extrapolation? — and averaging a per-molecule verdict does not answer it: two sets can have identical nearest-neighbour similarities and still occupy different regions.

Every set of molecules has a **mean field**, the mean of its feature maps, which for CMF is itself a function of space. The maximum mean discrepancy [23] is the distance between two of them:

$$\mathrm{MMD}^2(X,Y) = \overline{K(x,x')} - 2\,\overline{K(x,y)} + \overline{K(y,y')} \tag{7.10}$$

— three averages over blocks of the kernel matrix and nothing else. The unbiased estimator leaves the self-similarities out of the two within-set terms, which is why it can come out slightly negative: that is how "no difference at all" looks once the bias has been removed, not a defect. The null distribution comes from permuting the labels, which costs three block means per shuffle because the kernel is computed once.

```
python cmf_cli.py compare library.sdf --model model.cmf --permutations 999
python cmf_cli.py compare seriesA.sdf seriesB.sdf --charges file
```

With `--model` the library is compared with that model's training set, in that model's own fields, reusing the cross-kernel the prediction computes anyway. The p-value is never zero: it is one plus the number of shuffles that matched or beat the observed value, over one plus the number of shuffles, because a finite number of them cannot support a claim of impossibility.

The witness says where the difference lies, which is the useful form of a p-value. It is the function whose expectation separates the two sets — a molecule's mean kernel value against the first set minus its mean against the second — evaluated at every molecule: most negative for the compounds that make the library unlike the training set, most positive for those most typical of it.

The command prints both ends and `--out` writes the whole column. On the Sutherland benzodiazepine set against the COX2 one, the witness names nordazepam and delorazepam as the most typical benzodiazepines and the COX2 sulphonamides as the least — without being told that there are two targets.

Is the test any good?

A test is useless unless it is calibrated as well as powerful, and `benchmarks/run_structure.py` measures both halves. Twenty random half-splits of each of the eight Sutherland sets, where there is nothing to find: **5.6 % are called significant at the 5 % level**, which is what a correctly calibrated test does and the permutation construction guarantees. All twenty-eight pairs of *different* targets, where there is everything to find: **100 % are called significant, median z = +171**. The test is calibrated where there is nothing and saturated where there is.

7.20 How certain is that number?

A difference of 0.02 in q^2 is quoted all over the benchmarks in this repository. Some of those differences are real and some are the particular molecules they were computed over.

The **bootstrap** box of that row (`--bootstrap N`) resamples the (observed, predicted) pairs N times and reports a percentile confidence interval for q^2 and for R^2_{pred}. It is the uncertainty of the *statistic* given these predictions, which is exactly what decides whether two settings differ. On the 21 training steroids, $q^2 = 0.768$ carries a 95 % interval of [0.47, 0.89] — wide enough to show that a difference of a hundredth means nothing.

`--bootstrap-refit N` adds Cramer's bootstrap [26] as well: resample the training set with replacement, refit, and score the molecules that were left out of the bag. That one measures the variability of the fit rather than of the statistic, and it is the more pessimistic of the two. The kernel is never recomputed — a resample is an index into the matrix — so a hundred refits cost about as much as one cross-validation.

7.21 How certain is one prediction? Conformal intervals

The bootstrap says how firm a figure of merit is. It says nothing about the molecule in front of you, and neither does anything else the program has offered so far: the Gaussian-process posterior spread is the spread of a model that believes its own kernel, which for molecular fields it should not.

Conformal prediction supplies the missing guarantee and requires almost nothing in return. Take the absolute errors the model makes on molecules it did not fit, let q be their $(1-\alpha)$ quantile — the $\lceil (n+1)(1-\alpha) \rceil$-th smallest, the +1 being what makes the result finite-sample rather than asymptotic — and report $\hat{y} \pm q$. Then, as long as the calibration molecules and the new one are exchangeable, the truth lies inside at least $(1-\alpha)$ of the time, **whatever the learner, whatever the kernel, however wrong the model is**. A badly specified model does not lose its coverage; it pays with a wider interval instead.

```
python cmf_cli.py build data.sdf --charges file --conformal 0.9 --save model.cmf
python cmf_cli.py predict model.cmf new.sdf --out predictions.csv
```

`--conformal` calibrates on the errors of the cross-validation the program is running anyway, so it costs nothing and adds no step, prints the half width, and — when there is a test set — the coverage it actually achieved on molecules that were never part of the calibration. The rule is stored in the `.cmf` file, so `predict` prints an interval beside every number and writes `lower` and `upper` columns into the CSV. `--conformal LEVEL` on `predict` re-reads the stored errors at a different level;

the errors are kept, not just the width.

What it costs, measured

Calibrating on out-of-fold errors and then predicting with the model refitted on everything is not exactly exchangeable — the errors come from models fitted on nine tenths of the training set and the predictions from a model fitted on all of it — so the guarantee becomes an approximation, and the size of the gap is an empirical question. `benchmarks/run_conformal.py` answers it on the eight Sutherland sets, ten random 70/30 splits each, with the width in units of the activity (whose standard deviation averages 1.36):

nominal	out-of-fold (`cv`)	held-out third (`split`)	GP posterior
80 %	**80.3 %**, width 2.28	80.1 %, width 2.59	77.7 %, width 3.00
90 %	**89.9 %**, width 3.10	90.1 %, width 3.61	83.1 %, width 3.85
95 %	**95.1 %**, width 4.09	94.9 %, width 4.04	86.9 %, width 4.59

The cheap variant holds its level to a few tenths of a per cent at every level, and is **narrower** than the exact split version, which loses accuracy because it fits on less data after holding a third out. The Gaussian-process interval, which is what one would otherwise reach for, misses its level at all three — and misses it unpredictably: across the eight sets its 90 % coverage ranges from 63 % to 99 %, while every conformal rule stays between 89 and 96 %.

Normalised conformal — dividing each error by the distance of its molecule from the training set, so that the interval widens where the model extrapolates — is available as `--conformal-normalise`, and on these sets it gains nothing: it over-covers (93.4 % at a nominal 90 %) and is a third wider. Softening it until it stops over-covering turns it back into the plain rule. The error on these series simply does not grow with distance from the training set strongly enough for the normalisation to be useful, which is itself worth knowing about one's data.

Two limits worth stating. The guarantee is **marginal**, not conditional: 90 % of molecules are covered, not 90 % of *every kind* of molecule, so a subclass the model handles badly can be covered less often than that while the overall figure holds. And a level needs molecules to support it — 19 calibration errors for 95 %, 9 for 90 % — below which the program says the level is not reachable rather than returning its widest error and calling it an interval.

Regression only: conformal classification produces prediction *sets* rather than intervals, and is not implemented.

When the scale of the activity is the problem: a warped process

The interval above, and the standard deviation of the chapter after this one, assume one thing about the activity: that what the kernel does not explain is Gaussian noise on one scale. A pIC50 series is often not like that. The scale is skewed, a floor of inactive compounds sits at one value, a handful of very active ones stretch the top. A Gaussian process on such a series spends its noise on the shape of the scale instead of on the chemistry, and its interval is too wide at one end and too narrow at the other.

The remedy of Snelson, Ghahramani and Rasmussen (NIPS 2003) is to learn a monotone transform of the response *together with* the process, so that the transformed activity is what the GP models and every prediction is carried back through the inverse. `cmfqsar.warp` does this (plan item ND11). With $u = (y - \text{centre}) / \text{width}$ the standardised activity,

$$g(y) = u + \sum_{i=1}^{I} a_i \tanh(b_i(u + c_i)), \qquad a_i > 0,\ b_i > 0, \tag{7.11}$$

a straight line plus a few smooth steps, each of which stretches the scale around one place and compresses it far away. Two terms are the default, six parameters, so it fits thirty molecules. The parameters maximise the log marginal likelihood of the warped activities under the kernel -- with the amplitude profiled out and the constant mean marginalised, as the evidence of *Choosing the attenuation and the ridge by the evidence* is computed -- plus the Jacobian $\Sigma_i \log g'(y_i)$ that makes it the likelihood of the *original* activities. Because the amplitude is profiled, the objective does not change when the transform is scaled, so what is fitted is only its shape.

A series close to normal is left alone. Six free parameters gain a few nats of evidence on any series. The transform is therefore kept only if it beats the penalty of the Bayesian information criterion for its parameters, half their number times log n -- about twelve nats on sixty molecules, where a Gaussian series gains four -- and the identity is used otherwise. The output says which happened and by how much.

What comes back. The GP gives a mean and a deviation in the warped units. The prediction in the original units is the *median*, g^{-1}(mean), and an interval is [g^{-1}(mean − k sd), g^{-1}(mean + k sd)], which is no longer symmetric about the prediction -- that asymmetry is the point. The model stays linear in the field in the warped units: the co-field and the atom contributions are read there, and the output says so.

```
model = CMFModel(fieldset=fs, method="krr", params={"lam": 0.1}, warp=2)
model.fit(table, y)                                  # the transform is learned with the fit
print(model.warp_.describe())                         # or "the identity", and why
pred = model.predict(test)                            # the median, in the original units
median, lo, hi = model.predict_warped_interval(test, level=0.9)
```

On the command line it is `cmf build --warp 2`; the transform travels with the saved model, so `predict` and the conformal interval report in the original units without being told.

Measured on the eight Sutherland sets and on the 38 sets of the Mittal collection, five random 70/30 splits each, with alpha and lambda chosen by the evidence on the training part: the central 90 % interval of the warped process against the plain process's and the conformal interval calibrated on the plain fit's folds, on the held-out molecules (`benchmarks/run_warped.py`, `benchmarks/results/warped/table.md` and `benchmarks/results/warped_mittal/table.md`). Two of the collection's 38 sets are not in the table, APZ and RYR, which have 16 and 17 molecules, too few for a 70/30 split with an interval.

collection	sets	warp kept	q2, plain	q2, warped	coverage, plain GP	coverage, conformal	coverage, warped	width, plain GP	width, conformal	width, warped
Sutherland	8	32% of splits	+0.484	+0.482	88%	89%	88%	3.00	3.17	2.96
Mittal	36	41% of splits	+0.102	+0.057	88%	90%	87%	4.93	5.73	4.83

The last column of each coverage group is what the gate reads. Over the 44 series the warped interval is closer to the promised 90% than the plain process's on 8 and further on 14 (1 and 2 on the Sutherland sets, 7 and 12 on the collection). The paired differences over the sets, with their 95 % bootstrap intervals: on the Sutherland sets coverage -0.2% [-0.5%, -0.0%] against the plain GP and -1.8% [-3.6%, +0.3%] against the conformal interval, width -0.044 [-0.133, +0.013] and -0.213 [-0.345, -0.080], q2 -0.002 [-0.012, +0.007]; on the collection coverage -1.1% [-2.1%, -0.2%] and -3.1% [-4.6%, -1.8%], width -0.099 [-0.170, -0.041] and -0.904 [-2.107, -0.242], q2 -0.045 [-0.086, -0.016]. The transform was kept on 32% of the Sutherland splits and 41% of the collection's, gaining +10.3 and +16.3 nats of evidence over the identity on average; where it was kept, q2 moved by -0.004 [-0.024, +0.013] on the 4 Sutherland sets concerned and -0.065 [-0.122, -0.025] on the 25 of the collection, and coverage by -0.5% [-0.8%, -0.1%] and -1.6% [-2.9%, -0.3%]. Coverage does not improve on more series than it worsens, so the warp stays an option (`--warp`) and not a default, as the item's own rule says.

What writing it found. Three sets of the collection -- PTC, DIAZEPAM and EV71 -- have no hydrogen-bond donor on any molecule, so the donor field is zero throughout. The choice of attenuation and ridge by the evidence that the error benchmarks share divided each field by the mean of its training diagonal, which is zero for such a field, so the kernel filled with NaN and the evidence was finite for nothing. The model's own rule, a scale of zero for a field that is zero everywhere, is now used there too; the Sutherland sets all have donors, so no stored number changed.

7.22 Three estimates of the error, compared

The program now offers three answers to *should I believe this number*, and they come from three different places: the **Gaussian-process posterior variance**, which is the kernel's own statement about how much of this molecule lies in the span of the training set; the **applicability domain**, which is a geometric statement about how far it is from its nearest training neighbours; and the **conformal interval**, whose normalised form widens where the model is extrapolating. They are in three different units — a variance, a distance in field space, a width in log units of activity — so they cannot be compared by their numbers. What can be compared is the order they put the molecules in, because that is what they are used for: which prediction do I act on, which do I measure first.

```
python cmf_cli.py build data.sdf --test-file test.sdf --domain \
       --conformal 0.8 --conformal-normalise --error-predictors
```

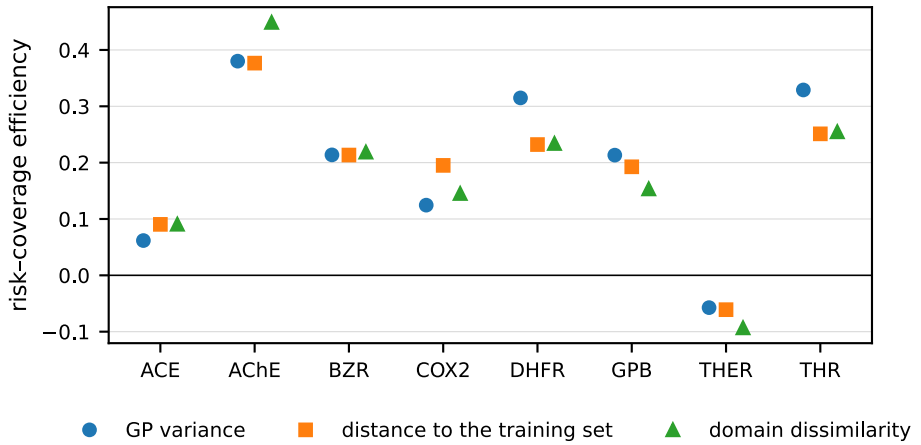
Each estimate is scored against the absolute error actually made on the test molecules, three ways. **Spearman's ρ** asks whether the order is right at all. The **risk–coverage efficiency** asks what is gained by acting on it: trust the most confident fraction, measure the RMSE of that fraction, and compare the area under that curve with two references — the *oracle*, which orders the molecules by the error they really made, and the flat curve of ignoring the estimate. One means the oracle, zero means no better than ignoring it, and a negative number means actively misleading. The **halves** are the crudest and most practical: the RMSE of the half it trusts against the half it doubts.

Measured, the three are indistinguishable. Eight Sutherland sets, five 70/30 splits each, α and λ chosen by the evidence on the training part, the conformal rule calibrated on that part's own out-of-fold errors:

estimate	ρ	efficiency	RMSE trusted	RMSE doubted	ratio
GP variance	+0.142	**+0.198**	0.835	1.024	1.38

estimate	ρ	efficiency	RMSE trusted	RMSE doubted	ratio
distance to the training set	+0.125	+0.186	0.853	1.009	1.31
domain dissimilarity	+0.133	+0.181	0.841	1.022	1.36
normalised conformal width	+0.133	+0.181	0.841	1.022	1.36

The GP variance is ahead by +0.011 in efficiency over the distance, with a 95 % interval of [−0.021, +0.044] — zero is inside it, and the choice between the three does not matter.



The risk–coverage efficiency of the three estimates on each of the eight Sutherland sets, mean over five splits. One would be the oracle that knows the error of every molecule, zero an estimate that carries no information. The three move together from set to set, and on THER all three are below zero.

Two things in that table deserve more attention than the ranking. The first is that the **normalised conformal width and the domain measure score identically, and must**: the normalisation divides the interval by the molecule's difficulty, which *is* its distance from the training set, so the width is a monotone function of the domain and cannot order the molecules differently. The conformal rule is justified by being calibrated — it covers what it says it covers, which neither of the others claims — and not by ordering. There are three estimates here, not four.

And the remaining two are nearly the same. Over the same splits the estimates agree with each other at a mean Spearman ρ of **+0.80 to +0.86** — the GP variance against the distance to the training set at +0.799, against the domain dissimilarity at +0.853 — while the conformal width against the domain measure is **+1.000**, exactly as the algebra says it must be. That is why the efficiencies in the table are indistinguishable: these are not three independent opinions about a molecule but one geometric fact, how far it sits from the training set, expressed in three sets of units.

The second is the size of all of them. The best captures about a fifth of what the oracle would, and the half it trusts has an RMSE of 0.84 against 1.02 for the half it doubts. That ratio of about 1.4 is real and useful — it is exactly the gain from measuring the confident half first — and it is nothing like a per-molecule error bar. On one of the eight sets (THER) every estimate comes out negative: an estimate of error is itself a model, and it can be wrong about a series. Use them to order molecules, and quote the conformal interval when a number has to be attached to one.

7.23 The model as the fast level of a hierarchy: a standard deviation in the units of the activity

A multiscale scheme puts a CMF model between an electronic-structure method and a coarser model: the fine level labels a few hundred structures, the model answers for the thousands the coarse model needs, and the coarse model turns them into a material property. For that the model must say, per structure, whether its answer can be used. The posterior variance of the Gaussian process (the *Gaussian process* row of the learner table) is the right quantity, but as it stands it is not usable for the purpose, for three reasons.

It is in the units of the kernel. The Gaussian process that matches the ridge fit has the prior covariance $s\,\mathbf{K}$ and the noise $s\lambda$, where s is the one amplitude the evidence profiles out in closed form (the section *Choosing α and λ by the evidence*): the mean squared whitened residual of the training set, $s = \mathbf{r}^{\mathrm{T}}(\mathbf{K}+\lambda\mathbf{I})^{-1}\mathbf{r}/(n-1)$ with $\mathbf{r}$ the residual about the fitted constant. The predictive variance of a new *measurement* at x is then

$$\sigma^2(x) = s\left[K(x,x) - \mathbf{k}^{\mathrm{T}}(\mathbf{K}+\lambda\mathbf{I})^{-1}\mathbf{k} + \frac{(1-\mathbf{k}^{\mathrm{T}}(\mathbf{K}+\lambda\mathbf{I})^{-1}\mathbf{1})^2}{\mathbf{1}^{\mathrm{T}}(\mathbf{K}+\lambda\mathbf{I})^{-1}\mathbf{1}} + \lambda\right] \qquad (7.12)$$

and its square root is in the units of the activity. The third term inside the bracket is the uncertainty of the constant mean that the ridge fits as its bias, marginalised under a flat prior (Rasmussen and Williams 2006, eq. 2.42 with one basis function); it is what makes a structure far from every training one uncertain about the *level* of the activity and not only about its shape. Without the last term, λ, the deviation is that of the underlying value rather than of a measurement, which is what two predictions of the same structure share.

Nobody had checked that it means what it says. On the folds, the standardised out-of-fold residual $z = (y - \hat{y})/\sigma$ should have a root mean square of one, and the interval of two standard deviations should hold about 95 % of the activities. The program reports both, and the factor that would make the root mean square one. A factor well above one says the model is surer than it should be, which is the usual finding when the kernel misses a part of the chemistry. The factor is reported and not applied.

A number is not a decision. A tolerance in the units of the activity turns it into one: every structure whose standard deviation exceeds the tolerance is marked as one to pass to the fine level (a DFT calculation, a measurement) instead of using the model's answer. The share the fine level can afford goes the other way, to the tolerance that sends it that many.

```
python cmf_cli.py build data.sdf --charges file --test-fraction 0.3 --calibration --save model.cmf
python cmf_cli.py predict model.cmf new.sdf --sd --tolerance 0.7 --out predictions.csv
```

`--calibration` prints the check on the folds and, when there is a test set, on it. `--sd` prints the prediction with its standard deviation and writes it to the CSV as `predicted_sd`; `--tolerance` adds the marks (`-> fine level` on the screen, a `defer` column of 0 and 1 in the CSV) and a line saying how many structures the model answers and, where the activities are known, the error on each side. In Python the same three things are `model.predict_with_sd(test)`, `model.calibration()` and `cmfqsar.surrogate.Handoff.of(sd, tolerance, y, mean)`; the deviation needs the linear kernel and the ridge learner, and a low-rank model refuses it, because the term of the mean needs the full training kernel.

Measured, the deviation is calibrated as it stands. Eight Sutherland sets, five random 70/30 splits each, α and λ by the evidence on the training part (`benchmarks/run_surrogate.py`):

set	n	test RMSE	mean σ	z rms, folds	z rms, test	inside 2σ
DHFR	361	0.751	0.679	1.10	1.08	93 %
COX2	282	0.811	0.825	0.97	0.99	93 %
BZR	147	0.610	0.620	1.03	0.98	94 %
ACE	114	1.512	1.378	0.99	1.15	89 %
AChE	111	0.809	0.830	1.05	0.96	94 %
THR	88	0.672	0.641	0.96	1.02	94 %
THER	76	1.635	1.605	1.03	1.07	93 %
GPB	66	0.767	0.722	0.99	1.10	92 %
mean		0.946	0.912	**1.02**	**1.05**	**93 %**

The root mean square of *z* is 1.02 on the folds and 1.05 on the held-out molecules, and two standard deviations hold 93 % of them against the 95 % of a Gaussian. The mean deviation is within 4 % of the error actually made on every set. So the scale that the evidence profiles out *is* the noise level of the series, and the correction factor, though reported, is not needed on these sets; applied to the held-out molecules it moves the root mean square from 1.05 to 1.03.

The tolerance rule does what it promises, and no more. With the tolerance set to the cross-validated RMSE of the training part, the error the model claims for a typical molecule, the model answers **61 %** of the held-out molecules at an RMSE of **0.90** and defers the rest, whose RMSE is **1.03**; the mean tolerance is 0.92, so the answered share's error sits where the rule said it would. Sending a fixed quarter of the molecules to the fine level gives 0.88 against 1.08. The separation is modest, and it must be: the section above showed that the posterior variance is the distance to the training set in other units and captures about a fifth of what an oracle would. On THER the deferred molecules are *better* predicted than the answered ones, as every estimate of the error was on that set. What calibration adds is not a better ordering but a fair size: the error the fast level reports for the structures it answers is the error it makes, which is what a coarse model needs in order to carry it forward.

On a hop set, where the hierarchy is real, the rule earns more. The sample of LiTraj used in the external check of the hop model — 1,801 hops of lithium ions in 393 crystals, each labelled by a bond-valence calculation, which is the fine level here — with ten folds that hold out whole structures and the ridge and the width of a Gaussian kernel chosen on an inner split of the same kind (`benchmarks/run_surrogate.py --hops`):

kernel	q^2	held-out RMSE (eV)	mean σ (eV)	z rms, inner folds	z rms, held out	inside 2σ	σ against the error (ρ)
linear, invariant local	+0.572	1.246	1.136	1.04	1.05	95 %	+0.20
Gaussian on the same rows	+0.623	1.169	1.053	1.01	0.99	96 %	+0.35

kernel	tolerance (eV)	answered	RMSE answered	RMSE deferred	answered at 25 % deferred	RMSE of that 25 %
linear	1.242	87 %	1.123	1.856	1.063	1.681
Gaussian	1.207	82 %	0.998	1.752	0.840	1.830

The deviation is calibrated here as well, for both kernels, which is what the scale absorbing the units of the kernel promises: the root mean square of z is 1.05 and 0.99, and two standard deviations hold 95 % and 96 % of the barriers. The rule now separates by a factor of 1.8 rather than 1.1: with the Gaussian kernel the model answers 82 % of the hops at an RMSE of 1.00 eV and sends back 18 % whose RMSE would have been 1.75 eV, and a fixed quarter sent back leaves 0.84 eV against 1.83 eV for the quarter. The reason is the data and not the method. A Sutherland series is congeneric, so every held-out molecule has close relatives in the training part and the deviations are all alike; the collection behind the hop sample covers every chemistry that holds lithium, so a held-out structure is often the only one of its chemical system, and the deviation says so. That is the situation a hierarchy is in, and it is where a calibrated deviation does its work: the hops the fast level keeps carry the error it claims, and the ones it returns are the ones the calculation was needed for.

The same deviation without the n by n kernel

Everything above needs the training kernel, factorised once. A set of a hundred thousand structures has no such kernel -- that is why the low-rank path of *Large training sets* fits in the space of a few hundred landmarks instead -- and until now that path could predict but could not say how sure it was: `surrogate()` refused it in those words. Plan item MS6 removes the refusal, and nothing of size n is built to do it.

The kernel of a low-rank model is $\Phi\Phi^T$, with Φ the $n \times m$ matrix of features against the landmarks, and every quantity the surrogate computes has an $m \times m$ form through the Woodbury identity. With $\mathbf{A} = \Phi^T\Phi + \lambda\mathbf{I}$ and $\mathbf{r}$ the residual about the fitted constant,

$$\mathbf{r}^T(\mathbf{K}+\lambda\mathbf{I})^{-1}\mathbf{r} = \frac{\mathbf{r}^T\mathbf{r} - (\Phi^T\mathbf{r})^T\mathbf{A}^{-1}(\Phi^T\mathbf{r})}{\lambda}, \qquad \mathbf{1}^T(\mathbf{K}+\lambda\mathbf{I})^{-1}\mathbf{1} = \frac{n - (\Phi^T\mathbf{1})^T\mathbf{A}^{-1}(\Phi^T\mathbf{1})}{\lambda}, \qquad (7.13)$$

so the scale s is the same mean squared whitened residual as before, the term of the unknown mean is (1 − boldsymbol{φ}$^T\mathbf{A}^{-1}\Phi^T\mathbf{1})^2$ over the second quantity, and the posterior variance is the one the low-rank path already had, with the exact self-kernel kept so that it does not fall to zero where the landmarks span nothing. The Gram the fit stores -- $\Phi^T\Phi$, $\Phi^T\mathbf{y}$, $\Phi^T\mathbf{1}$, n, Σ y and $\mathbf{y}^T\mathbf{y}$ -- holds everything these expressions need, which is why a model read back from a file can give its deviation without its training set.

```
model.fit_low_rank(train, y, landmarks, lam=0.1)
mean, sd = model.predict_with_sd(test)     # the same call as for the exact model
model.calibration(folds=10)                # on the same folds the exact path uses
```

`LowRankSurrogate` in `cmfqsar.surrogate` is the object behind it; `lowrank.cross_validate` returns the out-of-fold deviation with `with_sd=True`, and `lowrank.LowRank.joint_self` is the exact self-kernel of a table. The tests fix the claim the way the low-rank tests fix the fit: with the training molecules as the landmarks the kernel is reproduced exactly, so the scale, the mean, the deviation and the calibration must equal the exact path's to round-off, and they do.

Measured on the stored splits of the eight Sutherland sets -- the same five 70/30 splits, the same attenuation and ridge as the table at the start of this chapter, so the only thing that differs is the path -- with three landmark sets: every training molecule (the exact kernel computed another way), half of them and a quarter of them, drawn at random (`benchmarks/run_surrogate.py --low-rank`, `benchmarks/results/surrogate/table.md`).

landmarks	rank	z rms, folds	exact	z rms, test	exact	inside 2 sd	largest difference from the exact path
molecules	497	1.02	1.02	1.05	1.05	93%	1.1e-08
half	253	0.95	1.02	0.94	1.05	95%	6.0e-01
quarter	129	0.86	1.02	0.82	1.05	97%	1.2e+00

The averages are over the eight sets and five splits. The last column is what the gate reads: the largest difference from the exact path over every set and split, which is 1.1e-08 with the molecules as landmarks -- the same kernel computed another way, so round-off -- and 6.0e-01 and 1.2e+00 with half and a quarter of them. So the gate holds where the landmarks span the data, which is the reading's own claim, and the approximation loosens it in one direction: with half the molecules as landmarks the root mean square of z falls below the exact path's on every split, by 0.07 on the folds and 0.11 on the held-out molecules on average, and with a quarter by 0.16 and 0.23 -- the deviation of the approximate model is larger than its error. That is the term which keeps the variance from falling to zero where the landmarks span nothing being conservative by construction, not an error in reading it; the largest single-split differences, 0.60 and 1.20, are both GPB, where 46 training molecules leave a quarter at twelve landmarks.

What writing it found. Two faults of the low-rank path, both older than this item and both invisible on a set where every molecule carries every field. The exact fit scales each field by one over the mean of its normalised training diagonal; that diagonal is 1 for a molecule whose field is not identically zero and 0 for one whose field is -- no donor, no acceptor, no charge -- so on a set that holds such molecules the scale is above one. The low-rank path took one, which made it another model on exactly those sets: with the training molecules themselves as landmarks its kernel missed the exact one by 1 % on the ACE set, where 13 of 80 training molecules have no hydrogen-bond donor. And the self-kernel it kept for the variance counted every field as one for every molecule, where the normalisation leaves a zero field at zero, so a molecule without a donor had its deviation inflated by that field's whole share. Both are repaired -- the scale is now the exact path's, and `LowRank.joint_self` is the one definition of the self-kernel -- and the test that would have caught them, a series with esters and a nitrile among the acids, is in `tests/test_lowrank_surrogate.py`. The repair also reaches `cmf build --low-rank`, which had used a scale of one for choosing its ridge, and which passed over `--calibration` without a word; it reports the calibration now.

What this does not do is train on a hundred thousand structures. The path can -- its memory is the rank squared whatever the number of molecules, as *Large training sets* measured -- but the test case the plan names, the full LiTraj collection, is held in this repository as a sample of 1,801 hops, and the hop model of that sample is the rotation-invariant local kernel, which has explicit features and does not go through this path at all. The deviation at that scale is therefore shown to be *computable* here and not yet *measured*; the measurement needs the overlap kernel on a large set in one frame, and it is recorded as open.

7.24 What the model thinks the groups are: clustering the kernel

Every grouped validation in this manual splits the molecules by their Bemis–Murcko scaffold, and the reasoning is sound: random folds put a molecule's own scaffold-mates in the training part and greatly overstate the quality of a mixed series. But it rests on an assumption nobody checks — that a scaffold is what makes two molecules alike *to the model*. The model does not see scaffolds. It sees fields, and its notion of similarity is the kernel.

Spectral clustering partitions that similarity directly. The kernel is already a similarity matrix, so there is nothing to build: form the normalised Laplacian, take the eigenvectors belonging to its smallest eigenvalues, put the rows on the unit sphere and run k-means there [27]. Groups that are not linearly separable in field space are separable in those coordinates, which is the point of the construction. How many groups is decided by the **eigengap** unless it is given, and the gap is reported beside the answer so that a weak one can be seen for what it is.

```
python cmf_cli.py cluster data.sdf --charges file
python cmf_cli.py cluster data.sdf --clusters 5 --out clusters.csv
```

The command prints the clusters, the spectrum they came from, the activity of each, and the **adjusted Rand index** between them and the scaffold groups — the number that says whether the usual grouping groups by what the model actually responds to.

The two partitions do not agree, and it matters

Measured on the eight Sutherland sets, with both partitions cut into the same number of groups — holding out one of two clusters removes half the training set and holding out one of nine does not, so a difference in granularity would be read as a difference in difficulty:

	q^2
random 10-fold	+0.508
leave one **scaffold group** out	+0.271
leave one **field cluster** out	+0.057

and the adjusted Rand index between the two partitions averages **+0.140**, which is weak agreement: what the chemist calls a series and what the kernel finds similar are largely different things. On BZR they are different enough that holding out a field cluster takes q^2 to −0.740.

The consequence is uncomfortable and worth stating. This manual recommends the leave-one-chemotype-out figure as the more reliable one, and against random folds it is — 0.271 against 0.508. But it is not the hardest reliable test available: holding out a group of the model's own similarity costs another 0.214, because a scaffold group leaves molecules the kernel still considers close in the training part. Quote the scaffold figure, by all means; just do not read it as the worst case.

7.25 Where the validation appears

The three checks of *Building a model* — y-randomisation, the bootstrap intervals and the applicability domain — are set in one place and reported in another, which is worth stating plainly because there is no button and no separate window for them.

Setting them: the *Validation* row at the foot of the model panel's second group, directly under *External CV*. Three controls: **shuffles** (0 = off), **bootstrap** (0 = off), and the **applicability domain** tick. They are settings, not actions — choose them before pressing **Build model**, and they run as part of that build, after the fit and the test prediction.

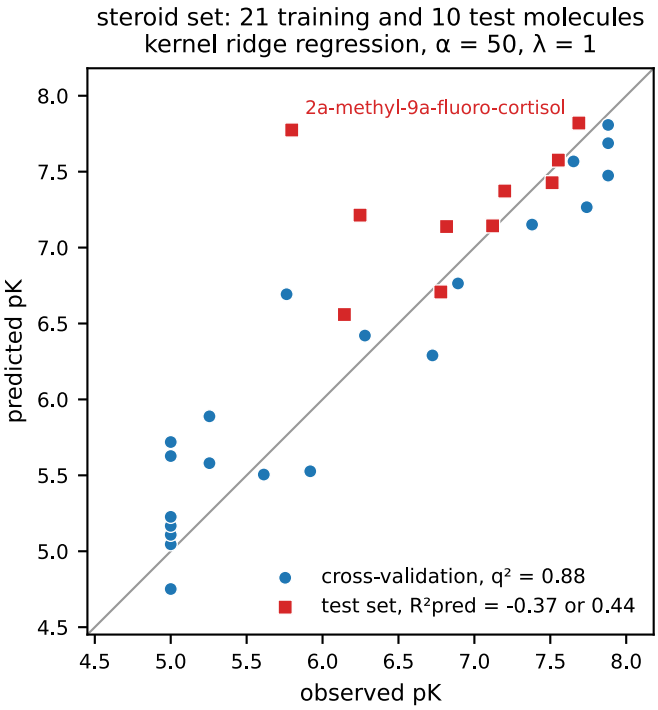
Reading them: the *Statistics* tab of the results panel, where they are appended below the usual figures under a heading *Validation*. The names of the molecules that fell outside the applicability domain go to the *Log* tab instead, with their similarity values, so that the statistics stay a summary and the list of molecules outside the domain stays with the rest of the log.

Three cases produce no validation block at all, and each says why rather than leaving a blank:

- all three controls at their defaults — nothing was asked for;
- a classification or one-class model — y-randomisation and the bootstrap are implemented for regression only, and the block says so;
- the autocorrelation kernel source — the applicability domain is measured in the field kernel, which that source does not provide.

7.26 Statistics reported

Regression: r^2 and RMSE for the fit, q^2 and RMSEcv for the cross-validation, R^2pred and RMSEpred for the test set in both variance conventions, q^2ex and RMSEcvex for the external CV. Classification: accuracy, balanced accuracy, sensitivity, specificity, Matthews correlation, AUC and the ROC curve, each for the fit, the cross-validation and the test set. Screening: AUC, enrichment factors at 0.5, 1, 5 and 10 %, the best achievable enrichment at each of those fractions, BEDROC and RIE.



Observed against predicted for the kernel ridge model of the steroid set at the best point of the grid of Attenuation, regularisation and the grids: the cross-validated predictions of the training molecules and the predictions of the ten test molecules, with the line of a perfect prediction. The two values of R²pred are the two conventions, against the variance of the test set and against the training mean; the test affinities span a narrow range, so the first is negative while the second is not. The molecule named is the only one with a fluorine, an atom the training set does not contain. The Scatter plot tab shows the same points and names the molecule under the cursor.

7.27 Saving and reusing

File → Save model... writes a `.cmf` file (joblib) holding the field set, the kernels' definition, the fitted coefficients, the training molecules, every setting needed to predict later, and the record of how the training set was prepared that *What has to match between fitting and predicting* describes. `cmf_cli.py predict model.cmf new.sdf --out predictions.csv` applies it, after checking the new molecules against that record; the GUI loads it with *File → Load model...* and writes the record's summary to the log.

7.28 A report to hand to a colleague

A `.cmf` file is for the program. The person who has to judge the model — a colleague, a reviewer, a supervisor — will never start the application, and what they need is one file they can open.

```
python cmf_cli.py report model.cmf --out model.html
```

That writes a single self-contained HTML page: every picture is inside it, so it survives being emailed, attached or put on a share, and it opens in any browser and prints to PDF. It has seven sections, in the order a sceptical reader asks for them.

- **What this model is** — fields, α, learner, kernel, how the charges were made.
- **How well it fits and predicts** — the statistics with the observed-against-predicted plots beside them, because a q² is a summary of that picture and not a substitute for it.

- **Whether to believe that** — the validation that was actually run: y-randomisation, the bootstrap intervals, the applicability domain, the conformal level, the comparison of error predictors. **A check that was not run is reported as not run**, never silently omitted, which is the difference between a report and an advertisement.
- **What the model rests on** — the mean absolute contribution of each field, which is what separates a model standing on its geometry from one standing on a single field.
- **The co-fields**, projected onto the three coordinate planes from fixed viewpoints: the strongest signed value along each viewing axis, with the training atoms as grey dots for scale. A projection rather than an isosurface, because a report has no camera and its reader has no mouse.
- **The extremes of the series** — the most and least active training molecules, each decomposed over the fields by exact Shapley values and over its own atoms by the contribution rule.
- **The pharmacophore** read out of the co-fields, and the provenance of the model file.

Everything comes out of the model file, so a report costs no refitting and **cannot disagree with the model it describes**. Missing parts are reported rather than causing a failure: a nonlinear kernel has no co-fields, no atom contributions and no pharmacophore, and the report says so in place of each and keeps the rest; a model fitted without a test set reports no R^2pred.

`--no-cofields` leaves out the pictures, which are the slow part — a report of a 50-molecule model takes about three seconds with them and under one without — and `--spacing` sets how finely they and the pharmacophore are computed. `--open` shows it in the browser straight away.

8 Reading the model

A CMF model is readable in a way a grid model is not, because the prediction can be taken apart exactly. Reference [6] states three decomposition rules, and the program implements all three.

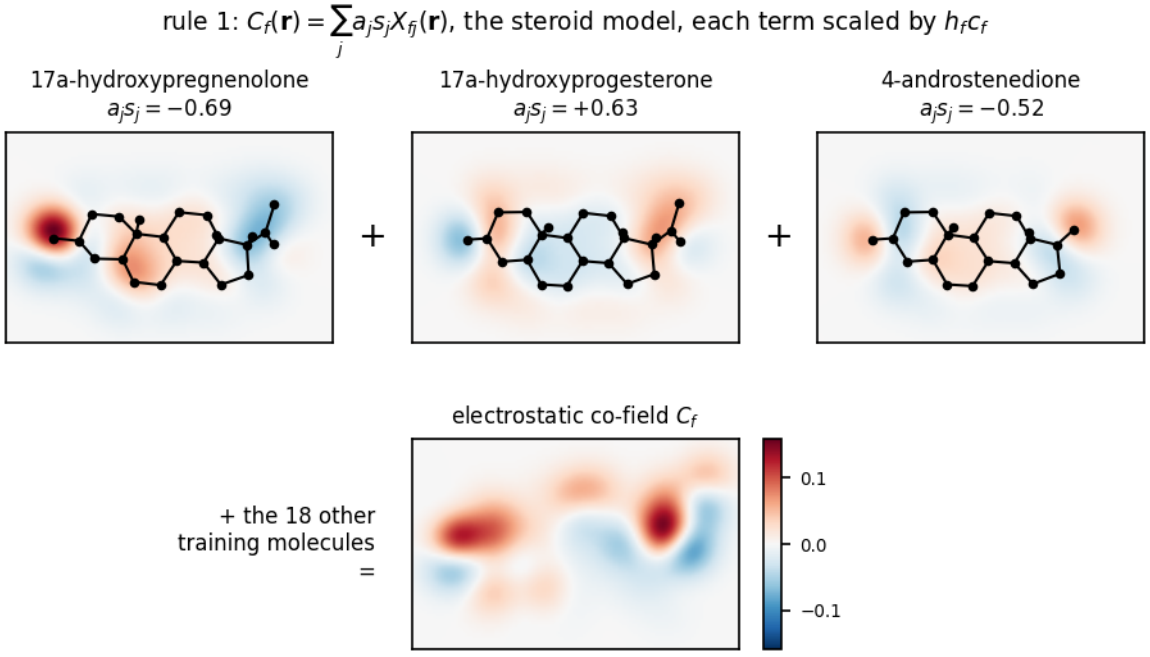
8.1 The three decomposition rules

Every model the program fits is a sum of kernel values with the training molecules, and the three rules below follow from that one fact. Reference [6] states them for the kernel of overlap integrals. The program keeps them for every model that is still such a sum, and it says so when a model is not: a nonlinear kernel layer keeps rule 2 and loses rules 1 and 3.

Rule 1: the co-field is a weighted sum of the fields of the training molecules.

$$C_f(\mathbf{r}) = \sum_j a_j \, s_j \, X_{fj}(\mathbf{r}) \tag{8.1}$$

This is Eq. (2.8) restated. The weights are the fitted dual coefficients a_j, one per training molecule, times the normalisation s_j of its field when the kernels are normalised. Every lobe of a co-field therefore comes from named molecules: a positive lobe is where the fields of the molecules with a positive weight are strong and the fields of the molecules with a negative weight are weak. The scheme below draws the three terms of the largest weight for the electrostatic co-field of the steroid model, and the co-field that they and the other eighteen terms add up to. In the program the co-field is the **Compute co-field** button, described in *Co-fields* below; the weights are `model.coef` in the library, and the *Contributions* tab lists them beside the products of rule 2.

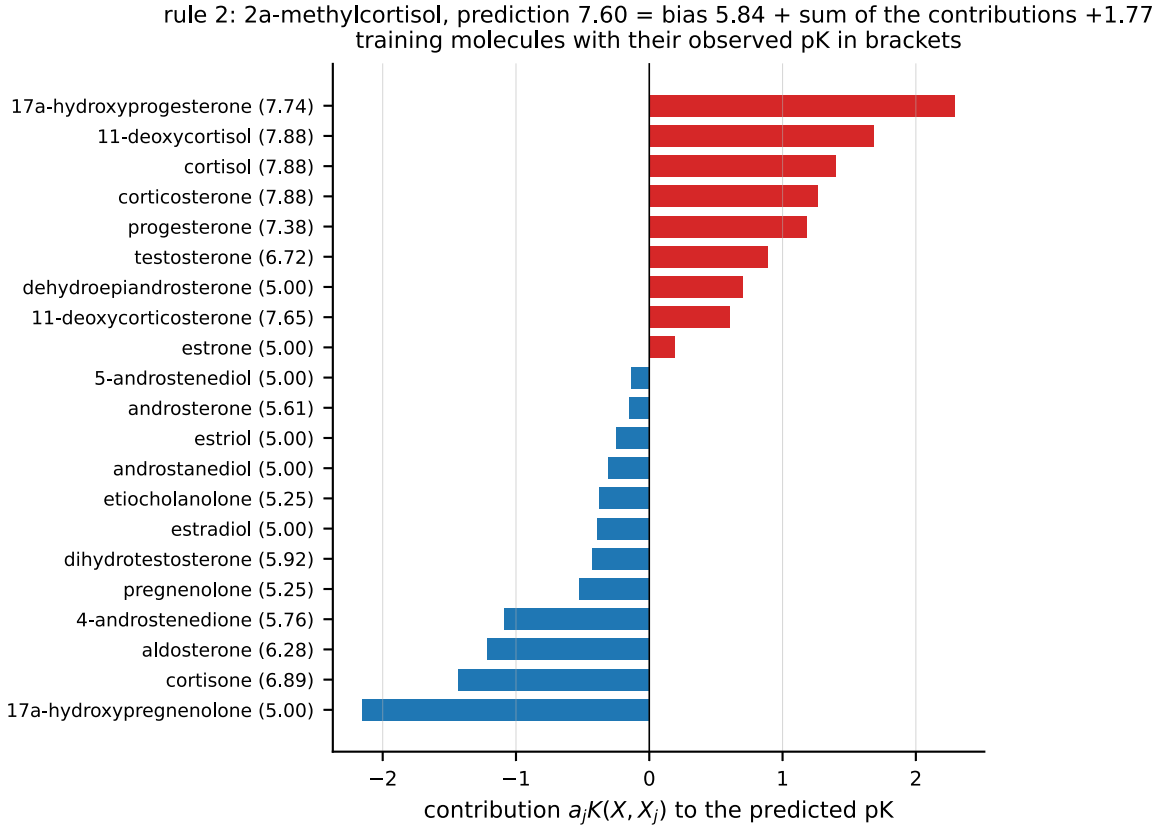


Rule 1 for the electrostatic co-field of the steroid model of this chapter: the three training molecules of the largest weight, each field drawn times its weight with the skeleton of the molecule over it, and the co-field that all 21 terms add up to, on one colour scale. The weight of a training molecule is its dual coefficient times the normalisation of its field, and the sign of the weight, not the activity itself, decides whether its field is added or subtracted.

Rule 2: a prediction is the bias plus one contribution per training molecule.

$$\hat{y} = b + \sum_j a_j K(X, X_j) \tag{8.2}$$

This is Eq. (2.7) restated. $K(X, X_j)$ is the kernel between the molecule and training molecule j, so the contribution of a training molecule is its similarity to the molecule times its weight, which is why the rule is also called the similarity decomposition. The rule holds for every kernel, a nonlinear layer included, because the layer changes the kernel and not the form of the sum. It is the first thing to read when a prediction surprises you: a prediction resting on one or two training molecules is a warning, and a support vector machine makes the sum short on purpose, since most of its coefficients are zero. The scheme shows the contributions of the 21 training molecules to one test molecule of the steroid set, sorted: the molecules the fit weighted positively and that resemble the test molecule pull the prediction up, the others pull it down. The *Contributions* tab lists them, `cmf report` prints the largest, and `model.training_contributions(mol)` returns them.



Rule 2 for one test molecule of the steroid set, the one the model of this chapter predicts highest: the contribution of each of the 21 training molecules, sorted, with the observed affinity of the training molecule in brackets; the bias and the sum of the bars are in the title. The bars are not an activity ranking of the training set, because the similarity to the test molecule is a factor in every term.

Rule 3: a prediction is a sum over the atoms of the molecule.

$$\hat{y} = b + \sum_k y_k \tag{8.3}$$

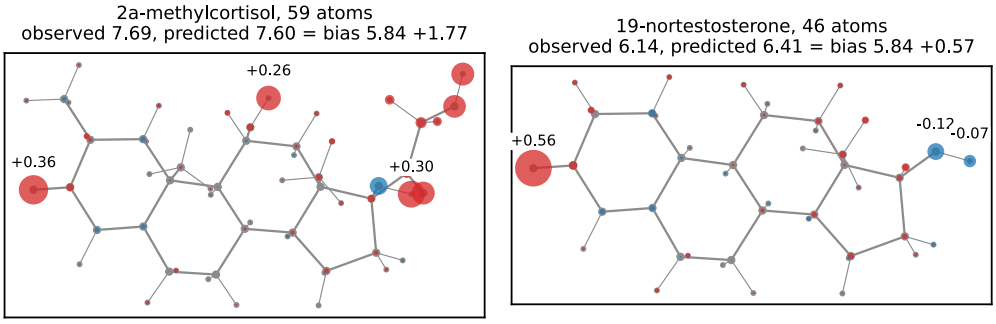
The field of a molecule is a sum of Gaussians centred on its atoms, and the overlap kernel is a sum over pairs of atoms, so the contribution of every training molecule splits further into one term y_k per atom k of the molecule being predicted, the overlap of that atom's own Gaussians with the

co-fields:

$$y_k = \sum_f \frac{h_f c_f}{\sqrt{K_f(x,x)}} \int X_{fk}(\mathbf{r})\, C_f(\mathbf{r})\, \mathrm{d}\mathbf{r}, \qquad X_{fx}(\mathbf{r}) = \sum_k X_{fk}(\mathbf{r}) \tag{8.4}$$

Here X_{fk} is the part of the field of the molecule that atom k carries, and the rest is Eq. (2.9) with the sum over the atoms taken outside the integral. The contribution of a substituent or a fragment is the sum over its atoms. This is the rule for design, because it says which part of a molecule helps and which part does not. It needs a kernel that is a sum over atom pairs, which the overlap kernel and the rotation-invariant autocorrelation kernel are and a nonlinear layer is not; under a layer the Shapley values of *Shapley values* below take its place. The scheme draws the atom contributions of the two test molecules the steroid model predicts highest and lowest, as Figure 3 of [6] draws them: red spheres raise the predicted affinity, blue ones lower it, and the area is the magnitude. The 3D view draws the same spheres for the selected molecule, and `model.atom_contributions(mol)` returns the values, per field with `per_field=True`.

rule 3: the prediction as a sum over the atoms; red raises the predicted pK, blue lowers it, the area is the magnitude



Rule 3 for the two test molecules of the steroid set that the model of this chapter predicts highest and lowest, drawn as Figure 3 of [6]: one circle per atom, hydrogens included, red for a positive contribution and blue for a negative one, with an area proportional to the magnitude and the three largest values written out. The sums in the titles are the predictions minus the bias. The largest contributions sit on oxygen atoms, where the electrostatic co-field has its lobes.

What to do with the map. An atom with a negative contribution is a candidate for removal or for replacement, an atom with a positive one is what to keep, and the sum over a substituent is what the model says that substituent is worth at its position. The predicted effect of exchanging one group for another at a position is the difference of the two sums, one integral per pair of groups and no refit: that is the measure behind the table of *Which groups are interchangeable: a bioisostere table*, `cmf design` in *Designing the next compound* ranks candidate groups by the prediction for the whole molecule, and on a series without a common frame the invariant route of that chapter reports how much of a candidate's prediction comes from the group that was put in. One caution: the contributions describe the molecule as it is posed in the model's frame. Taking a group away changes the normalisation of the field and may change the conformation, the alignment and the charges of what is left, so the map says where to act, and the prediction for the edited molecule says whether the action worked. The same split written as a file for another program to read is *A map as an output mode* in *A site of the cell as the sample* below, which covers molecules and cells alike.

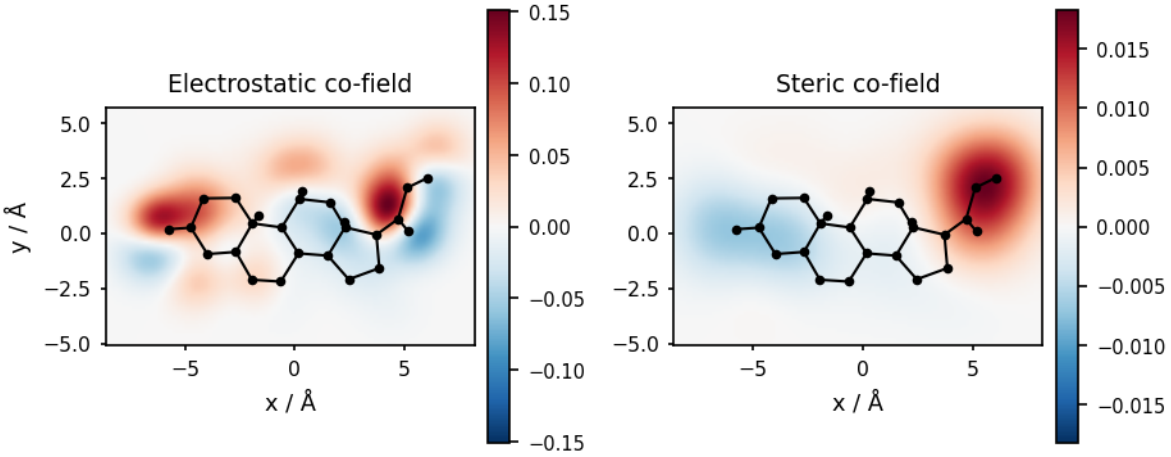
The two sums are the same number: the contributions of the training molecules and the contributions of the atoms both add up to the prediction minus the bias, and a test asserts it. For a classification model the coefficients carry the sign of the class, $a_j = \alpha_j\, y_j$, and the three rules read

the same way with the active class in place of the higher activity.

8.2 Co-fields

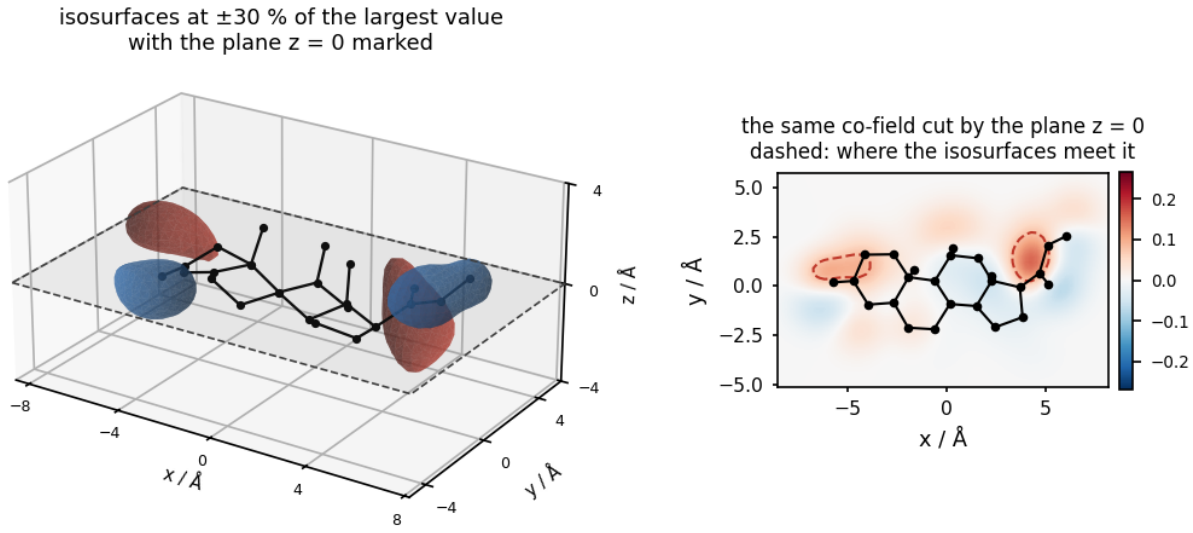
The co-field of Eq. (2.8), restated above as rule 1, is the model as a function of space: a weighted sum of the training molecules' fields with the fitted dual coefficients as weights, each divided by the norm $s_j = K_f(j,j)^{-1/2}$ of its own field when the kernels are normalised, which they are by default. The prediction of any molecule is then the overlap of its own field with the co-field, field type by field type. Press **Compute co-field** and the program evaluates it on a grid around the training set; choose the field with the *Field* selector and set the *iso level* slider, which is a percentage of the largest absolute value rather than an absolute number, so the surfaces stay meaningful when you change field.

What the lobes mean: a positive co-field lobe is a region where *more* of that field raises the predicted activity. For the electrostatic field that means positive charge is favoured there, for the steric field bulk, for the hydrophobic field lipophilicity, and for an indicator field it means an atom of that type is favoured at that position.



The electrostatic and the steric co-field of a model of the steroid training set (α = 1, λ = 1, q² = 0.79), cut in the mean plane of the series, with corticosterone, the most active training molecule, drawn over them. Red marks a region where more of that field raises the predicted affinity: positive charge for the electrostatic field, bulk for the steric one. The lobes sit at the two ends of the skeleton, which is where the series varies.

Isosurfaces and plane cuts. The 3D view, like the figures of the papers, draws a field as two closed surfaces, the places where it equals plus and minus the iso level, and everything weaker than that level is left invisible. The maps of this manual cut the same function with one plane and colour every point of the plane by its value, so weak regions are visible and anything off the plane is not. The two are the same object drawn two ways; the figure below shows the electrostatic co-field of the previous figure both ways, and the dashed lines on the map are where the two isosurfaces cross the plane. A weak but wide lobe that a map shows may fall below the iso level in the 3D view, and lowering the level with the slider brings it back.

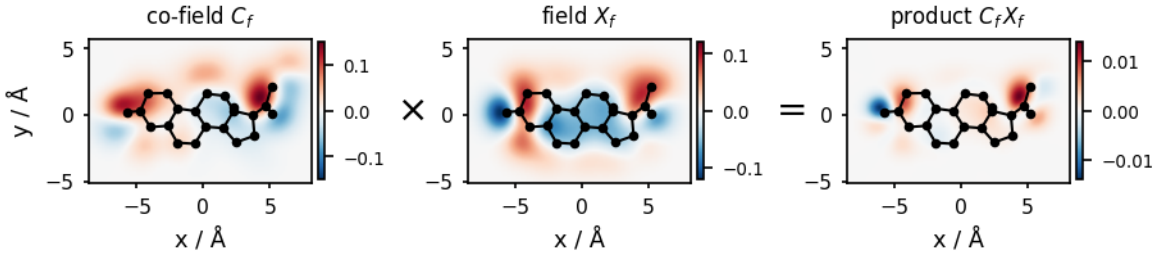


The same electrostatic co-field as the previous figure, drawn as the application draws it and as this manual draws it. Left: the isosurfaces at plus and minus 30 per cent of the largest absolute value, red positive and blue negative, with corticosterone inside them and the mean plane of the series marked; the lobes are the regions the iso level encloses, and nothing weaker is shown. Right: the cut of that plane through the same function, coloured by value, with the dashed lines where the two isosurfaces meet the plane. The map shows the whole plane, the surfaces show the whole space above one level.

Two further maps use the same grid, for the molecule currently shown:

- the **molecular field** $X_f(\mathbf{r})$ of that molecule, in the same colours. The two share a colour scale, so only the drawing style tells them apart: the style box is `auto` by default, which draws this one as a mesh while a co-field is on screen and solid when it is the only surface shown;
- the **overlap** $C_f(\mathbf{r})\,X_f(\mathbf{r})$, whose integral *is* that field's contribution to the prediction by Eq. (2.9), so the map shows where the prediction comes from.

$$\text{contribution of field } f \text{ to the prediction } = h_f c_f \int C_f(\mathbf{r})\,X_f(\mathbf{r})\,\mathrm{d}\mathbf{r}$$
$$= +0.65 \text{ for the electrostatic field of 19-norprogesterone}$$



The prediction as an overlap, for the electrostatic field of one test molecule of the steroid set and the model of the previous figure: the co-field of the model, times the molecule's own field, gives the product whose integral over space, with the field's scale and mixing coefficient, is that field's contribution to the predicted affinity. The prediction is the sum of these contributions over the fields plus the bias. This is Fig. 1 of [2] drawn with the program's own maps; the application shows the same three maps in 3D for the molecule on screen.

File → Export current co-field (Gaussian cube) writes a cube file readable by PyMOL, VMD or Jmol, which is the way to put a co-field into a publication figure.

How much of it is noise

A co-field is read as a fact about the chemistry. It is a linear map of the measured activities, so part of its shape is a fact about the measurement error instead, and the program can say which part in closed form.

The dual coefficients are a linear map of the activities — with a bias, the bordered system of Eq. (2.6), whose inverse's top-left block is the matrix below — so activities carrying independent noise of variance σ^2 give coefficients carrying covariance $\sigma^2\,\mathbf{G}\mathbf{G}^{\mathrm{T}}$, and the co-field, being linear in them, carries

$$\operatorname{Var}[C_f(\mathbf{r})] = \sigma^2 \left\| \mathbf{G}^{\mathrm{T}} \mathbf{x}_f(\mathbf{r}) \right\|^2, \qquad x_{fj}(\mathbf{r}) = s_{fj} X_{fj}(\mathbf{r}) \tag{8.5}$$

— one matrix product per block of grid points, no resampling and no refitting. The **hide noise** tick of the 3D view then draws only what lies more than two standard deviations from zero, and `pharmacophore --min-z 2` reads features off the same masked map, so that a lobe the noise could have produced never becomes a feature. The report (`report`) does both by default.

The noise level σ is the model's own out-of-fold error, RMSEcv, which is the assay's noise and the model's error together; `--sigma` sets it directly.

What this is, precisely. It is not the Bayesian posterior variance of the co-field, and the difference is worth stating. Under the Gaussian process whose kernel this is, the prior on the field weight function is white noise on L^2: its value at a point is not a random variable at all, and the pointwise posterior variance is infinite everywhere. What the prior does allow is the variance of $\langle w, g \rangle$ for a probe field g of non-zero width — which is exactly the GP variance of a prediction for a one-atom molecule placed there, and the program already computes that. The quantity above is the other half of the question: the molecules and the kernel are held fixed, and the noise in the activities is propagated. It answers *how much of this lobe would survive the error in the measurements*, which is the question a co-field picture provokes.

Measured, the mask keeps about a third of the picture, and that third is the part that is stable. Eight Sutherland sets, co-fields on a 0.8 Å grid (`benchmarks/results/cofield_error/table.md`); the training set is then resampled with replacement thirty times and refitted, and a voxel counts as stable when the sign of its co-field survives that — a harder perturbation than the formula models, since it moves the molecules too:

kept by the mask, of what would have been drawn	**30 %**
of the kept part, holds its sign under resampling	**97.8 %**
of the cut part	**90.6 %**
difference	**+0.072**, 95 % interval [+0.064, +0.083], in the same direction on 8 of 8 sets

One more figure, for completeness: the bootstrap moves the co-field only about 0.6 times as far as the closed form allows for, so cutting at two standard deviations is really cutting at about 3.2 of the bootstrap's — the threshold is conservative, which follows from taking σ to be the out-of-fold error. What it gets right is *where* the uncertainty is, and the two orderings agree across the grid at a rank correlation of +0.82. That is what a mask needs, since a mask is an ordering with a cut-off.

A co-field at a point, and what an atom there would do. A co-field is a value per unit of field, which is not a number anyone can check against anything: a field is not something one can put

somewhere. An atom is. `cofields.atom_effect` gives the overlap of the co-field with one atom's own Gaussian, which is the same closed-form integral the kernel sums over atom pairs, so the answer is in the units of the activity -- ppm for a shift, log units for an affinity -- and says what adding that atom there would add to the prediction. For a continuous indicator field the atom of its type carries a weight of one, so the number is directly "what one oxygen here does"; for the electrostatic field the weight is a partial charge and has to be given. `cofield_at_points` reads the co-field itself at a list of places, `cofield_covariance_at` and `atom_effect_covariance` give the error that the noise in the activities allows, and `contrast_at` compares two places with the covariance between them taken into account, which matters because two points a few angstrom apart are built from the same training fields and their errors are far from independent. Plan item LP3 uses all of this to read a co-field at the ortho, meta and para positions of a ring; nothing in it is specific to that use.

The co-field of a nonlinear model: one per molecule

A model with a kernel layer (*Nonlinear kernel layers and multiple kernel learning*) predicts

$$y(X) = \sum_j a_j \, g(K(X, X_j),\, K(X, X),\, K(X_j, X_j)) + b, \tag{8.6}$$

where K is the linear joint kernel of Eq. (2.7) and g is the layer: the identity for the linear model, $\exp(-\gamma\, d^2)$ for the Gaussian layer, the arc-cosine function for the arc-cosine layers. The prediction is no longer the overlap of the field with one fixed co-field, which is why the program refuses to draw one. It is still a differentiable function of the field, and its derivative with respect to the normalised field of type f, taken at molecule X, is a field:

$$C_{f,X}(\mathbf{r}) = \sum_j a_j \, g_s(X, X_j)\, s_{fj} X_{fj}(\mathbf{r}) \;+\; 2\Big[\sum_j a_j \, g_n(X, X_j)\Big] s_{fX} X_f(\mathbf{r}), \tag{8.7}$$

with $g_s = \partial\, g \,/\, \partial\, K(X, X_j)$ and $g_n = \partial\, g \,/\, \partial\, K(X, X)$. This is the **local co-field**: the co-field of the linear model tangent to the nonlinear one at X. The prediction of a molecule close to X changes by the overlap of $C_{f,X}$ with the change of its normalised field, summed over the field types with the usual factors $h_f\, c_{f'}$ and nothing else. The two derivatives are closed forms:

layer	g_s	g_n
linear	1	0
Gaussian, $g = e^{-\gamma\, d^2}$	$2\gamma\, g$	$-\gamma\, g$
arc-cosine, costheta = $K(X,X_j)/\sqrt{(K(X,X)K(X_j,X_j))}$	$(\pi - \text{theta})/\pi$	$\sqrt{(K(X_j,X_j)/K(X,X))}$ sintheta / 2π

For the linear layer Eq. (8.7) is the ordinary co-field whatever X is. For the Gaussian layer it is the ordinary co-field with every training molecule weighted by its kernel similarity to X, and with the field taken relative to X's own: the model reads a molecule through the training molecules that resemble it. The second term lies along X's own field. With cosine-normalised kernels, which are the default, $K(X, X)$ does not change when the normalised field moves, so the term has no effect on any prediction and is left out, as the ordinary co-field leaves out the dependence of the normalisation on the field; without normalisation it is kept. The unit test moves one atom's weight by a small amount and checks that the prediction moves by the overlap of Eq. (8.7) with the change of the normalised field, for every layer and both conventions.

Two things are built on the local co-fields of the training molecules:

- **the spread.** Their mean, point by point, is the one co-field to draw for a nonlinear model; their standard deviation is a map of where the effect of field depends on the rest of the molecule. It is zero everywhere for a linear model and large where the model is not additive. Both are read off the same matrix the noise band of the previous section uses, the normalised fields of the training molecules at the grid points, so the cost is that of one co-field with a matrix product per block of points;
- **the non-linearity index** of a field type,

$$\nu_f = 1 - \frac{\|\bar{C}_f\|^2}{\frac{1}{n}\sum_X \|C_{f,X}\|^2}, \tag{8.8}$$

the share of the mean squared norm of the local co-fields that their spread around the mean makes up: 0 for a linear model, 1 when the local co-fields have nothing in common. The norms are overlap integrals, and the local co-fields are combinations of the training fields, so the index is read off the Gram matrix $\mathbf{B}\,\mathbf{K}_f\,\mathbf{B}^{\mathsf{T}}$ with $\mathbf{B}$ the coefficient matrix of Eq. (8.7) and $\mathbf{K}_f$ the training kernel of the field, which the model keeps; nothing is evaluated on a grid. The mean cosine between the local co-fields of two training molecules is reported beside it, and a joint index takes all the fields together, weighted as the joint kernel weighs them.

`cmf cofield --kind local --molecule FILE` writes the local co-field at that molecule as cube files, and `--kind spread` writes the spread and the mean (`<prefix>_spread_<field>.cube` and `<prefix>_spreadmean_<field>.cube`) and prints the index of every field; in Python the three are `cmfqsar.nonlinearity.local_cofield_coefficients`, `cofield_spread` and `nonlinearity_indices`. The application does not draw them yet.

What it measures, and what it does not. The maps describe the tangent models, not the chemistry by themselves. The Gaussian layer makes every model nonlinear to a degree set by its bandwidth, so the spread is nonzero even when the measured relation is additive, as long as γ is finite. Three checks go with the maps: the linear model's q^2 beside the layered one (if the linear kernel predicts as well, the spread is a property of the kernel), the stability of the maps under refitting on part of the set, and a target made non-additive on purpose, whose index must come out higher. Second derivatives, how the effect of field at one place depends on the field at another, are a function of two points and not a field; they are not drawn here.

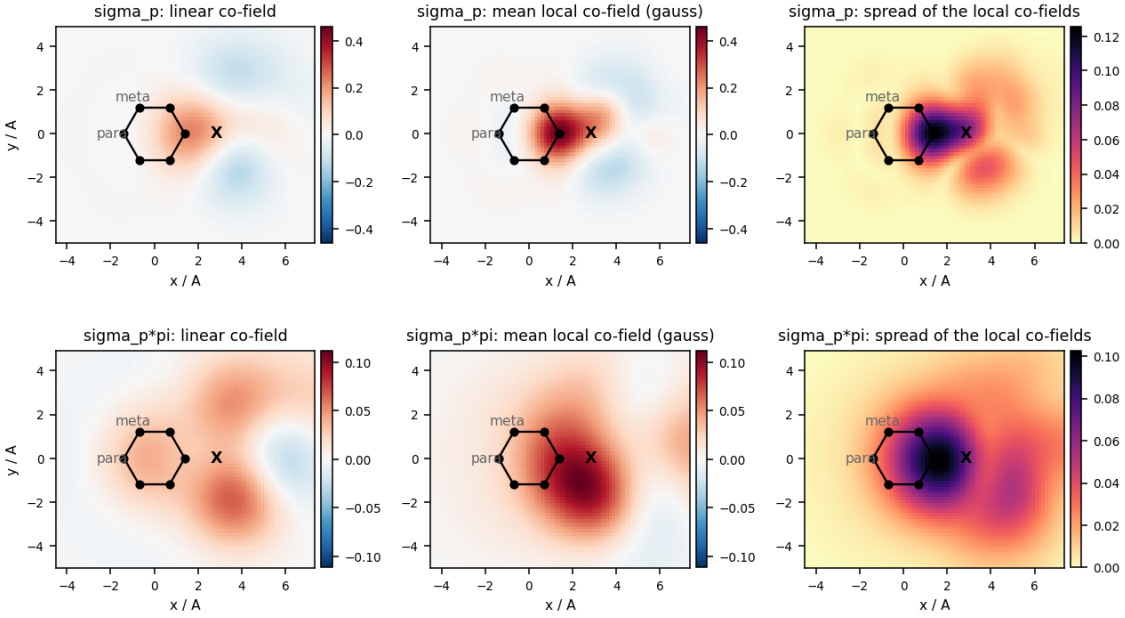
Measured (`benchmarks/run_nonlinearity.py`, `results/nonlinearity/`), on the Hammett set of `benchmarks/run_hammett.py`, the substituted benzenes in one frame: one conformer per substituent, Gasteiger charges, the five physicochemical fields, α, λ and γ by 10-fold cross-validation. The fourth target is $\sigma_p \cdot \pi$ on the 69 substituents that have both, a product of two constants and therefore quadratic in the fields if each factor is linear in them.

target	n	q² linear	q² Gaussian layer	γ	index, joint	index, electrostatic	spread map, stability	mean map, stability
σm	95	0.70	0.80	1	0.20	0.18	1.00	0.99
σp	95	0.66	0.78	0.5	0.10	0.08	0.99	0.99
π	69	0.72	0.88	1	0.32	0.41	0.87	0.54
σp · π	69	0.19	0.44	1	0.41	0.42	0.96	0.80

Stability is the mean correlation between the maps of five refits on random four-fifths of the set. Four things follow. The index orders the targets as it should: the product made non-additive on

purpose has the largest index, 0.41, and σ_p the smallest, 0.10, with a mean cosine of 0.94 between its local co-fields, so its model is close to additive and its mean local co-field is the linear co-field with slightly sharper lobes (figure). The constants themselves are not linear in the *normalised* field: the Gaussian layer gains 0.10 to 0.16 in q^2 on all three, most on π. One likely reason, not tested here, is that π grows with the size of the substituent while a cosine-normalised kernel does not see size. The spread maps survive refitting on three of the four targets at a correlation of 0.96 or better; on π the chosen λ is 10^{-4}, the fit is close to interpolation and the mean map changes from refit to refit (0.54), which is the stability check doing its job: that map should not be read. And the spread of σ_p sits at the ipso carbon and the first atom of the substituent, which says that the effect of charge there depends on what the rest of the substituent is, while for the product it covers the whole substituent region.

Electrostatic field, ring plane: the co-field of the linear model, and the mean and the spread of the molecule-dependent co-fields of the Gaussian layer



The electrostatic field in the ring plane, for σp (top) and for the product σp · π (bottom): the co-field of the linear model, the mean of the local co-fields of the Gaussian layer, and their spread over the 95 (69) training molecules. For σp the mean local co-field is the linear co-field, slightly sharpened, and the spread is confined to the ipso position; for the target made non-additive on purpose the mean map differs from the linear one and the spread covers the substituent.

Second derivatives: the pair-interaction field and the interaction modes

The local co-field says what more field at one place does. Whether that depends on the field at another place is a second derivative. Taken with respect to the field at two points it is a function of two points, not a field, and on its diagonal it carries a Dirac delta; so the program takes it with respect to the amplitudes of **probe atoms** instead: Gaussians of the field's own width added to the molecule's raw field, one at each place, with amplitudes varepsilon$_k$. With X_f + Σ_k varepsilon$_k$ p_k in place of the field, the prediction is an ordinary function of the amplitudes, and the chain rule through the layer gives

$$\frac{\partial y}{\partial \varepsilon_k} = \sum_j a_j [g_s S_{jk} + g_n n_k], \qquad \frac{\partial^2 y}{\partial \varepsilon_k \partial \varepsilon_l} = \sum_j a_j [g_{ss} S_{jk} S_{jl} + g_{sn}(S_{jk} n_l + n_k S_{jl}) + g_{nn} n_k n_l + g_s S_{jkl} + g_n n_{kl}], \tag{8.9}$$

where S_{jk} and S_{jkl} are the derivatives of the linear joint kernel $K(X, X_j)$ with respect to the amplitudes, n_k and n_{kl} those of the self-kernel $K(X, X)$, and the g's the derivatives of the layer. All of them are closed forms in three overlap integrals: the probe with the molecule's own field, the probe with each training field, and probe with probe (the docstring of `cmfqsar/nonlinearity.py` writes them out). The normalisation of the kernels is differentiated exactly rather than argued away, and the unit tests check the result against second differences of the prediction, with the probes on the molecule's own atoms and at arbitrary points in space, for both conventions. Three things follow from the formula. A linear model without normalisation has no second derivative at all. With cosine normalisation even the linear model has one, because a normalised field cannot grow without bound: that is a non-additivity of the kernel, not of the data, and the maps show it. The arc-cosine layers are refused: their kernel has a cusp where a molecule meets a training molecule, and the second derivative diverges there.

Three readings of Eq. (8.9) are implemented:

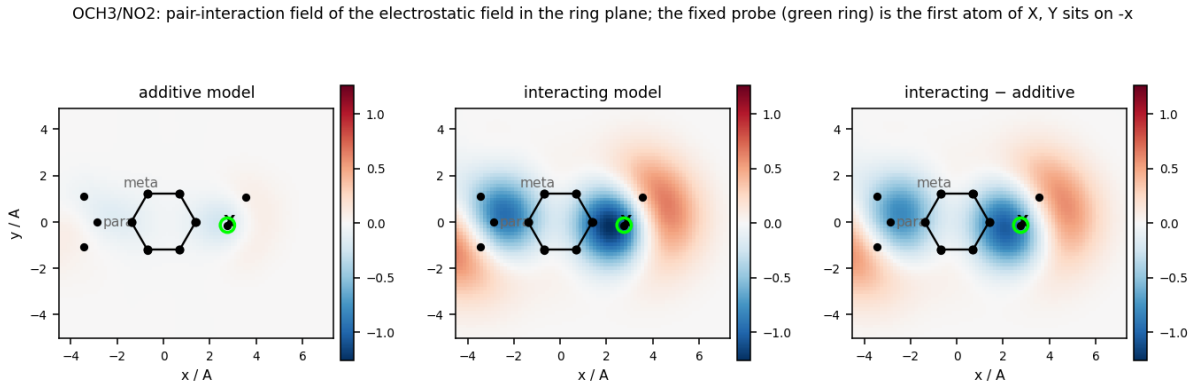
- the **pair-interaction field** (`cmf cofield --kind pair --molecule FILE --probe-atom K`, `nonlinearity.pair_interaction_field`): one probe fixed on atom K of the molecule, carrying that atom's weight in the field, the other scanned over the grid. The value at **r** is how much adding field there changes what the fixed atom does: positive where the two reinforce each other, negative where one undoes part of the other, zero where the model treats the two places additively. `--probe-field` puts the fixed probe in another field type than the one drawn;
- the **curvature field** (`--kind curvature`, `nonlinearity.curvature_field`), the diagonal: the second derivative along one probe, negative where the model saturates and positive where it accelerates. Its first-derivative companion `nonlinearity.gradient_field` is what one unit of a probe adds, the `atom_effect` of a linear model generalised to a layer;
- the **atom interaction matrix** and its **modes** (`--kind modes`, `nonlinearity.atom_interactions`, `nonlinearity.interaction_modes`): the second derivatives with respect to the weights of every pair of the molecule's own atoms -- two partial charges, say -- beside the first derivatives, which under a layer take the place of the atom contributions. The eigenvectors of the matrix are patterns of atoms whose weights the model reads together; the eigenvalue is the curvature along the pattern, positive when the changes in it reinforce one another, negative when they saturate. The command prints the leading modes with the atoms that carry them and writes the matrix with `--csv`.

All three are in the units of the activity per unit of amplitude squared, so a pair term is judged by multiplying it by two realistic amplitudes -- two partial charges, two indicator weights of one -- and comparing the product with what the first derivatives give for the same changes. Where the product is small against that, the model is additive there for every practical purpose, whatever the map looks like.

Measured (`benchmarks/run_interactions.py`, `results/interactions/`). No measured set in the repository has a known non-additive pair, so the benchmark makes one: 105 para-disubstituted benzenes X-C_6H_4-Y from 14 substituents, built in the frame of the Hammett example, with an additive activity $\sigma_p(X) + \sigma_p(Y)$ and an interacting one that adds $1.5\ \sigma_p(X)\ \sigma_p(Y)$, both with 0.02 units of noise; five fields, unnormalised kernels, the Gaussian layer, α, λ and γ by 10-fold cross-validation. The pair term costs the linear kernel 0.17 in q^2 (0.99 to 0.82) and the layer recovers most of it (0.95). The interacting activity has an exact second derivative: with c_k the sensitivity of σ to the charge of atom k, the cross derivative between an atom of X and an atom of Y is $1.5\ c_k\ c_l$ and there is none within a group; the additive activity has none anywhere. Reading c_k off the additive model's first derivatives, every cross pair of every molecule (78 molecules with X and Y both other than H and different) has a designed value, and the fitted values are regressed on it through the origin:

model	slope, atom pairs	correlation	slope, pair field near Y	correlation	within-group RMS over the designed cross RMS
additive	0.06	0.30	0.08	0.42	0.35
interacting	1.87	0.73	1.43	0.67	2.39

The pair field near Y is the field with the probe fixed on the first heavy atom of X, within 2 Å of the Y group, against its designed shape, 1.5 c_k times the fixed atom's charge times the additive model's first-derivative field there. Three things follow. The designed pair term is recovered where there is one and absent where there is none: slopes of 1.9 and 1.4 against 0.06 and 0.08, correlations of 0.7 against 0.3 to 0.4, and the pair field of one molecule is stable under refitting on four-fifths of the set (correlation 0.97). The size is overstated by a factor of 1.4 to 1.9, and the within-group block of the interacting model, which should be zero, is 2.4 times the designed cross block in root mean square: a Gaussian layer that has learnt a pair term is curved within the groups too. And *where* the curvature sits does not tell the two models apart: the share of the atom matrix between the two groups is 0.10 for the additive model and 0.09 for the interacting one (0.07 for the share of pairs), and the share of the pair field near Y is 0.32 against 0.28 (0.08 of the volume) -- the layer puts curvature where the training fields are, whatever the target. The signal of a pair term is the *size* of the second derivative, which is why the figure draws the two models on one colour scale and why the advice above is to judge a pair term by its product with realistic amplitudes. Between two sites the sign and the order of magnitude can be read; within one site the values are mostly the kernel's own curvature, and a reference model of an additive quantity, when one exists, is the right thing to subtract.



The pair-interaction field of the electrostatic field in the ring plane for OCH3-C6H4-NO2, the fixed probe (green ring) on the oxygen of the methoxy group, the nitro group on the far side: the model of the additive activity, the model of the interacting one, and their difference, on one colour scale. The additive model's field is small everywhere; the interacting model's is large, negative over the nitro group and around the probe, positive beyond the methoxy group.

8.3 Shapley values

The *Shapley* tab decomposes a single prediction into additive contributions by the Shapley construction, over **atoms**, over **substituents of the common scaffold**, or over **field types**, with second-order interaction indices available. It serves two purposes. With the linear kernel it reproduces the atom contributions exactly, which is a useful check. With a nonlinear kernel layer, where co-fields do not exist, it is the only decomposition left — and it still works, because it needs

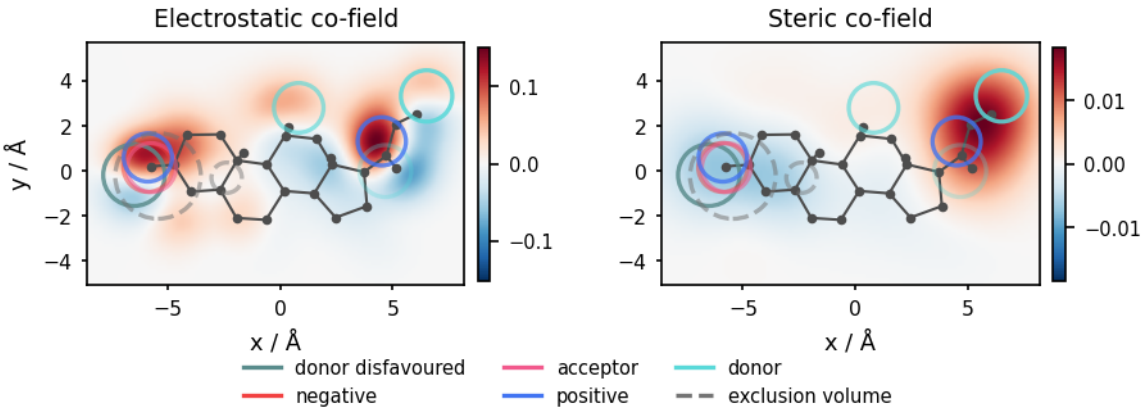
nothing but the ability to evaluate the model on subsets.

8.4 A pharmacophore out of the co-field

Tools → Extract pharmacophore from the co-field… turns the co-field into a discrete, typed pharmacophore: the favourable lobes of each field become feature points of the matching kind (positive and negative centres, hydrophobe, donor, acceptor), each with a tolerance radius taken from the size of its lobe, and the unfavourable steric lobes — or the pocket, when a protein is loaded — become exclusion volumes. The result is a table and a PDB-style file of feature spheres that other software can read, and it draws in the 3D view; untick *pharmacophore* in the viewer controls to clear it.

This is the bridge from a CMF model to a screening query: the model is continuous and needs an alignment, the pharmacophore is discrete and does not.

the pharmacophore read out of the steroid co-fields, over two of them



The pharmacophore extracted from the steroid model of chapter 8 at half height, drawn over the electrostatic and the steric co-field in the mean plane with the most active training molecule: each feature is a circle of its tolerance radius at its centre, fainter the further it lies from the plane, coloured by its kind, and an exclusion volume is dashed. The favourable lobes of each field have become typed points, the unfavourable steric lobes exclusion volumes, and that set of spheres is what the query files of the next part carry.

Taking the query out of the program

```
python cmf_cli.py pharmacophore model.cmf --min-z 2 --query query.json
python cmf_cli.py pharmacophore model.cmf --min-z 2 --query query.pml --enable-top 3
```

`.json` writes a **Pharmit** session and `.pml` the **LigandScout-style XML**; both are written to the documented shape of those formats, which is as far as this program can go — it cannot test any particular viewer. Three things about the file are worth knowing before it is opened somewhere else.

A query is a conjunction, so a dozen features at once matches nothing. The strongest `--enable-top` features are written switched on and the rest switched off, for the user to choose among in their own program; exclusion spheres are always on, since they only forbid.

Hydrogen-bond features have no direction. A co-field says that a donor *here* raises the activity; it says nothing about where the hydrogen points, so the features are written as isotropic points rather than as the vectors those formats usually carry.

Three of this program's feature types cannot be expressed at all and are reported rather than quietly dropped. `bulk` asks for atoms of any kind, which no query language expresses. The disfavoured `polar`, `acceptor_bad` and `donor_bad` say that one *particular* kind of group is unwelcome; the only negative a query has is an exclusion sphere, and that would forbid every atom, which is a much stronger claim than the co-field makes.

Is the query any good, measured

Eight Sutherland sets, five 70/30 splits each. The pharmacophore is read out of the training model's co-fields — masked at two standard deviations first, so that a lobe the measurement error could have produced never becomes a feature — and its strongest four features are matched against the molecules held out, by the simple rule a screening program uses: an atom of the right kind inside the sphere. The actives are the most active quarter.

set	n	features	AUC query	AUC random centres	AUC the model
DHFR	361	7.4	0.573	0.498	0.901
COX2	282	8.4	0.662	0.478	0.867
BZR	147	7.8	0.518	0.514	0.797
ACE	114	7.0	0.789	0.552	0.927
AChE	111	7.8	0.631	0.484	0.903
THR	88	8.4	0.595	0.468	0.780
THER	76	7.2	0.711	0.482	0.812
GPB	66	6.8	0.838	0.484	0.902
mean			**0.665**	**0.495**	**0.861**

The query separates the active quarter at an AUC of 0.665, against 0.495 for the same features moved to random places in the same box — a paired difference of +0.170 with a 95 % interval of [+0.102, +0.238], so the places the co-field picked are doing the work, and ahead on 36 of 40 splits. Over every split together the top tenth by the query holds 1.77 times as many actives as chance would put there.

The model it was read from reaches 0.861 on the same molecules, and that gap is the price of the summary: a pharmacophore is four spheres and a rule for counting them, while the model is every training molecule's field weighed by a fitted coefficient. What the query gains is that it can leave this program — it goes into a screening tool and runs over libraries no kernel method here could hold.

8.5 What a candidate is missing

Tools → Field difference from a reference molecule... subtracts two molecules' fields on the co-field grid and reports, per field type, the cosine between them, the distance, and how much of the reference's field the candidate **fails to reproduce** against how much it **adds**. The difference map draws as its own isosurface, and the table ranks the whole data set by field similarity to the reference. In a lead-optimisation reading: the "missing" column is what to put back, the "added" column is what may be causing trouble.

8.6 Several endpoints at once: the selectivity co-field

A separation problem, or any selectivity, is not one property but two: an extractant has an affinity for americium and an affinity for europium, and what matters is the difference. A model of the ratio works, but it throws information away. Each affinity has its own measurements, often on partly different compounds; a ligand that binds both metals strongly looks, to a model of the ratio, exactly like one that binds neither; and the ratio cannot say *which* of the two affinities a substituent moved.

Tools → Model several endpoints (selectivity co-field)… fits every endpoint you tick on the kernel of the model you have already built, with one bias per endpoint and a task covariance that lets them share information with each other:

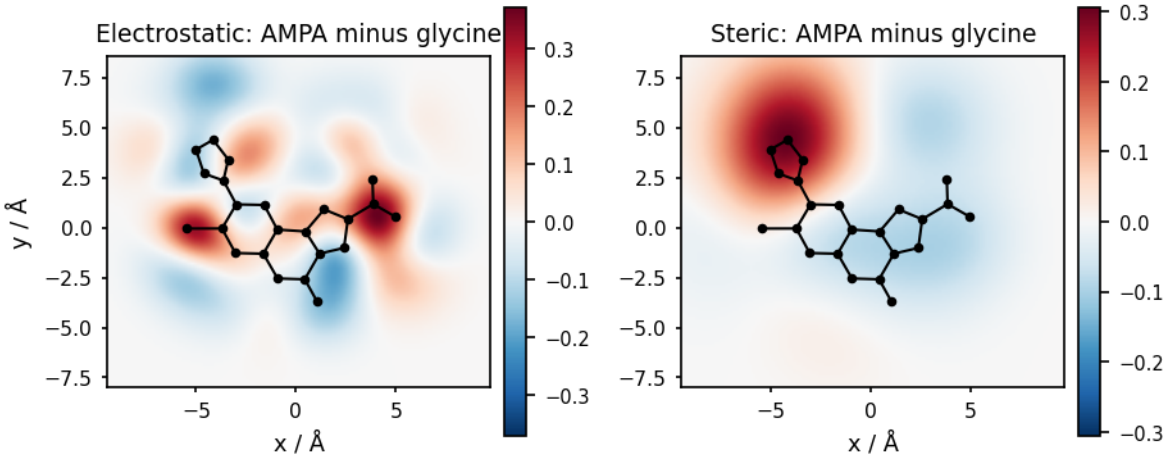
$$y_t(X) = \sum_j c_{jt}\, K(X, X_j) + b_t, \tag{8.10}$$

where the coefficients of the two endpoints are tied through a single correlation ρ. At ρ = 0 the endpoints are modelled separately and share only the kernel; as ρ rises they share information; at ρ = 1 the two sets of coefficients coincide and the model could only predict a *constant* selectivity, so 1 is excluded (and ρ may be negative, for endpoints that move in opposite directions). The dialog searches ρ and the ridge by cross-validation, and the cross-validation holds out whole **molecules**, never single measurements: with a molecule's first endpoint in the training set and its second in the test set, the correlation would reveal the answer.

Because each endpoint's prediction is linear in the field with its own coefficients, each endpoint has a co-field, and so does their difference — the **selectivity co-field**

$$C_A(\mathbf{r}) - C_B(\mathbf{r}) = \sum_j (c_{jA} - c_{jB})\, X_j(\mathbf{r}), \tag{8.11}$$

a map of where more of a field raises the first endpoint more than the second. Only a model linear in the field can produce it; it is available here for the same reason co-fields are available at all, and it is refused for a model with a nonlinear kernel layer.



The selectivity co-field of the 57 AMPA / glycine-NMDA antagonists of `examples/selectivity`*, AMPA minus glycine, in the plane of the tricyclic core, with the most AMPA-selective compound drawn over it. A red lobe is a place where more of that field shifts the preference towards the AMPA receptor, a blue one towards the glycine site. The two-endpoint fit has its correlation and ridge chosen by cross-validation.*

Once the fit is made, an **Endpoint** box appears in the *Maps* row of the control strip with one entry per endpoint and one for the difference of the pair you named — and every map, export and contribution table then refers to the chosen one; a cube exported while the selectivity is chosen

carries it in its file name. The endpoints are numeric properties of the data set, so the ones beyond the first are imported with *File → Import activities from CSV...* or read from columns of the file. Molecules that carry only some of them are kept: they inform the endpoints they have, and through the covariance the others. Each endpoint needs at least five molecules.

What the chosen ρ means is itself a result. On the corrosion-inhibitor series of the benchmark (`benchmarks/results/multitask/table.md`) the cross-validation chose ρ = 0 on the two largest series and a non-zero ρ of either sign on the small ones, where it gained at most 0.04 in q^2: an endpoint that already has its own molecules gains little from another, and the ρ the dialog reports in the log is the measurement that says how much.

The same model from the command line

`build --second-endpoint COLUMN` fits the pair without the application. The second endpoint is a numeric property of the same file, the correlation is chosen by the marginal likelihood unless `--task-rho` gives it, and the cross-validation that is reported holds out whole molecules. The fitted model then predicts the primary endpoint with the coefficients the two-task fit gives it, and carries the other endpoint with it:

```
cmf build guests.csv --second-endpoint alpha_logK --save model.cmf
cmf predict model.cmf new_guests.csv --out predictions.csv
cmf cofield model.cmf --endpoint selectivity --out-prefix maps
```

`predict` prints both endpoints and their difference and writes them as two more columns; `cofield --endpoint second|selectivity` draws the other endpoint's maps or the selectivity ones. A molecule with no primary activity is dropped before any model is built, as always, so the primary endpoint has to be the one every row carries; the second may be missing anywhere, which is the case the model exists for. `examples/make_two_endpoint_example.py` joins two of the cyclodextrin tables into such a file.

Two cavities, measured. `benchmarks/run_two_endpoints.py` takes the pairs of cyclodextrin hosts: the guests of two tables joined on the structure of the guest, one rotation-invariant kernel, the correlation chosen by the evidence, ten-fold cross-validation over whole molecules, and `alone` the same code at a correlation of zero, which is exactly the two separate fits.

pair	guests (both)	correlation	first endpoint, shared - alone	second endpoint
beta, alpha	1493 (496)	+0.788	-0.004 [-0.011, +0.004]	+0.005 [-0.008, +0.019]
beta, gamma	1253 (201)	+0.869	-0.001 [-0.006, +0.004]	+0.032 [-0.011, +0.082]
alpha, gamma	938 (181)	+0.706	-0.002 [-0.009, +0.004]	-0.008 [-0.029, +0.016]

Sharing changes nothing here, which is the same answer the corrosion series gave: every interval contains zero, the largest movement being +0.032 for the gamma endpoint when it borrows from beta's 1162 guests. An endpoint with a few hundred molecules of its own already has what it needs, and what the two endpoints have in common is carried by the kernel they share before any correlation is fitted.

The selectivity is a different matter. It is predictable -- q2 +0.273 for beta against alpha on the 496 guests measured against both, +0.223 for beta against gamma and +0.389 for alpha against

gamma -- and **what it rests on is the size of the guest**, which is the known difference between these cavities. The correlation between the predicted selectivity and the number of heavy atoms follows the ordering of the three cavities exactly: beta over alpha rises with size (+0.468, the measured values +0.382), while beta over gamma falls (-0.739 against -0.480) and alpha over gamma falls further (-0.676 against -0.613). A big guest prefers the bigger cavity, a small one the narrower, and the model says so without being told that a cavity has a size.

Beyond one correlation

One ρ says that every pair of endpoints is related to the same degree. For two endpoints that is no assumption at all; for seven it is a strong one, and wrong in the usual case where some endpoints are nearly the same and others are only weakly related. The dialog's **Task covariance** box therefore offers a free-form matrix,

$$\mathbf{T} = \mathbf{L}\mathbf{L}^{\mathrm{T}} + \mathrm{diag}(\mathbf{d}), \tag{8.12}$$

with **L** of shape (endpoints, rank) and **d** > 0. It is positive definite for every value of its parameters, which is why nothing has to be constrained; at rank 1 it contains the one-ρ model exactly, so the comparison is meaningful; and its **diagonal is free**, which matters whenever the endpoints are in different units — a potency and a free energy in kJ/mol need their own amplitudes, and one shared correlation cannot give them one.

Such a covariance has (rank + 1) × endpoints parameters, which is far too many for cross-validation: ten fits per candidate and several hundred candidates. It is chosen instead by the **marginal likelihood**, the multi-endpoint form of the criterion of *Choosing α and λ without holding anything out*: the mean is marginalised over the design columns under a flat prior — one constant per endpoint, or the task descriptors below — and the kernel amplitude is profiled out, so one factorisation scores a candidate and the value does not depend on the units of the activities. `cmfqsar/multitask.log_evidence` is that criterion and `fit_covariance` optimises it by Nelder–Mead from several starts, because the evidence over a covariance is not concave and a start at ρ = 0.8 and one at ρ = 0.3 do not always converge to the same answer.

An endpoint nobody measured

A free-form row for an endpoint with no measurements is unidentifiable: there is nothing to fit it to. A row read off **task descriptors** is not. Give the endpoints their own descriptor vectors $\mathbf{z}_t$ — the medium, the metal, the temperature; in a host-guest problem the structure of the host itself — and set

$$T(t,s) = (1-\varepsilon)\exp\left(-\frac{|\mathbf{z}_t - \mathbf{z}_s|^2}{2\ell^2}\right) + \varepsilon\,\delta_{ts}, \tag{8.13}$$

whose length scale ell and nugget varepsilon are chosen by the same evidence (`descriptor_covariance`). The nugget is not optional: without it two endpoints with identical descriptors would be perfectly correlated, the covariance singular, and their coefficients forced to coincide. A `linear` form is available too, and is the one to use when two endpoints are expected to be *anti*-correlated, since an RBF kernel cannot be negative.

The mean needs the same treatment, since an endpoint that was never measured has no bias to fit. Passing `task_features` to `Observations` replaces the indicator columns of the bias with the descriptors themselves, so that the mean is linear in them and extrapolates; with no features the design is the indicator matrix and the behaviour is exactly as before. Then

```
from cmfqsar.multitask import Observations, descriptor_covariance, fit
obs = Observations.from_columns(Y, names=hosts, task_features=Z_with_intercept)
dc = descriptor_covariance(K, obs, Z)
f = fit(K, obs, lam=dc.lam, T=dc.T)
y_new = f.predict_new_task(K_cross, dc.row_for(z_new), features=z_new_with_intercept)
```

predicts the unmeasured endpoint, and `f.new_task_coefficients(dc.row_for(z_new))` gives it a co-field like any other. What this cannot do is invent what is peculiar to that endpoint: it interpolates between the endpoints that were measured and inherits only what they have in common.

What it measures

Measured on `examples/cyclodextrin/` (`benchmarks/run_tasks.py`), which has seven real endpoints and descriptors for them: 1669 distinct guest molecules, 2671 association constants against the seven cyclodextrin hosts (beta 1162, alpha 827, gamma 292, hp_beta 135, sbe_beta 135, methyl_beta 77, dimethyl_beta 43), and the hosts are molecules too, so the task descriptors are a structure rather than a label — cavity size, lipophilicity, charge, hydroxyl count, heavy atoms. One kernel for every row, the rotation-invariant field autocorrelation at α = 0.1 (there is no frame available here, and *Working without an alignment* found that kernel the best of the three on this data), and 10-fold cross-validation over **guests**, so that no guest has one host in the training set and another in the test set.

task covariance	mean q^2	beta	alpha	gamma	hp_beta	sbe_beta	methyl_beta	dimethyl_beta	what it is
independent	**+0.412**	+0.448	+0.338	+0.473	+0.545	+0.331	+0.377	+0.374	rho = 0
one rho, CV	**+0.454**	+0.448	+0.338	+0.524	+0.586	+0.389	+0.441	+0.454	rho = +0.90
one rho, evidence	**+0.454**	+0.449	+0.341	+0.523	+0.592	+0.391	+0.435	+0.451	rho = +0.857 over 25 values
free rank 1	**+0.475**	+0.447	+0.339	+0.525	+0.604	+0.444	+0.504	+0.465	2 x 1 starts, 1603 evaluations of the evidence
free rank 2	**+0.488**	+0.447	+0.340	+0.527	+0.624	+0.452	+0.536	+0.490	2 x 1 starts, 2274 evaluations of the evidence
descriptors	**+0.448**	+0.446	+0.342	+0.505	+0.608	+0.390	+0.430	+0.413	rbf, length scale 4, nugget 0.1

Sharing is worth +0.076 mean q² — +0.412 fitting the seven hosts independently on the same kernel against +0.488 for the free rank 2 covariance — and the gain is largest on the endpoints with fewest measurements, which is what sharing information between endpoints is supposed to do (the gain and the number of measurements correlate at -0.89, and the biggest gain goes to methyl_beta, while the smallest set is dimethyl_beta). A free-form covariance beats one shared correlation by +0.034, so with seven endpoints the single number misses part of the gain.

The evidence was the objective for every row but the cross-validated one, and it is worth checking whether it chose what cross-validation would have:

task covariance	log evidence	mean q²
free rank 2	-2605.3	+0.488
free rank 1	-2608.8	+0.475
one rho, evidence	-2635.3	+0.454
one rho, CV	-2635.8	+0.454
descriptors	-2647.6	+0.448
independent	-2713.1	+0.412

It did: the covariance with the highest evidence is also the one with the highest out-of-fold q², which is the agreement that is assumed without a check elsewhere in this manual.

An endpoint nobody measured, measured

Each host was then removed **entirely** — the whole endpoint, not a fold of its guests — and predicted from the other six through the descriptor covariance, with the mean linear in the descriptors so that there is something to extrapolate. Spearman ρ is the column to read: the absolute scale of an endpoint nobody has titrated is not recoverable, and a screen ranks.

host removed	n	nearest host	ρ, descriptor covariance	ρ, nearest host alone	ρ, mean of the measured hosts	q², descriptor
beta	1162	sbe_beta	+0.577	+0.320	+0.623 (615)	+0.231
alpha	827	beta	+0.431	+0.433	+0.470 (522)	+0.066
gamma	292	sbe_beta	-0.284	+0.145	+0.533 (225)	-5.499
hp_beta	135	sbe_beta	+0.664	+0.294	+0.806 (105)	+0.483
sbe_beta	135	beta	+0.516	+0.515	+0.498 (74)	+0.200
methyl_beta	77	dimethyl_beta	+0.595	+0.529	+0.894 (73)	+0.416
dimethyl_beta	43	methyl_beta	+0.703	+0.708	+0.863 (38)	+0.044

Mean ρ **+0.457** through the descriptor covariance, against **+0.421** for the single-task model of the nearest host by descriptors and **+0.670** for simply averaging the other hosts' measured values of the same guest. So the covariance does carry real information about an endpoint it has never seen — it beats the nearest host's own model 4 times out of 7 — and it does not beat the cheapest baseline of all. That baseline is not a model, which is its only disadvantage: it can only say something about a guest some other host was measured against, and says nothing about a new molecule. The q^2 column is the necessary warning: it is negative on 1 of the 7 hosts, so the numbers this route produces are a ranking and not a binding constant.

What governs the spread is whether the removed host is interpolated between the others or extrapolated to. Cavity size is the first descriptor, and of the seven hosts five have seven glucose units — β-cyclodextrin and its four derivatives — while **α** has six and **γ** has eight. Remove one of those five and the covariance interpolates; remove α or γ and it has to extrapolate outside the range it was built on. The five interpolated hosts average ρ **+0.611** against **+0.473** for the nearest host's own model, a margin of +0.138; the two extrapolated ones give +0.431 (α) and **−0.284** (γ, with q^2 −5.50, a prediction that is worse than useless).

So the claim this route supports is *interpolation between endpoints that were measured*. γ is the warning: it is not merely the largest cavity but the only one of its size, the task kernel has no neighbour to take its shape from, and the mean — linear in the descriptors — is extrapolated past every host it was fitted on. The length scale the evidence chose is itself the symptom: 4 standardised units, which is the *top of the grid* on every one of the seven fits, so the criterion would prefer an even wider task kernel. With seven endpoints there is not enough to localise it, every endpoint is treated as a neighbour of every other, and that is generous towards γ and wrong.

8.7 Comparing two models by their co-fields

The selectivity co-field above is the *difference* of two co-fields. Their *inner product* is the other half of the same algebra, and it answers a different question: how far do two models ask the same thing of the chemistry?

```
python cmf_cli.py transfer targetA.cmf targetB.cmf
python cmf_cli.py transfer round1.cmf round2.cmf round3.cmf --fields --out map.csv
```

It needs no grid. With $C_f^A(\mathbf{r}) = \Sigma_j\, a_j\, s_{fj}\, X_{fj}(\mathbf{r})$ and the same for B, the overlap is a sum of overlap integrals of atom-centred Gaussians — which is the field kernel between the two training sets:

$$\langle C_f^A, C_f^B \rangle = \sum_{j,k} a_j b_k s_{fj} s_{fk} K_f^{\mathrm{raw}}(j,k) = \mathbf{a}^{\mathrm{T}} \mathbf{K}_f(A,B)\, \mathbf{b} \tag{8.14}$$

one matrix product on a block the program computes for a prediction anyway. The number to read is the **cosine**, because it does not move when either activity is rescaled, when the mixing coefficients change, or when the field scaling does: it compares the *shapes* of two demands rather than their sizes. A unit test checks it against the same integral taken on a 175 000-point grid; they agree to four decimals.

What it is for, in the order the question arises. **Two targets**: a positive cosine says both want the same thing in the same place, which is why selectivity is hard, and a negative one says they want opposite things, which is where selectivity comes from. **Two series on one target**: agreement means the SAR transfers, disagreement means two chemotypes are being fitted by two different stories and a model pooled over both will be an average of them. **One series across rounds**: the cosine with the previous round says how much the new compounds changed the model, and the

per-field cosines say in which field.

Is it a map of SAR transfer, measured

Every Sutherland set split into its chemotypes, a model on each group of fifteen molecules or more, the attenuation shared across the set, and every ordered pair scored twice: by the co-field cosine, and by how well one model orders the other group's molecules (Spearman's ρ — not R^2, since two chemotypes often sit at different activity levels and an offset is not a failure of the SAR).

	mean cosine	mean transfer ρ
two halves of one chemotype (the control, 16 pairs)	**+0.401**	**+0.673**
two different chemotypes of one target (64 pairs)	+0.178	+0.334

The cosine separates the two cases by 0.22, which is what it is for: two models of one SAR look alike and two models of two SARs do not. And within the chemotype pairs the cosine and the transfer **correlate at ρ +0.667** over the 64 pairs, which is the claim of this feature, measured — a table of co-field overlaps is a map of SAR transfer.

Everything depends on one assumption. Two co-fields are functions on one space only if their molecules were posed in one frame. The program checks what it can from the model files — the same fields, the same attenuations, training boxes that overlap — and refuses with a reason when they fail. What it cannot check is whether two alignments *mean* the same thing, and nothing in the cosine would reveal that they did not. A cosine is also not a prediction: that two models want the same thing does not mean either is right.

8.8 What the field sees that another view does not

A *view* is one description of the same molecules, which in this program means one kernel over them. The program builds several: the five-field kernel, the quantum fields, a fingerprint, the pocket complementarity, the same fields over a conformer ensemble. Nothing above says how much two of them say the same thing, or, more useful, **which part of the field is the part a second view does not contain**. Canonical correlation analysis answers that question: given two descriptions of the same objects it finds the pair of directions, one in each description, whose projections correlate most strongly, then the next such pair uncorrelated with the first, and so on. The kernel form does it with kernels in place of coordinates, which is what CMF needs because its description of a molecule has no coordinate vector.

```
python cmf_cli.py kcca examples/sutherland/THR.sdf --charges file                # against ECFP4
python cmf_cli.py kcca examples/sutherland/THR.sdf --charges file --save-components comps
python cmf_cli.py kcca set.sdf --other dataset --other-dataset set_other_poses.sdf   # two field views
python cmf_cli.py kcca set.sdf --fields steric,hydrophobic --other fields --other-fields electrostatic
hba,hbd
```

View A is always a field kernel, built with the same options as a model's. View B is the Tanimoto kernel of 2048-bit ECFP4 by default, which describes a molecule by its substructures and knows nothing of shape or pose; or the same molecules from another file, matched by name, which is how two poses or two conformer sets are compared; or another set of fields of the same molecules.

With centred kernels **K** and **L** on the same *n* molecules and dual directions **a**, **b** (one coefficient per molecule, the same object as a fitted model's coefficients), the pairs solve the two coupled

equations

$$\mathbf{KL}\,\mathbf{b} = \rho\,(\mathbf{K}^2 + c_K\mathbf{K})\,\mathbf{a}, \qquad \mathbf{LK}\,\mathbf{a} = \rho\,(\mathbf{L}^2 + c_L\mathbf{L})\,\mathbf{b}, \tag{8.15}$$

where ρ is the canonical correlation and c_K, c_L are ridges. **The ridge is not an option.** With $c = 0$ and invertible kernels every canonical correlation is exactly 1 and the directions are arbitrary: both views have as many directions as there are molecules, so any direction of one can be matched exactly by some direction of the other. That is the classical failure of the method, and it is why the program never judges a pairing by its training correlations. The ridge is chosen by the correlations on **held-out** molecules (the directions fitted on four fifths of the set, the variates correlated on the fifth), and every correlation is compared with the one a **shuffled pairing** of the two views produces, where each molecule's field is paired with another molecule's fingerprint. A correlation that does not clear the shuffled one is the degeneracy, not chemistry.

The output is a table of the leading components with their training correlation, their held-out correlation, the 95th percentile of the shuffled pairing, and a *variance share*: how much of the direction the ridge left, which is 1 for every direction when the ridge is too small. Then three verdicts, in the order the item of the development plan asks them:

- **(i) The correlations are real:** how many components hold out above the shuffled pairing.
- **(ii) Where the activity is.** A direction of the field counts as *contained* in the other view when that view explains more than half of its variance on held-out molecules. The field kernel is split into its part inside the contained directions and the rest, which is the part of the field that is the field's own, and kernel ridge regression is fitted on each with the regularisation chosen inside the folds. The difference of the two q^2 values comes with its bootstrap interval, and an interval that covers zero is not a result. The split is a projection of the kernel and not a regression on the canonical variates, because the variates are whitened by construction: a model on them has lost how much the molecules vary along each direction, which is the prior kernel ridge regression rests on.
- **(iii) The components are readable.** A canonical direction on a field side is a vector of dual coefficients, so it has a co-field, drawn by the same code as a model's. When both views are field kernels on the same fields in one frame, the two co-fields of one component are functions on one space, and the program prints their cosines (the one of *Comparing two models by their co-fields*): the two sides of one component should agree, and two different components should not. A fingerprint has no co-field, so against ECFP4 only the field side of each component can be drawn.

`--save-components DIR` writes each leading component as a model file, `A1.cmf` and so on, and `private.cmf`, the activity model on the part of the field the other view does not contain; `cmf cofield DIR/private.cmf` draws where that part carries the activity, which is the picture the question was asked for. `--out` writes the canonical variates of every molecule.

The degenerate case, which is the obvious misuse. With the activity itself as the second view, that view has one dimension, and the first canonical direction is the kernel ridge regression of the activity on the field at the matched ridge, up to a scale factor. "KCCA-QSAR" in that sense is kernel ridge regression under another name and predicts nothing new; the test suite uses the identity as a correctness check and nothing else. The method earns its place only when both views are rich.

What the fingerprint already holds, measured

The pairing that puts CMF's own premise to the test: the five-field kernel, at the attenuation the evidence picks for each set, against the ECFP4 Tanimoto kernel of the same molecules, on the eight

Sutherland sets (`benchmarks/run_kcca.py`). The field is three-dimensional and the fingerprint is not, so the directions of the field the fingerprint does not contain are the part of the description that is genuinely three-dimensional.

	mean over the eight sets
canonical pairs that hold out above the shuffled pairing	83 %
directions of the field the fingerprint contains (more than half their variance explained)	14 %
q^2 of kernel ridge regression on the whole field	+0.528
q^2 on the part of the field the fingerprint contains	+0.478
q^2 on the part it does not contain	+0.162
q^2 on a random part of the field of the contained part's size	+0.024

(i) holds: the leading pairs hold out at correlations of 0.93 to 0.99 on every set, far above the shuffled pairing's 0.2, so the two views agree about most of what varies between the molecules. **(ii)** comes out the other way from the premise. The part of the field the fingerprint contains, a tenth to a fifth of its directions, carries nearly all of the activity; the rest, which is the three-dimensional part, predicts at +0.16 on average and the difference is below zero with its interval on 7 of the 8 sets (−0.32 on average). A random subspace of the contained part's size predicts nothing, so it is not the size of the part that matters: the fingerprint picks out the directions along which the molecules vary most, and on these congeneric series those are where the activity is. **(iii)** holds, measured on two field views of the same molecules (the file's poses against the same poses moved by up to 10° and 0.5 Å): the co-fields of the two sides of one component agree at a cosine of +0.83 on average, against 0.05 between different components, on all eight sets.

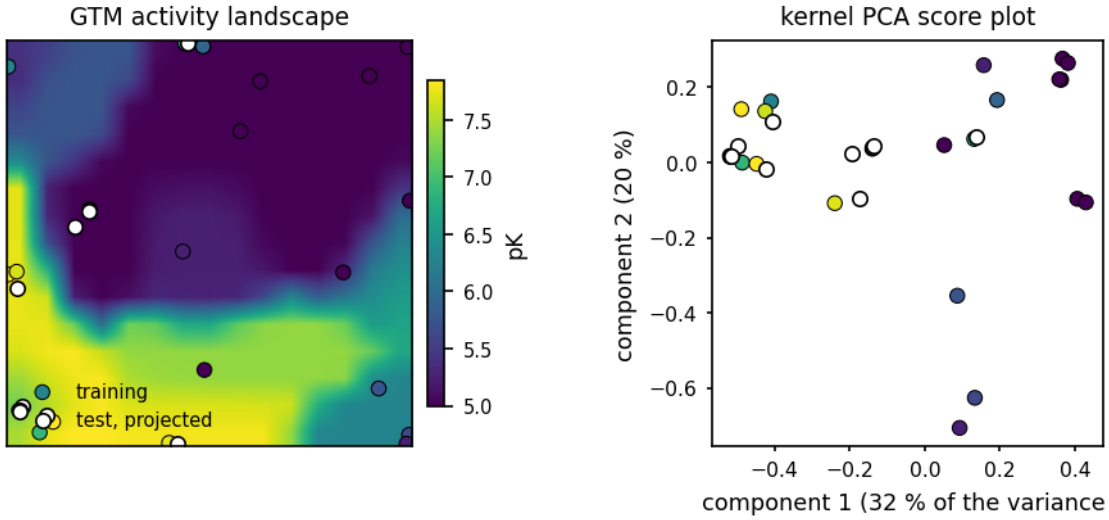
What this settles is what the item asked to be told either way. On the Sutherland series a two-dimensional descriptor already holds the directions of the field that carry the activity, and the three-dimensional remainder adds little, which is the finding of the fourth-dimension study again from the other side. It does not say the field is useless: the whole field predicts better than either part, and a fingerprint has no co-field, no pose and nothing to say about a molecule whose substructures are new. It says where, on series of one chemotype, the information is.

Two more pairings were measured the same way. The three reactivity fields of the cheap quantum stage (f+, f−, the dual descriptor) against the five empirical fields, on 49 series (`benchmarks/run_kcca_quantum.py`): the empirical view contains 23 % of the quantum view's directions, and that part predicts the activity at q^2 +0.18 while the quantum view's own part predicts at −0.06, below it with the interval on 20 of 49 series. So what the reactivity fields know about the activity, shape and charge already contain, which is why adding them gained little in the earlier chapters; the Hammett set is the exception, as it should be. And the ligand's fields against its interaction profile with the stored binding site, the pocket's view of the ligand, on 21 free-energy series (`benchmarks/run_kcca_pocket.py`): the profile contains 35 % of the field's directions, but on sets of 22 to 44 ligands neither part can be said to carry more of the activity (the interval covers zero on 16 of 21), the profile alone predicts nothing, and the whole field beats either part. The cross-target form of that pairing, a kernel over targets, needs several targets whose ligands share a frame and is not possible here.

8.9 Chemical space maps

Tools → Chemical space map... builds a two-dimensional map of the set from the CMF kernels. Fields are not vectors, so Generative Topographic Mapping cannot be applied to them directly; the program performs kernel PCA on the CMF kernel — which is functional PCA, and reproduces feature-space distances exactly — and runs the ordinary EM fit of GTM in those coordinates. The out-of-sample projection stays a single kernel row, so new molecules can be placed on an existing map.

Two projections, and they answer different questions. The *Projection* box chooses between the GTM landscape and the **kernel PCA score plot**, which draws the functional principal components themselves — by default the first two, with any other pair available. The GTM spreads the set over a square grid, which is what makes a landscape possible, but it is a nonlinear mapping: a tight cluster is pushed apart to fill the grid and no distance on the map can be read as a distance between molecules. The score plot is the linear projection the GTM is fitted on, so its distances are the kernel's own as far as the two eigenvalues reach, and each axis is labelled with the share of the kernel variance its component explains — on the steroid set, 32.9 % and 15.1 %. Ask the map *where the activity is*; ask the score plot *whether this series is one cluster or three*, and how much of the kernel a flat picture of it can genuinely show. The score plot needs no EM fit, so it also costs a fraction of the time.



The two projections of the steroid set under the model of chapter 8. Left: the GTM activity landscape on a 15 by 15 grid, the mean affinity the responsibilities assign to each node, with the 21 training molecules at their mean positions and the 10 test molecules projected onto the fitted map. Right: the kernel PCA score plot the GTM is fitted on, with the share of the kernel variance each axis carries. The map spreads the set over the square; the score plot keeps the kernel's own distances, and shows the set as two groups rather than one.

From the responsibilities follow the **activity landscape** (responsibility-weighted mean activity per node), the **class landscape** (the fraction of actives) and the **data density**, which is a picture of the applicability domain; nodes with no support are left blank. The landscape is a model in its own right: on the thrombin training set it reproduces the activities with R^2 0.92.

Two cautions. Use cosine-normalised kernels, or the leading components follow molecular size. And a CMF kernel between molecules that share no frame is meaningless, so a map spanning several series would partly be showing alignment quality — unless you use the pose-free count kernel of *Working without an alignment*, which separates three targets' ligands cleanly with no alignment at all.

8.10 The pre-image kernel PCA lacks: a decoder of pseudo-atoms

The map above runs kernel PCA on the CMF kernel, which is functional principal component analysis of the fields. It can place a molecule and it can reconstruct a field -- but the reconstruction is a weighted sum of training fields. That sum can be drawn on a grid, and it is not a set of atoms: nothing can be sampled from it, and no design step can take it as a target. Plan item E11 adds a decoder whose output *is* a set of atoms.

The three pieces (`cmfqsar.fieldae`, PyTorch, `pip install -e .[fieldae]`):

piece	what it is
encoder	a molecule to a code of a few dimensions. Its input is the whitened kernel against a few hundred landmarks, the feature vector of *Fitting in the space of the landmarks*, so it sees a molecule exactly as the kernel does: no grid, no atom order, no fixed atom count
decoder	a code to a fixed number of **pseudo-atoms**, each with a position and one weight per field. That is the form every other field in this program has, so a decoded field goes straight into the overlap kernel, a co-field reader or a matcher
loss	the squared distance between the two fields in the field Hilbert space

The decoded field has the form of any other field in this program, a sum of Gaussians on M pseudo-atoms at positions $\{\hat{r}\}_m$ with weights $\hat{w}_{mf}$,

$$\hat{X}_f(\mathbf{r}) = \sum_{m=1}^{M} \hat{w}_{mf} \exp\left(-\frac{b_f}{2} |\mathbf{r} - \hat{\mathbf{r}}_m|^2\right) \tag{8.16}$$

and what is minimised is its distance from the molecule's own field in the field Hilbert space,

$$\|X_f - \hat{X}_f\|^2 = k(X_f, X_f) - 2\,k(X_f, \hat{X}_f) + k(\hat{X}_f, \hat{X}_f) \tag{8.17}$$

All three terms are sums of overlap integrals of atom-centred Gaussians -- the same closed form as every kernel in this manual, (2.3) -- so the loss is exact and differentiable in the positions and the weights of the pseudo-atoms, and **no grid enters the training**. The derivative comes from automatic differentiation of that formula, and the test suite holds it to central finite differences of the ordinary NumPy kernel, which stays the reference.

The activity model does not change. The autoencoder is never the regressor. A code that enters a prediction enters it as one more kernel view, so the model stays a kernel model and the co-field, the Shapley values and every interpretation tool are untouched. This is why the item is not the convolutional network that the plan's *What is not planned* excludes: there a grid replaces the kernel and a network replaces the model, while here the loss *is* the closed-form kernel.

```
        from cmfqsar import fieldae

        geom = fieldae.geometry(fieldset, scale=fieldae.field_scales(table, fieldset))
        tens = fieldae.tensors(table, fieldset, geom)
        feats, low = fieldae.landmark_features(table, fieldset, n_landmarks=256)
        ae = fieldae.FieldAutoencoder(geom, feats.shape[1], code=8, n_pseudo=32)
        fieldae.fit(ae, feats, tens, train_rows, valid_rows)
        print(fieldae.relative_errors(ae, feats, tens, test_rows).mean())
        mf = ae.pre_image(ae.codes(feats[[0]])[0])      # the field as a set of atoms
        fieldae.export_decoded("decoded.cube", mf, geom, f=0)
```

Measured on 8,678 molecules. The item cannot be judged on a series of thirty, where kernel PCA has as many components as it needs and any network is underdetermined, so the measurement is on COMPAS-1D: the polybenzenoid hydrocarbons of *A frame by rule*, with the field set, the attenuations, the field scaling and the random0 split of the model of record there -- 6,942 training, 868 validation and 868 held-out molecules. The closed-form self-overlap of this module agrees with the diagonal of that benchmark's stored kernel to 7e-08 relative over all 8,678 molecules, which is the precision of the stored kernel and not of the formula.

The first gate of the item is a stopping rule: if the nonlinear decoder cannot reconstruct better than kernel PCA with the same number of latent dimensions, there is nothing to use. (`benchmarks/results/fieldae/table.md`, relative reconstruction error $|X_f - \hat{X}_f| \,/\, |X_f|$ on the held-out part, 32 pseudo-atoms; lower is better.)

latent dimensions	the decoder	kernel PCA	the decoder gains
2	**0.291**	0.340	0.049
4	**0.217**	0.285	0.068
8	**0.132**	0.216	0.083
16	**0.083**	0.145	0.062
32	**0.070**	0.082	0.012

The gate passes at every width, and the margin says why. The decoder is ahead everywhere, by most at eight dimensions, and by least at thirty-two, where kernel PCA has enough components to describe the set almost as well. The comparison is not even, and the unevenness favours kernel PCA: it gets the training mean for free on top of its components, and its reconstruction need not be a set of atoms. That a few dozen pseudo-atoms beat it is the result the item was asking for.

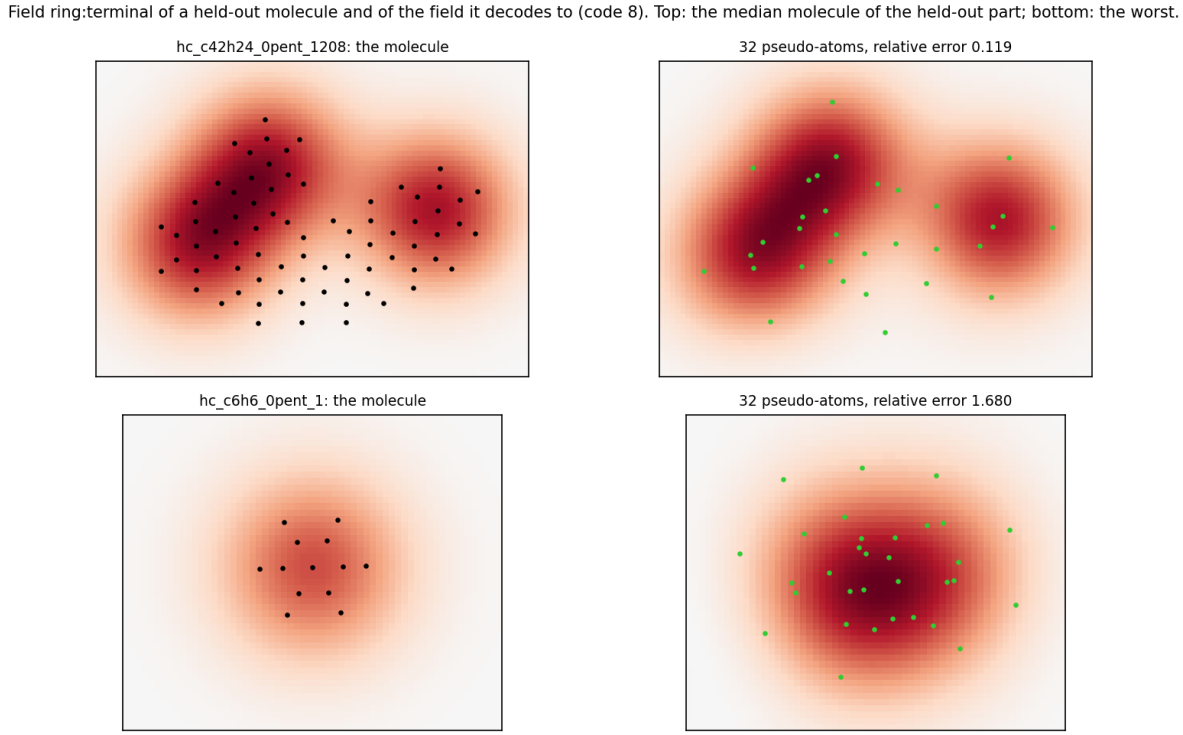
Where the limit is: the floor. Fitting pseudo-atoms to one molecule directly, with no code in between (`fieldae.fit_pseudo_atoms`), gives the best that that many pseudo-atoms can do on a field of this set. It separates the two ways a decoder can fail.

pseudo-atoms	the floor	the decoder at code 8	what limits it
8	0.188	0.200	the pseudo-atoms: the decoder is already at the floor
16	0.092	0.147	both
32	0.043	0.132	the code

pseudo-atoms	the floor	the decoder at code 8	what limits it
64	0.073	0.136	neither: the direct fit itself stops improving

With eight pseudo-atoms the decoder is within 0.012 of what eight pseudo-atoms can represent at all, so nothing is left for a wider code to find; with thirty-two the floor is three times lower than the decoder reaches, so the code of eight is the bottleneck. Beyond thirty-two the direct fit stops improving as well, which is the optimiser and not the representation: a field of fifty-six atoms is described by about thirty pseudo-atoms, and more of them collapse onto each other (`fieldae.min_pseudo_distance` reports that, and a repulsion term keeps it in check).

Where the reconstruction fails, and why.



A held-out molecule beside the field its code decodes to, for the ring-class field of the terminal rings: on the left the molecule's own field with its atoms as black points, on the right the field of the 32 pseudo-atoms the decoder places, drawn in green, with the relative reconstruction error in the title. The upper row is the molecule of median error, a forty-two-carbon benzenoid, where the two pictures are the same picture. The lower row is the worst of the whole held-out part: benzene, where a fixed thirty-two pseudo-atoms are spread over a field that one ring makes. Drawn by `benchmarks/run_fieldae.py`.

For the median molecule the decoded field and the real one are the same picture. The worst case of the whole held-out part is **benzene**, at a relative error of 1.68 -- the one molecule of one ring in a part where 652 of 868 molecules have ten. A decoder has a fixed number of pseudo-atoms and places thirty-two of them around a single ring, which is far more than that field needs. Among the sizes the set is actually made of the error is steady and rises only mildly with size (0.09 at seven rings, 0.14 at ten; rank correlation with the number of rings +0.25). A set with one extreme outlier in size will have that outlier as its worst reconstruction.

The second gate fails, and the failure is the informative half. Item E11 also asks whether the reconstruction error is a good applicability-domain score: whether a molecule the decoder

reconstructs badly is one the model predicts badly. Scored against the errors the model of record actually made on the held-out part, under the protocol of *Three estimates of the error, compared*:

estimate	rank correlation with the error	risk-coverage efficiency
distance to the centre of the training set	**+0.120**	**+0.566**
one-class SVM	+0.052	+0.531
distance from the principal subspace (kernel PCA)	+0.001	+0.519
distance to the fifth nearest training molecule	+0.061	+0.507
one-class Gaussian process	+0.050	+0.497
the autoencoder's reconstruction error	**-0.059**	+0.467

The reconstruction error is the worst of the seven, and its rank correlation is slightly negative. The reason is the same property that makes the decoder win gate (a): it is trained to reconstruct *every* molecule in the field space, and a nonlinear decoder generalises that ability to molecules unlike the training set, so a good reconstruction stops being evidence of being near the training set. A one-class score measures that distance and nothing else, and on this task even the best of them is weak (rank correlation +0.120, and the efficiency of the five is between 0.50 and 0.57 where 0.5 is no better than ignoring the estimate). **Use the one-class scores for the applicability domain; use the decoder for the pre-image.**

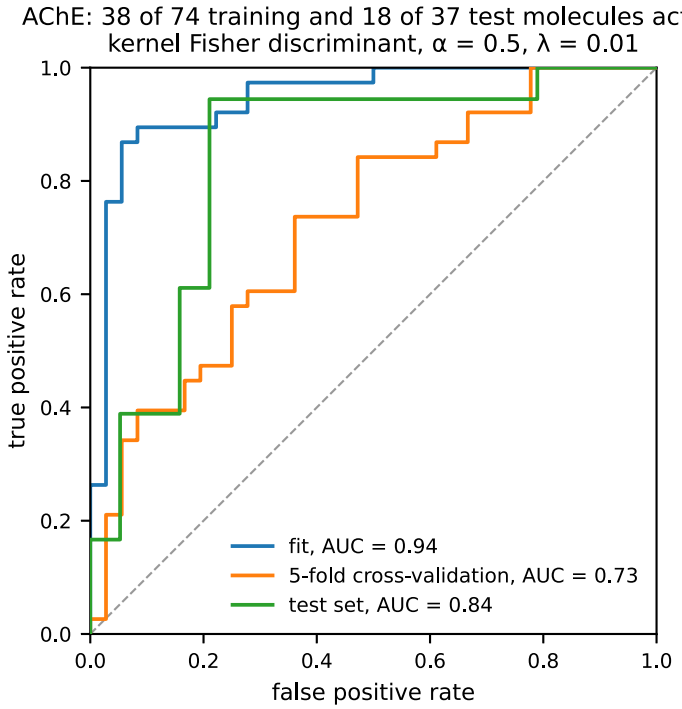
What the construction is for, then: a decoded field is a target a design step can aim at. `pre_image` returns one as ordinary `MoleculeFields`, `export_decoded` writes it as a cube file for the viewer, and `sample` draws codes around the training codes and decodes them -- which is a hypothesis field, not a molecule, and this manual labels it as such. The variational form that would make that sampling principled is version (iii) of the item and is not built.

9 Classification and virtual screening

9.1 Setting up a classification task

Choose *Binary classification* as the task. The classes come either from a threshold on the activity (with *active below* for data where small is potent, such as an IC50 in molar units) or from a class property in the file. The learners are C-SVC, ν-SVC and the Laplace Gaussian-process classifier; the *ROC* and *Scores* tabs replace the scatter plot.

```
python cmf_cli.py build examples/sutherland/AChE.sdf --charges file \
       --task classification --threshold 6.84 --method gpc --Cs 1,10,100
```



The ROC curves of a classification model of the AChE set with the classes of the command above, here the kernel Fisher discriminant of A classifier with a co-field on the split of the file: the fit on the training molecules, their five-fold cross-validation and the test set, each with its area under the curve. The curve of the fit says how well the model separates the molecules it was fitted on; the other two say how well it separates molecules it has not seen. The ROC tab draws the same curves.

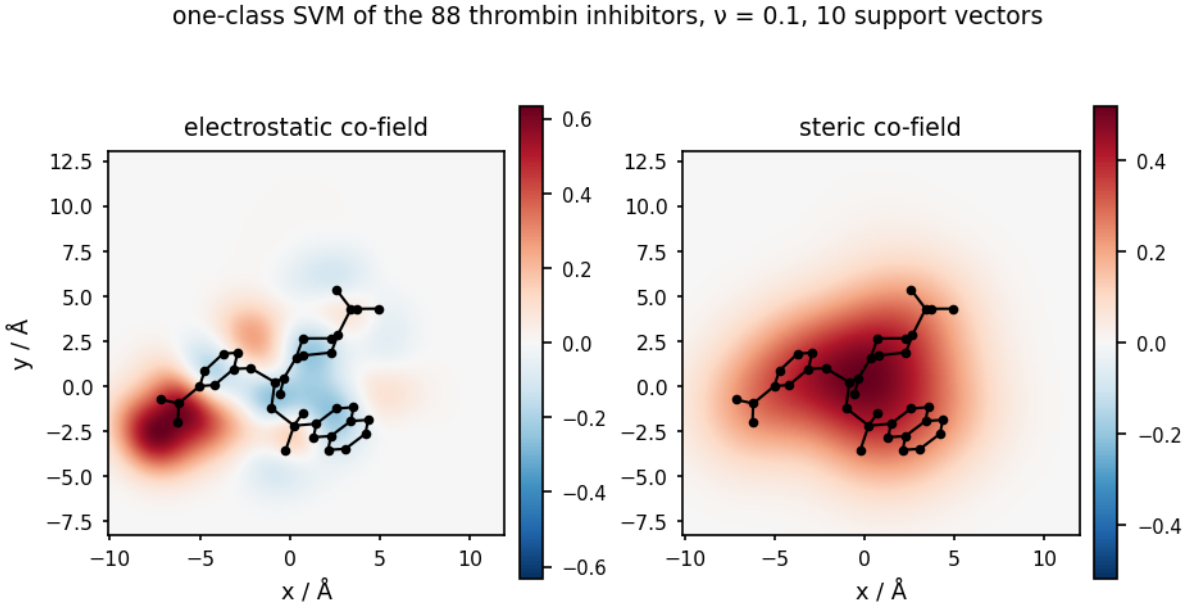
9.2 One-class models: screening with actives alone

Often there are no reliable inactives, only a set of known actives, and the question is whether a new molecule belongs to the same class. That is one-class classification, applied to CMF by Karpov et al. [7, 8]. Five descriptions of the active class are implemented:

Method	What it measures
`ocsvm`	the 1-SVM: a hyperplane in feature space separating the actives from the origin
`ocmean`	distance to the mean embedding of the actives, the centroid of the fields

Method	What it measures
`ocknn`	distance to the k-th nearest active
`ocgp`	the one-class Gaussian process, optionally scored by mean over standard deviation
`ockpca`	reconstruction error against the principal subspace of the actives (kernel PCA)

On the sets measured here the 1-SVM is the **most fragile** of the five: the centroid, k-NN and kernel-PCA descriptions are as good or better and have one parameter fewer to go wrong. Try more than one; they disagree in interesting places.



The co-field of the one-class SVM fitted on the 88 thrombin inhibitors alone, with the charges of the file, α = 0.5 and ν = 0.1, in the mean plane of the series, with the most active inhibitor drawn over it. A one-class model sees no inactives, so its co-field is a description of the active class: red is where the actives have field in common, the positive charge of the amidine at the left end in the electrostatic map and the whole skeleton in the steric one. The score of a new molecule is its overlap with this map, and the boundary is a threshold on that score.

9.3 Decoys and the figures that matter

A screening model is only as meaningful as the background it is tested against. *File → Add decoys (background set)…* or `--decoys` loads presumed-inactive molecules used **only** for evaluation, never for fitting. `examples/make_decoy_sets.py` builds property-matched decoy sets, which is the fair kind: decoys matched to the actives in molecular weight, logP, rotatable bonds and formal charge, so that the model cannot win by noticing size.

Two problems the program handles explicitly:

- **A decoy must not be rejected for its pose.** `--align-decoys` places every decoy in the frame of the training set by maximising its field similarity, with several random restarts, and `--decoy-conformers N` generates N conformations of each decoy and keeps the one that poses best (the file's own conformation is always among the candidates).
- **AUC saturates.** With cross-target decoys almost any method reaches 0.95 and above, which tells you nothing. Read the **enrichment factors** at 0.5, 1, 5 and 10 % instead, which the

program reports beside the maximum achievable value at each fraction, and **BEDROC/RIE** [16], which weight the top of the list.

A measured caution from the benchmarks in this repository: field similarity on these tasks performs about as well as ECFP4 fingerprint similarity, and cross-target decoys are trivially separable by either. If a screening result looks spectacular, check what the decoys were.

9.4 Measured inactives, and learning from positives alone

Every background in the previous section is *presumed* inactive, and a presumed inactive is an easier to reject than a measured one. A compound that was tested and did nothing is the kind of molecule a screen actually has to reject, and there is a public collection of them: **LIT-PCBA** [19], assembled from high-confidence PubChem dose-response assays — 15 targets, 7761 actives and 382674 inactives, every one with an experimental verdict. The AVE-unbiased subsets are the form the authors recommend, since the asymmetric validation embedding of Wallach and Heifets [20] removes the analogue bias that otherwise lets a screen score well by recognising that actives resemble actives.

What LIT-PCBA does not have is the thing a field method needs: a conformation and a pose. `examples/make_litpcba_examples.py` supplies them. For each target it embeds the actives and a seeded sample of the measured inactives with ETKDG and MMFF, then poses **both classes by one rule** — maximum field similarity to the target's crystal ligand, which the archive ships, so the frame is the one the receptor defines and the placement knows nothing about which molecule is active. The 57 MB archive is downloaded on demand and is not redistributed here; what is kept are the derived structures, `examples/litpcba/<target>.sdf` and `<target>_decoys.sdf`.

Why the posing rule is half the measurement

`--pose-against actives` poses the inactives to maximise their similarity to the posed actives instead, which is what `cmfqsar/decoys.py` does for the background sets of the previous section: no decoy is then rejected for having been posed badly. That sounds like the fair choice and is the generous one, for a reason that is not the obvious one. Posing a decoy where it most resembles the actives does make it harder to reject — but the pose search also *fails* on a molecule whose field cannot be made to match, and the similarity it settles at is then a shape-matching score in its own right. Optimising every candidate's pose against the actives silently makes the alignment part of the scoring function, which is what shape-based screening does on purpose and what a field model measured this way gets for free. The same molecules posed both ways say what it is worth, and the figure below says it is most of the result.

Unlabelled is not inactive

The compounds that came back negative in one assay at one concentration are **unlabelled** rather than inactive: some are inert, some were mis-measured, some would have worked at another concentration, and a few are actives nobody has recognised. Training a classifier on that mixture as a negative class teaches it to reject the very molecules a screen is looking for, and the damage is worst at the top of the ranking, which is the only part anyone reads.

`cmfqsar/pu.py` implements **bagging PU** [21], the method that needs neither an estimate of the class prior nor an assumption about how the positives were sampled:

1. draw a small random sample of the unlabelled set and call it negative;
2. fit on the positives against that sample;

3 score the unlabelled molecules that were *not* in the sample — out of bag, so a molecule's score never comes from a model that was told it was negative;

4 repeat, and average each molecule's out-of-bag scores.

An unlabelled molecule that looks like the positives is called negative in only a few bags and scores high in the ones that leave it out; one that is genuinely unlike them scores low wherever it appears. Everything works on a precomputed kernel, so the CMF kernel enters unchanged:

```
from cmfqsar.pu import pu_scores
scores = pu_scores(K, positives=train_actives, unlabelled=library, test=held_out, bags=30)
```

What it measures

Measured on two LIT-PCBA targets (ESR1_ant, TP53), 152 measured actives against 2999 measured inactives, 70/30 on the actives and half the inactives offered to the methods that can use them. AUC on the held-out actives against the held-out inactives:

method	ESR1_ant	TP53	what it sees
distance to the centre of the actives	0.536	0.365	the training actives
distance to the k-th active	0.515	0.382	the training actives
one-class Gaussian process	0.542	0.546	the training actives
distance from the kernel-PCA subspace	0.537	0.444	the training actives
ECFP4 Tanimoto to the actives	0.668	0.701	the training actives
classifier, inactives = negative	0.544	0.613	+ half the inactives, as negative
classifier, inactives = unlabelled (bagging PU)	0.662	0.715	+ half the inactives, as unlabelled

Three things in that table are worth saying plainly.

The one-class field descriptions do not work here. The best of them reaches 0.544 against 0.685 for the maximum ECFP4 Tanimoto to the same actives, and some are below chance (distance to the centre of the actives at 0.365 on TP53: the held-out actives look *less* typical of the training actives than the measured inactives do). Nor is that a badly chosen setting. Every parameter that can reorder the ranking was swept — the field attenuation over 0.3, 0.5, 1 and each description's own parameter, 45 fits per set — and keeping whichever scored best *on the test set*, a bound no legitimate protocol could reach, lifts the best description only to 0.574. (ν is not in that sweep, and cannot be: it places the threshold so that a fraction ν of the training actives falls outside the description, which shifts every decision value by the same constant and leaves every ranking metric unchanged.) What the gap reflects is the benchmark's design: the AVE debiasing removes the analogue bias on purpose, and a method whose entire content is "this molecule resembles the actives" then has nothing left to say.

The kernel is not what fails. The same CMF kernel used discriminatively, with the inactives treated as unlabelled, reaches 0.689 — level with the fingerprint, and +0.110 above the same learner told that the inactives are negatives. For screening against measured inactives, use the kernel discriminatively; the one-class descriptions are for the case the earlier sections describe, a

congeneric series against a structurally different background.

The posing is half of any screening number. Posed instead to maximise their similarity to the actives, the same inactives move the one-class AUCs by +0.141 on average while the fingerprint shifts by +0.000, because it never looks at a coordinate. That is the generous placement the background sets of the previous section are given, and the difference is a measurement of how much a field-based screening figure depends on how its decoys were placed.

9.5 Fitting in the space of the landmarks

The exact model holds one n by n matrix per field and factorises the n by n joint kernel. On a 32 GB machine that stops at ten to twenty thousand molecules: five field kernels of 10 000 molecules are 4 GB. Memory is the limit and not time. Plan item B6 assessed the ways out, found that one of them fits this kernel, and `cmfqsar/lowrank.py` is that one.

The construction. Choose m landmarks Z and replace every field kernel by

K(X, X) ~ K(X, Z) K(Z, Z)^+ K(Z, X),

the Nystrom approximation. That is the Gram matrix of the features `Phi = K(X, Z) W with W W^T = K(Z, Z)^+, so the model can be fitted in the space of the m columns instead of the space of the n molecules: (Phi^T Phi + lam I) w = Phi^T y is the same ridge as (K + lam I) a = y, in m by m memory. Nothing new is computed -- K(X, Z) is what the kernel module already produces -- and lowrank.gram accumulates Phi^T Phi and Phi^T y` over blocks of molecules, so the features of the whole set are never held at once.

Three kinds of landmark. `gaussian_centres` puts a cubic grid of one-atom landmarks over the frame: the rank such a basis needs depends on the frame and the width of the field, not on how many molecules there are, which is what makes the method worth having. `landmark_molecules` takes a subset of the molecules themselves, which has no width problem but is tied to the training set. `landmark_atoms` is between the two and is what a **narrow** field wants: one landmark on every atom of a few molecules, thinned to a spacing, each with the radius of the atom it stands on and unit weight in every field. A grid covers the box; the atoms cover the field, which is only ever large near an atom.

What stays exact. A molecule's self-kernel is its own integral and costs nothing, so it is never approximated. That matters twice. The cosine normalisation divides by the exact diagonal; and the posterior variance keeps the exact k(x, x):

var(x) = k(x,x) - phi^T phi + lam phi^T (Phi^T Phi + lam I)^-1 phi,

which is the exact Gaussian-process variance with K replaced by its approximation. Without that first pair of terms -- the plain subset-of-regressors form -- the variance falls towards zero far from the landmarks, which is the opposite of what a lower confidence bound needs.

The interpretation survives. A fit in the feature space is a dual fit whose coefficients sit on the landmarks, so `lowrank.cofield_gaussians` returns the same three arrays (centres, coefficients, widths) that `CMFModel.cofield_coefficients` returns for an exact model, with the index set changed from training molecules to landmarks. The co-field is drawn, read and decomposed exactly as before.

Measured by `benchmarks/run_lowrank.py` on the eight Sutherland sets:

- **It is the exact model when nothing is approximated.** With every molecule its own landmark, the kernel, the predictions, the log evidence, the posterior variance and the co-field on a grid all agree with the exact path to 2e-11 or better.

- **The approximation costs little on a broad field.** At an attenuation of 0.5, a grid of Gaussian centres 2 A apart is within 0.004 of the exact q2 on every set, and 20 to 80 landmark molecules within 0.076. The attenuation decides this, not the number of molecules.
- **At a narrow field the landmarks have to follow the atoms.** At an attenuation of 2 and above a grid 2 A apart reproduces the kernel only to 5e-01, where the atoms of the same molecules thinned to 0.5 to 0.75 A reach 2e-02 with a landmark count of the same order -- on GPB at an attenuation of 5, 6.5e-01 against 1.8e-02 for 259 landmark atoms. The q2 follows: +0.548 for the grid against +0.593 for the atoms, where the exact model gives +0.594. That is the case the assessment of this item named and left open.
- **What it saves is memory, and only beyond the crossing point.** At 361 molecules -- the largest aligned set here -- the exact kernels are 6 MB and the low-rank fit is 13 MB, because the rank of a 2 A grid over that frame is 1275. The low-rank memory does not grow with the molecules at all, so the two cross at about 1275 molecules and everything beyond that is saving. It is also eight times faster at that size, which was not the point: a kernel entry against a one-atom centre costs a thirtieth of one against a whole molecule.
- **The variance survives.** On molecules the landmarks do not reach, the form that keeps the self-kernel is off by 0.013 against the exact variance where the form that drops it is off by 0.025.

It is a flag on `build`, so the model is an ordinary model. `--low-rank grid:2.0` or `--low-rank molecules:200` fits this way, chooses the ridge by the evidence or on folds, and `--save` writes a file that `predict`, `cofield` and the rest read like any other:

```
cmf build series.sdf --activity pKi --low-rank grid:2.0 --alphas 0.5 --save model.cmf
cmf build series.sdf --activity pKi --low-rank atoms:0.5 --alphas 2 --save narrow.cmf
cmf predict model.cmf new.sdf --out predicted.csv
cmf cofield model.cmf --out-prefix map
```

A test set and a prediction interval, as on the exact path. `--test-fraction` leaves molecules out and the branch scores them: R2pred and RMSEpred are reported in the same block as the training statistics, because `evaluate_test` needs nothing but a prediction and the prediction is the one thing the approximation already provides. `--conformal LEVEL` calibrates on the out-of-fold errors the fit leaves behind, prints the half width, and reports what the interval covered on the test molecules, which were never part of the calibration. The interval travels in the model file, so `cmf predict --conformal` reports it from a saved low-rank model. `--predictions` writes both subsets with their `lower` and `upper` columns.

```
cmf build series.sdf --activity pKi --low-rank atoms:0.75 --alphas 2 \
    --test-fraction 0.3 --conformal 0.9 --predictions table.csv --save model.cmf
```

What this branch refuses is `--conformal-normalise`. A normalised interval divides each error by the distance of its molecule from every other training molecule, and that distance is read off the n by n kernel, which is the one matrix a low-rank model is fitted to avoid. The plain interval is the one to use here, and on the sets measured in `benchmarks/run_conformal.py` normalising gains no sharpness anyway.

What the file keeps is the landmarks, the whitening and one weight per feature instead of one coefficient per training molecule. The training table travels with it as before, because it is linear in the number of molecules and the provenance and the contract are read off it; what is never built, at any point, is the n by n kernel. `cmf lowrank` remains as the short path that fits and reports without saving anything.

What is not done. The low-rank branch of `build` does not take a second endpoint or write the JSON report, and the exact path is there for both. The inducing Gaussians take the field's own width; the application has no control for any of this; and the largest aligned set in the repository is 361 molecules, so the saving is shown where it is not yet needed. That last one is the alignment problem of the next part of the plan and not this item's to solve.

9.6 Screening a library that does not fit in memory

Everything above loads a data set, builds every molecule's fields and keeps them. That is right for a training set and impossible for a library: a hundred thousand molecules will not fit, and nothing about them needs to be kept once their score is written.

Predicting with a kernel method needs only the rows `K(X, X_train)` — one row per new molecule, as wide as the training set. So the `screen` subcommand reads the library in blocks and never holds more than one:

```
python cmf_cli.py screen model.cmf library.sdf --out scores.csv --beta 1.0 --top 25
```

It reads `--chunk` molecules (500 by default) with a forward supplier, so the file is streamed and never indexed; builds their fields with the model's own field set; computes the block's kernel rows; writes the scores to the CSV as it goes, so a run stopped after an hour keeps everything it had scored; and keeps only the best `--top` of them in memory. A SMILES file is accepted too and its molecules are embedded as they are read.

Measured against the same library loaded whole, scored by a 200-molecule model:

library	how it was read	seconds	molecules per second	peak process memory
4166 molecules (22 MB)	streamed, 500 per block	48	86	295 MB
4166 molecules (22 MB)	streamed, 100 per block	59	71	232 MB
4166 molecules (22 MB)	loaded whole	46	90	571 MB
16664 molecules (87 MB)	streamed, 500 per block	201	83	288 MB
16664 molecules (87 MB)	streamed, 100 per block	237	70	237 MB
16664 molecules (87 MB)	loaded whole	178	94	1706 MB

The memory column is what matters. Over a four-fold library the whole-file path grows 571 to 1706 MB, a factor of 3.0, while the streamed path at 100 molecules per block stays at 232 and 237 MB. Extrapolating the first to 100 000 molecules asks for about 10 GB; the second would still be using 237 MB. The price is 33 % of wall clock, from parsing the file in blocks rather than once.

What the score is. For a regression model it is the prediction, or with `--beta B` the lower confidence bound mean $-$ B sd, which is what a library should normally be ranked by, and the reason the standard deviation is always in the CSV: a library is mostly molecules unlike the training set, a kernel method answers the training mean for those, and a point estimate cannot be told from a real prediction. Screen the thrombin series against a cyclooxygenase model and every molecule comes back at the training mean with a spread four times the residual error — which is the model saying, correctly, that it has nothing to contribute.

What it does not do. It does not invent poses. A field kernel compares molecules in one frame, so the library has to be in the training set's frame already, or `--pose` has to be given, which places each molecule by maximising its field similarity to the training molecules. That is the placement the background sets of the benchmarks are given, it costs about half a second per molecule — far more than the scoring — and it is optimistic, because a real library has to choose a pose without knowing the actives. `--pose-conformers N` generates N conformations of each molecule and poses every one, which multiplies the cost by N.

9.7 Which molecule to make next

The model predicts an activity and, through the Gaussian-process view, a standard deviation beside it. Those two numbers answer a question [6] poses and leaves as a statement — that maximising the overlap tells one how to modify the molecule — as an operation with a number:

```
python cmf_cli.py suggest model.cmf candidates.sdf --acquire ei --top 10
```

The candidates have to be posed in the training set's frame, as for `predict`. What comes back is the ranked list with the prediction, the standard deviation, and the distance to the nearest molecule already made.

The rule depends on what the asking is for, and the program keeps the two apart rather than pretending one serves both.

`--acquire`	what it maximises	what it is for
`ei`	expected improvement over the best made so far	finding an active
`ucb`	prediction + β standard deviations	the same, more crudely
`greedy`	the prediction	the same, with no allowance for being wrong
`variance`	the posterior standard deviation	improving the model
`maxmin`	distance in field space to everything made	the same, with no model at all
`random`	nothing	the baseline the others have to beat

Expected improvement counts only the upside, which is why it is the usual choice:

$$\mathrm{EI}(x) = (\mu - y^{*} - \xi)\,\Phi(z) + \sigma\,\varphi(z), \qquad z = \frac{\mu - y^{*} - \xi}{\sigma} \tag{9.1}$$

Note that this is the **opposite** of how the same two numbers are used to rank a screening library, where `rank_score` subtracts the standard deviation instead of adding it. The difference is not a matter of taste. A screening library is dominated by molecules unlike anything in the training set, a kernel method returns the training mean presented as a prediction for exactly those, and a molecule chosen on that basis is wasted. A molecule *made* to answer a question is different: being wrong about it costs little, and being wrong is informative. Optimism is right for one and wrong for the other.

What it gains, measured

Eight Sutherland sets, ten runs each, starting from twenty molecules drawn at random that hold no hit and acquiring a quarter of what is left one at a time. α and λ are reselected by the evidence at every step from what has been acquired and nothing else — which is what the evidence was built for. A **hit** is a molecule in the top 5 % of its set.

Finding an active:

rule	best found, of the best in the set	runs that found a hit	median acquisitions to the first hit
random	92.2 %	79 %	6
maxmin	90.9 %	64 %	9
variance	90.6 %	71 %	9
greedy	95.0 %	100 %	4
ucb	**96.6 %**	95 %	4
ei	95.2 %	89 %	**3**

Improving the model, as the RMSE on everything not yet acquired (against the variance of the whole set, so that a rule which acquires the extremes is not rewarded for making the rest look alike):

rule	mean RMSE over the run	RMSE at the end	q^2 at the end	against random, paired
random	1.122	1.064	+0.366	—
maxmin	**1.108**	1.047	+0.384	−0.014 [−0.034, +0.006]
variance	**1.108**	1.042	+0.386	−0.014 [−0.034, +0.006]
greedy	1.179	1.195	+0.158	**+0.057 [+0.038, +0.078]**
ucb	1.144	1.127	+0.279	+0.022 [+0.001, +0.044]
ei	1.121	1.086	+0.331	−0.001 [−0.022, +0.021]

The optimistic rules work and the exploring rules do not. For finding actives, `ucb` reaches 96.6 % of the best activity in the set against 92.2 % for picking at random, hits in a median of four acquisitions against six, and the paired gain in hit rate is +0.163 [+0.062, +0.275]; `ei` hits soonest of all, in three. For improving the model, the best exploration rule gains −0.014 RMSE over random with an interval of [−0.034, +0.006] — zero is inside it, so on these sets **exploring gains nothing measurable**, while `greedy` is clearly worse, by +0.057 [+0.038, +0.078].

The two tables are kept apart because an acquisition rule cannot be judged without saying what the acquiring is for, and three things in the numbers are worth more than the ranking.

The uncertainty rule and the diversity rule are the same rule. `variance` and `maxmin` give the same result, which is the error-prediction finding seen from a different direction: the posterior standard deviation of a CMF model is very nearly the distance to the training set in other units.

Acquiring by what the model does not know and acquiring by what the series does not cover are the same experiment with two names.

Exploring gains nothing measurable. The molecule the model is least sure of is the molecule least like the series, and what is learned from an outlier does not transfer to the molecules anyone will ask about next. Uncertainty sampling is a good rule for a space one intends to cover and a poor one for a series one intends to understand.

`greedy` **does better here than it would in general.** It ignores the posterior entirely, so a candidate unlike anything made is predicted at the training mean and cannot be told from a real answer. These sets are series of analogues, where nothing is far enough away for that failure to matter; on a library of diverse candidates the protection is gone, and `ei`, which at least tempers its optimism with the posterior, is the rule to use.

9.8 Designing the next compound

`suggest` ranks candidates somebody has already drawn. The design loop makes the suggestion itself: take a parent compound of a series, take one of its attachment points, put every fragment of a library there in turn, and rank what comes out.

```
python cmf_cli.py design series.sdf --parent 45 --position 2 --acquire ei --top 15
```

The command does the whole thing: it finds the scaffold, builds a **topomer model** of the series, chooses α and λ by the evidence, reports the q^2 that the suggestions rest on, enumerates, scores and ranks. There is no model file to pass, because the model has to be a topomer model and the core and the attachment points come from the series anyway.

Why a topomer model and not an aligned one. A substituent nobody has made has no crystal structure and no alignment, and superimposing it on anything would be a guess. The topomer rule builds a fragment into a pose that is a deterministic function of its own topology, so a candidate is posed by exactly the rule that posed the molecules the model was fitted on, and the comparison is like with like at one attachment point. A model fitted on superimposed whole molecules has no way to place a group that was never synthesised, which is why the command builds the topomer model itself instead of taking a model file: there is no aligned model that could answer the question.

Three numbers come back for each candidate, and they answer different questions: the **prediction**, the **posterior standard deviation** (a fragment unlike anything in the series gets the training mean and a wide spread, which is what stops the loop from proposing molecules it knows nothing about), and the **acquisition score** that combines them — the same rules as `suggest`, expected improvement by default. Then the annotations: whether the series already uses that group at that position, whether the whole molecule has been made and what it measured — the only free check there is on a suggestion — and, for the top few, what the candidate's fields have that the parent's do not.

The library is `--fragments common` (a built-in list of about forty ordinary medicinal-chemistry substituents), `series` (what this series already puts there), `both`, or a file of SMILES with one attachment point each.

Would it have suggested the right group, measured

Leave one substituent out: every molecule carrying a given R-group at the varied position is removed, the model is refitted on what is left, and the whole library is enumerated and ranked from one fixed parent. The group's measured activity is the mean over the molecules that carry it.

set	groups held out	position	q² of the models	ρ of the ranking with the measurements	the group it puts first	library mean	library best
THER	46 of 46	0	+0.251	**+0.279**	6.53	5.24	10.17
GPB	41 of 47	2	+0.566	**+0.516**	4.40	2.94	6.80

Over 87 held-out groups the ranking agrees with the measurements at a mean Spearman ρ of **+0.397**, and the group the loop puts first is worth **+1.37** log units more than the library average — it lands in the best 20 % of the library **48 %** of the time. The sharper question is where the groups that really are good end up: of the five best held out on each set, the median rank is 7 of 42 on GPB and 12 of 46 on THER, and the single best lands at 13 and at 32. A loop run twice over would have found most of them and missed one.

Only 2 of the eight sets can be reached at all: ACE (the core comes off cleanly in only 44% of the series), AChE (the core comes off cleanly in only 26% of the series), BZR (the core comes off cleanly in only 42% of the series), COX2 (the core comes off cleanly in only 22% of the series), DHFR (the core comes off cleanly in only 12% of the series), THR (the core comes off cleanly in only 25% of the series).

What it cannot do

The loop varies **one position of one parent**, and it states plainly the three things that this limits.

It needs a series with a scaffold that comes off at acyclic single bonds. Of the eight Sutherland sets only two are that; on the rest the Bemis–Murcko framework is fused into the molecules and what comes off is not a substituent. The command says so instead of ranking the pieces anyway.

It ranks groups, not conformations. The topomer pose is a convention, and a suggestion is a statement about which group belongs at a position, not about how it will sit when it is there.

And it cannot tell you about a position the series never varied. A model learns from contrast; a site that is the same in every training molecule carries no information about what else might go there, and the posterior standard deviation is the only thing that will say so.

A series that has no scaffold: the invariant route

The three limits above belong to the topomer route. `--kernel autocorrelation` takes the other one (plan item SC7): the rotation-invariant kernel needs no frame, so **a candidate is built as a whole molecule and embedded once**, and nothing is superimposed on anything. The series then needs no common scaffold at all -- the core is only what names the position being varied on the parent -- which is what makes a design loop possible on a table of guests, where the molecules share nothing.

```
cmf design examples/cyclodextrin/beta_cyclodextrin.csv --kernel autocorrelation \
    --core "C(=O)O" --parent "CC(=O)O" --acquire greedy
```

Two things change with it. There is no field difference to report, because there is no frame to draw one in; in its place each candidate carries **how much of its prediction comes from the group that was put in**, which the invariant model's atom contributions give exactly. And the acquisition score needs watching: on a table that is not a congeneric series the best molecule is far above anything the loop can reach, expected improvement is then the same for every candidate, and the

command says so and asks for `--acquire greedy` rather than presenting an order that is no order.

Does the ranking mean anything? `benchmarks/run_guest_design.py` answers with a leave-one-group-out test on the 1162 guests of the beta-cyclodextrin table. For each group of an eleven-group library of R-COOH, **every guest carrying that group is removed**, a model is fitted on what is left, and the library is ranked by it; a group's prediction therefore comes from a model that never saw a molecule containing it. A group carried by more than 30% of the table cannot have all of its guests removed -- a methyl is in every one of them -- and those rows remove the molecule alone and say so.

group	guests removed	predicted	rank	measured
methyl	1 (the molecule only)	+1.545	11/11	+1.301
propyl	1 (the molecule only)	+1.864	10/11	+1.505
butyl	1 (the molecule only)	+2.022	9/11	+1.954
pentyl	1 (the molecule only)	+2.397	6/11	+2.467
hexyl	261	+2.483	3/11	+2.041
phenyl	1 (the molecule only)	+2.081	8/11	+2.127
4-methylphenyl	1 (the molecule only)	+2.435	5/11	+3.225
cyclopentyl	83	+2.134	7/11	+3.204
cyclohexyl	122	+2.243	6/11	+3.620
cyclohexylmethyl	111	+2.613	2/11	+3.519
adamantyl	6	+3.297	1/11	-

Over the 10 groups whose acid is a measured guest the ranking follows the measurements: Spearman +0.709. **The two kinds of row are not equally hard**, and the mean over both would hide it: where only the molecule was removed the model had seen hundreds of its relatives and the mean absolute error is 0.26 over 6 groups, while where every guest carrying the group was removed (adamantyl, cyclohexylmethyl, hexyl, cyclohexyl, cyclopentyl) it is 0.95 over 4, with a bias of -0.73. A group the table no longer contains is pulled towards the mean, which is what a model should do with a group it has never seen.

What survives that is the order at the top. Adamantyl -- whose six guests in the table measure 2.9 to 4.3 against a median near 2, and whose own acid is not in the table at all -- comes **first of the 11** with every one of those six removed, and cyclohexylmethyl, the strongest of the measured groups that were fully held out, comes second. Taken over every row the 4 groups measuring 3.0 or more have mean rank 5.0, so the list is worth reading at the top and not in the middle. That is the claim a design loop has to make, and it is the one this measurement supports: it will name the group worth trying, not tell you what it will measure.

9.9 Replacing the scaffold: scoring only where the co-field has weight

The section above replaces a substituent and keeps the scaffold, because the topomer rule needs a scaffold to pose against. Replacing the *scaffold* leaves no frame, and comparing two whole molecules then answers mostly with how similar their skeletons are -- which is the one thing a scaffold hop has decided to change. Plan item MC5 is the way through: stop comparing the molecules everywhere.

A **mask** is a soft region of space, a sum of Gaussians on the atoms whose contribution to the prediction is largest (`cofields.top_atom_mask`). Multiplying the co-field by it leaves the co-field inside that region and nothing outside, and because the product of two Gaussians is a Gaussian the masked co-field is again a sum of Gaussians, so every integral stays closed form (`kernels.gaussian_product`, `kernels.expansion_overlap`). `cmfqsar.maskedmatch` gives three numbers for every candidate:

number	what it is
inside	the part of the prediction that comes from the masked region, in the units of the activity
outside	the rest of the prediction: field the candidate places where the model rewards none
cosine	the same comparison with the size of the molecule divided out, the cosine in the inner product the mask defines

```
from cmfqsar.cofields import top_atom_mask
from cmfqsar.maskedmatch import MaskedScorer, screen_conformers

mask = top_atom_mask(model, training_fields, k=10)   # the ten atoms that carry the prediction
print(mask.describe())                               # which atoms, and what each contributes
best = screen_conformers(model, candidates, mask)    # one row per molecule, its best conformer
print(best[0])
```

Measured, on the question the item asks and on a harder one. A model is fitted on *one* chemotype of a series -- the largest Bemis-Murcko group -- and the molecules of every other chemotype that are at least as active have to be ranked above a background. Nothing of those scaffolds is in the training set, which is what a hop is. (`benchmarks/results/maskedmatch/table.md`, mask of 10 atoms, AUC of the ranking.)

background	masked cosine	masked overlap	field similarity	the model's prediction
the weakly active molecules of the same other chemotypes	**0.729**	**0.743**	0.566	0.718
the 100 property-matched presumed inactives	0.584	0.637	**0.918**	0.657

The two rows answer in opposite directions, and the reason is the same in both. The mask throws away everything about a molecule except what it does where the model rewards field -- including its size and its overall shape. Against decoys from other targets that is exactly the information that separates the two groups, so the mask makes the ranking worse and the unmasked field similarity is already at its ceiling (0.918, and 1.000 on two of the four sets). Against molecules of the same target in the same frame, where nothing but the activity distinguishes the two groups, the mask gains **+0.163** AUC over that same field similarity and is ahead on three of the four sets. So the gate the item states -- the hops above the decoys, by the masked score rather than by field similarity --

fails on its own background and passes on a harder one, and the second measurement is the one worth keeping.

What the mask does not do is beat the model. Predicting the activity of the candidates and ranking by that gives 0.718 against the masked cosine's 0.729 and the masked overlap's 0.743, a difference smaller than the spread between the sets. Two of the four single-chemotype models have a negative q^2 (THR -0.138, ACE -0.420), so two of the four rows are rankings from models that do not predict at all; of the two that do, GPB (q^2 +0.587) puts the prediction first (0.877 against 0.862) and AChE (+0.096) also puts it first (0.672 against 0.643).

What is worth using is the split, not the ranking. On every one of the four sets, and at every mask size, the active molecules of the unseen scaffolds collect more of their prediction inside the mask than the weak ones do -- on GPB +0.477 against +0.121, on AChE +0.309 against +0.078 -- so the inside number says how much of a candidate's promise comes from the region the model has evidence about, and the outside number says how much of it is extrapolation. That is a reading a chemist can act on, and it is what this construction should be used for.

It does not pose anything. The candidates must already stand in the model's frame, as for `cmf screen`; a mask makes the comparison local and does not make the placement free.

9.10 A design loop with more than one objective

The suggestion rules of the earlier section close the loop on potency alone. Lead optimisation has at least two objectives, and since the two-task model they live on one kernel: one model predicts the potency and the anti-target activity of the same candidate, each with a posterior. Plan item MC11 is what goes between that model and a chemist's list, and it is `cmfqsar/multiobjective.py`; the retrospective run at the end of this section is `benchmarks/results/designloop/table.md`.

The Pareto front is the candidates no other candidate beats on every objective at once. A weighted sum needs weights, and nobody knows them before the compounds are made; a front does not. `pareto_layers` gives the front and the fronts behind it, so a list can be as long as the chemist wants.

The acquisition is the expected improvement of each objective against the best already made, added with weights. Each term is already in the units of its own objective's improvement, so the weights say how much a log unit of one is worth against a log unit of another and nothing is hidden in them.

The batch is the rule of the previous section with more than one task: after a pick, every objective's posterior covariance is conditioned on it and the acquisition is computed again. What that takes away is the uncertainty and not the promise -- two compounds the model is sure are a log unit better are both still worth making -- so the spreading acts through what the model does not yet know, which is the only honest place for it to act.

Every suggestion comes with the trade it makes: `multiobjective.trade` reports, against the best compound already made, which objectives a candidate gains and which it loses, with a change smaller than the model's own error reported as no change. A loop that gives only a score hides the trade it is making.

What happened when it was run backwards

The series of the previous section comes in two parts, which makes a retrospective test possible. The loop is trained on the 78 compounds of part 1 with both their endpoints, and then scores 425 candidates: every other histamine H3 ligand in ChEMBL that fits the same frame. 16 of them are the

compounds **the same authors went on to make** in part 2 of the paper. The loop sees no activity of any candidate.

the list the loop reports	candidates	of them made next	share
the Pareto front	5	0	0.0%
the first 10 by the acquisition	10	0	0.0%
the first 50 by the acquisition	50	0	0.0%

The gate is not met. The front holds none of them, and the compounds the authors made sit at Pareto layer 23 of 47 against 19 for the rest of the pool, at rank 343 of 425 against 208: the loop puts them in the worse half. The best of them reaches layer 2 and rank 79, so they are not invisible to it.

Three things explain the miss, and they are worth more than the verdict. The model does see that these compounds are good: their measured H3 affinity averages 9.15 against 7.98 for the rest of the pool, and the model predicted 8.68 against 8.59 -- the order is right and the size is not, which is what a q2 of +0.28 on a narrow series buys. What sinks them in the ranking is the other objective: the model predicts their hERG activity at 6.29 against 5.89 for the pool, and an acquisition that adds improvements prefers a candidate extreme in one objective to one that is good in both. And no hERG number was ever measured for them, so whether the model is right about that cannot be checked.

The last point is the real limit of this test. The compounds the authors made next were chosen with knowledge the loop does not have -- synthetic access, a chemical idea, the rest of a programme -- so the ones they did not make are not known failures, and only one of the two objectives can be checked against what happened. What the item asked for and what this provides is the machinery: a front over several objectives on one kernel, a batch taken from it by the rule of the previous section, and for every suggestion the trade it makes, written out.

9.11 An anti-target beside the potency

Where may field be added for the target that is **not** also added for the off-target? The two-task model answers it as a difference of co-fields, and the section on two endpoints above measured that on the Am/Eu extractants. Plan item MC4 asks the same question in its medicinal-chemistry form, and what it needed was data: one series with a potency and an anti-target activity measured on the same compounds.

The series is `examples/antitarget/h3_herg.sdf`, built by `examples/make_antitarget_examples.py` from one ChEMBL document, which is one paper [25]. The paper's own subject is this question. 78 compounds carry both numbers: the H3 affinity as a displacement of [125I]iodoproxyfan, and the hERG activity as a displacement of [3H]dofetilide, **one assay each, in one laboratory**. That matters, and the assay descriptions travel with the file as columns: an anti-target number from another paper is a different measurement. The two endpoints have the same spread, 0.68 log units each, and correlate only -0.22, so there is a selectivity to find rather than a single axis seen twice.

The first part of the gate: the potency must not suffer. The task correlation chosen by the evidence is -0.431 -- in this series a compound good for H3 tends to be worse at hERG -- and with it:

endpoint	q2 with the other endpoint	q2 on its own
pKi_H3	+0.341	+0.319
pIC50_hERG	+0.357	+0.338

On the potency the two-task fit is +0.022 [-0.005, +0.048] against the single fit, paired over the molecules: not worse, so the selectivity co-field is not bought with a worse model of the thing that matters. The selectivity itself is predicted at q2 +0.471, better than either endpoint alone, which is the claim of [10] met again on a different kind of series.

The second part: where the difference co-field puts its weight. `cofields.atom_effect` says what one more atom of a given field type, at a given place, would do to the predicted selectivity, in log units, with the error the activities allow. Read at the basic nitrogen of every molecule, at the carbons within 5 A of it, and at every other heavy atom:

place	field	effect on the selectivity
the basic nitrogen	steric	+12.84 [+9.68, +15.99]
the basic nitrogen	hydrophobic	+6.96 [+5.46, +8.47]
the carbons beside it	steric	+11.56 [+8.86, +14.26]
the carbons beside it	hydrophobic	+7.35 [+5.92, +8.78]
every other heavy atom	steric	+0.03 [-2.39, +2.45]
every other heavy atom	hydrophobic	+0.70 [-0.77, +2.17]

The weight is of one sign and it is concentrated exactly where the item says hERG is driven from: at the amine and the carbons beside it two fields stand clear of the noise and both are positive, while over the rest of the molecule neither does. A co-field that said the same thing everywhere would be saying nothing about the basic nitrogen in particular.

The sign is the opposite of what the item expected, and that is the interesting part. Lipophilicity does drive hERG here as the textbook says: cLogP correlates +0.38 with the hERG activity and -0.31 with the selectivity over a range of 3.0 to 7.0. But the co-field is about a **place** and cLogP is about a molecule, and what this series says is that the lipophilicity costing selectivity is the lipophilicity spread over the rest of the molecule, while the region around the basic nitrogen is where the H3 affinity is made. That is also what the paper did: it kept that region and changed what is far from it. A whole-molecule number cannot make that distinction, and it is the only thing a co-field adds.

```
        cmf build examples/antitarget/h3_herg.sdf --activity pKi_H3 --second-endpoint pIC50_hERG \
            --save h3.cmf
        cmf cofield h3.cmf --endpoint selectivity --out-prefix selectivity
```

`benchmarks/run_antitarget.py` is the measurement above. It is one series from one paper, with hERG measured by displacement rather than by patch-clamp and an alignment by common substructure rather than by a crystal structure; a second series, measured another way, is what would turn this reading into a rule.

9.12 Choosing ten compounds at once

A laboratory makes ten or twenty compounds at a time, not one. The acquisition rules of the previous section pick one molecule and refit; a whole plate has to be chosen before any of it is measured, so something has to keep the plate from being one molecule ten times over.

The cheap rule has been in `cmfqsar/active.py` from the start: take the best scores in order, and skip a candidate whose cosine with a pick already made is above a threshold. It works, and it has a

threshold in it that nothing fixes.

The rule the posterior already implies is what plan item MC2 asks for. After a pick, the posterior covariance conditioned on that pick says how much uncertainty is left everywhere else:

cov' = cov - cov[:, j] cov[j, :] / cov[j, j],

which is one step of a Cholesky factorisation with `j` as the pivot. The acquisition is then evaluated again on the conditional standard deviations, so a candidate whose uncertainty the first pick has already removed scores low by itself, with no threshold and no refitting (`active.condition_on`, `active.batch_by_covariance`). With the **variance** acquisition the score is the conditional standard deviation, so the rule is the greedy maximisation of the determinant of the batch's own covariance block -- the classical D-optimal design -- and a test checks the greedy batch against the exhaustive best on a small matrix.

`model.predict_with_covariance` returns the whole posterior covariance of a block of candidates, which is what the rule needs and what `predict_with_variance` only gives the diagonal of.

Measured by `benchmarks/run_batch.py` with the protocol of the previous section -- the eight Sutherland sets, twenty molecules to start, the attenuation and the ridge reselected by the evidence at every fit, ten repeats, the same starting set for every rule -- in batches of ten:

how the batch is chosen	fits	best acquired	error on what is left
one at a time	35	8.878	1.034
ei, similarity filter	3	8.902	1.041
ei, covariance	3	8.939	1.025
variance, similarity filter	3	8.540	1.029
variance, covariance (largest determinant)	3	8.403	1.043
random	3	8.709	1.011

The gate is not met, and the item says that is an answer. Paired run by run, the covariance rule minus the filter is +0.037 [-0.015, +0.110] in the best acquired and -0.016 [-0.035, +0.002] in the error for expected improvement, and -0.137 [-0.267, -0.013] and +0.014 [-0.008, +0.037] for the variance acquisition. None of the four favours the covariance rule beyond its interval and one favours the filter, so the cheap filter is enough. That is the outcome plan item E2 predicted from the posterior standard deviation being nearly flat over these pools: if conditioning on a pick barely moves the ordering, the two rules choose almost the same plate by different arithmetic.

What a batch costs is nothing measurable here, which is the more useful finding. Against the one-at-a-time loop, which sees every measurement before choosing the next molecule and pays thirty-five fits for it, expected improvement with the filter differs by +0.025 [-0.025, +0.079] in the best acquired and +0.008 [-0.016, +0.033] in the error. A plate of ten chosen at once is worth as much as ten chosen one after another, at a tenth of the fitting.

Both rules are on the command line:

```
cmf suggest model.cmf --candidates library.sdf --kind ei -n 10 --batch-rule covariance
```

9.13 Growing a bound fragment into the gap

A fragment that binds fills part of a site and leaves the rest empty. The question a fragment campaign asks next is what to put there. `cmfqsar/fraggrow.py` answers it with the fields alone, which is what plan item MC7 asks for, and the answer it gives is a measured no.

The construction. The pocket's offer at a point is its own field with the sign convention of the complementarity section: the electrostatic field with a minus, because a positive ligand region should meet a negative protein region; the hydrophobic field with a plus; the pocket's donors paired with the ligand's acceptors and the other way round. The **gap** is that offer in the space that is free -- free of the protein, and free of the fragment, which is how "minus what the fragment already places" is taken here. An offer is open only where a new atom could sit. (The other reading, subtracting the fragment's own field from the pocket's offer, compares two different quantities and erases exactly the contact region a new group would use; it was tried and it does not work.)

The candidates are built from their SMILES with a distance-geometry embedding and a force field, never from a crystal structure, and placed on the growth vector by rule: the origin at the atom that grows, the x axis along the direction that atom's own bonds leave empty, the bond length the sum of two covalent radii, and the turn about the new bond scanned. The true answer is built and placed exactly like every other candidate, so nothing about it enters the geometry. A placement whose atoms come closer than 2.5 A to the protein is refused.

Measured by `benchmarks/run_fraggrow.py` on 25 growing cases from the fragment sets of the Schrodinger free-energy collection -- the only public sets here with a pocket and with the fragment and the compound it grew into in one crystal frame -- against a library of 201 groups taken from the ligands of the same collection:

ranking	median rank of the true group	among the first ten	first
at random	82	0%	0%
by whether it fits, at random inside that	15	40%	0%
by whether it fits, then by the size of the group	28	12%	0%
by whether it fits, then by the field score	25	28%	0%

The gate is not met, and the field score is worse than not scoring at all. Refusing the placements that clash and shuffling the rest puts the true group at a median rank of 15 of 201, and finds it among the first ten in 40 per cent of the cases. Sorting those same candidates by the field score moves it to 25 and 28 per cent. The score is not neutral: it rewards whatever fills most of the gap, so it prefers the largest group that still fits, while the additions in these cases are a fluorine, a methyl, a chlorine. What carries the ranking is the geometry -- whether the group fits at all -- and the field adds nothing to it.

Under rule 5 that makes the gap a display tool: a picture of where there is room next to a bound fragment and what the pocket would reward there, which is worth drawing before a chemist chooses a group, and not a way of choosing one. `fraggrow.gap` returns it on a grid, with one array per field type, ready for the same cube export as any other field of this program.

What the measurement cannot settle. The library holds 201 groups and the true one is always in it, which is the easy part of a real enumeration. The cases are few, because a public set with a pocket, a fragment and the compound it grew into is rare. And every candidate gets one embedded conformation and a turn about the new bond, where a real growing tool would minimise the group

inside the site. A negative result here is evidence against the one-body field score, not against fragment growing.

9.14 A matched pair as a field placed in space

A matched molecular pair is usually a row in a table: a transformation, written as two group names, and the activity change it produced. In a series whose molecules are in one frame it is also something else -- **a difference of two fields**, a local object sitting where the two groups differ, with its shape and its charge kept. Plan item MC6 asks whether that object carries the effect of the transformation with it: can a change measured in other series predict the change in a series the model has never seen?

Plan item D5 is the warning and the reference. Increments fitted on one series did not transfer, and the vectors fitted on two sets agreed to +0.89 at best. An increment is a number attached to an atom type; the field difference is the same transformation with its geometry kept, so it can tell a methyl that points into a pocket from one that points at the solvent. Whether that is enough is a measurement.

The construction (`cmfqsar/transformfield.py`). The two members of a pair are already in one frame, since the series is aligned, so their fields are comparable with each other. What they do not share with another series is a frame, so the difference field is placed in a frame of its own: the origin at the core atom that carries the group, the x axis along the bond to the group, and the plane fixed by one more neighbour of that atom, chosen by atom type so that both members pick the same one. The difference field is the second group minus the first, each restricted to its own atoms by the mask of plan item MC1, and it is an ordinary field object: every kernel and every display of this program applies to it. The overlap kernel is bilinear, so reversing a pair reverses its kernel row, and a ridge with no intercept therefore predicts exactly the opposite change for the reverse of a pair -- the one property such a model must have.

The data are the only sets here with a crystal pose for every ligand: the ten Merck KGaA FEP series of Schindler and co-workers and the eight Schrodinger JACS series, 3814 matched pairs in 17 congeneric series. The folds hold out a whole series, the ridge is chosen inside the training series the same way, and q2 is taken against each series' own mean change, which is the strongest form of the reference the item names.

what predicts the change	q2 against each series' own mean
no change at all	-0.025
the transformation as a field, inside one series (no transfer)	+0.465
the transformation as a number (plan item D5)	-0.073
the transformation as a field, without a frame	-0.025
the transformation as a field	-0.025

The gate is not passed, and the reason is not the construction. Across series the model reaches q2 -0.025, with an interval over series of -0.118 to +0.035: it predicts nothing, and the ridge chosen in every fold is at the strong end of the grid, which is the model saying so. The same pairs described without a frame, as the rotation-invariant autocorrelation of the difference field, give the same -0.025, so a frame convention is not what went wrong. Trained **inside** each series, where nothing has to transfer, the same difference fields reach q2 +0.465 with whole molecules

held out. The descriptor works; the transfer does not.

The data say it before any model is fitted. Of the twelve transformations that occur in most series, eleven raise the activity in one series and lower it in another:

transformation	series	pairs	mean change	lowest	highest
methyl to chloro	10	23	+0.15	-0.93	+1.07
methyl to ethyl	8	13	-0.17	-1.15	+1.01
fluoro to methyl	7	14	+0.12	-0.53	+1.37
chloro to cyano	5	12	-0.48	-1.05	+0.09
isopropyl to ethyl	5	8	+0.30	-0.64	+1.62
methyl to methoxy	4	13	-0.14	-1.49	+0.40

Replacing a methyl by a chlorine is worth about a tenth of a log unit on average and anything between -0.9 and +1.1 depending on the protein. There is no single effect for a field or for a number to carry, and that is the answer to the item rather than a failure of the machinery.

What is worth keeping is the object itself. `transformfield.average_field` builds the mean difference field of all the pairs of one transformation, which is a picture of what that transformation does to the field, and the within-series row above says that inside one series the difference field predicts a change well. The place for it is inside a project, on one target, where plan item MC1's table already uses the same masking.

9.15 Which groups are interchangeable: a bioisostere table

`cmf design` ranks candidate substituents by the predicted activity of the whole molecule. The table a medicinal chemist asks for answers a different question: **which groups can replace each other at this position**, so that one of them can be chosen for a property the model knows nothing about -- solubility, cost, a patent, a metabolic liability. Plan item MC1 builds that table (`cmfqsar/bioisostere.py`).

It needs no new model. The prediction of the overlap kernel is a sum over the atoms of the molecule (decomposition rule 3), so the part of a prediction that belongs to a substituent is the sum over that substituent's own atoms, and the predicted effect of exchanging group A for group B at one position is the difference of those two sums. That is one integral per pair of groups instead of one model fit per group. The masking is exact rather than approximate: the kernel is a sum over pairs of atoms, so restricting one side of it to a subset of the atoms is the same thing as building the fields of that subset and nothing else, and `masked_field` does that. A test checks the split: for any division of a molecule in two, K(whole, whole) is K(a,a) + 2K(a,b) + K(b,b).

Three ways to say how far apart two groups are, which is the ladder of rule 1:

1. the **fingerprint distance** between the two groups, one minus their Morgan Tanimoto: two dimensions, no geometry, no model;
2. the **field distance** between the two posed groups, restricted to their own atoms: geometry, but no model. This is not a weak reference -- it already knows the shape and the charge each group puts where;
3. the **predicted change**, the difference of the two atom-contribution sums: geometry and the co-field, so it also knows whether the activity depends on the place where the two groups differ.

Measured by `benchmarks/run_bioisostere.py` on matched molecular pairs found by RDKit's `rdMMPA` -- one single bond cut at a time, both pieces tried as the core, two molecules with the same core and different groups, nothing chosen by eye. Each measure is compared with the size of the measured activity difference by rank correlation. For every pair the dual coefficients are refitted **without both of its members**, so the predicted change has not seen the two activities it is judged against.

set	pairs	fingerprint	field	co-field (pair left out)	co-field (whole model)
GPB, 66 molecules	730	-0.025	+0.005	+0.561	+0.631
THER, 76 molecules	142	+0.084	+0.162	+0.277	+0.641
THR, 88 molecules	751	+0.144	+0.242	+0.335	+0.711

The co-field weighting earns its place. On GPB and THR the predicted change ranks the measured activity difference better than both references by more than the interval of rule 4: against the fingerprint distance +0.586 [+0.505, +0.669] and +0.191 [+0.106, +0.279], against the field distance +0.556 [+0.479, +0.632] and +0.093 [+0.008, +0.181]. On THER it is ahead of both as well, +0.193 [-0.059, +0.437] and +0.116 [-0.089, +0.313], but the interval contains zero; that is the set with the fewest pairs. On no set is it worse than either reference.

How much of it is memory is the gap between the last two columns, and the gap is large: the whole model, which has seen both activities of every pair it judges, reaches +0.631, +0.641 and +0.711, while refitting without the two members leaves +0.561, +0.277 and +0.335. The honest column is the one above. Two things still come from the whole set and are worth naming: the field types, the attenuation and the ridge were chosen on all the molecules, and the alignment is the published one.

What the table is read for is the size, not the ranking. A predicted change smaller than twice the conformal half width is a change the model cannot see, and the groups inside that bound are the ones the table calls interchangeable. That part is the model's own error bar rather than a claim about the chemistry. On a model of GPB whose half width is 1.5 log units the bound covers every pair of the 35 groups at that position, which is the table saying honestly that this model cannot separate them; what it still gives is the order, and the groups it puts closest to the carboxamide are the methyl carbamate (+0.02), butyramide (+0.04) and acetamide (+0.05).

```
cmf isosteres model.cmf --molecules series.sdf --activity pKi --csv table.csv
```

The command finds the position carrying the most different groups, prints for each group the partners the model puts closest with the predicted change of the exchange, marks those the model can actually tell apart, and writes the whole square table.

9.16 The gradient of a prediction

Everything above ranks candidates that somebody enumerated: a library of fragments, a table of guests, a set of poses. A prediction is also a differentiable function, and the derivative answers a different question -- not *which of these* but *which way*.

The kernel is a sum of Gaussian overlaps over pairs of atoms, and a Gaussian overlap is an analytic function of the distance between the two atoms. So the prediction of a molecule is an analytic function of the positions of its atoms and of the per-atom weights that carry its fields, and both

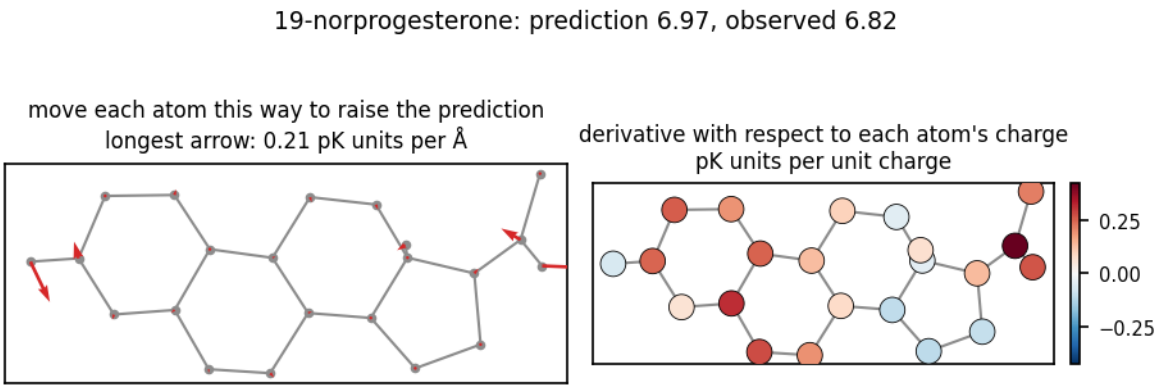
derivatives are available in closed form:

```
from cmfqsar.gradients import prediction_gradient

g = prediction_gradient(model, mol)
print(g.value)                      # the prediction, the same number predict() gives
print(g.coords.shape)               # (N, 3): d(prediction) / d(atom position)
print(g.weights.shape)              # (N, F): d(prediction) / d(per-atom field weight)
print(g.largest_weight_terms(3))    # the three atoms and fields that matter most
```

Nothing has to be enumerated first, and nothing is approximated: the normalisation, the field scaling and mixing, the nonlinear kernel layer and the interpolation between attenuation grid points are all differentiated with the kernel. The non-field descriptors contribute nothing, because they are computed from the molecular graph and do not change when an atom moves.

The co-field is the potential the atoms move in. For an unnormalised linear model the two readings of the model are the same statement. The derivative of the prediction with respect to the position of one atom is the atom's weight, times the gradient of the co-field at the atom's own position, smoothed by the atom's own Gaussian, summed over the fields. `cofield_gradient` evaluates that gradient anywhere, with or without the smoothing, and needs no molecule -- only a place to stand. A design rule that was a picture (*put something electronegative in the red lobe*) becomes a force.



The gradient of one prediction, for a test molecule of the steroid set under the model of chapter 8. Left: the derivative with respect to every atom position as an arrow, the direction in which moving that atom raises the predicted affinity; the two oxygens carry nearly all of it and the arrows on the carbons are below drawing size. Right: the derivative with respect to every atom's partial charge, red where more positive charge raises the prediction. Both come from one closed-form evaluation, and the first is what the flexible alignment and the generative uses of the next paragraph follow.

Three uses. A three-dimensional generative model proposes atom positions, and a gradient lets a fitted CMF model steer that proposal instead of only judging the result. A pose or a conformation can be refined by predicted activity, the way `varalign` and `flexalign` refine it by confidence. And the derivative with respect to one atom's weight is a local reading of the model: how much the prediction would change if that atom carried a little more charge, or were a little more hydrophobic -- the question a co-field picture is usually asked, answered as a number per atom.

Is it the derivative, and what does it cost

The only independent check is a finite difference of `predict` itself. A correct gradient disagrees with it by the square of the step until round-off in the predictions takes over, so a scan over the step shows a minimum, and the depth of that minimum is the accuracy of the comparison rather than of

the gradient (`benchmarks/run_gradient.py`).

set	molecule	atoms	step 1e-2	1e-3	1e-4	1e-5	1e-6
steroids	prednisolone	54	3.4e-04	3.4e-06	3.4e-08	**6.2e-09**	4.1e-08
THR	01	75	5.4e-05	5.4e-07	**5.4e-09**	2.3e-08	2.1e-07

The alternative to the analytic form is that finite difference, which costs two predictions for every degree of freedom -- 3N for the positions and N per field for the weights.

set	atoms	degrees of freedom	analytic	finite differences	gain
steroids	54	432	5.6 ms	1.07 s	190 times
THR	75	600	29 ms	7.4 s	258 times

What happens if you follow it

Steepest ascent on the atom positions of one held-out steroid, with every step capped so that no atom moves more than 0.05 A. The last two columns are the important ones.

step	prediction	largest displacement	rmsd	posterior sd	worst bond length change
0	+7.934	0.00 A	0.00 A	0.234	0.00 A
1	+8.149	0.05 A	0.01 A	0.247	0.06 A
5	+8.875	0.25 A	0.07 A	0.356	0.29 A
10	+9.534	0.50 A	0.12 A	0.508	0.54 A
20	+10.329	1.00 A	0.22 A	0.701	0.80 A

The prediction rises by 2.4 log units, which is more than the whole range of some series. Two things rise with it. The posterior standard deviation triples, so the molecule is leaving the training set at the same rate as the number is improving. And the bond lengths are pulled apart by up to 0.8 A, because nothing in the gradient knows that a molecule has bonds. The same walk on a thrombin ligand gains 2.0 log units and distorts a bond by 0.44 A.

The same gradient without a frame

Everything above compares the atoms of one molecule with the atoms of the training molecules, which is why it needs a common frame. The rotation-invariant kernel does not: its channels are the distance distribution of the field weights *inside* one structure, as the chapter on the rotation-invariant autocorrelation kernel describes. That kernel is differentiable in closed form as well, and the derivative is assembled in three steps.

A channel is a double sum over pairs of the structure's own atoms, so a position enters only through the distances of the pairs that contain that atom, and the derivative of a channel is a sum over the other atom of each pair. The model is then one vector in channel space, because the kernel is a dot product of the scaled rows: the sum over the training molecules collapses into the same vector the atom-wise decomposition uses. And the cosine normalisation divides each row by its own length, which adds the one extra term the overlap kernel does not have -- the derivative of that length, the

number item M15 of the development plan measured on its own.

```
        model = CMFModel(fieldset=fs, kernel_source="autocorrelation", ac_d_max=8.0, ac_sigma=0.5)
        model.fit(table, y)

        g = prediction_gradient(model, mol)     # the same call, the same two derivatives
```

Because no channel knows where the structure sits or how it is turned, this gradient moves a *conformation* and never a pose: turn the molecule and the gradient turns with it. A rigid motion leaves the prediction exactly where it was, which is why `cmfqsar.relax` refuses to relax a pose under an invariant model instead of returning zeros.

set	sample	atoms	fields	channels	closest agreement	analytic	finite differences	gain
steroids	prednisolone	54	5	255	3.1e-09	14 ms	6.1 s	432 times
THR	01	75	5	255	5.2e-09	21 ms	11.9 s	565 times
hop environments	one jump of mp-10620	22	10	935	1.9e-08	22 ms	9.1 s	414 times

The last row is the case this was wanted for. A hop environment is a cut-out of a crystal around the midpoint of a lithium jump, with its atoms weighted by an envelope; it has no frame of its own, which is why the hop model uses the invariant kernel. Its 935 channels and ten fields are differentiated as fast as the 255 channels of a molecular set, because the cost is set by the atom pairs and not by the channels.

Four relatives of this kernel are **not** differentiated: the in-plane surface channels with their heights, the periodic channels with their images and their uniform background, the reciprocal-space structure factors, and the signed three-point channels, whose sum is over triples of atoms rather than pairs. Each builds its channels in its own way and needs its own derivative; each is refused with that reason rather than approximated.

Relaxing a pose under it

A gradient is a direction, and a relaxation is what follows it to a stop. `cmfqsar.relax` does the one motion that leaves the chemistry untouched: the molecule is moved as a rigid body, six parameters about its own centroid, so every bond length and every field weight is what it was and only the pose changes.

```
        from cmfqsar.relax import relax_pose

        out = relax_pose(model, mol, max_rmsd=2.0)      # the pose where the model predicts most
        print(out.value_before, out.value_after, out.rmsd, out.limited)
```

`max_rmsd` is how far the atoms may move before a quadratic penalty holds them back, as in the confidence alignment; a relaxation that ends at the guard is reported as `limited`, because its answer is then the guard's and not the model's.

Does the gradient reach what the same objective reaches without it? Yes, and it is the reason to have it. The same search driven by the analytic gradient and searched without one reach the same value to three decimals, and the gradient needs 24 to 86 times fewer evaluations of the objective: 29 against 2,508 on a steroid, 87 against 2,116 on a thrombin ligand.

Is that the pose the confidence aligner finds? No, and the difference is the point. Started from the same pose, the prediction and the posterior variance move the same molecule to places one to two angstroms apart.

set	moved by	prediction	posterior variance	distance from the other pose
steroids	the prediction	+9.93	1.34	1.33 A
steroids	confidence	+7.77	0.04	1.33 A
THR	the prediction	+9.30	1.36	2.19 A
THR	confidence	+7.45	0.07	2.19 A

Neither column is wrong. Raising the prediction moves a molecule towards the training molecules that are *active*; raising confidence moves it towards the training set as a whole, with redundant members down-weighted by the inverse kernel. So the two answer different questions, and which one to ask is decided by what the pose is for. For a pose to predict from, use the confidence: it uses no activity value of the molecule being placed, so it is legitimate before anything is measured. The relaxation answers the other question -- where a molecule would have to sit for this model to call it active -- and its answer is a statement about the model, not about a receptor.

What it cannot do

A weight is not a molecule. The weight of an atom in the electrostatic field is its partial charge, and the charges of a real molecule are not free: they sum to its total charge and they follow from its structure. A positive derivative says the model would like more of that quantity there. It does not say that a molecule exists in which only that one atom has changed. The weight gradient is a direction in the field representation, to be realised by choosing another atom or another substituent.

Maximising an overlap is not designing a molecule. The table above is the demonstration: the overlap with a positive lobe of a co-field grows without limit as atoms are piled into that lobe. The gradient is a term to add to a generative model or a force field that enforces geometry, and not an optimiser to run on its own.

The model still has a domain. A step along the gradient goes where the training set is not. The posterior standard deviation, the conformal interval and the two-sample test are what say how far it can be trusted, and the lower confidence bound of *Which molecule to make next* is the objective to follow when the aim is a molecule to make rather than a number to maximise.

It needs a frame. The gradient is defined for the overlap-integral kernel, which is the one that needs the molecules superimposed. The rotation-invariant, surface and periodic kernels are built from the distances inside each molecule, and the program refuses them with that reason instead of returning a number.

10 Conformations: the fourth dimension

A 3D-QSAR model represents each molecule by one conformation, and the answer depends on which one. 4D-QSAR [12] replaces the choice by an ensemble and averages over it. *Model → Conformations* switches between the two; **single is the default and is unchanged**, and the tests assert that by comparing its fields against those built before ensembles existed.

In CMF the averaging needs no new mathematics. A field is a sum of atom-centred Gaussians and the kernel is bilinear in the fields, so the mean field of an ensemble is itself a field — its atoms are the atoms of every conformation with their weights multiplied by that conformation's weight — and the kernel of two ensembles comes out as

$$K(A,B) = \sum_{k=1}^{n_A} \sum_{l=1}^{n_B} w_k^A \, w_l^B \, K\left(X_A^{(k)}, X_B^{(l)}\right) \tag{10.1}$$

where the sums run over the n_A conformations of ensemble A and the n_B conformations of ensemble B, each conformation X enters with its weight w, and the weights of each ensemble sum to one. This is the mean-map kernel of the two conformational distributions, positive definite by construction. A test checks that identity against the pairwise kernels of the conformations. The model stays linear in that mean field, so **co-fields, the decomposition rules, Shapley values and the complementarity scores all keep working**; contributions arrive per atom per conformation and fold back onto the atoms by summation.

Tools → Generate conformations... builds the ensembles with ETKDG and MMFF94s, keeping the pose a molecule already has as its first conformation, so an earlier alignment survives. A multi-conformer file can be read instead, with `--ensemble-by name` grouping the records of one compound. Weights are uniform or Boltzmann from the stored energies, with kT and a cap on the number kept.

Where the other conformations go is a choice of its own, and not a small one: what matters is not where a conformation sits relative to its own molecule but where it sits relative to the rest of the series.

`--conformer-align`	**Where each generated conformation is put**
`self`	on the molecule's own pose, over all heavy atoms — the default, so nothing changes silently
`core`	on its own pose, over the common substructure with a reference only, so the scaffold stays where the published alignment put it and only the flexible part moves
`reference`	on a reference molecule of the series, through that same common substructure, which puts every conformation of the series in one frame
`field`	as `reference`, then refined by maximising the normalised CMF kernel — the criterion of [2], applied per conformation

Multi-instance learning (`cmf_cli.py mil`) goes further: instead of averaging, the model chooses which conformation of each molecule to use, alternating between fitting and selecting. Use it when you believe one conformation is the bioactive one and the others are noise; use the ensemble when you believe the molecule is genuinely flexible.

10.1 The variance of an ensemble: a flexibility map

The ensemble above represents a molecule by the **mean** of the fields of its conformations. A mean throws away how much the field moved between them, and that is a flexibility map: where a molecule's field is the same in every conformation, and where it is not. Plan item D10 asks whether that is worth carrying.

It is a field of the same class, which is what makes it cheap to ask. With the Gaussians written as exp(-b/2 |r - c|^2), the variance across conformations is

V(r) = (1/k) sum_m X_m(r)^2 - ((1/k) sum_m X_m(r))^2,

and every term is a product of two Gaussians. A product of two Gaussians is a Gaussian, so V is again a sum of Gaussians: at the **midpoints of atom pairs**, at **twice the width**, with a coefficient that falls off as the two atoms move apart. `cmfqsar/fluctuation.py` builds it, and `variance_fieldset` gives the field set it must be used with -- the widths are doubled, so a variance field and an ordinary one must not go into the same kernel call. The construction is exact and the tests check it against the variance evaluated directly on a grid.

What it costs. The pairs are what costs: k conformations and n atoms give k^2 n^2 pseudo-atoms before anything is dropped, so the field grows with the square of the molecule. A floor on the pair coefficient keeps that in hand -- pairs whose two atoms are far apart contribute almost nothing -- and `kept_share` says what the floor dropped. On the sets measured here that is 1065 to 2119 pseudo-atoms per molecule at an attenuation of 2 and a floor of 0.01, keeping 99% of the pair weight, and a kernel over a set takes 286 seconds at most. Hydrogens are dropped before the construction for the same reason: their fields are small and they double the atom count.

It does not help.

set	molecules	the mean alone	the variance alone	the best mixture	difference
the 31 CBG steroids, the classic CoMFA benchmark	31	+0.794	+0.425	+0.785 at 0.10	-0.009 [-0.079, +0.062]
glycogen phosphorylase b, 66 molecules	66	+0.460	+0.180	+0.462 at 0.10	+0.002 [-0.015, +0.020]
thrombin, 88 molecules	88	+0.308	-0.013	+0.302 at 0.10	-0.006 [-0.032, +0.023]

Every mixture that includes the variance is at or below the mean alone, and every interval contains zero or lies below it, so under rule 1 the block adds nothing. The best mixture is the smallest one offered on all three sets, which is what a block that is not helping looks like when a search is made to choose it.

What the variance carries by itself runs out as the series grows. Alone it reaches q2 +0.425 on the 31 steroids, +0.180 on the 66 of GPB and -0.013 on the 88 of thrombin: something on a small set of one rigid scaffold, nothing on a larger and more varied one. And on none of them is it something the mean does not already have. That is the reading the plan expected, and it leaves the variance field where it is useful: as a map. Where a series is rigid and where it moves is a different question from where it is active, and the two pictures are worth putting side by side -- `cmfqsar/cofields.py` draws a variance field like any other, with the field set that `variance_fieldset`

returns.

10.2 Scoring a conformational restriction

Tying two parts of a molecule together into a ring, so that a bond can no longer rotate, is one of the few moves in medicinal chemistry whose effect is meant to be purely conformational. The variance field of the previous section is a map of where a molecule places field only part of the time, which is exactly what such a move is supposed to change. Plan item MC9 asks whether it can be turned into a number: when a chemist closes a ring, does the field that matters stop moving, and does the potency follow?

Two amounts per molecule (`cmfqsar/restriction.py`). The co-field C_f(r) says how much the field of type f at point r matters for the activity, so its **square** is a weight: positive everywhere, large where the model reads the field, near zero where it does not. With that weight a molecule has

- **relevant**, the integral of C_f(r)^2 Xbar_f(r)^2 -- how much activity-relevant field the molecule places, averaged over its conformations;
- **intermittent**, the integral of C_f(r)^2 V_f(r) -- how much of that field is there only part of the time.

Both are summed over the field types and taken on a grid, the same grid machinery that draws a co-field. A pair of a flexible molecule A and its restricted analogue B is then two shares: **kept** = relevant(B)/relevant(A), and **removed** = 1 - intermittent(B)/intermittent(A).

The pairs are found by a rule and not by eye, so that the measurement cannot be steered. The restricted member has exactly one ring more, at least one rotatable bond fewer, at most two heavy atoms more, an atom composition differing by at most two atoms, and a maximum common substructure covering at least 80 per cent of the smaller member. A fingerprint is deliberately **not** used for the similarity: closing a ring changes the environment of every atom near the new bond, so butylphenol and 2-methylchroman -- the same eleven heavy atoms, one bond more -- are only 0.15 alike as Morgan fingerprints, and a threshold high enough to be useful would throw away exactly the pairs this item is about. Both members get the same conformer generator with the same settings, since the comparison is between two ensembles and not between two molecules.

Measured by `benchmarks/run_restriction.py` on four aligned Sutherland sets, 439 pairs in all, with three scores on the same pairs:

score	pairs	correlation with the potency gain
rotatable bonds removed	439	+0.236 [+0.151, +0.317]
variance removed, unweighted	439	-0.034 [-0.140, +0.062]
variance removed, weighted by the squared co-field	439	+0.043 [-0.030, +0.111]
the same, with the pair left out of the co-field	439	+0.020 [-0.061, +0.093]
relevant field kept, weighted	439	+0.220 [+0.120, +0.315]

The score does not pass the gate of the item. How much intermittent field a ring closure removes is not related to whether the closure gained potency: +0.043 [-0.030, +0.111], an interval that contains zero. Counting the rotatable bonds the closure removed reaches +0.236 [+0.151, +0.317] on the same pairs and needs no geometry, no conformations and no model. The co-field is

also rebuilt without both members of every pair, so the weight cannot have learned the answer from the two potencies it is compared with; that changes the score to +0.020, which says there was nothing to learn.

What does track the potency gain is the other half of the score. How much of the activity-relevant field the restricted analogue still places reaches +0.220 [+0.120, +0.315]. That is the **mean** field and not the variance, and it says the expected thing: a ring closure helps when the molecule still puts its field where the model reads it, and hurts when the ring holds it somewhere else.

This is the third place in this manual where the conformational ensemble carries less than it promises. Averaging a field over conformations does not predict better than one geometry; the variance adds nothing as a block of the kernel; and it does not explain what a restriction did either. The variance field stays a map, which is where the previous section left it.

From the command line. The score is a subcommand of its own, because it needs a model (for the co-field), a set of molecules in that model's frame, and conformations of each:

```
cmf restriction model.cmf --molecules series.sdf --activity pIC50 --conformers 3 \
    --csv pairs.csv
```

It finds the pairs by the rule above, prints the two shares of each, correlates **removed** with the potency gain when the file carries potencies, and writes one row per pair. It is the right tool for asking the question of a series of your own; the answer on the four public sets above is that the question has a negative answer there.

Two limits of the measurement. The ensembles are three conformations from a distance-geometry generator superimposed on the pose the series was aligned to, which is a narrow view of what a molecule does in water. And a ring closure is never purely conformational in practice: it changes the shape, and often an atom or two as well. A set of pairs that differ in nothing but a single bond would be a better test, and no public set of that kind with measured potencies was found.

10.3 Protonation and tautomers: ensembles of different molecules

A series measured in acid is modelled in one protonation state, chosen when the data was prepared. That choice is a guess wherever a pKa falls near the pH of the experiment: an azole with a pKa of 1 in 1 M HCl is about half protonated, and neither the neutral form nor the cation is the molecule that was measured. The same applies to a tautomer: an azole can be written with its proton on either nitrogen, and the two forms have different fields.

The averaging needs no new mathematics, and the members need not be one molecule. The argument of the previous section used only bilinearity: the mean field of an ensemble is itself a field whose atoms are the atoms of every member with their weights multiplied by that member's weight. Nothing in it requires the members to be conformations. They can be protonation states, or tautomers, which are different molecules with different numbers of atoms (`cmfqsar/protonation.py`, `cmfqsar/tautomers.py`). The kernel is still the mean-map kernel, still positive definite, and the model is still linear in the mean field, so the co-fields and the decomposition rules keep working. Tests check that identity against the pairwise kernels of the members for both kinds of ensemble, and that a conformational ensemble built through the general builder reproduces the tiled one atom for atom.

Option of `build`	What it does
`--ph 0`	represent every molecule by the pKa-weighted mean field of its protonation states at this pH
`--ph-states N`, `--ph-min-population F`	how many states to keep, and how light a state may be
`--tautomers score`	the mean field over the tautomers of the state the file gives
`--tautomers uniform`, `--tautomers given`	keep every enumerated form, or weigh the forms by energies in the molecule property `tautomer_energies`
`--tautomer-forms N`	forms kept at most

The weights of the protonation states are populations. Each ionisable site is found by Dimorphite-DL's patterns, given its regression pKa, and weighted by the Henderson-Hasselbalch fraction

$$f = \frac{1}{1 + 10^{\mathrm{pH} - \mathrm{p}K_\mathrm{a}}} \tag{10.2}$$

for the protonated form and its complement for the deprotonated one; a state's weight is the product over the sites. **The weights of the tautomers are not populations.** A tautomer has one only if the free energies are known, and none are computed here, so `score` keeps the forms RDKit scores highest and counts them the same. Energies from a semiempirical calculation would replace that assumption with a Boltzmann weight.

The states are built on the geometry the molecule arrives with: the heavy atoms keep their coordinates and only hydrogens are added, removed or moved, so **an aligned series stays aligned** and the ensemble costs no further superposition. Atom contributions come back on the atoms of the molecule as usual; a proton that only one state carries is reported on the heavy atom it sits on, which is the atom a chemist would read it on.

What it is worth, measured

`benchmarks/run_protonation.py` measures four models on the 23 corrosion-inhibitor series in 1 M HCl (997 molecules, `examples/corrosion_hcl`), with the same five fields, the same attenuation and ridge grids, the same ten-fold cross-validation and the same scaffold alignment as `run_corrosion.py`: `file`, the state the data file gives; `dominant`, the likeliest state by pKa; `ensemble`, the pKa-weighted mean over the states; and `tautomer`, the mean over the tautomers of the file's state.

Model at pH 0	mean q^2	best on
`file` (the state the data file gives)	**+0.324**	13 of 23 series
`dominant` (the likeliest state by pKa)	+0.260	3 of 23
`ensemble` (the pKa-weighted mean)	+0.261	1 of 23
`tautomer` (the mean over the forms)	+0.314	6 of 23

At pH 0 the ensemble has almost nothing to average over. Only 1% of the molecules are a real mixture in 1 M HCl, because a pKa two units away from the pH already settles the question, so

`ensemble` differs from `dominant` by +0.001. Where the two differ is in which state is chosen, and the pKa choice is the worse one here: -0.064 against the state the data file gives, with `file` best on 13 of the 23 series. The likeliest state by pKa carries the charge the file gives it for only 52% of the molecules, and its mean charge is +1.57 against +0.77 in the file: treating the sites as independent protonates a second and a third basic nitrogen that would not both be protonated in reality. **So the limit is the pKa model, not the ensemble.** Averaging the tautomers moves the result by -0.010, on the 5% of molecules that have more than one form worth keeping.

Where a mixture does exist, the ensemble has something to do: the share of molecules that are a real mixture is 1% at pH 0, 18% at pH 2, 83% at pH 3 and 27% at pH 7.4.

And it is not the lack of mixtures that holds the ensemble back. Repeating the measurement at pH 3, where 83% of the molecules are mixtures and a molecule has 3.0 states on average, gives the same order and no gain from the averaging: `file` +0.324, `dominant` +0.255, `ensemble` +0.246, which is -0.009 for the averaging itself. The endpoint was measured in 1 M HCl, so a pH-3 model is not the chemistry of the experiment; it is there to show what the averaging does when there is something to average, and the answer is nothing measurable.

When to use it. On this evidence, not as a default: the state prepared by hand gave the higher mean q2 at both pH values measured and was the best of the four models on 13 of the 23 series. Use it as a check -- if the ensemble and your chosen state disagree, the pKa of that molecule is near the pH and the state is worth looking at -- and use it as the model only where no one has chosen the states by hand. Read the mean charge the program prints beside the ensemble: where it is far above the charge of your file, the independent-site assumption is adding protons that a real molecule would not carry, and the ensemble is then worse than the single state, not better. The pKa values are Dimorphite-DL's and cannot be replaced from the command line yet; tautomer energies can be, through the molecule property `tautomer_energies`. Use the tautomer ensemble where the series contains azoles or other groups whose form the file decided arbitrarily, and read its numbers as the weaker claim they are. A conformational ensemble and a protonation or tautomer ensemble cannot be combined yet: the program refuses rather than build a product of members it cannot describe.

11 Working without an alignment

This chapter is the one to read if your molecules came from SMILES, if your series has several scaffolds, or if you simply do not trust the superposition you have. The program offers three frame-free routes, and the scripts in `benchmarks/` measure all of them.

11.1 Why the frame is the problem

An alignment is an assumption, and a wrong one is expensive. Measured on the nine targets that occur both in the curated Sutherland collection and in the automatically aligned Mittal collection — same compounds, same activities, same fields and grids — the curated superposition gives a mean q^2 of **+0.597** and the automatic one **+0.295**. Worse, the gain over a baseline with no geometry at all (atom-type counts) is +0.25 on the curated files and **−0.06** on the automatic ones: on five of the nine targets, fields in an automatic frame predict *worse* than counts. An alignment that is not trustworthy is not merely imperfect, it is actively harmful.

11.2 The pose-free count kernel

As α → 0 an indicator field tends to the count of atoms of that type, so the kernel tends to the dot product of the atom-type count vectors and nothing of the geometry survives. The program implements that limit in closed form (`chemotypes.count_kernel`), which is exact rather than approached, costs one pass over the atoms, and is 10 to 34 times faster than the Gaussian route. It is the **baseline** any aligned model should be made to beat, and it is what the leave-one-chemotype-out workflow reports beside the aligned model.

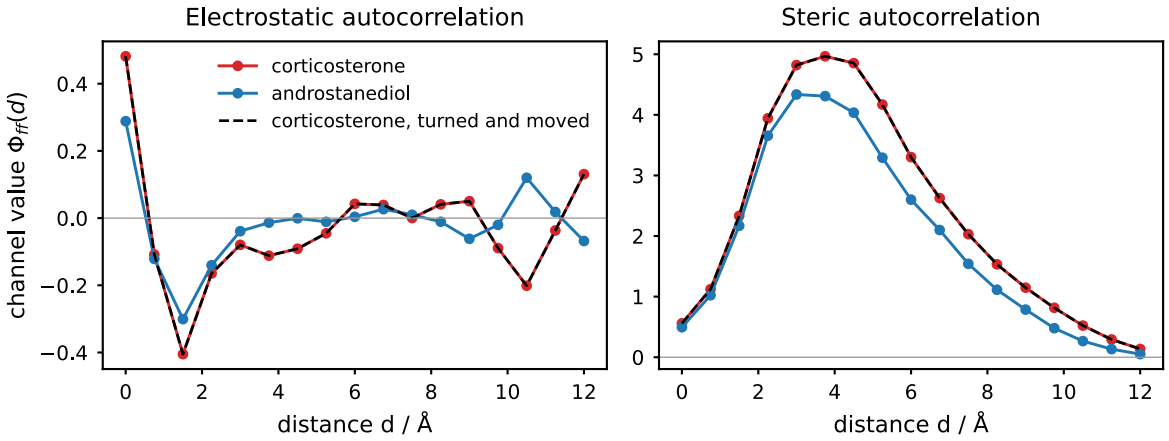
One subtlety, because it is easy to get wrong: the normalisation must be applied to the joint kernel, not field by field. A field normalised on its own tends to 1 whenever both molecules contain the type at all, so the counts cancel and the baseline loses the very descriptors it is built from.

11.3 The rotation-invariant autocorrelation kernel

Geometry without a frame. For each pair of field types the program computes the smoothed field autocorrelation

$$\Phi_{fg}(d) = \sum_{l}\sum_{m} w_{lf} w_{mg} \exp\left(-\frac{(|\mathbf{r}_l - \mathbf{r}_m| - d)^2}{2\sigma^2}\right) \tag{11.1}$$

as a function of interatomic distance d, which is invariant under rotation and translation by construction.



The autocorrelation channels of two steroids, the most and the least active of the training set, for the electrostatic and the steric field: one value per distance, every 0.75 Å, smoothed with a width of 0.75 Å. The black dashed line is the first molecule after an arbitrary rotation and translation, and it lies exactly on the red one: the channels depend on the conformation and on nothing else. The electrostatic channels change sign with distance because the charges do; the steric ones are a smoothed histogram of the interatomic distances weighted by bulk.

Where the two molecules are. That equation describes *one* molecule: both sums run over its own atoms, which is what makes it an autocorrelation. It is a descriptor, not a kernel — one number per channel *c*, a channel being one pair of field types at one distance — and the kernel is the cosine-normalised dot product of two such vectors:

$$K(A,B) = \frac{\sum_c s_c^2 \, \Phi_c(A)\, \Phi_c(B)}{\|s\,\Phi(A)\|\;\|s\,\Phi(B)\|} \tag{11.2}$$

with s_c the scale channel *c* inherits from its block: the reciprocal root mean square of that block over the training molecules, times the block's mixing weight. This is the opposite construction to the overlap kernel of *How a prediction is made*, which is defined on two molecules from the start and has no finite feature vector at all — its feature map is the field itself. That reversal is also what decides how each model can be read afterwards: a dual expansion over a feature map that is a function of space is itself a function of space, which is the co-field, while a dual expansion over channels is a vector of channel weights, which is the co-pattern.

The $d = 0$ channel reproduces the CMF self-overlap exactly when $\sigma^2 = 2/\alpha$, so the descriptor contains the ordinary kernel's diagonal information and adds the distance profile. Select *Autocorrelation* as the kernel source in the model panel (`--kernel autocorrelation`, with `--ac-sigma` and `--ac-dmax`), and the model fits, cross-validates, saves and predicts exactly as any other — only the co-field, which needs a frame, is unavailable.

On the chemotype benchmarks the invariant kernel recovers most of the value of the geometry without a frame (0.172 against 0.189 for the aligned model and 0.072 for counts), and on automatically aligned files it frequently beats the aligned model outright — on six of the nine shared targets, including thrombin at +0.452 against +0.021.

Two points, and what that misses. The descriptor counts pairs of atoms at a distance, and a mirror image has every pair at exactly the same distance, so this kernel cannot tell a molecule from its enantiomer. It is blind for the same reason to the orientation of one field with respect to another — whether the donor sits clockwise or anticlockwise from the bulk, going round the ring. The quantity that is not blind is the signed volume of three displacement vectors, and `--ac-chirality`

adds it as a second block of channels, the signed triple products of three field types resolved over `--ac-chirality-shells` radial shells (four by default). The block is appended rather than mixed in, so the two-point part is unchanged to the last bit, and it is scaled separately because a volume carries the cube of a length.

Measured, it does not justify its cost. `benchmarks/run_chirality.py` asks the question in the regime the alignment-free kernel exists for — leave-one-chemotype-out on the five chemotype-split sets, with three regularisations by three distance grids chosen inside the training part of every fold, and the number of shells kept out of the search so the signed channels never get more freedom than the kernel they extend:

	two-point	signed, 1 shell	signed, 2 shells	signed, 4 shells
mean q^2 over the five sets	+0.063	+0.082	+0.095	+0.108
paired difference	—	+0.019	+0.033	+0.045
95 % interval over the sets	—	[−0.051, +0.113]	[−0.031, +0.121]	[−0.028, +0.121]
better on	—	2 of 5	3 of 5	3 of 5

The mean moves in the right direction and the interval contains zero at every setting. More to the point, **one set accounts for all of it**: at four shells COX2 gains +0.187 and the other four sets average +0.009 between them, with AChE and BZR made worse. On five sets that is not a result, and the channels cost 16 % more columns.

The experiment those sets cannot do. None of the five contains an enantiomeric pair, so the kernel's blindness costs it nothing there and the table above is about something else. The same script therefore builds a set where handedness *is* the activity: twenty molecules, each with its exact mirror image — one conformation reflected, not a second embedding, so a pair differs in handedness and in nothing else and their two-point channels agree to one part in 10^{15} — with the activity `(1 − w)·composition + w·handedness`, the handedness read as the CIP descriptor rather than the SMILES tag, which is not the same label from one scaffold to the next.

At `w = 0` the model works: q^2 +0.97. At `w = 1`, where the answer is the handedness and nothing else, **no variant learns it** — two-point −0.391, and the signed channels −0.34 to −0.37 however many shells they are given and even when scaled up to carry half the kernel.

And the reason is measurable. A kernel method carries what it learnt from one molecule to another along a direction in feature space. The signed channels of a molecule and its own mirror image are exact negatives — that is what the block is for, and a test asserts it — but between *different* molecules they share nothing: the cosine of the signed block is **−0.016 for two molecules of the same handedness and +0.016 for two of opposite handedness**, a difference of 0.03 against a spread of 0.55. Even within one scaffold family, amino acids differing only in the side chain, the difference is 0.10 against a spread of 0.48. Handedness is not a direction in this space, so there is nothing for a linear model to transfer, and the two negative results above are one result seen twice.

That is the conclusion: the block is correct and does what it says about a molecule and its mirror image; it does not put the handedness of two different molecules on a common axis, which is what a QSAR model needs. The channels stay available and off by default.

The construction that does work reads one centre at a time. `cmfqsar/stereo.py` takes every atom with at least three neighbours, orders those neighbours by their weight in a field — the field

weights playing the part the CIP rules play for a chemist, so that the descriptor stays a property of the fields — and takes the signed volume of the ordered frame. It negates under reflection and survives any rotation, like the three-point block, and unlike it two *different* molecules of the same configuration agree: the cosine between same-handedness molecules is **+0.468 against −0.468 for opposite**, a difference of 0.935, where the three-point block manages 0.03.

That difference explains everything, and the synthetic set shows what it gains. Where the activity is handedness and nothing else, the stereo centres reach **q² +0.721** while every variant of the three-point block sits at −0.35; at every mixture from a quarter handedness upward they are ahead of the two-point kernel, and where there is no handedness in the activity at all they cost little (+0.924 against +0.970).

On the five real sets they add nothing — mean +0.039 against the two-point kernel's +0.063 — which is the expected answer rather than a disappointment: those sets contain no enantiomeric pair, so the channels carry no signal and 25 extra columns dilute the kernel. **Turn them on for a series whose stereochemistry varies, and leave them off otherwise.** An order that rests on two nearly equal weights is not an order, so a centre whose top three priorities are within `min_separation` of each other contributes nothing rather than a guessed sign; that guard is what takes the same-handedness separation from 0.18 to 0.86.

Mixing coefficients over the channel blocks. The channels come in blocks, one per pair of field types, each scaled to unit root mean square before all of them enter one cosine-normalised dot product. The overlap kernel weights its fields with h_f; the three invariant sources (autocorrelation, surface, periodic) weight their blocks with `--block-weights` (library: `ac_block_weights` on `CMFModel`), one coefficient per block applied to the scaled features before the normalisation, so that a block's share of the kernel goes as the square of its weight and a weight of zero removes the block exactly. Three ways to set them: `balanced` gives every *kind* of block — the pairs within a group of fields, the pairs across two groups — the same share of the kernel however many blocks each kind has; `KIND=W,...` names the kinds' weights (`indicator=0.3,indicator*physchem=0.5`; unnamed kinds weigh 1); `align` or `alignf` derive one weight per block by centred kernel alignment of the per-block kernels (`align_block_weights`, the `align_mixing` of the fields carried over, with `--block-rho` the shrinkage of the joint solution), re-derived from the training part of every cross-validation fold, so the reported q^2 never sees the left-out activities. The groups default to the continuous indicator fields (`indicator`) against the rest (`physchem`); `--block-groups 'GROUP=field,field;GROUP=cif:*'` names others. The fitted weights are printed by kind and stored with the model (`block_weight_report`).

What the row length is worth, and when the division helps

The kernel above divides every feature row by its own length, so what reaches the model is the direction of the row and not how long it is. That length is one number per sample. For a molecule or a cell it says how much field the sample carries in total, which is close to how large and how dense it is; for a local environment it says how crowded that environment is. Dividing it out is what makes samples of different sizes comparable, and that is why it is there. It also means the kernel cannot tell a molecule from the same molecule carrying four times the field.

`benchmarks/run_rowlength.py` measures what that costs, on five series that already exist and with no new data. Four arms are fitted on the same samples, the same folds and the same ridge grid, each with its setting chosen inside the training part:

arm	what the kernel is
the length divided out	the kernel as the program builds it: the reference

arm	what the kernel is
the length kept	the same kernel without the division, divided instead by its own mean diagonal, which is one constant for the whole matrix and takes no row's length away
the length as an extra channel	$(1-w)$ times the reference plus w times a kernel on the length alone, with w chosen inside the training part
the length alone	the kernel on that one number, which says how much of the target the length by itself carries

The answer depends on the series, and one number says which way it will go: how far the length goes with the target. Not how much the length varies. On the environments of a lithium hop the length correlates with the barrier at +0.60, and keeping it raises q^2 from +0.572 to +0.716, a gain of +0.144 with an interval of [+0.096, +0.198]; the length by itself reaches +0.354 there, so how crowded the environment is carries a third of the barrier and the cosine kernel was throwing it away. On the 2,000-cell defect series of MoS2 and WSe2 the length correlates with the formation energy at -0.88 and -0.93, because it is close to what the cell contains; q^2 is already +0.9989 and cannot show much, but the error falls from 36.5 to 2.4 meV per defect site on MoS2 and from 30.8 to 6.3 meV on WSe2.

Where the samples differ in size, the division earns its place. On the 1,162 beta-cyclodextrin guests, 2 to 42 heavy atoms, the length is two thirds of its own mean and tracks the heavy-atom count at +0.750 — but it tracks the binding constant at only +0.112, and keeping it *costs* 0.043 in q^2 with an interval of [-0.073, -0.015]. The size of a guest is not its affinity. On the single-atom catalysts, twelve combinations of four targets and three designs, keeping the length is behind the reference in three of the twelve beyond the interval and ahead in none. On the counts about a hop midpoint the length varies by 81 per cent of its mean and still buys nothing (+0.017 [-0.030, +0.055]), which is the clearest sign that the spread of the length is not what decides.

So the question is not whether to divide but when, and it costs one line to ask. Correlate the row length with the target on the training part before choosing: `cmfqsar.invariant.row_lengths` returns it. If that correlation is large, keep the length, with `normalize=False` in `cmfqsar.invariant.autocorrelation_kernel` or `cmfqsar.sites.site_kernel`, or add it as one more channel with `cmfqsar.invariant.length_kernel`. If it is small, divide it out. Keeping the length is the stronger of the two ways where it works at all: adding it as a channel recovered +0.070 of the +0.144 on the hop environments. No default changes, and every stored table in this repository was produced with the division in place.

Which columns count, and how far a weighted distance goes

The kernel measures one Euclidean distance over every channel at once. A forest on the very rows the kernel is built from does better than the kernel on some series, and the question is what it has that the kernel does not. On the hop environments of 1,801 lithium hops, 2,601 columns in 153 blocks, a forest on those rows reaches q^2 +0.742 against the kernel's +0.572, a lead of +0.171 with an interval of [+0.116, +0.226]. One explanation is that a single distance over 2,601 columns dilutes the few that carry the barrier among the many that do not, while a tree reads columns one at a time. `benchmarks/run_colweights.py` tests that explanation at four levels of detail, every weight computed inside the training part of each fold. The first two are the `--block-weights align` and `alignf` of the paragraph above, which the program already has; the other two are measured here and deliberately not added to it: one weight per column by the same alignment rule, and only the best columns with the rest set to zero, and a random forest's own importances as the weights.

The coarse weighting is the one that works, and it recovers about half of the lead. `--block-weights align`, one weight per pair of field types, takes the hop environments from +0.572 to +0.658, a gain of +0.086 [+0.042, +0.133], and the beta-cyclodextrin guests from +0.461 to +0.484, +0.024 [+0.003, +0.043]. Every finer rule falls inside the interval on both of those series: one weight per column gives +0.040 [-0.007, +0.089] on the hops, the best columns alone +0.029 [-0.031, +0.089], and a forest's own importances +0.056 [-0.001, +0.115]. So the forest's advantage is not simply that it knows which columns to read: handing the kernel the forest's own ranking does not reproduce it. That is also why nothing finer than a block weight is offered by the program.

On a short description the ordering reverses. The site profiles of the single-atom catalysts have 78 columns in 6 blocks, and there every rule gains beyond the interval -- the best columns alone most of all, +0.044 [+0.013, +0.084] from 23 to 47 of the 78 columns -- while the forest is 0.250 *behind* the kernel. Few columns and few samples is the regime where choosing among the columns is both possible and useful.

What to take from it. Set `--block-weights align` on a long description before reaching for anything finer: it is closed form, it costs one pass over the training features, it is re-derived inside every fold, and it is the only level that pays there. Expect no more than half of what a forest on the same rows would give you on a solid-state series. Read the rest as still unexplained: it is not the row length, which the section above measures, not a curvature the Gaussian layer can hold, which item BT7 measured, and not the weighting of the columns.

What the model says about one molecule

The invariant kernel has no co-field, because it has no frame, but **it does have a decomposition over atoms**. Every channel is a double sum over atom pairs, and the prediction is a fixed linear functional of the channels, so a pair of atoms is one term of the prediction and the only choice is how to divide a pair between its two atoms. Dividing it equally gives one number per atom that adds up to the prediction exactly:

$$y(x) - b = \sum_a q_a, \qquad q_a = \frac{1}{\|s\,\Phi(x)\|} \sum_c s_c\,\gamma_c\,\Phi_c^{(a)}(x) \tag{11.3}$$

with γ the co-pattern -- the training molecules' channel rows summed with their dual coefficients -- and $\Phi_c^{(a)}$ that atom's half of the channel. `CMFModel.atom_contributions` returns them for an invariant model as it does for an aligned one, the report lists the largest, and the viewer colours the atoms by them. `invariant_pair_contributions` splits the same total over the pairs of field types instead, which says which kind of channel the prediction came from. The signed three-point channels are a sum over *triples*, so a model built with `--ac-chirality` has no split over two atoms and the program says so rather than dropping them.

Does it name the right atoms? `benchmarks/run_guest_atoms.py` asks the question on the cyclodextrin guest tables, where the chemistry is known: binding is driven by a hydrophobic group entering the cavity, and polar groups stay in the water. For every guest the mean share per heavy atom inside a group is compared with the mean outside it, with the contributions taken from models fitted without that guest, so a guest's own constant never enters its own explanation.

Group on beta-cyclodextrin	guests	share per atom, inside minus outside
adamantyl	6	+0.0646 ± 0.0368
cyclohexyl	122	+0.0378 ± 0.0202

Group on beta-cyclodextrin	guests	share per atom, inside minus outside
phenyl	817	+0.0158 ± 0.0041
carboxyl	347	-0.0366 ± 0.0046
hydroxyl	221	-0.0916 ± 0.0126
amine	217	-0.0886 ± 0.0270

On beta-cyclodextrin it does. The three groups that are known to enter the cavity take more of the prediction than the rest of their own molecule and the three polar groups take less, and the order among the three -- adamantyl, then cyclohexyl, then phenyl -- is the order of the affinity they are known to give. The largest sets carry the claim: phenyl on 817 guests and the three polar groups on 347, 221 and 217, every interval away from zero. Adamantyl is the strongest effect and the weakest evidence, 6 guests.

On the other two hosts it does not. On alpha-cyclodextrin, whose cavity is too narrow for a benzene ring, phenyl comes out at -0.0158 and the carboxyl at +0.0703; on gamma-cyclodextrin, whose cavity is wide enough that none of these groups fills it, every one of the three is negative (-0.0333 for phenyl, -0.0558 for cyclohexyl). Read that as the limit of the reading rather than a fault in the arithmetic: the split is exact for all three, and what differs is whether the group a chemist would name is the group this host's data makes the model lean on.

It is a statement about a series, not about one molecule. Shuffling an atom's number among the atoms of the same molecule gives a null, and the group's z-score against it is +0.35 for phenyl and +1.38 for adamantyl: within one guest the group does not stand out far above the spread of its own atoms. What is solid is the mean over hundreds of guests. Use the atom numbers to see what a model has learned from a table, not to decide where to put the next substituent on one compound.

The bond graph as a view: the graph neural tangent kernel

The field kernel of this program is, as the chapter on nonlinear kernel layers puts it, the kernel of an infinitely wide radial-basis network on the aligned fields. The same limit exists for a network on the bond graph. The graph neural tangent kernel of Du and colleagues (NeurIPS 2019) is the kernel of an infinitely wide graph network *at initialisation*, computed in closed form from the graph with no training at all, and `cmfqsar.gntk` computes it (plan item ND7). Two molecules with n and m atoms give an $n \times m$ matrix over their atom pairs. Each atom starts with a feature vector, one hot over its atom type (the Sybyl type the fields are built with, which says element, hybridisation and aromaticity at once), so the covariance of a pair starts at the inner product of the two. One *block* of the network does two things to every pair: it averages over the bonded neighbours on both sides, which is the adjacency with self-loops applied left and right, and it passes through a layer of rectified units, whose infinite-width expectations are the arc-cosine terms of Cho and Saul that the chapter *Nonlinear kernel layers and multiple kernel learning* also uses. The tangent kernel is carried through the same steps as the sum of the covariance and the previous tangent kernel times the derivative term. After two such blocks the kernel of the two molecules is the sum over every atom pair, normalised to a cosine so that it sits on the scale of the cosine-normalised field kernels.

What the graph kernel knows is which atom is bonded to which; what it does not know is where any atom is. It therefore brings no co-field and no split in space, and a model that holds it reads its co-field from the field views alone. It enters as one more term of the sum of kernels under the weights of the chapter *Mixing the fields*: `mixing.with_views` stacks it under the field kernels, the evidence gives it a weight like any field, and `mixing.drop_one` gives the Bayes factor against *the graph view does nothing*, which is the question the plan asks of it.

```
from cmfqsar.gntk import gntk_kernel
from cmfqsar.mixing import evidence_mixing, joint_kernel, with_views

K_graph = gntk_kernel(table)                              # (n, n), cosine-normalised
stack, names, scale = with_views(K_fields, {"graph": K_graph}, names=field_names)
h = evidence_mixing(stack, y, scale=scale)                # one weight per field, one for the graph
K = joint_kernel(stack, h, scale)                         # the kernel the ridge is fitted on
```

Measured on the eight Sutherland sets and on 2,000 molecules of COMPAS-1D drawn at random, ten folds, the ridge by the evidence, every variant under the weights of the evidence: the five fields alone, the fields with the invariant autocorrelation kernel of item C5, the fields with the graph kernel, and the fields with both. The comparison the plan reads is the graph view against the invariant kernel, because a gain the invariant kernel already gives is not a result (`benchmarks/run_gntk.py`, `benchmarks/results/gntk/table.md` and `benchmarks/results/gntk_compas/table.md`).

set	n	q2, fields	+ invariant	+ graph	+ both	graph share	Bayes factor	graph against fields	graph against invariant
DHFR	361	+0.641	+0.667	+0.712	+0.683	95%	+17.1	+0.071 [+0.036, +0.110]	+0.045 [+0.010, +0.083]
COX2	282	+0.565	+0.599	+0.566	+0.601	7%	+0.7	+0.001 [-0.002, +0.004]	-0.033 [-0.058, -0.009]
BZR	147	+0.401	+0.569	+0.413	+0.574	20%	+0.4	+0.011 [-0.001, +0.025]	-0.156 [-0.243, -0.069]
ACE	114	+0.688	+0.720	+0.692	+0.726	24%	+2.1	+0.004 [-0.011, +0.022]	-0.028 [-0.059, -0.002]
AChE	111	+0.587	+0.611	+0.611	+0.626	19%	+0.9	+0.024 [+0.004, +0.049]	-0.001 [-0.031, +0.032]
THR	88	+0.723	+0.726	+0.728	+0.729	6%	+0.5	+0.005 [-0.001, +0.012]	+0.003 [-0.007, +0.015]
THER	76	+0.462	+0.477	+0.468	+0.481	13%	-0.2	+0.006 [-0.002, +0.014]	-0.009 [-0.025, +0.007]
GPB	66	+0.604	+0.610	+0.605	+0.610	17%	-0.0	+0.001 [-0.003, +0.005]	-0.004 [-0.013, +0.007]

set	n	q2, fields	+ invariant	+ graph	+ both	graph share	Bayes factor	graph against fields	graph against invariant
COMPAS-1D	2000	+0.439	+0.904	+0.908	+0.929	100%	+1845.3	+0.470 [+0.438, +0.503]	+0.004 [-0.002, +0.011]

Differences in q2 with their 95 % paired bootstrap intervals over the molecules. On the Sutherland sets the graph view changes q2 by +0.016 on average against the fields alone, with an interval that excludes zero on 2 of the eight sets in its favour and 0 against, and by -0.023 against the fields with the invariant kernel (1 in its favour, 3 against); added to the fields with the invariant kernel it changes q2 by +0.006 (1 in its favour, 0 against). The evidence gives the graph view 25% of the joint kernel on average, with a Bayes factor of +2.7 nats against it doing nothing. On the 2,000 molecules of COMPAS-1D the view changes q2 by +0.470 [+0.438, +0.503] against the fields and +0.004 [-0.002, +0.011] against the fields with the invariant kernel, holding 100% of the kernel with a Bayes factor of +1845.3. Against the invariant kernel the graph view wins on DHFR and loses on COX2, BZR, ACE. On DHFR the evidence hands the graph 95% of the kernel with a Bayes factor of +17.1, and the bond graph alone predicts better than the aligned fields (q2 +0.712 against +0.641): that is the largest and most varied of the eight series, the one an alignment serves worst, and there the graph takes over from the fields rather than adding to them. Everywhere else the graph holds a few per cent of the kernel and changes q2 within its interval, and what the fields with the invariant kernel gain over the fields alone, the graph does not give. On COMPAS-1D the aligned fields are weak (q2 +0.439), the graph alone reaches q2 +0.908, the same as the invariant kernel (+0.004 [-0.002, +0.011]), and the two together gain +0.025 [+0.020, +0.030] over the invariant kernel alone, an interval well clear of zero. The gate asked for a gain the invariant kernel does not already give: it is met on the two largest series (DHFR with 361 molecules and COMPAS-1D with 2,000) and on none of the seven small ones. Whether the graph needs the molecules or the poor alignment these two series share, this benchmark cannot separate. The view stays in the library as one more term for the weights of item E6 to judge series by series, which is what they did here.

11.4 Molecules on a surface: the in-plane invariant kernel

A molecule adsorbed on a surface has one direction that is not like the others. Its height above the metal and its tilt are physics — they are what the relaxation computed, and they are most of what decides the adsorption energy. Where it sits in the plane and how it is turned about the normal are not: the slab is periodic and its origin is wherever the cell was cut. The overlap kernel cannot tell the two apart, so it demands a frame for a degree of freedom that has no preferred value, and the invariant kernel of the last section throws the height away along with the rest.

This kernel keeps exactly what is physical. It is the two-point distribution of the field weights over the **in-plane** distance ρ, resolved by the heights of both atoms:

$$\Phi_{fg}(\rho_k, z_i, z_j) = \sum_l \sum_m w_{lf} w_{mg}\, G_\sigma(\rho_{lm} - \rho_k)\, G_{\sigma_z}(z_l - z_i)\, G_{\sigma_z}(z_m - z_j) \tag{11.4}$$

$$G_s(u) = \exp\left(-\frac{u^2}{2s^2}\right), \qquad \rho_{lm} = |\mathbf{r}_l^{xy} - \mathbf{r}_m^{xy}| \tag{11.5}$$

It is therefore invariant to translation in the plane and to rotation about the normal, and to nothing else: tilt, height and the vertical order of the atoms all survive. Averaged over the direction of ρ it is, by the Wiener–Khinchin theorem, the in-plane power spectrum of the fields resolved by height — the molecule described in reciprocal space across the surface and in real space along the normal. The surface is at $z = 0$ with the normal along $+z$, which is how the adsorption scripts of `examples/corrosion_fe110` write their poses; nothing checks that convention, so a set of poses in some other frame will be described accurately but uselessly.

The channels run over in-plane distances to 12 Å in steps of 0.75 Å (`--ac-dmax`, `--ac-sigma`) and heights from 1 to 6 Å in steps of 1 Å (`--surface-z ZMIN ZMAX ZSTEP`), the spacing of a channel being also its width on both axes. That is 17 × 6 × 6 = 612 channels for each pair of field types, 9,180 features on the five physico-chemical fields, which is why the blocks are scaled and may be weighted exactly as in the preceding section. Pairs of different field types are summed both ways round, as the three-dimensional descriptor counts them, and the $l = m$ terms are kept. Those channels are again a descriptor of one molecule, and the kernel of two of them is the dot product of the preceding section, over this section's channels:

$$K(A,B) = \frac{\sum_c s_c^2\, \Phi_c(A)\, \Phi_c(B)}{\|s\,\Phi(A)\|\ \|s\,\Phi(B)\|}, \qquad c = (f, g, \rho_k, z_i, z_j) \tag{11.6}$$

The command is `--kernel surface`; the model panel does not offer it.

What it is worth

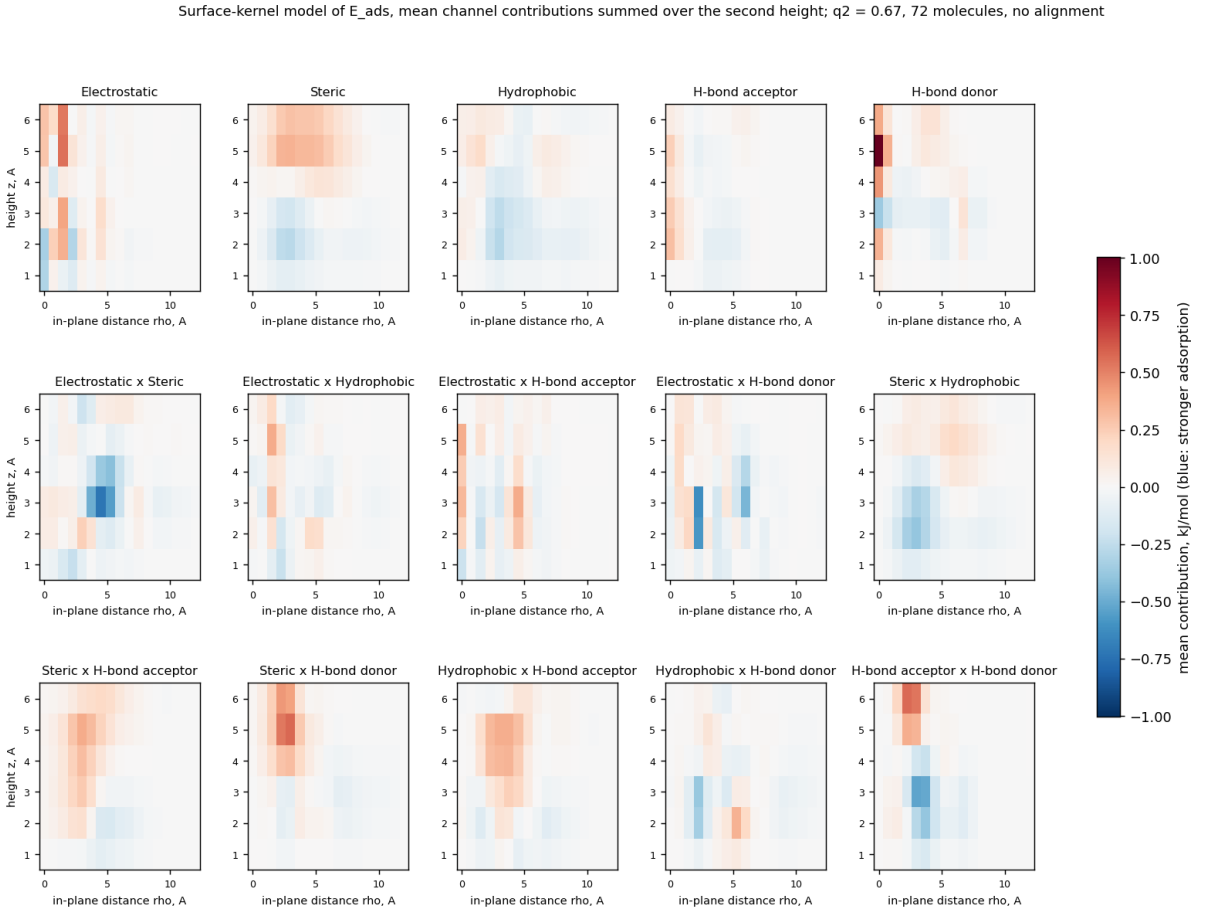
On the benzimidazole corrosion node — 72 molecules relaxed on Fe(110) with the UMA machine-learned potential, the target their adsorption energy, five physico-chemical fields, KRR with α and λ chosen in a 10-fold inner cross-validation, 5 outer folds, 10 repeats:

model	q^2	RMSE (kJ/mol)
surface kernel, poses as the relaxation wrote them	**0.655**	31.2
surface kernel, the same poses shuffled in the plane	**0.655**	31.2
overlap fields, poses as written	0.505	37.4
overlap fields, poses registered in the plane by their common substructure	0.517	36.9
overlap fields, poses shuffled in the plane	0.480	38.3
rotation-invariant autocorrelation, which discards the height	0.109	50.2

Three things follow. The **invariance is demonstrated rather than asserted**: turning every pose by a random angle about the normal and sliding it up to 6 Å in the plane changes the model by nothing at all, the second row reproducing the first to three decimals. **The height is what produces the score** — the fully invariant kernel differs from this one only in discarding z, and it falls from 0.655 to 0.109. And **the frame the overlap kernel wants is not worth building**: registering the poses laterally on the common substructure moves it from 0.505 to 0.517, a negligible difference, while the kernel that was never given a frame is 0.14 above the better of

them.

There is no co-field, for the reason every invariant source has none: the kernel is not an integral of a product of fields, so the prediction cannot be turned back into a function of space. What it offers instead is the co-pattern — the weight of every channel of the fitted linear model, which names the pair of field types, the in-plane distance and the two heights that carry the signal, which `top_channels` ranks against the labels of `surface_channel_labels`. The application draws it as a curve of weight against in-plane distance, one pair of field types at a time, in the *Co-pattern* tab of the Results panel; the section *The co-pattern as a chart* is about that chart and about what it may and may not be read for. The measurements above come from `surface_kernel_eads.py`, in `examples/corrosion_fe110`, which writes its table next to the script.



The co-pattern of the surface model of the adsorption energies of 72 molecules on Fe(110), $q^2 = 0.67$: the mean contribution of every channel to the prediction, in kJ/mol, summed over the second height, by pair of field types, in-plane distance and height. Blue means stronger adsorption. The signal sits in a few channels, among them the electrostatic and the hydrogen-bond donor field at zero in-plane distance. Drawn by `surface_channels_eads.py` *in* `examples/corrosion_fe110`*.*

11.5 Posed by rule: the surface frame without the slab

The surface kernel needs only the normal, but it cannot draw a co-field; the overlap kernel can, if the molecules are in the surface frame, and so far that frame came from relaxing every molecule on a slab. `cmfqsar/planealign.py` puts a molecule there by rule — flat, with the plane of its two longest principal axes parallel to the surface and the lowest heavy atom at a chosen height; upright, with its most basic nitrogen pointing a lone pair at the surface; or whichever of those is most similar in field to one trusted pose — and `cmf align --by plane` applies it to a set:

```
cmf align poses.sdf --by plane --pose flat --height 1.9 --out onsurface.sdf
cmf align poses.sdf --by plane --pose auto --refine plane --reference 0 --out onsurface.sdf
```

`--pose flat` turns the flattest face down and `flat-up` the other; `--pose auto` tries both faces and the upright pose and keeps the most similar to the reference, after `--refine plane` (moved in the plane and about the normal only) or `--refine plane+tilt` (also tilted by up to 30° and lifted by up to 1 Å). Nothing else moves: the conformation is never touched.

What the rule is worth was measured on the Fe(110) sets (`benchmarks/run_fe110.py`, the README section of the same name). A chemisorbed ring that the relaxation left flat is reproduced to 0.4 Å *given its face*; the face of a planar molecule is a mirror in the plane that no rule and no field similarity to one reference decides, and the molecules the relaxation left upright or oblique are 3 Å from any rule pose. The overlap kernel in the rule frame recovers of the adsorption energy exactly what it recovers in the relaxed frame once that frame's height — the outcome of the relaxation, which alone predicts E_ads with q^2 0.4–0.5 — is removed, and the surface kernel on the same rule poses does as well. The aligner therefore does not replace the slab; it is the way to put a set in a surface frame and draw a co-field in it when the frame is all that is wanted, and `--pose flat-up` is there because the user may know the face that the rule cannot.

11.6 A frame by rule: the gap of polybenzenoid hydrocarbons

The surface frame of the previous section was a rule with a slab behind it. A planar molecule needs no slab: its frame is its own inertia tensor, and the only thing left to decide is which way the axes point. That makes a series of rigid planar molecules the cleanest test the overlap kernel can get, because the alignment that usually decides a 3D-QSAR model is reduced to a rule that cannot be wrong. The series is COMPAS-1 (Wahab, Pfuderer, Paenurk and Gershoni-Poranne, *J. Chem. Inf. Model.* 2022, 62, 3704): every cata-condensed polybenzenoid hydrocarbon of one to eleven rings — 8,678 molecules at B3LYP-D3BJ/def2-SVP, 34,072 at GFN2-xTB — with its HOMO and LUMO energies. The property is the HOMO–LUMO gap in eV, a quantum-chemical quantity rather than a binding affinity, and the data set is downloaded by the benchmark (CC-BY-NC-SA, kept in `benchmarks/cache`, not shipped). `cmfqsar/pbh.py` holds what the study needs beyond the ordinary machinery, and `benchmarks/run_compas.py` is the study; its notes, table and one-page report are in `benchmarks/results/compas/`.

The frame. The heavy atoms are moved to their centre of mass and rotated onto the axes of their inertia tensor: x along the smallest moment (the long axis), z along the largest (the normal of the plane). Four proper rotations share those axes — the signs of x and y, with z following so that the frame stays right-handed — and `canonical_frame` picks one by comparing, in order, $\Sigma\, x^3$, $\Sigma\, y^3$, $\Sigma\, x^2 y$, $\Sigma\, x^3 y$ and $\Sigma\, x\, y^3$ over the heavy atoms and keeping the largest. Each of these changes sign under one of the flips, so a molecule for which all five vanish is symmetric under that flip and its field is the same in either frame. Benzene, triphenylene and the other molecules with a three-fold or higher axis have equal in-plane moments and no long axis; for them x is put along the direction of the largest third moment, or the fourth or sixth when the lower ones do not depend on the direction. That direction is solved for and not searched for: a maximum is flat, so comparing values of the moment over the angle cannot place the angle better than about 1e-8 radians, and the frame then depends on the machine at that level. The derivative of the moment is not flat at the maximum, so the program locates the angle by Newton's method on the derivative, which places it to the last digit. A molecule that is already in its frame stays where it is -- to 2e-15 A, which is what `tests/test_pbh.py` asks for -- a rotated or shifted copy lands on the same point set, and a mirror image lands on the mirrored one. The alternative that [2] offers — among the four candidate frames, the one whose field is most similar on average to the training set (`choose_frame_by_kernel`,

alignment by prediction confidence) — is implemented and off by default, since the rule leaves nothing to choose for a planar molecule without symmetry.

The fields. Five groups, each an ordinary entry of the field set:

- **A, steric**: the steric field of the program (Tripos well depths, widths scaled by the van der Waals radius, hydrogens included).
- **B, electrostatic**: GFN2-xTB partial charges computed by `cmfqsar/xtb.py` on the DFT geometry (0.3 s per ten-ring molecule), as the field `prop:q_xtb`; Gasteiger charges on request, and the substitution is then printed and written to the notes.
- **C, atom-class indicator fields**: the three carbon classes of a benzenoid, `cif:C.H` (a rim carbon bonded to a hydrogen), `cif:C.2r` (a carbon shared by two rings) and `cif:C.3r` (shared by three, which a cata-condensed set does not have and the benchmark drops): the typing scheme `pbh` of `--cif-scheme`.
- **D, ring-class indicator fields**: one Gaussian per ring *centroid*, in one field per annulation class — `ring:terminal` (one fused neighbour), `ring:linear` (two neighbours on opposite bonds, the middle ring of anthracene), `ring:angular` (two neighbours one bond apart, the middle ring of phenanthrene) and `ring:branched` (three, the centre of triphenylene). The centroids enter the molecular field as pseudo-atoms appended by `build_pbh_fields`, with weight zero in every atom field, so every kernel, the co-field and the atom contributions work on them unchanged; this is Clar's topology of a benzenoid written as a field in space.
- **E, Clar-sextet ring field** (`ring:sextet`): the same centroids weighted by the share of the maximal Clar structures in which the ring carries an aromatic sextet (`clar_weights`: naphthalene ½ and ½, anthracene ⅓ each, phenanthrene 1, 0, 1). It was meant to stand in for an aromaticity field until a field weighted by the NICS of every ring, which needs a DFT calculation per ring, could be built.

The protocol. Random 80/10/10 splits for five seeds, and size-extrapolation splits that train on the molecules with fewer than N rings and test on the molecules with N, for N = 8, 9 and 10 (179, 619 and 2,270 training molecules: the set is dominated by ten-ring molecules). The attenuation shared by the groups of a field set and then one attenuation per group (by coordinate descent) are chosen by 5-fold cross-validation on 2,500 training molecules; the ridge by 5-fold cross-validation on the whole training part; the choice between the shared and the per-group attenuation on the validation part. Kernel ridge regression with the bias term of Eq. (2.6), every field kernel scaled to unit mean diagonal over the set. The baselines are the attenuation-to-zero limit of the indicator and ring fields — ridge regression on the counts of the atom classes, the ring classes, the rings and the sextets, an additivity scheme, which the field model equals exactly at a vanishing attenuation (`tests/test_pbh.py`) — a random forest on RDKit descriptors, and a straight line in the number of rings.

What it measured. Test R^2 over the five random splits (mean ± sd), then R^2 on the three size-extrapolation splits:

field set	q^2	test R^2	test RMSE (eV)	N = 8	N = 9	N = 10
A steric	0.43	0.42 ± 0.04	0.30	−0.48	−0.40	−1.47
A, B	0.52	0.51 ± 0.03	0.28	−0.69	−0.40	−1.00

field set	q^2	test R^2	test RMSE (eV)	N = 8	N = 9	N = 10
C atom classes	0.49	0.49 ± 0.02	0.29	−0.02	−0.49	−1.15
D ring classes	0.66	0.65 ± 0.02	0.24	0.52	0.51	−0.04
A, B, C	0.60	0.60 ± 0.05	0.25	−0.05	−0.12	−1.07
C, D	0.75	0.71 ± 0.08	0.21	0.50	0.50	0.31
A, B, C, D	0.83	0.78 ± 0.11	0.18	0.59	0.53	0.44
A, B, C, D, E	0.87	0.87 ± 0.02	0.14	0.67	0.78	0.70
ridge on the counts of atom classes, ring classes, rings and sextets (additivity scheme)	–	0.70 ± 0.01	0.22	0.63	0.65	0.67
random forest on 66 RDKit descriptors	–	0.91 ± 0.02	0.12	0.32	0.35	−0.16
straight line in the number of rings	–	0.02 ± 0.03	0.39	−0.18	−0.08	−0.02

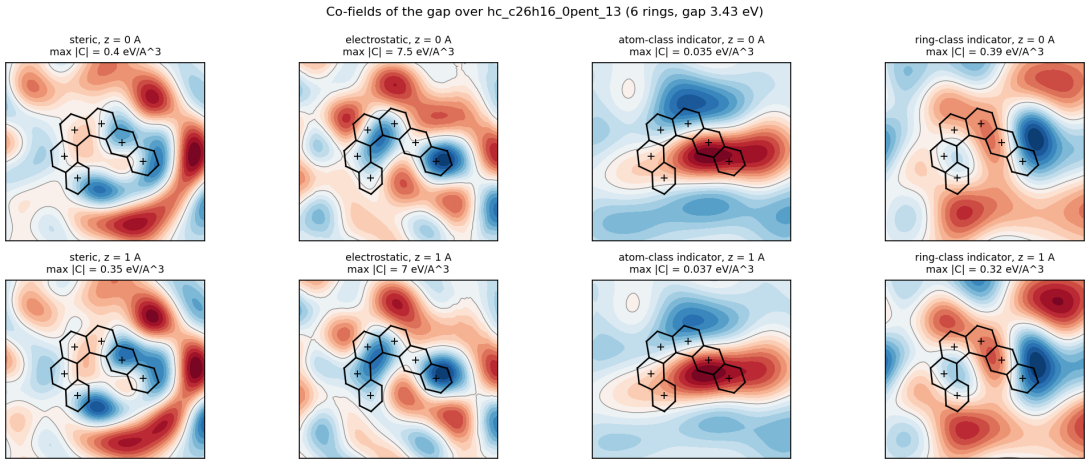
The gap of a benzenoid is topology. The steric field sees the shape and reaches 0.42; the carbon classes, which say where the rim is, 0.49; the four ring-class fields alone 0.65, more than the three atom groups together; and the Clar-sextet field, the placeholder, is the largest single gain on top of everything else. The size-extrapolation splits say the same thing more sharply: every set without a ring field has a negative R^2 on the larger molecules — it has learned the isomers it saw and nothing that carries over — and every set with one is positive. The spread of the A, B, C, D row over the seeds is one molecule: seed 0 puts benzene, the only one-ring molecule, in the test part, and the model extrapolates its gap to 10–13 eV (computed 6.72); over the molecules of three rings and more the RMSE is 0.164 ± 0.003 eV for A, B, C, D and 0.138 ± 0.002 eV with the sextet field. The baselines put the numbers in their place. The additivity scheme — ridge regression on the counts of the atom classes, the ring classes, the rings and the sextets — reaches 0.70 on the random splits and 0.63 to 0.67 on the size splits, so the four-group field set beats it on the random splits and not on the size splits, and only the set with the sextet field beats it on both. The random forest on 66 RDKit descriptors reaches 0.91 on the random splits, better than any field set, and 0.32, 0.35 and −0.16 on the size splits, worse than every set with a ring field: it knows the isomers it saw and nothing that carries over to a larger molecule, which is the question a model of an enumerated space is asked.

Against the published models. The same 8,678 molecules have been modelled before, on random splits like ours. The geometric graph networks of Weiss et al. (*J. Org. Chem.* 2023, 88, 9645, Table 1: an SE(3)-transformer on a graph of rings or of atoms, SphereNet, DimeNet++, SchNet) reach a test MAE of 0.011 to 0.022 eV; a SMILES language model 0.033; a recurrent network on augmented annulation strings 0.05 eV (Fite et al., *J. Phys. Org. Chem.* 2023, 36, e4458); extra-trees on selected RDKit descriptors R^2 0.91 (Zheng et al., *Chem. Phys. Lett.* 2023, 814, 140358), which our random forest reproduces. The field models, at 0.11 to 0.13 eV, are an order of magnitude

behind the graph networks on interpolation among isomers, and the table says why: the chosen widths are the widest of the grid and the ridge the smallest, so the gap is set by the ring arrangement to a precision the overlap of wide Gaussians does not resolve. Both remedies were measured (`--stage resolution`, the last table of `benchmarks/results/compas/table.md`): the attenuation grid extended to 0.02 to 20 Å^{-2} and the ridge down to 10^{-6} change nothing, and the Gaussian kernel layer of the section *Nonlinear kernel layers and multiple kernel learning* on the same kernel halves the error to 0.056 ± 0.004 eV MAE (R^2 0.964 ± 0.005 over three seeds), level with the annulation-string network and ahead of the descriptor forests, three to five times behind the graph networks. The resolution was the nonlinearity, not the width.

The width itself then sat at the edge of its own grid: the chosen value, 0.25 of the median squared distance, was the smallest of the grid and so its widest layer, and a width going to zero is the linear kernel again. Three wider layers were therefore measured as well (`--stage resolution --wide-gammas`, stored apart and reported in its own table of `benchmarks/results/compas/table.md`). Nothing better was hidden under the edge. The cross-validated error the width is chosen on changes by at most 0.016 eV between 0.03 and 0.25, while from 0.25 to 8 it grows by 0.05 to 0.21 eV: the minimum is a broad plateau whose edge the first grid already reached. Where the wider grid moved the choice, which was seven of the ten searches, the held- out error got worse, by up to 0.038 eV. The size-extrapolation splits show why, and the reason is worth keeping in mind beyond this series: the width is chosen by cross-validation inside the training part, which holds the smaller molecules, and there a layer wide enough to be almost the linear kernel loses little, while what the layer is worth appears on the larger molecules that are held out. A selection rule that cannot measure the quantity it selects for will pick the wrong width whatever grid it is given.

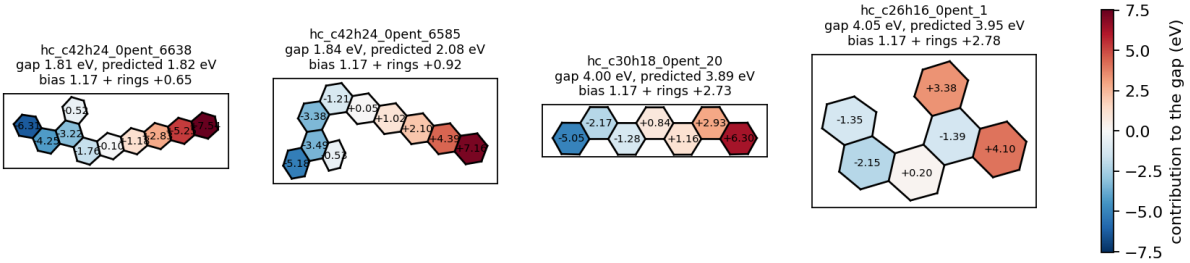
The layered model is no longer linear in the field, so the co-field and the per-ring contributions of this section belong to the linear model, and Shapley values are the reading of the layered one. On the size-extrapolation splits the layer gains as much: trained on the 2,270 molecules of up to nine rings, the layered five-group model predicts the 6,408 ten-ring molecules with R^2 0.92 and RMSE 0.113 eV, against 0.70 and 0.213 eV for the linear model and a negative R^2 for the descriptor forest. Nothing published measures that split, which stays the field model's own claim.



The co-fields of the gap over a six-ring reference molecule of the test set, one panel per field group, in the molecular plane and 1 Å above it; red raises the gap per unit of field, blue lowers it. Drawn by `benchmarks/run_compas.py`.

Reading a prediction ring by ring. Decomposition rule 3 gives every atom its share of a prediction; `ring_contributions` sums the shares over the atoms of every ring, splitting an atom shared by two rings in half, giving a hydrogen to its carbon and a ring centroid to its ring, so that

the per-ring values and the bias add up to the prediction to 10^{-8} eV. The benchmark draws them on the skeleton for four test molecules and writes, for the reference molecule, which rings and which zones of the co-fields raise or lower the gap (`benchmarks/results/compas/narrative_*.md`). The values are large and cancel: a ring on one side of the frame counts positive and one on the other side negative whatever its class, because a ridge of 10^{-4} lets the coefficients grow and the co-field acquires a smooth component across the frame. The sum is the prediction; the single values read position as much as chemistry.



Per-ring contributions to the gap (decomposition rule 3 summed per ring) of four test molecules: the two with the smallest and the two with the largest computed gap among the test molecules of four rings and more. Drawn by `benchmarks/run_compas.py`.

The Shapley reading of the layered model. The layered model predicts better and has neither a co-field nor an exact per-atom decomposition, so its reading is the Shapley values of the section *Shapley values*, with one ring as one player (`--stage ringshapley`, the table *The per-ring reading of the layered model* of `benchmarks/results/compas/table.md`). A benzenoid of up to fourteen rings has at most 16,384 coalitions, so the values are exact and not sampled. `cmfqsar.pbh.ring_players` builds the partition the game needs: an atom shared by two rings goes to the first of them, a hydrogen to its carbon, a ring centre to its own ring, because a player cannot be shared.

Three things come out of it. The first is a control. For the linear model the Shapley values are not a second opinion but the same numbers as rule 3: its per-atom shares summed over the same players, equal to within 10^{-10} eV. The per-ring decomposition above differs from them only in giving half of a shared atom to each of two rings, and on these molecules that convention alone moves a ring's value by 3.3 to 8.5 eV. How much the linear reading depends on one bookkeeping choice is itself worth knowing. The second is that the layered reading cancels far less. The absolute ring values of the layered model sum to 1.7 to 3.1 times what they explain, against 4.5 to 61 times for the linear model, whose single values reach 10 eV on a prediction of 1.8 eV. The model that gives up the co-field has the more stable reading of the two. The third is that they are not the same reading: the rank correlation between them over the rings of one molecule runs from -0.15 to +0.66, so the layer changes which ring the gap is attributed to, not only how well the gap is predicted. On this field set the layer is worth a test MAE of 0.0725 eV against 0.1323 eV for the linear model it is built on.

What the study does not settle is written in `benchmarks/results/compas/NOTES.md`: the size-extrapolation splits train on few molecules; most molecules are twisted (89 % have a heavy atom more than 0.5 Å from their best plane) and the two faces of their frame differ; the chosen steric attenuation and ridge sit at the edges of their grids; the NICS field is missing and has to beat the sextet share; the published COMPAS-1 numbers are not filled in.

The 34,072 molecules of COMPAS-1x. The same model in the space of landmark molecules (`--stage scaling`, the Nyström path of `cmfqsar/lowrank.py`, with the attenuations and the ridge of the COMPAS-1D model and the compiled kernel engine) reaches a test R^2 of 0.80 with 500 landmarks and the same with 1,000 and 2,000, at an RMSE of 0.133 eV on the GFN2-xTB gaps; the

rank of the feature map grows only from 2,244 to 2,722 where the number of columns quadruples. The fields of planar molecules in one frame span a few thousand directions, so the 34k set is a quarter of an hour of kernel evaluation against a few hundred landmarks and never its 34k × 34k matrix.

The four follow-ups, measured. The first report named four things to do next; they were run the next day with the same protocol (three random seeds, the size splits, the Gaussian layer as a third configuration), and `benchmarks/results/compas/table.md` holds every number.

A geometric aromaticity field. A NICS field for the whole set is one DFT calculation per molecule with a ghost atom above every ring (55 s for naphthalene, 5.5 min for six rings with the serial ORCA of one desktop: about a week for the set), so the field built is the **HOMA index** of every ring (harmonic-oscillator model of aromaticity, Krygowski 1993: from the bond lengths of the DFT geometry, 1 for benzene, below 0 for alternating bonds) as the weight of a ring-centred field, group F (`ring:homa`, `ring_homa` in `cmfqsar/pbh.py`). NICS(1) and NICS(1)zz were computed for the 179 molecules of up to seven rings as its calibration (`--stage nics`: B3LYP-D3BJ/def2-SVP, GIAO, one ghost hydrogen 1 Å above every centroid). On the 1,158 rings the HOMA index follows NICS(1)zz with r = −0.77 (the Clar-sextet share −0.50), so the geometric index is the better stand-in of the two; the linear rings are the most shielded (mean NICS(1)zz −30 ppm, against −25 for a terminal ring, −18 angular, −8 branched) and not the most bond-equalised (HOMA 0.68 against 0.79), so a NICS field would weight them more than the HOMA field does. Across the rings of the set the HOMA index follows the Clar-sextet share (r = 0.82; mean 0.80 for a terminal ring, 0.70 linear, 0.48 angular, 0.11 branched). In the linear model the HOMA field beats the Clar field by the MAE (0.097 against 0.108 eV) and on the size splits (R^2 0.83 against 0.70 on the ten-ring molecules), and the two together are the best linear model (MAE 0.085 eV; 0.84 on the ten-ring molecules); under the Gaussian layer the six groups reach 0.077 eV RMSE and 0.050 eV MAE, and R^2 0.93 on the ten-ring molecules from the 2,270 smaller ones. The sum of the HOMA indices added to the counts takes the additivity scheme from 0.70 to 0.80: most of what the field carries is a per-molecule sum, and the field is worth 0.05 eV of MAE beyond it.

Δ-learning from the GFN2-xTB gap. COMPAS-1x holds the xTB gap of every COMPAS-1D molecule, and a straight line in it, fitted on the training part of every split, already leaves 0.047 eV RMSE and 0.036 eV MAE (R^2 0.986). The field model trained on that residual (`--targets DELTA_eV`) reaches 0.026 ± 0.002 eV RMSE and 0.018 ± 0.001 eV MAE with the Gaussian layer, and 0.030 eV on the ten-ring molecules against 0.045 eV for the line: inside the range of the published graph networks (0.011 to 0.022 eV) for the price of one xTB calculation per molecule.

The peri-condensed set. COMPAS-3D (Wahab and Gershoni-Poranne, *Phys. Chem. Chem. Phys.* 2024, 26, 15344: 8,844 molecules of four to ten rings, 6,891 with ten, CAM-B3LYP-D3BJ/def2-SVP) brings the three-ring carbon `C.3r` and a fifth annulation class, `peri` (a ring two of whose shared bonds touch, or fused to four or more rings; 44,187 of its 85,960 rings). The picture repeats: the ring classes alone 0.46, the four groups 0.61, with the Clar field 0.83, with the Clar and HOMA fields 0.90 (linear); the layer takes the six groups to 0.097 eV RMSE and 0.072 eV MAE (R^2 0.95), and to R^2 0.86 on the 6,891 ten-ring molecules from 1,953 smaller ones, where the descriptor forest is at −2.3 and the additivity scheme at 0.75. The reading of the linear model does not survive this set: the four-group model refitted for the figures has rule-2 contributions of $\pm 5 \cdot 10^4$ eV that cancel to a prediction, and a float64 refit moves one test prediction by 0.89 eV (0.06 eV on COMPAS-1D), so the per-ring figure of this set is not a reading, and Shapley values of the layered model are.

Hetero atoms. A random 10,000 of the 52,000 neutral molecules of COMPAS-2D (Mayo Yanes, Chakraborty and Gershoni-Poranne, *Sci. Data* 2024, 11, 97: nitrogen, oxygen, sulfur and boron in rings of four to six atoms; read by `read_compas2` from the gzipped SDF) is where the frame by rule

stops working. The atom classes gain the types N, N.H, O, S, B and B.H, a group R holds one ring-centred field per ring type (pyridine, pyrrole, borole and the others), the HOMA field is defined where the Krygowski parameters exist (not for a bond to boron), and there is no Clar field. The best model (A, B, C, D, F, R with the layer) explains 69 % of the variance, 0.38 eV RMSE on a spread of 0.68 eV, and 0.60 on the 1,120 ten-ring molecules from the 8,880 smaller ones (0.38 eV RMSE), and the chosen parameters say why: the steric attenuation sits at the narrow end of its grid and the ridge at 0.1, a hundred times the ridge of the benzenoid sets or more, on every combined set, so the steric kernel is a near-identity and the fit rests on the indicator fields. The inertia axes depend on the masses, a sulfur or a boron at the rim moves the frame, and two molecules of one skeleton with different hetero atoms no longer lie on top of each other; the atom types alone reach 0.55, the ring types 0.48, and the HOMA field is the one clear gain on top (0.64 to 0.69). The descriptor forest reaches 0.56 and the additivity scheme on the 24 counts 0.51 (0.43 and 0.36 on the ten-ring molecules), so the fields stay ahead of both baselines, by a smaller margin than on the benzenoid sets. A frame from the skeleton rather than from the masses is the next thing to try on this set.

Planarity, measured. The frame rule was written for planar molecules, and the first version of the report called the non-planar ones a few helicene-like molecules. Measured, 89 % of the COMPAS-1D molecules have a heavy atom more than 0.5 Å from their best plane (median of the largest distance 1.48 Å, 95th percentile 2.91 Å; COMPAS-3D median 1.21 Å; COMPAS-2D median 0.13 Å): most ten-ring cata-condensed isomers hold a fjord and are twisted. The frame is defined for them, the two faces of it differ, and a mirror image lands in the mirrored frame, so the model is not symmetric under reflection although the gap is; a kernel averaged over the two faces is one line and was not run.

11.7 Crystals: the periodic autocorrelation kernel

The same descriptor for a periodic structure, summed over the neighbour list of one cell with its periodic images out to a cutoff $r_{\max}$, per atom of the cell, with the uniform background of each field pair removed and the cutoff tapered:

$$\Phi_{fg}(d) = \frac{1}{N}\left[P_{fg}(d) - \frac{S_f S_g}{V} B(d) \right] \tag{11.7}$$

The first term is the tapered pair histogram of the two fields, over every atom l of the cell and every neighbour m inside the cutoff, periodic images included:

$$P_{fg}(d) = \sum_{l \in \text{cell}} \sum_{m\,:\, r_{lm} < r_{\max}} w_{lf} w_{mg}\, \omega(r_{lm}) \exp\left(-\frac{(r_{lm} - d)^2}{2\sigma^2} \right) \tag{11.8}$$

The second is what that histogram would be if the same total weight of each field were spread uniformly through the cell, which is the part of P_{fg} that reports only how dense the crystal is:

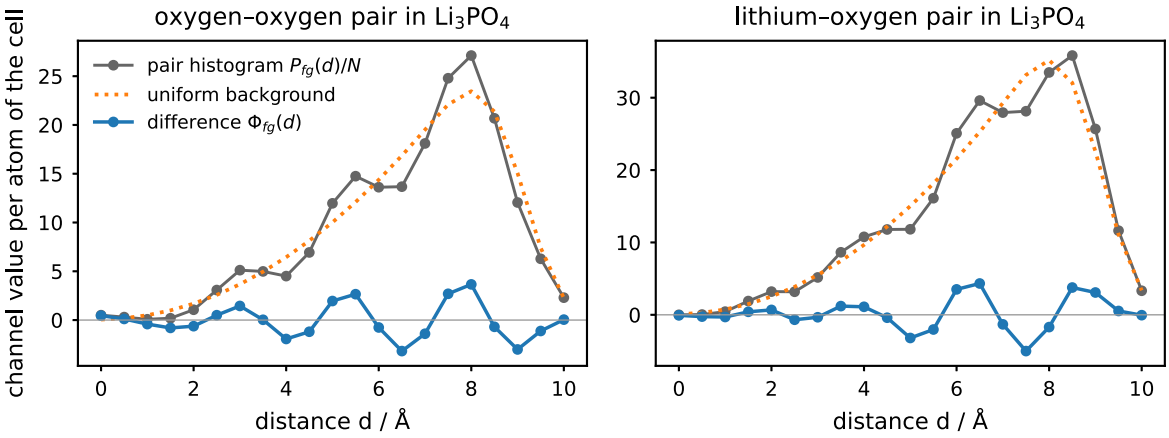
$$B(d) = \int_0^{r_{\max}} 4\pi r^2\, \omega(r) \exp\left(-\frac{(r - d)^2}{2\sigma^2} \right) \mathrm{d}r \tag{11.9}$$

Here N is the number of atoms in the cell, r_{lm} the distance from atom l to image m, $S_f = \Sigma_l\, w_{lf}$ the total weight of field f over the cell, V its volume, σ the width of the distance smearing (`--sigma`) and ω a half-cosine taper over the last `--taper` ångström. Both sums run over ordered pairs, so a pair of two different field types is counted both ways round, as the molecular descriptor counts it, and its background with it; the $l = m$ terms are kept, which makes the $d = 0$ channel the composition of the

cell. As in the two preceding sections Φ describes one structure, and the kernel of two of them is the same cosine-normalised dot product over the channels:

$$K(A,B) = \frac{\sum_c s_c^2\, \Phi_c(A)\, \Phi_c(B)}{\|s\,\Phi(A)\|\ \|s\,\Phi(B)\|}, \qquad c = (f, g, d_k) \tag{11.10}$$

It is invariant to the choice of cell, origin and orientation, so a supercell is the same material. Averaged over orientations it is the sine transform of the product of the typed structure factors, which makes the pair products of the element indicator fields the partial structure factors a diffraction experiment measures.



The three terms of Eq. (11.7) for the oxygen–oxygen and the lithium–oxygen pair of Li3PO4, the host of the hop section: the tapered pair histogram of the cell with its periodic images, the histogram the same atoms would give spread uniformly through the cell, and their difference, which is the channel the kernel sees. The histogram grows with the shell volume and falls under the taper; the difference keeps only the structure, the shells of neighbours at their distances, and says nothing about how dense the crystal is.

The fields of a solid are built by `inorganic_fieldset`: electrostatic (oxidation states guessed from the composition, or the `oxi_states/charges` array of the file), steric (atomic volume), electronegativity, one indicator field per element and one for the rest, and any per-atom array as `prop:<name>`. The command is `build-crystals` (an ASE-readable structure file, `--target KEY` or `--target-file CSV`, `--cutoff`, `--sigma`, `--taper`, `--no-background`, `--elements`, `--charges`, `--external-cv`, `--save`); in the library, `kernel_source="periodic"` on a `CMFModel` with a table from `crystal_table`. There is no co-field, but there is a co-pattern: the weight of every channel in the fitted linear model, which `top_channels` ranks by weight times spread, with the sign that says whether the pair at that distance raises or lowers the property. In the application the same model is fitted and read in the window of the section *Crystals and other periodic structures*, and `cmf predict` applies a saved one to new cells.

On the Matbench `phonons` task (1,265 crystals, the top of the phonon density of states) the linear kernel on four fields -- electrostatic, steric, electronegativity and the inverse square root of the mass -- reaches a mean absolute error of 117 cm^{-1} on the Matbench folds, 106 with a Gaussian layer, against 324 for the dummy predictor and 29 for the best graph network; keeping the uniform background improves the 58-field model from 181 to 119 on this task, so the switch is worth trying on a property that depends on composition and density (the four-field model kept it too reaches 92). The element indicator fields dilute the joint kernel — seventeen hundred equally scaled element-pair blocks against the ten physico-chemical ones — and `--block-weights` is the remedy: with the background kept, the aligned weights take the 59-field model from 119 to 97; they do not

make it better than the four physico-chemical fields alone. The full table and the co-patterns are in `benchmarks/results/matbench_phonons/table.md`.

Partial occupancy: the mean field of a disordered site

A disordered site, a Li site occupied in 54 % of the cells or a site shared by Zr and Ta, is a configurational ensemble, and this program treats an ensemble by its mean field (plan item M2; the mean-map kernel of the ensembles chapter). When the sites are occupied independently of one another the mean field over all configurations is exact and costs nothing: the weight of every field at a site is the occupancy-weighted mean of the weights of the species that share it,

$$\bar{w}_f = \sum_s p_s \, w_f(s), \tag{11.11}$$

which is what `crystal_fields` computes when a structure carries its site species (`periodic.site_species`; `periodic.read_disordered` reads a CIF with partial occupancies through pymatgen, and `read_structures` hands such a CIF to it). The overlap kernel is bilinear in the fields, so the kernel of two mean fields **is** the mean of the kernel over the two ensembles, with nothing approximated. The periodic autocorrelation is quadratic in one field, and its mean over the ensemble differs from the descriptor of the mean field wherever a site meets itself: in the self term at zero distance, in the pairs of a site with its own periodic images (a site is occupied in every cell or in none) and in the uniform background. There the mean of a product is not the product of the means, and the second moment of each site,

$$\overline{w_f w_g} = \sum_s p_s \, w_f(s) \, w_g(s), \tag{11.12}$$

takes its place. `CrystalFields` carries those second moments and `periodic_autocorrelation` uses them, so that the descriptor is the exact mean over the configurations too, per expected atom of the cell. The tests enumerate every configuration of a small disordered cell and check both statements to 1e-10; a structure without disorder carries none of this and goes through the same code unchanged. The oxidation states of a fractional composition, which a refined structure rarely balances to the last hundredth, come from the combination of common states nearest to charge balance, with every known state tried when the common ones cannot balance (a mixed-valence titanium phosphate). What this allows is in the section on migration paths: the garnets, perovskites, LGPS, NASICON and LISICON of a solid-electrolyte study, whose Li sublattices are never fully occupied.

Layers and chains: periodicity in fewer than three directions

A monolayer, a slab or a polymer chain comes in a cell like a crystal's, with vacuum along the directions that do not repeat, and the structure's periodicity flags (`pbc` in ASE; `--pbc ab` on the command line for a layer with the vacuum along *c*, `--pbc a` for a chain) say which directions do. Before plan item M1 the program read those flags in the neighbour list only. The background term divided by the volume of the cell, vacuum included, so the descriptor of a layer changed with a number the data set had chosen, the width of the vacuum; and the images of the periodic overlap kernel were generated along every lattice vector, so a layer close to a face of its cell was compared with its own copy across the vacuum. Both now read the flags.

The background per area or per repeat length. What is subtracted is the pair histogram of the same weights spread uniformly over the *periodic* directions alone; across the others every atom keeps its place. For a crystal that is the shell integral over the volume, as above. For a layer the pairs at distance r between two atoms a height h apart lie on a ring of radius $\sqrt{(r^2 - h^2)}$ in the plane,

so their count grows as 2π r dr over the area A for $r \geq h$; for a chain the two points of the axis at distance r from an atom a distance t off the axis are $\pm\sqrt{(r^2 - t^2)}$ along it, so the count is 2 du in the axial coordinate over the repeat length L. The background of a pair of field types is then a sum over the atom pairs of the cell,

$$\Phi_{fg}^{\mathrm{bg}}(d) = \frac{1}{NM} \sum_{l} \sum_{m} w_{lf} w_{mg} J_d(h_{lm}), \qquad J_d(h) = \int_h^{r_{\max}} 2\pi r\, \omega(r)\, e^{-(r-d)^2/2\sigma^2}\, \mathrm{d}r \quad \text{(layer)}, \tag{11.13}$$

with M the area or the repeat length, h_{lm} the separation of the pair across the non-periodic directions and, for a chain, $J_d(t) = \int_0^{\sqrt{(r_{\max})^2 - h_{lm}^2 - t^2}} 2\,\omega(r)\, e^{-(r-d)^2/2\sigma^2}\, \mathrm{d}u$ with $r = \sqrt{(u^2 + t^2)}$. The $l = m$ terms are in the sum, as $S_f\, S_g$ contains them for a crystal, and a disordered site enters with its second moment there (M2). The integrals are tabulated once per spec (`periodic._background_profile`) and read by interpolation; `CrystalFields.measure` is M, `periodic_measure` computes it from the cell and the flags. The tests check that a layer of atoms spread at random over the plane, at random heights within a slab, gives subtracted channels that are noise about zero and a subtracted term equal to the formula, and the same for a tube of atoms around a chain axis; and that the descriptor of a MoS_2 layer does not change when the vacuum goes from 5 to 25 Å, under an in-plane supercell, a rotation of the whole frame, a shift of the layer along the normal or a leaning third lattice vector -- while the same file flagged as a crystal changes with the vacuum, which is the defect this removes. A structure with no periodic direction has no background to remove and goes through as a molecule in a box.

The height channels of a layer. A pair-distance descriptor cannot see the normal, and for a layer the normal is physical: which atoms sit on the faces and which inside is what an adsorbate, a substrate or a Janus layer feels. With `--heights Z` (`PeriodicSpec(z_max=Z, z_step=...)`, `pbc_z_max` on the model) the descriptor of a layer is resolved by the in-plane distance ρ and the heights of both atoms above the mean plane of the layer, as the surface kernel of the chapter on molecules on a surface resolves a molecule above $z = 0$:

$$\Phi_{fg}(\rho_k, z_a, z_b) = \frac{1}{N}\left[\sum_{l} \sum_{m:\, \rho_{lm} < r_{\max}} w_{lf} w_{mg}\, \omega(\rho_{lm})\, e^{-(\rho_{lm} - \rho_k)^2/2\sigma^2}\, Z_a(z_l)\, Z_b(z_m) - \frac{R_k}{A} \Big(\sum_l w_{lf} Z_a(z_l) \Big) \Big(\sum_m w_{mg} Z_b(z_m) \Big) \right], \tag{11.14}$$

with Z_a a Gaussian of width `--height-step` about the height channel z_a on a grid from $-Z$ to Z, and $R_k = \int 2\pi\rho\, \omega(\rho)\, e^{-(\rho - \rho_k)^2/2\sigma^2}\, \mathrm{d}\rho$ the ring integral: a uniform spread over the plane puts 2πρ dρ / A pairs at every in-plane distance whatever the heights, so this background is exact. The sum runs over the neighbour list with the in-plane images, both orders of a pair of different field types are counted, and the self term is kept. A free-standing layer has no physical up, so the descriptor is the mean over the two faces (the same channels with both height axes reversed) unless `--oriented` says that the third lattice vector of the file is up, as it is for a layer on a substrate or a Janus layer with a known face: the tests show MoSSe told from its mirror image by the oriented descriptor and not by the default one, and the default equal to the mean of the two. The distance is a function of the in-plane distance and the two heights, so these channels carry strictly more than the distance channels, at the price of a block per pair of field types that is the number of height channels squared times larger: 49 times at $Z = 3$ Å every 1 Å, which is why the measurement below runs them on the four physico-chemical fields and not on the element indicator fields. Summed over both height axes with a wide grid they reduce to the in-plane distance distribution of the weights, which is a test. The height channels refuse a structure that is not periodic in exactly two directions.

Reading a layer. `read_structures` keeps the flags of the file, and a POSCAR or CIF has none, so `--pbc` sets them; a database export may leave a layer split across the cell boundary along the normal, one part at the bottom of the cell and the rest at the top, and `centre_layer` gathers it (the widest empty gap of the fractional coordinate, taken around the cycle, is put at the boundary and

the structure at the middle of the cell), which `build-crystals` does for every structure with an open direction. `lattice_spread` reports the area of a layer where it reported the volume of a crystal. The periodic overlap kernel of a series of layers on one lattice works as for crystals and does not see the vacuum either (a test). What this does not yet do is put a molecule on a periodic slab, which is the surface kernel with the slab as the receptor (item EC6 of the plan), and nanotubes have not been measured (NT4).

What the measurement shows (`benchmarks/run_jarvis2d.py`, the full table and the co-patterns in `benchmarks/results/jarvis2d/table.md`). The set is the 1,103 monolayers of JARVIS-DFT 2D (the file of 12 December 2022, downloaded on the first run), with three targets: the OptB88vdW band gap, the formation energy per atom and the exfoliation energy per atom (748 values). Five shuffled folds; the width of the channels (0.5 or 0.75 Å), the regularisation and, for the Gaussian layer, its scale are chosen by ten-fold cross-validation inside every training fold; cutoff 10 Å, taper 2 Å; the fields are the four physico-chemical ones of the phonons measurement above (electrostatic, steric, electronegativity, inverse square root of the mass; 210 channels) or those with the indicator fields of the elements present in at least ten structures (66 fields); the height channels run every 1 Å from -3 to 3 Å (the median layer is 3.8 Å thick; 10290 channels on the four fields). Every structure is run twice, once flagged as a crystal and once as a layer, and the vacuum sensitivity of a variant is the median relative change of its channel vector when the vacuum of every cell is doubled.

The vacuum. Flagged as a crystal, the descriptor changes by a median 50 % (at most 145 %) when the vacuum is doubled; flagged as a layer, with the background per area or kept, by 1e-15 -- the rounding error. The accuracy of the crystal flags does not suffer within this set, because JARVIS builds its cells alike (nine in ten have between 15 and 25 Å of vacuum, median 20.6 Å), so the dependence is nearly a constant factor across the set and the model learns it; it would not carry to a set built with another convention, to a slab model with a different vacuum or to the same layer in a supercell of another height, and that is what the flags remove.

The background. With the linear kernel the three pair-distance descriptors are alike on the band gap (0.82, 0.819 and 0.829 eV for the crystal flags, the layer flags with the background kept and the layer flags with the background per area; the mean predictor 1.05 eV), and the background per area costs on the formation energy (0.329, 0.349, 0.37 eV/atom; mean predictor 0.70) and on the exfoliation energy (66.4, 68.3, 74.7 meV/atom; mean predictor 71). The loss is spread over all thicknesses and not in the thick layers; what the subtraction removes is the areal density of every pair of field types, the same information whose removal hurt the phonons model above, and a linear model on the cosine-normalised kernel cannot put it back. The Gaussian layer on the same channels does better everywhere (0.68 eV, 0.283 eV/atom, 56.1 meV/atom with the background per area), and its controls, the same kernel on the crystal flags (0.67, 0.269, 53.5) and with the background kept (0.668, 0.274, 53.7), are within the fold spread of it: the gain is the kernel's, and on the four fields the Gaussian layer recovers what the per-area background removed. With the element indicator fields the linear model reaches 0.679 eV, 0.271 eV/atom and 67 meV/atom with the background kept, against 0.732, 0.311 and 64.3 with it removed per area; the Gaussian layer does not improve the 66-field model (0.719, 0.312, 63.3 per area; 0.67, 0.269, 62.6 kept; 0.675, 0.259, 61.1 crystal flags), and there the per-area background still costs on the gap and the formation energy. For a layer the switch `--no-background` is therefore worth trying as it is for a crystal, and it keeps the invariance to the vacuum.

The height channels. On the four fields with the linear kernel they gain on every target over the distance channels with the same background: 0.785 against 0.829 eV, 0.349 against 0.37 eV/atom, 63.7 against 74.7 meV/atom. With the Gaussian layer they gain on the exfoliation energy (51.2 against 56.1 meV/atom, R^2 0.30 against 0.14), the best result on that target here (the crystal flags with the same kernel reach 53.5 meV/atom, R^2 0.24; the margin in MAE is within one fold spread,

the margin in R^2 is not), and not on the band gap (0.692 against 0.68 eV) or the formation energy (0.286 against 0.283 eV/atom). The exfoliation energy is a property of the faces, which the heights see and the distances do not; the gap and the formation energy are properties of the bonding, which the distances already carry. The oriented descriptor is the symmetric one within the fold spread (63.9 against 63.7 meV/atom), as it should be on a set of mostly symmetric layers. What remains open: the height channels with the element indicator fields (a block per element pair times 49 is more memory than this measurement allowed), nanotubes (NT4), a molecule on a periodic slab (EC6) and a second monolayer set such as C2DB for the transfer across vacuum conventions.

11.8 A series on one lattice: the periodic overlap kernel

The autocorrelation above is invariant to rotation, which is what a set of unrelated crystals needs, and the price of that invariance is the co-field: a distance distribution can say which pair of field types at which distance matters, never *where* in the cell. A doped, substituted or defective series on **one parent lattice** is the other case. There the cells share a frame, exactly as the molecules of a congeneric series do, and the overlap integral -- this program's own kernel -- applies with one change: the integral runs over one cell and the fields are periodic.

The field of a cell is the lattice sum of its atoms' Gaussians,

$$\rho_{fi}(\mathbf{r}) = \sum_{l \in \text{cell}\, i} \sum_{\mathbf{T}} w_{lf} \exp\left(-\frac{b_{lf}}{2} |\mathbf{r} - \mathbf{r}_l - \mathbf{T}|^2 \right) \tag{11.15}$$

with **T** running over the translations of the lattice, and the overlap integral is taken over one cell Ω rather than over all space:

$$K_f(i,j) = \int_\Omega \rho_{fi}(\mathbf{r})\, \rho_{fj}(\mathbf{r})\, \mathrm{d}\mathbf{r} = \sum_{l \in i} \sum_{m \in j} \sum_{\mathbf{T}} k_f(|\mathbf{r}_l - \mathbf{r}_m - \mathbf{T}|) \tag{11.16}$$

where k_f is the closed-form Gaussian pair overlap of (2.3). One atom stays in the reference cell and the other runs over every periodic image. So the periodic kernel *is* the ordinary kernel evaluated against an image-expanded table, and every shortcut of the kernel code -- the grouping by width, the cutoff of plan item B2, the compiled threaded loop -- carries over untouched (`cmfqsar.periodic.periodic_overlap_kernels`).

Switch it on with `pbc_overlap=True`:

```
from cmfqsar import periodic
from cmfqsar.model import CMFModel

fs = periodic.inorganic_fieldset(elements=["Li", "P", "O"], alpha=2.0)
table = periodic.crystal_table(structures, fs, names)
model = CMFModel(fieldset=fs, method="krr", params={"lam": 1e-3},
                 normalize=True, pbc_overlap=True, engine="auto")
model.fit(table, y, cv_folds=10)
print(periodic.lattice_spread(table))        # how far the set is from one lattice
print(model.atom_contributions(table.molecules[0]))   # one number per site of the cell
```

The application fits the same model from the *Crystals and other periodic structures* window of the application chapter, draws the co-field inside the cell and colours the sites by these contributions.

It is the `fields` kernel source with the images added, so nothing above it changes: the cosine normalisation, the field scaling, the learners, the co-field and the per-site contributions are the

same code for a crystal as for a molecule. The co-field drawn on a grid is then periodic, which is what the grid code could not do with open boundaries.

Three properties, and the test suite holds the kernel to each. With a box far larger than the fields reach, the periodic kernel equals the open-boundary one exactly. Writing the atoms outside the cell instead of inside it changes nothing, while it changes the open-boundary kernel of the same atoms by more than one per cent -- a crystal is its lattice sum and does not care which atoms were called the cell. A supercell multiplies the raw integral and leaves the cosine-normalised kernel unchanged.

Two things it cannot do. The integral runs over the cell of one structure and the images of the other, so when two cells differ in size the two orders are different integrals; the kernel is their average, and `periodic.overlap_asymmetry` reports the disagreement -- 7e-09 on one lattice and 17 % across a 2.5 % lattice mismatch, which is the number that says whether a set really is one lattice. And a coalition of sites is not a structure (removing a site removes it from every image), so the Shapley decomposition is refused for this kernel while the co-field and the per-site contributions, which are sums and not coalitions, are exact.

Measured against the autocorrelation. `benchmarks/results/periodic_overlap/table.md`: 1,500 cells of `matbench_perovskites` with their formation energy -- the target of Matbench v0.1, the energy of the whole five-atom cell in eV, from Castelli et al. (2012) -- five Matbench folds, the same fields and the same ridge grid for every row. The cells are relaxed, so their lattice constants differ by up to 82 % in volume: a frame error for a point-by-point comparison. The second frame gives every cell the mean lattice constant and keeps its fractional coordinates, which puts the set on one lattice exactly.

kernel	frame	R^2	MAE, eV
periodic autocorrelation	as given	**+0.497**	**0.384**
periodic autocorrelation	one lattice	+0.494	0.405
periodic overlap	one lattice	+0.479	0.407
overlap, images left out	as given	+0.448	0.404
overlap, images left out	one lattice	+0.379	0.435
periodic overlap	as given	+0.350	0.449

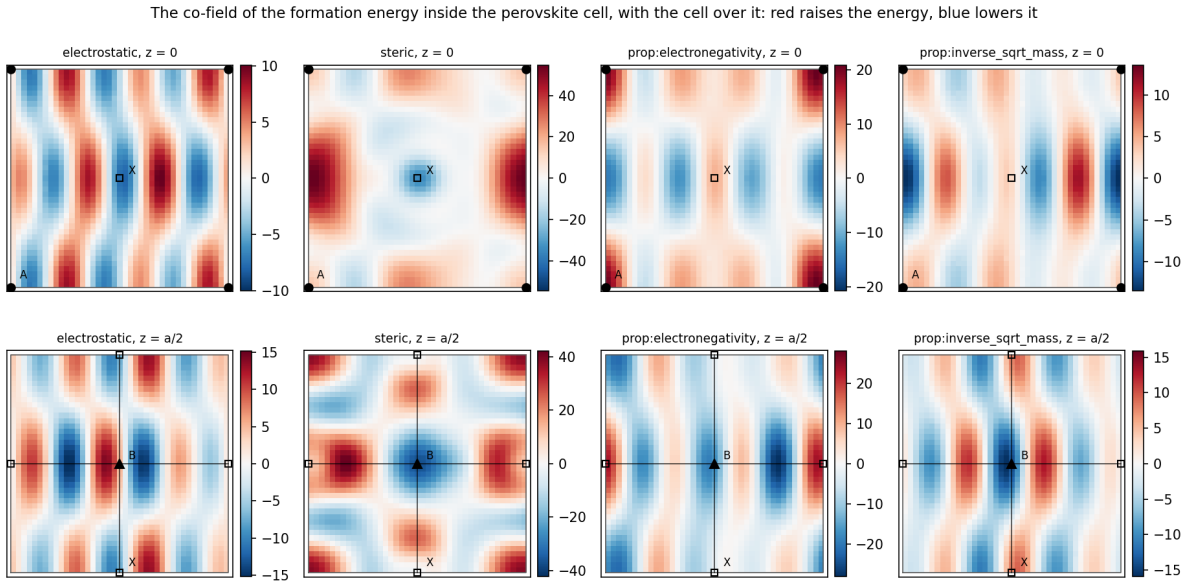
Read the table by what the frame does to each row. The autocorrelation does not notice it (+0.497 against +0.494), which it must not, since it is invariant. The periodic overlap gains **+0.129** from being given a frame, which is the largest effect in the table and is exactly what a kernel that compares fields point by point should show. The open-boundary overlap *loses* from the same operation, because without the images it compares five atoms as a cluster and the rescaling only distorts them.

And it does not beat the autocorrelation. The reason is in the same column: the operation that gives the overlap kernel its frame takes the lattice constant out of it, and for a formation energy the lattice constant is a strong predictor on its own. On a set of different compounds there is no frame to be had for free. **The overlap kernel for crystals is therefore for the series the item names -- one host lattice with a substitution, a vacancy or a dopant moved from site to site -- where the frame is given and nothing has to be rescaled to get it.**

The same measurement says how far the two frames are apart in the kernel's own terms. The disagreement between the two orders of the integral, which is zero when the cells are identical, is

0.0 % on one lattice and 200 % in the cells as they come -- that is, in the raw cells the two orders of the same overlap can have opposite signs, so those cells are not a frame at all. It is worth computing `periodic.overlap_asymmetry` on any new series before trusting a number from it.

What the co-field says. This is what the kernel is for, and no autocorrelation can produce it.



The co-field of the formation energy inside the perovskite cell with the cell drawn over it, from the model on one lattice: one column per field, the upper row in the plane through the A site and the X sites (z = 0), the lower row in the plane through the B site and the equatorial X sites (z = a/2). Red is field that raises the predicted formation energy per unit of field, blue field that lowers it. The circle is the A site, the triangle the B site, the square an X site, and the lines in the lower row are the four equatorial bonds of the BX_6 octahedron; the sites on the edges are the periodic images of the sites inside, because the field repeats with the lattice -- which is what the periodic sum is for. The steric map is blue on the X site at the centre of the upper row and on the B site at the centre of the lower one, and red away from both, along the faces that carry the A site; the other three oscillate with the lattice instead of sitting on the sites at all. Drawn by `benchmarks/run_periodic_overlap.py`.

It is periodic, and its strongest features sit on the sites: the steric co-field has a deep negative lobe exactly at the B site and at the oxygen, and a positive one at the A site, so steric weight at the B and X sites lowers the predicted formation energy while steric weight at the A site raises it. The electrostatic, electronegativity and mass co-fields oscillate with the lattice instead of sitting on the sites, and are correspondingly harder to read -- which is itself worth seeing, and is the kind of thing a picture shows and a channel weight does not.

Decomposition rule 3 gives the same reading as numbers, one contribution per site of every cell (a sample of 150 cells): A +2.37 ± 0.58, B -6.16 ± 0.81, X -3.87 ± 0.55 eV. The means carry a large constant that the bias cancels, so what they say is in the spread: **the B site's contribution varies most from compound to compound**, which is the site the chemistry of a perovskite turns on.

In three dimensions, over the structure. A plane is a cut, and the co-field of a crystal is a periodic function of all three coordinates. `cofield_volume` evaluates it on a grid inside the cell exactly as for a molecule -- the basis is the image-expanded training table, so the volume that comes out is already periodic -- and `cofields.export_cube` writes that grid as a Gaussian cube **with the atoms of a cell in the file**, which is what a viewer needs to draw the field around the structure:

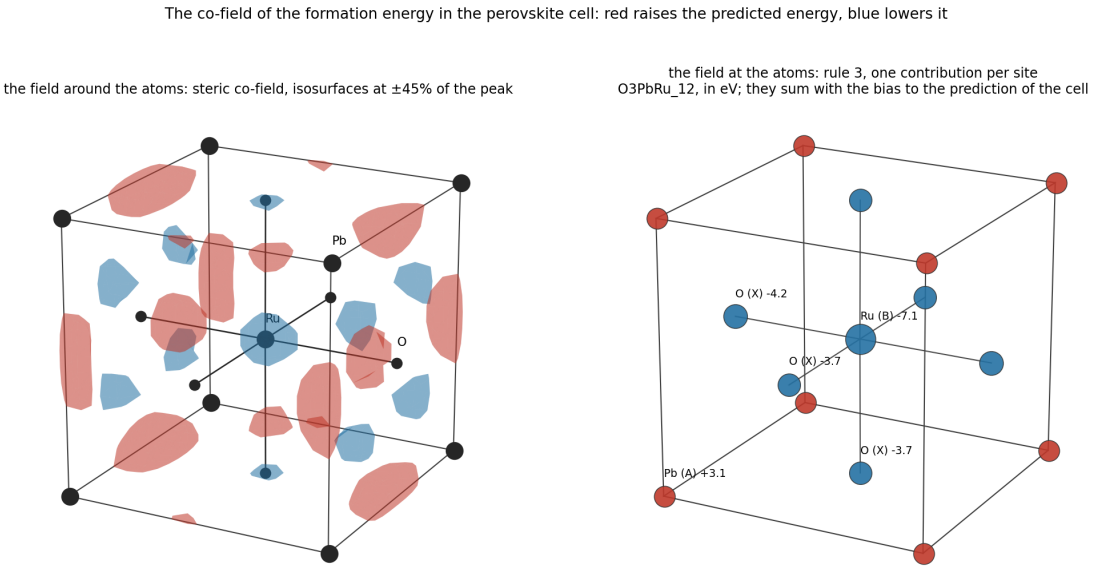
```
from cmfqsar.cofields import Grid, cofield_volume, export_cube

a = float(table.molecules[0].cell[0, 0])
grid = Grid(origin=np.zeros(3), spacing=a / 16, shape=(17, 17, 17))
volume = cofield_volume(model, f=1, grid=grid, include_scale=True)   # the steric co-field
export_cube("cofield_steric.cube", grid, volume, table.molecules[0])
```

The benchmark writes that file beside its results; one cell is enough, because the field repeats and any viewer will tile it. Nothing in this is special to crystals except the basis: the same two calls draw a co-field around a molecule, and the periodic sum is what makes the volume come out periodic.

It also draws the isosurfaces itself, reading that cube file back as an outside viewer would, which is both the picture and a check that the file says what it is meant to say (`run_periodic_overlap.py --stage isosurface`, `cofields.isosurface` for the marching cubes).

Two overlays, and they are not the same picture. The field can be drawn *around* the atoms, as a surface, and *at* the atoms, as one number per site: decomposition rule 3 splits the prediction over the sites of the cell exactly as it splits it over the atoms of a molecule, because the periodic kernel is the overlap kernel with the images summed in. The second is the crystal form of the atom-contribution figure of *Reading the model*.



Two overlays of one co-field on one cell. Left, the field **around** *the atoms: isosurfaces of the steric co-field at plus and minus 45 % of its largest value, red where field raises the predicted formation energy and blue where it lowers it, with the atoms the cube file carries -- lead at the eight corners, ruthenium at the centre, and the six oxygens of the octahedron with its arms drawn. Right, the field* **at** *the atoms: one sphere per site, its area the size of that site's contribution and its colour the sign. They say the same thing twice: the blue surface wraps the B site and the oxygens on the left, and those are the sites whose contributions are negative on the right. Drawn by* `benchmarks/run_periodic_overlap.py`*.*

For the cell drawn there, $PbRuO_3$, the sites contribute Pb +3.10, Ru −7.09 and the three oxygens −3.70, −3.67 and −4.20 eV, which with the bias of +16.84 add up to the prediction of +1.28 eV for the cell -- rule 3, in a crystal. The application draws the same two things in its 3D view, because a model fitted with `pbc_overlap=True` is still the overlap kernel: `atom_contributions` returns one number per site and the viewer colours the sites by it. The periodic **autocorrelation** kernel can do neither; it has no frame and no atom split, which is what this kernel was added for.

What a cube of a periodic co-field costs is worth knowing before asking for a fine one. The work is the number of grid points times the Gaussians each point sees, and the basis of this model is 1.8 million of them at 69 per cubic angstrom, so a point sees of order 10^5. Measured here, in seconds, for one field over one cell:

grid	spacing	electrostatic	steric, a 1.7 Å atom	steric, a 2.4 Å atom
10^3	0.46 Å	1.1	2.8	7.3
18^3	0.24 Å	3.4	13.8	55.5
26^3	0.17 Å	8.7	40.5	132.8

The cost is linear in the points and grows with the width of the field, because a wider Gaussian reaches further and every point then sees more of them; it is also linear in the size of the training set, since that is what sets the number of Gaussians. The memory is bounded whatever the density: `evaluate_gaussians` cuts its chunk from the neighbour count it measures, so one neighbour list stays under `cofields.MAX_PAIRS`.

A defect series on one host: the host symmetry and the arrangement kernel

The perovskites above are different compounds forced onto one lattice. The series the kernel was built for is the other case (plan item M14): one host, one supercell, and cells that differ only by which sites are changed -- a vacancy here, a substitution there. Two things are added for it.

Difference fields. A cell of the series is the host plus a change on a few sites, and the field of the cell is the field of the host plus the field of that change: a vacancy is the host atom with its weights negated, a substitution is that row and the new species on the same site (`periodic.defect_fields`, `periodic.defect_table`; `periodic.match_sites` finds the site of every atom at the nearest image). The kernel is bilinear, so the kernel of two difference fields is the kernel of the two whole cells minus terms that are the same for every pair, and a model with a bias on the differences is the model on the whole cells -- the tests check it to 1e-7 on real cells. The differences are small tables, a few rows per cell instead of two hundred, and they are not normalised: the pristine cell is the empty difference, and its norm is zero.

The host symmetry. Two arrangements of the same defects that differ by a symmetry operation of the host -- the same vacancy pair two sites along -- have the same energy, but in a fixed frame their fields do not overlap at all. The kernel is therefore averaged over the space group of the host, read from spglib (`periodic.host_symmetry`; 768 operations for an 8 x 8 supercell of a monolayer, 64 translations times 12 point operations):

$$\bar{K}(P,Q) = \frac{1}{|G|} \sum_{g \in G} K(P, gQ) \tag{11.17}$$

which is the kernel of the orbit-averaged fields and so a kernel in its own right. Averaged this way, the **linear** overlap turns out to be a count kernel: the orbit mean of a defect's field is the same wherever the defect sits, so the average is additive over the defects and blind to their arrangement. That is a useful baseline and not the kernel the series needs. The arrangement survives when the **Gaussian layer** is averaged over the group instead,

$$\bar{K}_\gamma(P,Q) = \frac{1}{|G|} \sum_{g \in G} \exp(-\gamma\,[K(P,P) + K(Q,Q) - 2K(P,gQ)]), \tag{11.18}$$

which is the orbit mean in the layer's feature space, positive semidefinite for the same reason (`periodic.symmetry_averaged_kernels`; `CMFModel(..., pbc_overlap=True, pbc_symmetry=...)`). A compiled loop (`fastkernel.defect_kernels`) takes every atom pair at its nearest image in fractional coordinates, with the cutoff capped at half the cell, and sums both averages at once: 125 s for 2,000 cells and 768 operations, against hours for the image-expanded reference, which the tests keep for agreement. The per-atom contributions of rule 3 go through the same loop.

Measured on 2DMD (Huang et al. 2023): one to three vacancies and substitutions on the 8 x 8 supercell of MoS2 and of WSe2, the DFT formation energy of the cell, 2,000 random cells per material and then all 5,933, five random folds with the attenuation, the bandwidth and the ridge chosen inside each training part (`benchmarks/run_2dmd.py`, `benchmarks/results/2dmd/table.md`). All 5,933 cells of each material are distinct under the host group, so every test cell is an arrangement the model has not seen. The error per defect site, which is what the literature on this set reports:

kernel	MoS2, 2,000 cells	WSe2, 2,000 cells	MoS2, all 5,933	WSe2, all 5,933
fixed frame, no symmetry	60	57	42	42
counts (the linear symmetry average)	41	42	41	41
periodic autocorrelation	8.3	15.7	3.0	6.0
arrangement (the symmetry-averaged layer)	**1.8**	**5.2**	**1.9**	**4.4**

On 2,000 cells the arrangement kernel beats the autocorrelation by 6.5 [5.3, 7.7] and 10.5 [9.1, 12.1] meV per site (paired bootstrap over the cells, 95 %) -- the half of the gate of the previous section that the perovskites could not test -- and the fixed frame and the counts lose to it by 26 to 52 meV: the symmetry average is what makes the overlap kernel work on a series, and the layer is what makes it see the arrangement. On all 5,933 cells the autocorrelation gains most from the extra training cells and the lead of the arrangement kernel shrinks to 1.0 [0.8, 1.2] and 1.5 [1.3, 1.9] meV per site, still with intervals that exclude zero; the fixed frame reaches the counts, because with enough cells it learns the orbit from the data. The published errors on this set, on the authors' own split of all 5,933 cells, are 2.6 and 4.8 meV for a sparse graph network and 11.8 and 14.7 for gradient boosting on descriptors (Kazeev et al. 2023). Trained on the 131 cells with one or two defects and predicting those with three (1,869 in the 2,000-cell sample, 5,802 in the full set), the arrangement kernel reaches 4.3 and 4.7 meV per site on MoS2 and 54 and 38 on WSe2, where the additive count kernel gives 38 on both; the autocorrelation does not extrapolate at all (R^2 near zero), and the fixed frame reaches R^2 0.6 to 0.8.

What each kind of defect costs. The per-defect contributions of the symmetry-averaged linear model (rule 3) are one number per kind of defect, the same to four decimals in every cell, as the symmetry requires -- and they are the chemistry the data hold. In MoS2 a sulfur vacancy costs 2.59 eV, a molybdenum vacancy 6.98, selenium for sulfur 0.31 and tungsten for molybdenum 0.19, against the four single-defect cells of the set at 2.65, 7.12, 0.28 and 0.17 eV; in WSe2 a selenium vacancy 2.62, a tungsten vacancy 5.09, sulfur for selenium -0.41 and molybdenum for tungsten -0.42, against 2.70, 5.30, -0.38 and -0.41. The difference is the mean interaction of the three defects in the cells the model was fitted on. The co-field is not the reading for this series: averaged over the host group it is one value per site type and says which species, not where, and the arrangement model has a layer, so rule 3 does not apply to it.

11.9 A site of the cell as the sample: the split of the autocorrelation

The pair histogram P_{fg} of the periodic autocorrelation above is a sum over ordered pairs of atoms of one cell, and such a sum can be grouped by the atom that stands first in each pair. The grouping is exact and costs nothing: the same neighbour list, read one row at a time. Writing $G_k(r)$ for the Gaussian of channel d_k times the taper,

$$\Phi_{fg}(d_k) = \frac{1}{N} \sum_{l \in \text{cell}} s_l(f, g, d_k), \qquad s_l(f, g, d_k) = w_{lf} \sum_m \sum_{\mathbf{T}} w_{mg}\, G_k(|\mathbf{r}_l - \mathbf{r}_m - \mathbf{T}|) \quad (11.19)$$

with the second order $s_l(g,f,d_k)$ added for a pair of different field types, the $l{=}m$ term where the spec keeps it, and the uniform background of that site subtracted, exactly as the whole cell has them. The row s_l is the **profile of site l**: which field type stands how far from it, with the periodic images summed in. It is the periodic counterpart of the atom-centred environment of the next section, with two differences. Nothing is cut out of the solid, so a site deep in a crystal is described as it is and not as the middle of a cluster; and the profile is one-centred -- pairs that start at the site -- rather than all pairs inside an envelope.

`cmfqsar/sites.py` (plan item M5) holds it. `site_autocorrelation` returns the rows, and their mean is the descriptor of the cell to machine precision, which is the property the module is tested on. The rows are not divided by the number of atoms, so a site profile does not change when the same site is written in a larger supercell.

A site is a sample. `site_fields(crystal, site)` returns the cell with that site as its centre, and the periodic autocorrelation of such a record is that site's row. Everything else follows without new machinery: `site_table` builds the table, `CMFModel(kernel_source="periodic")` fits it, and the targets are the ones `cmfqsar/atomtargets.py` already holds -- one array per structure, *not a number* where a site was not measured. The folds hold out whole **structures**: two sites of one cell, one in the training set and one in the test set, would let the model see the cell it is being tested on.

The descriptor needs no alignment, because it is built from distances. Its pair term is local -- only neighbours inside the cutoff enter it -- but its background is not, being the mean field of the whole cell times the shell integral, so two cells of different composition have different backgrounds at the same site. With `background=False` the profile depends on nothing beyond the cutoff.

What a site profile is worth. The first set is the single-atom catalysts of the EC8 pilot (`benchmarks/run_sac.py`, which cuts the site out of the sheet and is the thing to be level with): the metal site of 139 M@C(4-x)Nx sheets, with the binding energies of H, O and OH on 104 of them and of OOH on 91. Six bare sheets have been recomputed with plain PBE since the pilot ran and more are being recomputed, so these numbers are not comparable with the pilot's own and every reference here is recomputed on the same catalysts. Three designs of folds, as the pilot used: random catalysts, a whole metal held out, a whole motif held out. Every descriptor is a family -- three field sets by three cutoffs for the site profile and for the whole cell, the widths and attenuations of the pilot for the cut cluster -- and each family chooses inside an inner cross-validation of the same kind as the outer folds, so the comparison favours none. Mean q^2 over the four endpoints:

Descriptor	Random catalysts	One metal held out	One motif held out
site profile of the cell (M5)	**+0.792**	**+0.721**	**+0.755**

Descriptor	Random catalysts	One metal held out	One motif held out
the cut cluster, invariant (LP2)	+0.801	+0.739	+0.712
the cut cluster, overlap kernel (EC8)	+0.782	+0.695	+0.724
the whole cell, autocorrelation	+0.794	+0.756	+0.744
counts in shells around the site	+0.758	+0.096	+0.749

The site profile does what cutting the site out of the sheet does, without cutting it, and nothing more. Of the twelve paired comparisons (four endpoints by three designs, the catalysts resampled) it is level with everything built from fields: against the cut cluster better in two and worse in one for the overlap kernel, better in one and worse in none for the invariant one; against the whole cell better in one (the OH binding with a motif held out, +0.066) and worse in two (the O and OOH binding with a metal held out, -0.046 and -0.085). It is clearly better only than the counts, in five of the twelve, never losing to them. So the site resolution saves the cut and the termination of a cluster, which is worth having on a site that cannot be cut out cleanly, and it buys no accuracy on this set. The counts are the other reading of that table: they hold their own when the metals are known and collapse to +0.096 when a metal is new, which is what a descriptor made of element labels does, while every descriptor built from fields keeps most of its accuracy. The set is small enough to move: the recomputed sheet added between two runs of this benchmark took every method's OOH q^2 from about +0.93 to about +0.77, so the differences between the methods are worth reading and their absolute values are not. The full table is in `benchmarks/results/sites/table.md` (`benchmarks/run_sites.py`).

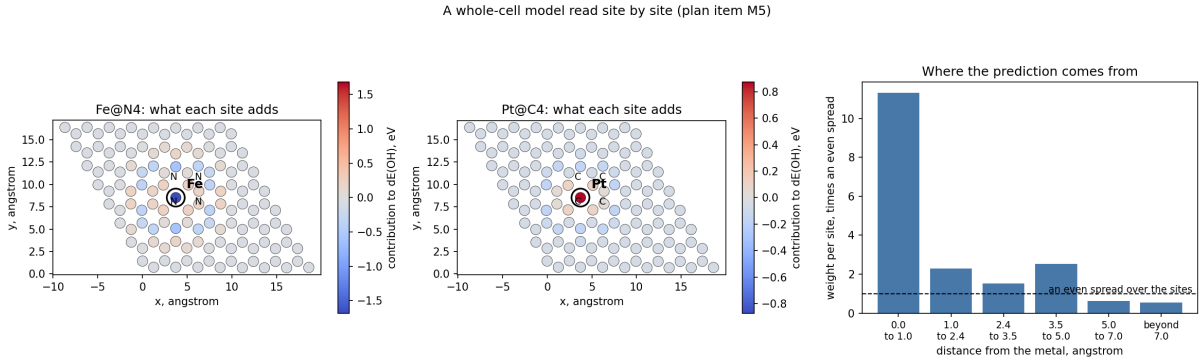
Many sites in one cell. The second set is the bond-valence site energy of every lithium site of the three solid electrolytes of `examples/hops`: 104 sites in beta-Li3PO4, Li3BP2O8 and alpha-LiBO2, where a fold is a whole compound the model has never seen. There the site profile **fails as an absolute predictor and works as a ranking**. It cannot place a compound it has not seen on the energy scale: the offsets are -0.12, +0.04 and +0.13 eV and the overall q^2 is negative for every method, the counts and the mean of the other compounds included, because with three compounds almost all of the variance is between them. Inside a compound it orders the sites -- Spearman +0.78, +0.84 and +0.55, and q^2 about the compound's own mean +0.60 for the phosphate and +0.77 for the borophosphate. The borate is the third case: its sites differ by 0.007 eV, which is nothing to predict.

A whole-cell model read site by site. The split works in the other direction as well. A model on the whole-cell descriptor is linear in the channels, the channels of a cell are the mean of its site profiles, so the prediction minus the bias is a sum of one number per site:

$$\hat{y}(i) - b = \sum_{l \in \text{cell } i} c_l, \qquad c_l = \frac{1}{N} \frac{(\boldsymbol{\sigma} \odot \mathbf{s}_l) \cdot \mathbf{w}}{\|\boldsymbol{\sigma} \odot \mathbf{x}_i\|} \tag{11.20}$$

with boldsymbol{σ} the channel scale, $\mathbf{x}_i$ the cell's channels and **w** the co-pattern of the fitted linear model (`periodic.channel_weights`). Nothing is attributed: this is rule 3 for the autocorrelation kernel, and `sites.site_contributions` returns the numbers. That is the local reading the autocorrelation could not give. A distance distribution says which pair of field types at which distance raises the property; the split says **which site of the cell** carries it.

On a model of the OH binding energy over the 104 catalysts (q^2 +0.893 in five folds, 0.51 eV) the weight falls where a chemist would put it: the metal site carries 8.9 % of it, which is 11.3 times what an even spread over the 127 sites of the sheet would put there, and the metal with its four neighbours carries 16.0 % against 3.9 %. Beyond 5 Å from the metal every site carries about half of an even share. The channels of the metal site say what the model reads there: for Pt@C4 the d electron count at the site itself, for Fe@N4 the electrostatic field at 2.0 Å, which is the first shell.



What each site of the sheet adds to the predicted OH binding energy of a whole-cell model, for Fe@N4 and Pt@C4: red raises the binding energy, blue lowers it, the circled atom is the metal and the labelled ones its four neighbours. The right panel is the weight per site by distance from the metal, divided by the weight an even spread over the sites would give; the dashed line is that even spread. The metal site carries eleven times its share and everything beyond 5 Å about half. Drawn by `benchmarks/run_sites.py`.

The limits. The height channels of a layer have no site split in this version: a site profile of a slab is taken with `z_max = 0` and does not tell a neighbour above the plane from one below it at the same distance. The contributions are defined for the linear kernel layer, as the co-pattern is. And the background of a site profile is a property of the whole cell, so a set of unrelated cells carries their compositions in it; a series on one host lattice does not have that difficulty.

A map as an output mode

The split above is a reading for a person: red and blue spheres on a molecule, a number per site of a cell. A coarser model needs the same numbers as data — a barrier per site, a reactivity per surface site, a growth rate per patch — and that is an output mode rather than a figure. It is `cmfqsar.sitemap` (plan item MS2), and it holds two different maps behind one entry.

The split of one prediction. `site_map(model, structure)` returns a `SiteMap`: one number per place, where each place is, the fitted constant as a place of its own, and the prediction the model itself made. A molecule is split over its atoms by rule 3 and a periodic cell over its sites by Eq. (11.20), so the caller does not have to know which kernel the model carries. The constant is kept apart because it belongs to no place, and with it the column adds up to the prediction exactly: `SiteMap.check()` verifies that, and the entry runs it by default.

```
from cmfqsar.sitemap import by_element, region_map, site_map

m = site_map(model, mol, units="log units")
print(m.prediction, m.total, m.bias)    # the second is the first, to round-off
m.write_table("map.csv")                # one row per place, with x, y, z
region_map(m, by_element(mol))          # the same map, grouped
```

Regions. A place of a coarse model is rarely one atom. `region_map` groups the places and sums their numbers, which needs no new mathematics because a split is additive: `by_element` collects

each element, `shells_around` collects them by distance from one place, measured through the images when the structure is periodic, and any grouping of indices can be given directly. A place may belong to one region at most, because a split is shared out and not copied, and the places left over are collected into one region so that the total still holds; dropping them instead is allowed and the map then says in a note that it no longer adds up.

A prediction per site. The second map is not a split. A site model (the first part of this chapter) takes one site of a cell as its sample, so it can be asked about every site of a cell: `site_predictions(model, crystal)` returns one answer per site, in the units of the property. The two must not be confused, and a map says which it holds in its `quantity` field. A contribution is a share of one structure's answer; a prediction is an answer of its own. `SiteMap.check` refuses a map of predictions for exactly that reason: there is no total for it to check.

On the command line. `cmf sites MODEL STRUCTURES --out map.csv` writes either map, choosing by what the model was fitted on, and `--regions elements` or `--regions shells --centre I` group the places first. The table has one row per place with the columns `structure, place, label, atoms, x, y, z, quantity, value`: a reader that steps on places keeps the rows it needs, and a reader that wants the structure's own answer back adds the `value` column of one structure, constant included. `--json` writes the same maps with every number.

What has no map. Four kernels are refused by name, with the reason: the in-plane surface channels, whose places are columns of a slab rather than atoms; the reciprocal-space structure factors, which are sums over the cell as a whole; the three-body channels, whose shares belong to triangles and not to single atoms; and the periodic overlap kernel, where a place of one cell overlaps places of every image of the other. A nonlinear kernel layer is refused as well, as rule 3 is, and the Shapley values take its place.

Does the map rank the sites as a site model ranks them? This is the gate of item MS2 and the answer is in two parts. The arithmetic half holds exactly: on the single-atom catalyst set the sum of the places and the constant differs from the prediction by at most 3.5e-13, which is round-off. The ranking half does not hold, and the measurement says why. Both models are fitted on the same catalysts with the same kernel, the same cutoff and the same ridge value — only the sample differs, a cell for one and the measured site of a cell for the other — and everything below is out of fold. The catalyst set has grown since the numbers earlier in this chapter were computed, so the counts differ between the two; the comparison here is internal to it.

Target	Catalysts	The metal is the largest place	Weight on the metal	Within a cell, mean rho	At the measured site, across cells
dE_H	133	72 %	11.7 %	+0.465	+0.219
dE_O	133	62 %	9.0 %	+0.476	+0.743
dE_OH	133	77 %	8.1 %	+0.350	+0.600
dE_OOH	119	69 %	8.6 %	+0.025	-0.001

The split locates the property. The binding energy of these catalysts belongs to the metal site, and the split puts its largest number there in 62 to 77 % of the cells -- its median rank is 1 on every target -- giving it 8.1 to 11.7 % of the weight where an even spread over the 127 sites of the sheet would give it 0.8 %, ten to fifteen times its share. What the split does not do is reproduce the site model's ordering of all the sites of a cell: the mean rank correlation inside a cell is +0.025 to +0.476 and it passes +0.7 on at most 13 % of the cells. Across the cells, at the one site of each cell

that was measured, it is -0.001 to +0.743, which passes +0.7 on one target of the four.

Two reasons, and neither is a fault in either model. The quantities differ: a contribution says how much of this cell's answer came from this place, and a prediction says what would be measured at that place, so the sites a cell model leans on need not be the sites that bind most strongly. And the site model was fitted on the metal site of every cell and on no other site, so its answers at the carbon and nitrogen sites are extrapolation — the gate compares the split against a model that was never shown the places the comparison runs over. The reading that stays inside both models' training data is the two predictions themselves, and there they agree closely:

Target	Cell model against site model	Cell model against the measurement	Site model against the measurement
dE_H	+0.968	+0.771	+0.762
dE_O	+0.987	+0.927	+0.927
dE_OH	+0.992	+0.952	+0.956
dE_OOH	+0.971	+0.883	+0.862

So the map is usable as an output mode — that is the first half of the gate, and the weight it puts on the metal — while the ordering it gives inside a cell is a statement about the cell model and not a site-level prediction. For a number per site to hand to a coarser model, fit a site model and use `site_predictions`; the split says which places the cell model read, which is what interpreting it needs. The full table is in `benchmarks/results/sites/table.md` (`benchmarks/run_sites.py --agreement`).

11.10 Order the cutoff cannot reach: the reciprocal-space channels

Everything above is a sum over a neighbour list, so a crystal is described out to $r_{\max}$ and no further. A superstructure with a long repeat, a modulation, the long-range part of the electrostatic interaction: none of them reaches the descriptor. The channels that do cost nothing for the distance, because they are sums over the atoms of **one cell** with no neighbour list at all (plan item M6, `cmfqsar/reciprocal.py`).

For a field *f* and a vector **G** of the reciprocal lattice -- the lattice of vectors with **G**·**T** = 2π *n* for every lattice translation **T** -- the **typed structure factor** is

$$S_f(\mathbf{G}) = \sum_{l \in \text{cell}} w_{lf} \exp(i\,\mathbf{G}\cdot\mathbf{r}_l) \tag{11.21}$$

It is the Fourier transform of the field of the whole infinite crystal at **G**, and it is what a diffraction experiment measures: $|S_f(\mathbf{G})|^2$ is the intensity of the Bragg reflection at **G** when field *f* is the scattering power. One value of S_f already holds information about the whole structure, however far the order extends, which is why these channels have no cutoff radius.

The channel at the reciprocal length q_k for a pair of field types is the intensity of that pair, averaged over the directions of **G**:

$$\Psi_{fg}(q_k) = \frac{1}{N}\left[\frac{(2\pi)^3}{V I_k}\sum_{\mathbf{G}\neq 0} A_{fg}(\mathbf{G})\,F(|\mathbf{G}|)\,B_k(|\mathbf{G}|) \;+\; (e_{fg}-c_{fg})\langle F\rangle_k\right] \tag{11.22}$$

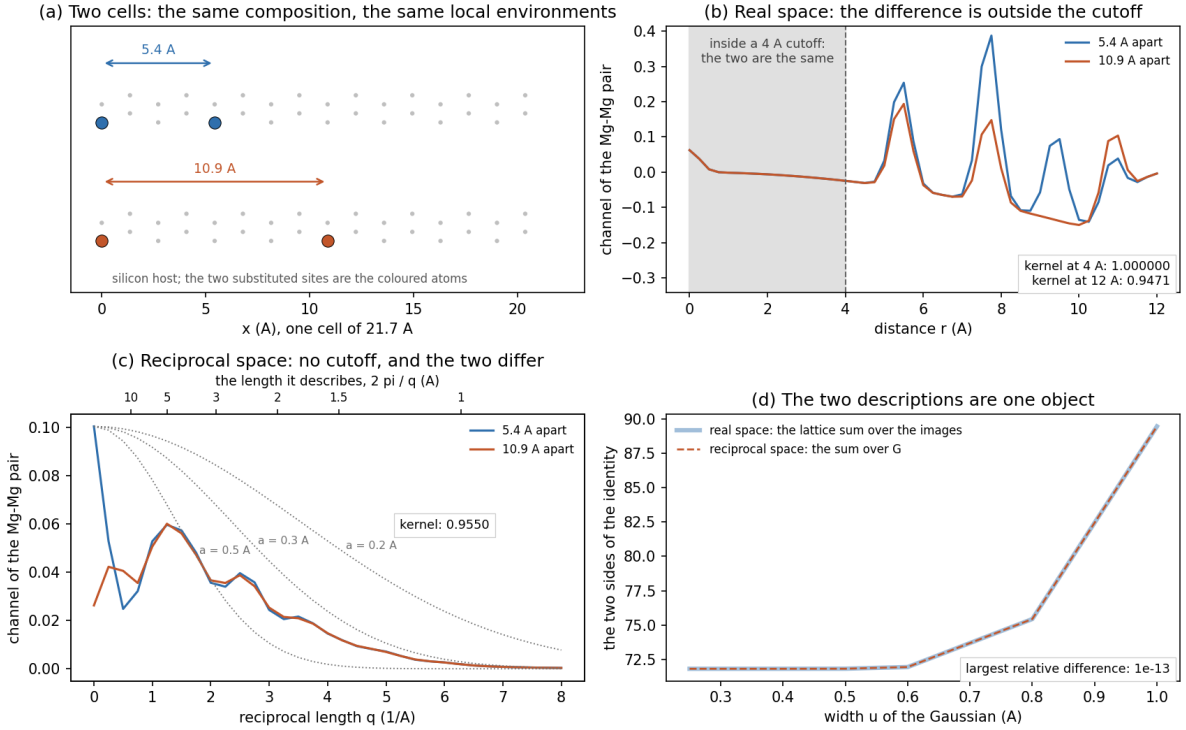
with $A_{fg} = |S_f|^2$ for one field type and 2 Re [S_f overline{S_g}] for two, which counts a pair of different types in both orders as the real-space descriptor does; $B_k(q) = \omega(q)\exp(-(q-q_k)^2/2\sigma^2)$ the channel with the same tapered window as the real-space cutoff; $I_k = \int 4\pi\, q^2\, B_k(q)\, dq$ its shell integral, which is how many reciprocal-lattice points the channel is expected to hold; V the volume of the cell and N its atoms. The term outside the sum is the part of A_{fg} that comes from an atom with itself, $c_{fg} = \Sigma_l\, w_{lf}\, w_{lg}$ from the weights and e_{fg} the same sum of the second moments of a disordered site. The two are equal for an ordered structure, so that term vanishes there; where they differ it is the correction partial occupancy asks for. Neither is summed over the lattice: a quantity that does not depend on **G** contributes $\langle\, F\, \rangle_k$, the mean of the form factor over the channel, and taking it from that integral instead of from the grid is what makes the descriptor independent of the choice of cell **exactly** -- a cell with few atoms holds few reciprocal-lattice points per channel, and a sum would estimate a constant differently in a supercell of the same structure.

The width of the field is a resolution. $F(q) = \exp(-a^2\, q^2)$ is the form factor: the two fields are Gaussians of width a, so their product in reciprocal space carries this factor. It falls with q, so a decides how much fine detail reaches the descriptor -- a wide field sees only the first few reflections. This is the same width the real-space descriptor uses along the distance axis, read in reciprocal space, and it is a resolution and not a cutoff: every atom of the cell enters every channel whatever the distances in the structure. A channel at q describes the structure on the scale $2\pi/q$, which is how these channels are compared with a cutoff radius.

Two things the real-space form has only as approximations. The uniform background is the **G** = 0 term, $A_{fg}(0) = (\Sigma_l\, w_{lf})(\Sigma_l\, w_{lg})$, the forward scattering: here it is one term of the sum, left out, with no integral to approximate. And the two descriptions are one object. Summing the field in real space and summing it over the reciprocal lattice give the same number -- Poisson summation, the identity an Ewald sum is built on -- so for any width u,

$$\sum_{l,m}\sum_{\mathbf{T}} w_{lf} w_{mg} \exp\left(-\frac{|\mathbf{r}_l - \mathbf{r}_m - \mathbf{T}|^2}{2u^2}\right) = \frac{(2\pi u^2)^{3/2}}{V} \sum_{\mathbf{G}} A_{fg}(\mathbf{G}) \exp\left(-\frac{u^2 |\mathbf{G}|^2}{2}\right) \quad (11.23)$$

`parseval_residual` computes both sides. The left side is this program's neighbour list and the right side is the sum over the reciprocal lattice, so the identity tests the two pieces of code against each other, and it is the statement that the reciprocal channels are the same descriptor at a different resolution rather than a second one.



What a reciprocal-space channel adds. (a) Two cells of a silicon host with two substituted sites, 5.4 angstrom apart in one and 10.9 in the other: the same composition and the same local environment around every atom. (b) The real-space channel of the substituted pair. Everything that separates the two structures lies outside a 4 angstrom cutoff, so the descriptor with that cutoff gives them a kernel of 1.000000 -- for it they are one structure -- while a 12 angstrom cutoff reaches the first peak and gives 0.947. (c) The reciprocal channels of the same pair, which have no cutoff: the two profiles differ from the lowest reciprocal length upwards, and the kernel is 0.955. The dotted curves are the form factor for three widths of the field, which is the resolution; the second axis gives the length 2 pi / q that each channel describes. (d) The identity of equation (11.23): the lattice sum over the images and the sum over the reciprocal lattice, as the width of the Gaussian is varied, agree to one part in 1e13. Drawn by `benchmarks/run_reciprocal.py`*.*

Using them. `reciprocal_feature_matrix` builds the channels of a set and `CMFModel(kernel_source="reciprocal")` fits them with the machinery the periodic autocorrelation already uses -- the per-block scaling, the cosine normalisation, the learners, the file format. `joint_features` puts the two descriptions side by side in one matrix, and `reciprocal_pattern` reads a fitted model the way the co-pattern reads the real-space one: which pair of field types at which reciprocal length, that is, at which spacing of planes.

What it measured. Three Matbench tasks, 1,200 structures of each, the five published folds, the setting of the descriptor and the ridge chosen inside every training fold (`benchmarks/run_reciprocal.py`; the table is `benchmarks/results/reciprocal/table.md`). The reciprocal channels were measured alone and added to the real-space descriptor as a second kernel, `K = (1-t) K_real + t K_reciprocal`, with the weight *t* chosen in the training fold. Three things came out of it.

They carry as much as the real-space descriptor, with no neighbour list. Alone they reach R^2 +0.518 on the formation energy of the perovskites against +0.528 for the best real-space cutoff, +0.778 against +0.773 on the bulk modulus and +0.728 against +0.735 on the shear modulus. All three differences have an interval that contains zero. A descriptor built only from sums over the atoms of one cell is level with a tuned pair-distance descriptor, which is the first thing worth knowing about it.

Added to the real-space descriptor they gain on one task of the three. Against the best real-space cutoff: +0.066 [+0.045, +0.090] on the formation energy, +0.015 [+0.002, +0.027] on the shear modulus and +0.008 [-0.005, +0.022] on the bulk modulus, the intervals from 2,000 bootstrap resamples of the structures. So the gain is clear on one set, small on the second and absent on the third. The mixing weight the folds chose is 0.25 to 0.55, so the training sets do put real weight there.

Reaching further in real space costs accuracy; reaching further in reciprocal space does not. On both moduli, taking the real-space cutoff from 4 to 10 Å *loses* about 0.10 of R^2 (+0.773 to +0.671 and +0.735 to +0.635): the longer sum adds many weakly informative distance channels that dilute the kernel. Adding the reciprocal channels to that 10 Å descriptor gives back +0.102 [+0.052, +0.172] and +0.076 [+0.051, +0.103]. Long-range information is worth having, and this is the cheap way to carry it: the number of reciprocal channels does not grow with the range at all.

The width of the field is the resolution, and the cutoff in q is not a parameter. On the bulk modulus the accuracy of the reciprocal channels falls steadily as the field widens, R^2 +0.785 at a = 0.15 Å down to +0.587 at 0.7 Å, and changes by at most 0.002 when the sum over **G** is taken to 12 $Å^{-1}$ instead of 8. Once the form factor has decided what is visible, summing further changes nothing, which is what the word resolution means here.

The identity holds on real structures. Over 120 crystals of the three tasks the two sides of equation (11.23) agree to about one part in 10^{13}, worst case 2×10^{-11}.

What a model reads there. On the bulk modulus the channels that move the prediction most sit at q = 2 to 3.5 $Å^{-1}$, which is a spacing of 1.8 to 3.1 Å -- the nearest-neighbour scale that sets an elastic constant -- with the electronegativity and inverse-square-root-mass fields and their products carrying them. Neighbouring channels are strongly correlated, so single weights alternate in sign and it is the envelope over q that carries the reading; `benchmarks/results/reciprocal/pattern.png` draws it.

The limits. The channels need a structure periodic in all three directions; a layer or a chain has a reciprocal lattice of lower dimension, and the intensity then depends on the direction across the vacuum, which these channels do not resolve. There is no site split of them (a structure factor is a sum over the whole cell, so the intensity cannot be divided among the sites the way the pair sum can), and a site record is refused for that reason. And the intensity of a reflection is the same for a structure and its mirror image, so these channels share the one blindness of every two-point descriptor.

11.11 A measured curve as the sample: the untyped descriptor

A diffraction experiment on a powder, a glass or a nanoparticle gives one curve and no coordinates. That curve is not a different object from the descriptors above. It is the **untyped** one (plan item M12, `cmfqsar/scattering.py`).

Every descriptor so far is *typed*: one channel block per pair of field types. An experiment cannot separate the types, because every atom scatters into the same detector. What it measures is the single block of one field whose weight is the atom's **scattering length** -- the neutron value b, or for X-rays the atomic number Z. Writing ρ_0 for the number density and $\langle b \rangle$ for the mean scattering length, the reduced pair distribution function is

$$G(r) = \frac{\Phi_{bb}(r)}{\sigma\sqrt{2\pi}\, r \langle b \rangle^2} \tag{11.24}$$

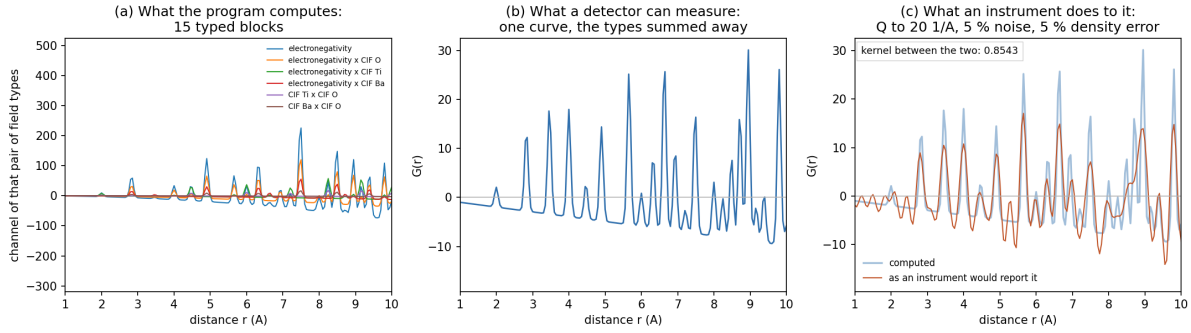
where Φ_{bb} is the periodic autocorrelation of that one field with the uniform background kept and the self term dropped. Every piece of the standard definition is already there: the background term of the descriptor *is* the $-4\pi\ r\ \rho_0$ that the definition subtracts, and $\sigma\sqrt{(2\pi)}$ is the integral of the channel's Gaussian. The identity is exact as the channel narrows, and `pair_distribution_residual` measures the difference against a direct sum over the neighbour list.

The same holds in reciprocal space, which is the other thing an experiment reports. The Faber-Ziman total structure factor is the one-field channel of the previous section divided by $\langle\, b \,\rangle^2$, and $F(Q) = Q\,[S(Q) - 1]$ and $G(r)$ are a sine transform apart. The *distinct* scattering is what enters: the term an atom gives with itself is taken out, so that S approaches 1 and F approaches zero at large Q. Leaving it in would add a straight line $(\langle\, b^2 \,\rangle / \langle\, b \,\rangle^2 - 1)\,Q$ to the curve of every structure with more than one element.

What this gains. A sample where there is no structure. `read_curve` reads the two-column text the common measurement programs write, `resample` puts it on the grid a model uses, and `curve_kernel` compares it with computed curves -- so a model fitted on structures can be asked about a measurement, and materials whose atomic structure nobody knows become describable.

What it costs: the typing. That cost is a question about data and not about code, and `benchmarks/run_scattering.py` measures it on the three Matbench tasks of the previous section, with the same folds.

What a measurement is not. A computed curve is sharper and cleaner than any measurement. Four things separate them, and `measured_like` applies all four so that a model can be tested against them before any experiment is trusted: a finite range in Q, which puts ripples of period $2\pi/Q_{\max}$ around every peak; counting noise on $F(Q)$; an error in the number density, which adds a straight line to $G(r)$ because ρ_0 is what the baseline is drawn from; and a limited range in r.



What a detector sums away. One cell of cubic BaTiO3, five atoms, described three ways by `benchmarks/run_scattering.py --stage concept`*. (a) What the program computes: 15 typed channel blocks, one per pair of field types, of which the six with the largest values are drawn. (b) What a detector can measure: the reduced pair distribution function G(r), one curve, with the types summed away by the weighting that scattering applies. (c) The same curve as an instrument would report it, from a transform cut at Q = 20 angstrom^-1, with 5 % counting noise and the number density wrong by 5 %. The kernel between the computed curve and the measured one is 0.8543: the gap in (c) is what the measurement section below measures the cost of.*

What the measurement says. `benchmarks/run_scattering.py` ran the three Matbench tasks of the previous section, 600 structures of each, with the same five published folds; the numbers are in `benchmarks/results/scattering/table.md`. It reports five things.

The identity holds to the accuracy the derivation promises. The curve from the descriptor and the curve summed over the neighbour list differ by 8.2e-05 of the curve at a channel width of 0.05 Å (median over 30 structures of each task; worst case 3.2e-04) and by 5.5e-06 at 0.02 Å. The

difference falls with the channel width, as it must.

Summing away the types costs most of the accuracy. The typed descriptor reaches R^2 +0.771 on the bulk modulus, +0.684 on the shear modulus and +0.418 on the perovskite formation energies. The one X-ray curve of the same structures reaches +0.303, +0.192 and +0.012: a difference of -0.469 [-0.560, -0.389], -0.492 [-0.579, -0.414] and -0.406 [-0.498, -0.311], with the interval from 2,000 bootstrap resamples of the structures. On the formation energies the curve alone is no better than predicting the mean. The neutron curve is worse than the X-ray one on the structures its table covers, +0.093, +0.086 and +0.101 (78 % of the elasticity sets and 96.5 % of the perovskites have a tabulated scattering length for every element). The likely reason is that the X-ray weight, the atomic number, orders the elements, while the neutron scattering lengths of neighbouring elements are irregular and change sign, so one weighted sum separates the structures less well. Both curves were offered at two ranges, 10 and 20 Å, and all 30 folds chose the shorter one.

A computed curve is too sharp to compare, and what an instrument does to it is what makes the kernel work. As computed, with a channel 0.05 Å wide, the 20 Å curve predicts nothing: R^2 +0.003 on the bulk modulus, -0.050 on the shear modulus and -0.001 on the formation energies. Cutting the transform at $Q_{\max}$ = 20 Å^{-1}, which broadens every peak by about $\pi/Q_{\max}$ = 0.16 Å, raises the same curve to +0.327, +0.235 and +0.030, and at $Q_{\max}$ = 10 Å^{-1} to +0.432, +0.344 and +0.050. Two Gaussians 0.05 Å wide and 0.2 Å apart barely overlap, so a kernel over sharp curves sees two structures as unrelated however their peaks moved; the resolution of the instrument is a width the comparison needs.

Of the other instrument effects, counting noise is the one that matters. Noise of 2 % leaves the bulk modulus at +0.324 against +0.327 without it, and 10 % cuts it to +0.150 and the shear modulus to +0.040. An error of 10 % in the number density costs nothing measurable (+0.336), because it adds a straight line that the kernel largely ignores. Dropping the curve below 2 Å costs a little (+0.260), and dropping it above 10 Å gains (+0.438), which is the same reading as the fit stage's choice of the shorter range.

A model must be fitted at the resolution of the measurement. Fitting on the computed curves and then predicting the degraded ones -- what someone with a structure database and one measurement in hand would do -- never reaches R^2 +0.06 in any of the 36 settings tried and is negative in 15 of them, against +0.327 for the same curves when the model is fitted on curves degraded alike. The two are not the same object: one is sharp and the other is broad. The remedy is `measured_like`: put the computed curves through the instrument's $Q_{\max}$ and noise first, and fit on those.

A curve written to a file finds its structure again. 25 curves of the bulk modulus set, written in the two-column form with a header, degraded as above and read back, all match the structure they came from best; the kernel with their own structure is 0.891 and with the others 0.001.

The limits. No experimental data is used in any of this and none is implied: there is no public set of measured curves with measured properties to fit, so what the measurement reports is how good an instrument would have to be, not that an instrument worked. A disordered cell is refused, because the mean over the species is right for the pair term and not for the self term that the total curve keeps. The X-ray weight is the atomic number, the $Q \to 0$ limit of the form factor, rather than the form factor itself. And a neutron scattering length is taken from a table that leaves out the elements whose value depends strongly on the wavelength; a structure containing one of those is refused rather than guessed at.

11.12 A microstructure as the sample: the field on a voxel grid

A micrograph has no atoms. A two-phase microstructure on a voxel grid is nevertheless a field -- the indicator of each phase, one value per voxel, is one channel -- and the kernel of this program, the overlap integral of two fields, applies to it as it stands. What changes is only how the field is made: not from atom-centred Gaussians but from the voxels themselves, with the fast Fourier transform doing the overlaps on the periodic grid. This is the way into the hierarchical materials informatics of the Materials Knowledge Systems (MKS) of Kalidindi and co-workers, whose descriptor is the **two-point statistics** of the microstructure: the probability that two voxels a vector *r* apart are in phases *a* and *b*. That statistic is the translation-averaged self-overlap of the phase field with its own translate, which is to say the periodic autocorrelation of the chapter above with voxels in place of atoms. `benchmarks/run_microstructure.py` measures what the field kernels are worth against that workflow, on the smallest fair benchmark.

The data. 400 synthetic two-phase cells on a periodic 51 x 51 grid (and 400 on a 21^3 grid), by the recipe of the `pymks` generator: Gaussian-filtered white noise cut at the quantile that gives the phase fraction. The fraction, the grain size, the elongation of the grains and their direction all vary from sample to sample. The label is the effective thermal conductivity of the cell from a finite-volume solver written in the script (`pymks` does not build on Python 3.13, and its elastic solver needs a finite-element package); phase 0 conducts 1, phase 1 conducts 10 or 100. Two labels: the component along *x*, which depends on the direction of the grains, and the mean over the directions, which does not.

The descriptions. The plain kernel of this program -- the overlap of the smoothed phase fields at zero shift, in the frame the samples were generated in; the same averaged over the point group of the grid; the overlap of two samples' two-point autocorrelations, which is the invariant route and is also the overlap of their power spectra, raw and in the cosine form of the autocorrelation kernel, averaged over the point group, and binned by the length of *r*; and the MKS baseline, two-point statistics, principal components fitted on the training part and a polynomial of the scores. Every kernel is tried linear and under the Gaussian layer, with the Gaussian smoothing width of the field (0 to 5 voxels), the layer and the ridge chosen by cross-validation inside the training part. Training sets of 20, 50, 100 and 200 samples are drawn five times each and the rest of the set is held out.

What it says (`benchmarks/results/microstructure/table.md`, with the notes beside it). On the direction-dependent label at contrast 10 the overlap of the autocorrelations reaches q2 0.89 at 20 training samples, 0.97 at 100 and 0.98 at 200; the MKS baseline 0.93, 0.965 and 0.97; at contrast 100 the two are 0.81 against 0.81 at 20 samples and 0.90 against 0.88 at 200; on the 3D cells 0.97 against 0.96 at 20 and 0.989 against 0.987 at 200. So the field-overlap kernel on the two-point statistics matches the baseline at 20 to 50 samples and is a little better from 100 on, and the gain holds where the contrast makes the arrangement matter. The zero-shift overlap in the generating frame is poor, and should be: these samples share a grid and nothing else, so it compares two structures at a random relative shift, and what it keeps as the smoothing grows is the phase fractions. A rotation average loses the direction of the grains and with it the direction-dependent label, and recovers everything on the direction-free one. The phase fraction alone explains 0.87 to 0.98 of the variance, as it does in the published homogenisation examples, which is why the arrangement is a second-order signal here.

The length scale. The smoothing width is the knob a multi-scale kernel would turn. The inner cross-validation sets it to zero for the autocorrelation kernels on the direction-dependent label -- the statistic wants its full resolution -- and to the widest width tried, 5 voxels, close to the grain size, for the zero-shift overlap and for the direction-free label, where the kernel then sees little beyond the fractions. A hierarchical model would therefore carry the fine width for the arrangement and the coarse one for the composition, which is what two field types with their own widths already do in this program.

The gate: the current baseline, a real micrograph set, two scales, and the map. The pilot's margin over polynomial ridge regression on principal components was small and that baseline is old, so four more tests were run, reported in the notes beside the table. *First*, every method was given the same budget of 40 settings, twenty draws per training size, and the current MKS baseline was added: a Gaussian process on the principal-component scores with one length scale per score, fitted by the marginal likelihood; and the label of the published MKS benchmark, the effective elastic stiffness from the finite-element solver of PyMKS, was computed beside the conductivity (PyMKS needed three patches to run on a current Python and `sfepy`; `benchmarks/prepare_elastic.py` lists them). The two labels correlate at 0.98 to 0.995, so the pilot's conductivity was a fair stand-in. Read this way, the overlap of the two-point autocorrelations and the Gaussian process are the same model to within the noise of the draws from 100 samples on, and below 100 samples the kernel is the steadier of the two, ahead in 14 to 17 of 20 draws by 0.02 to 0.15 in q2, where the GP's fit of twenty length scales on 16 to 40 points is the unstable step. *Second*, a real micrograph set, the 961 ultrahigh-carbon steel micrographs of DeCost and co-workers (2017) with seven microconstituent classes and the heat treatment of 598 of them: here the autocorrelation kernel of the grey level, computed with zero padding because a micrograph is one window on a medium that does not repeat, is the best of the second-order descriptions (accuracy 0.61 at 500 training images against 0.55 for the polynomial and 0.51 for the GP on the principal components) and is far behind the published route, features of a convolutional network with a linear model on top (0.87), in 0 of 20 draws ahead at any training size, because the classes are named after shapes that a second-order statistic does not carry; and when whole specimens are held out, every method loses 0.1 to 0.2 in accuracy and none predicts the treatment of a specimen it never saw. *Third*, a property that depends on two scales: phase 1 was given a defect concentration that sets its conductivity by a known law, and the cells were solved again. Each scale alone explains a third of the variance; the sum of an arrangement kernel and a descriptor kernel reaches 0.74; their product reaches 0.97, ahead of the MKS baseline with the descriptor appended in 19 or 20 of 20 draws. The kernel across scales is the product, because an effective property is close to the property of the conducting phase times a factor of the arrangement -- as two molecular fields that act together, not in turn. A second width of the same arrangement adds nothing. *Fourth*, the interpretation map: for the linear autocorrelation kernel the model is a linear function of the two-point statistics and its coefficient field over the lag vector is read straight from the dual coefficients (`benchmarks/results/microstructure/map_2d51_n400_c100.png`). For the conductivity along x it is a four-lobed figure that credits phase 1 found again 2 to 8 voxels away along the gradient, peaking at 3 to 4 voxels, and debits the same lags across it; for the direction-free mean it is a ring that credits every direction at the grain size, 5 voxels, and nothing larger. A principal-component regression has no such map, because its coefficients do not live on lags. The conclusion of the whole is scope rather than victory: a method built for molecules reaches microstructure informatics through its invariant route, lands on the established statistic with the same accuracy as the current baseline, is steadier below 100 samples, combines scales by a product, carries a continuous interpretation map, and shares the limit of that statistic on real micrographs, where a trained network sees more.

11.13 Three atoms at a time: what a histogram of distances cannot see

Every descriptor of a crystal above is a **two-point** one. It counts how much field weight sits at each distance from how much other field weight, and that is all it knows. Two limitations follow, and both belong to the description and not to the data (plan item M10, `cmfqsar/threebody.py`).

The first is that two structures can have exactly the same multiset of distances and not be the same structure. Such a pair is called **homometric**. It is not a curiosity of mathematics: a defect series on

one lattice holds many arrangements whose distance histograms are equal or very nearly so, and the two-point kernel then gives them a kernel of one and the model cannot tell them apart. The second is that a mirror image has the same distances as the structure it reflects, so no two-point descriptor can tell a left-handed crystal from a right-handed one.

The three-body block. Take every atom of the cell as a centre in turn. Two neighbours and the centre form a triangle, and a triangle is fixed by its three sides: the two distances from the centre and the distance between the two neighbours. Writing r_a for the distance of neighbour a from the centre, d_{ab} for the distance between two neighbours and N for the atoms in the cell,

$$\Phi_{fg}(k,k',m) = \frac{1}{N}\sum_{l}\sum_{a,b\in\mathcal{N}(l)} w_{af}\,w_{bg}\,R_k(r_a)\,R_{k'}(r_b)\,D_m(d_{ab}) \tag{11.25}$$

with R_k a Gaussian window on a grid of radii from the centre and D_m one on a grid of distances between neighbours. The neighbourhood N(l) holds the centre itself, at radius zero, so its own field types enter the channels, and every atom of every periodic image within the cutoff. Dividing by N makes a supercell give exactly the same channels as the cell it repeats, as it does for the two-point descriptor. The channel layout is the one of the triangle block that `cmfqsar/triangles.py` builds around one atom of a molecule: one block per pair of field types, and inside a block one column per pair of radial windows and distance window.

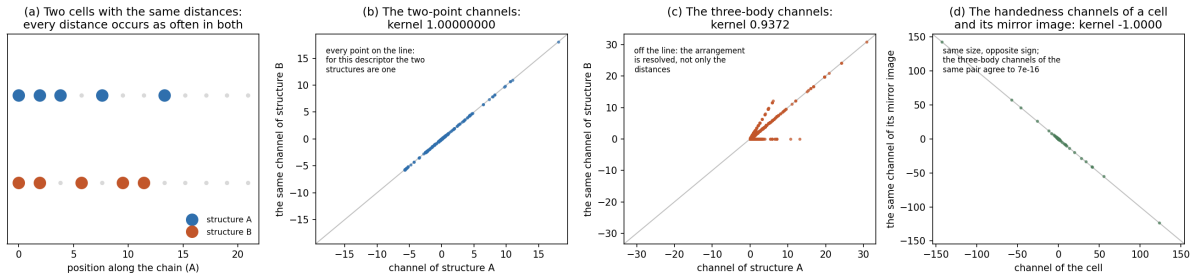
The handedness block. Three distances fix a triangle only up to its mirror image, so the block above is still blind to a reflection. The signed quantity that is not is the triple product of three vectors. About each centre,

$$X_{fgh}(s) = \frac{1}{N}\sum_{l}\det[\,A_f(l,s),\,A_g(l,s),\,A_h(l,s)\,], \qquad A_f(l,s) = \sum_{a\in\mathcal{N}(l)} w_{af}\,G_s(r_a)\,(\mathbf{r}_a-\mathbf{r}_l) \tag{11.26}$$

with G_s a Gaussian window on a grid of radii, which keeps the handedness resolved by distance from the centre instead of reduced to one number for the whole structure. Under a proper rotation all three vectors turn together and the determinant does not change; under a reflection it changes sign; a centrosymmetric neighbourhood gives zero, as an achiral environment must.

The cost, and a correction to what the plan expected. The development plan assumed this block would need a sum over triples of atoms, because the factorisation that the molecular chirality block uses needs an origin and a periodic structure has no centroid to use. With the centre atom as that origin the factorisation survives in full: a determinant is linear in each of its three arguments, so the sum over triples of neighbours **is** the determinant of three summed vectors. Both blocks therefore cost one pass over the pairs of neighbours of each atom, and nothing cubic is summed.

When the handedness block is empty, which is a property of the block and not a fault. A determinant of three vectors is zero whenever they lie in a plane. A field weight that depends only on the species of the atom makes every A_f a combination of one vector per species, so a structure of two species gives vectors in a plane at most and every channel is zero; three species are the minimum. A site with a rotation axis through it also gives zero, because a sum that the rotation leaves unchanged can only lie along the axis. The block is a sharp test for handedness rather than a general descriptor, and it is off by default.



The two blind spots of a two-point descriptor, drawn by `benchmarks/run_threebody.py --stage concept`*. (a) Two cells of five atoms on a ring of twelve sites, at 0, 1, 2, 4, 7 and at 0, 1, 3, 5, 6: every distance occurs as often in one as in the other. (b) Every two-point channel of structure A against the same channel of structure B. Every point lies on the line, so for that descriptor the two cells are one structure: kernel 1.00000000. (c) The same plot for the three-body channels. The points leave the line and the kernel falls to 0.9372, so the arrangement is resolved and not only the distances. (d) The handedness channels of a cell of four species with no symmetry against those of its mirror image: the same size and the opposite sign, kernel -1.000000, while the three-body channels of that same pair agree to 6.9e-16 of the largest channel.*

What the measurement says. `benchmarks/run_threebody.py` ran three stages: the two blind spots on constructed structures, three Matbench tasks of 600 structures each with the same five published folds as the sections above, and the defect series of plan item M14. The numbers are in `benchmarks/results/threebody/table.md`.

Both blind spots are real, and the two blocks close them. Two cells of five atoms on a ring of twelve sites, at positions 0, 1, 2, 4, 7 and at 0, 1, 3, 5, 6, have the same multiset of distances. The two-point channels give them a kernel of 1.00000000: for that description they are one structure. The three-body channels give 0.9372. A cell of four species with no symmetry and its mirror image agree in the three-body channels to 6.9e-16 of the largest channel, as every description built from distances must, while their handedness channels have the same size and the opposite sign (kernel -1.000000). Rocksalt, which is centrosymmetric, gives 2.5e-45 in every handedness channel.

On a property, the triangles gain where an angle is the signal. The three Matbench tasks were run with the two descriptions alone and with their kernels mixed as $K = (1-t)\ K_{two-point} + t\ K_{three-body}$, with t chosen inside every training fold. R^2 over the five folds:

Descriptor	formation energy	bulk modulus	shear modulus
the two-point descriptor	+0.453	+0.781	+0.699
the three-body channels	+0.542	+0.789	+0.727
the two kernels mixed	+0.515	+0.799	+0.721

The perovskite set is compounds of one composition type whose cells differ mainly by how the octahedra tilt, which is an angle and not a distance. There the three-body channels gain +0.089, with a 95 % interval of [+0.041, +0.140] from 2,000 bootstrap resamples of the structures. On the two elastic moduli the gains are +0.009 [-0.022, +0.041] and +0.028 [-0.003, +0.065]: both intervals include zero, so on those tasks the two descriptions are level. The mixture is the best of the three on the bulk modulus, but its gain there is +0.018 [-0.006, +0.044], which includes zero as well, and on the formation energies the mixture is worse than the three-body channels alone. Mixing adds a setting to choose and, on these three tasks, no accuracy.

On a set where the arrangement is the whole signal, the triangles do what a two-point description cannot. The 2DMD defect series (plan item M14) is one host lattice and one supercell with one to three point defects placed differently in every cell. Two splits were run over 2,000 cells of each of

two hosts, with the same folds, the same ridge grid and the same Gaussian layers as the two-point rows. Energies are given as the mean absolute error per site of the 191-atom cell.

The set	Descriptor	random folds: R^2	meV per site	one and two defects to three: R^2	meV per site
MoS2	two-point, 10 Å	+0.9997	8.3	-0.015	1222
MoS2	overlap of M14, 10 Å	+1.0000	1.8	+1.000	4
MoS2	three-body, 4 to 5 Å	+0.9972	36.0	+0.940	240
MoS2	the two kernels mixed	+0.9988	25.6	+0.021	1201
WSe2	two-point, 10 Å	+0.9989	15.7	+0.060	1096
WSe2	overlap of M14, 10 Å	+0.9998	5.2	+0.996	54
WSe2	three-body, 4 to 5 Å	+0.9861	38.3	+0.954	198
WSe2	the two kernels mixed	+0.9977	23.8	+0.091	1078

On random folds, where the training set holds arrangements much like the tested one, the two-point description is the better of the two, because it reaches 10 Å and the three-body channels only four to five. The second split is the one the item was built for: it trains on the 131 cells with one or two defects and predicts the cells with three, so every tested arrangement is new. There the two-point description fails -- R^2 -0.015 on MoS2, an error of 1,222 meV per site, which is no better than predicting the mean -- and the three-body channels hold, at +0.940 and 240 meV per site. The overlap kernel of M14 is better still, but it needs every cell to share one lattice and one supercell, which is what it uses to put two cells in a common frame; the three-body channels assume nothing about the set and work on structures that have nothing in common.

A mixed kernel does not inherit what the three-body one can do. The mixture lies between the two descriptions on random folds and fails with the two-point one on the extrapolation split, at R^2 +0.021 and +0.091. Its weight is chosen inside the training fold: on the random folds every fold gave the three-body kernel a quarter of the weight, and on the extrapolation split the choice fell on three quarters, so the mixture failed even where the three-body kernel carried most of it. The likely reason is that the two-point kernel is close to one for every pair of these cells, which share a lattice and differ in a few sites, so what it adds to the mixture is nearly constant and compresses the differences that the three-body part supplies. A second reading is about model selection and not about this descriptor: the weight and the kernel layer were chosen on a random inner split, where both descriptions look nearly perfect, so the choice could not see the failure that the outer split tests for. A model that is meant to extrapolate needs an inner split that extrapolates too.

The cost. A cell of 191 atoms takes 0.26 s for the 14,112 channels of the first setting, so the 2,000 cells of one host take about nine minutes, and their feature matrix is 0.11 GB; the second setting has 29,120 channels and 0.23 GB. The whole three-body row of one host -- the features of both settings, then twelve kernels over five folds and the second split -- took 35 minutes, and the mixed

row eleven.

The limits. A disordered cell is refused: a three-body channel is a product of three weights, and the mean of that product over the species sharing a site is not the product of their means, which is all the module has. No uniform background is subtracted, unlike the two-point descriptor, so these channels carry how dense the structure is as well as how it is arranged; a cosine-normalised kernel removes most of that, and the two-point channels are there to carry the density properly. The number of channels grows as the square of (fields times radii), so the radial grid is coarse and the cutoff short -- four to five angstrom, against the ten a two-point descriptor reaches cheaply. The two are complements and not alternatives, which is what the measurement says as well.

11.14 Designing a crystal: a prototype, a budget and a loop

Every section above asks how well a model predicts. A design study asks the other question: with a budget of calculations, can the model find a better structure than the ones it was fitted on? For molecules that question has the sections on which compound to make next; for crystals it needs a way to propose candidates, because there is no parent molecule to substitute on (plan item M11, `cmfqsar/prototypes.py`).

A prototype is a cell with site classes. One cell, its sites grouped into classes, and a list of the elements each class may take. Two modes cover what a materials study asks for. In *composition* mode every site of a class takes the same element, which is the ABO3 case: one prototype, many compositions. In *arrangement* mode a fixed number of the sites of a class take a substituent and the rest keep what the prototype has, which is the point-defect case: one composition, many arrangements. The element name `vac` removes the atom instead of replacing it, so a vacancy series is an arrangement series.

The number of candidates is a product of the choices and grows quickly, so nothing is built until it is asked for. `Prototype.n_candidates` counts them, `candidate_at` builds the one with a given index, and `candidates` either walks the enumeration or draws a sample from it without replacement. A pool of a few thousand can therefore be taken from an enumeration of millions without building the candidates that are left out.

Two candidates can be one structure. Any symmetry operation of the prototype that maps the substituted sites onto each other gives the same structure a second index. `site_permutations` reads those operations from spglib and turns them into permutations of the substitutable sites, and `unique` keeps one candidate per orbit. The other test is the descriptor itself: `channel_keys` gives each candidate the key of its channels, and two candidates with the same key are one structure *for the model*, whatever their coordinates say. The two tests do not agree, and the measurement below says what that costs.

An oracle is what computes the property. A design loop has to evaluate what it proposes. `LookupOracle` reads the value from a table, which is how the loop is measured on a published set, and it counts the calls, because a budget is counted in calls. `CalculatorOracle` runs any ASE calculator and relaxes the cell first if asked; the same interface takes a machine-learned potential or a VASP calculator, and what changes is the cost of a call, not the loop.

The loop. `design_loop` is given a pool, an oracle, a start set and a budget. At every step it fits the kernel model on what the oracle has answered so far, chooses the ridge by the evidence of plan item B4, scores every candidate that is left by one of the rules of `cmfqsar/active.py`, and asks the oracle about the best-scoring one. It never looks at a value it has not asked for and never asks twice about the same candidate, so the budget it reports is the budget a study would pay. A formation energy is to be made small and every rule in that module prefers a large score, so

`maximise=False` negates the target inside the loop; with that, `ucb` reads as the lower confidence bound of the section on which molecule to make next -- a candidate is proposed only when the model is both optimistic and sure.

How large a design space is, and how much of it is already computed. The five-atom cubic ABO3 prototype with the 52 A elements and 52 B elements of `matbench_perovskites` allows 2,704 candidates, and that set already holds a computed formation energy for 2,661 of them: 98.4 % of the space, with 43 candidates left. One more site class changes that. The doubled cell with its two B sites free of each other (A2BB'O6) allows 140,608 candidates, and the repository holds a computed formation energy for none of them. Counting the space is therefore the first step of a design study, and it is what tells a materials project whether the generator is needed at all.

Which candidates are one structure. On the 16 sites of an fcc supercell, one substituted site gives 16 placements and 1 arrangement, two give 120 and 5, and three give 560 and 10. The descriptor's own test is the weaker of the two: of those 10 arrangements, both descriptors give 8 channel sets at every reach tried -- the two-point channels at 6 and at 14 Å, the three-body channels at 5 and at 9 Å. The two merged pairs are not the same structure. ASE's symmetry comparator separates them, their space groups are 25 and 25 with different Wyckoff positions in one pair and 25 and 65 in the other, and an EMT calculation on the unrelaxed cells puts the two energies of each pair 0.6 meV per atom apart. That difference is an error no model built on these channels can avoid, and it is the limit of the whole family: a loop can be told that two candidates are one structure only as far as its channels see.

The gate. The pool is 4,000 cells of `matbench_perovskites`, the target is the formation energy and is to be made small, the oracle reads the tabulated value, and the budget is 100 calls after the start set. Ten start sets are drawn, every rule begins from the same ones, and two start sizes are run, because the model is what ranks the candidates. Two rules meet the gate, and both of them act on the prediction. With the three-body kernel and 100 computed cells the greedy rule reaches the best cell of the 4,000 (median rank 1) and improves on its start set in 7 of the 10 runs -- one of the ten start sets already held the best cell of the pool, so 9 runs could improve at all -- and it enters the best one per cent of the pool in 8 of 10 runs after a median of 8 calls; the lower confidence bound improves on its start set in 7 of 10 runs as well and reaches median rank 9. With 25 computed cells it still reaches median rank 1 with the three-body kernel, and only rank 15 with the two-point kernel: this is the one place in the study where the three-body channels of plan item M10 decide the outcome. Random choice reaches median rank 27 with 125 computed cells and 14 with 200, which is what chance gives (the median best rank of 125 and 200 draws from 4,000 is 22 and 14), and the rules that explore do no better here -- median rank 28 to 82 for `variance` and `maxmin` and 28 to 40 for `ei`, and they improve on their start set in 1 to 4 of the 10 runs against 5 of 10 for random choice.

Where the arrangement is the signal. The second measurement is the question a defect study asks: of 2,000 arrangements of one to three point defects on the 8 x 8 supercell of MoS2 and of WSe2 (the data of plan item M14), which arrangement is the most stable. The kernels are the ones items M14 and M10 already cached, so this stage costs minutes. On WSe2 the loop does what a design study hopes for: `greedy` and the lower confidence bound reach median rank 1 of 2,000 and enter the best one per cent in 10 of 10 runs after 20 to 38 calls, with either descriptor and either start size. On MoS2 the same two rules fail: median rank 10 of 2,000 with 100 computed cells and 37 with 25, and the best one per cent reached in 0 of 10 runs, while random choice reaches median rank 5 and enters the best one per cent in 6 of 10. There the rules that explore are the ones that work: on the three-body kernel with 100 computed cells `ei` reaches median rank 1 and `maxmin` enters the best one per cent in 10 of 10 runs.

A model that predicts well can still be useless for design. The stage that explains this (`--stage ordering`) fits the same model on the same start sets and asks where it puts the best candidate of the pool. On MoS2 the model is accurate: rank correlation +0.87 and R^2 +0.84 on the cells it has not computed, which is far above anything the perovskite pool reaches (+0.53 and +0.31 with 100 computed cells). Yet it puts the best candidate of the pool at place 894 of the 1,900 left out, so a rule that proposes the best-predicted candidate does not come near it. On WSe2 the same model at the same accuracy (+0.93 and +0.88) puts the best candidate at place 59. The difference is in the pools and not in the models. On WSe2 the best one per cent of the arrangements lies within 0.002 eV of the best one, so a rule that lands anywhere in that group has found what the study wants. On MoS2 the best arrangement lies 0.41 eV below the boundary of the best one per cent: it is an isolated candidate, and a kernel ridge model shrinks an isolated value toward the mean along with everything else. Two things follow. The number to report for a design loop is not the accuracy of the model but the place it gives the best candidate, and that place is measured before the loop starts, while the loop refits after every answer -- on the perovskites the best cell starts at place 227 of 3,900 and `greedy` still reaches it, because each answer improves a model that began weak, while on MoS2 the model is already as good as it will get and the 100 further answers do not move the best candidate forward. Three pools are too few for a general rule, but they are enough to show that the accuracy of a model does not say whether a design loop will work.

A closed loop, with a calculator instead of a table. The last stage computes every value inside the loop, so that nothing is read from a table anywhere. Eight of the 32 sites of an fcc copper cell take gold: one composition and 10,518,300 arrangements. A sample of 3,000 placements leaves 2,459 distinct candidates after the symmetry reduction, and 400 of those are the pool. The oracle is an ASE EMT calculator that relaxes the cell in every call, at 0.22 s per call, which is cheap enough to compute the whole pool exhaustively as well (88 s) and make the rank of what the loop found exact. The energies run from +0.0480 to +0.0805 eV per atom, a spread of 33 meV per atom. With 10 computed candidates to start, a budget of 100 calls and five start sets, every rule that uses the model -- `greedy`, `ucb`, `ei`, `variance` and `maxmin` -- finds the best candidate of the 400 in 5 of 5 runs, the greedy rule already at its first call, while random choice reaches median rank 3 and finds the best candidate in 1 of 5 runs. The budget is a quarter of the pool here, so this stage shows that the loop runs from end to end with a calculator and that using the model is better than not using it; it is too easy a search to say by how much. Replacing EMT with a machine-learned potential or with VASP changes the cost of a call and nothing else in the loop, and that is the step this item has not measured.

The limits. The generator substitutes on a fixed cell: it does not relax what it proposes, so the structure the model sees is the prototype's geometry with other elements on it, and the relaxation is the oracle's business. That is how a screening study works, but it means the descriptor of a candidate is not the descriptor of the relaxed structure the oracle returns, and the difference grows as the substituted ions differ more in size. The loop takes one pool and does not grow it, so a space too large to enumerate must be sampled first; a generator that proposes near what the model likes, rather than from a fixed pool, is not built. The gate above is met by the two rules that act on the prediction and by none of the three that only explore, and the defect series shows those same two rules failing on one of two materials, so the rule has to be chosen for the pool and not once for the program. And the loop is only as good as the model that ranks for it, which is the measurement this section ends with.

11.15 An atom instead of a molecule: the local invariant kernel

Every model so far predicts one number per molecule. A carbon shift, an acidity, a local reactivity or a site's energy belongs to an **atom in a molecule**, so the sample has to be the atom and its surroundings (plan item LP2). The section on per-atom targets above holds the targets of that shape (`cmfqsar/atomtargets.py`); `cmfqsar/localenv.py` builds the samples that go with them.

The sample is the field under an envelope. The environment of atom *a* of molecule *A* is the molecule's own field, for every field type *f*, multiplied by a Gaussian envelope centred on that atom:

$$X_{fa}(\mathbf{r}) = X_{fA}(\mathbf{r}) \exp\left(-\rho\,|\mathbf{r}-\mathbf{r}_a|^2\right) \tag{11.27}$$

Here $\mathbf{r}_a$ is the position of the atom and ρ sets the width of the envelope, in $Å^{-2}$. An atom near the centre keeps its weight and one far away loses it. A CMF field is a sum of atom-centred Gaussians and the envelope keeps that form, which is the point: an environment **is** a field of the same class as a molecule's, so the kernels, the regression, the variance and the atom decomposition all apply to it unchanged. The centre is the first atom of the environment, and `member_atom_map` records which atom of the molecule each atom of the environment came from, so a per-atom quantity can be put back where it belongs.

ρ says how far the model looks, and it is a hyper-parameter like α (the option is `rho`): the envelope falls to 1/*e* of its height at a distance 1/√(ρ) from the centre, which `envelope_width` reports. The default ρ = 0.5 $Å^{-2}$ is a width of 1.4 Å, which is a first-shell view; ρ = 0.1 $Å^{-2}$ is 3.2 Å and takes in most of a small molecule. It has to be chosen for the property, and the measurement below shows that it decides everything. Atoms whose envelope factor falls below 1e-4 are left out, which costs nothing measurable and saves carrying them; `kept_share` says what share of the enveloped weight that dropped.

The kernel needs no alignment. The kernel to use on an environment first is the rotation-invariant field autocorrelation of the sections above, taken from the centre instead of over a whole molecule: two environments of two different molecules need not be superimposed for their autocorrelations to be compared. That matters here more than anywhere else, because a common frame for carbons of unrelated molecules does not exist. On an environment the autocorrelation becomes a double sum over atom pairs in which each atom carries its envelope factor, and the distance axis is shorter -- `LOCAL_SPEC` runs to 8 Å in steps of 0.5 Å, because the envelope has already removed everything beyond a few Å.

An acidity is the field of a change. A pKa is the free energy of losing a proton, so the sample for it is not the acid but the **difference field** of the acid and its anion, which `difference_field` builds: a kernel on CMF fields is bilinear, so a difference of two fields is again a field, whose atoms are the atoms of both molecules with the second one's weights negated. That is the signed counterpart of the ensemble builder of the protonation section, which averages members and therefore needs non-negative weights. The two molecules have to be in one frame for the difference to mean anything, which is why the anion is built on the geometry of the acid.

The reference is an atom-centred count. What such a model has to beat is the simplest thing that could work: the type of the centre itself plus the counts of atom types in distance shells around it, which is what a per-atom increment scheme is. `count_features` builds it from the same molecules, the same samples and the same folds, so the only difference between the two numbers is the descriptor.

The folds hold out whole molecules. `AtomTargets.folds` splits molecules, never atoms. Two carbons of one molecule, one in the training set and one in the test set, would let the model see the molecule it is being tested on. For the same reason the bootstrap interval resamples molecules, with every measured atom of a drawn molecule going in with it.

```
cmf atoms examples/local/shifts_cdcl3.sdf --list        # what could be a target here
cmf local examples/local/shifts_cdcl3.sdf --target shift --reference
cmf local examples/local/carboxylic_pka.sdf --target pka --rho 0.1 0.5 --reference \
    --out with_predictions.sdf
```

`cmf local` fits the local kernel on the per-atom target, searches the ridge on the folds that hold out whole molecules, and with `--reference` fits the count model on the same folds and bootstraps the difference over molecules. `--out` writes an SDF carrying the measured value and the out-of-fold prediction on the atom each belongs to.

What it is worth, measured

`benchmarks/run_local.py` measures the two properties the plan names, against the count reference on the same folds. Both sets are prepared by `examples/make_local_examples.py` from free downloads and are in `examples/local/`, with their origin, their licences and every filter in `examples/local/README.md`. The last column but one is the mean kernel value between two different samples: a cosine-normalised kernel of 1 says the descriptor cannot tell two samples apart, whatever the regression does with it afterwards.

13C shifts in chloroform, 5096 assigned carbons in 400 molecules.

descriptor	envelope width rho	q2	error (ppm)	mean kernel between two samples	difference in q2 against the counts
atom-centred counts (18 atom types in 6 shells)	--	+0.853	20.464	0.566	--
local field	0.25	+0.902	16.731	0.462	+0.049 [+0.030, +0.067]
local field	0.5	+0.962	10.404	0.486	+0.109 [+0.097, +0.123]
local field	1	+0.973	8.824	0.537	+0.120 [+0.107, +0.133]
local field	2	+0.969	9.460	0.562	+0.116 [+0.103, +0.130]

The field around a carbon predicts its shift better than a count around it does, and the envelope width decides how much. At a width of 1.0 Å the error is 8.8 ppm against 20.5 ppm for the counts, a difference in q2 of +0.120 [+0.107, +0.133], which is far outside the interval. At the module's default width of 1.4 Å, which was fixed before either set here was measured, it is +0.109 [+0.097, +0.123]. A wider envelope is worse (+0.902 at a width of 2.0 Å) and so is a narrower one (+0.969 at 0.7 Å), so the best view of a carbon is about one angstrom across, which is what a shift should ask for: it is a property of the atom, its bonds and the next shell.

What these numbers are **not** is a shift predictor. Programs built for that task reach one to three ppm on 13C by using tens of thousands of spectra and often a calculated geometry; this model has 400 molecules and one force-field conformation each. The question the item asks is narrower, and

the measurement answers it.

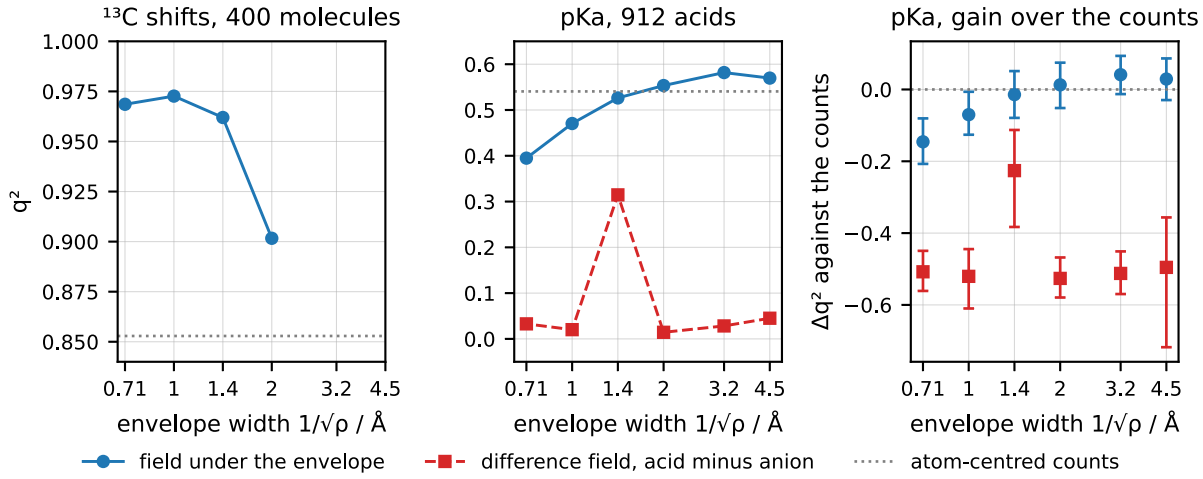
Carboxylic acid pKa, 912 acids with the value on the carboxyl carbon.

descriptor	envelope width rho	q2	error (pKa units)	mean kernel between two samples	difference in q2 against the counts
atom-centred counts (20 atom types in 6 shells)	--	+0.541	0.563	0.577	--
local field	0.05	+0.570	0.545	0.716	+0.029 [-0.030, +0.086]
local difference field	0.05	+0.045	0.811	1.000	-0.495 [-0.718, -0.356]
local field	0.1	+0.582	0.537	0.775	+0.041 [-0.013, +0.094]
local difference field	0.1	+0.028	0.818	1.000	-0.512 [-0.570, -0.451]
local field	0.25	+0.553	0.555	0.877	+0.013 [-0.052, +0.075]
local difference field	0.25	+0.014	0.824	1.000	-0.526 [-0.579, -0.468]
local field	0.5	+0.526	0.571	0.957	-0.014 [-0.079, +0.051]
local difference field	0.5	+0.315	0.687	1.000	-0.226 [-0.383, -0.113]
local field	1	+0.470	0.604	0.990	-0.070 [-0.126, -0.006]
local difference field	1	+0.020	0.822	0.999	-0.520 [-0.610, -0.445]
local field	2	+0.395	0.646	0.997	-0.146 [-0.207, -0.081]
local difference field	2	+0.033	0.816	0.998	-0.508 [-0.561, -0.449]

Here the local kernel is no better than the count, and the difference field, which the plan proposed as the sample, is far worse. The best of the plain environments is a width of 3.2 Å, +0.582 against +0.541 for the counts, a difference of +0.041 [-0.013, +0.094] -- inside the interval,

so by rule 4 it is not a result. The direction the plan expected is right, though: an acidity wants a **wider** envelope than a shift, and a tight one fails: +0.526 at a width of 1.4 Å and +0.395 at 0.7 Å, both below the count.

The mean-kernel column says why, and it is the useful part of this measurement. For the difference field it is 0.998 or more at every width, and 0.9999 at the three widest: **every carboxylic acid loses its proton in nearly the same way**, so the difference of the two fields is nearly the same pattern for all 912 of them, and after the cosine normalisation the samples are indistinguishable. The free energy of deprotonation is indeed a response to that change, but what distinguishes one acid from another is not in the change itself -- it is in what surrounds it, which is exactly what the difference removes. The same column explains the plain environments: at a width of 1.4 Å the mean kernel is already 0.957, because the first shell of a carboxyl group is the same group in every molecule, and the model only starts to see the substituent as the envelope widens.



q² against the envelope width 1/√ρ for the two per-atom targets, with the atom-centred counts as the dotted line. Left: a ¹³C shift wants a narrow envelope of about 1 Å. Middle: an acidity wants a wide one, and the difference field fails at every width. Right: the gain of the pKa models over the counts with its 95 % bootstrap interval; only an interval that excludes zero is a result.

The conformations the molecule visits. A measured shift is an average over the conformations a molecule visits, and so is an acidity; the single geometry that distance geometry returns is a stand-in for that average, and both measurements above named it as their weakest part. Plan item LP4 is the test of what it costs. It needs no new kernel: an environment is a field, so the mean of the per-conformation environments is a field too and the kernel on it is the mean-map kernel (`localenv.ensemble_environment`). What it cannot be is the environment of the mean field of the molecule: the atom sits in a different place in every conformation, so each conformation has its own environment, centred on its own copy of the atom, and it is those that are averaged. With a frame the same order applies -- pose each conformation, then average (`localalign.posed_ensemble`) -- because the site moves with the conformation.

`benchmarks/run_local.py` asks for 5 conformations per molecule from the distance-geometry generator with a force-field refinement, keeps the prepared geometry as the first of them, and measures four samples: that geometry alone, the lowest in energy of the pool alone, and the ensemble weighed equally and by the force field's energies.

On the shifts, averaging costs, and the control says it is the averaging.

conformations	q2	error (ppm)	atoms per sample	difference in q2 against the prepared one
one, as prepared	+0.973	8.824	13	--
one, lowest in energy	+0.972	8.880	13	-0.000 [-0.001, +0.000]
the ensemble, weighed equally	+0.952	11.632	71	-0.020 [-0.024, -0.017]
the ensemble, weighed by energy	+0.956	11.149	71	-0.016 [-0.019, -0.014]

The prepared geometry gives q2 +0.973 and 8.82 ppm. Averaging over the ensemble gives 11.63 ppm weighed equally and 11.15 ppm weighed by energy, which is -0.020 [-0.024, -0.017] and -0.016 [-0.019, -0.014] in q2 -- both outside their intervals, so both are real losses. The second row is the control: a different single conformation, the lowest in energy of the same pool, which differs from the prepared one for 285 of the 400 molecules and costs -0.000 [-0.001, +0.000] -- nothing. So the loss is not that the generated geometries are worse than the prepared one. It is the averaging.

The reading is that the mean field smears every atom over the positions it takes in the conformations. At an envelope width of 1.0 Å what is left inside the envelope is the first shells, and those are the same in every conformation: the averaging adds blur and no information. The atom count per sample says the same thing from the other side -- 13 atoms become 71 and the model gets worse.

On the acidities, it neither helps nor costs.

conformations	q2	error (pKa units)	atoms per sample	difference in q2 against the prepared one
one, as prepared	+0.582	0.537	25	--
one, lowest in energy	+0.576	0.540	25	-0.005 [-0.043, +0.035]
the ensemble, weighed equally	+0.562	0.550	127	-0.020 [-0.061, +0.025]
the ensemble, weighed by energy	+0.549	0.557	127	-0.032 [-0.070, +0.013]

The prepared geometry gives q2 +0.582, the ensemble +0.562 weighed equally and +0.549 weighed by energy, which is -0.020 [-0.061, +0.025] and -0.032 [-0.070, +0.013]. The control costs -0.005 [-0.043, +0.035]. Both ensemble intervals include zero, so by rule 4 this is not a measurable loss; what it is not is a gain. The envelope here is wider, 3.2 Å, and does reach the parts of a molecule that move between conformations, which is the case where averaging had the best chance of paying. It did not take it.

```
cmf local examples/local/shifts_cdcl3.sdf --target shift --conformers 5 \
    --conformer-weighting boltzmann
```

The option is there because the question is a fair one to ask of another set, not because it helps on these two.

Many atoms at once. A per-atom target is a large problem where a per-molecule one is a small one: the 400 molecules in `examples/local/` are 5096 carbons, and the free NMR databases hold hundreds of thousands. An exact kernel over samples costs the square of their number in memory and the cube in time, so plan item LP5 asks for a Nyström or inducing-point approximation. For this kernel none is needed, and that is the first thing `benchmarks/run_local_scale.py` measures.

The rotation-invariant local kernel is a **linear kernel on a finite feature map**. The field autocorrelation of an environment is a fixed number of channels -- 255 for the five physico-chemical fields -- scaled per block and cosine-normalised, and the kernel is the inner product of those rows. So the ridge regression can be solved in the space of the 255 coefficients instead of the space of the samples (`cmfqsar/locallarge.py`), which is the same model and not an approximation of it:

route	q2	error (ppm)	memory	the folds
dual, an n by n kernel	+0.972645	8.8240	0.19 GB	13 s
primal, n by d rows	+0.972645	8.8240	10 MB	0.1 s

The predictions of the two differ by 2e-06 ppm, which is round-off. What differs is the cost, and it is the memory that decides: the kernel grows with the square of the samples and the rows grow with the samples. At twenty times these 5096 carbons the kernel would be 77 GB and the rows 198 MB.

More data does not help. With the primal route a much larger set becomes easy, so the question the item really asks can be put: is the error of the sections above the method's or the data set's? The set is 58705 carbons in 6000 molecules, built from the whole nmrshiftdb2 dump rather than from its chloroform spectra alone, so it is noisier by about a ppm and far larger. An exact kernel over it would be 25.7 GB; the primal route cross-validates it in 7 seconds. Each size is a subset of the next.

molecules	carbons	q2	error (ppm)	the folds
100	1105	+0.953	10.84	0.2 s
200	2131	+0.961	10.13	0.3 s
400	4113	+0.964	9.73	0.4 s
800	8079	+0.965	9.61	1.0 s
1600	15780	+0.965	9.70	1.7 s
3200	31221	+0.964	9.83	3.2 s
6000	58705	+0.965	9.75	6.8 s

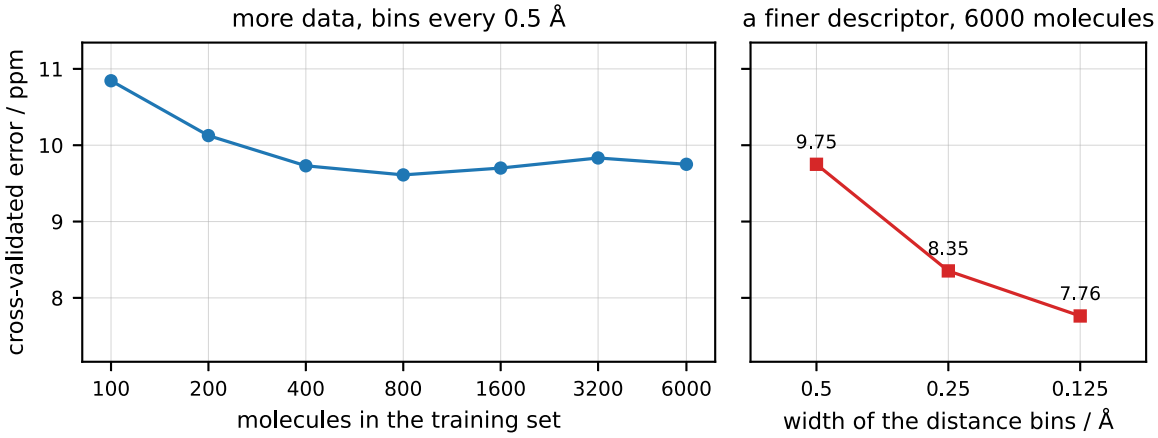
From 100 molecules to 6000 the error goes from 10.84 to 9.75 ppm, and from eight hundred molecules onwards it does not move at all. Sixty times the data is worth 1.23 ppm, nearly all of it spent before the first thousand molecules.

A finer descriptor helps more than sixty times the data. A flat curve says the error is not the data's. What is left is the descriptor, and the cheapest test of that is a finer one on the same samples: the distance axis of the autocorrelation at 0.5 A per bin against two finer spacings, everything else unchanged.

distance bins	channels	q2	error (ppm)
every 0.5 A	255	+0.965	9.75
every 0.25 A	495	+0.974	8.35

distance bins	channels	q2	error (ppm)
every 0.125 A	975	+0.978	7.76

Halving the bin width gains 1.40 ppm and halving again gains 1.99 ppm in all -- against the 1.23 ppm that sixty times the data gave. **The error of this model is set by what the descriptor can tell apart, not by how many carbons it has seen.** The route to the shift predictors this item names runs through a richer descriptor, which is plan item C5, rather than through a larger training set. What the scale work gives is that the question could be asked at all: the whole curve above, every fold and every ridge, is a few minutes of arithmetic.



The learning curve of the local shift model on the 6000-molecule nmrshiftdb2 set, cross-validated error against the number of training molecules, each size a subset of the next, and the same error at the full size when the distance bins of the descriptor are halved twice. Sixty times the data gains about one ppm and stops gaining after eight hundred molecules; two halvings of the bin width gain two. Drawn from benchmarks/results/local_scale/curve.json.

The kernel that does need landmarks. The posed overlap kernel of the next section has no finite feature map -- its features are the field itself, a function on space -- so there the dual form is the only exact one and the Nyström approximation is the route the plan names. `locallarge.nystrom_kernel` builds it from landmark environments, and on the 1225 aromatic carbons of that section it works very well: 200 landmarks of 1225 samples, 16 per cent of the rank, reproduce the exact q2 of +0.899, and 100 already give a relative kernel error of 0.001, while 25 give 0.072. The environments of one site are alike, which is why so few span them.

landmarks	relative error of the kernel	q2	error (ppm)
all 1225 (exact)	0.000	+0.899	4.217
25	0.072	+0.801	5.908
50	0.031	+0.859	4.970
100	0.001	+0.883	4.539
200	0.000	+0.899	4.203
400	0.000	+0.898	4.231
800	0.000	+0.899	4.217

What the descriptor still lacks, measured with SOAP. The learning curve above said the local model is limited by its descriptor. `benchmarks/run_soap_gate.py` asks which part is missing, by

computing a descriptor that has it on the same carbons, with the same ridge and the same folds: SOAP (Smooth Overlap of Atomic Positions), the power spectrum of one Gaussian density per element around the atom, from the dscribe package. The autocorrelation of this section is a two-point quantity. It resolves the distance between two neighbours of the centre and not their distances from it, so the triangle the three atoms form stays open. SOAP at angular order 0 resolves the distances from the centre alone and is two-body as well; each higher order adds the angle at the centre.

descriptor	5096 carbons, error (ppm)	1225 aromatic carbons, error (ppm)
atom-centred counts	20.46	6.38
field autocorrelation, best envelope width	8.82	5.19
posed overlap kernel (next section)	--	4.22
SOAP, angular order 0	8.15	4.94
SOAP, angular order 8, same density width and kernel exponent	6.62	3.81
SOAP, angular order 8, best density width and kernel exponent	5.72	3.50

The two-body SOAP matches the autocorrelation (+0.004 [-0.001, +0.008] in q2 on the 5096 carbons, bootstrapped over molecules). The angular orders are what lowers the error: +0.008 [+0.006, +0.011] there and +0.057 [+0.045, +0.070] on the aromatic carbons. On the aromatic carbons the best SOAP is better than the posed overlap kernel of the next section (+0.031 [+0.015, +0.052]), with no frame at all. So the part the local descriptor lacks is the angle at the centre. A CMF form of it would resolve each neighbour pair over its two distances from the centre as well as the distance between them, with the field weights as the channels in place of the elements. The next section builds that form and measures it.

The three-distance autocorrelation: the angle at the centre, with the fields as channels. `cmfqsar/triangles.py` resolves every pair of atoms in the environment over three distances: the distance of each atom from the centre and the distance between the two. Three sides fix a triangle, so this carries what the SOAP power spectrum carries, in a different basis, and it keeps the closed pair sum of the two-point descriptor: for every pair of fields, every pair of radial windows and every distance window, the sum over atom pairs of the two field weights times the three windows. Its channels are the fields. Where SOAP has one density per element, this descriptor has one weight per field, so a channel says which two atomic properties, at which two distances from the centre and how far apart, the model reads. An element indicator is itself a field, so SOAP's channels are one setting of the same code, and the two kinds of channel can be mixed. `cmf local --triangles` selects it; `--r-max` is the cutoff and the reach of the radial grid, `--grid-step` the spacing of both grids, `--grid-sigma` the window width, `--scaling` how the channel blocks are scaled before the kernel (per field, per pair of fields as the two-point kernel does, or not at all) and `--power` an integer exponent on the cosine-normalised kernel, which is what SOAP calls zeta. It works with `--primal` too, at power 1. An environment and its mirror image give the same channels: three distances fix a triangle up to reflection, which is the blind spot measured on whole molecules under plan item C5, and it is unchanged here.

`benchmarks/run_triangles.py` measures it on the carbons, ridge grid and folds of the SOAP gate above, with the element indicators of hydrogen, carbon, nitrogen and oxygen as a second set of channels through the same code, and both sets together. The settings below were chosen on the sets: cutoff 5 Å, grid step 0.5 Å, window 0.35 Å, kernel power 1, envelope width parameter 0.5 on the 5096 carbons and 0.1 on the aromatic carbons. Errors are root-mean-square out of fold, in ppm.

descriptor and channels	5096 carbons	1225 aromatic carbons
two-point field autocorrelation, best envelope width	8.82	5.19
SOAP, best of the gate	5.72	3.50
three-distance, five fields, scaled per field	5.45	4.13
three-distance, four elements, scaled per field	5.47	3.93
three-distance, four elements, no scaling	5.58	3.67
three-distance, fields and elements, scaled per field	4.96	3.86
three-distance, fields and elements, no scaling	4.83	3.81

Three things follow, each with its interval in q2, bootstrapped over molecules. **The basis reproduces SOAP.** With the element channels the three-distance descriptor is SOAP within the interval on both sets (+0.001 [-0.001, +0.003] on the 5096 carbons, -0.007 [-0.015, +0.001] on the aromatic ones), so the angular term the gate asked for is in the program. **The fields are a good set of channels on the broad set.** The five fields alone match SOAP there (+0.001 [-0.001, +0.003]) and improve on the two-point kernel by +0.017 [+0.014, +0.020]; put together with the elements they give 4.83 ppm, better than SOAP (+0.003 [+0.002, +0.005]) and than either set of channels on its own (+0.002 [+0.001, +0.004] against each). **On the aromatic carbons the fields carry less than the elements.** They improve on the two-point kernel by more (+0.056 [+0.033, +0.079]) but stay behind SOAP (-0.027 [-0.050, -0.010]) and behind the element channels in the same basis (-0.020 [-0.042, -0.001]), and adding them to the elements gains nothing (-0.006 [-0.035, +0.011]). The aromatic shift follows the substitution pattern of the ring, and the element channels state that pattern more directly than the field weights, which are smooth functions of it, do; on the broad set, where the carbons differ in hybridisation and neighbours, the field weights add information the elements do not have.

The smaller choices: a window of 0.35 Å is better than 0.5 or 0.7 Å on both sets; a kernel power above one, which SOAP needs, gains nothing here (5.46 ppm at power 2 against 5.45); scaling the channel blocks per field is best for the field channels and no scaling is best for the element channels, which is why the option exists. The cost is in the channel count, which grows as the square of fields times radii: 32 340 channels for five fields at the 5 Å cutoff and 128 205 for ten, and ten to twenty-five minutes per ten-fold cross-validation on the 5096 carbons against one minute for the two-point kernel. The default cutoff is 4 Å, which halves the count and loses 0.02 ppm on the shift set and nothing on the aromatic carbons. The command of the best shift model is

```
cmf local examples/local/shifts_cdcl3.sdf --target shift --rho 0.5 --triangles --r-max 5 --reference
```

The element channels of the benchmark are built in Python with `element_weights` of the module; the command line offers the fields. Results in `benchmarks/results/triangles/`.

11.16 A co-field around one atom: the local alignment

The local kernel of the previous section needs no frame, and that is also its limit: with no frame there is no co-field, so it cannot say **where** around the atom, and in which field, a feature raises or lowers the property. For one site a frame costs nothing, and plan item LP3 is that frame (`cmfqsar/localalign.py`).

The rule. The centre is already the origin and the first atom of the environment, so what is left is the rotation, and two of the centre's bonded neighbours fix it: the first gives the x axis, the second the xy plane. Which two is decided by the **atom type and nothing else**. A bond length would make a poor rule -- the two ring bonds of an aromatic carbon differ by hundredths of an angstrom, and by which of them is longer in one conformation -- so neighbours of the same type are treated as interchangeable. When that leaves the choice open, as it does for the two ring neighbours of an aromatic carbon or the three hydrogens of a methyl group, **every pose is kept and averaged**: the environment becomes the mean field of its poses, which is a mean-map kernel over the frames the site allows. That is the device the symmetrised Hammett set uses for the four images of a monosubstituted benzene, applied to one atom instead of one molecule.

The ring site. `ring_frames` is the site the plan names: an aromatic carbon of a six-membered ring, with the bond that leaves the ring on +x, the ring in the xy plane, the ring normal on z, and the two mirror images kept. In that frame the positions a substituent occupies on the ring are fixed numbers, which `RING_POSITIONS` holds: ipso at 1.4 A on +x, then ortho at 2.4 A, meta at 3.7 A and para at 4.2 A from the centre. A carbon where two rings meet has three aromatic neighbours and no such frame, and the rule refuses it rather than posing it anyway.

Then everything applies. The posed environments are in one frame, so the ordinary overlap kernel of this program works on them exactly as it works on molecules, and so does the co-field: `overlap_kernel` builds the `AtomTable` and the kernel, and `CMFModel.cofield_coefficients` gives the co-field around the atom. `cofields.cofield_at_points` reads it at places a chemist can name, `cofield_covariance_at` says how far the noise in the measured values moves it, and `contrast_at` compares two places with the right error bar: the co-field at two positions a few angstrom apart is built from the same training fields, so the two values are correlated and the error of their difference is not the two error bars added.

```
        cmf local examples/local/shifts_cdcl3.sdf --target shift --align --site ring --rho 0.25 \
            --cofield cofield.cube
```

`--align` replaces the rotation-invariant kernel by the posed overlap kernel, `--site ring` uses the ring rule and leaves out every atom it does not fit, and `--cofield` writes the co-field around the centre as a cube file in the frame of the pose, which any molecular viewer draws.

What it is worth, measured

`benchmarks/run_local_cofield.py` measures two things on the aromatic carbons of the shift set: 1225 carbons of six-membered aromatic rings in 170 molecules, with folds that hold out whole molecules, and the atom-centred count of the previous section as the reference.

The frame gains where the envelope is wide, and nothing where it is narrow.

descriptor	envelope width rho	width (A)	q2	error (ppm)	ridge	difference in q2 against the counts
atom-centred counts (18 atom types in 6 shells)	--	--	+0.768	6.383	0.01	--
posed overlap	0.05	4.5	+0.872	4.745	0.1	+0.104 [+0.070, +0.144]
invariant local	0.05	4.5	+0.116	12.461	0.1	-0.652 [-0.695, -0.594]
posed overlap	0.1	3.2	+0.887	4.448	0.01	+0.119 [+0.086, +0.158]
invariant local	0.1	3.2	+0.366	10.554	0.1	-0.402 [-0.457, -0.344]
posed overlap	0.25	2.0	+0.899	4.217	0.01	+0.131 [+0.098, +0.170]
invariant local	0.25	2.0	+0.803	5.882	0.001	+0.035 [+0.003, +0.068]
posed overlap	0.5	1.4	+0.889	4.425	0.0001	+0.121 [+0.083, +0.162]
invariant local	0.5	1.4	+0.847	5.187	0.01	+0.079 [+0.048, +0.116]
posed overlap	1	1.0	+0.861	4.935	1e-06	+0.093 [+0.046, +0.143]
invariant local	1	1.0	+0.836	5.361	0.01	+0.068 [+0.036, +0.105]

At the same envelope width the posed kernel is ahead of the invariant one by +0.756 [+0.718, +0.791] at a width of 4.5 A; +0.522 [+0.477, +0.572] at a width of 3.2 A; +0.096 [+0.069, +0.124] at a width of 2.0 A; +0.042 [+0.022, +0.061] at a width of 1.4 A; +0.025 [-0.015, +0.060] at a width of 1.0 A. The last of those is inside its interval, so at a width of 1 A the frame adds nothing that can be measured. Each at its own best width -- posed at rho 0.25 (+0.899) against invariant at rho 0.5 (+0.847) -- the difference is +0.052 [+0.033, +0.071].

That is what a frame is and is not for. An invariant descriptor of a wide environment has only distances, and at a width of 4.5 A the surroundings of an aromatic carbon are nearly the same distances in every molecule; the pose keeps the directions, so the ortho position and the meta position are two places rather than two atoms at similar distances. Squeeze the envelope down to 1 A and there is little left but the first shell, which distances alone describe perfectly well, and the frame adds nothing that can be measured.

Where the co-field sits, and what it does not say.

field	ipso	ortho	meta	para	\	para\	- \	meta\	
electrostatic	-16.7 +- 20.3	+33.0 +- 15.4	-28.5 +- 15.0	-10.0 +- 20.0	-18.5 +- 24.7				
steric	+15.5 +- 1.3	-2.6 +- 1.0	-2.7 +- 0.9	-4.2 +- 1.2	+1.5 +- 1.5				
hydrophobic	+1.4 +- 4.5	-2.0 +- 3.4	+6.6 +- 3.3	+2.2 +- 4.7	-4.4 +- 5.9				
hba	-57.4 +- 21.4	+17.8 +- 15.0	+29.6 +- 13.4	-41.1 +- 18.8	+11.5 +- 25.2				
hbd	+7.1 +- 33.6	+58.4 +- 25.4	-0.2 +- 23.7	+62.6 +- 30.4	+62.4 +- 39.2				

The co-field says the large thing and not the subtle one. The numbers above are the co-field at the four positions with the error the noise in the shifts allows, and one feature stands far above everything else: the **steric field at the ipso position**, +15.5 +- 1.3, 12 standard deviations from zero where nothing else reaches four. It says that putting anything bulkier than hydrogen on the measured carbon raises its shift, which is the dominant term of the 13C substituent effect and true of nearly every group: methyl +9 ppm, chlorine +6, nitro +20, methoxy +31.

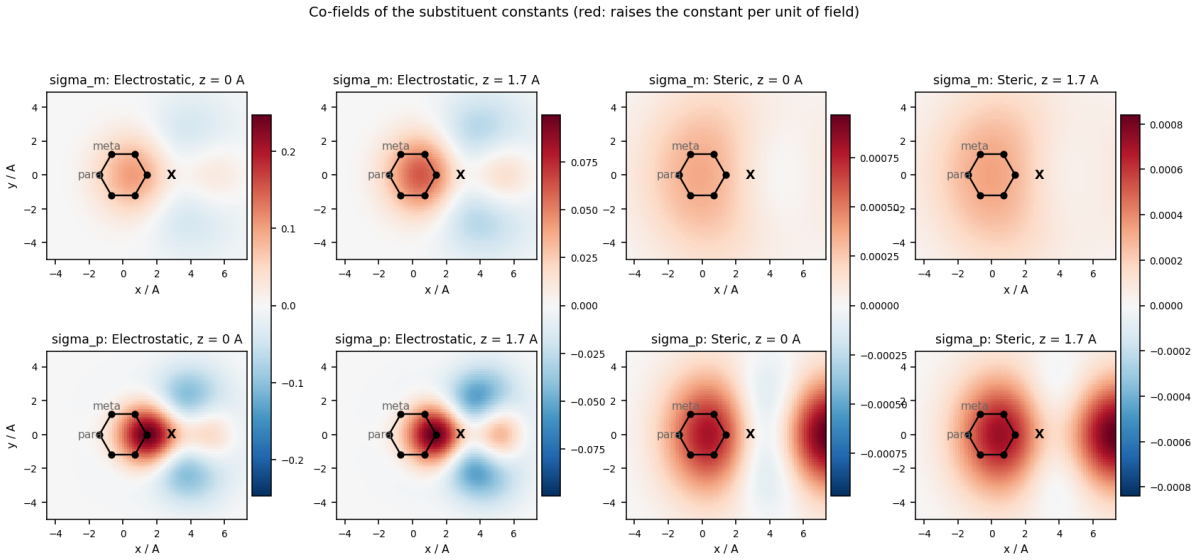
What it does not reproduce is the alternation around the ring, and that is the half of this item that fails. The question the plan asks is whether the co-field puts more at the para position than at the meta one. The para position is further from the measured carbon (4.2 A) than the meta one (3.7 A), so the envelope and the overlap both favour meta, and a co-field that still came out larger at para would be saying what the geometry does not. None does. For the electrostatic field the difference is -18.5 +- 24.7, the wrong sign and inside the error. The closest any field comes is the hydrogen-bond-donor one, +62.4 +- 39.2 (z +1.6), which is the right sign and the right field -- a hydroxyl or an amino group is where the ortho and para effects on an aromatic shift are largest -- and still short of two standard deviations.

Read the five co-fields together rather than one at a time. The fields overlap in what they describe -- an oxygen is a heavy atom, an acceptor and a negative charge at once -- so a co-field per field is a decomposition of one effect over five bases, not five independent statements, and the hydrogen-bond-acceptor co-field correcting the steric one at the ipso position is that decomposition at work rather than a claim about acceptors.

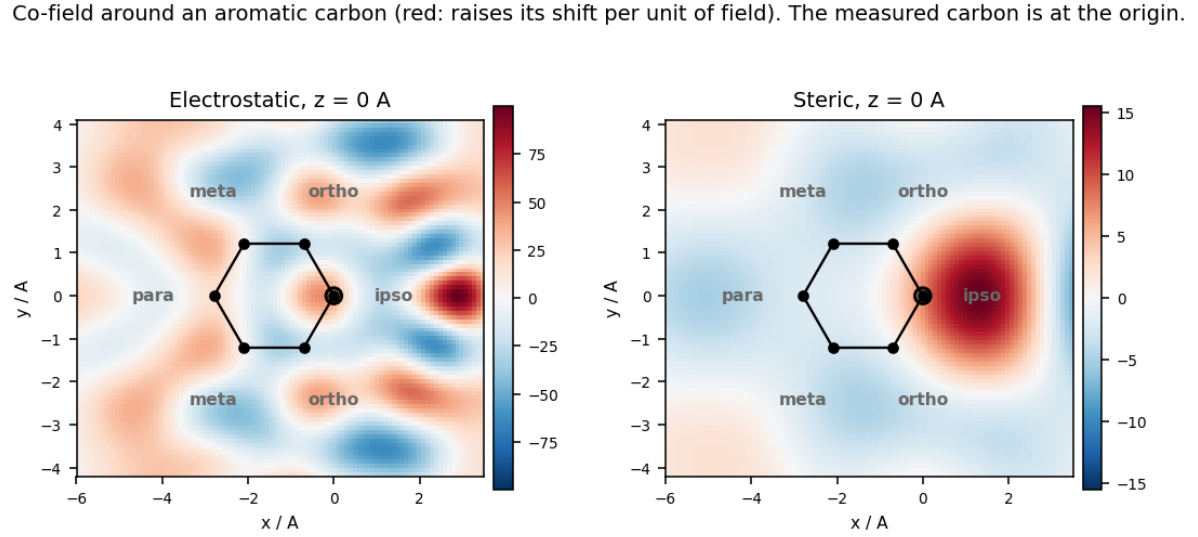
The signs are the more interesting half, and they are not nothing. The electrostatic co-field comes out -16.7 at ipso, +33.0 at ortho, -28.5 at meta and -10.0 at para, and a negative value means that negative charge at that place raises the shift of the carbon at the origin. A methoxy group raises the shift of the carbon it sits on by 31 ppm and lowers the ortho and para carbons by 14 and 8 ppm, and its oxygen is negatively charged, so a co-field that had learned that effect would be negative at ipso and positive at ortho and para. This one agrees at 3 of the four positions, with a correlation of +0.55 over them, but the y-scramble null for that correlation is wide (z +1.2): a correlation over four points cannot be sharp, and this one is not.

There is a reason to expect no better from these fields. A nitro group, whose nitrogen is **positively** charged, also raises the ipso carbon, by 20 ppm, and raises the para carbon where methoxy lowers it. No model linear in fragment charges can follow both groups, because what separates them is resonance and fragment charges do not carry it. That is the same wall the Hammett co-fields of this program met for sigma_p, and the remedy there was quantum charges computed on the whole

molecule, which carry the polarisation of the ring by the substituent (`examples/orca_atom_properties.py` and the `prop:` fields). The shift set here has no such charges. The other way round the wall is to stop asking the charge and ask the atom type, which is what the next part does and what settles the question.



The co-fields of the Hammett constants σ_m and σ_p, from the Hammett benchmark: the electrostatic and the steric co-field in the ring plane and 1.7 Å above it, with the ring and the substituent position X marked. Red raises the constant per unit of field. The electrostatic map of σ_p has lobes of both signs around the substituent position, a pattern that fragment charges cannot deliver for a nitro group and a methoxy group at once. Drawn by `benchmarks/run_hammett.py`.



The electrostatic and the steric co-field of the shift model around the measured carbon, at the origin, in the ring plane, with the ring and its four positions marked. Red raises the shift of that carbon per unit of field. The steric lobe at the ipso position is the feature twelve standard deviations from zero; the electrostatic map alternates in sign around the ring. Drawn by `benchmarks/run_local_cofield.py`.

The same co-field, read from the indicator fields. A partial charge cannot tell a pi-donor from a pi-acceptor, which is why the electrostatic co-field above could not reproduce the ring picture. A continuous indicator field carries the atom **type** instead: there is one field per Sybyl type, so an ether oxygen at the ortho position and a carbonyl carbon at the ortho position are different features

with co-fields of their own. That is how a table of substituent effects is written, and the second part of `benchmarks/run_local_cofield.py` asks whether the model writes the same table. It uses 17 types beside the five physico-chemical fields, at an envelope width of 4.5 A -- wider than the co-field above, because an envelope that damps the para position to nothing cannot be asked what sits there -- and predicts about as well (q2 +0.882, 4.55 ppm out of fold).

Two things make this a comparison of numbers rather than of pictures. First, `cofields.atom_effect` turns a co-field into **what one atom placed at a point would add to the prediction**, in ppm: the co-field is a sum of Gaussians and so is the atom, the overlap of two Gaussians is the closed-form integral the kernel itself sums over atom pairs, and the test suite checks the result against what the model's own prediction does when that atom is added. Second, the atom enters with the weight the envelope gives it at that distance (ipso 0.91, ortho 0.75, meta 0.50, para 0.41), so the number is what a real substituent at that ring position contributes rather than what an undamped one would.

type	ipso	ortho	meta	para	measured, for the group named
H	-1.6 +- 0.6 (n 774)	+0.2 +- 0.4 (n 1483)	+0.5 +- 0.3 (n 1448)	-0.1 +- 0.3 (n 733)	--
O.3	+5.4 +- 1.6 (n 170)	-6.4 +- 1.1 (n 342)	-1.3 +- 0.8 (n 334)	-1.5 +- 0.9 (n 157)	+31.4, -14.4, +1.0, -7.7 (methoxy (anisole))
C.3	+13.6 +- 0.8 (n 126)	+0.5 +- 0.5 (n 242)	-1.1 +- 0.4 (n 239)	-0.5 +- 0.5 (n 120)	+9.3, +0.7, -0.1, -2.9 (methyl (toluene))
C.2	+9.0 +- 1.1 (n 87)	-0.2 +- 0.7 (n 172)	-0.7 +- 0.5 (n 168)	+0.7 +- 0.7 (n 84)	+9.3, +0.2, +0.2, +4.2 (acetyl (acetophenone))
C.ar	+1.0 +- 0.2 (n 0)	-0.3 +- 0.2 (n 68)	-0.1 +- 0.1 (n 108)	-0.1 +- 0.2 (n 56)	--
N.pl3	+3.1 +- 2.0 (n 25)	-9.1 +- 1.4 (n 50)	-2.2 +- 1.1 (n 50)	-5.9 +- 1.4 (n 25)	+18.0, -13.3, +0.9, -9.8 (amino (aniline))
Cl	+7.4 +- 2.2 (n 16)	+0.1 +- 1.5 (n 32)	+1.7 +- 1.3 (n 32)	-2.4 +- 1.7 (n 16)	+6.2, +0.4, +1.3, -1.9 (chlorine (chlorobenzene))

The pattern comes out right. Over the 15 (type, ring position) pairs that have a measured value -- the ipso position left out, because it is the largest effect in every row and would carry the comparison by itself -- the correlation with the measured table is +0.89. Including the ipso position it is +0.72. The sharpest of these is the question the electrostatic field could not answer at all: a pi-donor lowers the para carbon and a pi-acceptor raises it, although the first atom of the donor is negatively charged and that of the acceptor is positive. Here the amino nitrogen gives -5.9 +- 1.4 ppm at the para position, 4 standard deviations from zero, against -9.8 measured, while the sp2 carbon gives +0.7 +- 0.7 against +4.2 measured. The signs are opposite, and the ether oxygen sits

on the donor side as well (-1.5 +- 0.9). That is the resonance distinction no model linear in fragment charges can make.

The sizes are compressed. A straight line through those 15 pairs has a slope of 0.46, so the model reproduces the shape of the table and about 46 per cent of its size. The root mean square difference from the measured row, per type, is Cl 0.7, C.2 1.8, C.3 2.5, N.pl3 8.1, O.3 14.0 ppm: the halogen and the two carbons come within a few ppm and the two strong donors are the ones held back. Three things pull that way: the ridge, which shrinks every coefficient; the envelope, which lets a para atom in at 0.41 of itself; and the set, where most rings carry several substituents whose effects the model shares out, while the table is measured on rings carrying one. The type labels are coarser than the table's groups too -- O.3 is every ether and hydroxyl oxygen, not only a methoxy, and C.2 is every sp2 carbon, not only an acetyl -- so the measured column is the nearest published row rather than the same quantity.

The same field set is a flag on the command, so the table above can be read off any per-atom target without the benchmark:

```
cmf local examples/local/shifts_cdcl3.sdf --target shift --align --site ring \
    --rho 0.05 --cif --cofield cofield.cube
```

`benchmarks/results/local_cofield/indicator.png` draws the two co-fields in the ring plane side by side. Both are positive where a substituent sits on the measured carbon and negative at the ortho positions, which is the large part of the effect; what separates the donor from the acceptor is the weaker pattern further out, and that is read off at the points in the table rather than by eye.

The other site the plan names, the carboxyl carbon, is not rescued by the frame either. The previous section found no signal on the 912 carboxylic acids: the invariant local kernel was no better than an atom-centred count there, and the difference field of the acid and its anion was far worse. A carboxyl carbon has two oxygens and a carbon of its own, which the atom types separate, so its pose is unique and nothing has to be averaged -- and with it the model loses to the count, -0.101 [-0.164, -0.037], and differs from the invariant kernel by -0.142 [-0.198, -0.088]. The frame was not what was missing. Only the model is measured there: a co-field read off a model that does not predict would be an interpretation of noise, which is why the pKa co-field of the substituted benzoic acids that this item's gate asks for is not drawn.

What is not done. The plan asks for a panel in the application that shows the environment with its co-field instead of the molecule. There is none: the co-field comes out of `cmf local --cofield` as a cube file, which a viewer draws, and the application still knows only models whose sample is a molecule.

11.17 A migration path as the sample: the barrier of a Li hop

Ionic conductivity in a solid electrolyte is set by the barrier a mobile ion meets between two sites, not by either site. That barrier is a field property in the sense of this program: the ion moving from one site through the bottleneck to the next feels the electrostatic potential, the repulsion and the polarisability of the framework around the path. In the language of the rest of the manual the framework is the receptor, the ion is the ligand and the barrier is the binding energy. Plan item BT7 makes the **hop** the sample (`cmfqsar/hops.py`), and everything the per-atom sections above built carries over with three changes.

The environment is cut from a periodic cell. `point_environment` is the `environment` of the local sections for a crystal: it collects every atom of the cell and every periodic image within the reach of the envelope, measured from a *point* rather than from an atom, multiplies the weights by $e^{-\rho\, d^2}$

and returns an ordinary field. The centre of a hop is the midpoint of the two sites, the migrating ion is removed from both its positions, and the first atom of the result is a weightless marker at the centre, so that the convention of the local modules that the centre is atom 0 holds. The fields are the inorganic set of the crystal section: the electrostatic field with oxidation states or, from a VASP run, net Bader charges (`cmfqsar/vasp.py`, plan item M3, reads them from `ACF.dat` and the `OUTCAR` with nothing written by hand), the steric field, the electronegativity, and one indicator field per element.

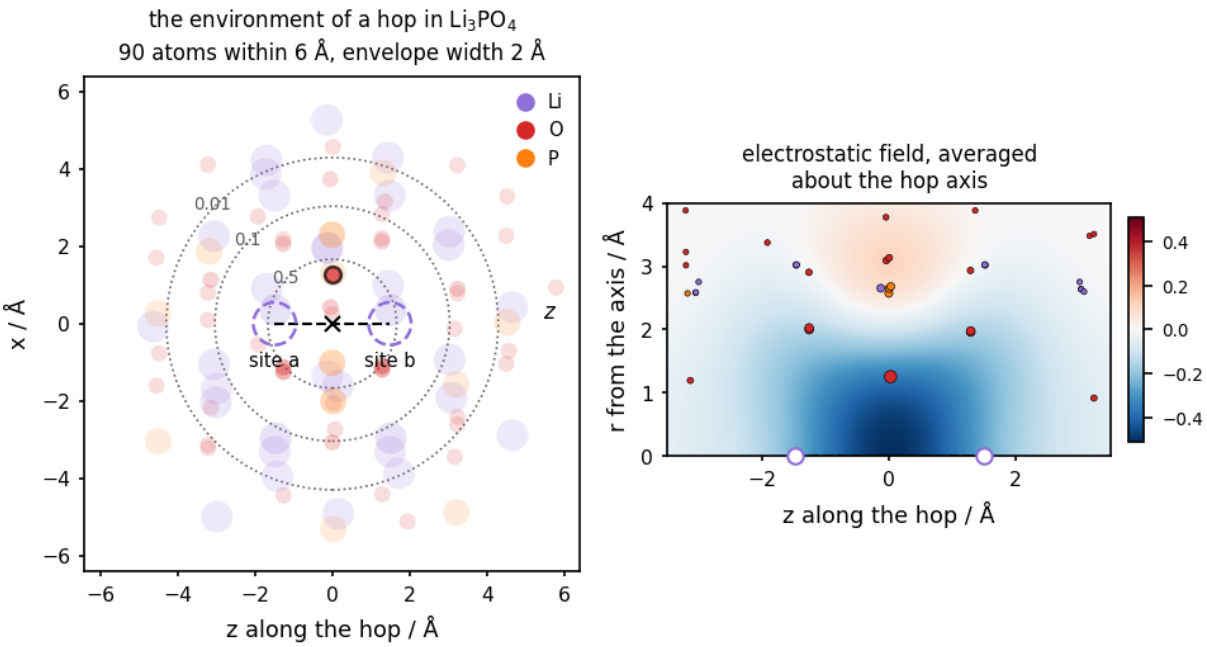
The frame is the hop itself. The origin is the midpoint and z points from the first site to the second, which is the direction the barrier is measured in; a hop and its reverse are two samples with two barriers. What is left is the rotation about z, and the program does not choose it: the kernel is **averaged over the whole rotation group about the hop axis**, which has a closed form. For two Gaussians of widths b_l and b_m at (ρ_l, z_l) and (ρ_m, z_m) in cylindrical coordinates about the axis, the overlap at a relative angle theta is

$$\left(\frac{2\pi}{b_l+b_m}\right)^{3/2}\exp\left(-c\left[(z_l-z_m)^2+\rho_l^2+\rho_m^2-2\rho_l\rho_m\cos\theta\right]\right), \qquad c=\frac{b_l b_m}{2(b_l+b_m)}, \tag{11.28}$$

and its mean over theta replaces the cosine term by the modified Bessel function $I_0(2c\rho_l\rho_m)$:

$$\bar{k}(l,m)=w_l w_m\left(\frac{2\pi}{b_l+b_m}\right)^{3/2}\exp\left(-c\left[(z_l-z_m)^2+(\rho_l-\rho_m)^2\right]\right)e^{-x}I_0(x), \qquad x=2c\rho_l\rho_m. \tag{11.29}$$

This is the mean-map kernel over the rotation group, the device the local alignment uses for a frame ambiguity, taken over a continuous group instead of a finite set of poses. The rotation average is a projection, so the kernel of two averaged fields equals the kernel of one plain field with one averaged field, and (11.29) costs one Bessel evaluation per atom pair, the same as the plain overlap kernel; the finite poses of `posed_hop` cost the number of poses squared and agree with it to machine precision at 96 poses. `HopModel` holds the kernel, one block per field divided by its diagonal and summed with the mixing coefficients, and the ridge regression on it; the rotation-invariant local kernel of the atom sections applies to the same environments unchanged and is the frame-free comparison.



A hop as the sample, for the hop of Li3PO4 the interface study computed with DFT, site 34 into site 0. Left: the environment cut from the cell, every atom within 6 Å of the midpoint with the two positions of the migrating ion removed (dashed circles), projected onto a plane through the hop axis, which is z; the dotted circles are where the envelope has fallen to 0.5, 0.1 and 0.01, and an atom is drawn fainter as its weight falls. The oxygen with the black edge forms the bottleneck, the point of least free space on the straight path. Right: the electrostatic field of the same environment averaged about the hop axis, as a function of the position along the hop and the distance from its axis, with the atoms at their (z, r) and the two sites on the axis; this is the view the rotation-averaged kernel of Eq. (11.29) and the co-field profile take.

The co-field is a profile. Averaged about the axis, the co-field depends on the position along the hop and the distance from its axis, so `HopModel.profile` reads it as a table in (z, r): where along the path, and how far from it, an atom of a given field raises or lowers the barrier. Its error follows the dual coefficients, whose covariance under independent noise of level σ in the barriers is $\sigma^2\,(K + \lambda I)^{-2}$, and `atom_contributions` splits a prediction over the atoms of the environment exactly, as the overlap model does.

The sample index is a pair. A hop is (structure, site a, site b, lattice offset of b). `enumerate_hops` lists every pair of sites of the mobile species between 1.5 and 4.5 A apart, periodic images included, both ways; `reduce_by_symmetry` keeps one of every set of hops with the same fingerprint (the sorted symbols, distances from the midpoint and positions along the hop of the atoms within 5 A), records the multiplicity, and `bottleneck` reports the point of least free space on the straight path with the atoms that form it. `HopSet.targets` turns the labelled hops into an `AtomTargets` whose "molecule" is the structure, so the folds hold out whole structures and the bootstrap resamples structures, since the hops of one cell are not independent.

The labels. `bvse_barriers` reads the barrier of every hop from the bond-valence site-energy landscape of the softBV model (the `bvlain` package): the energy is evaluated on a box around the path, the two sites are the lowest points within 0.6 A of them, and the barrier is the lowest level at which the two are connected through the box, found by bisection with a connected-components labelling at each step. That is a minimax path, which is what a migration barrier on a grid is, and it is cheap, so it labels thousands of hops; under rule 1 of the plan it is also the reference a model trained on DFT barriers has to beat. Measured or DFT barriers come from a table (`read_hop_labels`): structure, the two site indices, the image of the second site, the barrier and, if known, the reverse one.

From hops to conduction. The model predicts per hop. `percolation_thresholds` takes one energy per hop of a structure, adds the hops in order of increasing energy to a union-find over the sites that keeps the lattice offset of every site to its root, and reports the energy at which the network first connects through the cell in one, two and three independent directions (a path that comes back to a site at a different offset is a wrap, and the rank of the wraps is the dimensionality). `structure_percolation` does this from the bond-valence saddles or from predicted barriers, measured from the lowest site energy, which is what the effective activation energy of a conductor is read from.

```
cmf hops examples/hops/phosphate.extxyz examples/hops/borophosphate.extxyz examples/hops/borate.extxyz
    --charges file --bvse --reference --invariant --profile --percolate --out hops.csv
cmf hops structure.cif --labels barriers.csv --rho 0.05 0.1 0.25 --reference
cmf hops calcs/phosphate/pristine --charges file --list
```

The first command labels the hops of the three hosts of the Li interface study with the bond-valence landscape, fits the count reference, the invariant kernel and the rotation-averaged overlap kernel with folds that hold out whole structures, prints the co-field profile of the best model and the percolation thresholds of each structure from the labels and from the predictions, and writes every hop with its labels and out-of-fold prediction. The second takes barriers from a table; the third reads a VASP run directory (`CONTCAR`, `ACF.dat`, `OUTCAR`) and lists the distinct hops with their bottlenecks.

What it is worth, measured

`benchmarks/run_hops.py` measures the hop model on a public set and on the three hosts of the Li interface study (`benchmarks/results/hops/`).

The public set. 472 ternary compounds of lithium and oxygen (Li, O and one more element) of the Materials Project within 0.05 eV/atom of the convex hull, fetched through its OPTIMADE interface and prepared by `benchmarks/prepare_hops.py`: 24004 Li hops of 1.5 to 4.5 A, of which 1732 are distinct by symmetry, labelled with the bond-valence barrier and kept as the sample (at most six per structure, so that no structure dominates); 10 folds holding out whole structures; environments within 6 A of the midpoint; the fields are the electrostatic field with oxidation-state charges, the steric field, the electronegativity and one indicator field per element. The bond-valence barrier is a known function of the framework around the path, so the question is whether the kernel can learn such a function across structure types, and which descriptor it needs. One of the descriptors is the **two end sites** of the hop, each described the way plan item M5 describes a single site of a cell: the row of the cell's periodic autocorrelation that belongs to that site (`cmfqsar.sites.site_autocorrelation`), with the two rows combined as their sum and the absolute value of their difference, so that the descriptor does not change when the hop is reversed. That is the site-resolved model the gate of plan item BT7 asks the path model to beat. It is taken at the three cutoffs of the site profile M5 chose between, 4, 6 and 8 A, where it gives +0.439, +0.446 and +0.462; the row below is the one at 6 A, the radius the hop environments themselves use.

descriptor	envelope width (A)	q2	error (eV)	against the counts	against length and bottle neck	against the two end sites
counts of atoms in shells about the midpoint	--	+0.491	0.576	--	--	--
hop length and bottleneck distance	--	+0.103	0.765	--	--	--

descriptor	envelope width (A)	q2	error (eV)	against the counts	against length and bottle neck	against the two end sites
the two end sites, as item M5 describes a site (6 A)	--	+0.446	0.601	-0.045 [-0.142, +0.065]	+0.344 [+0.178, +0.452]	--
invariant local kernel	4.5	+0.553	0.540	+0.062 [-0.003, +0.170]	+0.451 [+0.321, +0.538]	+0.107 [+0.030, +0.218]
overlap kernel in the hop frame	4.5	+0.630	0.491	+0.138 [+0.085, +0.219]	+0.527 [+0.402, +0.608]	+0.184 [+0.087, +0.298]
invariant local kernel	3.2	+0.624	0.495	+0.133 [+0.066, +0.231]	+0.522 [+0.391, +0.609]	+0.178 [+0.082, +0.289]
overlap kernel in the hop frame	3.2	+0.649	0.478	+0.158 [+0.096, +0.250]	+0.546 [+0.423, +0.628]	+0.203 [+0.102, +0.318]
invariant local kernel	2.0	+0.657	0.473	+0.165 [+0.097, +0.266]	+0.554 [+0.421, +0.643]	+0.211 [+0.117, +0.329]
overlap kernel in the hop frame	2.0	+0.662	0.469	+0.170 [+0.112, +0.265]	+0.559 [+0.432, +0.641]	+0.216 [+0.109, +0.334]

Overlap in the hop frame against the invariant kernel at the same width: 4.5 A: +0.076 [-0.002, +0.145]; 3.2 A: +0.024 [-0.049, +0.099]; 2.0 A: +0.005 [-0.060, +0.071]. Learning curve of the frame model, q2 against the number of training structures (three random draws each): 25: +0.078; 50: +0.153; 100: +0.441; 200: +0.498; 400: +0.660; 459: +0.662.

The field beats the counts, and the frame pays only where the envelope is wide. The overlap kernel in the hop frame is ahead of the count reference by +0.170 [+0.112, +0.265] and of the two chemist's numbers by +0.559 [+0.432, +0.641], so the barrier is not a function of the hop length and the bottleneck width alone, nor of how many atoms of which kind surround the midpoint; where they sit matters. Against the rotation-invariant kernel at the same width the frame gains +0.076 [-0.002, +0.145] at 4.5 A, at the edge of the interval, and nothing at 3.2 and 2.0 A. That is the same picture the local alignment gave for the ring carbons: a wide environment is nearly the same set of distances in every structure and the directions separate it, while a narrow one is its first shell, which distances describe perfectly well. The best width is the narrowest (2.0 A), so at this size of set the frame is not what limits the model. The learning curve has not levelled off, +0.08 at 25 structures, +0.44 at 100 and +0.66 at 400, which is the opposite of what the shift set showed in LP5: here the set, not the representation, is the limit. The error of 0.47 eV on barriers whose quartiles are 0.34 and 1.13 eV is enough to rank hops and not to give a number; part of it is in the label, since a rigid landscape and a box around the straight path give some hops barriers of several eV that a relaxed path would not.

The path is worth more than its two ends, which is the gate of plan item BT7. The hop as an object has to earn its place against the obvious alternative: describe the two sites the ion moves between, each as well as the site-resolved model of item M5 describes a site, and let the pair stand for the path. It does not reach the path model. The frame kernel is ahead of it by +0.216 [+0.109,

+0.334] at a site cutoff of 6 A, +0.222 [+0.134, +0.310] at 4 A and +0.200 [+0.104, +0.322] at 8 A, clear of zero in all three, and the rotation-invariant kernel on the same hop environments is ahead by +0.211 [+0.117, +0.329]. The two end sites are also no better than the count of atoms in shells about the midpoint, -0.045 [-0.142, +0.065] at 6 A, so describing the two sites well is worth about what counting the neighbours of the midpoint is worth. The shortfall is not the descriptor confusing one hop with another: the two end sites name 1195 distinct pairs for the 1732 hops and 923 hops share their pair with another hop, since a hop and its reverse, and two hops through different images of the cell, get the same row, but predicting the mean barrier of each such group would still reach q2 +0.950 at an error of 0.181 eV. The site description loses the barrier for want of the path between the sites, not for want of a way to tell two paths apart. That is the reason the hop is a sample of its own: where the bottleneck sits between the two sites is not a property of either of them.

The co-field is a profile along the hop and away from its axis. Read at the midpoint and 2 A either side of it, 0 to 4 A from the axis, with the standard deviation the out-of-fold error (0.469 eV) allows, for the model at a width of 2.0 A:

Electrostatic field (a positive charge where the value is positive raises the barrier)

z \\ r	0 A	1 A	2 A	3 A	4 A
-2 A	-1.42 (0.81)	-0.01 (0.45)	-0.39 (0.32)	-0.07 (0.27)	+0.32 (0.26)
-1 A	-2.62 (1.07)	+0.10 (0.43)	+0.26 (0.31)	+0.36 (0.27)	+0.48 (0.25)
+0 A	-1.15 (1.11)	+0.38 (0.41)	-0.46 (0.32)	-0.10 (0.23)	-0.16 (0.22)
+1 A	-1.52 (1.07)	-0.02 (0.43)	+0.16 (0.31)	+0.82 (0.27)	+0.62 (0.24)
+2 A	-1.77 (0.80)	-0.67 (0.44)	-0.47 (0.33)	+0.35 (0.27)	+0.30 (0.26)

Steric field

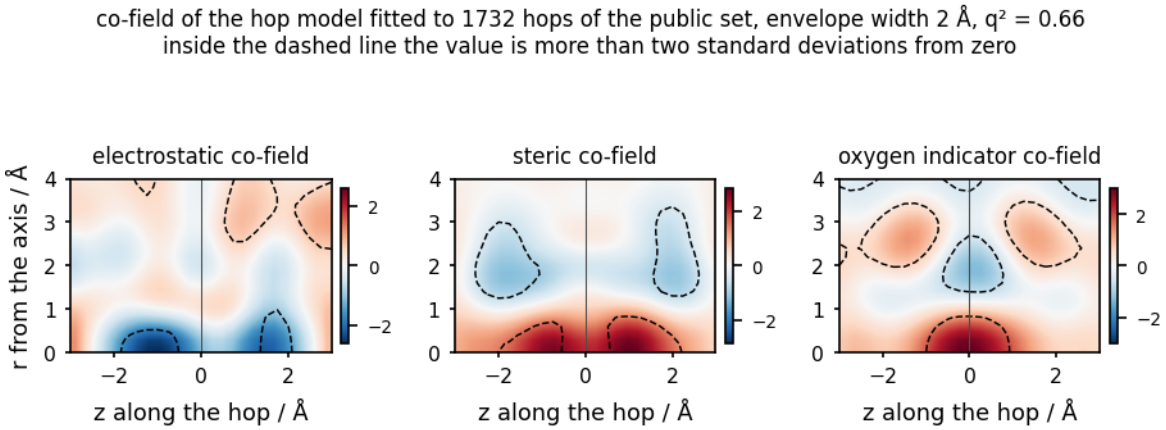
z \\ r	0 A	1 A	2 A	3 A	4 A
-2 A	+1.49 (0.74)	-0.23 (0.44)	-1.13 (0.36)	-0.50 (0.25)	+0.02 (0.21)
-1 A	+2.61 (0.96)	+0.33 (0.56)	-0.86 (0.44)	-0.09 (0.27)	+0.14 (0.20)
+0 A	+2.15 (1.74)	+0.60 (0.72)	-0.49 (0.47)	+0.24 (0.31)	+0.07 (0.22)
+1 A	+2.84 (0.94)	+0.70 (0.56)	-0.56 (0.44)	-0.03 (0.27)	+0.08 (0.19)
+2 A	+1.68 (0.73)	-0.11 (0.43)	-1.04 (0.37)	-0.64 (0.25)	-0.10 (0.21)

Indicator field of oxygen

z \\ r	0 A	1 A	2 A	3 A	4 A
-2 A	+0.77 (0.62)	-0.23 (0.39)	+0.56 (0.31)	+0.65 (0.24)	-0.38 (0.20)
-1 A	+1.96 (0.98)	+0.10 (0.44)	+0.18 (0.35)	+0.89 (0.25)	-0.27 (0.20)
+0 A	+3.03 (1.06)	+0.42 (0.44)	-1.31 (0.31)	-0.28 (0.25)	-0.50 (0.21)
+1 A	+1.89 (0.98)	+0.19 (0.44)	-0.13 (0.35)	+0.76 (0.25)	-0.12 (0.20)
+2 A	+0.67 (0.62)	-0.14 (0.40)	+0.61 (0.31)	+0.63 (0.24)	-0.39 (0.20)

Three things stand above their error, and they are the rules a chemist would state. First, anything on the axis raises the barrier: the steric co-field is +1.5 to +2.8 along the whole path at r

= 0 (two to three standard deviations), the oxygen indicator +3.0 (1.1) and the lithium indicator +5.2 (1.2) at the midpoint, because a path that runs through an atom is not a path. The electrostatic co-field on the axis is negative, -1.2 to -2.6, for the same reason: the atoms that lie on a straight line between two Li sites are oxygens, whose charge is negative, and a negative co-field times a negative weight raises the barrier. Second, an oxygen about 2 A from the axis at the midpoint **lowers** the barrier, -1.31 (0.31), four standard deviations: that is the bottleneck rule, a wide ring of anions around the saddle makes a low barrier and a narrow one a high barrier, and the co-field places the ring where the bottleneck of a 3 A hop in an oxide sits. Third, lithium and cations beside the sites raise it: the lithium indicator is +1.2 and +1.4 (0.25) at 3 A from the axis beside either site, five standard deviations, and the electrostatic co-field is +0.8 (0.27) there, so a positive charge within 3 A of a site repels the mobile ion out of it. The element indicators also separate cobalt (+3.1 at the midpoint) and manganese (+1.8 beside a site) from nickel (-1.5 at the midpoint), with the caution the local alignment section gives: the fields overlap in what they describe, so the twelve co-fields are one effect decomposed over twelve bases and not twelve statements, and an element read alone is partly the oxygen coordination that comes with it.



The co-field of the public hop model as a profile in the hop frame: the position along the hop against the distance from its axis, for the electrostatic field, the steric field and the oxygen indicator, on a finer grid than the tables above. Red raises the barrier per unit of field, blue lowers it; inside the dashed line the value is more than two standard deviations from zero. Anything on the axis raises the barrier, an oxygen about 2 Å from the axis at the midpoint lowers it, and a positive charge about 3 Å from the axis beside a site raises it. Drawn from `benchmarks/results/hops/profile_fine.json`*, written by* `benchmarks/run_hops.py --fine-profile`*.*

From hops to conduction. The out-of-fold barriers of every hop (the equivalent hops taking their representative's value) were run through the percolation step and compared with the thresholds of the labelled network, on the structures whose every distinct hop was labelled:

direction	structures	correlation	mean absolute difference (eV)	finite or not agrees
1D	164	+0.808	0.380	100%
2D	110	+0.857	0.351	100%
3D	60	+0.754	0.343	100%

The predicted network agrees with the labelled one on whether a structure percolates in one, two or three directions in every case, and gives the threshold to within 0.35 to 0.38 eV with a correlation of 0.75 to 0.86. The difference is about the model's error on one hop, which is what it should be, since a threshold is the barrier of one hop of the network. That is the step from a per-hop model to a per-structure number, and it holds.

The three hosts. Applied unchanged to the 48 distinct hops of Li3PO4, Li3BP2O8 and LiBO2 (`examples/hops/`, bond-valence labels, Bader charges on the atoms but the public model's oxidation-state charges for the prediction), the public model gives q2 -1.415, correlation +0.683 and an error of 0.279 eV. Within the hosts, leaving one host out, the frame kernel gives -0.120 and the counts -0.115; three structures cannot turn that into an interval, which is why the public set exists. The two hops the study computed with DFT are listed next to the bond-valence value and the prediction:

host	hop	length (A)	DFT barrier (eV)	bond-valence (eV)	predicted (eV)	bottleneck (A)
phosphate	34 -> 0	2.98	0.287	0.732	0.798	1.26 (O)
borophosphate	18 -> 8	2.64	0.003	0.393	0.561	1.48 (O O)

The DFT values are plain elastic bands for a vacancy hop into a relaxed vacancy, so they are lower bounds and include the relaxation the rigid landscape does not; the columns are not expected to agree in value. The public model ranks the hops of Li3PO4 and Li3BP2O8 (correlations +0.84 and +0.78) and fails on LiBO2, whose ten distinct hops span 0.03 eV, less than the error. The negative q2 says it does not carry the scale over: the public barriers average 0.89 eV and the hosts' 0.46, and the hosts are a phosphate, a borophosphate and a borate where the public set holds six silicates, four borates and no phosphate. A model for these hosts wants a set of their own chemistry, and the chemistry the public set has is that of the Materials Project, which is cathodes. The DFT values are lower than the bond-valence ones by about 0.4 eV, as a relaxed vacancy hop against a rigid landscape should be, and the order of the two hops is the same in every column.

The disordered conductors (plan item M2). The thirteen structures of `examples/hops/disordered/` are the families a solid-electrolyte study is about, read with the partial occupancies their refinements report: 6408 hops between sites the Li ion can hold, 298 distinct, 298 with a bond-valence barrier of the ordered framework. The public model applied as it stands gives q2 -1.075, correlation +0.273 and an error of 1.438 eV. A model cross-validated within the set, one structure held out at a time, gives -0.064 for the frame kernel and +0.166 for the counts, -0.230 [-0.886, +0.143] between them.

structure (COD)	what is disordered	hops	distinct	mean barrier (eV)	correlation with the public model	E_1D / E_2D / E_3D of the labelled network (eV)	of the public model's network
LLZO:Ta cubic (Li6.5La3Zr1.5Ta0.5O12)	Li sites 0.38/0.62, Zr/Ta shared	1728	18	0.22	-0.09	--	--
LLZO:Ta cubic (Li6.2La3Zr1.2Ta0.8O12)	Li sites 0.28/0.96, Zr/Ta shared	1824	19	0.25	-0.06	--	--
LLZO:Ta tetragonal	ordered	416	13	1.53	+0.61	--	--

structure (COD)	what is disordered	hops	distinct	mean barrier (eV)	correlation with the public model	E_1D / E_2D / E_3D of the labelled network (eV)	of the public model's network
LLTO rhombohedral (La0.5Li0.5TiO3)	Li sites 0.16	252	7	1.86	-0.32	--	--
LLTO tetragonal (La0.56Li0.33TiO3)	La/Li shared sites	12	4	4.00	-0.76	--	--
LGPS (Li10GeP2S12)	Li sites 0.47 to 0.89, Ge/P shared	416	26	0.40	+0.52	--	--
LiTi2(PO4)3 NASICON	ordered	0	0	nan	--	inf / inf / inf	inf / inf / inf
Li3.18Ti2(PO4)3 NASICON	Li sites 0.34/0.72	216	12	0.62	+0.84	--	--
Li2.92N	Li site 0.96	72	9	0.14	+0.06	--	--
LISICON (Li13.7Zn1.2Ge4O16)	Li sites 0.16 to 0.98, Li/Zn shared	416	55	0.52	+0.23	--	--
Li3PO4 (Pmnb)	ordered	144	18	0.35	+0.69	--	--
beta-Li3PS4	Li sites 0.3/0.7	152	19	0.39	+0.18	--	--
Li3.16Zn0.24GeO4	Li sites 0.06 to 0.92, Li/Zn shared	760	98	0.29	+0.50	--	--

What the table says. The percolation thresholds of the hop network agree with those of the bond-valence grid wherever the Li sublattice is the disordered part: cubic LLZO 0.40 against 0.41 eV, LGPS 0.53 to 0.59 against 0.57 to 0.60, beta-Li3PS4 0.25 to 0.40 against 0.27 to 0.42, Li2.92N 0.16 against 0.16, LISICON 0.45 to 0.49 against 0.47 to 0.52. That is the check that the hops between 1.5 and 4.5 A are the paths the landscape uses, and it holds for the families the item was built for. The ordered NASICON has no pair of Li sites in that window, because its M2 sites are empty and not in the file, so it has no hops; the tetragonal garnet and Li3PO4 differ from the grid because the network measures from the lowest site and the grid from its own minimum. The model is a different matter. The public model does not carry over (correlation +0.27 over all hops; +0.84 on the lithiated NASICON, +0.69 on Li3PO4, +0.52 on LGPS, nothing on the two cubic garnets whose eighteen distinct hops span 0.1 eV), and within the set the frame kernel is behind the counts, inside an interval that reaches from -0.9 to +0.1: twelve labelled structures, one of them with 98 distinct hops, and barriers up to 15 eV where a straight path runs through an atom, are not a set a

model can be measured on. What the item delivers is the data model that lets these structures in, exact and at no cost; a result on conductors needs the DFT-labelled set the previous section asks for.

Against published numbers on the same labels

Everything above measures this program against references built inside it. One published benchmark predicts the same quantity, so the models can be compared with other people's numbers: the nebBVSE122k part of the LiTraj collection (Dembitskiy et al., *npj Computational Materials* **11**, 131 (2025), MIT licence), which holds 122,421 bond-valence barriers of single Li hops in 11,290 structures of the Materials Project over 1,644 chemical systems. Its label is the barrier of one hop on a rigid framework, as here, relaxed with the nudged elastic band method instead of taken as the minimax path on a grid. Its split holds out whole structures, which was read from its index rather than from the paper: none of its 1,172 test structures appears in its training part. So the protocol is the one used here, and the numbers are comparable in kind.

Two differences remain. The collection covers every chemistry that holds lithium, so its barriers spread 2.4 times as widely as those of the public set above (standard deviation 1.97 eV against 0.81 eV), and the same error in eV therefore gives a higher q2 on it. And a kernel holds one number per pair of samples, so `benchmarks/run_litraj.py` reads a sample -- 400 structures with at most six hops each, 1801 hops of 393 structures -- where the published models are fitted on 96,849 hops of 8,945 structures.

model	q2	error (eV)	mean absolute error (eV)
counts about the midpoint (rule 1)	+0.468	1.389	0.812
hop length and bottleneck	+0.179	1.725	1.050
the two end sites, as item M5 describes a site (6 A)	+0.114	1.793	1.075
rotation-invariant local kernel, rho 0.25	+0.600	1.204	0.726
overlap kernel in the hop frame, rho 0.25	+0.580	1.234	0.730
published: M3GNet, a graph network marking the midpoint	+0.95	0.44	0.21
published: gradient-boosted forest on Voronoi features	+0.94	0.49	0.26
published: nearest neighbours on the same features	+0.92	0.58	0.27

Three things the table says. The ordering inside this program holds on a chemistry it was not built on: the field kernels are ahead of the count reference by +0.132 [+0.056, +0.209], so rule 1 is met here too. The size of the training set does not account for the distance to the published models: between 200 and 393 training structures the model gains +0.028 in q2 for every doubling, and the 4.5 doublings up to 8,945 structures carry it to about +0.71 at that rate, well short of +0.95. And the two findings of the sections above hold on the new set as well. The hop frame is again worth nothing in accuracy over the rotation-invariant kernel (-0.020 [-0.073, +0.039] here, +0.005 [-0.060, +0.071] on the public set), so the frame earns its place by the co-field it makes readable

and not by its predictions. The site-resolved descriptor falls from +0.446 on ternary lithium oxides to +0.114 here, so describing two end sites carries a barrier only inside one chemistry.

What the distance to those numbers is made of

The two published models closest to the graph network are a random forest and a gradient-boosted forest on hand-made features of the hop, not kernel models. The table above therefore mixes two questions: is this program's **description** of a hop behind, or is fitting that description with a **kernel**? They come apart by fitting both forests on this program's own descriptions, over the same hops and the same folds. One care is needed first. Both kernels here rescale the feature channels and then divide every row by its own length, so a kernel entry is the cosine of the angle between two rows and the row length never reaches the model; a forest handed the raw rows would be handed more than the kernel ever had. So each description is fitted twice, on the rows the kernel is built from and on the raw rows.

description	rows	fitted with	q2	mean absolute error (eV)	against the kernel
counts about the midpoint	as the kernel	random forest	+0.559	0.643	+0.091 [+0.024, +0.164]
hop length and bottleneck	as the kernel	boosted forest	+0.202	0.916	+0.023 [-0.094, +0.119]
rotation-invariant local	as the kernel	random forest	+0.742	0.481	+0.142 [+0.091, +0.198]
rotation-invariant local	as the kernel	boosted forest	+0.758	0.488	+0.158 [+0.110, +0.212]
rotation-invariant local	raw	random forest	+0.844	0.374	+0.243 [+0.188, +0.305]
rotation-invariant local	raw	boosted forest	+0.840	0.379	+0.240 [+0.183, +0.305]

The comparison favours the kernel, which is given the ridge that maximises the same out-of-fold q2 it reports while the forests are fitted at fixed settings and search for nothing.

The fitting method is worth a large part of the distance. On exactly the rows the kernel is built from -- the same description, nothing added -- a forest reaches q2 +0.758 where the kernel reaches +0.600, and the mean absolute error falls from 0.726 to 0.488 eV. So the description of a hop is not what was behind the published models here. Fitting it with a kernel ridge was.

And the cosine normalisation throws away something that carries the barrier. Adding the raw row lengths back, so that the model can also see how large an environment's features are and not only their direction, gains a further +0.086 and takes the mean absolute error to 0.374 eV. The kernel cannot use that length by construction. This is a property of the invariant kernel of this program and not of the hop models, so it bears on every kernel built on `cmfqsar.invariant.autocorrelation_kernel`; the plan opens it as item M15, and nothing is changed until it has been measured where that kernel is used. The same mechanism is already recorded as a caution for particles, where the per-atom normalisation removes the particle size.

Set beside the published numbers: this program's description of a hop, fitted with a forest, reaches a mean absolute error of 0.374 eV on 1801 hops, against 0.21 eV for the graph network and 0.26 eV for the published forest, both fitted on 96,849 hops. The forest also learns faster, +0.053 in q2 for every doubling of the training set against +0.028 for the overlap kernel.

Can a kernel hold what the forest found?

The kernels here are **linear** kernels: `autocorrelation_kernel` rescales the channels, divides every row by its own length and takes the inner product, so kernel ridge on one is a linear model in the feature space, while a forest is not. The first guess is therefore that the barrier is not a linear function of these features. A Gaussian kernel on the same rows tests that guess without leaving the kernel framework: the rows are unit vectors, so the squared distance between rows i and j is $2 - 2K_{ij}$ and the whole family of widths is a function of the matrix already computed, with no field touched again.

These arms are fitted more honestly than the table above. The width and the ridge are chosen inside each training part, on an inner split that holds out whole structures, and the test fold is touched once with the choice already made; the linear kernel is given the same treatment so the two can be set beside each other. That is why the nested linear row scores below the same kernel in the earlier table.

description	kernel	q2	mean absolute error (eV)	against the first table
counts about the midpoint	linear, nested	+0.445	0.818	-0.024 [-0.070, +0.003]
counts about the midpoint	Gaussian, nested	+0.430	0.849	-0.038 [-0.080, +0.000]
rotation-invariant local	linear, nested	+0.572	0.736	-0.029 [-0.048, -0.006]
rotation-invariant local	Gaussian, nested	+0.623	0.653	+0.023 [-0.002, +0.053]

The guess is the smaller part of the answer. On the local description the width is worth +0.052 [+0.023, +0.080] over the linear kernel given the same treatment. That is clear of zero, but the whole distance from that kernel to the random forest on the very same rows is +0.171 in q2, so a curved response to these features covers about 30 per cent of it, and the Gaussian kernel is still -0.119 [-0.170, -0.071] behind the forest. On the counts the width is worth nothing at all (-0.015 [-0.060, +0.032]), so this is not a general gain from curving a kernel; it belongs to the description that has many columns.

What the rest is more likely to be: the metric. A Gaussian kernel measures one distance over all 2601 columns at once, so the few columns that carry the barrier are diluted by the many that do not, and no single width repairs that; a tree reads columns one at a time and ignores the rest. On that reading the open question is which columns count and how they are weighted, which is a different repair from putting a width on a kernel. It is a reading and not a measurement, and the plan records it as the second half of item M15 -- together with the warning that learning weights has already failed once, in the pooled mixing pilot on molecular sets, so this case has to say why it differs before it claims anything.

One number came out in passing and applies to every kernel row of the earlier table. The nested linear kernel scores -0.029 [-0.048, -0.006] against the same kernel fitted with the ridge chosen on the folds it reports, which is what that search is worth. So the kernel rows above are overstated by

about that much and the forest rows by nothing.

The window the ion passes through

A barrier is set by the narrowest place along the path: a window of three or four framework atoms the ion has to pass. The two-point description of an environment carries the distance between two neighbours but not the two sides that meet at the centre, so it does not resolve the angle there, and two different windows can leave the same distance histogram. The section on the SOAP gate above measured what that costs on a carbon-shift model; here it is measured on a barrier. `cmfqsar.triangles` resolves the triangle for an environment whose first atom is the centre, and the first atom of a hop environment is a weightless marker at the midpoint, so its triangles are exactly the midpoint with two framework atoms. The reach, the grid spacing, the exponent and whether the row length is kept are all chosen inside each training part.

description	q2	error (eV)	mean absolute error (eV)
two-point, nested, same environments	+0.572	1.246	0.736
two-point with a Gaussian width, nested	+0.623	1.169	0.653
random forest on the two-point rows	+0.742	0.967	0.481
three-distance, nested	**+0.789**	**0.875**	**0.421**

The three-distance kernel against the two-point description fitted the same way is +0.217 [+0.159, +0.286] -- four times what a Gaussian width was worth -- and +0.209 [+0.145, +0.274] against the overlap kernel in the hop frame, which was the best model of the sections above. It passes the random forest on the two-point rows (+0.047 [-0.004, +0.096]) and the boosted forest (+0.031 [-0.022, +0.083]), so on this task a kernel is no longer behind a forest given the same description. **And it is still a linear kernel on fields, so the co-field survives**, which neither a forest nor a Gaussian width offers.

Three further readings. The nested arm chose the setting that keeps the row length in every fold, so resolving the angle and keeping the length are different pieces of information that add together. Mixing the two-point kernel back in is worth -0.004 [-0.032, +0.028] and the arm chose the pure three-distance kernel in every fold, so the two-point description adds nothing once the three-distance one is there. And the gain does not rest on the grid: reaches of 3, 4 and 5 angstrom with spacings of 0.5 and 0.75 land within 0.003 of each other at the first power, and only the exponent moves the figure.

Two cautions. A random forest on the raw two-point rows is still ahead of this kernel by -0.055 [-0.090, -0.023], so a flexible learner finds something a linear kernel on triangles does not; the second subsection below measures a forest on these same rows and says what that something is. And the range of the three-distance grid was placed after a first look at this same sample, so while nothing was chosen on a test fold, the grid is not independent of the data.

A Voronoi volume is a ratio of fields

The published models' own feature ranking puts the **smallest volume of the Voronoi polyhedra** around the three sites of a hop -- its start, its midpoint and its end -- first, and the largest of the three fourth. A Voronoi cell is defined by a minimum over atoms, which looks unlike anything this program computes, but it is the narrow-width limit of a ratio of fields. With $d_i(\mathbf{r}) = |\mathbf{r} - \mathbf{r}_i|^2 - R_i^2$ the

squared distance to atom *i* less its squared radius, the radical (power) Voronoi cell of a centre *c* is where d_c is the smallest, and

$$w_c(\mathbf{r}) = \frac{e^{-\beta d_c(\mathbf{r})}}{\sum_i e^{-\beta d_i(\mathbf{r})}} \tag{11.30}$$

tends to the indicator function of that cell as β grows, everywhere except its boundary. The denominator is a steric field with unit weights and a Gaussian of width 1/√(2β), so the volume is an integral of a ratio of fields and the Voronoi cell is its sharp limit.

Both are measured, on four columns from the three sites (their smallest, largest and mean volume, and the midpoint alone):

setting	q2	mean absolute error (eV)
the Voronoi cell itself	+0.465	0.824
the field version, β = 32	+0.465	0.823
the field version, β = 8	+0.468	0.820
the field version, β = 2	+0.467	0.817
the field version, β = 0.5	+0.376	0.894

A field carries the quantity with nothing to spare. The cell and the field version are level, and the nested arm chose a field over the cell in every fold. Only the widest setting fails, where one point of space is shared between neighbours instead of belonging to the nearest.

The feature is strong in few terms, and it adds nothing here. Those four numbers are level with the whole count reference (-0.000 [-0.074, +0.092]), which has hundreds of channels, and ahead of the hop length with the bottleneck by +0.289 [+0.238, +0.338]; they remain behind the whole two-point description (-0.103 [-0.168, -0.039]). But adding them to the three-distance kernel is worth +0.010 [-0.005, +0.023], which does not clear zero: the triangles around the midpoint already carry what the free volume says, as a description of those triangles should. That mixture is the best kernel model of this section at q2 +0.799 and a mean absolute error of 0.418 eV, and it is still -0.045 [-0.080, -0.013] behind the random forest on the raw two-point rows. So the published models' strongest feature is not what this program was missing; what the forest has is accounted for by the next subsection.

The forest is the fitting method, not the description

Two things closed most of the distance in the sections above. One is the **description**: the length of a row, which the kernel divides out, and the angle at the midpoint, which a two-point description cannot resolve. The other is the **fitting method**: a forest instead of a linear kernel on the same rows. The best model so far mixes the two -- it is a forest, and it is given the raw two-point rows -- so it is not clear which of them it owes its lead to. The arm below separates them by giving a forest the **three-distance** rows, with the same hops, the same folds and the same forest settings as every other forest in this section. If the better description is worth to a forest what it was worth to the kernel, the description side is still open; if it is worth nothing, what is left is the fitting method alone.

model	q2	error (eV)	mean absolute error (eV)
three-distance kernel, nested	+0.789	0.875	0.421

model	q2	error (eV)	mean absolute error (eV)
three-distance kernel with the free volume	+0.799	0.854	0.418
random forest on the two-point rows, raw	+0.844	0.753	0.374
random forest on the three-distance rows	+0.842	0.758	0.356
boosted forest on the three-distance rows	**+0.846**	**0.746**	**0.354**

The reading is in two comparisons. Against the kernel on the same rows the two forests are +0.053 [+0.015, +0.091] and +0.058 [+0.029, +0.089], both clear of zero, so a flexible learner still finds something a linear kernel on this description does not. But against the random forest on the raw two-point rows the best of them is +0.003 [-0.026, +0.028], which does not clear zero: **the better description is worth nothing to a forest, while it was worth +0.217 to the kernel.**

So the description side of the distance is closed. What a forest has over this program on this task is not a description of the hop that the fields cannot express. It is the fitting method: a model that can combine the rows in ways a linear kernel cannot. The Gaussian width above had already recovered about 30 per cent of that, and what is left is about +0.05 in q2, or 0.06 eV of mean absolute error. For this program that is a trade rather than a loss, because the field kernel keeps the co-field and a forest does not; and it says where to look next for accuracy on this task, which is the fitting method and no longer the description.

One note on what was measured. The forests are given the rows the winning kernel is built from: the channels scaled as that kernel scales them, with the row length left in, which is the setting the nested arm chose in every fold. The normalised form of the same rows was not measured. It would pair with a setting that was never chosen, and this feature matrix is 63,036 columns wide, so one arm on it costs about half an hour.

What this means for the program. On this task a graph network fitted on a hundred thousand barriers is still the most accurate model, and the published work shows a further step: a machine-learned interatomic potential that runs the elastic band itself reaches a mean absolute error of 0.07 to 0.08 eV against DFT barriers on structures without a transition metal, and 0.11 eV over the whole set once fine-tuned on it, which is a different kind of accuracy again. What the field model offers on migration is not the lowest error. It is the cost -- no relaxation at all, one field evaluation per hop -- and the co-field above, which says where along the path and at what distance from its axis the barrier is decided. Those are reasons to use it for screening and for reading, not for a number to quote.

What the measurements above add is where the distance to those numbers came from, and it was mostly not the fields. Fitting the same description with a forest, keeping the row length the kernel divides out, and resolving the angle at the midpoint take the mean absolute error from 0.726 eV to 0.354 eV; the one of those that keeps the model a linear kernel on fields, and so keeps the co-field, takes it to 0.421 eV. The last arm says how the rest divides: the description side is closed, and what a forest still has over the kernel is the fitting method. The published graph network reaches 0.21 eV, on fifty-four times as many hops, and it carries three-body terms of its own -- which is the same ingredient that moved this model furthest.

11.18 Reading the model from another program

Everything above reads a model from inside this program: the window, the command line, the library. A multiscale scheme needs it the other way round -- the model as a component of somebody else's calculation, the fast level between an electronic-structure method and a coarse model. Three interfaces are what the other programs ask for, and they are in `cmfqsar.interfaces` (plan item MS7). None of them is a new model. Each is a way in to the one that is already fitted, and the measurement at the end of this chapter is the check that nothing changes on the way.

A model as an ASE calculator

The Atomic Simulation Environment is how a structure reaches most of the atomistic tools in use. `CMFCalculator` makes a fitted model one of its calculators:

```
        from ase.optimize import BFGS
        from cmfqsar.interfaces import CMFCalculator

        atoms.calc = CMFCalculator(model, sign=-1.0)   # -1: move towards a *higher* prediction
        BFGS(atoms).run(fmax=0.02, steps=50)
```

What the calculator calls the energy is `sign` times the prediction, in the units of the activity and not in eV. An ASE optimiser minimises, so `sign=+1` drives the structure towards a lower prediction and `sign=-1` towards a higher one, which is the direction a design loop wants. Nothing converts units, because a predicted activity has no energy scale: the step sizes of the optimiser have to be chosen for the numbers the model gives.

The forces are minus the gradient of the prediction, from the analytic derivative of the chapter *The gradient of a prediction*. They exist for the two kernels that derivative covers -- the overlap integral and the rotation-invariant autocorrelation -- and the calculator says so when they do not: it reports `energies only`, drops `forces` from the properties it offers, and the refusal names the kernel and the reason. A model with no gradient is still useful through this interface, because a single-point screen needs energies alone.

One caution carries over from the gradient chapter, and it is the reason this is a component and not a force field: the gradient knows nothing about valence, and following it alone pulls bonds out of shape. Added to a force field, or used to turn a rigid body, it moves a structure towards what the model calls active. Used on its own, it will break the molecule and raise the number.

A periodic model answers energies and refuses forces, and writing this interface is what found the reason. The periodic **overlap** kernel evaluates the ordinary overlap against an image-expanded table, so moving one atom moves every image of it, and that term is not in the derivative. It had not been refused before, because the refusals of the gradient chapter are written per kernel source and the periodic overlap carries the ordinary one. It is refused now, by name and with the reason, rather than answering with a number nobody has checked.

A barrier per migration path

The hop model of the chapter above answers for a path, not for a structure, and a kinetic Monte Carlo or a percolation analysis asks about a structure. `HopBarriers` closes that gap: it enumerates the symmetry-distinct paths of the mobile ion, builds each environment the way the training set was built, predicts, and hands back a table.

```
        from cmfqsar.interfaces import HopBarriers

        reader = HopBarriers(model, fieldset, rho=0.25, r_max=6.0, mobile="Li")
        table = reader.table(atoms, name="Li3PO4")
        table.write_table("barriers.csv")        # one row per path, with where it is
        print(table.percolation())               # E_1D, E_2D, E_3D under the predicted barriers
```

The settings are arguments and not guesses. An environment built with another envelope width or another reach is a different descriptor, so the width, the reach, the mobile element and the path length limits have to be the ones the model was fitted with. The table carries each path's two end sites, its length, its midpoint, how many paths of the structure it stands for, and its predicted barrier; `barriers` maps every path of the structure, not only the distinct ones, because a percolation step needs the whole network. A caller who already has the network -- this program's own benchmark does -- passes it as `hopset=` and only the environments and the prediction are done here.

A field on a grid

`write_cofield_cube` writes the co-field as a Gaussian cube file, which is what a visualiser or a grid-based code reads. For a molecular model this is the export the command line already had. What is new is the periodic case: for a model on one lattice the cube is one cell, with the **lattice vectors as the cube's own axes**, so the file is the periodic field itself and tiles to as much of the crystal as a viewer is asked for. The earlier export could only write a cubic cell, because a cube file was built from one spacing and three counts; a general cell needs general axes, which this writes. On the command line it is `cmf cofield MODEL --cell STRUCTURE`.

The rotation-invariant kernels have no co-field at all -- their channels know no place -- and are refused by name rather than drawn. The map of `cmf sites` is what such a model offers instead.

Does the measurement survive the journey out?

This is the gate of item MS7, and it is a reproduction rather than a new number: the public hop model is rebuilt at its stored width and ridge, and then read twice. Once by the route the benchmark script uses -- environments built in the script and handed to the model -- and once through `HopBarriers`, which is given a structure and does the rest itself. The structures are the three hosts of the lithium interface study, which the public model has never seen.

Paths of the hosts	Distinct by symmetry	Compared by both routes	Largest difference
1040	48	48	1.2e-11 eV

The two routes agree to 1.2e-11 eV, which is round-off: the interface is a way in and not another model. The two paths the benchmark lists against a DFT elastic-band barrier come back as the stored table has them:

Host	Path	Stored	Through the interface
phosphate	34 -> 0	0.7979	0.7979
borophosphate	18 -> 8	0.5610	0.5610

And so do the percolation thresholds, which is what a conductivity estimate starts from -- the lowest barrier at which the paths form a connected network in one, two and three dimensions:

Host	E_1D	E_2D	E_3D
phosphate	0.403 / 0.403	0.452 / 0.452	0.798 / 0.798
borophosphate	0.528 / 0.528	0.654 / 0.654	1.571 / 1.571
borate	0.281 / 0.281	0.437 / 0.437	none / none

In eV above the lowest site, by the script's route / through the interface. The full table is in `benchmarks/results/hops/table.md` (`benchmarks/run_hops.py --interfaces`).

What this does not show is a faster calculation. Both routes pay for the same two things, the environments and the kernel against the training set, and the interface pays a little more because it works one structure at a time. What it shows is that the number a coarse model reads through the interface is the number this program measured, which is the only property a component of somebody else's calculation has to have.

11.19 The voltage of an insertion electrode: the cell against its composition

A cathode is judged by its **average voltage**: the work per electron of taking the working ion out of the host. For the insertion of $(y-x)$ lithium into $\mathrm{Li}_x\mathrm{Host}$ the average voltage over that interval is the change in energy per electron, referred to lithium metal,

$$V = -\frac{E(\mathrm{Li}_y\mathrm{Host}) - E(\mathrm{Li}_x\mathrm{Host}) - (y-x)\,E(\mathrm{Li})}{y-x}, \tag{11.31}$$

with the energies per host formula unit and in electronvolts. Plan item BT3 makes that pair the sample (`cmfqsar/battery.py`), and the benchmark asks what an electrode paper asks: does the **arrangement** of the atoms say anything about the voltage that the **composition** does not already say? Composition is a strong baseline here, because the voltage is mostly decided by which metal is reduced and by how much.

The data need no account. The Materials Project serves its structures and their formation energies through OPTIMADE without an API key, and a formation energy already carries the elemental reference: it is the energy of the cell minus the energies of the elements it is made of. Elemental lithium is one of those elements, so it cancels in the difference above and the voltage follows from the two formation energies alone. One query per chemical system, the phases within 0.03 eV per atom of the convex hull kept, each cell reduced to its **host framework** (its composition with the lithium removed, divided by the common factor) and its **lithium content per host formula unit**, and neighbouring steps of one framework's ladder paired. The official insertion-electrode documents, with topotactic pairs and the corrections a battery database applies, need a free key and the `mp-api` package; `insertion_electrodes` fetches them when a key is given.

The reconstruction is checked before it is used (`python benchmarks/run_battery.py --stage check`). Over the first insertion, up to one lithium per host formula unit:

cathode	published, V	reconstructed, V
LiCoO2, layered	4.10	3.77
LiFePO4, olivine	3.45	3.33
LiMn2O4, spinel	4.00	3.23
LiNiO2, layered	3.80	3.47

The reconstruction is **0.39 V low on average, and low every time**. Both reasons are limits of the data and not of the kernel: the energies are the published GGA/GGA+U ones without the corrections a battery database applies, and the average here runs over the intermediate phases of the ladder, which a two-phase measurement does not see. The window above the hull was chosen on that table. At 0.01, 0.02 and 0.03 eV per atom the agreement is the same, 0.39 V, while the set grows from 83 to 303 pairs; at 0.08 it is worse, 0.57 V. So 0.03 is the default, and the table is in `results/battery/table.md` so the choice can be read rather than taken on trust.

The protocol. 303 insertion pairs over 157 host frameworks, the discharged cell describing the pair, 23 fields. Five folds keep a framework together: two steps of one ladder share a framework, a chemistry and most of their structure, so a model that has seen one of them has been told most of the answer for the other. The ridge and the width of the Gaussians are chosen inside the training part by ten-fold cross-validation. The composition model is a linear kernel on the element fractions of the cell and the crystal descriptors of item BT2.

model	mean absolute error, V	spread over the folds	R^2
the mean voltage of the set	1.203	-	0.000
composition	0.569	0.115	+0.728
fields	0.650	0.146	+0.624
fields and composition	0.582	0.117	+0.684

The voltages of the set average 2.77 V and predicting that mean costs 1.203 V, so every model here is well inside the question.

What the numbers say. The composition decides this property and the arrangement adds nothing. The field kernel is 0.081 V worse than the composition baseline and the two together 0.013 V worse, both inside the fold-to-fold spread of 0.146 V, which under rule 4 is not a result. That is the answer the data support for an average voltage: it is a thermodynamic quantity of a composition, and the geometry enters only through which phase that composition takes. The item is kept under rule 5 because it is the loader and the baseline every other battery item needs, and because a negative result measured against a baseline is worth more than an unmeasured kernel.

By family of hosts, for the model that uses both kernels:

family	pairs	mean absolute error, V	mean voltage, V
phosphate	113	0.509	3.61
oxide	105	0.664	2.83
other	51	0.603	0.34
fluoride	21	0.678	4.23
sulfide	10	0.466	1.86
silicate	3	0.178	2.81

The family called *other* is the lithium alloys: Li-Mg, Li-Sn, Li-Ag, Li-Ga, Li-Si, Li-Pb, Li-Zn and Li-Al, whose voltages run from -0.25 to 1.18 V with a mean of 0.34 V. They are anode materials rather than cathodes, and they are kept because the voltage of an alloying reaction is the same quantity measured the same way. They are also most of what makes the set wide, and so most of the reason

that predicting its mean costs 1.203 V.

The difference of the two cells, as a field (item BT6). A voltage is a property of a pair and not of a structure, and the fields are linear in the atoms, so the difference of the two cells' fields is itself an ordinary field in the cell: the atoms of the lithiated cell with their weights, and the atoms of the delithiated one with their weights negated (`cmfqsar/celldiff.py`). Where the two cells agree -- most of the host -- the two contributions cancel, and what is left is the inserted ions and the relaxation their arrival caused. Two things do not cancel, and neither is a fault: the formal charge of the metal, which item BT1 makes Co(+3.5) at half lithiation and Co(+3) at full, and the electronegativity proxy, which is a deviation from the mean over the cell and so moves when lithium enters it. Because a field difference is a field, every kernel takes it unchanged; it is bilinear in the weights and nothing has to be re-derived for signed ones.

What the construction costs. A difference is defined only when the two cells stand on one lattice and hold the same host atoms, and two cells relaxed separately do not: their lattice constants differ by the volume change of the insertion, which is a quantity a battery paper reports in its own right. Of the 303 pairs, **42 have cell lengths within 5% of each other and the same host atoms**, over 33 frameworks; the other 261 are refused, with the reason recorded for each, so what the frame costs is reported and not hidden.

What it gives, and what it does not. The reading the item asked for comes out of the construction rather than out of a model: over the 42 pairs the inserted ion sits in **5.0 +/- 1.1 oxygens at 2.22 +/- 0.13 angstrom**, which is the first shell of a lithium in an oxide. The model is another matter. On the same five folds, grouped by framework:

model	mean absolute error, V	spread over the folds	R^2
the mean voltage of these pairs	0.791	-	0.000
composition	0.656	0.100	+0.228
the discharged cell alone	0.699	0.143	+0.138
the difference field	0.797	0.083	+0.014
the difference field and the composition	0.680	0.118	+0.250

The gate the item set itself is not met. Paired over the same samples, the difference field is +0.090 V against the discharged cell alone, with an interval of [-0.034, +0.216] that contains zero, so the two are indistinguishable here; against the composition it is +0.134 V [+0.004, +0.273], which does not contain zero, so on this set the difference field is **worse** than the element fractions. The set is the reason to be careful either way: 42 pairs over 33 frameworks leave about eight pairs per fold, and every model sits within 0.135 V of simply predicting the mean voltage. What the numbers do support is the conclusion of the stage before them, now with the pair itself as the sample: the voltage follows the composition, and the arrangement does not add to it.

The co-field in the cell the item promised is not here, and the reason is worth stating. A co-field needs every sample in one frame, which for cells means one parent lattice for the whole set; 33 different hosts do not have one, so the kernel used here is the autocorrelation, whose reading is a co-pattern over distance channels rather than a map. With R^2 +0.014 that co-pattern has nothing to say, and the channels it names are element indicators of single frameworks, which means the model is recognising hosts and not geometry. The map needs a ladder of one host on one lattice -- which is the beta-Li3PO4 vacancy series the conductivity literature argues about, and the one case

this program still has to compute for itself.

The limits. The pairs are matched by host *composition* and not by host *structure*: two cells with the same framework formula and different frameworks can be paired, which a topotactic criterion would refuse. The target carries the 0.39 V offset of the check above, so an error below that is not a claim about measured voltages but about reproducing these energies. And 303 pairs is a large set for this program and a small one for a materials benchmark: the same code reaches 528 pairs at a hull window of 0.05 eV per atom, at the cost of a worse agreement with the published values.

11.20 An ion as the probe: where a cation sits, and what it gains

A probe field answers "what would this group gain by sitting here?". With an ion as the probe the answer is the quantity an electrolyte or an ionophore is designed for, and it can be checked against a number that was computed rather than fitted. `examples/ether/` holds the set that makes the check possible: 1,085 saturated acyclic ethers -- glymes and their branched relatives -- with the binding energy of Li^+, Na^+ and Mg^{2+} at wB97X-D3/def2-TZVP, and 1,050 of them in the relaxed geometry of the lithium complex, from the distribution of Wang et al. (Chem. Commun. 56 (2020) 8920). `examples/make_ether_examples.py --fetch` downloads it and checks the hashes; the distributor states no licence, so the status of the copy is *unclear* in the sense of public_3dqsar_datasets.md. `benchmarks/run_ether.py` is the measurement (plan item EC7).

Where the ion sits. A probe field is a map of an energy, so its deepest point is a prediction of where the ion goes, and it needs no model and no fitting at all. On the 1,050 relaxed complexes, with the ion removed and its position kept, the deepest point of the lithium probe field lies 0.36 ± 0.55 Å from the lithium the DFT relaxation placed, and within 1.0 Å in 97 % of the complexes. Two references that also need no model: the centroid of the molecule's oxygens is 0.65 ± 0.74 Å away (86 % within 1.0 Å) and the centroid of its heavy atoms 0.67 ± 0.80 Å (82 %). The probe field is thus a usable guess of the binding site in a flexible chain, which is the one thing a docking program would otherwise be needed for.

The binding energy. Nested cross-validation, five outer folds, the ridge and the field width chosen inside each fold, no frame anywhere (the alignment-free kernel). Mean absolute error in eV, folds drawn at random:

model	Li^+ (1048)	Na^+ (980)	Mg^{2+} (982)
predicting the mean of the set	0.454	0.449	1.235
oxygen count, C:O ratio, heavy atoms	0.296	0.279	0.505
the same with the molecule's HOMO energy	0.254	0.246	0.444
the electrostatic field alone	0.209	0.248	0.557
the five fields	**0.124**	**0.181**	**0.266**
the matching ion probe field alone	0.140	0.193	0.321
the five fields and the probe field	**0.117**	**0.171**	**0.254**

Paired over the same molecules, with a bootstrap interval: the ion probe beats the counts by 0.156 eV [0.139, 0.175] on lithium, and it beats *the electrostatic field alone* -- which is the comparison the plan item names -- by 0.069 eV [0.060, 0.078] on lithium, 0.055 on sodium and 0.236 on magnesium, every interval clear of zero. The probe field is slightly behind the five fields together (+0.015 eV [+0.009, +0.022] on lithium) and the two together are ahead of either. With one oxygen count held out at a time, which asks for a chain length the model has not seen, the five fields and the probe reach 0.129, 0.204 and 0.279 eV against 0.284, 0.291 and 0.571 for the counts. **This is the first measurement in this part of the manual where the arrangement of the atoms beats what the formula already says**, and by about a factor of two: the voltage of an insertion electrode went the other way.

The selectivity between two ions is a much harder quantity. The difference between the lithium and the sodium energy is what an ionophore is designed for. Over the 980 molecules that carry both it has a mean of −1.07 eV and a spread of 0.36, and the oxygen count tells one nothing about it (correlation −0.05, q^2 +0.01). Predicting its mean costs 0.190 eV; the five fields and the probe together reach 0.171 eV (q^2 +0.22) and the two ion probes 0.176. The field models are ahead of the counts by 0.012 eV [0.003, 0.022], which is a real difference and a small one. Adding the sodium probe to the lithium probe changes nothing at all (+0.001 eV [−0.000, +0.003]), which is what the probe definitions predict: two singly charged probes report the same potential at two distances, and that is not enough to separate two ions. **Selectivity needs what a fixed probe charge cannot carry**, namely the polarisation of the ion and of the donor atoms.

Three ions as three endpoints. The three columns are the multi-task problem of plan item E1 with descriptors of the endpoint rather than a label: 3,010 measurements over 1,048 molecules, one kernel, and the covariance between the endpoints taken from the ion's radius, charge and polarisability. Fitting the endpoints independently costs 0.204 eV; one shared correlation of 0.5 gives 0.198 and the descriptor covariance 0.188, with q^2 +0.893, +0.698 and +0.852 for Li^+, Na^+ and Mg^{2+}. The covariance it chooses is worth reading: lithium is more closely related to magnesium (0.46) than to sodium (0.27), and sodium and magnesium least of all (0.20). What decides which endpoint helps another is therefore the size of the ion -- Li^+ 0.76 Å, Mg^{2+} 0.72, Na^+ 1.02 -- and not its charge.

An ion that was never measured cannot be predicted here. Each ion in turn was removed entirely and predicted from the other two. Magnesium is refused before any arithmetic: the mean of an endpoint is a linear function of the charge, and with two monovalent ions left that function cannot be evaluated at charge +2, so the design matrix has two identical rows. Lithium and sodium can be evaluated and both fail: 1.484 eV and 0.539 eV against their own means of 0.454 and 0.449. They are far better than every reference a model that never saw the ion has -- the mean of the measured ions is 3.5 and 4.7 eV away -- and still much worse than knowing the removed ion's own average. The part that fails is the mean: lithium's is −3.84 eV and the model, which may use only the charge to place it, predicts −2.36. Three endpoints are too few for more than one descriptor column, so this is a limit of the set and not of the method.

The limits. The coordinates are the relaxed *lithium* complexes, so the sodium and magnesium models are built on a lithium-shaped conformation; the three energies correlate (+0.80 and +0.85), which hides the error in a single-ion model but limits what the selectivity may be read as. The targets are computed energies of a one-to-one complex in the gas phase, not solvation free energies, so nothing here is a statement about a real electrolyte, where a lithium ion is surrounded by four or five molecules. And the published model of this set is a neural network potential trained by active learning, which is a different object from a kernel on a field: the comparison made here is against the formula and against the electrostatic field, not against it.

11.21 The ion with its shell as the molecule: a solvation complex

The section above models the solvent molecule. For almost every property of an electrolyte the object that carries the property is not the solvent but the complex: the ion together with the molecules that touch it. `cmfqsar/solvation.py` builds one (plan item BT4), and it does three separate things so that each can be checked on its own.

It places. A donor atom is an oxygen or a nitrogen with a lone pair, and the direction of that lone pair is the reverse of the sum of the atom's bond directions — the bisector for an ether oxygen, the C=O axis for a carbonyl. `place_shell` puts the ion at the origin, takes as many directions as there are molecules from the polyhedron of that coordination number (linear, trigonal, tetrahedral, and the bipyramid and octahedron above that), turns each molecule so its lone pair points back at the ion, and sets the donor at the coordination distance — 2.01 Å for Li^+ and oxygen, the distance measured in the relaxed complexes of `examples/ether/`. Each molecule is then spun about its own ion–donor axis to the angle that leaves the most room. An anion may join the shell, which makes a contact ion pair, and the total charge follows from the fragments. `wrap_ion` is the other case: one molecule that wraps the ion with several of its own oxygens, as a glyme does.

It optimises. GFN2-xTB through `cmfqsar.xtb`, which caches by molecule, charge, method and starting geometry, so a run resumes where it stopped. A complex of fifty atoms takes under a second. The binding energy is E(complex) − E(ion) − ΣE(solvent) and the desolvation energy E(n−1) + E(solvent) − E(n), both written out in the functions that compute them.

It frames. This is what the complex gives that the free solvent cannot. A series of glymes has no scaffold to superimpose, but a series of complexes has one atom in common, and the polyhedron of its donors fixes the orientation: `cation_frame` puts the origin on the ion, the axes along the principal axes of the donor set, and resolves the four proper sign choices by the third moments of the donors. Where two moments are equal — a regular polyhedron — the axes inside that plane are taken from the heavy atoms of the whole complex instead, and `frame_quality` reports the moment ratios so that a caller can see when the frame still leaves a rotation free. With a frame there is an overlap kernel, and with an overlap kernel there is a co-field.

What the builder is worth, measured against a published relaxation

`benchmarks/run_solvation.py` measures the builder on 150 of the 1,050 Li^+–ether complexes of `examples/ether/`, which were relaxed with density functional theory. Three runs per molecule: the free solvent, the published complex re-optimised with xTB, and the complex the builder makes from five generated starting conformers, each optimised, the lowest kept.

Quantity	The published relaxation	xTB from that geometry	The builder's own complex
oxygens reaching the ion	4.24 ± 0.87	4.38	3.81 ± 0.84
nearest oxygen (Å)	1.935	1.874	1.856
mean oxygen distance (Å)	1.995 ± 0.068	—	1.926 ± 0.088
binding energy, correlation with the published one	—	**+0.907**	+0.555
binding energy, mean error (eV)	—	−1.323	−0.919

Two statements follow, and they are about different things. **The method overbinds**: GFN2-xTB puts a lithium ion about 1.3 eV deeper into an ether than wB97X-D3 does, and gets the ordering right while doing it (+0.91). It ranks and it must not be quoted as an absolute number. **The builder loses the longer chains**: it reaches 3.81 of the 4.24 oxygens and its correlation with the published energy is +0.555, not +0.907. The reason is measured rather than guessed: the five starting conformers of one molecule spread 1.52 ± 0.71 eV, so which conformer the builder starts from decides the answer, and five of them are not enough for a chain with five oxygens. What the builder lacks is a conformational search, not a better placing rule.

The gate: does the shell carry what the solvent alone does not

Item BT4 asks that question, and the 1,048 complexes with a published binding energy answer it. Five outer folds, the ridge and the field width chosen inside each fold. Predicting the mean of the set costs 0.454 eV.

Model	MAE (eV)	R^2
the solvent alone, alignment-free kernel (the model of the section above)	0.124 ± 0.016	+0.894
the complex, alignment-free kernel	**0.114 ± 0.013**	**+0.914**
the complex, overlap kernel in the cation frame	0.145 ± 0.018	+0.868
the solvent again, overlap kernel in the same frame	0.143 ± 0.009	+0.886

The gate is met, and by a small margin. Paired over the same molecules, putting the ion back gains 0.010 eV [0.004, 0.016], an interval clear of zero and eight per cent of the error. The shell does carry something the solvent alone does not, and it is not much: the solvent in the geometry it holds in the complex already says most of what there is to say.

The frame costs accuracy. The overlap kernel in the cation frame is 0.018 eV [0.010, 0.027] worse than the alignment-free kernel on the same molecules, and inside that frame the ion adds nothing at all (+0.002 eV [−0.003, +0.008]). The two models were run precisely to separate the ion from the frame, and the separation says the gain above belongs to the ion. The frame is not useless, but what it buys is not accuracy.

What it buys is the co-field, which the alignment-free kernel cannot have at all. Fitted on every complex and evaluated on a grid, the largest magnitude of the electrostatic co-field lies 1.37 Å from the ion and 1.37 Å from the nearest donor oxygen, and that of the hydrogen-bond acceptor field 1.03 Å from the ion. Both sit inside the first shell, which is where the chemistry is, so the item's reading gate is met. The hydrogen-bond donor field is the control that costs nothing: an ether has no donor, that field carries no weight anywhere, and its largest magnitude lands 12.5 Å away, in the corner of the box.

The series computed here: carbonates, ethers and fluorinated ethers

Nineteen solvents with one to four molecules around Li^+, every complex placed and optimised by this module (75 complexes, one failure). At one solvent molecule the carbonates bind −2.49 ± 0.12 eV, the ethers −3.01 ± 1.08 and the fluorinated ethers −2.91 ± 0.61. The wide spread of the ethers is not noise: twelve of the nineteen solvents give a complex in which the ion touches one oxygen, and those twelve agree closely (−2.36 ± 0.18 eV), while the other seven are chains that fold and

donate two or three oxygens at once, so their "one molecule" is already a three-coordinate complex. Two numbers from the table are worth reading on their own. **Fluorination weakens the donor**: FEC binds 0.29 eV less than EC, which is the effect those additives are used for. **The fluorinated ethers hold the fourth molecule weakly**: the desolvation energy at four solvents is 0.92 to 1.35 eV for the carbonates and 0.02 to 0.33 eV for the fluorinated ethers, which is what a diluent is chosen to do. Two of the 56 desolvation energies are negative, meaning the optimiser did not find a bound complex at that size, and the table names them.

11.22 A distribution of shells instead of one: sampling a bulk electrolyte

The section above builds one complex and optimises it. A liquid electrolyte does not hold one complex. At any moment every lithium ion in the box has a different set of neighbours, and the property an experiment measures — a conductivity, a transference number, a diffusion coefficient — is an average over that distribution. Plan item BT5 asks whether the distribution is worth sampling: does a model built on shells cut out of a bulk simulation predict better than the same model built on item BT4's single optimised complex? Two modules answer it. `cmfqsar/electrolyte.py` builds and runs the box. `cmfqsar/shells.py` cuts the shells out of the trajectory and turns them into one field.

Why a distribution of shells is still one field

This is the reason the route needs little new code. A CMF field is a sum of Gaussian functions centred on atoms, and the kernel between two fields is bilinear: linear in each of them. So the weighted mean field of a set of shells is itself a field of the same kind. Its atoms are all the atoms of all the shells, and each atom keeps its own weight multiplied by the weight of the shell it came from. Nothing new has to be defined, and three statements follow at once.

- The kernel between two such mean fields is exactly the weighted mean of the shell-to-shell kernels. No pairwise sum has to be written out, and a 16-member ensemble costs one kernel evaluation on a larger atom list, not 256 of them.
- The kernel matrix stays positive definite, because a mean of fields is a field and the kernel is the same kernel.
- The model is still linear in that mean field, so the co-field and the Shapley decomposition keep their meaning. The co-field of an ensemble is the weight function that the average shell is integrated against.

`combine_members` in `cmfqsar/fields.py`, written for plan item D4, needed no change for this. It already accepted members that carry their own atoms, and that is what shells are: two shells of the same electrolyte need not even hold the same molecules.

The per-atom reading needs one decision, because a per-atom contribution has to point at *an* atom. `atom_maps` sends the ion to the ion, and every solvent atom to the matching atom of the first molecule of its own species in the first member. A per-atom number is therefore per atom of a species, summed over however many molecules of that species the shell happens to hold. That is the only reading the geometry supports: "the second ethylene carbonate" is not a well-defined object in a liquid.

The box: what is simulated, and with what

A `Formulation` is a salt, a set of solvent weight fractions and a molality. From those three `counts` works out the exact number of molecules of each species: 250 solvent molecules in the proportions

asked for, and the number of ion pairs that reproduces the molality, with at least four pairs so that the shells are not all cut from one ion. The arithmetic needs no density, which is why molality and weight fractions are the only concentration units accepted. A molarity or a volume fraction would require a density to be assumed, and the simulation is supposed to produce the density, not to be given it.

The rest is an ordinary molecular-dynamics protocol, and every part of it is a stated setting of `Settings` rather than a default buried in a script:

Step	What is done
Starting geometry	packmol packs the molecules into a cube whose side comes from the pure-component densities, loosened by ten per cent
Topology	assembled from the same RDKit templates packmol was given, in a fixed species order; the packed file supplies coordinates only, because packmol writes no bond records
Solvent and anion parameters	GAFF-2.11 through `openmmforcefields`, with atom types from `antechamber`
Charges	the OpenFF NAGL graph-network AM1-BCC model, which gives AM1-BCC charges without a semi-empirical calculation per molecule
Cation parameters	the Amber monovalent ion set in `amber14/tip3p.xml`, because GAFF has no lithium
Electrostatics	particle-mesh Ewald, 1.0 nm real-space cutoff
Integration	Langevin middle integrator, 2 fs steps, hydrogen bonds constrained
Ensemble	298.15 K, Monte Carlo barostat at 1 bar
Equilibration	150 ps, during which the box finds its own density
Production	1,000 ps, one frame every 2 ps

The net charge of the assembled system is checked against zero before the run starts, since a wrong anion template would otherwise give a silently charged box. The versions used for the series below are OpenMM 8.3.1, `openmmforcefields` 0.15.1, OpenFF Toolkit 0.18.0, OpenFF NAGL 0.5.5, AmberTools 24.8, packmol 20.14.3 and MDTraj 1.11.1, run on a CUDA device inside WSL.

The reach of the force field decides which electrolytes can be in the series at all, and the module makes that a refusal rather than an approximation. GAFF has no boron, so `SALTS` holds no borate: asking for $LiBF_4$ or LiBOB raises an error that names the reason. This matters for the series below, because those two salts are a large part of the open conductivity table it is drawn from.

Cutting a shell out of a periodic box

A shell is every molecule with at least one donor atom — oxygen, nitrogen, fluorine, sulfur or chlorine — within a cutoff of the ion. The cutoff is 1.3 times the ion–donor coordination distance measured in item BT4's relaxed complexes, which is 2.613 Å for Li^+ and oxygen. The ion's own counter-ion is not excluded: a shell that holds the anion is a contact ion pair, and whether one forms is exactly what changes with concentration.

Two faults of a periodic box have to be repaired, and both of them produce a wrong geometry rather than an error, so neither announces itself:

- **A molecule split across a face of the box.** Half its atoms are at one edge and half at the other. `trajectory.make_whole`, written for item D7, puts it back together by following bonds.

- **A shell on the far side of the box from its ion.** The molecule is whole but it sits a box length away. `trajectory.reimage` moves it to the image nearest the ion.

The selection itself is written for speed, because a single electrolyte here needs 3,500 to 31,000 shells. All donor atoms of the frame are screened at once: one array of distances to the ion under the minimum-image convention, which is a matrix product and a rounding. Only the residues that pass are then made whole and reimaged. The first version repaired every residue before measuring any distance, which meant about half a million repairs for one electrolyte, and it was too slow to use.

Each shell records what it is made of: the species and the number of molecules of each, the distances of the donor atoms that reached the ion, and a `fingerprint` that does not depend on the order the molecules were found in, so that two shells of the same shape are recognised as the same shape.

Thinning, and the two numbers that say whether it converged

Twelve thousand shells cannot all be carried into a kernel. `cluster_shells` keeps a limited number of representatives — 16 by default — by grouping the shells whose fingerprints lie within 0.75 of each other and taking one shell from each group. The weight of a representative is the share of all sampled shells in its group.

Those weights are populations, not Boltzmann factors. This is item D7's rule and it is worth repeating here: an equilibrated trajectory has already sampled the distribution, so a shell that is common appears often. Weighting those occurrences again by a Boltzmann factor would raise the distribution to a power and count the energy twice.

Item D8 requires a sampling error next to any accuracy figure obtained from a trajectory, and `convergence` reports two separate numbers. Both are computed exactly from one shell-to-shell overlap matrix, with no model fitted and no grid involved, as one minus the normalised overlap of two mean fields:

- **The block error of the mean field.** The frames are split into blocks, the mean field of each block is formed from an equal number of shells, and the error reported is how far those block means lie from the mean of the whole run. This is the sampling error the item asks for: it falls as the trajectory grows.
- **The cost of keeping 16 members.** How far the thinned ensemble's mean field lies from the mean field of all the shells. This is not a sampling error. It is the price of the thinning, and it does not fall when the trajectory grows.

The first version mixed the two, by measuring the block error on the thinned ensembles. It rose with more sampling, from 0.055 at 40 frames to 0.145 at 500, because thinning noise is larger than sampling noise and more frames give more distinct shapes to throw away. A diagnostic that gets worse as the data get better is reporting the wrong quantity. Measured separately on the same trajectory the two numbers are 0.00008 and 0.00496, and they mean different things.

The series: 62 measured electrolytes from an open table

The item allows simulated or measured transport data. Measured data is the better choice here, because taking the target and the descriptor from the same trajectory would test the force field against itself. CALiSol-23 is an open collection of 13,825 measured ionic conductivities of lithium electrolytes, published under CC BY 4.0, and `examples/make_electrolyte_shells.py` fetches it, checks its hash, and selects a series from it by stated rules.

Two targets are read off each formulation, not one. A fit of $\log_{10} k = a + b/T + c/T^2$ to that formulation's own measurements gives **$\log_{10}$ of the conductivity at 298.15 K**, which is what an electrolyte is chosen for, and **the activation energy at 298.15 K**, which is its temperature dependence. The second is the quantity a solvation shell has the better claim on: an ion moves by leaving one shell and entering the next, so the barrier should depend on how tightly the shell holds it. The fit is an interpolation and is never extrapolated.

Every selection rule is in the script, and each is there for a reason that can be checked:

Rule	Why
$LiPF_6$ only	GAFF has no boron, so $LiBF_4$ and LiBOB cannot be simulated here
molality and weight fractions only	those two units fix the molecule counts exactly, with no density assumed
solvents the module can parameterise	eleven carbonates, esters, ethers and acetonitrile
at least four temperatures, spanning at least 20 K and containing 298.15 K	so that both targets are interpolated
fit residual at most 0.03 in $\log_{10} k$, activation energy between 0 and 40 kJ/mol	drops formulations whose measurements do not follow one smooth curve
a source paper whose conductivity scale is below a tenth of the collection's is dropped	one paper's values are 0.019 of everyone else's at the same salt, solvent and temperature, which is a unit error in the source, not a property of an electrolyte
at most three molalities per solvent composition	keeps the concentration axis without letting one well-measured mixture dominate

What comes out is **62 formulations over 22 solvent compositions** of six solvents, from 0.245 to 2.373 mol/kg. The conductivity at 298 K spans 0.169 to 1.172 in $\log_{10}$ mS/cm, where predicting the mean of the set costs 0.146, and the activation energy spans 5.5 to 37.8 kJ/mol, where predicting the mean costs 3.535 kJ/mol.

The concentration axis is kept on purpose, because that is where a sampled shell can say something an optimised complex cannot. The share of shells that hold the anion rises with the molality (correlation +0.729), from 0.222 in the most dilute box to 0.978 in the most concentrated. A single four-molecule complex has no way to express that: the 22 compositions give only 12 distinct complexes, because rounding a mixture to four molecules makes several compositions identical, and 9 of those 12 are shared by more than one composition. Two of them, DEC/PC at 50:50 and at 60:40, round to two molecules of each and so give the same geometry and the same energy to six figures.

What the simulations cost, and what they produced

Quantity	Over the 62 boxes
atoms in a box	3,500 ± 398 (2,853 to 4,285)
simulation rate	456 ± 49 ns per day
time per box, including the shell extraction	191 ± 17 s (139 to 223)
failures	none

Quantity	Over the 62 boxes
density the barostat settled at, g/cm³	1.205 ± 0.081 (1.058 to 1.445)
molecules in a shell	4.26 ± 0.35 (3.72 to 4.98)
donor atoms reaching the ion	6.95 ± 0.40 (5.83 to 7.43)
shells cut per electrolyte	13,427 ± 7,716 (3,500 to 31,000)
block error of the mean field	0.00019 ± 0.00037 (0.00003 to 0.00253)
cost of keeping 16 members	0.0065 ± 0.0070 (0.0010 to 0.0394)

The whole series takes about three and a half hours on one CUDA device. The trajectories are 1.2 GB and are not kept in the repository; what is kept is one small structure file of 16 shells per electrolyte, which is all a model needs.

One number here is a statement about the force field rather than about the electrolytes. The 6.95 donor atoms at the ion is far above the 4.24 oxygens item BT4 measured on complexes relaxed with density functional theory. Part of the difference is that anion fluorine atoms count as donors here, and the donor count also rises with the molality (+0.769). The rest is the missing polarisation of a fixed-charge force field, which is known to bind a small cation to a carbonate too strongly. The number is recorded in `shells.json` rather than corrected.

The gate: do the sampled shells beat the single optimised complex

That is the whole question of the item, and `benchmarks/run_shells.py` answers it with seven models, two targets and two fold schemes. Five outer folds, the ridge and the field width chosen inside each fold. The two fold schemes ask different questions: random folds allow the same solvent composition at two molalities to fall on both sides of the split, and the by-composition folds do not, so that a model must predict a mixture it has never seen.

Four of the models are fields and three are controls. The controls are there because a field model that cannot beat the molality alone is not describing an electrolyte at all.

Model	What it is given
`conditions`	the molality, and nothing else
`composition`	the molality and the solvent weight fractions
`solvent`	the free solvent molecules as an ensemble, alignment-free kernel
`complex`	item BT4's single optimised complex, overlap kernel in the cation frame
`complex-free`	the same complex, alignment-free kernel
`shells`	the 16 sampled shells as one mean field, overlap kernel in the cation frame
`shells-free`	the same mean field, alignment-free kernel

The complex appears twice and the shells appear twice on purpose. The kernel is not part of the item's question, and item BT4 measured the alignment-free kernel as the better of the two for its own descriptor, so a comparison in one kernel only would answer nothing. The gate is therefore read twice: framed against framed, and alignment-free against alignment-free.

Each field kernel is mixed in equal parts with the kernel on the molality, at a fixed weight of 0.5 rather than a weight chosen inside the fold. Item M10 showed why: if the weight is free, every model may quietly discard its own field and land on the `conditions` model, and the comparison then measures nothing. A separate reading with the weight chosen in the fold is reported next to the fixed one, and it says how much of each result survives that freedom.

The activation energy, where the gate is met. Predicting the mean of the set costs 3.535 kJ/mol.

Model	Random folds, MAE (kJ/mol)	By-composition folds, MAE (kJ/mol)
`conditions`	2.983	3.035
`composition`	1.374	1.429
`solvent`	1.726	2.023
`complex`	2.001	2.424
`complex-free`	1.934	2.232
`shells`	2.236	2.903
`shells-free`	**1.612**	**1.685**

Paired over the same electrolytes, **the sampled shells beat the single optimised complex by 0.323 kJ/mol [-0.082, +0.786] on random folds and by 0.546 kJ/mol [+0.137, +1.038] on the harder by-composition folds**, in the alignment-free kernel. The by-composition interval is clear of zero and the random-fold one is not, and both of the variants with the kernel weight chosen inside the fold are clear of it (0.436 [0.057, 0.889] and 0.586 [0.133, 1.163]). So the item's gate is met on this target in three readings of four.

These numbers were revised by item D11, and the revision is instructive. Until then 72 % of these shells carried a **net charge of +1** in the electrostatic field: the phosphorus of the hexafluorophosphate anion is a valence state the Gasteiger scheme has no parameters for, so it was given a charge of zero while the lithium kept its +1. Correcting it makes the shell model slightly *worse* -- 1.456 to 1.612 kJ/mol on random folds -- and takes the random-fold gain over the single complex from 0.480 [0.096, 0.960], clear of zero, to 0.323 [-0.082, +0.786], which is not. The variants that choose the kernel weight inside the fold barely moved at all (0.436 to 0.436, 0.586 to 0.586), which says the weight search was already absorbing part of the fault.

The difference between the two targets was predicted before any of it was measured, and that is the strongest statement the item supports. The activation energy is the barrier an ion meets as it leaves one shell and enters the next, so it is the quantity a distribution of shells has a claim on. The conductivity at one temperature is set largely by how much salt is dissolved and how viscous the mixture is, and a shell geometry carries neither.

The conductivity, where the gate is not met. Predicting the mean costs 0.146 in $\log_{10}$ mS/cm. The shells reach 0.109 against the complex's 0.094 on random folds, and 0.103 against 0.103 on by-composition folds, and the paired interval contains zero both ways: −0.016 [−0.036, +0.004] and +0.000 [−0.018, +0.019]. There is no gain here to keep. One comparison on this target did change sign of verdict under item D11's correction: against the free-solvent model on by-composition folds the shells are now worse by 0.021 [0.002, 0.039], where the interval had held zero.

The frame costs accuracy, and it costs the ensemble much more than the single complex. In the cation frame the sampled shells are 0.621 kJ/mol [0.223, 1.003] worse than their own alignment-free version on random folds, and 1.230 [0.697, 1.792] worse on by-composition folds. For the single complex the same frame costs 0.068 kJ/mol, with an interval containing zero. This is the same direction item BT4 found and a much larger size. The reason is geometric: one complex has one coordination polyhedron and sits in the frame that polyhedron fixes, while sixteen shells of different shapes and sizes have sixteen polyhedra, and no single frame holds them all. The accuracy and the co-field, which needs the frame, therefore pull against each other in this item.

The composition is still not beaten. On the activation energy the shell field only draws level with the molality and the solvent weight fractions: −0.081 kJ/mol [−0.397, +0.256] on random folds and −0.159 [−0.502, +0.221] on by-composition folds, both intervals containing zero. On the conductivity the composition is ahead beyond the interval. This is the fourth battery item in a row in which a simple composition descriptor is at least as good as a field, and it should be read as a property of these targets rather than of the implementation: a conductivity is an average over a whole liquid, and a field describes one object in it.

Against the free solvent molecules the gain is not established: +0.118 kJ/mol [−0.284, +0.583] and +0.346 [−0.100, +0.870]. The shells beat the optimised complex, which is what the item asked, but an ensemble of free solvent molecules, which needs no simulation at all, is not clearly behind them.

One thing about how this reading was reached belongs here and not in a footnote. The `complex-free` model was added after the first run showed the framed comparison failing, and it is the model that turns the gate from unmet into met. The reason for adding it is stated above and does not depend on the result: comparing a framed ensemble only against a framed complex leaves out the better of item BT4's two models. But the decision came after the first numbers were seen, and a reader should weigh it as such. What argues against it being a lucky choice is that the gain holds in both fold schemes, holds with the kernel weight chosen inside the fold, and runs in the direction the item predicted in advance.

The co-field, and what it must be read against

The alignment-free kernel has no co-field at all, so this reading belongs to the framed model, the one that costs accuracy. Fitted on all 62 electrolytes with the activation energy as the target and evaluated on a grid. The donor atoms of these shells sit 2.14 ± 0.17 Å from the ion, and that is the number the last column is read against.

Field	Largest magnitude	Distance from the ion (Å)	Distance from the nearest donor atom (Å)
electrostatic	−3.51	3.03	0.81
steric	−0.41	5.04	2.85
hydrophobic	−1.54	5.00	2.86
hydrogen-bond acceptor	−1.23	4.22	1.92
hydrogen-bond donor	+0.00	16.12	13.72

The second distance is what makes the first one readable, and it was added for that reason. A point 3.03 Å from the ion and a point 2.14 Å from it are both inside the first shell, but only the second is where an atom actually is. Read together, the electrostatic field has its largest magnitude 0.81 Å from a coordinating atom and the hydrogen-bond acceptor field 1.92 Å from one, while the steric

and hydrophobic fields peak 5 Å out, which is the molecule tails and the second shell rather than the coordination sphere.

This reading was revised by item D11 and the revision is the point of that item. With the hexafluorophosphate anion carrying no charge, the electrostatic co-field peaked **+3.62 at 0.69 Å from the ion** -- on the cation itself -- and the hydrophobic field peaked +3.11 almost exactly on a donor atom. With the anion's charge restored the electrostatic peak is **−3.51 at 3.03 Å from the ion and 0.81 Å from a donor**: negative, and on the coordinating atoms rather than on the metal. The second reading is the chemically sensible one, and the first was an artefact of a charge the program had quietly set to zero. The hydrogen-bond donor field is the control that costs nothing: a carbonate and an ester have no hydrogen-bond donor, that field carries no weight anywhere, its largest magnitude is exactly zero, and the point it is reported at is a corner of the box 16 Å away.

What this item settles

- **The route works and is cheap.** A distribution of shells is one field, the kernel needs no new closed form, and 62 bulk simulations with their shell extraction cost three and a half hours.
- **It is kept for the temperature dependence and not for the conductivity.** The gate is met on the activation energy, in the alignment-free kernel, on the by-composition folds and in both fold-chosen-weight variants, and not on the random folds after item D11 corrected the charges; it is not met on the conductivity under any reading.
- **The co-field needs the frame, and the frame costs 0.6 to 1.2 kJ/mol here.** A caller who wants the picture should expect to pay for it; a caller who wants the prediction should use the alignment-free kernel.
- **A composition descriptor remains the baseline to beat on a property of a whole electrolyte**, and on these two targets it is not beaten.

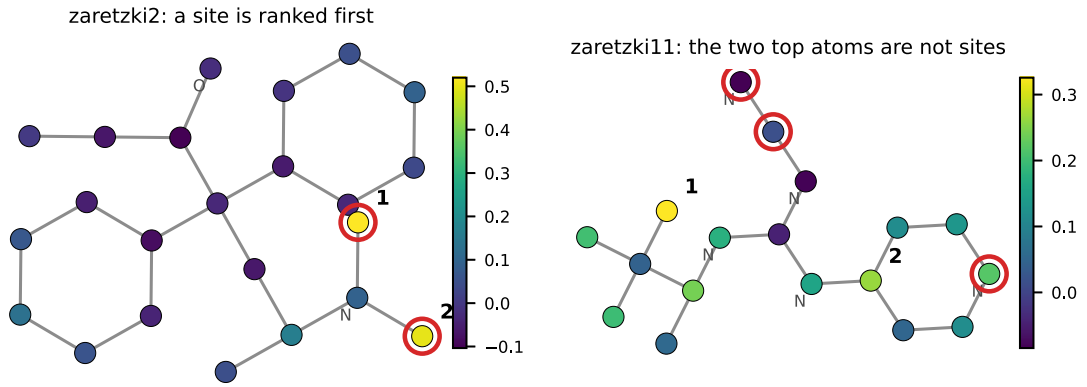
11.23 Which atom is attacked: a site of metabolism

Blocking a metabolic position is one of the standard moves in medicinal chemistry, and the question it asks is about one atom: of the twenty-odd heavy atoms of a drug, which one does a cytochrome P450 attack? That is the shape of target the two previous sections built the machinery for, and plan item MC8 is the measurement of it on drug-like molecules rather than on the small ones those sections use.

The data set. `examples/local/metabolism.sdf` holds 636 molecules, 1301 sites among 14 488 heavy atoms, from the benchmark of Zaretzki, Matlock and Swamidass [24] in the form the FAME.AL project publishes under the MIT licence. The target is one number per heavy atom: one if the atom is a site, zero if it is not. The sites are merged over the P450 isoforms, so the question is the general one -- where is this molecule attacked -- and not which enzyme attacks it. `examples/local/README.md` gives the selection and what was changed.

The measure is a ranking inside a molecule. A q^2 on a target of zeros and ones is not what a chemist needs. The question is which atom, so the measure is the one the site-of-metabolism literature uses: **in what share of the molecules is a true site among the one, or the two, atoms the model ranks highest**. The ranking happens inside each molecule, so the scale of one molecule's predictions cannot help or hurt another, and a molecule with no site asks no question. `localenv.top_hits` and `localenv.top_k_accuracy` are that measure.

out-of-fold score of every heavy atom; red ring: a measured site of metabolism; 1, 2: the atoms ranked highest



The site-of-metabolism model read on two molecules of the set, with every heavy atom coloured by its out-of-fold score under the best model of the table below, the count and the local field together; a red ring marks a measured site and 1 and 2 the two atoms the model ranks highest. Left, a molecule where the first-ranked atom is a site; right, one where neither of the two is, the kind of miss the hit rate counts. The scores are the model's own, so this is what `cmf local --rank` *prints, drawn.*

The baseline the item names is a model with no geometry: the same atom types, counted over **bonds** instead of over distance. `localenv.topological_counts` builds it -- the type of the centre, and the types one, two, three and four bonds away -- which is what an atom's topological fingerprint holds. It is the counterpart of `shell_counts`, and the two differ in exactly the thing a three-dimensional model is supposed to add.

`benchmarks/run_metabolism.py` measures all of it on ten folds that hold out whole molecules, with the ridge searched the same way for every row:

descriptor	the site is first	among the first two	area under the curve	against the count over bonds
count over bonds (2D)	53.6%	70.9%	0.859	--
count over distance (3D)	57.5%	73.9%	0.864	+3.0% [+0.3%, +5.5%]
local field, rho 0.15	54.4%	70.1%	0.843	-0.8% [-4.2%, +2.7%]
local field, rho 0.25	55.0%	71.1%	0.853	+0.2% [-3.3%, +3.8%]
local field, rho 0.5	55.2%	69.7%	0.854	-1.3% [-4.9%, +2.5%]
local field, rho 1	55.3%	69.3%	0.844	-1.6% [-5.2%, +2.2%]
local field, rho 2	53.0%	66.5%	0.837	-4.4% [-8.0%, -0.8%]
count and field, rho 0.25	60.8%	76.9%	0.875	+6.0% [+3.3%, +8.8%]
centre type and field, rho 0.25	59.9%	74.8%	0.860	+3.9% [+0.6%, +7.2%]

The field adds next to the count, not instead of it. On its own the local field is no better than the two-dimensional baseline: 71.1% against 70.9%, +0.2% [-3.3%, +3.8%]. Counting the same atom types over distance instead of over bonds does better on its own, 73.9%, +3.0% [+0.3%, +5.5%] -- geometry is worth something here, and it is worth it in the simplest form. The two together reach **76.9%**, which is +3.0% [+0.5%, +5.7%] over the count alone and +6.0% [+3.3%, +8.8%] over the baseline. That is the gate of the item, and it is passed by the pair rather than by the field. The two descriptions hold different things: the count says which atom types are at which distance, the field says how the charge and the shape are arranged there, and neither carries the other.

Why the field alone is weak here is worth stating, because it is a property of the descriptor and not of the set. Most of what the count contributes to that mixture is the name of the atom being asked about: replacing the whole count by its first block -- the centre's own atom type, one column per type -- already reaches 74.8%, against the 76.9% of the whole count. The autocorrelation of an environment never names its centre. It is built from distances between pairs of weighted atoms, and the centre is only the atom the envelope weights most. How easily an atom is oxidised depends first of all on what the atom is, so the invariant local field starts without the single most informative variable and has to recover it indirectly. On the shift set of the first section that costs little, because there every sample is a carbon; here the candidates are carbons, nitrogens, oxygens and sulfurs at once.

The width is chosen and not taken from the edge of the range. The field rows rise to rho 0.25, an envelope 2.0 Å wide, and fall on both sides of it. A wider envelope makes every atom of a molecule look alike, because the features are measured from the molecule as a whole and not from the centre; a narrower one leaves too little of the molecule inside the envelope to tell two carbons apart.

From the command line. This is the first set in the manual large enough for the exact kernel to be the wrong route: 14 488 atoms give a kernel of 1.7 GB. `--primal` solves the same model in the space of the 255 features instead (the previous section shows the two agree to round-off), and `--rank` reports the ranking measure next to the q^2:

```
cmf local examples/local/metabolism.sdf --target som --rho 0.25 \
    --primal --rank --reference
```

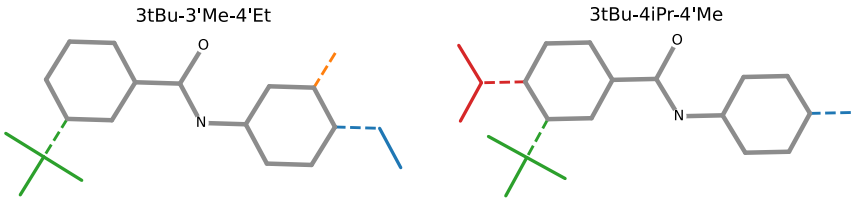
That runs in about fifteen seconds and prints both the field model and the atom-centred count, with the share of molecules whose site each puts first and among the first two.

What is not done. The mixture of the count and the field, which is the row that passes the gate, is in the benchmark and not in `cmf local`: the command fits one descriptor at a time. An isoform-resolved target is the other open question. The sites here are merged over the isoforms, so the target is a property of the atom; a target resolved by enzyme would ask about the fit of the molecule into one binding site, which is what a field describes best, and that is the test this set cannot give.

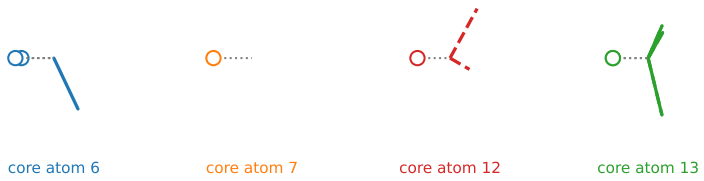
11.24 Topomer fields

Cramer's topomer idea [10], applied to fields. Cut the molecule at the bonds that attach its substituents to a common core and rebuild each substituent in a pose that is a **deterministic function of its topology**: canonical atom numbering, one embedding with a fixed seed, every rotatable torsion set to 180°, the attachment atom at the origin with its open valence along −x, and the remaining roll fixed by the principal axis of the heavy atoms. The same R-group is then always

placed in the same position, whatever molecule it came from — a test asserts the coordinates are identical, not merely close.



the substituents rebuilt by the rule: 3tBu-3'Me-4'Et solid, 3tBu-4iPr-4'Me dashed; open circle: the open valence, along −x; an oblique view



The topomer cut, on two N-phenylbenzamides of `examples/benzamides_synthetic.sdf` *that share an isopropyl group at the same position. Top: the two molecules with the core in grey, the substituents coloured by the core atom they hang from and the cut bonds dashed. Bottom: the same substituents rebuilt by the rule, each at its own attachment point with the open valence along −x, the first molecule's solid and the second's dashed. The shared group falls on the same atoms in both, which is what lets a fragment library be tabulated once; the groups the second molecule carries at two further positions occupy regions of their own.*

Two things make this fit CMF better than it fits CoMFA. There is no grid, so a fragment's field is just its atoms and a fragment–fragment kernel is an analytic number, which means the fragment library can be tabulated once and reused. And each attachment point keeps **its own region of space** (the substituent of position k is placed at $x = 100k$ Å, far enough that no Gaussian reaches the next site), so a molecule's kernel is exactly the sum of its per-position overlaps — which is what topomer CoMFA obtains by giving every position its own block of columns. A test checks that identity to 1e-9.

In the application it is *Tools → Replace molecules by their topomer substituents…*, and it is a **transformation of the data set** rather than a hidden switch: the molecules are replaced by their posed substituents, names, activities and the split are kept, and the data-set line says *[topomer substituents of …]*. That is the correct form for it, because a topomer model's co-field and contributions live in the substituent frames and not around the original ligands. Reload the file to get the molecules back.

Choosing the core matters more than anything else, which is why the program now chooses it differently. The maximum common substructure of a chemotype mixture is not a scaffold — on GPB it comes out as `CCO`, fused into every ring system, and cuts nothing off at all, losing every molecule. The default is therefore the **Bemis–Murcko framework of the largest chemotype**, the scaffold of one series, with every molecule that framework comes cleanly off forming the series; on GPB that is `C1CCOCC1` and 60 of the 66 molecules. The common substructure is still available for a set you know is congeneric (*maximum common substructure of the series* in the dialog, `--core-mode mcs`), and you can always give the core yourself as SMILES or SMARTS. Check the cuttable fraction the program reports either way: a core fused to the rest of the molecule cannot be removed without opening a ring, and what is left is not a substituent.

11.25 What the three routes are worth, measured

Every column below uses the same molecules, fields, α and λ grids and folds.

Collection	Series	Aligned q^2	Topomer q^2	The cut costs	The pose costs
Sutherland, curated frame	8	**+0.445**	+0.343	+0.035	+0.066
Mittal, automatic frame	31	+0.200	**+0.261**	-0.027	-0.033
Corrosion inhibitors, neutral	17	+0.248	**+0.307**	+0.024	-0.083
Corrosion inhibitors, 1 M HCl	17	+0.303	**+0.358**	+0.037	-0.092

The last two columns separate the two things the topomer construction does at once, by way of a control that keeps the same cut substituents at the same separated sites but leaves each one exactly where the alignment put it. **What the pose rule costs depends entirely on the alignment it replaces.** Against the curated Sutherland frame it costs 0.066 q^2; against an automatic superposition, or against the frames of the corrosion series, it *gains* 0.033 to 0.092, because a canonical pose is more consistent than an arbitrary embedded conformation. The **cut** costs 0.02 to 0.04 on average and is the expensive half only where the scaffold carries information — separate sites discard the scaffold's own field and the geometry between one substituent and another, which matters when the arms' mutual placement is the pharmacophore (0.21 on thrombin, 0.14 on thermolysin) and costs nothing or even helps when the scaffold is constant and the decorations small (BZR gains 0.102 q^2, DHFR 0.054).

Practical guidance. With a trustworthy curated alignment, keep it: the geometry adds about 0.25 to q^2 over counts. With an automatic alignment or none, prefer the frame-free routes — topomer for a congeneric series with a clear scaffold whose R-groups come off at acyclic single bonds, and the invariant kernel for anything pooled, multi-scaffold or ring-fused. Always report the count baseline beside the model, so that the value of the geometry is visible rather than assumed.

11.26 Heterogeneous series, step by step

When a set holds several chemotypes with no one common substructure, the *Tools* → *Heterogeneous sets* submenu is a four-step workflow:

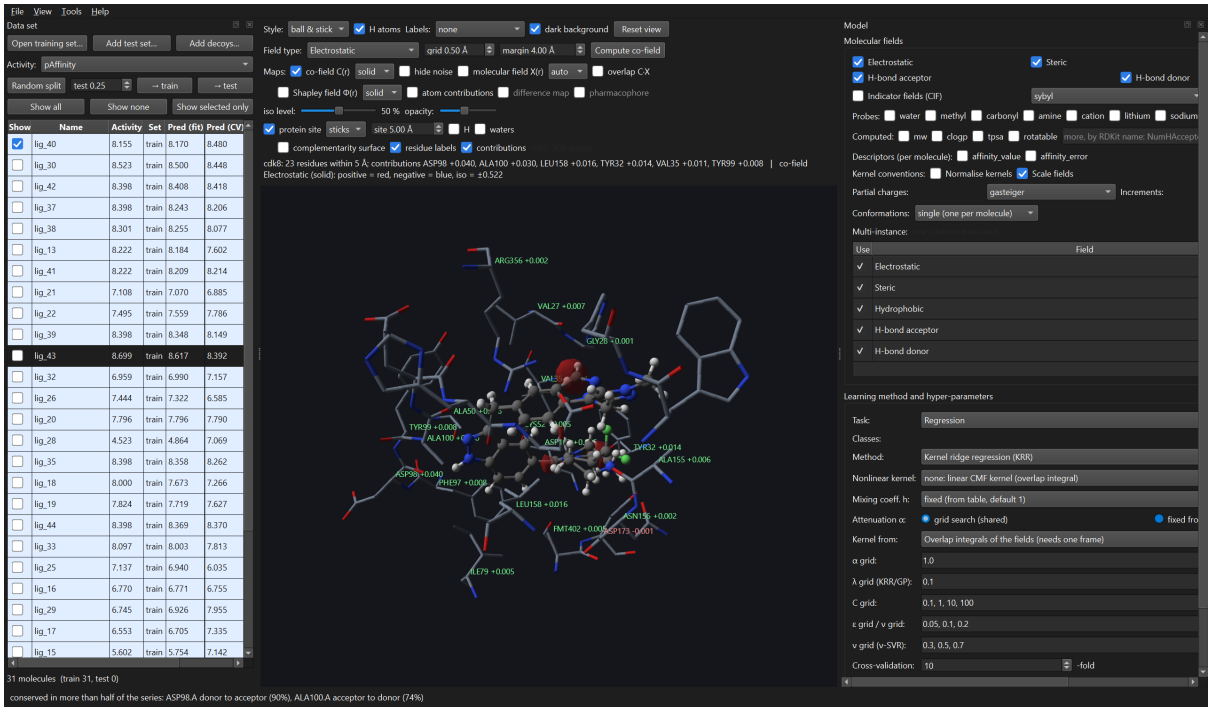
1. **Find the chemotypes** — Bemis–Murcko frameworks, optionally generic, with small groups merged so that leave-one-chemotype-out does not degenerate into leave-one-molecule-out.
2. **Align within each chemotype** — a superposition that is meaningful, because the members of one cluster do share a substructure.
3. **Place the chemotypes in one frame** — each cluster positioned by maximising field similarity to the others, which is the best that can be done when no common substructure exists.
4. **Leave one chemotype out, with the alignment-free baseline** — the reliable validation, reported beside the count kernel. Random folds overstate q^2 on such sets by 0.11 to 0.47 on the five chemotype benchmarks here, which is why this step exists.

12 Protein pockets

12.1 Getting a pocket on screen

File → Fetch protein from the PDB... downloads an entry from RCSB and caches it under `~/.cmfqsar/pdb`, with the dialog pre-filled with the identifier the data set names. *File → Load protein structure...* reads a local PDB or mmCIF file instead. The reader is deliberately independent of RDKit, because crystal structures carry alternate locations, missing elements and non-standard residues that a chemistry toolkit would refuse to sanitise, and none of that matters for drawing.

The free-energy benchmark sets in `examples/openff` and `examples/schindler` give their ligands in the frame of a reference crystal structure and name it per record, so **nothing has to be superimposed**: opening such a set brings its pocket with it from `examples/pockets/`, which holds the residues within 8 Å of any ligand, taken whole, without waters and without the co-crystallised ligand. The *protein site* controls above the 3D view then draw the residues within a chosen distance as wire frame or sticks, with hydrogens, waters and residue labels optional.

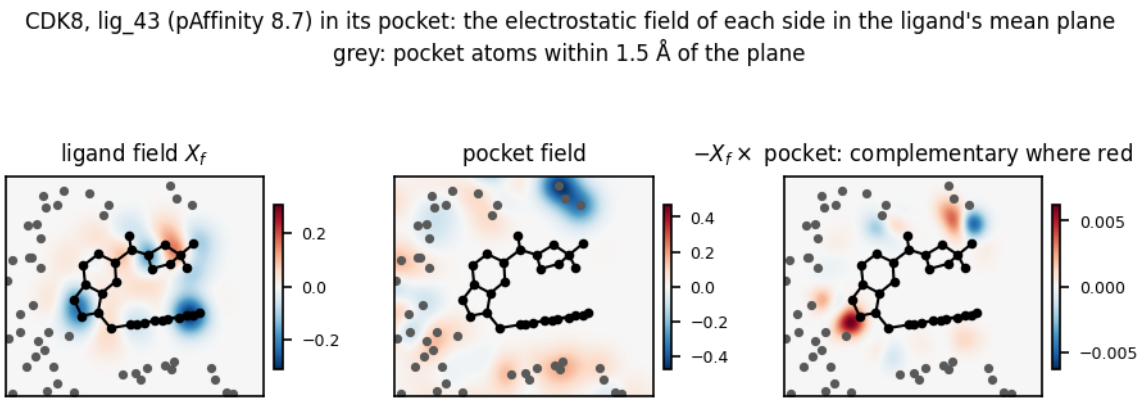


The application with the CDK8 series of `examples/openff` *and the binding site of its reference crystal structure, which the set brings with it from* `examples/pockets`*: the most active ligand in ball and stick, the residues within 5 Å of it as sticks, each labelled with its name and with what it contributes to the ligand's complementarity, green where it helps and red where it does not, and the electrostatic co-field of a kernel ridge model as isosurfaces. The line above the view lists the residues of the site and the largest contributions, and the status line names the contacts conserved across the series. Taken by* `docs/make_screenshot.py`*, which drives the application through these steps.*

12.2 Complementarity as numbers

A pocket is a set of atoms, so the same parameterisation that builds a ligand's fields builds the protein's, and the overlap of two Gaussian fields has the same closed form on either side. The *Complementarity* tab (CLI: `cmf_cli.py complement`) turns that into four answers:

- **How complementary a molecule is to the pocket** — overlap integrals between ligand and pocket fields, exact and without a grid, each divided by the two norms so that it is a cosine in $[-1, 1]$ and does not grow with molecular size. Complementarity is not similarity: the electrostatic term carries a minus sign, because a positive ligand region should meet a negative protein region; the hydrophobic term carries a plus; the hydrogen-bond terms pair the donors of one side with the acceptors of the other. Steric overlap is reported separately as *burial*, because bulk against the protein is contact and clash at once, with the number of clashing heavy-atom pairs beside it.
- **Where on the molecule it holds** — the same quantities on a smooth van der Waals surface give a value per vertex, drawn as a green-to-red surface, with a correlation per field over the surface. Electrostatics uses potentials rather than Gaussian densities here, because the potential is what the other side feels.
- **Whether the co-field agrees with the pocket** — the model's own map against the protein, by sign agreement, cosine or correlation. This is the strongest external check a CMF model can be given: the co-field was fitted to activities and never saw the protein.
- **Which residues say so** — the same scores, resolved by receptor residue.



The electrostatic field of the most active CDK8 ligand of `examples/openff` *and of its pocket, both in the ligand's mean plane with the pocket atoms near that plane in grey, and their product with the sign convention of the complementarity score: red where a positive region of one side meets a negative region of the other. The two fields are built by the same parameterisation, and the overlap of the third panel over all of space, divided by the two norms, is the electrostatic term of the score.*

Which residues say so

An overlap integral is a double sum over atom pairs, so keeping the per-atom column sums instead of the grand total costs nothing and is exact: every term of the score above splits over the residues of the pocket and adds back to it. One cosine becomes an **interaction profile**, a table of residues against interaction types.

This is worth having because a global score cannot say which contact a pose has lost, and neither can the ligand RMSD — a pose can move very little and still give up the one interaction that mattered. Across a series the profiles also average into a **consensus**: an interaction seen in most of the complexes is a property of the target, while one seen in a single complex is more likely a property of that ligand, which is the distinction a pharmacophore query needs.

On the ten CDK2 ligands of `examples/openff/cdk2.sdf` against their pocket, every molecule reaches 18 of the 53 residues, and the contacts conserved across the whole series are Glu81 donating to an acceptor, Leu83 accepting from a donor, and Ile10 and Leu134 by burial — the hinge and the hydrophobic floor, recovered without being told that either exists.

In the application, tick **by residue** beside *surface scores* before pressing *Compute*. The table under the Complementarity tab then lists every residue within 4.5 Å of the selected molecule, one column per interaction type, sortable by any of them, with the share of the series that shows each residue's interactions in the last column; the header says how much of the whole score those residues carry, the rest being the tail of the Gaussians over the residues behind them. Tick **contributions** beside *residue labels* in the protein-site controls and the 3D view writes each residue's total on its alpha carbon, green where it helps and red where it does not.

From the command line:

```
python cmf_cli.py complement data.sdf --structure pocket.pdb --residues --profile-out profiles.csv
```

`--residues` prints the strongest contacts of each molecule and the conserved pattern of the series, and scores every molecule against that consensus by cosine and by Tanimoto over the thresholded bits; `--reference NAME` compares against one molecule's profile instead of the consensus, `--residue-threshold` sets what counts as present, and `--profile-out` writes one row per molecule and residue.

12.3 A macrocyclic host as a pocket

`complement` was written for proteins: a structure with residues, and a score read residue by residue. A cyclodextrin, a cucurbituril or a calixarene is a pocket too, and comes as a molecule file with no residues at all. The program now reads one (plan item SC3): `read_molecule_host` takes an SDF, MOL, MOL2 or PDB file and **names the host's repeating units**, which are its small ring systems -- one glucose of a cyclodextrin, one glycoluril of a cucurbituril -- with every atom outside a ring given to the unit it hangs on, and the macrocycle that joins them left out of the grouping, since it is not a unit. Everything the pocket machinery does per residue is then done per unit.

`cavity_frame` gives the middle of the ring of units and the direction in which they vary least, which is the axis of the channel. `complement --place-in-cavity` uses it: every molecule is put in the cavity, its centroid on the centre and its longest axis along the channel, which is how a guest threads a cyclodextrin, and is then refined there by the local rigid-body placement of `cmfqsar.docking`. Without it the guests are scored where they happen to sit, which for a host read from a file is nowhere near the site.

```
python examples/make_host_structures.py
cmf complement examples/cyclodextrin/beta_cyclodextrin.csv \
    --structure examples/cyclodextrin/hosts/beta_cyclodextrin.sdf --place-in-cavity
```

The structures. `examples/make_host_structures.py` embeds a host from its SMILES and keeps the most symmetric of forty conformations -- a cyclodextrin is a ring of identical sugars, so symmetry is the rule, and it uses nothing about the guests. The three it builds have cavity radii of 2.6, 3.0 and 3.9 Å for the six-, seven- and eight-sugar rings, against published values near 2.4, 3.1 and 4.0, so the cavity it offers is the right size before any guest is scored.

What it is worth, measured

This is the largest set a complementarity score has been given: one host, one cavity, and 1162 guests with a measured constant, where no protein set in the repository offers more than a few dozen ligands per pocket. `benchmarks/run_host_pocket.py` places every guest and correlates the score with log K.

host	units	guests	score vs log K	burial vs log K	heavy atoms vs log K	score, size held constant
beta	7	1162	-0.348	-0.077	+0.348	-0.285
alpha	6	827	-0.206	-0.102	+0.184	-0.160
gamma	8	292	+0.025	-0.022	+0.383	-0.072

host	acceptor to donor	donor to acceptor	electrostatic	hydrophobic
beta	-0.285	-0.281	-0.104	-0.405
alpha	-0.217	-0.188	-0.028	-0.277
gamma	+0.037	-0.073	-0.174	+0.064

The score does not predict the constant, and on the largest table it points the wrong way. On beta-cyclodextrin the correlation is -0.348; the guest's heavy-atom count alone gives +0.348, and with the size held constant what the score adds is -0.285. On beta and alpha the hydrophobic term, which is the one a hydrophobic cavity should reward, is the most negative of the four; on gamma, whose cavity is wide enough that a guest of this size need not touch its walls, every correlation is near zero.

That is the limit of a one-body score, not a fault in the structures. Binding to a cyclodextrin is driven by what leaves the cavity: the water inside it is poorly coordinated, and a guest that displaces it gains free energy that no overlap between the guest's fields and the host's can contain. A score computed from the two molecules alone has no term for the solvent it displaces, and this measurement is the clearest statement of that the repository has, because nothing else here has a thousand ligands in one pocket to say it with. Read the score as a check on a pose -- does this molecule fit, does it clash -- and not as an affinity.

12.4 Docking

Tools → Dock into the pocket... places the conformations of a molecule in the site by rigid-body optimisation. It is a convenience for getting a pose into the right frame, not a scoring function; the program's purpose is what happens after the poses exist.

13 Three series, worked through

Everything above is a tool. This part uses them, on three series taken from the free-energy benchmark collections because those come with the crystal structure their ligands were posed in, which is what makes the complementarity step possible at all. They were chosen by measuring every set in `examples/openff` and `examples/schindler` that has a pocket and keeping the three best models among different target families; `examples/case_studies.py` reproduces every number below and `benchmarks/results/cases/table.md` records them.

case	target	n	activity range	α	q²cv	R²pred	worst of 30 shuffles
`ptp1b`	protein tyrosine phosphatase 1B	22	3.77	3	+0.499	+0.147	**+0.539**
`mcl1`	Mcl-1, a Bcl-2 family protein	25	3.05	0.2	+0.548	+0.581	+0.098
`cdk8`	cyclin-dependent kinase 8	33	4.18	3	+0.496	+0.180	+0.026

The protocol is the same for all three. The attenuation and the ridge are chosen by cross-validated grid search on two thirds of the series, and the other third — every third molecule by activity rank, so that it spans the range instead of sampling one end of it — is held out. The model whose co-fields are then read is refitted on the whole series with those hyper-parameters, because a co-field read from two thirds of a series is a co-field of two thirds of a series.

One thing to say before any of the three. What follows is not a chemist's reading of these series. The development plan asks for these cases to be judged by a medicinal chemist, and that judgement has not been made; what is here is every number such a judgement would need, with the places where the program contradicts itself left in rather than tidied away.

13.1 The case that does not survive its own check: `ptp1b`

Twenty-two ligands of a phosphatase, spanning 3.77 log units. The model looks good: q²cv +0.499, and on the two thirds it was selected on, +0.824. Then the activities are shuffled thirty times and refitted, and **the best shuffle reaches +0.539** against the real +0.499, for p = 0.065. On twenty-two molecules with five fields, a q² of one half is what chance produces.

So this case is not interpreted, and it is kept for exactly that reason: it is what *Is it chance? y-randomisation* is for, and it is the first thing to run on a series of this size. Everything the program would have said is still computed, and two parts of it are worth noting as warnings rather than findings:

- the fit is R² +0.975 against a cross-validated +0.499. A model that reproduces its own training set almost exactly and half the variance of a molecule it has not seen is memorising;

- the external third gives R^2pred +0.147 while the selection fold gave +0.824. The gap between those two numbers is the clearest single sign of a hyper-parameter chosen on too little data.

One measured result does survive, because it needs no model: of the five fields, the one this series does **not** need is the electrostatic one. Removing each field in turn and refitting costs electrostatic -0.009, steric +0.113, hydrophobic -0.016, hba +0.043, hbd +0.025 in q^2cv — the steric field carries what there is, and the electrostatic field can go without loss. A phosphatase site is charged; the series does not care.

13.2 Shape and lipophilicity in a groove: `mcl1`

Twenty-five ligands of a protein-protein interaction groove. q^2cv +0.548, R^2pred +0.581 on the 9 held out, and the best of thirty shuffles reaches only +0.098 ($p = 0.032$). This one can be read.

Which fields. Removing each in turn: electrostatic +0.032, steric +0.062, hydrophobic +0.071, hba +0.037, hbd -0.077. The hydrophobic and steric fields carry it and the hydrogen-bond donor field is worse than useless — removing it *improves* the model by 0.077. For a groove that binds a helix face, shape and lipophilicity being the answer is the expected answer, and it is also the measured one.

What the co-fields say. At the noise level the activities imply (0.559 log units), the share of each co-field that stands more than two standard deviations from zero is electrostatic 2%, steric 23%, hydrophobic 4%, hba 0%, hbd 0%. Only the steric map has a substantial part that the measurement error could not have produced; the electrostatic map, at 2 %, is almost entirely noise and should not be drawn. The pharmacophore read from what survives has 5 features: bulk +44.64; hydrophobic +29.18; positive +4.25; polar -0.71. A large favourable bulk region, a hydrophobic region inside it, and an exclusion volume of almost the same weight (-33.21) — which is the series saying that the groove has a size and that exceeding it costs as much as filling it earns.

Where one prediction comes from. The most active ligand (lig_53, measured +7.26, predicted +7.15) decomposes by rings and the pieces between them into CH4 at 37 +2.01, Cl1 at 36 +1.63, ring 2 -1.56. The two largest positive contributions are a small aliphatic piece and a chlorine, and one of the rings contributes *negatively*: against the scaffold baseline, that ring is not what makes this molecule good.

What the pocket knows. Scoring all twenty-five ligands against the crystal site (1007 atoms of 61 residues) gives a mean complementarity of +0.201, and its correlation with the measured activity is Spearman +0.254. But the steric overlap alone — burial, how deep in the groove the ligand sits — correlates at **+0.698**. The score that averages five complementarity terms is three times worse than one of its terms. On this target, how far the ligand reaches into the groove is most of the structure-activity relationship, and a composite score hides it.

13.3 An acceptor-driven kinase: `cdk8`

Thirty-three ligands spanning 4.18 log units, the widest range of the three. q^2cv +0.496, best shuffle +0.026, so the model is real; but R^2pred +0.180 on the 11 held out, and a training fit of R^2 +0.998 at $\lambda = 0.01$, which is a model interpolating its training set.

Which fields. electrostatic +0.043, steric -0.015, hydrophobic -0.024, hba +0.099, hbd -0.061. The hydrogen-bond acceptor field is the one that matters, and three of the five can be removed with no loss. For a kinase, an acceptor-driven model is the expected shape of the answer — the hinge donates — and this is the only one of the three cases where the dominant field is a polar one.

What survives the noise. electrostatic 4%, steric 6%, hydrophobic 4%, hba 7%, hbd 0%, at a noise level of 0.697. These are the smallest shares of the three cases: the model predicts, and almost none of its map is distinguishable from what the measurement error could have produced. The pharmacophore has 10 features (bulk +1.34; acceptor_bad -1.18; positive +1.09; positive +1.00), and the second and fifth of them are *unfavourable* acceptor regions — places where an acceptor lowers the activity. A pharmacophore from this series would consist mostly of places to avoid.

The pocket says nothing. Complementarity +0.002 with burial 0.014: these ligands barely overlap the trimmed site at all, and the correlation of the score with activity, +0.326, is on a quantity that is almost zero for every ligand. Compare `mcl1`, where burial is 0.588. The difference is not the method but the pocket file: a site trimmed at 8 Å around a shallow, solvent-exposed hinge leaves most of a ligand outside it.

13.4 What the three have in common

- **Every one of them loses at least one field without losing accuracy**, and in two of the three the field a chemist would expect to matter is not the one that does.
- **The share of a co-field that survives its own error bar is small**: 23 % at best, 6 % on `cdk8`. Drawing an unmasked co-field from twenty-five molecules shows mostly noise, which is what *The uncertainty of a co-field* exists to prevent.
- **A composite score can be worse than one of its parts** (`mcl1`: +0.254 against +0.698).
- **The check comes first.** One of the three does not beat a shuffle of its own activities, and no amount of interpretation would have revealed that; only the shuffle did.

13.5 One command for the alignment: `cmf align`

Alignment was spread over flags of `build` and `predict` and the `hetero` workflow, which made the criteria the program has look like unrelated features. They are one question — where do these molecules go — asked five ways, and `align` asks it:

```
cmf align data.sdf --by mcs    --out aligned.sdf          # on the common substructure
cmf align data.sdf --by field  --reference 0 --out aligned.sdf
cmf align data.sdf --by variance --model m.cmf --max-rmsd 2 --out aligned.sdf
cmf align data.sdf --by pocket --model m.cmf --pocket site.pdb --out aligned.sdf
cmf align data.sdf --by plane  --pose auto --refine plane --reference 0 --out onsurface.sdf
```

`--by`	what it maximises	what it needs
`mcs`	the overlap of the maximum common substructure with a reference molecule	a reference, or it takes the first of the most active quarter
`field`	the CMF field similarity to a reference molecule, a model's training set, or the set itself	nothing, but `--restarts` decides how hard it tries
`variance`	the confidence of the model's own prediction — the criterion of *Alignment by prediction confidence*	a model (`--model`); `--flexible` adds the torsions, and `--any-orientation` with `--max-rmsd 0` searches orientations as well as placements

`--by`	what it maximises	what it needs
`pocket`	that confidence with a complementarity term, $\sigma^2 - w$ [fit − clash]	a model and a structure; see the warning in *Alignment inside the pocket*
`plane`	nothing, by default: it poses each molecule on a surface $z = 0$ by rule (`--pose flat` or `upright`); with `--pose auto` and `--refine`, the field similarity to one reference pose, moved only as a surface allows	a reference whose input pose is trusted, for `auto` and `--refine`; see *Posed by rule: the surface frame without the slab*

Every mode prints what it did per molecule and then how far the set moved, and writes nothing unless `--out` is given — an alignment that is not kept is a measurement, and saying so avoids the surprise of a command that silently rewrote a file. `--max-rmsd` limits how far a molecule may travel from the pose it arrived in, which is the guard the confidence criterion needs: without it a molecule is free to leave the series entirely and sit wherever the variance happens to be lowest. `--judge` adds the mean field similarity within the set afterwards, which is the one number that says whether the set now shares a frame.

What `align` does **not** do is tell you whether the result is better. Only *A protocol that can judge an alignment* can, and `benchmarks/run_poses.py` is where that protocol lives.

13.6 Co-fields without the application: `cmf cofield`

The co-fields were readable only from the 3D view. `cofield` writes them as Gaussian cube files, which any molecular viewer opens, and prints the numbers that go with them:

```
cmf cofield model.cmf --out-prefix maps --spacing 0.5 --z 2
cmf cofield model.cmf --kind overlap --molecule candidate.sdf --csv agreement.csv
```

`--kind cofield` is the model's own map, one cube per field. `--kind molecule` is one molecule's field in the model's frame, and `--kind overlap` is that field weighted by the co-field — where the molecule and the model agree, which is the picture that explains a prediction. `--z` masks the map where it lies closer than that many standard deviations to zero (*The uncertainty of a co-field*), and the printed share of the covered grid that survives is the quickest way to see how much of a map is worth looking at:

```
electrostatic    peak     0.6146 (signed    -0.6146)   7% of the covered grid above 2 sd
hydrophobic      peak     0.8206 (signed    +0.8206)   18% of the covered grid above 2 sd
```

On a classification or one-class model there is nothing to mask with. The band propagates noise in the activities and a class label carries none, so `--z` is reported as ignored and the map is drawn unmasked. The map itself is the same expression for every learner whose score is a kernel expansion, the kernel Fisher discriminant included.

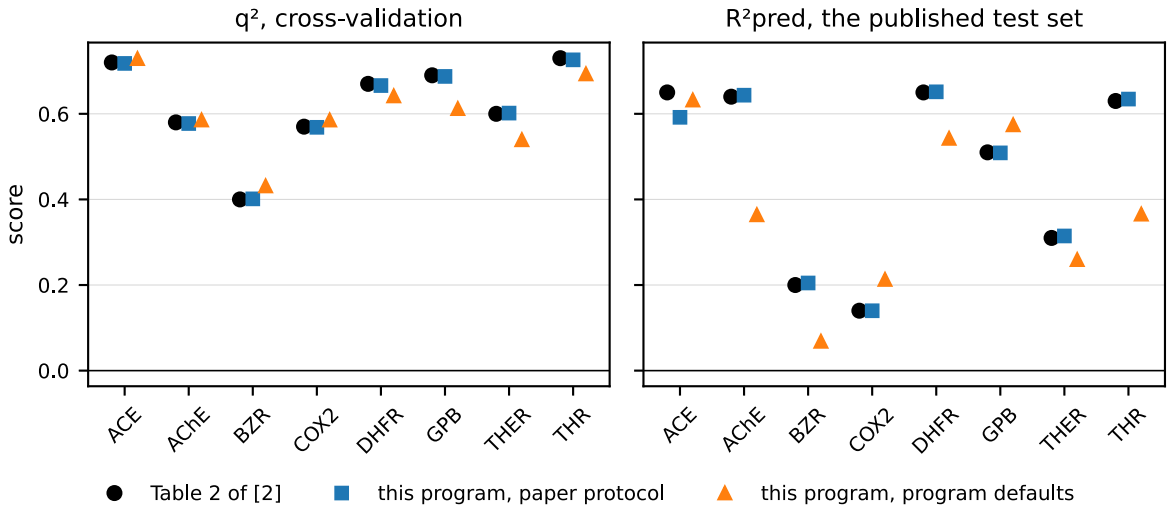
A model with a nonlinear kernel layer has no single co-field, and `--kind cofield` says so. Five kinds serve it instead (*The co-field of a nonlinear model: one per molecule* and the section after it): `--kind local --molecule FILE` is the co-field of the linear model tangent to it at that molecule; `--kind spread` writes the mean of those over the training molecules and their standard deviation, and prints the non-linearity index of every field; `--kind pair --molecule FILE --probe-atom K` is the pair-interaction field with the fixed probe on atom K; `--kind curvature` the second derivative along

one probe at every point; `--kind modes` prints the atom interaction matrix of the molecule as its leading modes and writes it with `--csv`.

13.7 The benchmarks from the tool: `cmf benchmark`

`benchmarks/run_all.py` is the one entry point for every measurement in this manual, and `cmf benchmark` is the same program reached through the tool:

```
cmf benchmark --list            # what there is, what it needs, how long it takes
cmf benchmark --check           # do the tables and the documents match the stored results?
cmf benchmark alignment poses   # run these two
```



The eight Sutherland sets: q^2 of the cross-validation and R^2pred on the published test sets, from Table 2 of [2] and from this program with the protocol of [2] and with its defaults. The numbers of [2] are reproduced by its protocol; the defaults differ most on R^2pred, where the test sets are small.

It needs the source checkout, because the benchmarks and their data are not installed with the package; the command says so rather than failing obscurely if they are missing.

What `--check` checks. It is the statement that the documents of this repository say what the measurements say, and it answers five questions:

question	what fails it
Is every result table what its result files give?	a table rebuilt from the stored results differs from the committed one
Is every spliced passage current?	a passage between `<!-- tag -->` markers in a document differs from what the result files give
Does the manual name every measurement?	a benchmark is registered that the manual mentions nowhere, by name, by its results folder or by its script
Is the typeset manual the manual?	`docs/manual.md` has been edited since `docs/CMF_manual.pdf` was built
Is the example data the data that was measured?	a file under `examples/` differs from its hash in `benchmarks/manifest.json`

The third and fourth questions are the part of the working rule below that a program can decide. The last one is what makes a published number traceable: the table, the document, the result file and the molecules all have to agree, or the command fails and says which of them does not.

A PDF cannot be compared with a rebuild of itself, because it carries the date it was built on, so two builds of one source are different files. `python docs/make_manual.py` therefore writes `docs/CMF_manual.sha256`, the hash of the Markdown it read, and `python docs/make_manual.py --check` compares that hash with the file as it stands. The question it answers is the useful one: has the manual been edited since the PDF was made?

One command per stored result. Some benchmark scripts are run several times with different options: `benchmarks/run_hammett.py` is run six times, once per charge model, conformer set and conformer weighting, and `benchmarks/run_hops.py` twice, once for the measurement and once for the fine co-field profile the figures need. For such a script, two rules keep the results traceable (plan item G6). Every option that changes a number enters the names of the files that run writes, as a tag: `summary_ens_boltzmann.csv` is the conformer ensemble with Boltzmann weights, and a run over one target writes `summary_sigma_m.csv`, so no run writes over another's numbers. And the run writes its own command to `run<tag>.json` beside the summary, which the result table prints above each table, so a reader can repeat any block and a stored file whose command is unknown says so. `tests/test_hammett_runs.py` and `tests/test_hops_runs.py` hold the registry in `benchmarks/run_all.py` and the stored files to these rules: one documented command per stored result file, no two commands writing the same file, and no stored result without a command. Before this was checked, two of the four commands the registry listed for the Hammett benchmark were the same command, one wrote a file the repository does not keep, and four of the six stored results came from commands the registry did not list; the hop benchmark gave every run the same file names, so the quick look its own usage lines offer wrote over the stored measurement, and the fine profile the manual draws was rebuilt by no command the registry listed. The registry also names the command that prepares an input a benchmark only reads: `python benchmarks/prepare_hops.py` fetches the public hop set from the Materials Project. `cmf benchmark --compute` does not run such a command, because it needs the network and its result is kept in the repository; `cmf benchmark --list` prints it.

The working rule. A feature lands with its manual section and its benchmark script in the same commit (plan item G4). Nothing can check that a section was *written*, and these two checks make the omissions that matter visible: a measurement nobody can find, and a PDF that disagrees with the Markdown beside it. `tests/test_documents.py` runs both in the test suite, so the rule holds on every commit and not only when someone remembers to run the benchmarks.

What depends on the machine. The test suite runs on Ubuntu, Windows and macOS, and the first runs on machines other than the author's showed which quantities a test may assert and which it may not (plan item A2). Three are worth knowing if you run the suite yourself or write a test. *An embedded conformer is not the same geometry everywhere.* RDKit's distance-geometry embedding takes a seed, but its random stream is not the same in every build, so a geometry built in a test is reproducible on one machine and not between machines. A test that measures a geometry therefore reads a stored structure; the width of the cavity of alpha-cyclodextrin, for example, ranged over the embedding seeds from 0.88 A to 3.14 A. *A maximum located by comparing values is worth half the digits.* Near a maximum a smooth function is flat, so an argument found by comparing values of the function carries about half of the available digits, that is eight and not sixteen. Where the answer has to be the same everywhere, as the canonical frame of a polybenzenoid hydrocarbon does, the program solves for the derivative instead. *A pseudo-inverse is exact up to the conditioning.* `nystrom_kernel` drops the eigenvalues below `NYSTROM_CUTOFF = 1e-12` of the largest, so with every molecule a landmark it reproduces the kernel to about `eps / NYSTROM_CUTOFF`, which is two parts in 1e5 and not the 1e-6 a test first asked for. A test of such a round trip takes its bound from the eigenvalues of the kernel it is given. What was *not* a cause, although it was the first guess: the linear-algebra library. The runners and the author's machine use

the same OpenBLAS, which the numpy wheels carry.

One more thing a test that drives the window has to respect. Every window, panel and chart of the suite belongs to **one** Qt application, which `tests/conftest.py` creates once and holds for the whole run; a test that needs it asks for the `qt_application` fixture. A test must not build its own with `QApplication.instance() or QApplication([])`: that call returns the one application that already exists, and the fixture it is stored in then owns it, so the end of that module deletes the application under every module that follows. Qt reports it as "shared QObject was deleted directly" and the process crashes while shutting down, after the last test has passed, which is a failure no test can catch. `pytest_sessionfinish` asks after the last test whether the application is still there and fails the run with the reason if it is not.

Taking Qt apart when Python shuts down is not reliable on every machine, and that is after the last test too: on some runners the process ended with a segmentation fault, on others with exit code 1 and no message, on a suite that had just reported every test passed. A run that has built a Qt application therefore **ends with the status pytest arrived at**, in `pytest_unconfigure`, instead of shutting Python down. A failing run still fails, since that status is what is passed on; what is skipped is only what Python does after the report is written. If you need the ordinary shutdown — for a tool that writes its output at the very end, such as coverage — set `CMF_TESTS_FULL_SHUTDOWN=1`.

14 Command-line reference

Everything the application does is available without it, which is what makes the program scriptable for a benchmark or a cluster run. The general form is

```
python cmf_cli.py <subcommand> <dataset> [options]
```

and `python cmf_cli.py <subcommand> --help` prints the full option list; what follows is an overview, not every option.

Subcommand	What it does
`build`	build, cross-validate, test and save a model
`build-crystals`	the same on periodic structures (CIF, POSCAR, extended XYZ, and every other format ASE reads; the list is in the section on crystals of chapter 4)
`diagnose`	ask a data set what it can support, before modelling it
`align`	put a set in one frame: by substructure, by field similarity, by prediction confidence or in a pocket
`predict`	predict with a saved model, optionally aligning by confidence first; a model fitted on crystals takes crystals
`screen`	score a library against a saved model, reading it in blocks, so its size does not matter
`report`	one self-contained HTML page describing a saved model
`cofield`	the co-fields of a saved model as cube files and as numbers
`benchmark`	run, list or check the benchmarks of the checkout
`suggest`	which posed candidate to make next (expected improvement and the rest)
`design`	what to put at a position of a parent, ranked by the model
`transfer`	compare models by their co-fields: do they ask the same thing?
`mixing`	the posterior over the field weights: which fields the series needs
`hetero`	the heterogeneous-series workflow: cluster, align, place, leave-one-chemotype-out
`topomer`	model a series by its substituents, posed by rule
`map`	chemical-space map and activity landscape (kernel PCA + GTM)
`explain`	Shapley decomposition of one prediction, for any kernel
`mil`	multi-instance learning over conformations
`pharmacophore`	read a pharmacophore out of a fitted model's co-fields
`difference`	what one molecule's fields have that another's do not
`complement`	field complementarity with a protein pocket
`compare`	are two sets the same population? (MMD, with a witness)
`cluster`	cluster a set by its field kernel (spectral clustering)

Subcommand	What it does
`atoms`	a target that belongs to the atoms: what a file carries, and how it would be split
`local`	a model over atoms: the field around an atom against a count around it
`lowrank`	fit in the space of a few hundred landmarks, for a set too large for the exact kernel
`isosteres`	which groups are interchangeable at a position, by the co-field
`restriction`	score a ring closure: how much of the field that matters stops moving
`unalign`	give every molecule a random pose, the control experiment

14.1 The options of `build`, by group

Data. `--activity PROP`, `--activity-file TABLE`, `--test-file FILE`, `--test-fraction F`, `--ensemble-by KEY`.

Fields. `--fields electrostatic,steric,hydrophobic,hba,hbd[,prop:name]`, `--cif`, `--cif-scheme sybyl|element|element+aromatic`, `--cif-min N`, `--cif-types LIST`, `--pharm`, `--pharm-min N`, `--pharm-features LIST`, `--charges gasteiger|mmff|file`, `--param rdkit|tripos|tripos-legacy`, `--prop-missing error|zero`, `--increments FILE`, `--learn-increments`, `--learn-increments-alpha A`, `--learn-increments-lam L`, `--learn-increments-rho R`, `--save-increments FILE`.

Kernel. `--kernel fields|autocorrelation|surface`, `--ac-sigma S`, `--ac-dmax D`, `--surface-z ZMIN ZMAX ZSTEP`, `--ac-chirality`, `--ac-chirality-shells S`, `--block-weights W`, `--block-groups G`, `--block-rho R`, `--kernel-layer linear|gauss|arccos1|arccos2`, `--gammas LIST`, `--normalize`, `--raw-scale`, `--optimize-h`, `--mkl [align|alignf]`, `--mkl-rho R`.

Task and learner. `--task regression|classification|oneclass`, `--method krr|gp|svr|nusvr|svc|nusvc|gpc|ocsvm|ocmean|ocknn|ocgp|ockpca`, `--threshold T`, `--active-below`, `--class-property PROP`, `--oc-k K`, `--oc-components Q`, `--oc-lam L`, `--oc-score mean|mean_over_sd`, `--no-bias`.

Hyper-parameters. `--alphas LIST`, `--lams LIST`, `--Cs LIST`, `--epsilons LIST`, `--nus LIST`, `--lam-search grid|brent`, `--alpha-search grid|nelder-mead`, `--nm-start A,L`.

Conformations. `--conformations single|ensemble`, `--conformers N`, `--conformer-window A`, `--prune-rmsd A`, `--conformer-align self|core|reference|field`, `--conformer-align-ref I`, `--conformer-weighting uniform|boltzmann|property`, `--kt KCAL`, `--max-conformers N`.

Protonation and tautomers. `--ph PH`, `--ph-states N`, `--ph-min-population F`, `--tautomers score|uniform|given`, `--tautomer-forms N`.

Validation. `--folds K`, `--cv-no-shuffle`, `--nested`, `--nested-on train|all`, `--outer-folds K`, `--repeats R`, `--seed S`.

Screening. `--decoys FILE`, `--align-decoys`, `--decoy-restarts N`, `--decoy-conformers N`.

Alignment. `--align-train-variance [MAX_RMSD]`, `--align-train-rounds N`, `--align-train-folds K`.

Output. `--save model.cmf`, `--predictions out.csv`, `--json out.json`.

14.2 Worked examples

```
# indicator fields, a test split and nested cross-validation
python cmf_cli.py build examples/benzamides_synthetic.sdf --activity pActivity --cif \
       --alphas 0.3,0.5,0.8,1.2 --lams 1e-3,1e-2,0.1,1 --test-fraction 0.25 \
       --nested --save model.cmf

# predict, with the new molecules aligned by prediction confidence first
python cmf_cli.py predict model.cmf new.sdf --align-variance 2.0 \
       --aligned-out aligned.sdf --out predictions.csv

# the published protocol of [2] on a Sutherland set
python cmf_cli.py build examples/sutherland/THR.sdf --charges file --param tripos-legacy \
       --raw-scale --cv-no-shuffle --lam-search brent --nested --nested-on all

# a frame-free model: the rotation-invariant autocorrelation kernel
python cmf_cli.py build series.sdf --kernel autocorrelation --ac-sigma 0.75 --ac-dmax 12

# a topomer model with the core given explicitly
python cmf_cli.py topomer examples/benzamides_synthetic.sdf --activity pActivity \
       --core "c1ccccc1C(=O)N" --alphas 0.3,0.5,1 --lams 1e-2,0.1,1

# one-class screening against a property-matched background
python cmf_cli.py build actives.sdf --task oneclass --method ockpca \
       --decoys background.sdf --align-decoys --decoy-conformers 5

# a classification model with the Gaussian-process classifier
python cmf_cli.py build examples/sutherland/AChE.sdf --charges file --task classification \
       --threshold 6.84 --method gpc --Cs 1,10,100
```

15 Using it as a library

The library is the layer the application and the command-line tool are built on, and it is small enough to use directly in a notebook.

```
from cmfqsar import FieldSet, CMFModel, load_dataset
from cmfqsar.fields import AtomTable, build_fields, cif_types_in
from cmfqsar.model import grid_search

ds = load_dataset("aligned.sdf", activity_prop="pIC50")
fs = FieldSet.create(cif_types=cif_types_in(ds.mols, "sybyl", 5))
mf = [build_fields(m, fs, n) for m, n in zip(ds.mols, ds.names)]
train = AtomTable.from_molecules(mf)
model = CMFModel(fieldset=fs, method="krr")
grid_search(train, ds.activity, model, alphas=[0.3, 0.5, 1.0], lams=[1e-3, 1e-2, 0.1, 1])
model.fit(train, ds.activity, cv_folds=10, reuse_kernels=True)
print(model.stats.summary())
```

Decomposing one prediction, which works for the nonlinear kernels as well:

```
from cmfqsar.shapley import atom_shapley, field_shapley, scaffold_decomposition

res = atom_shapley(model, mf[0])                       # one value per atom
res = field_shapley(model, mf[0], interactions=True)   # one value per field type
res = scaffold_decomposition(model, mf[0], ds.mols[0], ds.mols[1:20])   # per substituent
print(res.prediction, res.baseline, res.table(), res.top_interactions(5))
```

Three more tasks that a notebook user is likely to want outside the application. Each script runs as written on a file from `examples/`.

A prediction interval with a coverage guarantee. The calibration uses the cross-validated errors that `fit` has already computed, so nothing is recomputed; see *How certain is one prediction? Conformal intervals* for what the guarantee means.

```
from cmfqsar import FieldSet, CMFModel, load_dataset
from cmfqsar.fields import AtomTable, build_fields
from cmfqsar.conformal import coverage

ds = load_dataset("examples/steroids_cbg.sdf", activity_prop="CBG_pK")   # 21 train, 10 test
fs = FieldSet.create()
mf = [build_fields(m, fs, n) for m, n in zip(ds.mols, ds.names)]
tr, te = ds.train_indices, ds.test_indices
model = CMFModel(fieldset=fs, method="krr", params={"lam": 0.01})
model.fit(AtomTable.from_molecules([mf[i] for i in tr]), ds.activity[tr], cv_folds=10)
model.calibrate_conformal(level=0.9)     # from the cross-validated errors fit() has already made
print(model.conformal_.describe())
yp, lo, hi = model.predict_interval(AtomTable.from_molecules([mf[i] for i in te]))
print(coverage(ds.activity[te], lo, hi))
```

A model on a series that has no alignment. A SMILES table is enough: `load_dataset` gives every molecule its own 3D structure, in no common frame, and the rotation-invariant autocorrelation kernel of *Working without an alignment* does not need one. The last line shows which atoms the prediction for one molecule came from.

```
from cmfqsar import FieldSet, CMFModel, load_dataset
from cmfqsar.fields import AtomTable, build_fields
from cmfqsar.model import grid_search

ds = load_dataset("examples/cyclodextrin/gamma_cyclodextrin.csv", activity_prop="activity")
fs = FieldSet.create(alpha=1.0)
mf = [build_fields(m, fs, n) for m, n in zip(ds.mols, ds.names)]
train = AtomTable.from_molecules(mf)
model = CMFModel(fieldset=fs, method="krr", normalize=True,
                 kernel_source="autocorrelation", ac_sigma=0.75, ac_d_max=12.0)
grid_search(train, ds.activity, model, alphas=[1.0], lams=[1e-3, 1e-2, 0.1, 1])
model.fit(train, ds.activity, cv_folds=10)
print(model.stats.summary())
print(model.atom_contributions(mf[0]))      # one value per atom of the first molecule
```

Two endpoints on one kernel. The multi-task model of *Several endpoints at once* is a function of the kernel matrix, so it takes the kernel of any model, aligned or not. Here every guest carries its binding constant to beta-cyclodextrin and some guests carry the alpha-cyclodextrin constant as well; the two are fitted together, tied by one correlation chosen by cross-validation.

```
import numpy as np
from cmfqsar import FieldSet, CMFModel, load_dataset
from cmfqsar.fields import AtomTable, build_fields
from cmfqsar import multitask as MT

ds = load_dataset("examples/cyclodextrin/beta_with_alpha.csv", activity_prop="activity")
alpha = np.array([float(m.GetProp("alpha_logK")) if m.HasProp("alpha_logK") and
                  m.GetProp("alpha_logK").strip() else np.nan for m in ds.mols])
fs = FieldSet.create(alpha=1.0)
mf = [build_fields(m, fs, n) for m, n in zip(ds.mols, ds.names)]
model = CMFModel(fieldset=fs, method="krr", normalize=True, kernel_source="autocorrelation")
K = model.training_kernel(AtomTable.from_molecules(mf))    # one kernel for both endpoints
obs = MT.Observations.from_columns(np.column_stack([ds.activity, alpha]), names=("beta", "alpha"))
rho, lam, score, per_task = MT.search(K, obs, rhos=[0.0, 0.5, 0.9], lams=[0.01, 0.1, 1.0])
print(rho, lam, per_task)                    # cross-validated q2 of each endpoint
fit = MT.fit(K, obs, rho=rho, lam=lam)
print(fit.predict_selectivity(K[:5], 0, 1))  # beta minus alpha for the first five guests
```

Five notebooks under `docs/tutorials/` take the library through the same ground step by step, with plots and 3D views: a first model on an aligned series, a series with no alignment, the checks that say how far a prediction can be trusted, two endpoints with their selectivity, and screening a library against a model of one series. Each runs on the example data of the repository, locally or in Google Colab from the link at its top. `python docs/tutorials/run_tutorials.py` executes them, so that the outputs in the files come from the current code.

15.1 The stable surface

Nine names are the supported interface of the package. They are what the examples above import, what the notebooks use, and what this chapter describes:

name	what it is
`load_dataset`	reads an SDF, MOL2 or SMILES file and returns a `Dataset`
`Dataset`	the molecules, their names, the activity, and the indices of the training and test subsets

name	what it is
`FieldSet`	which fields a model uses and how wide their Gaussians are; `FieldSet.create` makes one
`FieldSpec`	one field of a field set
`build_fields`	one molecule plus a field set, as a `MoleculeFields`
`MoleculeFields`	the fields of one molecule
`AtomTable`	the fields of several molecules in one table, which is what a model is fitted on; `AtomTable.from_molecules` makes one
`CMFModel`	the model: `fit`, `predict`, `predict_interval`, `calibrate_conformal`, `rank_score`
`ModelStats`	what the fit measured: `q2`, `r2`, `rmse_cv`, `r2_pred`, and `summary()`

`from cmfqsar import ...` imports every one of them, and `cmfqsar.__all__` lists them in the source. The promise is this: within a major version a name on this list is not removed or renamed, and an argument of it that `tests/test_stable_surface.py` records keeps its name, its position and its default. New arguments may be added after the recorded ones, so a call written today keeps working; a call that relies on an argument's position beyond those recorded does not have that promise.

Everything else in the package is internal. It is listed below and documented in `docs/api.md`, and it may change in any release, so a script that imports a module directly should say which version of the package it was written for. The two surfaces are also tested differently: the stable one by `tests/test_stable_surface.py`, which fails when a promised argument disappears or when an example in this chapter imports a name the package no longer has, and the rest by the tests of the feature that owns it.

The function-by-function reference is a separate file, `docs/api.md`. It is generated from the docstrings of the source by `python docs/make_api.py`, so it lists every public class, method and function with its signature, and it is as current as the last time that command was run. This chapter says what the modules are for; that file says what each function takes and returns.

The modules, one line each. The package has more modules than the application has panels: many of them exist only for the command-line tool and for use as a library.

Data, fields and kernels:

```
cmfqsar/dataio.py        reading SDF / MOL2 / SMILES, model persistence, exports
cmfqsar/fields.py        field parameterisation; FieldSet / MoleculeFields / AtomTable
cmfqsar/atomtypes.py     Sybyl-like atom typing and van der Waals parameters
cmfqsar/increments.py    atomic increments of the original CMF implementation [2]
cmfqsar/learnincrements.py  increments fitted to the data instead of taken from the table
cmfqsar/pharmtypes.py    six pharmacophore features as indicator fields
cmfqsar/probes.py        GRID-like probe fields written as sums of Gaussians
cmfqsar/anisotropic.py   atom functions with a direction (Gaussian times a linear factor)
cmfqsar/exposure.py      solvent exposure as a weight on the fields
cmfqsar/descriptors.py   descriptors that are not fields: one number per molecule
cmfqsar/orca.py          quantum-chemical properties from ORCA, as fields or descriptors
cmfqsar/cdft.py          conceptual-DFT fields on a grid, and their Gaussian fits
cmfqsar/kernels.py       analytic field kernels, vectorised
cmfqsar/fastkernel.py    a compiled, threaded atom-pair loop for the kernels
cmfqsar/mixing.py        a posterior distribution over the field weights
cmfqsar/perfield.py      a separate attenuation for every field, chosen by the evidence
```

Models, their parameters and their validation:

```
cmfqsar/model.py         KRR / SVR / GP, cross-validation, grid search, nested CV, statistics
cmfqsar/evidence.py      choosing α and λ by the marginal likelihood instead of by cross-validation
cmfqsar/neldermead.py    the Nelder-Mead minimiser of the original R scripts
cmfqsar/ranking.py       the ranking SVM: learning the order of the molecules
cmfqsar/multitask.py     two endpoints at once; selectivity as a co-field
cmfqsar/gpc.py           Laplace Gaussian-process classification
cmfqsar/oneclass.py      centroid, k-NN, GP and kernel-PCA one-class descriptions
cmfqsar/pu.py            learning from positives and unlabelled molecules
cmfqsar/fusion.py        one score from the similarities to several known actives
cmfqsar/validation.py    y-randomisation, applicability domain and the error of q²
cmfqsar/conformal.py     prediction intervals with a coverage guarantee
cmfqsar/errorpred.py     which error estimate knows when a prediction is bad
cmfqsar/mmd.py           the two-sample test: is this library like the training set?
cmfqsar/contract.py      what has to match between fitting and predicting, checked
cmfqsar/diagnose.py      what to ask of a data set before modelling it (`cmf diagnose`)
```

Reading a model:

```
cmfqsar/cofields.py      co-fields and molecular fields on grids, isosurfaces, cube export
cmfqsar/shapley.py       Shapley decomposition over atoms, substituents, field types
cmfqsar/pharmacophore.py typed features with tolerance spheres, from the co-field
cmfqsar/fielddiff.py     field differences against a reference molecule
cmfqsar/transfer.py      comparing two models by their co-fields
cmfqsar/kcca.py          what the field sees that another view does not (kernel CCA)
cmfqsar/spectral.py      clustering a set by its kernel rather than by its scaffolds
cmfqsar/gtm.py           chemical-space maps and activity landscapes
cmfqsar/report.py        the HTML report of a model
```

Alignment and poses:

```
cmfqsar/align.py         3D embedding, MCS alignment, field-similarity refinement, unalign
cmfqsar/coalign.py       co-alignment of a training set by expectation-maximisation over poses
cmfqsar/varalign.py      alignment by prediction confidence (GP posterior variance)
cmfqsar/flexalign.py     the same search with the torsions free
cmfqsar/fieldgraph.py    field-point graphs: alignment without a common substructure
cmfqsar/poses.py         a protocol that judges an alignment against reference poses
cmfqsar/stereo.py        handedness as a property of a stereocentre
```

Working without an alignment:

```
cmfqsar/invariant.py     rotation-invariant field autocorrelation kernels
cmfqsar/periodic.py      the periodic autocorrelation kernel for crystals
cmfqsar/topomer.py       topomer fields: substituents posed by rule
cmfqsar/chemotypes.py    scaffold clusters, placement, grouped folds, the count kernel
```

Conformations and ionisation states:

```
cmfqsar/conformers.py    conformational ensembles: generation, weighting, grouping
cmfqsar/mil.py           multi-instance learning over conformations
cmfqsar/protonation.py   protonation ensembles: a molecule as a weighted mean of its states
cmfqsar/tautomers.py     tautomer ensembles
```

Screening and design:

```
cmfqsar/decoys.py        posing a background set for screening evaluation
cmfqsar/screen.py        screening a library that does not fit in memory
cmfqsar/active.py        which molecule to make next
cmfqsar/design.py        what to put at a position, ranked by the model
cmfqsar/gradients.py     which way to move an atom: the derivative of a prediction
```

Protein pockets:

```
cmfqsar/protein.py       PDB / mmCIF reading, binding-site selection, frame check
cmfqsar/complementarity.py  pocket fields, complementarity scores, co-field agreement
cmfqsar/pocketalign.py   aligning a molecule inside the pocket
cmfqsar/docking.py       rigid-body placement of conformations in the pocket
```

The two front ends:

```
cmfqsar/cli.py           the `cmf` command-line tool
cmfqsar/gui/             the PySide6 application
```

16 The example data sets

Set	Contents
`examples/benzamides_synthetic.sdf`	109 aligned N-phenylbenzamides with a *synthetic* activity: the overlap of their fields with a hidden pseudo-receptor co-field plus noise (σ = 0.25). A model has to recover those zones; $q^2 \approx 0.87$ against a noise ceiling of 0.94. The set to learn the program on, because the right answer is known.
`examples/steroids_cbg.sdf`	the classic CoMFA benchmark: 31 steroids [9] with corticosteroid-binding-globulin affinities, in the corrected Gasteiger-group geometries, re-superimposed on the steroid skeleton of testosterone. Customary split 1–21 / 22–31, leave-one-out $q^2 \approx 0.79$.
`examples/sutherland/*.sdf`	the eight literature sets of [2], from Sutherland et al. [17]: ACE, AChE, BZR, COX-2, DHFR, GPB, THER, THR, with the aligned MMFF94s conformations as distributed, the published split, and the original Sybyl types and MNDO ESP charges as per-atom properties. Choose *Partial charges: file*.
`examples/selectivity/ampa_glycine.sdf`	the 57 AMPA / glycine-NMDA antagonists of the 2003 selectivity-fields paper (Baskin, Tikhonova, Palyulin & Zefirov) with `pKi_AMPA`, `pKi_glycine`, their difference `delta_AMPA_glycine` and the published split (52 / 5), superimposed on the tricyclic core; the set to try *Model several endpoints (selectivity co-field)…* on. Use Gasteiger charges: the file charges of sixteen compounds are unreliable. `docked_ampa.sdf` and `docked_glycine.sdf` hold the same compounds docked in the two receptor models, `receptor_ampa.pdb` and `receptor_glycine.pdb`.
`examples/mittal/*.sdf`	38 of the 40 benchmark sets of Mittal, McKinnon and Sorich [18], built here from their 2D supplementary data by ETKDG and an MCS superposition, with each molecule's fit quality recorded in `align_rmsd`. The collection to use when you want to know how a method behaves on an *automatic* alignment.
`examples/openff/*.sdf`	21 congeneric series, 513 ligands, from the Open Force Field protein–ligand benchmark, each posed in its target's crystal structure; activity tag `pAffinity`.
`examples/pockets/*.pdb`	the binding sites of those series, trimmed and already in the ligands' frame, so they open together with the data.
`examples/thrombin_atom_properties.sdf`	a worked example of `prop:<name>` fields: per-atom quantum properties carried in an SDF.

`docs/benchmark.md` compares the program with Table 2 of [2] on the eight literature sets. `benchmarks/run_sutherland.py` reruns that comparison, `benchmarks/run_topomer.py` measures the alignment against the topomer rule (`--mittal` for the automatically aligned collection), `benchmarks/run_alignment.py` compares the value of a curated alignment with that of an automatic one, and `benchmarks/run_corrosion.py` does the same for corrosion-inhibitor series.

17 When something goes wrong

The 3D view is black, or the program will not start. The viewer needs working OpenGL through PyOpenGL and pyqtgraph. On a remote desktop or in a virtual machine, software rendering is usually the fix (on Windows, a Mesa `opengl32.dll` beside the program; on Linux, `LIBGL_ALWAYS_SOFTWARE=1`). Everything except the 3D view works regardless, and the command-line tool needs no graphics at all.

q² is high but the test predictions are poor. Three usual causes, in order of frequency: the hyper-parameters were chosen on the same cross-validation you are quoting (run *External CV*); the set holds several chemotypes and random folds are overstating it (use leave-one-chemotype-out); or the alignment carries information that will not transfer (compare against the count baseline).

q² is negative. The model predicts worse than the mean: check that the activity column is the right one and on a logarithmic scale, and that the molecules really share a frame. For R^2_{pred} on a narrow test set it may be an artefact of the variance convention, and the program reports both.

The optimum sits at the edge of the α or λ grid. Extend the grid. The *Hyper-parameter search* tab exists to make this visible.

A per-atom property is rejected. The hydrogen count almost certainly changed when the file was loaded. Write the property for all atoms including hydrogens, or pass `--prop-missing zero` deliberately.

The topomer transformation drops most of the molecules. The core is fused to the rest of them. Look at the cuttable fraction in the log and give a core joined to its R-groups by acyclic single bonds.

A decoy set gives suspiciously good enrichment. Check that the decoys are property-matched and posed in the training frame; cross-target decoys are separable by almost anything.

Co-fields are unavailable. A nonlinear kernel layer or the autocorrelation kernel source is selected, and neither is linear in the field, so the co-field does not exist. Use the *Shapley* tab instead; for a kernel layer, `cmf cofield --kind local` and `--kind spread` draw the co-field of the tangent model at one molecule and how far those differ over the training set (*The co-field of a nonlinear model: one per molecule*).

It is slow. Kernel cost grows with the square of the total atom count, so the expensive sets are those with many large molecules. Cut the α grid first, since each α needs its own kernel, and remember that the count kernel is 10 to 34 times faster than the Gaussian route for a baseline.

Appendix A — Field parameterisation

Two parameterisations are available (*Increments* in the model panel, `--param`):

Field	`rdkit`, the default	`tripos`, the original CMF scripts
electrostatic	partial charge: Gasteiger, MMFF94, or read from the file	the same
steric	Tripos vdW well depth and width α/R^2_vdW, parameters by element	parameters by Sybyl atom type, Tripos 5.2 table
hydrophobic	Wildman–Crippen atomic logP contribution	in-house one-atom fragment increments by Sybyl type and attached hydrogens
H-bond acceptor / donor	approximate Abraham-type increments, donor weight on the polar hydrogen	in-house increments from Abraham's A and B constants, donor weight on the heavy atom
indicator field	1 for atoms of the given type	the same
`prop:<name>`	the value of a per-atom property carried by the file	the same

`tripos-legacy` reproduces the hydrogen-counting bug of the original R scripts, which is needed to match the published numbers exactly and should not be used for new work.

A.1 Increments fitted to the data

The hydrophobic field and the two hydrogen-bond fields are built from one number per atom type: the increment of a Sybyl type carrying a given number of hydrogens. Those numbers come from regressions of logP and of Abraham's solute constants, not from the activity being modelled, and the two parameterisations in the table above differ by 0.20 in q2 on one Sutherland set. That difference says the increments are not settled, so the program can fit them instead (`cmfqsar/learnincrements.py`, `build --learn-increments`).

Fitting them is cheap, because the kernel is a quadratic form in the increments. Every atom of a type carries the same weight, so

$$K(A,B) = \sum_{t,u} c_t c_u M_{tu}(A,B) \tag{A.1}$$

where `M` sums the Gaussian overlap integral over the atom pairs whose types are `t` and `u` and does not depend on the increments at all. `M` is built once per attenuation, every candidate kernel is one contraction, and the derivative of the cross-validated error can be carried back to the increments exactly, through the cosine normalisation and the fold solves. The program minimises the same cross-validated squared error it selects models on, with the published values as the starting point and a penalty on the distance from them. Tests compare every link of that chain with finite differences, and compare the quadratic form with the kernel the program builds the ordinary way.

Two properties of the parameterisation are worth knowing. A cosine-normalised kernel does not change when all the increments of a field are multiplied by the same number, so only their relative sizes can be learned; the fitted values are reported at the length of the published ones, which

changes no kernel. And the penalty cannot be chosen on the cross-validation being minimised, which always prefers the freest fit, so it is chosen on training molecules that the fitting does not see.

What it is worth, measured

`benchmarks/run_increments.py` measures four models on the eight Sutherland sets, on their published training and test split, with the same five fields, the same attenuation and ridge grids and the same ten-fold cross-validation: the published increments; increments fitted on this set's own training molecules; increments fitted on each of the other seven sets and applied here; and the best of those seven.

Model	mean q2 (training cross-validation)	mean R2 on the test set
published increments	+0.596	**+0.383**
fitted on this set	**+0.816**	+0.310
fitted on another set (mean of seven)	+0.593	+0.363
the best of those seven	+0.646	+0.449

Fitting the increments improves the number it is fitted on and does not improve prediction. The cross-validated q2 rises from +0.596 to +0.816, which is what minimising it must do. On the test set, which the fitting never saw, R2 goes from +0.383 to +0.310; fitted increments predict better than the published ones on 2 of the 8 sets. The gap between the two movements is the selection bias of reading a q2 off the cross-validation that chose the parameters.

What is learned does not transfer. Increments fitted on one set and applied to another give +0.363 against +0.383 for the published table; transferred increments beat the published ones on 3 of the 8 sets. The best of the seven donors reaches +0.449, which is above the table, but that donor is picked with the test set already in hand. The fitted vectors do not agree with each other either: the mean cosine between the increments fitted on two different sets is +0.89 for the hydrophobic field, +0.72 for the acceptor field and +0.54 for the donor field, where every fit started from the same table.

The "best of seven" column is the most generous reading there is, and it is not a rule anyone could follow: the best donor is picked with the test set already in hand. Read it as an upper bound on what choosing a donor set could ever give.

So the increments are a per-set nuisance parameter, which is the answer the plan asked for. The published table is a reasonable prior and should stay the default; fitting the increments is worth doing to see how much a set wants to move them, not to make a model that predicts better. The fit moves the two hydrogen-bond fields much further than the hydrophobic one (+0.99, +0.86 and +0.70 mean cosine with the published values), which is also where the two parameterisations of Appendix A differ most: they put the donor weight on different atoms. How far a set moves its increments does not say how much it will lose, though -- over the eight sets the correlation between the two is -0.25, which is nothing.

`--learn-increments` is therefore not a default. It prints a warning that the q2 of the same cross-validation is optimistic, and `--save-increments` writes the table so that it can be read back with `--increments FILE` and used on another series -- which is how the transfer above was measured.

Appendix B — Files and tags the program understands

Tag or convention	Meaning
any numeric SD property	candidate activity; the loader guesses and the dialog confirms
`subset`, `set`, `split`	the train/test split, as `train`/`test` or 1/2
`atom.dprop.<name>`	a per-atom double property, written and read by RDKit
`atom.prop.sybyl_type`	per-atom Sybyl types, used automatically for indicator fields
a molecule property with one number per atom	the same per-atom property in quantum-chemistry dump form
`.cmf`	a saved model (joblib): field set, kernels, coefficients, training molecules, settings
Gaussian cube	the export format for co-fields, molecular fields, Shapley fields and difference maps
`REMARK 400` in a pocket PDB	the provenance of a trimmed binding site

Appendix C — Glossary

The terms this manual uses in a particular sense, with the chapter that treats each one.

term	what it means here
alignment, or **frame**	the common coordinate system the molecules are placed in before their fields are compared. The overlap kernel needs one and is only as good as it is; the count, autocorrelation, surface and periodic kernels do not. *Preparing the data*; *Working without an alignment*
applicability domain	how close a molecule outside the training set is to it, measured in the kernel the model itself uses, with a threshold taken from the training set's own spread. Reported per molecule as the **AD** column and carried into the exported predictions. *Where does the model apply?*
attenuation, α	the width parameter of a field's Gaussians: α large makes a field that falls away within an atom's own volume, α small one that reaches across the molecule. The single most consequential setting after the alignment. *The fields*; *Attenuation, regularisation and the grids*
autocorrelation kernel	geometry without a frame: the smoothed distribution of field-weight products over interatomic distance, invariant to rotation and translation by construction. *The rotation-invariant autocorrelation kernel*
BEDROC, RIE	screening figures that weight the top of a ranked list, where AUC saturates. *Decoys and the figures that matter*
Bemis–Murcko framework	the ring systems of a molecule together with the bonds that join them. `cmf diagnose` measures the common framework to ask whether one frame could exist at all. *Before you model*
block weights	one coefficient per block of channels — a block being one pair of field types — applied before the invariant kernels are normalised, so a weight of zero removes a block exactly. *Mixing coefficients over the channel blocks*
CHELPG, **Hirshfeld**, **Löwdin**, **Mulliken** charges	four ways of dividing a molecule's electron density among its atoms, all of which ORCA can report and any of which can become the electrostatic field's weights. They disagree, sometimes substantially, which is why the charge model is part of what must match between fitting and predicting. *Quantum-chemical properties with ORCA*
chemotype	a group of molecules sharing a scaffold. Splitting a benchmark by chemotype rather than at random is the test of whether a model carries beyond the series it was fitted on. *The example data sets*
co-field, $C_f(\mathbf{r})$	the model as a function of space: the training molecules' fields summed with the fitted coefficients, one per field type. Every prediction is the overlap of a molecule's own field with it. *The same prediction, as a field*; *Co-fields*
co-pattern	what the invariant kernels offer in place of a co-field — the weight of every channel of the fitted linear model, named by its field pair and distance, ranked by `top_channels`. *Crystals*; *Molecules on a surface*
contributions	the decomposition of one prediction over the training molecules, and of a training molecule's share over its atoms. *Contributions*

term	what it means here
count kernel	the pose-free baseline: a kernel on atom-type counts, with no geometry at all. Worth running before any aligned model, because it sometimes wins. *The pose-free count kernel*
decoys	presumed-inactive molecules used **only** to evaluate a screening model, never to fit it; property-matched decoys are the fair kind. *Decoys and the figures that matter*
`cmf diagnose`	the command that runs the five baselines — aligned, counts, autocorrelation, topomer, descriptors — on a data set before any model is built, and says which route to take. *Before you model*
dual coefficients, **a**, and **bias** b	what fitting produces: one coefficient per training molecule and one constant, from which every later prediction is a weighted sum of kernel values. *Fitting: what is learned from the training set*
ECFP4	a circular fingerprint of the atoms' environments out to two bonds: the standard 2D similarity measure, and the baseline the field similarity of this program is compared against on screening tasks, where it does about as well. *Decoys and the figures that matter*
ETKDG	RDKit's distance-geometry conformer generator, which builds the 3D structure whenever the input is 2D and supplies the starting conformations for the conformer ensembles, the flexible alignment and the topomer rebuilds. Its poses are arbitrary until they are aligned. *What the program reads*; *Conformations: the fourth dimension*
external cross-validation, q^2_{ex}	the whole model-building procedure, hyper-parameter search included, repeated inside each outer fold; the only figure that is not optimistic about the search. *Validation*
field, $X_f(\mathbf{r})$	a continuous function of space: one Gaussian per atom, weighted by that atom's charge, bulk, lipophilicity, H-bond strength, indicator value or file-borne property. The object CMF models. *The fields*
field scaling, c_f	the factor that puts a normalised field kernel's mean diagonal at 1 over the training set, so the fields enter the mixture on equal terms. Fixed on the training set and stored. *Normalisation and scaling*
field set	the list of field types a model uses, with their attenuations, charge model and parameterisation. It must match exactly between fitting and predicting. *What has to match between fitting and predicting*
flexible alignment	alignment that turns the rotatable bonds as well as moving the molecule, with the force field as a prior on the conformation and several generated starting conformations. The best pose method measured here: 0.86 Å median against the crystal poses. *Judging an alignment, and aligning flexibly*
Fukui index, f^+, f^-, f^0	a per-atom measure of where a molecule gives or takes electron density, computed from the N+1 and N−1 states by ORCA and used as a `prop:` field. The **dual descriptor** is $f^+ - f^-$. *Quantum-chemical properties with ORCA*
h, the **mixing coefficient** h_f	the weight of one field's normalised kernel in the joint kernel. Set by hand, optimised, or derived by kernel alignment. *Mixing the fields*
increments	the table of per-atom parameters a field is built from — the `rdkit` default or the `tripos` tables of the original CMF scripts. Not a setting of the method but of the chemistry it reads. *Appendix A*
indicator field	a field whose weight is 1 for atoms of a given type and 0 elsewhere, so that its co-field says where atoms of that type help. *Choosing the fields*

term	what it means here
joblib	the serialisation library a saved `.cmf` model is written with. A `.cmf` file is therefore a Python object graph, not an interchange format: it is read by this program and not meant to be read by another. *Saving and reusing*
joint kernel, $K(A,B)$	the sum of the normalised, scaled field kernels with their mixing coefficients: the single similarity the learner sees. *Mixing the fields*
Kabsch superposition	the least-squares rotation and translation that best superimposes two sets of matched points. It is the second half of an MCS alignment -- the substructure search finds the correspondence, this places the molecule -- and the pose benchmark measures how far the result lands from the crystal pose. *Alignment*; *Judging an alignment, and aligning flexibly*
kernel layer	a nonlinear function applied to the joint kernel — Gaussian or arc-cosine — after which there is no single co-field, because the prediction is no longer linear in the fields; a local co-field per molecule remains. *Nonlinear kernel layers and multiple kernel learning*
local co-field	the derivative of a layered model's prediction with respect to the normalised field, taken at one molecule: the co-field of the linear model tangent to it there. For the linear kernel it is the ordinary co-field for every molecule. *The co-field of a nonlinear model: one per molecule*
non-linearity index	the share of the mean squared norm of the local co-fields of the training molecules that their spread around the mean makes up: 0 for a linear model, 1 when they have nothing in common. *The co-field of a nonlinear model: one per molecule*
pair-interaction field	the second derivative of a prediction with respect to the amplitudes of two probe atoms, one fixed on an atom of the molecule and the other scanned over space: how much adding field at a point changes what the fixed atom does. Zero everywhere for a linear model without normalisation. *Second derivatives: the pair-interaction field and the interaction modes*
interaction modes	the eigenvectors of the atom interaction matrix, the second derivatives with respect to the weights of every pair of a molecule's atoms: patterns of atoms whose weights the model reads together, with the curvature along the pattern as eigenvalue. *Second derivatives: the pair-interaction field and the interaction modes*
MCS alignment	superimposing molecules on their maximum common substructure. The conventional frame, and the baseline every pose method here is measured against. *Alignment*
MKL	multiple kernel learning: deriving the mixing coefficients from the kernels and the activities by centred kernel alignment rather than setting them. *Nonlinear kernel layers and multiple kernel learning*
nested cross-validation	choosing the hyper-parameters inside each training fold and scoring outside it, so that the reported figure has not seen the molecules the choice was made for. *Validation*
normalisation	dividing a field kernel by the geometric mean of the two self-kernels, so every molecule has unit self-similarity and a large molecule cannot look important merely for being large. *Normalisation and scaling*

term	what it means here
one-class model	a model fitted on actives alone, which asks whether a molecule resembles them rather than how active it is. *One-class models: screening with actives alone*
overlap integral, $K_f(A,B)$	the integral over all space of the product of two molecules' fields, which for Gaussians is a closed-form double sum over atom pairs. The kernel CMF is built on. *The overlap integral*
periodic kernel	the autocorrelation descriptor for a crystal: the pair histogram of one cell with its periodic images, per atom, with the uniform background removed and the cutoff tapered. *Crystals: the periodic autocorrelation kernel*
pharmacophore	a set of typed spheres extracted from the lobes of a co-field, which is the model's own statement of what a molecule should carry and where. *A pharmacophore out of the co-field*
probe field	a GRID-like interaction energy with a chosen probe, expanded in Gaussians so that it keeps the closed-form kernel and the co-field. *GRID-like probe fields*
`prop:` **field**	a field whose per-atom weights are read from the file or computed by ORCA rather than from a parameter table. *Charges and per-atom properties*
q^2, R^2_{pred}, RMSE	the cross-validated, test-set and fit figures of merit. q^2 is not a test: it is what the search optimises, and a model can be chosen for it. *Validation*; *Statistics reported*
selectivity co-field	the difference of two endpoints' co-fields in a multi-task model, which draws where a molecule should differ to be selective rather than merely potent. *Several endpoints at once*
Shapley value	the share of a prediction attributed to each field or atom by averaging over all subsets, with the pairwise interaction indices beside it. *Shapley values*
solvent exposure weighting	multiplying each atomic weight by its relative solvent accessibility. Measured here and found not to help: ligands of this size have no interior. *Weighting the fields by solvent exposure*
surface kernel	the in-plane invariant kernel for molecules on a surface: distances in the plane, resolved by the heights of both atoms, invariant to lateral position and rotation about the normal and to nothing else. *Molecules on a surface*
topomer field	a field built from substituents rebuilt in a pose that is a deterministic function of their topology, so the same R-group always lands in the same place and no alignment is needed. *Topomer fields*
y-randomisation	refitting the whole procedure on shuffled activities many times; if the shuffled models score as well as the real one, the real one is chance. *Is it chance? y-randomisation*

Appendix D — Bibliography

References are grouped by what they are to this program: the papers that define the method it implements, those that apply it to one-class classification and virtual screening, and the work it takes methods or benchmark data from. Journal names follow the ACS abbreviations; a DOI is given wherever the work has one.

D.1 The method

[1] Zhokhova, N. I.; Baskin, I. I.; Bakhronov, D. K.; Palyulin, V. A.; Zefirov, N. S. Method of continuous molecular fields in the search for quantitative structure–activity relationships. *Dokl. Chem.* **2009**, *429*, 273–276. doi:10.1134/S0012500809110056

[2] Baskin, I. I.; Zhokhova, N. I. The continuous molecular fields approach to building 3D-QSAR models. *J. Comput.-Aided Mol. Des.* **2013**, *27*, 427–442. doi:10.1007/s10822-013-9656-4

[3] Baskin, I. I.; Zhokhova, N. I. Continuous molecular fields approach applied to structure–activity modeling. In *Application of Computational Techniques in Pharmacy and Medicine*; Gorb, L., Kuz'min, V., Muratov, E., Eds.; Challenges and Advances in Computational Chemistry and Physics, Vol. 17; Springer: Dordrecht, 2014; pp 433–459. doi:10.1007/978-94-017-9257-8_13

[4] Sitnikov, G. V.; Zhokhova, N. I.; Ustynyuk, Y. A.; Varnek, A.; Baskin, I. I. Continuous indicator fields: a novel universal type of molecular fields. *J. Comput.-Aided Mol. Des.* **2015**, *29*, 233–247. doi:10.1007/s10822-014-9818-z

[5] Baskin, I. I.; Zhokhova, N. I. Descriptors based on continuous indicator fields for 3D-QSAR studies. In *Proceedings of MOL2NET 2016, International Conference on Multidisciplinary Sciences*, 2nd ed.; MDPI: Basel, 2017. doi:10.3390/mol2net-02-03854

[6] Baskin, I. I.; Zhokhova, N. I. Continuous molecular fields and the concept of molecular co-fields in structure–activity studies. *Future Med. Chem.* **2019**, *11*, 2701–2713. doi:10.4155/fmc-2018-0360

D.2 One-class classification and virtual screening

[7] Karpov, P. V.; Baskin, I. I.; Zhokhova, N. I.; Zefirov, N. S. Method of continuous molecular fields in the one-class classification task. *Dokl. Chem.* **2011**, *440*, 263–265. doi:10.1134/S0012500811100016

[8] Karpov, P. V.; Baskin, I. I.; Zhokhova, N. I.; Nawrozkij, M. B.; Zefirov, A. N.; Yablokov, A. S.; Novakov, I. A.; Zefirov, N. S. One-class approach: models for virtual screening of non-nucleoside HIV-1 reverse transcriptase inhibitors based on the concept of continuous molecular fields. *Russ. Chem. Bull.* **2011**, *60*, 2418–2424. doi:10.1007/s11172-011-0372-8

D.3 Methods and data taken from other work

[9] Cramer, R. D.; Patterson, D. E.; Bunce, J. D. Comparative molecular field analysis (CoMFA). 1. Effect of shape on binding of steroids to carrier proteins. *J. Am. Chem. Soc.* **1988**, *110*, 5959–5967. doi:10.1021/ja00226a005

[10] Cramer, R. D. Topomer CoMFA: a design methodology for rapid lead optimization. *J. Med. Chem.* **2003**, *46*, 374–388. doi:10.1021/jm020194o

[11] Bemis, G. W.; Murcko, M. A. The properties of known drugs. 1. Molecular frameworks. *J. Med. Chem.* **1996**, *39*, 2887–2893. doi:10.1021/jm9602928

[12] Hopfinger, A. J.; Wang, S.; Tokarski, J. S.; Jin, B.; Albuquerque, M.; Madhav, P. J.; Duraiswami, C. Construction of 3D-QSAR models using the 4D-QSAR analysis formalism. *J. Am. Chem. Soc.* **1997**, *119*, 10509–10524. doi:10.1021/ja9718937

[13] Bishop, C. M.; Svensén, M.; Williams, C. K. I. GTM: the generative topographic mapping. *Neural Comput.* **1998**, *10*, 215–234. doi:10.1162/089976698300017953

[14] Cho, Y.; Saul, L. K. Kernel methods for deep learning. In *Advances in Neural Information Processing Systems 22 (NIPS 2009)*; Curran Associates: Red Hook, NY, 2009; pp 342–350.

[15] Rasmussen, C. E.; Williams, C. K. I. *Gaussian Processes for Machine Learning*; MIT Press: Cambridge, MA, 2006. doi:10.7551/mitpress/3206.001.0001

[16] Truchon, J.-F.; Bayly, C. I. Evaluating virtual screening methods: good and bad metrics for the "early recognition" problem. *J. Chem. Inf. Model.* **2007**, *47*, 488–508. doi:10.1021/ci600426e

[17] Sutherland, J. J.; O'Brien, L. A.; Weaver, D. F. A comparison of methods for modeling quantitative structure–activity relationships. *J. Med. Chem.* **2004**, *47*, 5541–5554. doi:10.1021/jm0497141

[18] Mittal, R. R.; McKinnon, R. A.; Sorich, M. J. Comparison data sets for benchmarking QSAR methodologies in lead optimization. *J. Chem. Inf. Model.* **2009**, *49*, 1810–1820. doi:10.1021/ci900117m

[19] Tran-Nguyen, V.-K.; Jacquemard, C.; Rognan, D. LIT-PCBA: an unbiased data set for machine learning and virtual screening. *J. Chem. Inf. Model.* **2020**, *60*, 4263–4273. doi:10.1021/acs.jcim.0c00155

[20] Wallach, I.; Heifets, A. Most ligand-based classification benchmarks reward memorization rather than generalization. *J. Chem. Inf. Model.* **2018**, *58*, 916–932. doi:10.1021/acs.jcim.7b00403

[21] Mordelet, F.; Vert, J.-P. A bagging SVM to learn from positive and unlabeled examples. *Pattern Recognit. Lett.* **2014**, *37*, 201–209. doi:10.1016/j.patrec.2013.06.010

[22] Platt, J. C. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*; Smola, A. J., Bartlett, P., Schölkopf, B., Schuurmans, D., Eds.; MIT Press: Cambridge, MA, 1999; pp 61–74.

[23] Gretton, A.; Borgwardt, K. M.; Rasch, M. J.; Schölkopf, B.; Smola, A. A kernel two-sample test. *J. Mach. Learn. Res.* **2012**, *13*, 723–773.

[24] Zaretzki, J.; Matlock, M.; Swamidass, S. J. XenoSite: accurately predicting CYP-mediated sites of metabolism with neural networks. *J. Chem. Inf. Model.* **2013**, *53*, 3373–3383. doi:10.1021/ci400518g

[25] Levoin, N.; Labeeuw, O.; Calmels, T.; Poupardin-Olivier, O.; Berrebi-Bertrand, I.; Lecomte, J.-M.; Schwartz, J.-C.; Capet, M. Novel and highly potent histamine H3 receptor ligands. Part 1: withdrawing of hERG activity. *Bioorg. Med. Chem. Lett.* **2011**, *21*, 5378–5383. doi:10.1016/j.bmcl.2011.07.006

[26] Cramer, R. D.; Bunce, J. D.; Patterson, D. E.; Frank, I. E. Crossvalidation, bootstrapping, and partial least squares compared with multiple regression in conventional QSAR studies. *Quant. Struct.-Act. Relat.* **1988**, *7*, 18–25. doi:10.1002/qsar.19880070105

[27] Ng, A. Y.; Jordan, M. I.; Weiss, Y. On spectral clustering: analysis and an algorithm. In *Advances in Neural Information Processing Systems 14*; Dietterich, T. G., Becker, S., Ghahramani, Z., Eds.; MIT Press: Cambridge, MA, 2002; pp 849–856.